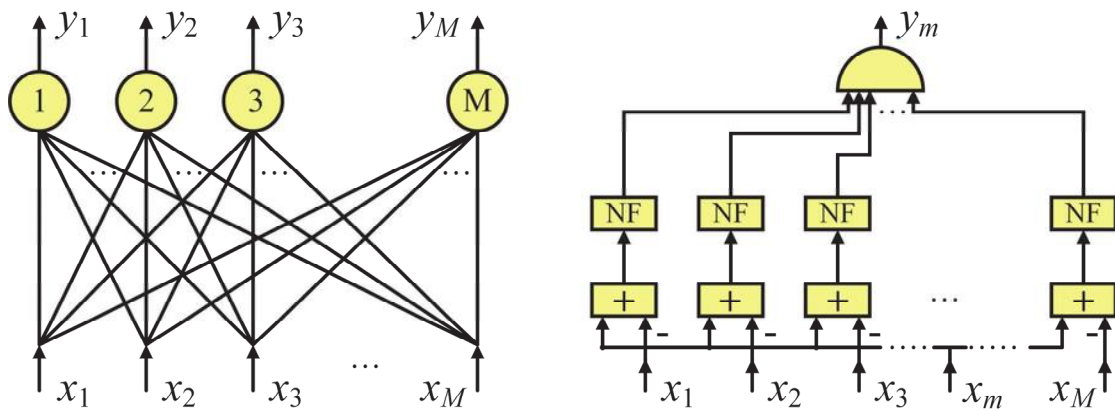


Intelligent Computing: A Computing Approach to Artificial Intelligence

Edition 1.5

Hon Keung Kwan



Intelligent Computing: A Computing Approach to Artificial Intelligence

Edition 1.5

Hon Keung Kwan

Professor
University of Windsor
Department of Electrical and Computer Engineering
401 Sunset Avenue, Windsor, Ontario, Canada N9B 3P4

Publication date: Edition 1.5, 4 February 2020.

ISBN 978-1-988307-09-1

Copyright © 2020 by Hon Keung Kwan

All rights reserved. This work in whole or part cannot be translated or copied or reproduced or distributed in any form or by any means, or stored in a data base or retrieval system, without the prior written permission of the copyright holder.

dfisp.org

Preface

The Edition 1.5 of this eBook on Intelligent Computing gives a clear description of the theories, models, algorithms, and design methodologies of intelligent computing to mimic and explain human-like reasoning, learning, and memory.

The eBook consists of 20 chapters organized into 5 parts. The Part I on Intelligent Computing consists of Chapter 1 which first gives an overview of each of computational intelligence, artificial intelligence, intelligent computing, and intelligent system design before presenting an introduction to the basics of biological and artificial neural networks.

The Part II on Neural Computing starts with an overview of neural computing and consists of Chapters 2 to 8 covering various neural networks, supervised and unsupervised learning algorithms.

The Part III on Fuzzy Neural Computing first provides an overview of fuzzy neural computing and consists of Chapter 9 to 12 explaining fuzzy set, fuzzy logic and reasoning, fuzzy systems, and fuzzy neural networks.

The Part IV on Evolutionary Computing begins with an overview of evolutionary computing and consists of Chapter 13 to 17 describing four popular evolutionary algorithms, genetic algorithms, differential evolution, particle swarm optimization, and simulated annealing; and aspects on coding, initialization, and constraints.

Finally, the Part V on Deep Neural Computing first presents an overview of deep neural computing and consists of three chapters on multiplierless neural networks, convolutional neural networks, and deep residual learning and networks.

The eBook gives a comprehensive coverage of intelligent computing as a computing approach to artificial intelligence. The topics covered by this eBook are clearly described and explained with the aid of step-by-step hand-calculation examples, Matlab examples, Matlab design examples, equations, and self-explanatory figures and tables.

This eBook can be used as a senior undergraduate and graduate textbook on Intelligent Computing for students in engineering, computer science, informatics, management science, and other programs. This eBook contains fundamental and advanced topics to meet the curriculum requirements of both undergraduate and graduate courses.

The eBook is also suitable for practicing engineers and computing professionals who wish to self-learn and understand intelligent computing as a computing approach to artificial intelligence, and to explore new possibilities for research and practical projects. No specific prerequisites are required but a background of elementary courses in linear algebra and calculus would be useful.

This eBook is of great use to those who are interested in (a) the models and learning algorithms of neural networks, fuzzy systems, fuzzy neural networks, and deep neural networks; and (b) their design methods for a specific application from the problem formulation and parameter vector representation, to the use of backpropagation or evolutionary algorithms for training.

This eBook is designed to be used together with an accompanying eBook written by the same author entitled “Matlab and Worked Problems in Intelligent Computing and System Design” to reinforce and enrich learning. It contains worked problems and assignment problems for the purposes of course assignments and self-learning.

There are a total of 98 examples in which 37 of them are design examples. The design examples illustrate how neural networks, fuzzy systems, fuzzy neural networks, and deep neural networks can be designed to build intelligent systems. The eBook contains over 150 tables, 176 figures, 195 references, and a total of 721 pages to support and clarify explanations.

A complete list of all the chapters and their topics along with all the examples, design examples and their titles are listed under the table of contents; notations for numbering equations, tables, and figures; and terms for examples and problems are explained immediate after the

table of contents; all the tables and figures and their titles are also listed at the end of the table of contents.

A clickable table of contents also comes with the eBook to allow a quick link of a specific topic.

The eBook has been designed and formatted to be readable and usable directly by an instructor in lecture classes and tutorial classes, without the need for preparing PowerPoint presentation files.

For ease of viewing the contents, a group of related paragraphs are started with an indented paragraph except at the beginning of a section or subsection or subsubsection.

The contents of this eBook have been developed under research and classroom environments for courses taught by the author at the University of Windsor. Input and/or feedback regarding typo errors and improvements would be appreciated.

H. K. Kwan, 4 February 2020

<http://www.uwindsor.ca/isplab/>

Intelligent Computing: A Computing Approach to Artificial Intelligence

Edition 1.5

Hon Keung Kwan

	Pg
Front cover	1
Title	1
Preface	3
Table of Contents	24
Notations for Equations, Tables, and Figures	1
Terms for Problems and Examples	
List of Tables	9
List of Figures	11
Part I Intelligent Computing	
I.1 Overview	1
Chapter 1 Introduction to Intelligent Computing	19
1.1 Introduction	
1.2 Overview of Computational Intelligence	
1.3 Overview of Artificial Intelligence	

1.4	Overview of Intelligent Computing	
1.5	Overview of Intelligent System Design	
1.6	Basics of Biological Neural Networks	
1.6.1	Transmission mechanism	
1.6.2	Neurons	
1.6.3	Synapses	
1.6.4	Human brain and mind	
1.7	Basics of Artificial Neural Networks	
1.7.1	Comparisons	
1.7.2	What is an ANN?	
1.7.3.	Typical structure of an ANN	
1.7.3.1	Structural features	
1.7.3.2	Nonlinear quantization functions	
1.7.3.2.1	Step function	
1.7.3.2.2	Ramp function	
1.7.3.2.3	Sigmoid function	
1.7.4	Applications of ANNs	
Part II	Neural Computing	
II.1	Overview	2
Chapter 2	One-Layer Feedforward Neural Networks	49
2.1	Introduction	
2.2	1LF-MaxNet for Maximum/Minimum Determination	
2.2.1	Introduction	

2.2.2	One-layer feedforward MaxNet	
2.2.3	Comparisons	
2.2.4	One-layer feedforward MinNet	
2.2.5	Summary	
2.3	Pattern Associators	
2.4	Hebb Rule	
2.4.1	Learning	
2.4.2	Recall	
2.4.2.1	Using weight matrix	
	Example 2.1 (Calculate Hebb-rule weights and recalls of PA)	
2.4.2.2	Using pattern association	
	Example 2.2 (Test input recalls using Hebb-rule pattern association)	
2.5	Properties of Hebb-Rule Trained PAs	
2.5.1	Pattern rectification	
	Example 2.3 (Pattern rectification property of Hebb-rule trained PA)	
2.5.2	Pattern completion	
	Example 2.4 (Pattern completion property of Hebb-rule trained PA)	
2.5.3	Fault tolerance	
	Example 2.5 (Fault tolerance property of Hebb-rule trained PA)	
2.5.4	Central tendency	
	Example 2.6 (Central tendency property of Hebb-rule trained PA)	M
2.5.5	Repeated learning	

	Example 2.7 (Repeated learning for improved pattern rectification and completion)	
2.5.6	Robust learning	
	Example 2.8 (Robust learning for improved pattern rectification and completion)	
2.6	Delta Rule	
2.6.1	Learning	
2.6.2	Recall	
	Example 2.9 (Delta-rule trained pattern associator)	M
2.7	Properties of Delta-Rule Trained PAs	
2.7.1	Central tendency	
	Example 2.10 (Central tendency property of delta-rule trained PA)	M
2.7.2	Higher ability to learn	
	Example 2.11 (Difficult pattern learnt by delta-rule trained PA)	M
2.8	Similarities between Hebb rule and delta rule	
Chapter 3	One-Layer Recurrent Neural Networks	31
3.1	Introduction	
3.2	Autoassociators	
3.3	Hebb Rule	
3.3.1	Learning	
	Example 3.1 (Hebb-rule learning in AA)	
3.3.2	Recall	
	Example 3.2 (Input pattern recalls in AA)	M
3.4	Properties of Hebb-Rule Trained AAs	

3.4.1	Pattern rectification	
	Example 3.3 (Pattern rectification property of Hebb-rule trained AA)	M
3.4.2	Pattern completion	
	Example 3.4 (Pattern completion property of Hebb-rule trained AA)	M
3.4.3	Central tendency	
	Example 3.5 (Central tendency property of Hebb-rule trained AA)	M
	Example 3.6 (Hebb-rule trained AA for pattern rectification and completion)	M
3.5	Delta Rule	
3.5.1	Learning	
3.5.2	Recall	
	Example 3.7 (Calculate weight matrix and recalls of delta-rule trained AA - 2 pattern-pairs)	M
	Example 3.8 (Calculate weight matrix and recalls of delta-rule trained AA - 5 pattern-pairs)	M
3.6	Properties of Delta-Rule Trained AAs	
	Example 3.9 (Pattern rectification and completion properties of delta-rule trained AA)	M
	Example 3.10 (Central tendency property of delta-rule trained AA)	M
3.7	Comparisons	
3.8	Autoassociators with Increased Storage Capacity	
Chapter 4	Multilayer Neural Networks	29
4.1	Introduction	

4.2	Backpropagation Learning Algorithm	
4.3	Backpropagation with Adaptive Bias	
4.4	Backpropagation with Adaptive Slope	
4.5	Backpropagation with Momentum	
4.6	Performances	
4.7	Multilayer Feedforward Neural Network Design	
	Design Example 4.1 (XOR problem learning using two-layer feedforward neural network)	M
4.8	Two-layer Recurrent Neural Network Design	
4.8.1	Network structure	
4.8.2	Gradient-based learning algorithm	
4.8.3	Genetic-based learning algorithm	
4.8.3.1	Encoding	
4.8.3.2	Crossover	
4.8.3.3	Mutation	
4.8.3.4	Reproduction	
	Design Example 4.2 (50 commands and 24 phone numbers learning using two-layer recurrent neural networks)	M
4.8.4	Summary	
Chapter 5	Associative Memories	45
5.1	Introduction	
5.2	Bidirectional Associative Memories	
5.2.1	Weight matrix	
5.2.2	Recall procedure	
5.2.3	Properties of BAMs	

	Example 5.1 (Learning and recalls of BAM)	M
5.3	Temporal Associative Memories	
5.3.1	Weight matrix	
5.3.2	Recall procedure	
5.3.3	Storage capacity	
	Example 5.2 (Learning and recalls of TAM)	M
5.4	Use of Bipolar Step Function	
5.5	Cascaded and Ring Associative Memories	
5.5.1	Classical BAM	
5.5.2	TAM using single BAM	
5.5.3	Cascade bidirectional associative memories	
5.5.4	Ring unidirectional associative memories	
5.5.5	Stability	
5.5.6	Pattern partitioning	
	Design Example 5.3 (Learning, recall and comparison of BAM, CBAM, and RUAM)	M
5.5.7	Summary	
5.5.8	BP-trained ring unidirectional associative memories	
5.6	Multilayer Symmetric and Asymmetrical Associative Memories	
5.6.1	Network Structures	
5.6.1.1	Symmetrical network	
5.6.1.2	Asymmetrical network	
5.6.2	Learning algorithm	
5.6.2.1	Symmetrical network	
5.6.2.2	Asymmetrical network	

5.6.3	Memory recalls	
	Design Example 5.4 (Symmetrical and asymmetrical associative memories for alpha-numeral pattern problem)	M
	Design Example 5.5 (Symmetrical and asymmetrical associative memories for face image problem)	M
5.6.4	Summary	
Chapter 6	Self-Organizing Maps	49
6.1	Introduction	
6.2	Competitive and Cooperative Clustering through Feedback Connections	
6.2.1	Mexican hat function	
6.2.2	Clustering phenomenon	
	Example 6.1 (Bubble formation by clustering)	M
	Example 6.2 (Impact of feedback factor on bubble formation)	M
	Example 6.3 (Bubble formation occurs at initial maximum location)	M
	Example 6.4 (Two-dimensional bubble formation by clustering)	M
6.3	Self-Organization Map Adaptation through Feedforward Connections	
6.3.1	One-dimensional SOM algorithm	
	Example 6.5 (Weight order formation depends on initial random input)	M
6.3.2	Causes of self-organization	
6.3.3	Effect of parameters	

	Example 6.6 (SOM requires gradually decreasing neighborhood width)	M
	Example 6.7 (SOM requires monotonically decreasing learning rate)	M
	Example 6.8 (SOM performs negatively with pre-ordered initial vector)	M
6.4	Clustering by SOM	
	Design Example 6.9 (SOM for clustering)	M
6.5	Two-Dimensional Self-Organizing Map	
6.5.1	Two-dimensional SOM algorithm	
	Design Example 6.10 (Two-dimensional self-organization map)	M
Chapter 7	Learning Vector Quantization	18
7.1	Introduction	
7.2	LVQ Network	
7.3	LVQ Learning Algorithm	
7.4	Weight Initialization Methods	
	Example 7.1 (LVQ for clustering vectors)	M
	Example 7.2 (Weight movements after LVQ learning)	M
	Example 7.3 (XOR problem cannot be learnt by LVQ classifier)	M
7.5	Improved LVQ Algorithms	
Chapter 8	Radial Basis Function Networks	25
8.1	Introduction	
8.2	Radial Basis Function Networks	

8.3	Separability of Patterns	
8.4	Gaussian Mapping Function	
	Example 8.1 (XOR problem solved by nonlinear transform)	
8.5	Interpolation RBF Network	
	Example 8.2 (XOR problem solved by interpolation RBF network)	M
8.6	Generalized RBF Network	
8.6.1	For $w_0 = 0$	
8.6.2	For $w_0 \neq 0$	
	Example 8.3 (XOR problem solved by generalized RBF network)	M
8.7	Determination of Radial Basis Function Centers	
	Design Example 8.4 (Generalized RBF network design for input to output mapping)	M
Part III	Fuzzy Neural Computing	
III.1	Overview	2
Chapter 9	Fuzzy Sets	46
9.1	Introduction	
9.2	Crisp Sets	
9.3	Fuzzy Sets	
9.3.1	Fuzzy sets with a discrete universe	
9.3.2	Fuzzy sets with a continuous universe	
9.3.3	Notations for fuzzy sets	
9.3.4	Subset	

9.3.5	Complement, union, and intersection
9.3.5.1	Complement (Negation)
9.3.5.2	Union (Disjunction)
9.3.5.3	Intersection (Conjunction)
9.3.6	Fuzzy partition
9.4	Terminologies of Membership Functions
9.4.1	Symmetry
9.4.2	Open left, open right, closed
9.4.3	Normality
9.4.4	Support
9.4.5	Fuzzy singleton
9.4.6	Core
9.4.7	Crossover points
9.4.8	α -cut and strong α -cut
9.4.9	Convexity
9.4.10	Widths of normal and convex fuzzy sets
9.4.11	Fuzzy number
9.5	One-Dimensional Membership Functions
9.5.1	Triangular membership function
9.5.2	Trapezoidal membership function
9.5.3	Gaussian membership function
9.5.4	Generalized bell membership function
9.5.5	Sigmoidal membership function
9.5.6	Asymmetric and close membership functions
9.6	Two-Dimensional Membership Functions

9.7	Derivatives of Parameterized Membership Functions	
9.8	Linguistic Variables	
9.8.1	Term set and its components	
9.8.2	Concentration and dilation	
9.8.3	Contrast intensifier $INT(A)$ and contrast diminisher $DIM(A)$	
	Example 9.1 (Contrast intensifier and contrast diminisher)	M
9.8.4	Negation (NOT) and connectives (AND OR)	
	Example 9.2 (Membership function constructions of composite linguistic terms)	
	Example 9.3 (Membership function plots of composite linguistic terms)	M
9.8.5	Orthogonality	
9.8.6	Similarity measure	
	Example 9.4 (Linguistic meaning of fuzzy set)	
9.9	Extension Principle	
	Example 9.5 (Fuzzy set construction by extension principle)	
9.9.1	Extension principle of universe X to universe Y	
	Example 9.6 (Fuzzy set mapping using quadratic polynomial by extension principle)	
	Example 9.7 (Fuzzy set mapping using cubic polynomial by extension principle)	M
9.9.2	Extension principle of universe $X_1 \times X_2 \times \dots \times X_M$ to universe Y	

Example 9.8 (Two-dimensional to one-dimensional universal mapping using linear function by extension principle)

Chapter 10	Fuzzy Logic and Reasoning	25
10.1	Introduction	
10.2	Fuzzy Relations	
10.2.1	Binary fuzzy relations	
	Example 10.1 (Compute binary fuzzy relation matrix)	
10.2.2	Composition of two fuzzy relations in same product space $X \times Y$	
10.2.3	Composition of two fuzzy relations in different product spaces $X \times Y$ and $Y \times Z$	
10.2.3.1	Max-Min composition	
10.2.3.2	Max-Product composition	
	Example 10.2 (Choice between programs using fuzzy relation matrices)	
10.3	Fuzzy If-Then Rules	
10.4	Compositional Rule of Inference	
	Example 10.3 (Linguistic variables and interpretations)	
10.5	Fuzzy Aggregation	
10.6	Defuzzification	
10.6.1	Centroid of area – Continuous universe of discourse	
10.6.2	Centroid of area – Discrete universe of discourse	
10.6.3	Centroid computation	

10.7	Fuzzy reasoning	
	Example 10.4 (2-rule 2-input 1-output fuzzy reasoning system)	M
Chapter 11	Fuzzy Systems	33
11.1	Introduction	
11.2	Mamdani Fuzzy System	
	Design Example 11.1 (4-rule 1-input 1-output Mamdani fuzzy system)	M
	Design Example 11.2 (4-rule 2-input 1-output Mamdani fuzzy system)	M
11.3	Tsukamoto Fuzzy System	
	Design Example 11.3 (4-rule 1-input 1-output Tsukamoto fuzzy system)	M
	Design Example 11.4 (4-rule 2-input 1-output Tsukamoto fuzzy system)	M
11.4	TSK Fuzzy System	
	Design Example 11.5 (4-rule 1-input 1-output TSK fuzzy system)	M
	Design Example 11.6 (4-rule 2-input 1-output TSK fuzzy system)	M
11.5	Comparisons	
Chapter 12	Fuzzy Neural Networks	46
12.1	Introduction	
12.2	Fuzzy Neurons	
12.2.1	Input fuzzy neuron (Input-FN)	
12.2.2	Transit fuzzy neuron (Tran-FN)	

- 12.2.3 Competitive fuzzy neuron (Comp-FN)
- 12.2.4 Sum fuzzy neuron (Sum-FN)
- 12.2.5 Maximum fuzzy neuron (Max-FN)
- 12.2.6 Minimum fuzzy neuron (Min-FN)
- 12.3 Fuzzy Inference Models
 - 12.3.1 Min-Competitive inference model
 - 12.3.2 Min-Sum inference model
 - 12.3.3 Min-Max inference model
- 12.4 Fuzzy Neural Networks
 - 12.4.1 Min-Competitive FNN
 - 12.4.1.1 One-input one-output problems
 - 12.4.1.2 Two-input one-output problems
 - 12.4.2 Min-Sum FNN
 - 12.4.2.1 One-input one-output problems
 - 12.4.2.2 Two-input one-output problems
 - 12.4.3 Min-Max FNN
 - 12.4.3.1 One-input one-output problems
 - 12.4.3.2 Two-input one-output problems
- 12.5 Learning Algorithms
- 12.6 Min-Competitive Fuzzy-Neural Networks for Pattern Recognition
 - 12.6.1 Unsupervised Min-Competitive Fuzzy Neural Network (UMCFNN)
 - 12.6.2 Supervised Min-Competitive Fuzzy Neural Network (SMCFNN)
 - 12.6.3 Application of MCFNN to pattern recognition

	Design Example 12.1 (Pattern recognition using FNNs)	M
12.7	Fuzzy Neural Networks as Classifiers	
12.7.1	Application to non-fuzzy classification	
	Design Example 12.2 (Circle-in-the-square problem using FNNs)	M
12.7.2	Application to fuzzy classification	
	Design Example 12.3 (Fuzzy XOR problem using FNNs)	M
12.8	Summary	
Part IV	Evolutionary Computing	
IV.1	Overview	2
Chapter 13	Genetic Algorithms	32
13.1	Introduction	
13.2	Genetic Algorithms	
13.2.1	Reproduction	
	Example 13.1 (Roulette wheel reproduction of next population)	
13.2.2	Crossover	
	Example 13.2 (Crossover two parents to generate two children)	
13.2.3	Mutation	
	Design Example 13.3 (Solving second-order polynomial function by GA)	M
13.3	Schema, Order, Length	

	Example 13.4 (Order, length and mutation immunity of schema)	
	Example 13.5 (Numbers of schemas and mutation immunities of binary and hex codes)	
13.4	Improved Reproduction Methods	
13.4.1	Improved selection	
	Example 13.6 (Remainder stochastic sampling without replacement)	
13.4.2	Linear scaling	
	Example 13.7 (Linear scaling of fitness values)	M
13.5	Fitness Function Mapping	
13.5.1	68-95-99.7 rule	
13.5.2	Maximization problems	
13.5.3	Minimization problems	
13.6	Neural Network Design using Genetic Algorithm	
	Design Example 13.8 (Two-layer feedforward neural network learning of integer-valued XOR problem using GA)	M
	Design Example 13.9 (2-layer feedforward neural network learning of mixed-valued XOR problem using GA)	M
	Design Example 13.10 (2-layer feedforward neural network learning of real-valued XOR problem using GA)	M
Chapter 14	Differential Evolution	28
14.1	Introduction	
14.2	Algorithm	
14.2.1	Initialization	

14.2.2	Mutation	
14.2.3	Crossover	
14.2.4	Selection	
14.2.4.1	Maximization problem	
14.2.4.2	Minimization problem	
	Design Example 14.1 (2-layer feedforward neural network learning of XOR problem using DE)	M
14.3	Population-based Crossover and Selection	
14.3.1	Crossover	
14.3.2	Selection	
14.3.2.1	Maximization problem	
14.3.2.2	Minimization problem	
	Design Example 14.2 (Third-order polynomial interpolation by DE)	M
	Design Example 14.3 (Joint-objective 2-variable quadratic function minimization by DE)	M
Chapter 15	Particle Swarm Optimization	64
15.1	Introduction	
15.2	Social Cognition and Particle Swarm	
15.3	Choices of Local and Global Neighbourhoods	
15.4	Continuous-valued Particle Swarm Optimization	
15.4.1	Continuous number model	
15.4.2	Bipolar sigmoid function as saturating function	
15.5	Discrete-valued Particle Swarm Optimization	
15.5.1	Binary number model	
15.5.2	Binary sigmoid function as saturating function	

	Design Example 15.1 (Exponential function interpolation by PSO)	M
	Design Example 15.2 (1-dimensional sine function interpolation by PSO)	M
	Design Example 15.3 (2-layer feedforward neural network learning of XOR problem using PSO - I)	M
	Design Example 15.4 (2-layer feedforward neural network learning of XOR problem using PSO - II)	M
	Design Example 15.5 (2-variable Sphere function minimization using BPSO)	M
	Design Example 15.6 (Third-order polynomial function interpolation using BPSO)	M
	Design Example 15.7 (2-layer feedforward neural network learning of XOR problem using BPSO)	M
15.6	Remarks	
Chapter 16	Simulated Annealing	18
16.1	Introduction	
16.2	Simulated Annealing	
16.2.1	Algorithm	
16.2.2	Choices of parameter values	
16.3	Pseudocode for Simulated Annealing	
	Design Example 16.1 (2-layer feedforward neural network learning of XOR problem using SA - I)	M
	Design Example 16.2 (2-layer feedforward neural network learning of XOR problem using SA - II)	M
16.4	Population-Based Simulated Annealing	
	Design Example 16.3 (2-layer feedforward neural network learning of XOR problem using population-based SA)	M

17.6	Multi-objective Evolutionary Optimization Algorithms	
------	--	--

Part V Deep Neural Computing

V.1	Overview	2
-----	----------	---

Chapter 18 Multiplierless Neural Networks 13

18.1	Introduction	
18.2	Simple Sigmoid Activation Functions	
18.2.1	Second-order activation function	
18.2.2	Comparison	
	Design Example 18.1 (Pattern recognition using feedforward neural networks with simple activation function)	M
18.3	Discrete Multilayer Feedforward Neural Network Design	
18.3.1	Weight and bias adaptations	
18.3.2	SSAF and OPOT weights	
18.3.3	Design algorithm	
	Design Example 18.2 (Ten-digit problem learning using discrete multilayer feedforward neural network)	M
18.4	SOPT Quantized Activation Functions	
18.5	Systolic Realization of Neural Networks	

Chapter 19 Convolutional Neural Networks 26

19.1	Introduction	
19.2	Definitions of Convolutional Operations	

19.3	Structure of Convolutional Neural Networks
19.4	Properties of Convolutional Neural Networks
19.4.1	Sparse weights
19.4.2	Weight sharing
19.4.3	Equivariant representation
19.5	Layers of Convolutional Neural Networks
19.5.1	Convolutional layer
19.5.1.1	Filter and stride
19.5.1.2	Dilated convolution
	Example 19.1 (Calculate size of dilated filter)
19.5.1.3	Zero padding
19.5.1.4	Feature maps
	Example 19.2 (Numbers of weights in a convolutional layer)
19.5.1.5	Output size
	Example 19.3 (Numbers of neurons in a convolutional layer)
19.5.1.6	Learning parameters and number of convolutional layers
19.5.2	Batch normalization layer
19.5.3	ReLU layer
19.5.4	Cross channel normalization layer
19.5.5	Maximum or average pooling layer
19.5.6	Fully connected layer
19.5.7	Regularization techniques
19.5.7.1	L1 or L2 regularization
19.5.7.2	Dropout

19.5.7.3	Data augmentation	
19.5.7.4	Early stopping	
19.5.8	Output layer	
19.5.8.1	Softmax and classification output layer	
19.5.8.2	Regression output layer	
19.5.9	Batch size	
19.6	Training of Convolutional Neural Networks	
	Design Example 19.4 (Create and train simple convolutional neural network for classification)	M
Chapter 20	Deep Residual Learning and Networks	25
20.1	Introduction	
20.2	Residual Units	
20.3	Identity Mapping by Shortcuts	
20.4	Deep Residual Networks	
20.4.1	Bottleneck 3-layer residual units	
20.4.2	Performance and comparison of original ResNets with other networks	
20.5	Identity Mappings in Deep Residual Networks	
20.5.1	Importance of identity skip connections	
20.5.2	Performance and comparison of original ResNets and ResNets	
20.6	Aggregated Transformations for Deep Residual Networks	
20.6.1	Template	
20.6.2	Neuron operation	
20.6.3	Aggregated transformations	

- 20.6.4 Model capacity
- 20.6.5 Performance and comparison of ResNets and ResNeXts

References

Back cover 1

Total 721 pages

Number of Examples: 98

Number of Design Examples: 37

Number of Tables: 150

Number of Figures: 176

Number of references: 195

Keys Pg: Total page number of a chapter

M: Matlab-based Example

The front cover shows the structure of the one-layer Max/Min Net¹ that requires the time of two-input adder, one hardlimiting function, and one AND gate to compute the maximum or minimum of any arbitrary set of M real values. ¹ H. K. Kwan, "One-layer feedforward neural network for fast maximum/minimum determination", Electronics Letters, 13 August 1992, volume 28, number 17, pages 1583-1585.

Notations for Equations, Tables, and Figures

Symbol	Description	Example
C	Chapter number	
S	Section number	
X	Example number	
N	Table/Figure/Equation number	
E	Example	
(C.S.N)	<i>N</i> th equation of Chapter C Section S	(2.1.1)
(EC.X.N)	<i>N</i> th equation of Example C.X	(E4.1.1)
Table C.S.N	<i>N</i> th table of Chapter C Section S	Table 2.2.2
Table EC.X.N	<i>N</i> th table of Example C.X	Table E4.2.2
Figure C.S.N	<i>N</i> th figure of Chapter C Section S	Figure 2.3.3
Figure EC.X.N	<i>N</i> th figure of Example C.X	Figure E4.3.3

Terms for Examples and Problems

Term	Description
Examples	As listed in a chapter
Design Examples	As listed in a chapter
Problems	As listed in Worked Problems eBook

For ease of reading, a group of related paragraphs are started with an indented paragraph except at the beginning of a section or subsection or subsubsection.

List of Tables

Table 1.1 Key topics in computational intelligence and artificial intelligence and their relationships.

Table 1.6.1 Transmission mechanism in neurons.

Table 1.6.2 Typical types of neurons.

Table 1.6.3 Features of type I (excitatory) synapses and type II (inhibitory) synapses.

Table 1.7.1 General and special structural features of ANN.

Table 2.2.1 Computational requirements of MaxNets.

Table 2.2.2 Hardware requirements of MaxNets.

Table E2.1.1 Input patterns (1x4) and output patterns (1x2) for learning.

Table E2.1.2 Noisy input patterns (1x4) for recalls.

Table E2.2.1 Input patterns (1x4) and output patterns (1x4) for learning.

Table E2.3.1 Input patterns (1x6) and output patterns (1x6) for learning.

Table E2.3.2 Noisy input patterns (1x6) for recalls.

Table E2.4.1 Incomplete input patterns (1x6) for recalls.

Table E2.5.1 Input patterns (1x4) and output patterns (1x4) for learning.

Table E2.6.1 Input patterns (1x4) and output patterns (1x4) for learning.

Table E2.6.2 Noisy input patterns (1x4) for recalls.

Table E2.7.1 Input patterns (1x4) and output patterns (1x1) for learning.

Table E2.7.2 Noisy input patterns (1x4) for recalls.

Table E2.7.3 Incomplete input patterns (1x4) for recalls.

Table E2.9.1 Input patterns (1x4) and output patterns (1x1) for learning.

Table E2.10.1 Noisy input patterns (1x4) for recalls.

Table E2.11.1 Input patterns (1x4) and output patterns (1x1) for learning.

Table E2.11.2 Noisy input patterns (1x4) for recalls.

Table E2.11.3 Incomplete input patterns (1x4) for recalls.

Table 4.2.1 Notations for backpropagation learning algorithms.

Table 4.7.1 XOR problem.

Table E4.2.1 List of 50 commands consisting of 44 words.

Table E4.2.2 Input, hidden, and output configurations of different neural networks.

Table E4.2.3 Time requirements of different neural networks reaching ASSE=2.5 for 50 commands learning.

Table E4.2.4 Recall accuracies of different neural networks for 50 commands (not bracketed) and 24 phone numbers (bracketed).

Table E4.2.5 Performance effects of population size.

Table E5.3.1 Recall results for storing two-word sequences.

Table E5.3.2 Recall results for storing four-word sequences.

Table E5.4.1 Autoassociator: Pattern-pair recalls.

Table E5.4.2 Autoassociator: Single-pattern recall from layer 0.

Table E5.4.3 Autoassociator: Single-pattern recall from layer 2.

Table E5.4.4 Pattern associator: Pattern-pair recalls.

Table E5.4.5 Pattern associator: Single-pattern recall from layer 0.

Table E5.4.6 Pattern associator: Single-pattern recall from layer 2.

Table E5.4.7 Asymmetrical network: Pattern-pair recalls.

Table E5.4.8 Asymmetrical net: Single-pattern recall from layer 0.

Table E5.4.9 Asymmetrical net: Single-pattern recall from layer 2.

Table E5.5.1 Numbers of weights of different associative memory networks.

Table E6.1.1 Bubble formation by clustering at outputs of neurons.

Table E6.3.1 Outputs of neurons showing activity center.

Table E6.5.1 Choices of neighborhood width and learning rate parameter.

Table 6.5.1 Two-dimensional SOM algorithm.

Table E6.10.1 (Choice of learning rate parameter and neighborhood width)

Table 7.3.1 Notations for LVQ.

Table 7.4.1 LVQ weight initialization methods.

Table E7.1.1 Clustering vectors using LVQ.

Table E7.1.2 Initial weights for LVQ.

Table E7.1.3 Classification results obtained by LVQ.

Table E7.2.1 Initial centers for LVQ.

Table E7.2.2 Clustering 2-input vectors using LVQ.

Table E7.3.1 XOR problem.

Table E7.3.2 Initial weights for XOR problem.

Table E7.3.3 XOR classification results obtained by LVQ.

Table 8.2.1 Comparison between RBF networks and MLFNNs.

Table E8.1.1 XOR input to output mapping.

Table E8.1.2 XOR classification using nonlinear transform.

Table E8.3.1 XOR input to output mapping.

Table E8.3.2 XOR problem classification using GRBF network.

Table E8.4.1 Input (1x4) to output mapping.

Table E8.4.2 GRBF network for (1x4) input to output mapping.

Table 9.2.1 Notations and definitions of crisp sets.

Table 9.4.1 Definitions of open left, open right, and closed fuzzy sets.

Table 9.7.1 One-dimensional Gaussian membership function and its derivatives.

Table 9.7.2 One-dimensional generalized bell membership function and its derivatives.

Table 9.7.3 Two-dimensional Gaussian (composite) membership function and its derivatives.

Table 9.7.4 Two-dimensional generalized bell (non-composite) membership function and its derivatives.

Table E9.6.1 Fuzzy set mapping using quadratic function by extension principle.

Table E9.7.1 One-to-one mapping using cubic polynomial.

Table E9.7.2 Two-to-one mapping using cubic polynomial.

Table E9.7.3 Three-to-one mapping using cubic polynomial.

Table E9.8.1 Two-dimensional to one-dimensional universal mapping using linear function by extension principle.

Table 10.1.1 Notations for fuzzy logic and reasoning.

Table 10.2.1 Properties of Max-Min composition.

Table 12.1.1 Notations for fuzzy neural networks.

Table 12.5.1 Steps of learning algorithm.

Table 12.6.1 UCMFNN for pattern recognition learning algorithm.

Table 12.6.2 SMCNN for pattern recognition learning algorithm.

Table E12.1.1 Recognition rates when trained by training set 1.

Table E12.1.2 Recognition rates when tested by shifted patterns.

Table 12.7.1 MSFNN classifier learning algorithm.

Table 12.7.2 MMFNN classifier learning algorithm.

Table E12.3.1 Fuzzy XOR problem (9 data-pairs).

Table E12.3.2 Fuzzy XOR problem (36 data-pairs).

Table E12.3.3 Fuzzy inference rules (from 9 samples) learnt by MSFMM.

Table E12.3.4 Fuzzy inference rules (from 9 samples) learnt by MMFNN.

Table 13.2.1 Notations for genetic algorithms.

Table 13.2.2 Steps of a basic genetic algorithm.

Table 13.2.3 Steps of roulette wheel reproduction.

Table E13.1.1 Total fitness of 6 chromosomes.

Table E13.1.2 Roulette wheel parent selection.

Table 13.2.2 Pseudocode for genetic algorithm using three basic genetic operators (Start: S; End: E; RW: Roulette Wheel; C: Crossover; M: Mutation).

Table E13.3.1 GA calculations involving reproduction, crossover, and mutation ($c1(t)$ to $c6(t)$ refer chromosomes 1 to 6 at generation t) (Hints: For both Crossover and Mutation Processes: View the Roulette Wheel Copies, the New index i , and the binary pattern of each chromosome i to keep track its movement).

Table 13.3.1 Examples for schema, order, length.

Table E13.4.1 Order and length of schema.

Table E13.6.1 Calculations of remainder stochastic sampling without replacement.

Table E13.8.1 Two-layer neural network parameters for GA.

Table E13.8.2 Additional guidelines for learning XOR problem using GA.

Table E13.8.3 Matlab program for integer-valued XOR design problem using genetic algorithm.

Table E13.8.4 Design parameters and result of integer-valued XOR problem using genetic algorithm.

Table E13.9.1 Design parameters and result of mixed-valued XOR problem using genetic algorithm (Case 1).

Table E13.9.2 Design parameters and result of mixed-valued XOR problem using genetic algorithm (Case 2).

Table E13.10.1 Design parameters and result of real-valued XOR problem using genetic algorithm.

Table 14.2.1 Notations for DE algorithm.

Table 14.2.2 Pseudocode for DE algorithm (M: Mutation; C: Crossover; S: Selection).

Table E14.1.1 DE parameters for two-layer neural network.

Table E14.1.2 Matlab program for XOR design problem using differential evolution.

Table E14.1.3 Design parameters and result of XOR problem using differential evolution.

Table 14.3.1 Pseudocode for DE algorithm using population-based crossover and selection (M: Mutation; C: Crossover; S: Selection).

Table 15.3.1 Choices of local and global neighbourhoods.

Table 15.4.1 Notations for PSO continuous number model.

Table 15.4.2 Relationship between slope parameter r_g and range of $v_{pa}(t)$ for bipolar sigmoid function s_g .

Table 15.4.3 Pseudocode for PSO continuous number model (E: Evaluate; C: Compare; I: Inmitate).

Table 15.5.1 Notations for PSO binary number model.

Table 15.5.2 Pseudocode for PSO binary number model (C: Compare; I: Inmitate).

Table 15.5.3 Relationship between slope parameter r_f and range of $v_{pd}(t)$ for binary sigmoid function s_f .

Table E15.3.1 PSO parameters for two-layer neural network.

Table E15.3.2 Matlab program for XOR design problem using particle swarm optimization (particleswarm.m).

Table E15.3.3 Design parameters and result of XOR problem using particle swarm optimization (particleswarm.m).

Table E15.4.1 Matlab program for XOR design problem using particle swarm optimization (pso_icsd.m).

Table E15.4.2 Design parameters and result of XOR problem using particle swarm optimization (pso_icsd.m).

Table E15.5.1 BPSO parameters for sphere function minimization.

Table E15.6.1 Probability function slope parameters and coefficient wordlength.

Table E15.6.2 BPSO parameters for third-order polynomial function interpolation.

Table E15.7.1 BPSO parameters for two-layer neural network.

Table 16.3.1 Pseudocode for simulated annealing.

Table E16.1.1 SA parameters for 2-layer neural network.

Table E16.1.2 Matlab program for XOR design problem using simulated annealing.

Table E16.1.3 Design parameters and result of XOR problem using simulated annealing.

Table 17.2.1 Parameter vectors represented by different coding methods.

Table E17.1.1 Concatenated multi-parameter unsign-magnitude binary integer coding.

Table E17.2.1 Concatenated multi-parameter sign-magnitude binary integer coding.

Table E17.3.1 Concatenated multi-parameter sign-magnitude binary fixed-point coding.

Table 17.3.1 Specifications of initialization functions.

Table E18.1.1 Performance of two-layer and three-layer neural networks using SSAF.

Table E18.2.1 Convergence speed and generalization capability of networks with one hidden layer.

Table E18.2.2 Convergence speed and generalization capability of networks with two hidden layers.

Table E19.4.1 Matlab program to create and train CNN.

Table 20.4.1 Architectures for ImageNet. Building blocks are shown in brackets (details and other variants are shown for Figure 20.4.1), with the numbers of blocks stacked. Down-sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.

Table 20.4.2 Error rates (10-crop testing) on ImageNet validation. VGG-16 is based on [He1512a].

Table 20.4.3 Error rates of single-network results on the ImageNet validation set. VGG is based on [He1512a].

Table 20.4.4 Error rates of ensembles. The top-5 error is on the test set of ImageNet and reported by the test server.

Table 20.5.1 Comparisons with state-of-the-art methods on CIFAR-10 and CIFAR-100 using moderate data augmentation (flip/translation), except for ELU [Cle1602] with no augmentation. Better results of [Gra1505], [Spr1504] have been reported using stronger data augmentation and ensembling. For the ResNets, the number of parameters is listed. Results are obtained by taking the median of 5 runs with mean $\pm$ std in the brackets. All ResNets results are obtained with a mini-batch size of 128 except * with a mini-batch size of 64.

Table 20.6.1 ResNet-50 (3rd column) and ResNeXt-50 (4th column) with a $32 \times 4d$ template (using the reformulation in Figure 20.10(c)). Inside the brackets are the shape of a residual block, and outside the brackets is the number of stacked blocks on a stage. “ $C=32$ ” suggests grouped convolutions [Kri1212] with 32 groups. The numbers of parameters and FLOPs are similar between these two networks.

Table 20.6.2 Relations between cardinality and width (for the template of conv2), with roughly preserved complexity on a residual block. The number of parameters is $\sim 70k$ for the template of conv2. The number of FLOPs is ~ 0.22 billion ($\# \text{ parameters} \times 56 \times 56$ for conv2).

Table 20.6.3 State-of-the-art networks on the ImageNet-1K validation set (single-crop testing). The test size of ResNet/ResNeXt is 224×224 and 320×320 as in [He1704] and of the Inception networks is 299×299 . Top-1: Top-1 error; Top-5: Top-5 error.

List of Figures

Figure 1.6.1 Sketch of a neuron.

Figure 1.6.2 Some neurons of the cerebral cortex.

Figure 1.6.3 Type I synaptic contact.

Figure 1.6.4 Type II synaptic contact.

Figure 1.7.1 L -layer feedforward neural network.

Figure 1.7.2 Internal structure of a neuron.

Figure 1.7.3 Typical nonlinear quantization functions (for $L = 2$): (a) Step function (Binary); (b) Ramp function (Binary); (c) Sigmoid function (Binary); (d) Step function (Bipolar); (e) Ramp function (Bipolar); (f) Sigmoid function (Bipolar).

Figure 2.2.1 One-layer MaxNet/MinNet.

Figure 2.2.2 Internal structure of m th neuron of Figure 2.2.1.

Figure 2.3.1 4-input and 1-output pattern associator.

Figure 2.3.2 4-input and 2-output pattern associator.

Figure 3.2.1 Autoassociator with M neurons.

Figure 3.2.2 Autoassociator with $M = 3$ neurons.

Figure 4.2.1 L -layer feedforward neural network.

Figure 4.2.2 Structure of a neuron.

Figure 4.7.1 Two-layer neural network for XOR problem.

Figure 4.8.1 Two-layer recurrent neural network.

Figure 4.8.2 Chromosome structure containing weight-connection pairs.

Figure E4.2.1 ASSE of different neural networks for 50 commands learning.

Figure E4.2.2 ASSE of different neural networks for 24 phone numbers learning.

Figure E4.2.3 Number of weight connections in different neural networks for 50 commands.

Figure E4.2.4 Number of weight connections in different networks for 24 phone numbers.

Figure E4.2.5 ASSE for GRNN with different crossover probabilities.

Figure E4.2.6 ASSE for GRNN with different mutation rates.

Figure E4.2.7 ASSE for GRNN with discrete and continuous weights.

Figure 5.2.1 Structure of BAM.

Figure 5.3.1 Forward recall of a TAM.

Figure 5.3.2 Backward recall of a TAM.

Figure 5.5.1 Temporal associative memory using single BAM.

Figure 5.5.2 Temporal associative memory using CBAM.

Figure 5.5.3 Temporal associative memory using RUAM.

Figure 5.6.1 Three-layer bidirectional symmetrical associative memories.

Figure 5.6.2 Three-layer bidirectional asymmetrical associative memories.

Figure E5.4.1 0 to 9 numerals of 10x10 bipolar pixels.

Figure E5.4.2 Samples of 26 small alphabets of 16x16 bipolar pixels.

Figure E5.4.3 Samples of 26 capital alphabets of 16x16 bipolar pixels.

Figure E5.5.1 Four original front-face images.

Figure E5.5.2 Four sets of 2 side-face images for training.

Figure E5.5.3 Four sets of 2 side-face images for testing.

Figure E5.5.4 Symmetrical network recalls by test-face images of Figure E5.5.3.

Figure E5.5.5 Asymmetrical network recalls by test-face images of Figure E5.5.3.

Figure 6.2.1 One-dimensional Mexican hat function ($a = -0.05$; $b = 0$; $c = 1$; $d = 0.01$).

Figure 6.2.2 Two-dimensional Mexican hat function ($a = -0.05$; $b = 0$; $c = 1$; $d = 0.01$).

Figure 6.2.3 Discrete approximations of one-dimensional Mexican hat function ($a = 5$, $M = 21$; $b = 0.05$, $c = -0.02$).

Figure 6.2.4 Discrete approximations of two-dimensional Mexican hat function ($a_i = 3$, $M_i = 2a_i + 1$, for $i = 1, 2$; $b = 0.09$, $c = -b/3$).

Figure 6.2.5 P feedforward external inputs to neuron j for feature detection and one self-feedback and $2M$ lateral feedbacks of neuron j for clustering.

Figure 6.2.6 One self-feedback and $2M$ lateral feedbacks connections of a neuron j with excitatory and inhibitory effects for clustering (total external input to neuron j denoted by $I_j(0)$).

Figure E6.1.1 Bubble formation by clustering.

Figure E6.2.1 Bubble formation of neurons 1 to 83 at $k = 0$ to 20 using different α : (a) No (or decreasing) bubble formation for $\alpha = 0.15$; (b) Bubble formation for $\alpha = 0.2$; (c) Saturated bubble formation for $\alpha = 0.25$.

Figure E6.3.1 Activity center remains at initial maximum location.

Figure E6.4.1 Discrete two-dimensional Mexican hat function ($a = 3$, $M = 3a$, $N = 4M + 1$, $b = 0.06$, and $c = -b/3$).

Figure E6.4.2 Effect of feedback factor on bubble formation: (a) No (or decreasing) bubble formation for $\alpha = 0.05$; (b) Bubble formation for $\alpha = 0.056$; (c) Saturated bubble formation for $\alpha = 0.062$.

Figure 6.3.1 N -input M -output SOM.

Figure 6.3.2 Two-dimensional topological neighborhoods ($R=0$: dot line; $R=1$: solid line; $R=3$: desh line): (a) Square neighborhood (Top); (b) Diamond neighborhood (Left); (c) Octagonal neighborhood (Right).

Figure E6.5.1 1-input and 11-output SOM.

Figure E6.5.2 SOM results: (a) Forward order; (b) Backward order; (c) Two compressed ends.

Figure E6.6.1 SOM results with fixed neighborhood width of 4.

Figure E6.6.2 SOM results with fixed neighborhood width of 2.

Figure E6.7.1 SOM results with fixed learning rate of 0.4.

Figure E6.8.1 SOM results with evenly spread initial weights.

Figure E6.10.1 Two-dimensional self-organization map at $k = 0$.

Figure E6.10.2 Two-dimensional self-organization map at $k = 100$.

Figure E6.10.3 Two-dimensional self-organization map at $k = 200$.

Figure E6.10.4 Two-dimensional self-organization map at $k = 500$.

Figure E6.10.5 Two-dimensional self-organization map at $k = 1000$.

Figure E6.10.6 Two-dimensional self-organization map at $k = 2000$.

Figure E6.10.7 Two-dimensional self-organization map at $k = 5000$.

Figure 7.2.1 2-input 4-output LVQ network.

Figure E7.1.1 4-input 2-output LVQ network.

Figure E7.2.1 Movements of original weights after four 2-input vector learning.

Figure 8.3.1 Linearly separable problem.

Figure 8.3.2 Linearly nonseparable problem.

Figure E8.1.1 Linearly nonseparable XOR problem shown on two-dimensional plane.

Figure E8.1.2 Linearly separable XOR problem obtained by nonlinear transform.

Figure 8.5.1 Interpolation RBF network ($P = K$).

Figure 8.6.1 Generalized RBF network ($P < K$).

Figure E8.3.1 Generalized RBF network for XOR problem.

Figure E8.4.1 Generalized RBF network for 4-input to 1-output mapping problem.

Figure 9.3.1 Crisp set and fuzzy set of hot water: (a) Crisp set; (b) Fuzzy set.

Figure 9.3.2 Membership grades: (a) Discrete universe; (b) Continuous universe.

Figure 9.3.3 Fuzzy set A is contained in fuzzy set B ($A \subseteq B$).

Figure 9.3.4 Fuzzy set operations: (a) Fuzzy sets A, B ; (b) Fuzzy set “NOT A ”; (c) Fuzzy set “ A OR B ”; (d) Fuzzy set “ A AND B ”.

Figure 9.3.5 Typical membership functions.

Figure 9.4.1 Subjective membership functions.

Figure 9.4.2 Membership function terminologies.

Figure 9.4.3 Convex and nonconvex membership functions.

Figure 9.5.1 Parameterized membership functions: (a) Triangular; (b) Trapezoidal; (c) Gaussian; (d) Generalized bell.

Figure 9.5.2 Parameter effects on generalized bell membership function: (a) Increasing $c=[-20,0,20]$, $a=10$, $b=2$; (b) Increasing $a=[5,10,15]$, $b=2$, $c=0$; (c) Increasing $b=[1,2,4]$, $a=10$, $c=0$; (d) Increasing $a=[5,10,15]$, $b=[1,2,4]$, $c=0$.

Figure 9.5.3 Construction of close and asymmetric membership

functions: (a) Sigmoidal functions $z1, z2$; (b) $|z1 - z2|$; (c) Sigmoidal functions $z1, z3$; (d) $z1 * z3$.

Figure 9.6.1 Min and Max operators for two-dimensional membership function construction: (a) $z1$; (b) $z2$; (c) $\min(z1, z2)$; (d) $\max(z1, z2)$.

Figure E9.1.1 Effects of applying contrast intensifier.

Figure E9.1.2 Effects of applying contrast diminisher.

Figure E9.3.1 Membership functions of primary and composite linguistic values: (a) Primary; (b) Composite.

Figure 9.8.1 Orthogonal membership functions.

Figure E9.6.1 Extension principle on fuzzy sets with discrete universes.

Figure E9.7.1 Extension principle on fuzzy sets with continuous universes.

Figure 10.2.1 Sagittal diagram of a binary fuzzy relation $R(X, Y)$.

Figure E10.2.1 Composition of fuzzy relations $\mu_{R_1 \circ R_2}(x_2, z_1)$.

Figure 10.3.1 Transformation by fuzzy implication functions: (a) $z1$; (b) $z2$; (c) $\min(z1, z2)$; (d) $z1 * z2$.

Figure 10.4.1 Compositional rule of inference: (a) $R(X, Y)$; (b) $c(A)$; (c) $\min(R(X, Y), c(A))$; (d) $\mu_B(y)$.

Figure E10.4.1 2-rule 2-input 1-output fuzzy reasoning (Algebraic product for weights, $w_1 = 0.27998$, $w_2 = 0.21832$; centroid defuzzified output, $y_o = 0.68096$).

Figure 11.1.1 Block diagram of a fuzzy system.

Figure 11.2.1 Mamdani fuzzy system (2-rule 2-input 1-output; minimum operator for weights, $w_1=0.3733$, $w_2=0.2500$; centroid defuzzified output, $y_0=0.4815$).

Figure E11.1.1 Mamdani fuzzy system (4-rule 1-input 1-output).

Figure E11.2.1 Mamdani fuzzy system (4 rule 2-input 1-output).

Figure 11.3.1 Tsukamoto fuzzy system (2-rule 2-input 1-output, minimum operator for weights).

Figure E11.3.1 Tsukamoto fuzzy system (4-rule 1-input 1-output).

Figure E11.4.1 Tsukamoto fuzzy system (4-rule 2-input 1-output).

Figure 11.4.1 TSK fuzzy system (2-rule 2-input 1-output).

Figure E11.5.1 TSK fuzzy system (4-rule 1-input 1-output).

Figure E11.6.1 TSK fuzzy system (4-rule 2-input 1-output).

Figure 12.2.1 Diagram of a fuzzy neuron.

Figure 12.4.1 Three-layer fuzzy neural network.

Figure 12.4.2 Min-Competitive Fuzzy Neural Network (MCFNN).

Figure 12.4.3 Weight function $w_{ij}^{[1]}(x_i)$ of MCFNN.

Figure 12.4.4 Min-Sum Fuzzy Neural Network (MSFNN).

Figure 12.4.5 Weight function $w_{ij}^{[1]}(x_i)$ of MSFNN.

Figure 12.4.6 Min-Max Fuzzy Neural Network (MMFNN).

Figure 12.4.7 Weight function $w_{ij}^{[1]}(x_i)$ of MMFNN.

Figure 12.6.1 Unsupervised Min-Competitive Fuzzy Neural Network for pattern recognition.

Figure 12.6.2 Fuzzification function ($\beta = 0.3, m_1 = 10, n_1 = 10$).

Figure 12.6.3 Output function of Max-FN in second layer of UMCFNN.

Figure 12.6.4 Supervised Min-Competitive Fuzzy Neural Network.

Figure 12.6.5 Output function of Max-FN in second layer of SMCFNN.

Figure 12.6.6 Core operations of pattern recognition.

Figure E12.1.1 36 universal font patterns for training.

Figure E12.1.2 36 bold font patterns for testing.

Figure E12.1.3 36 Italic font patterns for testing.

Figure E12.1.4 36 non-standard font patterns for testing.

Figure E12.1.5 Shifted patterns of exemplar E.

Figure E12.1.6 Distorted patterns for testing.

Figure 12.7.1 Circle-in-the-square classification problem.

Figure E12.2.1 Classification results obtained by MSFNN.

Figure E12.2.2 Classification results obtained by MMFNN.

Figure E12.3.1 Fuzzy XOR classification problem and two training sets: (a) Fuzzy XOR problem (m.v.: membership value); (b) Training set 1 (9 samples), Training set 2 (36 samples).

Figure E12.3.2 Membership functions learnt by MSFNN.

Figure E12.3.3 Fuzzy boundaries obtained by MSFNN (9 samples).

Figure E12.3.4 Fuzzy boundaries obtained by MSFNN (36 samples).

Figure E12.3.5 Membership functions learnt by MMFNN.

Figure E12.3.6 Fuzzy boundaries obtained by MMFNN (9 samples).

Figure E12.3.7 Fuzzy boundaries obtained by MMFNN (36 samples).

Figure 13.2.1 Roulette wheel with 6 chromosomes weighted with 30%, 25%, 20%, 15%, 7%, 3%.

Figure E13.3.1 Minimizing $e(x)$ is equivalent to maximizing $f(x)$ (with $\varepsilon = 10^{-3}$).

Figure E13.7.1 Fitness values before (black) and after (blue) linear scaling.

Figure E14.1.1 Convergence curve of XOR problem using differential evolution.

Figure E14.2.1 (a) Optimal function; (b) Optimized function at generation 4681.

Figure 15.4.1 Bipolar sigmoid function with slope parameter $r_g = [2, 1, 0.5, 0.25, 0.125]$ correspond to black, blue, red, green, and magenta color curve respectively.

Figure 15.5.1 Binary sigmoid function with slope parameter $r_f = [2, 1, 0.5, 0.25, 0.125]$ correspond to black, blue, red, green, and magenta color curve respectively.

Figure E15.1.1 (a) Optimal exponential function; (b) Optimized exponential function at generations 1, 34, and 2650.

Figure E15.2.1 (a) Optimal sine function; (b) Optimized sine function at generations 1, 100, and 152.

Figure 17.3.1 Plot of e^{-6x} for $x \in [0, 1]$ for parameter initialization.

Figure 17.3.2 Two 2-side exponential functions for parameter initialization.

Figure 18.2.1 Signal-flow graph of $H_{s1}(z)$ for digital implementation.

Figure 18.2.2 Bipolar sigmoid function $G_s(z)$ ($\tau = 2$) and second-order piecewise function $G_{s1}(z)$ ($\tau = 2$).

Figure E18.1.1 Ten-digit input and output pattern pairs for Design Example 18.1.

Figure E18.2.1 Ten-digit input and output pattern pairs for Design Example 18.2.

Figure 19.4.1 Sparse weights – Top view [Goo1600]. For a 3-element kernel (top), the receptive field consists of only three inputs which are connected to each output. For fully connected (bottom), the receptive field consists of all inputs which are connected to each output.

Figure 19.4.2 Weight sharing [Goo1600]. Black arrows indicate the connections that use a particular weight in two different models. The black arrows (top) indicate uses of the central weight of a 3-element

kernel in a convolutional model. For weight sharing, each central weight is used for connecting the lower central neuron to its corresponding 3 upper neurons and also for connecting the upper central neuron from its 3 lower neurons. The single black arrow (bottom) indicates the use of each central weight of the weight matrix in a fully connected model. This model does not use weight sharing, so each central weight is used only once.

Figure 19.4.3 Equivariant representation [Goo1600]. The receptive field of the neurons in the upper layers of a convolutional network is larger than the receptive field of the neurons in the lower layers.

Figure 19.5.1 The components of a typical convolutional neural network layer. Left: In this terminology, the CNN is viewed as a small number of relatively complex layers, with each layer having many stages. Right: In this terminology, the CNN is viewed as a larger number of simple layers; every step of processing is regarded as a layer.

Figure E19.4.1 Samples of input images.

Figure E19.4.2 Training progress plot showing the mini-batch loss and accuracy and the validation loss and accuracy (=0.9884).

Figure 20.1.1 CIFAR-10 training errors and testing errors for CNN consisting of 20 and 56 layers.

Figure 20.2.1 2-layer residual unit.

Figure 20.4.1 A deeper residual function $F(x)$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 20.1 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

Figure 20.4.2 Example network architectures for ImageNet. Left: the VGG-19 model [Sim1504] (19.6 billion FLOPs) as a reference. Middle: a CNN with 34 parameter layers (3.6 billion FLOPs). Right: a ResNet

with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions.

Figure 20.4.3 ImageNet training errors (thin curves) and validation errors (bold curves) for CNN (left) and ResNets (right) with same number of parameters.

Figure 20.5.1 (a) Original residual unit [He1512a]; (b) Residual unit [He1607]. The wide grey arrows (on left of (a) and (b)) indicate the easiest paths for the information to propagate, corresponding to the additive term $\mathbf{x}_l$ in (20.5.1) (forward propagation).

Figure 20.5.2 Various types of shortcut connections. The grey arrows indicate the easiest paths for the information to propagate. The shortcut connections in (b)-(f) are impeded by different components. For simplicity, the batch normalization layers right after the weight layers for all units are not displayed.

Figure 20.5.3 Training curves on CIFAR-10 of 1001-layer ResNets. Solid lines denote test error (y-axis on right), and dashed lines denote training loss (y-axis on left).

Figure 20.6.1 A block of ResNet (left) [He1512a] and a block of ResNeXt (right) with cardinality = 32, with roughly the same complexity. A layer is labelled by (# in channels, # filter size, # out channels).

Figure 20.6.2 Equivalent building blocks of ResNeXt. (a): Aggregated residual transformations, the same as Figure 20.6.1 (right). (b): A block equivalent to (a), implemented as early concatenation. (c): A block equivalent to (a, b), implemented as grouped convolutions [Kri1212]. Notations in bold text highlight the reformulation changes. A layer is denoted as (# input channels, # filter size, # output channels).

Figure 20.6.3 Training curves on ImageNet-1K. ResNet/ResNeXt-50 (left) with preserved complexity (~ 4.1 billion FLOPs, ~ 25 million parameters); ResNet/ResNeXt-101 (right) with preserved complexity (~ 7.8 billion FLOPs, ~ 44 million parameters).

Part I

Intelligent Computing

I.1 Overview

The Part I on Intelligent Computing consists of one chapter covering an overview of computational intelligence, an overview of artificial intelligence, an overview of intelligent computing as a computing approach to artificial intelligence, an overview of intelligent system design, and an introduction to the basics of biological and artificial neural networks.

Chapter 1

Introduction to Intelligent Computing

1.1 Introduction

In this chapter, overviews of computational intelligence, artificial intelligence, intelligent computing, and intelligent system design are first given.

In addition, the basics of biological neural networks and artificial neural networks are introduced.

For biological neural networks, transmission mechanism, neurons, and synapses are described. The differences between numerical and neural computing are identified.

The definition, structure, features, nonlinear activation functions, and applications of artificial neural networks are presented.

1.2 Overview of Computational Intelligence

Computational intelligence (CI) [WikCoIn] usually refers to the ability of a computer to learn a specific task from external data or experimental observation.

In general, computational intelligence is a set of nature-inspired computational methodologies and approaches to address a complex real-world problem in which the process could be too complex for mathematical reasoning or could contain some uncertainty during the process or could simply be stochastic in nature.

Indeed, many real-world problems cannot be translated into binary language (as unique values of 0 and 1) for computers to process them. Instead fuzzy logic can translate a real-world problem into linguistic variables and term sets and represent them in terms of membership functions for processing by computers.

Computational intelligence approximates the human's reasoning by using inexact and incomplete knowledge to produce actions in an adaptive way.

Computational intelligence employs a combination of different techniques including: (a) Artificial neural networks as a neural system to learn from experiential data to mimic a biological system; (b) Fuzzy logic which enables a fuzzy system to reason and process natural language; (c) The combination of the parallel structure of neural networks and the reasoning of fuzzy system to form fuzzy neural systems; (d) Evolutionary computing based on natural selection and probability for evolution, learning, and optimization; and (e) Back-propagation, supervised learning, unsupervised learning, and reinforcement learning.

1.3 Overview of Artificial Intelligence

Artificial intelligence (AI) [WikArIn], sometimes called machine intelligence, is intelligence realized by machines, in contrast to human intelligence.

Artificial intelligence research can be defined as the study of intelligent agents represented by a device that perceives its environment and takes actions that maximize its chance of success to achieve its goals.

More specifically, artificial intelligence is defined [Kap1901] as a system's ability to correctly interpret external data, to learn from such data, and to use those learnings to achieve specific goals and tasks through flexible adaptation.

The term artificial intelligence is applied when a machine mimics the cognitive functions of human minds, such as learning and problem solving.

Machine learning is the design, study, and development of algorithms to allow machines to learn from external data without human intervention to conduct data mining.

Data mining is to extract meaningful information from a vast amount of data to discover new and useful patterns in the data which can then be used to train machine.

Both machine learning and data mining fall under the field of data science and both processes are used for solving complex problems.

The traditional problems of artificial intelligence research include perception, reasoning, learning, planning, knowledge representation, natural language processing, and the ability to move and manipulate objects. General intelligence is one of the long-term goals.

Approaches include computational intelligence, statistical methods, and traditional symbolic artificial intelligence. Many tools are used in artificial intelligence, including heuristic search and mathematical optimization, and methods based on statistics, probability and economics.

The major areas of artificial intelligence include deep learning neural networks, data mining and machine learning, and natural language processing.

1.4 Overview of Intelligent Computing

The fields of computational intelligence and artificial intelligence are largely originated respectively from electrical engineering research and computer science research. Both fields share many common interests and problems including neural networks and deep learning; supervised, unsupervised, and reinforcement learning; machine learning and data mining; fuzzy neural computing and natural language processing; and heuristic search and evolutionary computing.

In this eBook, the author presents intelligent computing as a computing approach to artificial intelligence by grouping the key topics under four parts consisting of neural computing, fuzzy neural computing, evolutionary computing, and deep neural computing. The key topics covered by computational intelligence and artificial intelligence and their relationships are listed in Table 1.1.

Table 1.1 Key topics in computational intelligence and artificial intelligence and their relationships.

Computational intelligence	Artificial intelligence
Feedforward neural networks	Convolutional neural networks Deep residual networks
Recurrent neural networks Boltzmann machine	Long short-term memory networks Restricted Boltzmann machine Deep belief networks Gated recurrent units
Associative memories	Autoencoders Generative adversarial networks
Multiplierless neural networks	Quantized neural networks
Hebb rule learning Delta rule learning Backpropagation algorithms Mathematical optimization	Deep learning algorithms
Learning vector quantization Backpropagation algorithms Mathematical optimization Support vector machine Neural networks Fuzzy neural networks	Machine learning (Supervised – Classification, Regression)
Self-organizing maps Data clustering algorithms Evolutionary clustering Fuzzy neural networks	Machine learning (Unsupervised – Clustering)
Reinforcement learning	Dynamic decision-making
Radial basis function networks Fuzzy sets, logic & reasoning Fuzzy systems Fuzzy neural networks	Natural language processing
Genetic algorithms (GA)	Heuristic search
Evolutionary algorithms (DE)	
Swarm algorithms (PSO)	
Physical algorithms (SA)	

1.5 Overview of Intelligent System Design

The theory, methods, networks, systems, and algorithms of intelligent computing can be used to train intelligent systems. An overview of intelligent system design is given below.

A system (such as one for signal processing) can be defined as an organized and purposeful entity consisting of interrelated and interdependent processors (or methods) all designed to perform collectively a specific processing task for solving a problem.

A processor can be a shallow or deep neural network, a fuzzy system, a fuzzy neural network, a digital filter, an adaptive digital filter, a discrete transform, and etc.

An intelligent system can be defined as a smart system that has an awareness of its state and operations and can detect, analyze, decide, adapt, and predict.

A practical intelligent system can take advantage of integrating more than one processor, with the fuzzy neural approach or the deep learning approach emerges as a natural choice when decision is to be made using nonlinear processing.

Before proceeding to optimize a shallow or deep neural network, a fuzzy system, a fuzzy neural network, a digital filter, and etc., a parameter vector $\mathbf{c}$ representing the network or system or filter needs to be defined.

The optimization of a shallow or deep neural network, a fuzzy system, and a fuzzy neural network is usually performed in the data domain. Given a number of input to output data pairs, the parameters of each applicable network or system are optimized to acquire learning for classifying or recognizing or interpolating a learnt or unlearned input to its desired output.

For example, a simple 2-layer neural network with a bias at each of the two hidden neurons and the output neuron can be designed to learn the XOR problem as explained in Chapters 4, 13-16.

The optimization of a digital filter is most often performed in the frequency domain to approximate specified passband(s) and stopband(s) to filter out unwanted frequency components of a signal with a filter order depending on the filtering requirement. Design examples are described and explained in [Kwa1803].

1.6 Basics of Biological Neural Networks

1.6.1 Transmission mechanism

There are two major types of mechanism, graded potential and action potential, which are responsible for the transmission of chemical substance (or information) from one neuron to another as described in Table 1.6.1.

Table 1.6.1 Transmission mechanism in neurons.

Mechanism	Description
Graded potential	Graded potential is like diffusion process. Difference in levels of the chemical concentration between 2 adjacent neurons will cause diffusion of chemical substance from the higher level of concentration to the lower level.
Action potential	When the chemical concentration in a neuron reaches up to a certain level, the neuron will fire and transmit excessive chemical substance into adjacent neurons.

1.6.2 Neurons

A typical neuron of a vertebrate animal is shown in Figure 1.6.1. A neuron receives synapses from its dendrites and generates action potentials which travel down the axon.

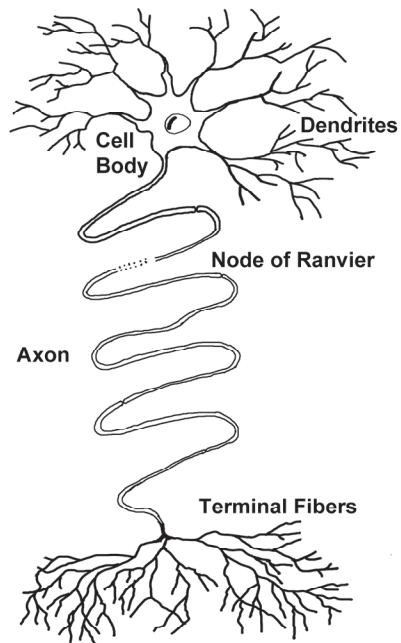


Figure 1.6.1 Sketch of a neuron.

The axon sub-divides into various branches that emit synapses onto dendrites and cell bodies of other neurons.

The neuron can carry nerve impulses for a considerable distance. The nerve impulses originate in the cell body, and are propagated along the axon, which may have one or more branches.

This axon, which is folded for diagrammatic purpose, would be a centimeter long at actual size. Some axons are more than one meter long.

The axon's terminal branches form synapses with as many as 1,000 other neurons.

Most synapses join the axon terminals of one neuron with the dendrites of other neurons. Thus the dendrites surrounding the neuron in the diagram might receive incoming signals from tens, hundreds, or even thousands of other neurons.

Many axons, such as this one, are insulated by a myelin sheath interrupted at intervals by the regions known as nodes of Ranvier.

A general reference for neurons is given by [And8800].

Sketches of some neurons of the cerebral cortex are shown in Figure 1.6.2. Axons' branches may form synapses onto other axons. Dendrites may form synapses onto other dendrites.

Graded potentials (instead of action potentials) can be effective if the axon is short. Graded potentials can also be effective at local levels.

Synapse formed by one axon can be modulated by another axon. Additional details can be found in [Cri8600].

Typical neurons of the cerebral cortex are the pyramidal cells. They are characterized by a pyramid shaped cell body, a thick apical dendrite, and a number of basal (of the base part) dendrites.

There are a variety of different types of pyramidal cells. The remainder of neurons in the cerebral cortex can be broadly referred to as non-pyramidal cells.

As described in Table 1.6.2 and shown in Figure 1.6.2, there are a variety of different types of neurons which include Spindle neurons, Unipolar neurons, Astrocyte neurons, and Purkinje neurons.

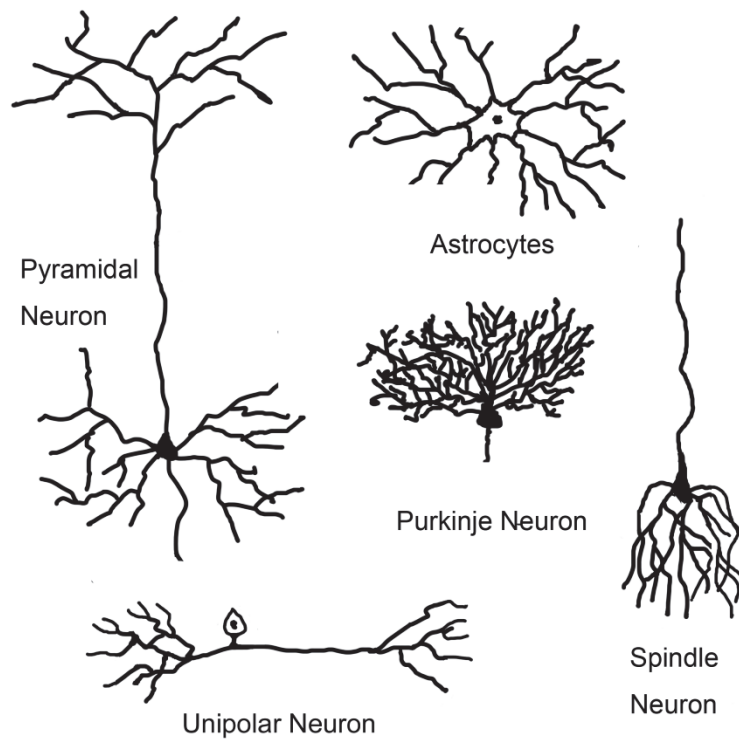


Figure 1.6.2 Some neurons of the cerebral cortex.

Table 1.6.2 Typical types of neurons.

Neuron	Description [WikNeur]
Spindle	A Spindle neuron is characterized by a large spindle-shaped soma, gradually tapering into a single apical axon in one direction, with only a single dendrite facing the opposite direction. Spindle neurons are relatively large cells that may allow rapid communication across the relatively large brains of great apes (chimpanzee, gorilla), elephants, and cetaceans (whales, dolphins). While rare in comparison to other neurons, spindle neurons are abundant, and large, in humans.
Unipolar	A Unipolar neuron has only one protoplasmic process

	(neurite) extends from the cell body. Unipolar neurons are common in insects, where the cell body is often located at the periphery of the brain and is electrically inactive. In all species, including vertebrates and invertebrates, many types of primary sensory neurons are pseudo-unipolar. A pseudo-unipolar neuron is a sensory neuron in the peripheral nervous system. This neuron contains an axon that has split into two branches; one branch runs to the periphery and the other to the spinal cord.
Astrocyte	Astrocyte neurons have no obvious axon but only processes for both receiving and transmitting information. Astrocyte neurons are known collectively as astroglia which are characterized by star-shaped glial cells in the brain and spinal cord. They are the most abundant cell of the human brain. Astrocytes are a sub-type of glial cells in the central nervous system.
Purkinje	These neurons form some of the largest neurons in the human brain. Such a neuron has an intricately elaborate dendritic arbor which is characterized by a large number of dendritic spines. Purkinje neurons are found within the Purkinje layer of the cerebellum. Purkinje cells are aligned like dominos stacked one in front of the other. Purkinje cells send inhibitory projections to the deep cerebellar nuclei, and constitute the sole output of all motor coordination in the cerebellar cortex.

1.6.3 Synapses

The great majority of synapses in the cerebral cortex are chemical. Depending on the size and shape of a neuron, it may receive a few hundred up to tens of thousands of synapses.

Most synaptic contacts can be classified into types I and II [Cri8600] with features as shown in Figures 1.6.3-1.6.4 and listed in Table 1.6.3.

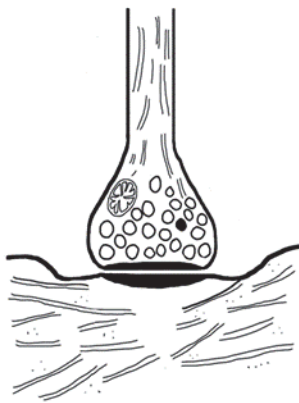


Figure 1.6.3 Type I synaptic contact.

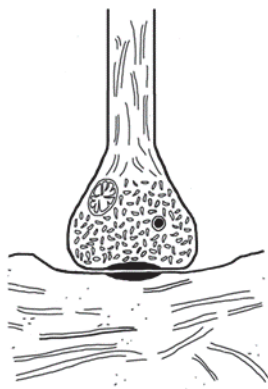


Figure 1.6.4 Type II synaptic contact.

Table 1.6.3 Features of type I (excitatory) synapses and type II (inhibitory) synapses.

Synapse	Features
Type I	(a) The membrane thickening is greater on the postsynaptic side.
	(b) The pre-synaptic process contains fairly large round synaptic vesicles.
	(c) The synaptic gap is about 30 nanometers.
Type II	(a) The membrane thickening is the same on both sides.
	(b) The zone of apposition is usually smaller.
	(c) The synaptic vesicles are smaller and often ellipsoidal in shape.
	(d) The synaptic gap is about 20 nanometers.

1.6.4 Human brain and mind

For interested readers, a number of references in human brain and mind are listed here.

Reference [Gre9600] gives a very readable reference containing many full-color graphics which explains the human mind. Reference [Fra9800] gives an exploration of possible mechanisms of mind, and of their accompanying paradigms of mind. Reference [Fus9900] presents the role of the cerebral cortex of the primate in memory.

Reference [Koc9800] is a handbook of computational methods and techniques for modeling the functional properties of single and groups of nerve cells. Reference [Mac8700] gives a literature review of modeling and theoretical studies of the nervous system and brain from a number of different perspectives.

Reference [Arb8900] gives a collection of chapters with a focus on three major aspects of neural network functions, namely, learning, perception, and action.

Reference [Arb0211] presents the progress made in recent years in the many subfields related to the two great questions: How does the brain work? How can we build intelligent machines?

1.7 Basics of Artificial Neural Networks

1.7.1 Comparisons

Digital computers are fast and accurate for complicated and numerical computations, but are slow and not accurate for the following types of operations:

- a. Pattern recognition: Recognizing m -dimensional ($m > 1$) patterns.
- b. Association: Associating with a set of typical characteristics that some or all stored patterns have.
- c. Generalization: Generalizing to a set of typical characteristics that some or all stored patterns have.
- d. Intelligent decision: Deciding intelligently on a new input situation which has not encountered before.

ANNs perform well in the above weak areas of digital computers and have the following features:

- a. Asynchronous parallel computation: Computations are operating asynchronously and carried out in parallel.
- b. Learning ability: It can learn by updating values stored inside its processing elements.
- c. Error correction: Ability to correct incorrect input information and to identify correct learned information.
- d. Distributed memory: Memory or information is being stored everywhere in an ANN.
- e. Fault tolerance: As a result of redundancy, ability to function normally while having a number of faults or disconnections.

1.7.2 What is an ANN?

An ANN is a computing network which attempts to simulate the structure, interconnections, and interactions of the nerve cells of a biological brain, while having the ability to update continuously its knowledge from experience.

1.7.3 Typical structure of an ANN

1.7.3.1 Structural features

Figure 1.7.1 shows a typical structure of a L -layer feedforward neural network.

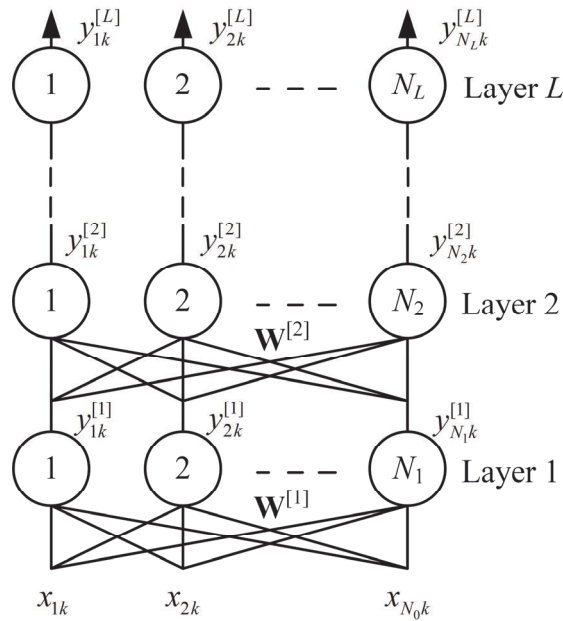


Figure 1.7.1 L -layer feedforward neural network.

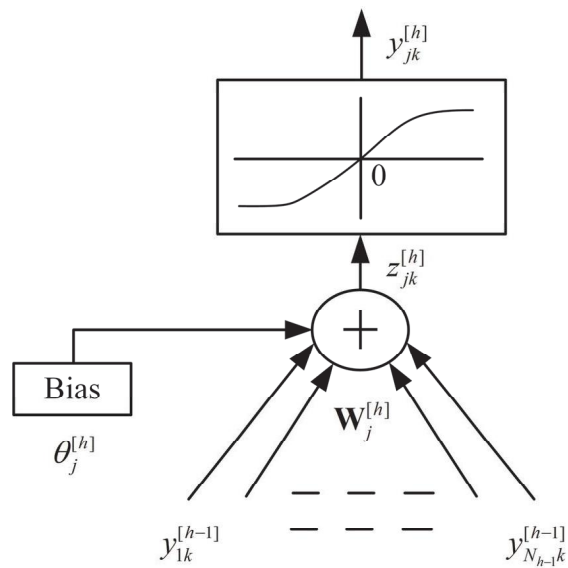


Figure 1.7.2 Internal structure of a neuron.

General and special structural features of an ANN are listed in Table 1.7.1.

Table 1.7.1 General and special structural features of ANN.

General structural features	
1	Each processing element typically stores a bias, one or more adaptive weights and their inputs from corresponding outputs of the previous layer, and a nonlinear function.
2	The operation of each processing element is determined by a given input to output transfer function.
3	Processing elements of the same layer usually have the same transfer function.
4	The interconnection topology among processing elements determines the operations of an ANN.

Special structural features	
1	It contains simple processing elements which are usually operating in parallel and independently.
2	It learns by updating weight values.
3	Its memories are stored in weights distributed throughout all connections.
4	It has a great degree of fault tolerance due to redundant interconnections.

1.7.3.2 Nonlinear quantization functions

Processing elements or neurons used in an ANN are usually nonlinear. As shown in Figure 1.7.2, a processing element sums N weighted inputs together with a bias (or threshold) and passes the sum through a nonlinear function.

Figure 1.7.3 illustrates 3 typical nonlinear quantization functions, namely, step function, ramp function, and sigmoidal function. Both their binary and bipolar versions are shown.

The type of nonlinear function suitable to be used for training artificial neural networks depends on the input to output representation format.

Bipolar sigmoidal function is suitable for real-valued input to output mapping; and binary sigmoidal function is suitable for non-negative-valued input to output mapping.

Bipolar step function is suitable for bipolar step input to output mapping; and binary step function is suitable for binary step input to output mapping.

Bipolar ramp function can be used for bipolar real-valued or bipolar step input to output mapping; and binary ramp function can be used for binary real-valued or binary step input to output mapping.

The operations of these nonlinear quantization functions can be defined by (1.7.1)-(1.7.6).

1.7.3.2.1 Step function

Binary

$$F_h(z) = \begin{cases} 1 & \text{for } z \geq L \\ 0 & \text{for } z < L \end{cases} \quad (1.7.1)$$

Bipolar

$$G_h(z) = \begin{cases} +1 & \text{for } z \geq L \\ -1 & \text{for } z < L \end{cases} \quad (1.7.2)$$

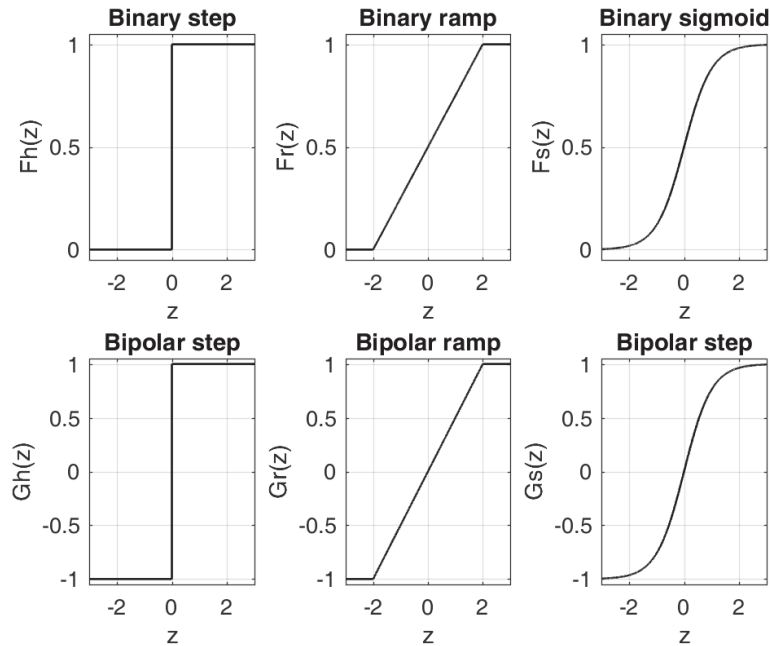


Figure 1.7.3 Typical nonlinear quantization functions: (a) Step function (Binary, $L=0$); (b) Ramp function (Binary, $L=2$); (c) Sigmoid function (Binary, $L=2$); (d) Step function (Bipolar, $L=0$); (e) Ramp function (Bipolar, $L=2$); (f) Sigmoid function (Bipolar, $L=2$).

1.7.3.2.2 Ramp function

Binary

$$F_r(z) = \begin{cases} 1 & \text{for } z \geq L \\ (\theta_f z + 1)/2 & \text{for } -L \leq z \leq L \\ 0 & \text{for } z \leq -L \end{cases} \quad (1.7.3)$$

The parameter θ_f represents the slope of the linear function in the transition region within the input range $-L \leq z \leq L$. The three piecewise functions coincide at $-L$ and L between the respective two adjacent functions.

Bipolar

$$G_r(z) = \begin{cases} 1 & \text{for } z \geq L \\ \theta_g z & \text{for } -L \leq z \leq L \\ -1 & \text{for } z \leq -L \end{cases} \quad (1.7.4)$$

The parameter θ_g represents the slope of the linear function in the transition region within the input range $-L \leq z \leq L$. The three piecewise functions coincide at $-L$ and L between the respective two adjacent functions.

1.7.3.2.3 Sigmoid function

Binary

$$F_s(z) = \frac{1}{1 + e^{-r_f z}} \quad (1.7.5)$$

The parameter r_f determines the slope of the transition region within the input range $-L \leq z \leq L$, as well as determining the value of $\pm L$ in which the function saturates to 1 or 0.

Bipolar

$$G_s(z) = \frac{1 - e^{-r_g z}}{1 + e^{-r_g z}} \quad (1.7.6)$$

The parameter r_g determines the slope of the transition region within the input range $-L \leq z \leq L$, as well as determining the value of $\pm L$ in which the function saturates to ± 1 .

Both the binary and bipolar sigmoidal functions are represented in terms of exponential functions which are complicated to implement directly.

1.7.4 Applications of ANNs

Some applications of ANNs are

- a. Object, image, and pattern recognition regardless to orientation, scale, and small distortion such as object identification seen from different directions; automobile, ship and aircraft identification; and handwritten character recognition and translation [Hec9000].
- b. Independent speech recognition such as continuous independent speech recognition; and spoken language translator [Hec9000].
- c. Nonlinear signal processing such as pulse radar detection [Kwa8906], [Kwa9301]; and noise filtering [Kwa9109a], [Kwa9905], [Tao9905].
- d. Detection of late potentials in electrocardiogram signals [Bay0008], [Bay0108]; and biomedical image analysis [Hec9000];
- e. Data analysis such as prediction of gas-phase adsorption isotherms [Bas0208]; stock market prediction; loan application scoring; big data analysis; and time-series analysis [Hec9000].
- f. Control such as vision-based broomstick balancer; and automobile autopilot [Hec9000].

Part II

Neural Computing

II.1 Overview

A neural network (NN) has a parallel structure which is composed of nonlinear processing elements (called neurons) interconnected through weights. Neural networks (NNs) are built after biological neural systems.

A neural network stores patterns with distributed coding and is a trainable nonlinear dynamic system. A neural network has a faster response due to parallel processing and a higher performance due to nonlinear processing (than that of a sequential digital computer) in emulating the capabilities of the human brain.

Neural network classifiers are model-free estimators. Neural network classifiers do not make assumptions of how outputs depend on inputs. Instead, they adjust themselves to a given training set by a learning algorithm and decide the boundaries of classes.

Traditionally, backpropagation (BP) learning algorithm [Tan9301] has been used to adjust the parameters of a neural network to acquire input to output learning of a given problem.

The main challenge of BP type learning is that it needs many training epochs to converge and this takes a long time to train a network with a large set of training data.

A neural network has a structure of one or more layers and a sigmoidal-type of nonlinear operation at the output of each hidden neuron and the output of each output neuron.

For supervised learning, the number of inputs and the number of outputs to be used are determined by the specified mapping problem under

consideration. For unsupervised learning, the number of outputs is to be determined by the learning algorithm.

The number of hidden neurons to be designed depends on the complexity of the specified mapping problem under consideration.

A two-layer neural network (with one input layer, one hidden layer, and one output layer) is a universal approximator which has an ability to learn any input to output relationship with an appropriate choice on the number of hidden neurons.

Part II on neural computing covers seven chapters including one-layer feedforward neural networks, one-layer recurrent neural networks, multilayer neural networks, associative memories, self-organizing maps, learning vector quantization, and radial basis function networks as presented in Chapters 2 to 8.

The Hebb rule and the delta rule are respectively soft and hard learning algorithms for simple neural networks like one-layer feedforward neural networks and one-layer recurrent neural networks as described in Chapters 2 and 3.

The backpropagation algorithms [Tan9301] are powerful hard learning algorithms for the general types of multilayer feedforward and recurrent neural networks as described in Chapter 4.

Chapter 2

One-Layer Feedforward Neural Networks

2.1 Introduction

In this chapter, one-layer feedforward neural networks are described. We first introduce a one-layer feedforward neural network for fast determination of the maximum or minimum of a set of real values [Kwa9208].

Fast determination of the maximum or minimum of a set of real values has practical applications including max/min operations in fuzzy systems [Kwa9408], [Cai9806].

Maximum determination using related winner-take-all neural networks are also mentioned [Hou9005], [Yen9203].

Then, a simple form of artificial neural networks (ANNs), called pattern associators (PAs) is introduced. A pattern associator can be trained to learn the mapping of input to output patterns using either Hebb rule or delta rule. A classical reference for pattern associators is given by [McC8800].

The topics covered in this Chapter include one-layer MaxNet and MinNet; the structure of pattern associators; Hebb rule learning, recall, and properties; delta rule learning, recall, and properties; and similarities between the two learning rules. The two learning rules and their properties are illustrated with numerical examples.

2.2 1LF-MaxNet for Maximum/Minimum Determination

In [Lip8704], a recursive network, called MAXNET, for determining the maximum of a set of intermediate outputs from the Hamming network had been proposed (box 2 and Figure 6 of [Lip8704]). We call such a MAXNET the recursive MaxNet (R-MaxNet).

In [Lip8704], a modification of the R-MaxNet was suggested to reduce the number of connections, which offers a significant reduction in the number of required calculations.

It was then shown [Lip8704] that the speed of convergence of the R-MaxNet can be increased by adaptively updating the strength of mutual inhibitions between each processing element resulting in an improved R-MaxNet called IMAX.

The R-MaxNet is also called a winner-take-all network which can be implemented using analogue technique [Joh9105] or using optical technique [Pan9003].

According to the simulation results of the six data sets obtained in [Yen9203], the R-MaxNet [Pan9003] requires several thousand recursive iterations and IMAX [Yen9203] requires 8 to 14 recursive iterations to converge to the maximum values of the six data sets.

In general, the numbers of recursive iterations required by the R-MaxNet and any of its modifications are much larger [Hou9005], [Yen9203], [Joh9105], [Pan9003] but they can filter noises contain in the original inputs to determine the embedded maximum.

A multilayer feedforward neural network, which can be implemented digitally, had been suggested [Lip8704] to determine the maximum of a set of real values (Figures 8 and 9 of [Lip8704]). We shall call this multilayer feedforward neural network as the multilayer feedforward MaxNet (MLF-MaxNet).

The advantage of this MLF-MaxNet is that the maximum of a set of real values can be determined in only one feedforward iteration.

In this section, a one-layer feedforward maximum determination network (1LF-MaxNet) [Kwa9208], which requires roughly $1/[3 \log_2(M)]$ (See Table 2.2.1) of the time of the MLF-MaxNet for the maximum determination of M inputs, is described and explained.

Each of the 1LF-MaxNet and 1LF-MinNet can be implemented digitally using two-input adders, hard-limiting elements, and ADD gates.

With an inversion of the hard-limiting function, the 1LF-MaxNet can be converted to a one-layer feedforward minimum determination network (1LF-MinNet) for the minimum determination of a set of real values.

The 1LF-MaxNet/1LF-MinNet is suitable for maximum/minimum value determination in any application, where speed is a critical issue, and where all inputs and outputs are represented in digital format.

2.2.1 One-layer feedforward MaxNet

The structure of the 1LF-MaxNet is shown in Fig. 2.2.1. The network consisted of M neurons for M inputs. The output of the m th neuron is defined as

$$y_m = \text{AND} \{F_{\max}[x_m - x_n] \text{ for } n = 1 \text{ to } M, n \neq m\}, m = 1 \text{ to } M \quad (2.2.1)$$

where $\text{AND}\{\cdot\}$ represents the logic operation of an $(M - 1)$ -input AND gate; and $F_{\max}[\cdot]$ represents the operation of a hard-limiting nonlinear function (NF) which is defined as

$$F_{\max}[z] = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases} \quad (2.2.2)$$

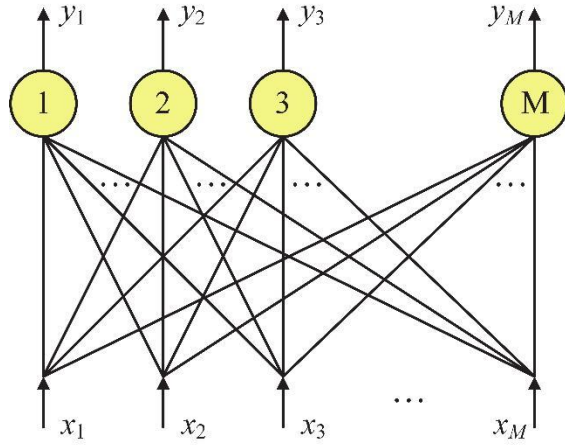


Figure 2.2.1 One-layer MaxNet/MinNet.

The internal structure of the m th neuron of the 1LF-MaxNet is shown in Figure 2.2.2.

It is obvious that the operations defined in (2.2.1) and (2.2.2) will always give a “1” at the output of the neuron with the maximum input and give a “0” at the outputs of all other neurons after one feedforward iteration.

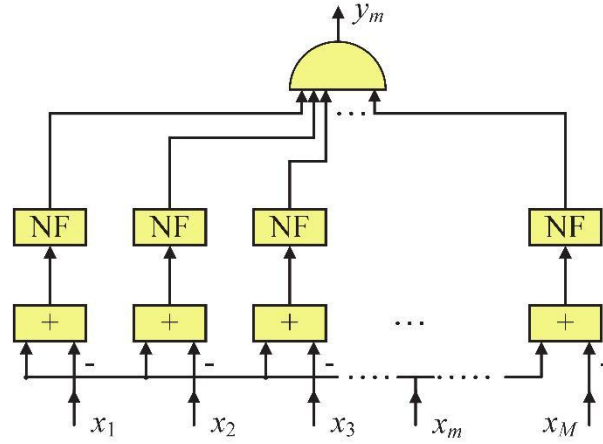


Figure 2.2.2 Internal structure of m th neuron of Figure 2.2.1.

2.2.2 Comparisons

In this Section, we compare the convergence speeds and hardware requirements between the R-MaxNet of the Hamming network, the MLF-MaxNet, and the ILF-MaxNet.

Let the time taken by one two-input addition be T_a , one two-input multiplication T_m , one two-input power-of-two (POT) multiplication T_{m2} , one threshold nonlinear operation T_t , one hardlimiting nonlinear operation T_h , and one $(M - 1)$ -input AND gate operation as T_{AND} .

The R-MaxNet is an iterative network which requires a number of recursive iterations I_r , before converging to one nonzero output at the neuron having the maximum input. Both the MLF-MaxNet and the ILF-MaxNet are feedforward networks; they will take only one feedforward iteration for convergence.

Because the number of inputs of the MLF-MaxNet has to be expressible in $M = 2^L$, so $M = 2^L$ is selected in the comparisons. In the MLF-MaxNet, we assume that:

- (a) All computations are computed immediately in parallel once intermediate data are available without waiting.
- (b) The time taken by one T_h is less than the total time taken by one T_{m2} , two T_a , and one T_t .
- (c) The time taken by one T_a is less than the total time taken by one T_t , and one T_h .
- (d) All the four POT multiplications in each comparator subnet are grouped together as one POT multiplication.

The results of the convergence speed computations are summarized in Table 2.2.1.

As seen from Table 2.2.1, the 1LF-MaxNet is the fastest in terms of convergence speed in which its speed gain increases, as the number of recursive iterations I_r , of the R-MaxNet increases, and also as L (or M) of the MLF-MaxNet increases.

Table 2.2.1 Computational requirements of MaxNets.

MaxNet	Computational requirement
R-MaxNet [Lip8704]	$I_r[MT_a + T_m + T_t]$
MLF-MaxNet [Lip8704]	$(L - 1)[3T_a + T_{m2} + 2T_t]$ $+ [T_a + T_t + T_h] + [T_a + T_h]$
1LF-MaxNet [Kwa9208]	$[T_a + T_h + T_{AND}]$

A comparison of the hardware requirement of the three MaxNets is summarized in Table 2.2.2.

Table 2.2.2 Hardware requirements of MaxNets.

MaxNet	Hardware requirements
R-MaxNet [Lip8704]	$M(M-1)$ -input adders
	M 2-input multipliers
	M 2-input adders
	M thresholding elements
MLF-MaxNet [Lip8704]	$[5(N_c - 1) + 2 + ML]$ 2-input adders
	$[N_c - 1]$ 2-input POT multipliers
	$[3(N_c - 1) + 2]$ thresholding elements
	$[2N_c + M]$ hard-limiting elements
	$N_c = \left\lceil \frac{M}{2} + \frac{M}{4} + \frac{M}{8} + \dots + 1 \right\rceil$
1LF-MaxNet [Kwa9208]	$M(M-1)$ 2-input adders
	$M(M-1)$ hard-limiting elements
	$M(M-1)$ -input AND gates

2.2.3 One-layer feedforward MinNet

With a change of each of the nonlinear functions (NEs) in the 1LF-MaxNet as shown in Figure 2.2.2, from $F_{max}[\cdot]$ to $F_{min}[\cdot]$ as defined in (2.2.3)-(2.2.4) below, the 1LF-MaxNet becomes the 1LF-MinNet which can determine which of the M inputs of the network has the minimum value.

To do so, the output of the m th neuron of the 1LF-MinNet is defined as

$$y_m = \text{AND} \{F_{min}[x_m - x_n] \text{ for } n = 1 \text{ to } M, n \neq m\}, m = 1 \text{ to } M \quad (2.2.3)$$

where $F_{min}[\cdot]$ represents the operation of an inverted hard-limiting nonlinear function which is defined as

$$F_{min}[z] = \begin{cases} 1 & \text{if } z \leq 0 \\ 0 & \text{if } z > 0 \end{cases} \quad (2.2.4)$$

The operations defined in (2.2.3) and (2.2.4) will always give a “1” at the output of the neuron with the minimum input and give a “0” at the outputs of all other neurons after one feedforward iteration.

2.2.4 Summary

A one-layer feedforward MaxNet and MinNet called 1LF-MaxNet and 1LF-MinNet have been described and explained.

The special feature of the 1LF-MaxNet and 1LF-MinNet is its fast convergence speed which takes only a total time of one two-input addition, one hard-limiting operation, and one $(M - 1)$ -input AND gate operation to converge to the maximum of a set of M real values.

This is achieved by a fully parallel digital architecture which enables it to be of practical use in real-time high speed applications in diverse applications such as implementing max/min operations in fuzzy systems.

The convergence speed of the MaxMinNet is faster than those of the MaxNet of the Hamming network and its modified forms, and the multilayer feedforward MaxNet.

2.3 Pattern Associators

A pattern associator (PA) is a one-layer network that is capable of associating specified input patterns to corresponding output patterns through a set of connection weights.

Weights can be trained according to specified input to output mapping requirements. Nonlinear operation is applied at each output.

For illustration, a four-input and one-output PA is shown in Figure 2.3.1 and a four-input and two-output PA is shown in Figure 2.3.2.

The properties of a PA include

- (a) It can learn to act as a content-addressable memory.
- (b) It learns to extract the features of a set of repeated experiences and store them as weights.

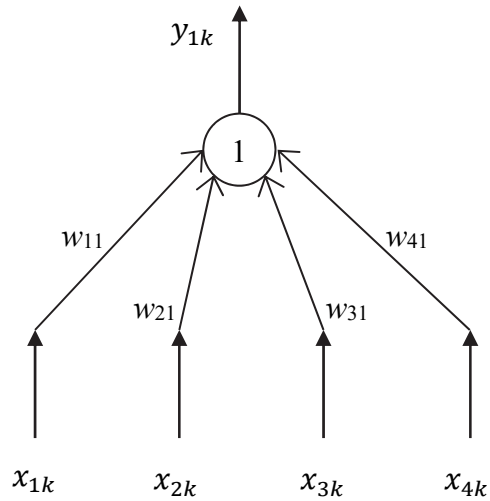


Figure 2.3.1 4-input and 1-output pattern associator.

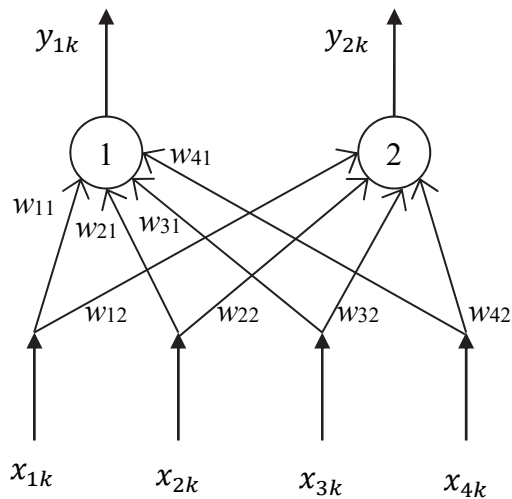


Figure 2.3.2 4-input and 2-output pattern associator.

- (c) It generalizes its responses to new inputs similar to those responses of its trained inputs.
- (d) It has abilities to rectify corrupted and recall incomplete learnt patterns.

- (e) It exhibits central tendency property.
- (f) It can tolerate missing connections or faults.
- (g) Its performances degrade with increasing noise and damage.

2.4 Hebb Rule

2.4.1 Learning

Define

$$\Delta w_{ijk} = \varepsilon x_{ik} y_{jk} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M \quad (2.4.1)$$

where Δw_{ijk} represents the change of weight between the input i and the output j due to the pattern pair k ; x_{ik} represents the input i and y_{jk} represents the output j of the k th input-output pattern pair $(\mathbf{X}_k, \mathbf{Y}_k)$ of dimensions $1 \times N$ and $1 \times M$; and ε is the learning rate parameter.

Starting from all-zero weights, the resultant weight w_{ij} after learning K pattern-pairs is

$$w_{ij} = \sum_{k=1}^K \Delta w_{ijk} = \varepsilon \sum_{k=1}^K x_{ik} y_{jk} \quad (2.4.2a)$$

for $i = 1 \text{ to } N$ and $j = 1 \text{ to } M$

From (2.4.2a), the corresponding weight matrix $\mathbf{W}$ consisting of elements w_{ij} can be expressed as the sum of the vector products of all the K pattern pairs as

$$\mathbf{W} = \varepsilon \sum_{k=1}^K \mathbf{X}_k^T \mathbf{Y}_k \quad (2.4.2b)$$

Alternatively, (2.4.2a-b) can be expressed as

$$\mathbf{W} = \varepsilon \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_K \end{bmatrix}^T \begin{bmatrix} \mathbf{Y}_1 \\ \mathbf{Y}_2 \\ \vdots \\ \mathbf{Y}_K \end{bmatrix} = \varepsilon [\mathbf{X}_1^T \quad \mathbf{X}_2^T \quad \dots \quad \mathbf{X}_K^T] \begin{bmatrix} \mathbf{Y}_1 \\ \mathbf{Y}_2 \\ \vdots \\ \mathbf{Y}_K \end{bmatrix} \quad (2.4.2c)$$

The resultant $N \times M$ weight matrix, $\mathbf{W}$, can be expressed as

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & \dots & w_{1M} \\ w_{21} & w_{22} & w_{23} & \dots & w_{2M} \\ w_{31} & w_{32} & w_{33} & \dots & w_{3M} \\ \dots & \dots & \dots & \dots & \dots \\ w_{N1} & w_{N2} & w_{N3} & \dots & w_{NM} \end{bmatrix} \quad (2.4.3)$$

For bipolar vectors of ± 1 elements, ε could range from $1/(K \max(N, M))$ to $1/\max(N, M)$.

In (2.4.1)-(2.4.2), ε acts as a scaling factor to ensure each of the elements of any output vector $\mathbf{Y}_k$ obtained from (2.4.3) be appropriately bounded before applying the output nonlinear function.

2.4.2 Recall

2.4.2.1 Using weight matrix

After a PA has been trained, the j th output due to an input test vector, $\mathbf{X}_k$, is

$$y_{ik} = G \left[\sum_{i=1}^N x_{ik} w_{ij} \right] \text{ for } j = 1 \text{ to } M \quad (2.4.4a)$$

The corresponding k th output vector $\mathbf{Y}_k$ can be obtained as

$$\mathbf{Y}_k = G[\mathbf{X}_k \mathbf{W}] \quad (2.4.4b)$$

where $G[.]$ represents the operation of the output nonlinear quantization.

For bipolar patterns, the appropriate choice is the bipolar step function $G_h[.]$ defined in (1.3.2).

Example 2.1 (Calculate Hebb-rule weights and recalls of PA)

For the 4-input and 2-output PA as shown in Figure 2.3.2, assuming $G_h[.]$ is used at the output neurons, and that the following 3 pattern pairs as defined in Table E2.1.1 are to be trained. Apply the Hebb rule to

- calculate the weights;
- verify the recall correctness under noiseless condition; and

(c) determine the output of each of the three test vectors given in Table E2.1.2.

Table E2.1.1 Input patterns (1x4) and output patterns (1x2) for learning.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	[1 1 1 1]	[1 1]
2	[1 -1 1 -1]	[1 -1]
3	[1 1 -1 -1]	[-1 1]

Table E2.1.2 Noisy input patterns (1x4) for recalls.

k	$\mathbf{X}_{kt}$
1	[-1 1 1 1]
2	[-1 -1 1 -1]
3	[-1 1 -1 -1]

Solution

(a)

$$\mathbf{W} = \varepsilon \sum_{k=1}^3 \mathbf{X}_k^T \mathbf{Y}_k = \varepsilon \begin{bmatrix} 1 & 1 \\ -1 & 3 \\ 3 & -1 \\ 1 & 1 \end{bmatrix} \quad (\text{E2.1.1})$$

(b)

$$\text{For } \mathbf{X}_t = \mathbf{X}_1 = [1 \quad 1 \quad 1 \quad 1]$$

$$\mathbf{Y}_t = G_h[\mathbf{X}_1 \quad \mathbf{W}] = G_h[4\varepsilon \quad 4\varepsilon] = [1 \quad 1] = \mathbf{Y}_1$$

$$\text{For } \mathbf{X}_t = \mathbf{X}_2 = [1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t = G_h[\mathbf{X}_2 \quad \mathbf{W}] = G_h[4\varepsilon \quad -4\varepsilon] = [1 \quad -1] = \mathbf{Y}_2$$

$$\text{For } \mathbf{X}_t = \mathbf{X}_3 = [1 \quad 1 \quad -1 \quad -1] \quad \mathbf{X}_t = \mathbf{X}_3 = [1 \quad 1 \quad -1 \quad -1]$$

$$\mathbf{Y}_t = G_h[\mathbf{X}_3 \quad \mathbf{W}] = G_h[-4\varepsilon \quad 4\varepsilon] = [-1 \quad 1] = \mathbf{Y}_3$$

(c)

For $\mathbf{X}_t = [-1 \ 1 \ 1 \ 1]$

$$\mathbf{Y}_t = G_h[\mathbf{X}_t \ \mathbf{W}] = G_h[2\varepsilon \ 2\varepsilon] = [1 \ 1] = \mathbf{Y}_1$$

For $\mathbf{X}_t = [-1 \ -1 \ 1 \ -1]$

$$\mathbf{Y}_t = G_h[\mathbf{X}_t \ \mathbf{W}] = G_h[2\varepsilon \ -6\varepsilon] = [1 \ -1] = \mathbf{Y}_2$$

For $\mathbf{X}_t = [-1 \ 1 \ -1 \ -1]$

$$\mathbf{Y}_t = G_h[\mathbf{X}_t \ \mathbf{W}] = G_h[-6\varepsilon \ 2\varepsilon] = [-1 \ 1] = \mathbf{Y}_3$$

2.4.2.2 Using pattern association

It can be shown that the relationship of the output vector $\mathbf{Y}_t$ due to a test input vector $\mathbf{X}_t$ can be expressed by

$$\mathbf{Y}_t = G \left[c \sum_{k=1}^K \mathbf{Y}_k (\mathbf{X}_k \mathbf{X}_t^T / N) \right] \quad (2.4.5)$$

Equation (2.4.5) offers an alternative way to recall the output vector $\mathbf{Y}_t$ due to any test input vector $\mathbf{X}_t$ without using the weight matrix.

The following derivation shows how (2.4.5) can be obtained. Assuming all weights are initially zero, we have

$$w_{ij} = \Delta w_{ij} = \varepsilon x_{ik} y_{jk} \quad (2.4.6)$$

where x_{ik} and y_{jk} represent, respectively, the i th input and j th output of a single learning process k .

The j th output neuron when tested with an input pattern $\mathbf{X}_t$ is

$$y_{it} = G \left[\sum_{i=1}^N x_{it} w_{ij} \right] \quad (2.4.7)$$

Substituting (2.4.6) into (2.4.7), we obtain

$$y_{it} = G \left[\sum_{i=1}^N x_{it} (\varepsilon x_{ik} y_{jk}) \right] = G \left[\varepsilon y_{jk} \sum_{i=1}^N (x_{it} x_{ik}) \right] \quad (2.4.8)$$

$$y_{it} = G [c y_{jk} (\mathbf{X}_k \mathbf{X}_t^T / N)] \quad (2.4.9)$$

where

$$c = N\varepsilon \quad (2.4.10)$$

In general, for a collection of M outputs at the output neurons, we have

$$\mathbf{Y}_t = G [c \mathbf{Y}_k (\mathbf{X}_k \mathbf{X}_t^T / N)] \quad (2.4.11)$$

$$\mathbf{Y}_t = G [c \mathbf{Y}_k] \text{ for } \mathbf{X}_t = \mathbf{X}_k \quad (2.4.12)$$

Equation (2.4.11) implies that the output depends on the similarity between the test pattern and the trained pattern k .

For K pattern-pairs training, we have

$$w_{ij} = \varepsilon \sum_{k=1}^K x_{ik} y_{jk} \quad (2.4.13)$$

Substituting (2.4.13) into (2.4.7), we obtain

$$\begin{aligned} y_{it} &= G \left[\sum_{i=1}^N x_{it} \left(\varepsilon \sum_{k=1}^K x_{ik} y_{jk} \right) \right] \\ &= G \left[\varepsilon \sum_{k=1}^K y_{jk} \left(\sum_{i=1}^N x_{it} x_{ik} \right) \right] = G \left[c \sum_{k=1}^K y_{jk} (\mathbf{X}_k \mathbf{X}_t^T / N) \right] \end{aligned} \quad (2.4.14)$$

For a collection of N outputs at the output neurons, we obtain

$$\mathbf{Y}_t = G \left[c \sum_{k=1}^K \mathbf{Y}_k (\mathbf{X}_k \mathbf{X}_t^T / N) \right] \quad (2.4.15)$$

Equation (2.4.15) implies that if the training input patterns are not orthogonal, wrong association could occur.

Example 2.2 (Test input recalls using Hebb-rule pattern association)

Given four pattern-pairs of $\mathbf{X}_k$ and $\mathbf{Y}_k$ for $k = 1$ to 4 as shown in Table E2.2.1 which are to be trained in a PA using the Hebb rule.

Determine the output vector due to an arbitrary input test vector given by $\mathbf{X}_t = [x_{1k} \ x_{2k} \ x_{3k} \ x_{4k}]$.

Table E2.2.1 Input patterns (1x4) and output patterns (1x4) for learning..

k	$[x_{1k} \ x_{2k} \ x_{3k} \ x_{4k}]$	$[y_{1k} \ y_{2k} \ y_{3k} \ y_{4k}]$
1	$[1 \ 1 \ -1 \ -1]$	$[1 \ -1 \ -1 \ -1]$
2	$[1 \ -1 \ 1 \ -1]$	$[-1 \ 1 \ -1 \ -1]$
3	$[1 \ -1 \ -1 \ 1]$	$[-1 \ -1 \ 1 \ -1]$
4	$[1 \ 1 \ 1 \ 1]$	$[-1 \ -1 \ -1 \ 1]$

Solution

From (2.4.5), we have

$$\begin{aligned}
 \mathbf{Y}_t = & G \left[\frac{c}{N} \{ [1 \ -1 \ -1 \ -1]([1 \ 1 \ -1 \ -1][x_{1t} \ x_{2t} \ x_{3t} \ x_{4t}]^T) \right. \\
 & + [-1 \ 1 \ -1 \ -1]([1 \ -1 \ 1 \ -1][x_{1t} \ x_{2t} \ x_{3t} \ x_{4t}]^T) \\
 & + [-1 \ -1 \ 1 \ -1]([1 \ -1 \ -1 \ 1][x_{1t} \ x_{2t} \ x_{3t} \ x_{4t}]^T) \\
 & \left. + [-1 \ -1 \ -1 \ 1]([1 \ 1 \ 1 \ 1][x_{1t} \ x_{2t} \ x_{3t} \ x_{4t}]^T) \} \right]
 \end{aligned}
 \tag{E2.2.1}$$

2.5 Properties of Hebb-Rule Trained PAs**2.5.1 Pattern rectification**

Pattern rectification is the ability of a pattern associator to retrieve a learned output pattern from a learned input vector that contains one or more incorrect elements.

Example 2.3 (Pattern rectification property of Hebb-rule trained PA)

Consider 4 pattern-pairs of $\mathbf{X}_k$ and $\mathbf{Y}_k$ for $k = 1$ to 4 as shown in Table E2.3.1 which are to be stored in a Hebb-rule trained PA.

Using (2.4.5) and $G_h[.]$, recall the following corrupted version of each of the original input vectors as defined in Table E2.3.2 to illustrate the pattern rectification property of the PA.

Table E2.3.1 Input patterns (1x6) and output patterns (1x6) for learning.

k	$[x_{1k} \ x_{2k} \ x_{3k} \ x_{4k} \ x_{5k} \ x_{6k}]$	$[y_{1k} \ y_{2k} \ y_{3k} \ y_{4k} \ y_{5k} \ y_{6k}]$
1	$[-1 \ -1 \ 1 \ 1 \ 1 \ 1]$	$[-1 \ 1 \ -1 \ 1 \ -1 \ 1]$
2	$[1 \ 1 \ -1 \ -1 \ 1 \ 1]$	$[1 \ -1 \ 1 \ 1 \ -1 \ 1]$
3	$[1 \ 1 \ 1 \ 1 \ -1 \ -1]$	$[1 \ 1 \ -1 \ -1 \ 1 \ 1]$
4	$[1 \ -1 \ 1 \ -1 \ 1 \ -1]$	$[-1 \ 1 \ 1 \ 1 \ 1 \ -1]$

Table E2.3.2 Noisy input patterns (1x6) for recalls.

k	$[x_{t1k} \ x_{t2k} \ x_{t3k} \ x_{t4k} \ x_{t5k} \ x_{t6k}]$
1	$[-1 \ 1 \ 1 \ 1 \ 1 \ 1]$
2	$[1 \ 1 \ 1 \ -1 \ 1 \ 1]$
3	$[1 \ 1 \ 1 \ 1 \ 1 \ -1]$
4	$[-1 \ -1 \ 1 \ -1 \ 1 \ -1]$

Solution

From (2.5), we obtain

$$\begin{aligned} \mathbf{Y}_t &= G \left[c \sum_{k=1}^4 \mathbf{Y}_k (\mathbf{X}_k \mathbf{X}_t^T / 6) \right] \\ &= G_h [c \{ \mathbf{Y}_1 (\mathbf{X}_1 \mathbf{X}_t^T) + \mathbf{Y}_2 (\mathbf{X}_2 \mathbf{X}_t^T) + \mathbf{Y}_3 (\mathbf{X}_3 \mathbf{X}_t^T) + \mathbf{Y}_4 (\mathbf{X}_4 \mathbf{X}_t^T) \} / 6] \end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_t &= G_h \left[\frac{c}{6} \{ [-1 \ 1 \ -1 \ 1 \ -1 \ 1] ([-1 \ -1 \ 1 \ 1 \ 1 \ 1] \mathbf{X}_t^T) \right. \\
&\quad + [1 \ -1 \ 1 \ 1 \ -1 \ 1] ([1 \ 1 \ -1 \ -1 \ 1 \ 1] \mathbf{X}_t^T) \\
&\quad + [1 \ 1 \ -1 \ -1 \ 1 \ 1] ([1 \ 1 \ 1 \ 1 \ -1 \ -1] \mathbf{X}_t^T) \\
&\quad \left. + [-1 \ 1 \ 1 \ 1 \ 1 \ -1] ([1 \ -1 \ 1 \ -1 \ 1 \ -1] \mathbf{X}_t^T) \} \right]
\end{aligned}
\tag{E2.3.1}$$

On substituting each of the four corrupted vectors into the above expression, it can be shown that a correct output recall can be obtained. Thus the property of pattern rectification is illustrated.

2.5.2 Pattern completion

Pattern completion is the ability of a pattern associator to retrieve a learned output pattern from a learned input vector that contains one or more missing elements.

Example 2.4 (Pattern completion property of Hebb-rule trained PA)

Consider the 4 pattern-pairs of $\mathbf{X}_k$ and $\mathbf{Y}_k$ for $k=1$ to 4 as shown in Table E2.3.1 which are to be stored in a Hebb-rule trained PA.

Using $G_h[.]$, recall the incomplete version of each of the original input vectors as given in Table E2.4.1 to illustrate the pattern completion property of the PA.

Table E2.4.1 Incomplete input patterns (1x6) for recalls.

k	$[x_{t1k} \ x_{t2k} \ x_{t3k} \ x_{t4k} \ x_{t5k} \ x_{t6k}]$
1	$[-1 \ 0 \ 1 \ 1 \ 1 \ 1]$
2	$[1 \ 1 \ 0 \ -1 \ 1 \ 1]$
3	$[1 \ 1 \ 1 \ 1 \ 0 \ -1]$
4	$[0 \ -1 \ 1 \ -1 \ 1 \ -1]$

Solution

On substituting each of the 4 incomplete vectors into the expression of

Example 2.3, it can be shown that a correct output recall can be obtained. Thus the property of pattern completion is illustrated.

2.5.3 Fault tolerance

Fault tolerance is the ability of a pattern associator to function normally in the presence of some missing connections or weights.

In other words, a pattern associator can lose or be subject to a corruption of a part of its interconnection weights without affecting its performance.

Example 2.5 (Fault tolerance property of Hebb-rule trained PA)

Assuming that a Hebb-rule trained pattern associator is to be used to store the four pattern pairs as shown in Table E2.5.1, and that the bipolar step function is used at each neuron.

Table E2.5.1 Input patterns (1x4) and output patterns (1x4) for learning.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	[-1 1 1 1]	[1 -1 1 -1]
2	[1 -1 1 1]	[-1 1 -1 1]
3	[1 1 -1 1]	[1 -1 -1 1]
4	[1 1 1 -1]	[-1 1 1 -1]

(a) Given the learning rate parameter is equal to 1, it can be shown that the weight matrix $\mathbf{W}_1$ of the pattern associator can be obtained as

$$\mathbf{W}_1 = \begin{bmatrix} -2 & 2 & -2 & 2 \\ 2 & -2 & 2 & -2 \\ -2 & 2 & 2 & -2 \\ 2 & -2 & -2 & 2 \end{bmatrix} \quad (\text{E2.5.1})$$

It can be verified that this weight matrix $\mathbf{W}_1$ is able to recall correctly all the four pattern-pairs $\mathbf{X}_k$ to $\mathbf{Y}_k$ for $k=1$ to 4.

(b) Given a weight matrix $\mathbf{W}_2$ for the pattern associator as

$$\mathbf{W}_2 = \begin{bmatrix} -2 & 2 & -2 & 0 \\ 2 & -2 & 0 & -2 \\ -2 & 0 & 2 & -2 \\ 0 & -2 & -2 & 2 \end{bmatrix} \quad (\text{E2.5.2})$$

Using this new weight matrix $\mathbf{W}_2$, it can also be verified that this weight matrix $\mathbf{W}_2$ is able to recall correctly all the four pattern-pairs $\mathbf{X}_k$ to $\mathbf{Y}_k$ for $k=1$ to 4.

This property shown in the above example demonstrates that information stored inside a pattern associator is distributed among weights and there is no need to have all the weights of a pattern associator to be connected in order to be fully functional.

2.5.4 Central tendency

Central tendency is the ability of a pattern associator to learn to respond better to the prototype of a set of distorted examples of a category than to any individual distortion.

This is because the characteristics of any individual distorted example tend to average out as more and more distorted examples are presented.

Example 2.6 (Central tendency property of Hebb-rule trained PA)
Matlab

Suppose we want to train a PA using the Hebb rule with noisy versions corrupted by random noise within ± 0.5 of the following input and output patterns as given in Table E2.6.1.

Table E2.6.1 Input patterns (1x4) and output patterns (1x4) for learning.

k	$[x_{1k} \ x_{2k} \ x_{3k} \ x_{4k}]$	$[y_{1k} \ y_{2k} \ y_{3k} \ y_{4k}]$
1	$[1 \ 1 \ 1 \ 1]$	$[1 \ -1 \ 1 \ -1]$
2	$[1 \ -1 \ 1 \ -1]$	$[1 \ 1 \ 1 \ 1]$
3	$[1 \ 1 \ -1 \ -1]$	$[1 \ -1 \ -1 \ 1]$
4	$[1 \ -1 \ -1 \ 1]$	$[1 \ 1 \ -1 \ -1]$

Assuming $G_h[\cdot]$ is used:

- (a) Determine the weight matrix.
- (b) Verify whether the PA can recall correctly all the original output patterns using the original input patterns with and without corruption of random noise.

Solution

- (a) We train the PA with 10 different sets of noisy original input and output patterns and obtain

$$\mathbf{W} = \begin{bmatrix} 10.062 & 0.524 & -0.717 & 0.514 \\ -0.053 & -11.160 & 0.180 & -1.390 \\ 0.499 & -0.589 & 11.258 & -0.822 \\ -0.299 & -1.107 & 0.234 & -9.270 \end{bmatrix} \quad (\text{E2.6.1})$$

- (b) Now we try to recall the original output patterns using the original input patterns corrupted with arbitrary random noise as defined in Table E2.6.2.

Table E2.6.2 Noisy input patterns (1x4) for recalls.

k	$\mathbf{X}_{tk}$
1	[0.500 0.501 1.490 0.542]
2	[1.332 -0.877 1.275 -0.759]
3	[1.075 1.386 -0.990 -0.529]
4	[1.094 -0.806 -1.014 1.346]

From (2.6b), we have

$$\mathbf{Y}_{tk} = G_h[\mathbf{X}_{tk}\mathbf{W}] \quad \text{for } k = 1 \text{ to } 4 \quad (\text{E2.6.2})$$

Hence, we obtain

$$\mathbf{Y}_{t1} = G_h[5.586, -6.819, 16.637, -6.698] = [1 \ -1 \ 1 \ -1]$$

$$\mathbf{Y}_{t2} = G_h[14.316, 10.577, 13.072, 7.895] = [1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_{t3} = G_h[10.407, -13.738, -11.801, -4.344] = [1 \ -1 \ -1 \ -1]$$

$$\mathbf{Y}_{t4} = G_h[10.151, 8.680, -12.036, -9.961] = [1 \ 1 \ -1 \ -1]$$

When using the original input patterns, correct recalls can also be obtained.

2.5.5 Repeated learning

In the Hebb rule, correlations between input and output patterns are not always sufficient to allow a network to learn some simple non-orthogonal associations.

A repeated Hebb rule learning could be used to learn difficult-to-learn pattern pairs to correct the situation.

After the Hebb rule learning, a pattern-pair that cannot be learned could be subsequently learned by repeated learning one or more times on a difficult to learn pattern-pair.

A simple example is given below to illustrate this property.

Example 2.7 (Repeated learning for improved pattern rectification and completion)

Consider a 4-element to 1-element of four pattern-pairs learning problem using the Hebb rule as shown in Table E2.7.1, use the learning rate ε of $1/N$ and the bipolar step function $G_h[z]$ in the recall:

- Calculate the weight vector and show that all the four pattern-pairs can be learned by repeated learning;
- Demonstrate the pattern rectification property; and
- Demonstrate the pattern completion property.

Table E2.7.1 Input patterns (1x4) and output patterns (1x1) for learning.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	$[-1 \ 1 \ -1 \ 1]$	-1
2	$[1 \ -1 \ -1 \ -1]$	1
3	$[1 \ -1 \ 1 \ 1]$	1
4	$[1 \ -1 \ -1 \ 1]$	-1

Solution

(a) The Hebb rule learned weight vector $\mathbf{W}$ can be calculated by

$$\begin{aligned}
 \mathbf{W} &= \varepsilon \sum_{k=1}^4 \mathbf{X}_k^T \mathbf{Y}_k \\
 &= \varepsilon \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \end{bmatrix} [-1] + \varepsilon \begin{bmatrix} 1 \\ -1 \\ -1 \\ -1 \end{bmatrix} [1] + \varepsilon \begin{bmatrix} 1 \\ -1 \\ 1 \\ 1 \end{bmatrix} [1] + \varepsilon \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [-1] \\
 &= \varepsilon \begin{bmatrix} (-1) \times (-1) + 1 \times 1 + 1 \times 1 + 1 \times (-1) \\ 1 \times (-1) + (-1) \times 1 + (-1) \times 1 + (-1) \times (-1) \\ (-1) \times (-1) + (-1) \times 1 + 1 \times 1 + (-1) \times (-1) \\ 1 \times (-1) + (-1) \times 1 + 1 \times 1 + 1 \times (-1) \end{bmatrix} = \varepsilon \begin{bmatrix} 2 \\ -2 \\ 2 \\ -2 \end{bmatrix}
 \end{aligned}
 \tag{E2.7.1}$$

Given the learning rate $\varepsilon = 1/N$ ($N = 4$, $\varepsilon = 0.25$) and the bipolar step function $G_h[z]$, the weight vector $\mathbf{W}$ is used to recall the outputs as

$$\begin{aligned}
 \mathbf{Y}_{1t} &= G_h[\mathbf{X}_1 \mathbf{W}] \\
 &= G_h[0.25\{(-1) \times (2) + (1) \times (-2) + (-1) \times (2) + (1) \times (-2)\}] \\
 &= G_h[-2.0] = -1 = \mathbf{Y}_1 \\
 \mathbf{Y}_{2t} &= G_h[\mathbf{X}_2 \mathbf{W}] \\
 &= G_h[0.25\{(1) \times (2) + (-1) \times (-2) + (-1) \times (2) + (-1) \times (-2)\}] \\
 &= G_h[1.0] = 1 = \mathbf{Y}_2 \\
 \mathbf{Y}_{3t} &= G_h[\mathbf{X}_3 \mathbf{W}] \\
 &= G_h[0.25\{(1) \times (2) + (-1) \times (-2) + (1) \times (2) + (1) \times (-2)\}] \\
 &= G_h[1.0] = 1 = \mathbf{Y}_3 \\
 \mathbf{Y}_{4t} &= G_h[\mathbf{X}_4 \mathbf{W}] \\
 &= G_h[0.25\{(1) \times (2) + (-1) \times (-2) + (-1) \times (2) + (1) \times (-2)\}]
 \end{aligned}$$

$$= G_h[0.0] = 1 \neq \mathbf{Y}_4$$

From this recall, we find that the 4th pattern-pair cannot be learned. We repeat the difficult-to-learn pattern-pair one time to calculate the weight vector $\mathbf{W}_1$ as

$$\mathbf{W}_1 = \mathbf{W} + \varepsilon \mathbf{X}_4^T \mathbf{Y}_4 = \varepsilon \begin{bmatrix} 2 \\ -2 \\ 2 \\ -2 \end{bmatrix} + \varepsilon \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [-1] = \varepsilon \begin{bmatrix} 1 \\ -1 \\ 3 \\ -3 \end{bmatrix} \quad (\text{E2.7.2})$$

Using the learning rate of $\varepsilon = 0.25$ for the weight vector $\mathbf{W}_1$, and the bipolar step function $G_h[z]$, the outputs can be recalled as

$$\begin{aligned} \mathbf{Y}_{1t} &= G_h[\mathbf{X}_1 \mathbf{W}_1] \\ &= G_h[0.25\{(-1) \times (1) + (1) \times (-1) + (-1) \times (3) + (1) \times (-3)\}] \\ &= G_h[-2.0] = -1 = \mathbf{Y}_1 \\ \mathbf{Y}_{2t} &= G_h[\mathbf{X}_2 \mathbf{W}_1] \\ &= G_h[0.25\{(1) \times (1) + (-1) \times (-1) + (-1) \times (3) + (-1) \times (-3)\}] \\ &= G_h[0.5] = 1 = \mathbf{Y}_2 \\ \mathbf{Y}_{3t} &= G_h[\mathbf{X}_3 \mathbf{W}_1] \\ &= G_h[0.25\{(1) \times (1) + (-1) \times (-1) + (1) \times (3) + (1) \times (-3)\}] \\ &= G_h[0.5] = 1 = \mathbf{Y}_3 \\ \mathbf{Y}_{4t} &= G_h[\mathbf{X}_4 \mathbf{W}_1] \\ &= G_h[0.25\{(1) \times (1) + (-1) \times (-1) + (-1) \times (3) + (1) \times (-3)\}] \\ &= G_h[-1.0] = -1 = \mathbf{Y}_4 \end{aligned}$$

The above calculations show that the fourth pattern-pair that cannot be initially learned using the Hebb rule can be subsequently learned by one repeated learning.

(b) Consider a noisy version of the four pattern-pairs with inverted elements highlighted in bold as shown in Table E2.7.2.

Table E2.7.2 Noisy input patterns (1x4) for recalls.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	[-1 -1 -1 1]	-1
2	[1 1 -1 -1]	1
3	[-1 -1 1 1]	1
4	[1 1 -1 1]	-1

Using the learning rate $\varepsilon = 0.25$ for the weight vector $\mathbf{W}_1$, and the bipolar step function $G_h[z]$, the outputs can be recalled as

$$\begin{aligned}
 \mathbf{Y}_{1n} &= G_h[\mathbf{X}_{1n} \mathbf{W}_1] \\
 &= G_h[0.25\{(-1) \times (1) + (-1) \times (-1) + (-1) \times (3) + (1) \times (-3)\}] \\
 &= G_h[-1.5] = -1 = \mathbf{Y}_1
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{Y}_{2n} &= G_h[\mathbf{X}_{2n} \mathbf{W}_1] \\
 &= G_h[0.25\{(1) \times (1) + (1) \times (-1) + (-1) \times (3) + (-1) \times (-3)\}] \\
 &= G_h[0.0] = 1 = \mathbf{Y}_2
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{Y}_{3n} &= G_h[\mathbf{X}_{3n} \mathbf{W}_1] \\
 &= G_h[0.25\{(-1) \times (1) + (-1) \times (-1) + (1) \times (3) + (1) \times (-3)\}] \\
 &= G_h[0.0] = 1 = \mathbf{Y}_3
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{Y}_{4n} &= G_h[\mathbf{X}_{4n} \mathbf{W}_1] \\
 &= G_h[0.25\{(1) \times (1) + (1) \times (-1) + (-1) \times (3) + (1) \times (-3)\}] \\
 &= G_h[-1.5] = -1 = \mathbf{Y}_4
 \end{aligned}$$

The above calculations demonstrate the pattern rectification property of a pattern associator.

(c) Consider an incomplete version of the four pattern-pairs with zero elements highlighted in bold as shown in Table E2.7.3.

Table E2.7.3 Incomplete input patterns (1x4) for recalls.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	[0 1 -1 1]	-1
2	[1 0 -1 -1]	1
3	[1 0 1 1]	1
4	[1 -1 -1 0]	-1

Using the learning rate $\varepsilon = 0.25$ for the weight vector $\mathbf{W}_1$, and the bipolar step function $G_h[z]$, the outputs can be recalled as

$$\begin{aligned}
 \mathbf{Y}_{1m} &= G_h[\mathbf{X}_{1m} \mathbf{W}_1] \\
 &= G_h[0.25\{(0) \times (1) + (1) \times (-1) + (-1) \times (3) + (1) \times (-3)\}] \\
 &= G_h[-1.75] = -1 = \mathbf{Y}_1
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{Y}_{2m} &= G_h[\mathbf{X}_{2m} \mathbf{W}_1] \\
 &= G_h[0.25\{(1) \times (1) + (0) \times (-1) + (-1) \times (3) + (-1) \times (-3)\}] \\
 &= G_h[0.25] = 1 = \mathbf{Y}_2
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{Y}_{3m} &= G_h[\mathbf{X}_{3m} \mathbf{W}_1] \\
 &= G_h[0.25\{(1) \times (1) + (0) \times (-1) + (1) \times (3) + (1) \times (-3)\}] \\
 &= G_h[0.25] = 1 = \mathbf{Y}_3
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{Y}_{4m} &= G_h[\mathbf{X}_{4m} \mathbf{W}_1] \\
 &= G_h[0.25\{(1) \times (1) + (-1) \times (-1) + (-1) \times (3) + (0) \times (-3)\}] \\
 &= G_h[-0.25] = -1 = \mathbf{Y}_4
 \end{aligned}$$

The above calculations demonstrate the pattern completion property of a pattern associator.

2.5.6 Robust learning

To achieve a robust recall of a noisy or incomplete input pattern, a sufficient noise margin between -1 and $+1$ outputs for bipolar patterns

(or between 0 and 1 outputs for binary patterns) must be acquired during learning.

Using the bipolar ramp function for bipolar patterns (or the binary ramp function for binary patterns) with $L > 0$, a robust learning and recall is achievable through multiple applications of repeated learning of the Hebb rule until a sufficient noise margin is attained.

After all the pattern-pairs have been correctly learnt, the bipolar ramp function for bipolar patterns (or the binary ramp function for binary patterns) with $L = 0$ (or each respective step functions) must be used for recalls so that the noise margin obtained through learning can be released to accommodate deviations caused by a noisy input and arrive at correct recalls.

Example 2.8 (Robust learning for improved pattern rectification and completion)

Repeat Example 2.7 using robust learning and state any properties observed.

Solution

(a) Using the learning rate $\varepsilon = 0.25$ and the bipolar ramp function $G_r[z]$ with $L = 1$, the outputs can be recalled using the weight vector $\mathbf{W}_1$ as

$$\begin{aligned} \mathbf{Y}_{1t} &= G_r[\mathbf{X}_1 \mathbf{W}_1] \\ &= G_r[0.25\{(-1) \times (1) + (1) \times (-1) + (-1) \times (3) + (1) \times (-3)\}] \\ &= G_r[-2] = -1 = \mathbf{Y}_1 \end{aligned}$$

$$\begin{aligned} \mathbf{Y}_{2t} &= G_r[\mathbf{X}_2 \mathbf{W}_1] \\ &= G_r[0.25\{(1) \times (1) + (-1) \times (-1) + (-1) \times (3) + (-1) \times (-3)\}] \\ &= G_r[0.5] = 0.5 \neq \mathbf{Y}_2 \end{aligned}$$

$$\begin{aligned} \mathbf{Y}_{3t} &= G_r[\mathbf{X}_3 \mathbf{W}_1] \\ &= G_r[0.25\{(1) \times (1) + (-1) \times (-1) + (1) \times (3) + (1) \times (-3)\}] \end{aligned}$$

$$= G_r[0.5] = 0.5 \neq \mathbf{Y}_3$$

$$\mathbf{Y}_{4t} = G_r[\mathbf{X}_4 \mathbf{W}_1]$$

$$= G_r[0.25\{(1) \times (1) + (-1) \times (-1) + (-1) \times (3) + (1) \times (-3)\}]$$

$$= G_r[-1] = -1 = \mathbf{Y}_4$$

Apply repeated learning to the $\mathbf{X}_2$ & $\mathbf{Y}_2$ and $\mathbf{X}_3$ & $\mathbf{Y}_3$ pattern-pairs, we have

$$\mathbf{W}_2 = \mathbf{W}_1 + \varepsilon \mathbf{X}_2^T \mathbf{Y}_2 + \varepsilon \mathbf{X}_3^T \mathbf{Y}_3$$

$$= \varepsilon \begin{bmatrix} 1 \\ -1 \\ 3 \\ -3 \end{bmatrix} + \varepsilon \begin{bmatrix} 1 \\ -1 \\ -1 \\ -1 \end{bmatrix} [1] + \varepsilon \begin{bmatrix} 1 \\ -1 \\ 1 \\ 1 \end{bmatrix} [1] = \varepsilon \begin{bmatrix} 3 \\ -3 \\ 3 \\ -3 \end{bmatrix} \quad (\text{E2.8.1})$$

Using the learning rate $\varepsilon = 0.25$ and the bipolar ramp function $G_r[z]$ with $L = 1$, the outputs can be recalled using the weight vector $\mathbf{W}_2$ as

$$\mathbf{Y}_{1t} = G_r[\mathbf{X}_1 \mathbf{W}_2]$$

$$= G_r[0.25\{(-1) \times (3) + (1) \times (-3) + (-1) \times (3) + (1) \times (-3)\}]$$

$$= G_r[-3] = -1 = \mathbf{Y}_1$$

$$\mathbf{Y}_{2t} = G_r[\mathbf{X}_2 \mathbf{W}_2]$$

$$= G_r[0.25\{(1) \times (3) + (-1) \times (-3) + (-1) \times (3) + (-1) \times (-3)\}]$$

$$= G_r[1.5] = 1 = \mathbf{Y}_2$$

$$\mathbf{Y}_{3t} = G_r[\mathbf{X}_3 \mathbf{W}_2]$$

$$= G_r[0.25\{(1) \times (3) + (-1) \times (-3) + (1) \times (3) + (1) \times (-3)\}]$$

$$= G_r[1.5] = 1 = \mathbf{Y}_3$$

$$\mathbf{Y}_{4t} = G_r[\mathbf{X}_4 \mathbf{W}_2]$$

$$= G_r[0.25\{(1) \times (3) + (-1) \times (-3) + (-1) \times (3) + (1) \times (-3)\}]$$

$$= G_r[0] = 0 \neq \mathbf{Y}_4$$

Apply repeated learning to the $\mathbf{X}_4$ and $\mathbf{Y}_4$ pattern-pair, we have

$$\mathbf{W}_3 = \mathbf{W}_2 + \varepsilon \mathbf{X}_4^T \mathbf{Y}_4 = \varepsilon \begin{bmatrix} 3 \\ -3 \\ 3 \\ -3 \end{bmatrix} + \varepsilon \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [-1] = \varepsilon \begin{bmatrix} 2 \\ -2 \\ 4 \\ -4 \end{bmatrix} \quad (\text{E2.8.2})$$

Using the learning rate $\varepsilon = 0.25$ and the bipolar ramp function $G_r[z]$ with $L = 1$, the outputs can be recalled using the weight vector $\mathbf{W}_3$ as

$$\begin{aligned} \mathbf{Y}_{1t} &= G_r[\mathbf{X}_1 \mathbf{W}_3] \\ &= G_r[0.25\{(-1) \times (2) + (1) \times (-2) + (-1) \times (4) + (1) \times (-4)\}] \\ &= G_r[-3] = -1 = \mathbf{Y}_1 \\ \mathbf{Y}_{2t} &= G_r[\mathbf{X}_2 \mathbf{W}_3] \\ &= G_r[0.25\{(1) \times (2) + (-1) \times (-2) + (-1) \times (4) + (-1) \times (-4)\}] \\ &= G_r[1.0] = 1 = \mathbf{Y}_2 \\ \mathbf{Y}_{3t} &= G_r[\mathbf{X}_3 \mathbf{W}_3] \\ &= G_r[0.25\{(1) \times (2) + (-1) \times (-2) + (1) \times (4) + (1) \times (-4)\}] \\ &= G_r[1.0] = 1 = \mathbf{Y}_3 \\ \mathbf{Y}_{4t} &= G_r[\mathbf{X}_4 \mathbf{W}_3] \\ &= G_r[0.25\{(1) \times (2) + (-1) \times (-2) + (-1) \times (4) + (1) \times (-4)\}] \\ &= G_r[-1.0] = -1 = \mathbf{Y}_4 \end{aligned}$$

As observed from the above, all the four pattern-pairs have been learnt and correctly recalled.

(b) Consider a noisy version of the four pattern-pairs with inverted elements highlighted in bold as shown in Table E2.7.2, assume the learning rate $\varepsilon = 0.25$ and the bipolar step function $G_h[z]$ is used, the outputs can be recalled using the weight vector $\mathbf{W}_3$ as

$$\begin{aligned}
\mathbf{Y}_{1n} &= G_h[\mathbf{X}_{1n} \mathbf{W}_3] \\
&= G_h[0.25\{(-1) \times (2) + (-1) \times (-2) + (-1) \times (4) + (1) \times (-4)\}] \\
&= G_h[-2.0] = -1 = \mathbf{Y}_1
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{2n} &= G_h[\mathbf{X}_{2n} \mathbf{W}_3] \\
&= G_h[0.25\{(1) \times (2) + (1) \times (-2) + (-1) \times (4) + (-1) \times (-4)\}] \\
&= G_h[0.0] = 1 = \mathbf{Y}_2
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{3n} &= G_h[\mathbf{X}_{3n} \mathbf{W}_3] \\
&= G_h[0.25\{(-1) \times (2) + (-1) \times (-2) + (1) \times (4) + (1) \times (-4)\}] \\
&= G_h[0.0] = 1 = \mathbf{Y}_3
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{4n} &= G_h[\mathbf{X}_{4n} \mathbf{W}_3] \\
&= G_h[0.25\{(1) \times (2) + (1) \times (-2) + (-1) \times (4) + (1) \times (-4)\}] \\
&= G_h[-2.0] = -1 = \mathbf{Y}_4
\end{aligned}$$

The above calculations demonstrate the pattern rectification property of a pattern associator.

(c) Consider an incomplete version of the four pattern-pairs with zero elements highlighted in bold as shown in Table 2.12, assume the learning rate $\varepsilon = 0.25$ and the bipolar step function $G_h[z]$, the outputs can be recalled using the weight vector $\mathbf{W}_3$ as

$$\begin{aligned}
\mathbf{Y}_{1m} &= G_h[\mathbf{X}_{1m} \mathbf{W}_3] \\
&= G_h[0.25\{(0) \times (2) + (1) \times (-2) + (-1) \times (4) + (1) \times (-4)\}] \\
&= G_h[-2.5] = -1 = \mathbf{Y}_1
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{2m} &= G_h[\mathbf{X}_{2m} \mathbf{W}_3] \\
&= G_h[0.25\{(1) \times (2) + (0) \times (-2) + (-1) \times (4) + (-1) \times (-4)\}] \\
&= G_h[0.5] = 1 = \mathbf{Y}_2
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{3m} &= G_h[\mathbf{X}_{3m} \mathbf{W}_3] \\
&= G_h[0.25\{(1) \times (2) + (0) \times (-2) + (1) \times (4) + (1) \times (-4)\}] \\
&= G_h[0.5] = 1 = \mathbf{Y}_3
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{4m} &= G_h[\mathbf{X}_{4m} \mathbf{W}_3] \\
&= G_h[0.25\{(1) \times (2) + (-1) \times (-2) + (-1) \times (4) + (0) \times (-4)\}] \\
&= G_h[0.0] = 1 \neq \mathbf{Y}_4
\end{aligned}$$

It can be observed that insufficient noise margin occurs even after multiple repeated learning in the Hebb rule. Under this circumstance, the problem can be corrected by apply repeated learning to the $\mathbf{X}_4$ and $\mathbf{Y}_4$ pattern-pair as

$$\mathbf{W}_4 = \mathbf{W}_3 + \varepsilon \mathbf{X}_4^T \mathbf{Y}_4 = \varepsilon \begin{bmatrix} 2 \\ -2 \\ 4 \\ -4 \end{bmatrix} + \varepsilon \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [-1] = \varepsilon \begin{bmatrix} 1 \\ -1 \\ 5 \\ -5 \end{bmatrix} \quad (\text{E2.8.3})$$

Using the learning rate $\varepsilon = 0.25$ and the bipolar step function $G_h[z]$, the outputs of the incomplete input patterns of Table E2.7.3 can be recalled using the weight vector $\mathbf{W}_4$ as

$$\begin{aligned}
\mathbf{Y}_{1m} &= G_h[\mathbf{X}_{1m} \mathbf{W}_4] \\
&= G_h[0.25\{(0) \times (1) + (1) \times (-1) + (-1) \times (5) + (1) \times (-5)\}] \\
&= G_h[-2.75] = -1 = \mathbf{Y}_1
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{2m} &= G_h[\mathbf{X}_{2m} \mathbf{W}_4] \\
&= G_h[0.25\{(1) \times (1) + (0) \times (-1) + (-1) \times (5) + (-1) \times (-5)\}] \\
&= G_h[0.25] = 1 = \mathbf{Y}_2
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{3m} &= G_h[\mathbf{X}_{3m} \mathbf{W}_4] \\
&= G_h[0.25\{(1) \times (1) + (0) \times (-1) + (1) \times (5) + (1) \times (-5)\}] \\
&= G_h[0.25] = 1 = \mathbf{Y}_3
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{4m} &= G_h[\mathbf{X}_{4m} \mathbf{W}_4] \\
&= G_h[0.25\{(1) \times (1) + (-1) \times (-1) + (-1) \times (5) + (0) \times (-5)\}] \\
&= G_h[-0.75] = -1 = \mathbf{Y}_4
\end{aligned}$$

The above calculations demonstrate the pattern completion property of a pattern associator.

To verify the pattern rectification ability of the weight vector $\mathbf{W}_4$: Using the learning rate $\varepsilon = 0.25$ and the bipolar step function $G_h[z]$, the outputs of the noisy input patterns of Table E2.7.2 can be recalled as

$$\begin{aligned}
\mathbf{Y}_{1n} &= G_h[\mathbf{X}_{1n} \mathbf{W}_4] \\
&= G_h[0.25\{(-1) \times (1) + (-1) \times (-1) + (-1) \times (5) + (1) \times (-5)\}] \\
&= G_h[-2.5] = -1 = \mathbf{Y}_1
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{2n} &= G_h[\mathbf{X}_{2n} \mathbf{W}_4] \\
&= G_h[0.25\{(1) \times (1) + (1) \times (-1) + (-1) \times (5) + (-1) \times (-5)\}] \\
&= G_h[0.0] = 1 = \mathbf{Y}_2
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{3n} &= G_h[\mathbf{X}_{3n} \mathbf{W}_4] \\
&= G_h[0.25\{(-1) \times (1) + (-1) \times (-1) + (1) \times (5) + (1) \times (-5)\}] \\
&= G_h[0.0] = 1 = \mathbf{Y}_3
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{4n} &= G_h[\mathbf{X}_{4n} \mathbf{W}_4] \\
&= G_h[0.25\{(1) \times (1) + (1) \times (-1) + (-1) \times (5) + (1) \times (-5)\}] \\
&= G_h[-2.5] = -1 = \mathbf{Y}_4
\end{aligned}$$

The above calculations demonstrate the pattern rectification property of a pattern associator.

2.6 Delta Rule

2.6.1 Learning

In the delta rule, the algorithm adjusts the weights in such a way as to reduce the difference between the desired and obtained outputs of a k th pattern-pair. Starting from $k=1$, the obtained j th output is

$$y'_{jk} = G \left[\sum_{i=1}^N x_{ik} w_{ij} \right] \text{ for } j = 1 \text{ to } M \quad (2.6.1a)$$

The corresponding k th output vector of dimensions $1 \times M$ is obtained as

$$\mathbf{Y}'_k = G[\mathbf{X}_k \mathbf{W}] \quad (2.6.1b)$$

For bipolar vectors, $G_r[.]$ is preferred in training to allow a gradual learning. The j th output error e_{jk} is given by

$$e_{jk} = y_{jk} - y'_{jk} \text{ for } j = 1 \text{ to } M \quad (2.6.2a)$$

The corresponding k th output error vector of dimensions $1 \times M$ is given by

$$\mathbf{E}_k = \mathbf{Y}_k - \mathbf{Y}'_k \quad (2.6.2b)$$

where y_{jk} and y'_{jk} represent, respectively, the ideal output and the obtained output at the j th output neuron during the k th pattern-pair presentation. The weight change between the i th input and the j th output neuron is

$$\Delta w_{ijk} = \varepsilon x_{ik} e_{jk} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M \quad (2.6.3a)$$

The corresponding weight matrix change due to the k th pattern-pair is

$$\Delta \mathbf{W}_k = \varepsilon \mathbf{X}_k^T \mathbf{E}_k \quad (2.6.3b)$$

where ε could range from $1/(K \max(N, M))$ to $1/\max(N, M)$ for bipolar vectors. Hence, the updated weight becomes

$$w_{ij} = w_{ij} + \Delta w_{ijk} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M \quad (2.6.4a)$$

The corresponding updated weight matrix of dimensions $N \times M$ is

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_k \quad (2.6.4b)$$

The above procedure is repeated for $k = 1$ to K to complete a training iteration.

The training is repeated for more iterations until the sum of squared errors (SSE) of an iteration falls below a specified level.

Alternatively, a weight can be updated once in an iteration by summing up all the weight changes due to all the pattern-pairs $k = 1$ to K within an iteration according to (2.6.5a-b).

The approach has the advantage of averaging out the effects of all such incremental learning steps to avoid undesirable fluctuations caused by some pattern-pairs. Hence

$$w_{ij} = w_{ij} + \sum_{k=1}^K \Delta w_{ijk} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M \quad (2.6.5a)$$

The corresponding weight matrix update can be carried out as

$$\mathbf{W} = \mathbf{W} + \sum_{k=1}^K \Delta \mathbf{W}_k \quad (2.6.5b)$$

2.6.2 Recall

In the same manner as the Hebb-rule trained PA, after training, the k th input vector can be applied to check recall correctness obtained at the j th output as

$$y'_{jk} = G \left[\sum_{i=1}^N x_{ik} w_{ij} \right] \text{ for } j = 1 \text{ to } M \quad (2.6.6a)$$

The corresponding k th output vector can be expressed as

$$\mathbf{Y}'_k = G[\mathbf{X}_k \mathbf{W}] \quad (2.6.6b)$$

$G_h[.]$ is an appropriate choice for recall.

Example 2.9 (Delta-rule trained pattern associator) Matlab

Assuming the bipolar ramp function $G_r[\mathbf{z}]$ (with $L = 1$) is used for training, $\varepsilon = 0.25$ and zero initial weights, weight update is carried out at the end of each pattern presentation, use the delta rule to obtain the weights of Table E2.9.1.

All computational results are rounded to 2 decimal places.

Table E2.9.1 Input patterns (1x4) and output patterns (1x1) for learning.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	[1 1 1 1]	1
2	[1 -1 1 -1]	1
3	[1 1 -1 -1]	-1
4	[1 -1 1 1]	-1

Solution

The network is started with zero initial weights. When the pattern 1 is presented, from (2.16a), the output is

$$y'_{11} = G_r \left[\sum_{i=1}^4 x_{i1} w_{i1} \right]$$

$$= G_r[(1)(0) + (1)(0) + (1)(0) + (1)(0)] = 0 \quad (E2.9.1)$$

From (2.17a), the error is

$$e_{11} = y_{11} - y'_{11} = 1 - 0 = 1 \quad (E2.9.2)$$

Hence, from (2.18a-2.19a), we have

$$\Delta w_{i1} = \varepsilon x_{i1} e_{11} = 0.25 x_{i1} \quad \text{for } i = 1 \text{ to } 4 \quad (E2.9.3)$$

$$w_{i1} = w_{i1} + \Delta w_{i1} \quad \text{for } i = 1 \text{ to } 4 \quad (E2.9.4)$$

$$\begin{aligned}
& [w_{11} \ w_{21} \ w_{31} \ w_{41}]^T \\
& = [0 + \Delta w_{11} \ 0 + \Delta w_{21} \ 0 + \Delta w_{31} \ 0 + \Delta w_{41}]^T \\
& = [0.25 \ 0.25 \ 0.25 \ 0.25]^T
\end{aligned} \tag{E2.9.5}$$

When the pattern 2 is presented, from (2.16a), the output is

$$\begin{aligned}
y'_{12} &= G_r \left[\sum_{i=1}^4 x_{i2} w_{i1} \right] \\
&= G_r [(1)(0.25) + (-1)(0.25) + (1)(0.25) + (-1)(0.25)] \\
&= 0
\end{aligned} \tag{E2.9.6}$$

Hence from (2.17a), the error is

$$e_{12} = y_{12} - y'_{12} = 1 \tag{E2.9.7}$$

From (2.18a), we have

$$\Delta w_{i1} = \varepsilon x_{i2} e_{12} = 0.25 x_{i2} \quad \text{for } i = 1 \text{ to } 4 \tag{E2.9.8}$$

$$[\Delta w_{11} \ \Delta w_{21} \ \Delta w_{31} \ \Delta w_{41}]^T = [0.25 \ -0.25 \ 0.25 \ -0.25]^T \tag{E2.9.9}$$

From (2.19a), we obtain

$$[w_{11} \ w_{21} \ w_{31} \ w_{41}]^T = [0.50 \ 0.00 \ 0.50 \ 0.00]^T \tag{E2.9.10}$$

When the pattern 3 is presented, from (2.16a), the output is

$$\begin{aligned}
y'_{13} &= G_r \left[\sum_{i=1}^4 x_{i3} w_{i1} \right] \\
&= G_r [(1)(0.5) + (1)(0.0) + (-1)(0.5) + (-1)(0.0)] = 0
\end{aligned} \tag{E2.9.11}$$

Hence from (2.17a), the error is

$$e_{13} = y_{13} - y'_{13} = -1 \tag{E2.9.12}$$

From (2.18a), we have

$$\Delta w_{i1} = \varepsilon x_{i3} e_{13} = -(0.25) x_{i3} \quad \text{for } i = 1 \text{ to } 4 \tag{E2.9.13}$$

$$[\Delta w_{11} \quad \Delta w_{21} \quad \Delta w_{31} \quad \Delta w_{41}]^T = [-0.25 \quad -0.25 \quad 0.25 \quad 0.25]^T \quad (\text{E2.9.14})$$

From (2.19a), we obtain

$$[w_{11} \quad w_{21} \quad w_{31} \quad w_{41}]^T = [0.25 \quad -0.25 \quad 0.75 \quad 0.25]^T \quad (\text{E2.9.15})$$

When the pattern 4 is presented, from (2.16a), the output is

$$y'_{14} = G_r \left[\sum_{i=1}^4 x_{i4} w_{i1} \right] = G_r[1.5] = 0 \quad (\text{E2.9.16})$$

Hence from (2.17a), the error is

$$e_{14} = y_{14} - y'_{14} = -2 \quad (\text{E2.9.17})$$

From (2.18a), we have

$$\Delta w_{i1} = \varepsilon x_{i4} e_{14} = (0.25)(-2)x_{i4} \quad \text{for } i = 1 \text{ to } 4 \quad (\text{E2.9.18})$$

$$[\Delta w_{11} \quad \Delta w_{21} \quad \Delta w_{31} \quad \Delta w_{41}]^T = [-0.5 \quad 0.5 \quad -0.5 \quad -0.5]^T \quad (\text{E2.9.19})$$

From (2.19a), we obtain

$$[w_{11} \quad w_{21} \quad w_{31} \quad w_{41}]^T = [-0.25 \quad 0.25 \quad 0.25 \quad -0.25]^T \quad (\text{E2.9.20})$$

When all the four patterns have been trained once, this completes the first iteration. The resultant sum of squared errors (SSE) is calculated as

$$\text{SSE} = \sum_{j=1}^4 e_{1j}^2 = (1)^2 + (1)^2 + (-1)^2 + (-2)^2 = 7 \quad (\text{E2.9.21})$$

At the start of second iteration, when the pattern 1 is presented, the latest set of weights should be used. Other steps are identical to those in the first iteration.

$$\begin{aligned} y'_{11} &= G_r \left[\sum_{i=1}^4 x_{i1} w_{i1} \right] \\ &= G_r[(1)(-0.25) + (1)(0.25) + (1)(0.25) + (1)(-0.25)] = 0.0 \end{aligned} \quad (\text{E2.9.22})$$

$$e_{11} = y_{11} - y'_{11} = 1 - (0.0) = 1.0 \quad (\text{E2.9.23})$$

$$\Delta w_{i1} = \varepsilon x_{i1} e_{11} = (0.25)x_{i1} \quad \text{for } i = 1 \text{ to } 4 \quad (\text{E2.9.24})$$

$$[\Delta w_{11} \quad \Delta w_{21} \quad \Delta w_{31} \quad \Delta w_{41}]^T = [0.25 \ 0.25 \ 0.25 \ 0.25]^T \quad (\text{E2.9.25})$$

Hence

$$[w_{11} \quad w_{21} \quad w_{31} \quad w_{41}]^T = [[0.00 \ 0.50 \ 0.50 \ 0.00]^T \quad (\text{E2.9.26})$$

The above procedures for the pattern 1 are repeated for patterns 2 to 4, which completes the iteration 2.

The training continues with more iterations until a specified sum of squared output errors or a specified number of iterations are reached. At the end of iteration 15, we have

$$[w_{11} \quad w_{21} \quad w_{31} \quad w_{41}]^T = [-0.98 \ 0.98 \ 1.96 \ -0.98]^T \quad (\text{E2.9.27})$$

With a sufficient number of iterations, the sum of squared errors will eventually be converged to zero with a weight vector becomes

$$[w_{11} \quad w_{21} \quad w_{31} \quad w_{41}]^T = [-1 \ 1 \ 2 \ -1]^T \quad (\text{E2.9.28})$$

2.7 Properties of Delta-Rule Trained PAs

A pattern associator trained by both the Hebb rule and the delta rule shares similar properties of pattern rectification, pattern completion, fault tolerance, and central tendency, but different properties in their abilities to learn.

2.7.1 Central tendency

Example 2.10 (Central tendency property of delta-rule trained PA)
Matlab

Suppose we want to train a PA using noisy versions corrupted by random noise within ± 0.5 of the original input and output patterns as given in Table E2.6.1.

Assuming $G_r[.]$ is used for training with $L = 1$, $G_h[.]$ is used for recall, $\varepsilon = 0.25$, zero initial weights, and the delta rule is used.

- Determine the weight matrix assuming weight update is carried out at the end of each pattern presentation.
- Recall all the original output patterns using the original input

patterns with and without corruption of random noise to illustrate the central tendency capability of the PA.

Solution

(a) We train the PA using a different set of noisy original input and output patterns in each iteration by delta rule. After 100 iterations, we obtain the weight matrix as

$$\mathbf{W} = \begin{bmatrix} 5.378 & -0.844 & 0.670 & 0.431 \\ 0.130 & -3.645 & 0.553 & 1.510 \\ 0.200 & 0.414 & 3.503 & -0.954 \\ -1.618 & 0.073 & 0.652 & -4.136 \end{bmatrix} \quad (\text{E2.10.1})$$

In this example, a large number of iterations of 100 were used to ensure sufficient training to illustrate the central tendency property. Hence the SSE limit was not needed to ensure sufficient training.

(b) Now we try to recall the original output patterns using the original input patterns corrupted with arbitrary random noise as given in Table E2.10.1.

Table E2.10.1 Noisy input patterns (1x4) for recalls.

k	$\mathbf{X}_{tk}$
1	[0.500 0.501 1.490 0.542]
2	[1.332 -0.877 1.275 -0.759]
3	[1.075 1.386 -0.990 -0.529]
4	[1.094 -0.806 -1.014 1.346]

Since

$$\mathbf{Y}_{tk} = G_h[\mathbf{X}_{tk} \mathbf{W}] \text{ for } k = 1 \text{ to } 4 \quad (\text{E2.10.2})$$

Using the corrupted version of the input patterns to recall, we obtain

$$\mathbf{Y}_{t1} = G_h[2.174 \ -1.594 \ 6.188 \ -2.693] = [1 \ -1 \ 1 \ -1]$$

$$\mathbf{Y}_{t2} = G_h[8.534 \ 2.545 \ 4.382 \ 1.172] = [1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_{t3} = G_h[6.620 \ -6.410 \ -2.327 \ 5.691] = [1 \ -1 \ -1 \ 1]$$

$$\mathbf{Y}_{t4} = G_h[3.401 \ 1.693 \ -2.388 \ -5.345] = [1 \ 1 \ -1 \ -1]$$

When using the original input patterns to recall, correct recalls can also be obtained.

2.7.2 Higher ability to learn

In general, the delta rule is a more powerful learning rule as compared to that of the Hebb rule.

A simple example is shown to demonstrate that the delta rule is able to learn and recall successfully one particular difficult to learn pattern-pair whereas the Hebb rule fails to do the same.

Example 2.11 (Difficult pattern learnt by delta-rule trained PA) Matlab

A four-input and one-output pattern associator is to learn the four pattern-pairs as given in Table E2.11.1.

Table E2.11.1 Input patterns (1x4) and output patterns (1x1) for learning.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	$[-1 \quad 1 \quad 1 \quad -1]$	1
2	$[1 \quad -1 \quad -1 \quad 1]$	-1
3	$[1 \quad -1 \quad 1 \quad -1]$	1
4	$[1 \quad 1 \quad 1 \quad -1]$	-1

Assuming the bipolar step function $G_h[\cdot]$ is used at each output and a learning rate $\varepsilon = 1/N$ is used.

- Using the Hebb rule to calculate the weight vector $\mathbf{W}$ and verify the recall correctness at the output of the pattern associator.
- Using the delta rule to calculate the weight vector $\mathbf{W}$ and verify the recall correctness at the output of the pattern associator.
- Explain your observations in the recall results obtained in (a)-(b).

(d) Using the delta-rule trained weight vector, recall some noisy versions of the given four pattern-pairs as shown in Table E2.11.2 to demonstrate the pattern rectification property.

Table E2.11.2 Noisy input patterns (1x4) for recalls.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	$[-1 \quad 1 \quad -1 \quad -1]$	1
2	$[1 \quad 1 \quad -1 \quad 1]$	-1
3	$[-1 \quad -1 \quad 1 \quad -1]$	1
4	$[1 \quad 1 \quad 1 \quad 1]$	-1

(e) Using the delta-rule trained weight vector, recall some incomplete versions of the given four pattern-pairs as shown in Table E2.11.3 to demonstrate the pattern completion property.

Table E2.11.3 Incomplete input patterns (1x4) for recalls.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	$[0 \quad 1 \quad 1 \quad -1]$	1
2	$[1 \quad 0 \quad -1 \quad 1]$	-1
3	$[0 \quad -1 \quad 1 \quad -1]$	1
4	$[1 \quad 1 \quad 1 \quad 0]$	-1

Solution

(a) Using the Hebb rule with a learning rate $\varepsilon = 0.25$, the weight vector $\mathbf{W}$ is

$$\begin{aligned}
 \mathbf{W} &= \varepsilon \sum_{k=1}^4 \mathbf{X}_k^T \mathbf{Y}_k \\
 &= \varepsilon \begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [1] + \varepsilon \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [-1] + \varepsilon \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} [1] + \varepsilon \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [-1]
 \end{aligned}$$

$$= \varepsilon \begin{bmatrix} (-1) \times 1 + 1 \times (-1) + 1 \times 1 + 1 \times (-1) \\ 1 \times 1 + (-1) \times (-1) + (-1) \times 1 + 1 \times (-1) \\ 1 \times 1 + (-1) \times (-1) + 1 \times 1 + 1 \times (-1) \\ (-1) \times 1 + 1 \times (-1) + (-1) \times 1 + (-1) \times (-1) \end{bmatrix} = \varepsilon \begin{bmatrix} -2 \\ 0 \\ 2 \\ -2 \end{bmatrix} \quad (\text{E2.11.1})$$

Using the bipolar step function $G_h[.]$ and the weight $\mathbf{W}$, all the outputs are recalled as

$$\begin{aligned} \mathbf{Y}_{1t} &= G_h[\mathbf{X}_1 \mathbf{W}] \\ &= G_h[0.25\{(-1) \times (-2) + (1) \times (0) + (1) \times (2) + (-1) \times (-2)\}] \\ &= G_h[1.5] = 1 = \mathbf{Y}_1 \\ \mathbf{Y}_{2t} &= G_h[\mathbf{X}_2 \mathbf{W}] \\ &= G_h[0.25\{(1) \times (-2) + (-1) \times (0) + (-1) \times (2) + (1) \times (-2)\}] \\ &= G_h[-1.5] = -1 = \mathbf{Y}_2 \\ \mathbf{Y}_{3t} &= G_h[\mathbf{X}_3 \mathbf{W}] \\ &= G_h[0.25\{(1) \times (-2) + (-1) \times (0) + (1) \times (2) + (-1) \times (-2)\}] \\ &= G_h[0.5] = 1 = \mathbf{Y}_3 \\ \mathbf{Y}_{4t} &= G_h[\mathbf{X}_4 \mathbf{W}] \\ &= G_h[0.25\{(1) \times (-2) + (1) \times (0) + (1) \times (2) + (-1) \times (-2)\}] \\ &= G_h[0.5] = 1 \neq \mathbf{Y}_4 \end{aligned}$$

Among the four given pattern-pairs, $\mathbf{X}_1$ and $\mathbf{Y}_1$ and $\mathbf{X}_2$ and $\mathbf{Y}_2$ are complementary to each other; $\mathbf{X}_3$ is orthogonal to $\mathbf{X}_1$ and $\mathbf{X}_2$; the normalized correlations of $\mathbf{X}_4$ to $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$ are respectively, 0.5, -0.5, and 0.5.

In other words, $\mathbf{X}_4$ is a difficult to learn pattern with a 50% similarity to all other input patterns $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$.

(a) Using the delta rule to learn these four pattern-pairs, we have

$$y'_{jk} = G_h \left[\sum_{i=1}^N x_{ik} w_{ij} \right] \text{ for } j = 1 \text{ to } M$$

$$e_{jk} = y_{jk} - y'_{jk}$$

$$\Delta w_{ijk} = \varepsilon x_{ik} e_{jk} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M$$

$$w_{ij} = w_{ij} + \Delta w_{ijk}$$

or in its matrix format as

$$\mathbf{Y}'_k = G_h[\mathbf{X}_k \mathbf{W}]$$

$$\mathbf{E}_k = \mathbf{Y}_k - \mathbf{Y}'_k$$

$$\Delta \mathbf{W}_k = \varepsilon \mathbf{X}_k^T \mathbf{E}_k$$

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_k$$

for $N=4$ and $M=1$

Assuming zero initial weights, i.e. $\mathbf{W} = [0 \ 0 \ 0 \ 0]^T$.

Iteration 1

$$\begin{aligned} \mathbf{Y}'_1 &= G_h[\mathbf{X}_1 \mathbf{W}] = G_h[(-1)(0) + (1)(0) + (1)(0) + (-1)(0)] \\ &= G_h[0] = 1 \end{aligned}$$

$$\mathbf{E}_1 = \mathbf{Y}_1 - \mathbf{Y}'_1 = 1 - 1 = 0$$

$$\Delta \mathbf{W}_1 = \varepsilon \mathbf{X}_1^T \mathbf{E}_1 = 0.25 \times [-1 \ 1 \ 1 \ -1]^T \times 0 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_1 = [0 \ 0 \ 0 \ 0]^T$$

$$\begin{aligned} \mathbf{Y}'_2 &= G_h[\mathbf{X}_2 \mathbf{W}] = G_h[(1)(0) + (-1)(0) + (-1)(0) + (1)(0)] \\ &= G_h[0] = 1 \end{aligned}$$

$$\mathbf{E}_2 = \mathbf{Y}_2 - \mathbf{Y}'_2 = -1 - 1 = -2$$

$$\Delta \mathbf{W}_2 = \varepsilon \mathbf{X}_2^T \mathbf{E}_2 = [-0.5 \ 0.5 \ 0.5 \ -0.5]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_2 = [-0.5 \ 0.5 \ 0.5 \ -0.5]^T$$

$$\begin{aligned}
\mathbf{Y}'_3 &= G_h[\mathbf{X}_3 \mathbf{W}] \\
&= G_h[(1)(-0.5) + (-1)(0.5) + (1)(0.5) + (-1)(-0.5)] \\
&= G_h[0] = 1
\end{aligned}$$

$$\mathbf{E}_3 = \mathbf{Y}_3 - \mathbf{Y}'_3 = 1 - 1 = 0$$

$$\Delta \mathbf{W}_3 = \varepsilon \mathbf{X}_3^T \mathbf{E}_3 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_3 = [-0.5 \ 0.5 \ 0.5 \ -0.5]^T$$

$$\begin{aligned}
\mathbf{Y}'_4 &= G_h[\mathbf{X}_4 \mathbf{W}] \\
&= G_h[(1)(-0.5) + (1)(0.5) + (1)(0.5) + (-1)(-0.5)] \\
&= G_h[1] = 1
\end{aligned}$$

$$\mathbf{E}_4 = \mathbf{Y}_4 - \mathbf{Y}'_4 = -1 - 1 = -2$$

$$\Delta \mathbf{W}_4 = \varepsilon \mathbf{X}_4^T \mathbf{E}_4 = [-0.5 \ -0.5 \ -0.5 \ 0.5]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_4 = [-1 \ 0 \ 0 \ 0]^T$$

$$\text{SSE} = \mathbf{E}_1^2 + \mathbf{E}_2^2 + \mathbf{E}_3^2 + \mathbf{E}_4^2 = 0^2 + (-2)^2 + 0^2 + (-2)^2 = 8$$

Iteration 2

$$\begin{aligned}
\mathbf{Y}'_1 &= G_h[\mathbf{X}_1 \mathbf{W}] = G_h[(-1)(-1) + (1)(0) + (1)(0) + (-1)(0)] \\
&= G_h[1] = 1
\end{aligned}$$

$$\mathbf{E}_1 = \mathbf{Y}_1 - \mathbf{Y}'_1 = 1 - 1 = 0$$

$$\Delta \mathbf{W}_1 = \varepsilon \mathbf{X}_1^T \mathbf{E}_1 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_1 = [-1 \ 0 \ 0 \ 0]^T$$

$$\begin{aligned}
\mathbf{Y}'_2 &= G_h[\mathbf{X}_2 \mathbf{W}] = G_h[(1)(-1) + (-1)(0) + (-1)(0) + (1)(0)] \\
&= G_h[-1] = -1
\end{aligned}$$

$$\mathbf{E}_2 = \mathbf{Y}_2 - \mathbf{Y}'_2 = -1 - (-1) = 0$$

$$\Delta \mathbf{W}_2 = \varepsilon \mathbf{X}_2^T \mathbf{E}_2 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta\mathbf{W}_2 = [-1 \ 0 \ 0 \ 0]^T$$

$$\begin{aligned}\mathbf{Y}'_3 &= G_h[\mathbf{X}_3\mathbf{W}] = G_h[(1)(-1) + (-1)(0) + (1)(0) + (-1)(-0)] \\ &= G_h[-1] = -1\end{aligned}$$

$$\mathbf{E}_3 = \mathbf{Y}_3 - \mathbf{Y}'_3 = 1 - (-1) = 2$$

$$\Delta\mathbf{W}_3 = \varepsilon\mathbf{X}_3^T\mathbf{E}_3 = [0.5 \ -0.5 \ 0.5 \ -0.5]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta\mathbf{W}_3 = [-0.5 \ -0.5 \ 0.5 \ -0.5]^T$$

$$\begin{aligned}\mathbf{Y}'_4 &= G_h[\mathbf{X}_4\mathbf{W}] \\ &= G_h[(1)(-0.5) + (1)(-0.5) + (1)(0.5) + (-1)(-0.5)] \\ &= G_h[0] = 1\end{aligned}$$

$$\mathbf{E}_4 = \mathbf{Y}_4 - \mathbf{Y}'_4 = -1 - 1 = -2$$

$$\Delta\mathbf{W}_4 = \varepsilon\mathbf{X}_4^T\mathbf{E}_4 = [-0.5 \ -0.5 \ -0.5 \ 0.5]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta\mathbf{W}_4 = [-1 \ -1 \ 0 \ 0]^T$$

$$\text{SSE} = \mathbf{E}_1^2 + \mathbf{E}_2^2 + \mathbf{E}_3^2 + \mathbf{E}_4^2 = 0^2 + 0^2 + 2^2 + (-2)^2 = 8$$

Iteration 3

$$\begin{aligned}\mathbf{Y}'_1 &= G_h[\mathbf{X}_1\mathbf{W}] \\ &= G_h[(-1)(-1) + (1)(-1) + (1)(0) + (-1)(0)] = G_h[0] = 1\end{aligned}$$

$$\mathbf{E}_1 = \mathbf{Y}_1 - \mathbf{Y}'_1 = 1 - 1 = 0$$

$$\Delta\mathbf{W}_1 = \varepsilon\mathbf{X}_1^T\mathbf{E}_1 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta\mathbf{W}_1 = [-1 \ -1 \ 0 \ 0]^T$$

$$\begin{aligned}\mathbf{Y}'_2 &= G_h[\mathbf{X}_2\mathbf{W}] \\ &= G_h[(1)(-1) + (-1)(-1) + (-1)(0) + (1)(0)] = G_h[0] = 1\end{aligned}$$

$$\mathbf{E}_2 = \mathbf{Y}_2 - \mathbf{Y}'_2 = -1 - 1 = -2$$

$$\Delta\mathbf{W}_2 = \varepsilon\mathbf{X}_2^T\mathbf{E}_2 = [-0.5 \ 0.5 \ 0.5 \ -0.5]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta\mathbf{W}_2 = [-1.5 \ -0.5 \ 0.5 \ -0.5]^T$$

$$\mathbf{Y}'_3 = G_h[\mathbf{X}_3\mathbf{W}]$$

$$= G_h[(1)(-1.5) + (-1)(-0.5) + (1)(0.5) + (-1)(-0.5)]$$

$$= G_h[0] = 1$$

$$\mathbf{E}_3 = \mathbf{Y}_3 - \mathbf{Y}'_3 = 1 - 1 = 0$$

$$\Delta\mathbf{W}_3 = \varepsilon\mathbf{X}_3^T\mathbf{E}_3 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta\mathbf{W}_3 = [-1.5 \ -0.5 \ 0.5 \ -0.5]^T$$

$$\mathbf{Y}'_4 = G_h[\mathbf{X}_4\mathbf{W}]$$

$$= G_h[(1)(-1.5) + (1)(-0.5) + (1)(0.5) + (-1)(-0.5)]$$

$$= G_h[-1] = -1$$

$$\mathbf{E}_4 = \mathbf{Y}_4 - \mathbf{Y}'_4 = -1 - (-1) = 0$$

$$\Delta\mathbf{W}_4 = \varepsilon\mathbf{X}_4^T\mathbf{E}_4 = [0 \ 0 \ 0 \ 0]$$

$$\mathbf{W} = \mathbf{W} + \Delta\mathbf{W}_4 = [-1.5 \ -0.5 \ 0.5 \ -0.5]^T$$

$$\text{SSE} = \mathbf{E}_1^2 + \mathbf{E}_2^2 + \mathbf{E}_3^2 + \mathbf{E}_4^2 = 0^2 + (-2)^2 + 0^2 + 0^2 = 4$$

Iteration 4

$$\mathbf{Y}'_1 = G_h[\mathbf{X}_1\mathbf{W}]$$

$$= G_h[(-1)(-1.5) + (1)(-0.5) + (1)(0.5) + (-1)(-0.5)]$$

$$= G_h[2] = 1$$

$$\mathbf{E}_1 = \mathbf{Y}_1 - \mathbf{Y}'_1 = 1 - 1 = 0$$

$$\Delta\mathbf{W}_1 = \varepsilon\mathbf{X}_1^T\mathbf{E}_1 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta\mathbf{W}_1 = [-1.5 \ -0.5 \ 0.5 \ -0.5]^T$$

$$\mathbf{Y}'_2 = G_h[\mathbf{X}_2\mathbf{W}]$$

$$= G_h[(1)(-1.5) + (-1)(-0.5) + (-1)(0.5) + (1)(-0.5)]$$

$$= G_h[-2] = -1$$

$$\mathbf{E}_2 = \mathbf{Y}_2 - \mathbf{Y}'_2 = -1 - (-1) = 0$$

$$\Delta \mathbf{W}_2 = \varepsilon \mathbf{X}_2^T \mathbf{E}_2 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_2 = [-1.5 \ -0.5 \ 0.5 \ -0.5]^T$$

$$\mathbf{Y}'_3 = G_h[\mathbf{X}_3 \mathbf{W}]$$

$$= G_h[(1)(-1.5) + (-1)(-0.5) + (1)(0.5) + (-1)(-0.5)]$$

$$= G_h[0] = 1$$

$$\mathbf{E}_3 = \mathbf{Y}_3 - \mathbf{Y}'_3 = 1 - 1 = 0$$

$$\Delta \mathbf{W}_3 = \varepsilon \mathbf{X}_3^T \mathbf{E}_3 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_3 = [-1.5 \ -0.5 \ 0.5 \ -0.5]^T$$

$$\mathbf{Y}'_4 = G_h[\mathbf{X}_4 \mathbf{W}]$$

$$= G_h[(1)(-1.5) + (1)(-0.5) + (1)(0.5) + (-1)(-0.5)]$$

$$= G_h[0] = 1$$

$$\mathbf{E}_4 = \mathbf{Y}_4 - \mathbf{Y}'_4 = -1 - (-1) = 0$$

$$\Delta \mathbf{W}_4 = \varepsilon \mathbf{X}_4^T \mathbf{E}_4 = [0 \ 0 \ 0 \ 0]^T$$

$$\mathbf{W} = \mathbf{W} + \Delta \mathbf{W}_4 = [-1.5 \ -0.5 \ 0.5 \ -0.5]^T$$

$$\text{SSE} = \mathbf{E}_1^2 + \mathbf{E}_2^2 + \mathbf{E}_3^2 + \mathbf{E}_4^2 = 0^2 + 0^2 + 0^2 + 0^2 = 0$$

After 4 iterations of learning, the SSE becomes zero and the weight vector becomes

$$\mathbf{W} = [-1.5 \ -0.5 \ 0.5 \ -0.5]^T \quad (\text{E2.11.2})$$

Using the bipolar step function $G_h[.]$ to recall all the outputs with the weight vector $\mathbf{W}$, we obtain

$$\begin{aligned}
\mathbf{Y}_{1t} &= G_h[\mathbf{X}_1 \mathbf{W}] \\
&= G_h[\varepsilon\{(-1) \times (-1.5) + (1) \times (-0.5) + (1) \times (0.5) + (-1) \\
&\quad \times (-0.5)\}] \\
&= G_h[2\varepsilon] = 1 = \mathbf{Y}_1 \\
\mathbf{Y}_{2t} &= G_h[\mathbf{X}_2 \mathbf{W}] \\
&= G_h[\varepsilon\{(1) \times (-1.5) + (-1) \times (-0.5) + (-1) \times (0.5) + (1) \\
&\quad \times (-0.5)\}] \\
&= G_h[-2\varepsilon] = -1 = \mathbf{Y}_2 \\
\mathbf{Y}_{3t} &= G_h[\mathbf{X}_3 \mathbf{W}] \\
&= G_h[\varepsilon\{(1) \times (-1.5) + (-1) \times (-0.5) + (1) \times (0.5) + (-1) \\
&\quad \times (-0.5)\}] \\
&= G_h[0] = 1 = \mathbf{Y}_3 \\
\mathbf{Y}_{4t} &= G_h[\mathbf{X}_4 \mathbf{W}] \\
&= G_h[\varepsilon\{(1) \times (-1.5) + (1) \times (-0.5) + (1) \times (0.5) + (-1) \\
&\quad \times (-0.5)\}] \\
&= G_h[-\varepsilon] = -1 = \mathbf{Y}_4
\end{aligned}$$

The above calculations show that the weight vector trained by the delta rule can recall the all the four pattern-pairs correctly.

(c) The calculations in (a)-(b) show that the Hebb rule fails to learn a difficult-to-learn pattern-pair (i.e., the pattern-pair 4) whereas the delta rule is capable of learning all the four pattern-pairs in four iterations.

(d) Using the delta-rule trained weight vector, all the outputs can be recalled from the noisy input vectors as

$$\begin{aligned}\mathbf{Y}_{1t} &= G_h[\mathbf{X}_{1t}\mathbf{W}] \\ &= G_h[\varepsilon\{(-1) \times (-1.5) + (1) \times (-0.5) + (-1) \times (0.5) + (-1) \\ &\quad \times (-0.5)\}] \end{aligned}$$

$$= G_h[\varepsilon] = 1 = \mathbf{Y}_1$$

$$\begin{aligned}\mathbf{Y}_{2t} &= G_h[\mathbf{X}_{2t}\mathbf{W}] \\ &= G_h[\varepsilon\{(1) \times (-1.5) + (1) \times (-0.5) + (-1) \times (0.5) + (1) \\ &\quad \times (-0.5)\}] \end{aligned}$$

$$= G_h[-3\varepsilon] = -1 = \mathbf{Y}_2$$

$$\begin{aligned}\mathbf{Y}_{3t} &= G_h[\mathbf{X}_{3t}\mathbf{W}] \\ &= G_h[\varepsilon\{(-1) \times (-1.5) + (-1) \times (-0.5) + (1) \times (0.5) + (-1) \\ &\quad \times (-0.5)\}] \end{aligned}$$

$$= G_h[3\varepsilon] = 1 = \mathbf{Y}_3$$

$$\begin{aligned}\mathbf{Y}_{4t} &= G_h[\mathbf{X}_{4t}\mathbf{W}] \\ &= G_h[\varepsilon\{(1) \times (-1.5) + (1) \times (-0.5) + (1) \times (0.5) + (1) \\ &\quad \times (-0.5)\}] \end{aligned}$$

$$= G_h[-2\varepsilon] = -1 = \mathbf{Y}_4$$

The above calculations demonstrate the pattern rectification property.

(e) Using the delta-rule trained weight vector, all the outputs can be recalled from the incomplete input vectors as

$$\begin{aligned}\mathbf{Y}_{1t} &= G_h[\mathbf{X}_{1t}\mathbf{W}] \\ &= G_h[\varepsilon\{(0) \times (-1.5) + (1) \times (-0.5) + (1) \times (0.5) + (-1) \\ &\quad \times (-0.5)\}] \end{aligned}$$

$$= G_h[0.5\varepsilon] = 1 = \mathbf{Y}_1$$

$$\begin{aligned}\mathbf{Y}_{2t} &= G_h[\mathbf{X}_{2t}\mathbf{W}] \\ &= G_h[\varepsilon\{(1) \times (-1.5) + (0) \times (-0.5) + (-1) \times (0.5) + (1) \\ &\quad \times (-0.5)\}] \end{aligned}$$

$$= G_h[-2.5\varepsilon] = -1 = Y_2$$

$$Y_{3t} = G_h[X_{3t}W]$$

$$= G_h[\varepsilon\{(0) \times (-1.5) + (-1) \times (-0.5) + (1) \times (0.5) + (-1) \times (-0.5)\}]$$

$$= G_h[1.5\varepsilon] = 1 = Y_3$$

$$Y_{4t} = G_h[X_{4t}W]$$

$$= G_h[\varepsilon\{(1) \times (-1.5) + (1) \times (-0.5) + (1) \times (0.5) + (0) \times (-0.5)\}]$$

$$= G_h[-1.5\varepsilon] = -1 = Y_4$$

The above calculations demonstrate the pattern completion property.

2.8 Similarities between Hebb Rule and Delta Rule

Case 1: In a PA, with orthogonal input patterns and zero initial weights:

At the end of the first iteration:

Both the Hebb rule and the delta rule produce identical weights.

At the end of more training iterations:

For the case of the Hebb rule, the weights will keep on increasing without bound for more and more training iterations (for both orthogonal and non-orthogonal input patterns).

As an identical set of weights is added at the end of each iteration, there is no need to do Hebb rule training after the first iteration.

The delta rule will stop learning after the first iteration as all obtained outputs will be exactly the same as the ideal outputs.

Case 2: In a PA, with non-orthogonal input patterns and zero initial weights:

At the end of the first iteration:

If weights are updated at the end of an iteration, the weights for both the

Hebb rule and the delta rule will be identical (as that of orthogonal input patterns in the Case 1).

If weights are updated at the end of each pattern-pair learning, the weights for both the Hebb rule and the delta rule will not be identical.

At the end of more training iterations:

For the case of the Hebb rule, the weights will keep on increasing (by adding the same set of weights obtained at the end of the first iteration) without bound for each additional training iteration.

If weights are updated at the end of an iteration, the delta rule will stop learning in the Hebb rule manner after the first iteration.

Instead, it adapts the weights and hence the output so as to produce exact target patterns.

If weights are updated at the end of each pattern-pair learning, the delta rule will stop learning in the Hebb rule manner immediately after the first pattern-pair learning.

Chapter 3

One-Layer Recurrent Neural Networks

3.1 Introduction

In this chapter, a one-layer recurrent neural network called autoassociator (AA) is introduced.

Similar to a pattern associators, both the Hebb rule and the delta rule can be used for training an autoassociator.

The topics covered in this Chapter include the structure of the autoassociator, the Hebb rule learning, recall, and properties; the delta rule learning, recall, and properties; and a comparison between the Hebb rule learning and the delta rule learning for autoassociators. Design of autoassociators with increased storage capacity are mentioned at the end. A classical reference for autoassociators is given by [McC9800].

3.2 Autoassociators

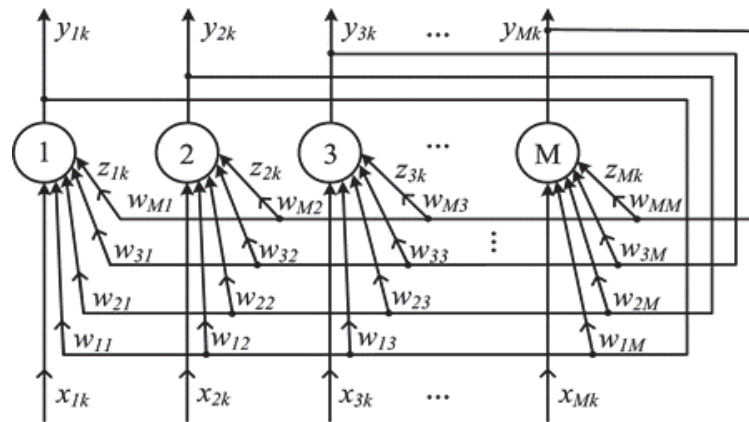


Figure 3.2.1 Autoassociator with M neurons.

An autoassociator (AA) is a one-layer recurrent network that is capable

of associating each of a specified set of input patterns to itself through a set of feedback connection weights.

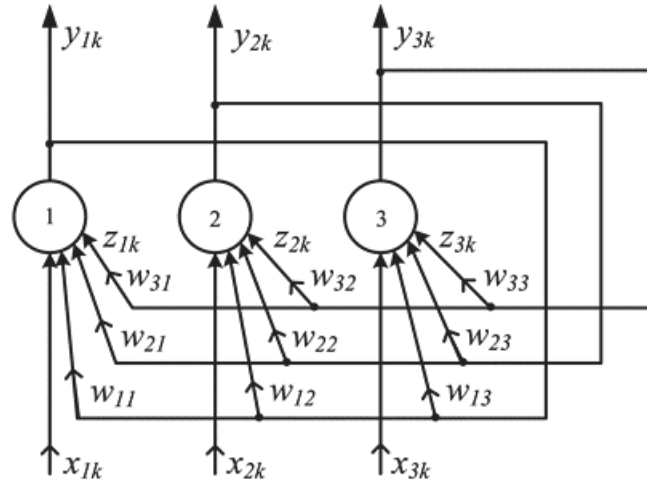


Figure 3.2.2 Autoassociator with $M = 3$ neurons.

For illustration, a M -neuron AA is shown in Figure 3.21 and in Figure 3.22 for $M = 3$. In general:

- (a) Both Hebb rule and delta rule can be used for AA training.
- (b) For an AA with a small number of neurons, self-feedbacks should be avoided to ensure stability if delta rule is used.
- (c) An AA can learn to rectify a distorted learned pattern to its original form and to do pattern completion.
- (d) An AA can also learn to extract the prototype of a set of patterns from distorted exemplars presented during training.

3.3 Hebb Rule

3.3.1 Learning

In the learning process of K patterns by an N -neuron AA, the change in weights due to the first input pattern, $\mathbf{X}_1$, is equal to

$$\delta w_{ij} = \varepsilon x_{i1} x_{j1} \quad \text{for } i, j = 1 \text{ to } N \quad (3.3.1a)$$

$$\delta \mathbf{W} = \varepsilon \mathbf{X}_1^T \mathbf{X}_1 \quad (3.3.1b)$$

where ε is the learning rate parameter. The learning of the pattern $\mathbf{X}_1$ is then completed. Now the second input pattern $\mathbf{X}_2$ will be used instead and so on until all the remaining patterns have been learned.

With zero initial weights, the overall weight matrix will be

$$\mathbf{W} = \varepsilon \sum_{k=1}^K \mathbf{X}_k^T \mathbf{X}_k \quad (3.3.2)$$

There are totally N^2 number of w_{ij} forming a $N \times N$ weight matrix, $\mathbf{W}$ as

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & \dots & w_{1N} \\ w_{21} & w_{22} & w_{23} & \dots & w_{2N} \\ w_{31} & w_{32} & w_{33} & \dots & w_{3N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w_{N1} & w_{N2} & w_{N3} & \dots & w_{NN} \end{bmatrix} \quad (3.3.3)$$

In (3.3.1), ε could range from $(1/KN)$ to $1/N$. In the Hebb rule, ε acts as a scaling factor to ensure all the elements of the internal input vectors, $\mathbf{Z}_k$ for $k = 1$ to K , to be appropriately bounded before applying the nonlinear quantizer.

Example 3.1 (Hebb-rule learning in AA)

Let $\varepsilon = 0.25$, two patterns $\mathbf{X}_1 = [1 \ -1 \ 1 \ -1]$ and $\mathbf{X}_2 = [1 \ -1 \ -1 \ 1]$ are to be stored in a four-neuron AA using the Hebb rule. Assuming zero initial weights, determine all weight values w_{ij} .

Solution

Assuming zero initial weights, from (3.2), we have

$$\mathbf{W} = \varepsilon \sum_{k=1}^K \mathbf{X}_k^T \mathbf{X}_k \quad (\text{E3.1.1})$$

Hence

$$\begin{aligned}
\mathbf{W} &= 0.25 \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} [1 \quad -1 \quad 1 \quad -1] + 0.25 \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [1 \quad -1 \quad -1 \quad 1] \\
&= \begin{bmatrix} .50 & -.50 & .00 & .00 \\ -.50 & .50 & .00 & .00 \\ .00 & .00 & .50 & -.50 \\ .00 & .00 & -.50 & .50 \end{bmatrix} \tag{E3.1.2}
\end{aligned}$$

3.3.2 Recall

The recall algorithm is as follows

$$\mathbf{Y}_t(1) = \mathbf{X}_t \tag{3.3.4}$$

$$\mathbf{Z}_t(n) = \mathbf{Y}_t(n-1)\mathbf{W} \quad \text{for } n \geq 2 \tag{3.3.5}$$

$$\mathbf{Y}_t(n) = G[\mathbf{Z}_t(n)] \quad \text{for } n \geq 2 \tag{3.3.6}$$

where $G[\]$ denotes an output nonlinear function.

The procedures described in (3.3.5) and (3.3.6) are repeated until $\mathbf{Y}_t(n)$ converges to $\mathbf{Y}_t(n-1)$. For bipolar input and output patterns, the bipolar step function is an appropriate choice for recall.

Example 3.2 (Input pattern recalls in AA) Matlab

Assuming that the bipolar step function is used, for the test input vectors, $\mathbf{X}_1$ and $\mathbf{X}_2$ in the Example 3.1, compute the test output vectors.

Solution

From Example 3.1, the weight matrix of the AA is equal to

$$\mathbf{W} = \begin{bmatrix} .50 & -.50 & .00 & .00 \\ -.50 & .50 & .00 & .00 \\ .00 & .00 & .50 & -.50 \\ .00 & .00 & -.50 & .50 \end{bmatrix} \tag{E3.2.1}$$

The test output vectors can be computed as

For $k=1$:

$$\mathbf{X}_t = \mathbf{X}_1$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t = [1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = [1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = [1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = [1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \quad -1 \quad 1 \quad -1]$$

...

For $k=2$:

$$\mathbf{X}_t = \mathbf{X}_2$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t = [1 \quad -1 \quad -1 \quad 1]$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = [1 \quad -1 \quad -1 \quad 1]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \quad -1 \quad -1 \quad 1]$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = [1 \quad -1 \quad -1 \quad 1]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \quad -1 \quad -1 \quad 1]$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = [1 \quad -1 \quad -1 \quad 1]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \quad -1 \quad -1 \quad 1]$$

...

In each of the two cases, the test output vector converges to its correct recall output after the second recall iteration or after one iteration.

3.4 Properties of Hebb-Rule Trained AAs

For a Hebb-rule trained Pattern Associator, from (2.3.5) we obtain

$$\mathbf{Y}_t = G \left[c \sum_{k=1}^K \mathbf{Y}_k \{(\mathbf{X}_k)(\mathbf{X}_t)^T / N\} \right] \quad (3.4.1)$$

replacing the training output vectors $\mathbf{Y}_k$ with the training input vector $\mathbf{X}_k$, (3.4.1) becomes

$$\mathbf{Y}_t = G \left[c \sum_{k=1}^K \mathbf{X}_k \{(\mathbf{X}_k)(\mathbf{X}_t)^T / N\} \right] \quad (3.4.2)$$

Equation (3.4.2) gives the test output vector of a Hebb-rule trained AA.

With the use of bipolar input and output vectors, the bipolar step function $G_h[\]$ is applied at the test output vector as

$$\mathbf{Y}_t(n) = G_h \left[c \sum_{k=1}^K \mathbf{X}_k \{(\mathbf{X}_k \mathbf{Y}_t(n-1))^T / N\} \right] \quad \text{for } n \geq 2 \quad (3.4.3)$$

3.4.1 Pattern rectification

Pattern rectification is an ability to retrieve the original pattern from an external input vector which contains one or more incorrect elements.

Example 3.3 (Pattern rectification property of Hebb-rule trained AA)
Matlab

Consider two patterns $\mathbf{X}_1$ and $\mathbf{X}_2$ which are to be stored in a Hebb-rule trained AA as

$$\mathbf{X}_1 = [1 \quad 1 \quad 1 \quad 1 \quad -1 \quad -1 \quad -1 \quad -1]$$

$$\mathbf{X}_2 = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1]$$

where $c = N\varepsilon = 1$. Given a corrupted version of $\mathbf{X}_1$, denoted by $\mathbf{X}_t$, as an external input test vector as

$$\mathbf{X}_t = [1 \quad 1 \quad 1 \quad 1 \quad -1 \quad -1 \quad 1 \quad -1]$$

Assuming that the bipolar step function is used, show whether the AA can be used to retrieve the original pattern $\mathbf{X}_1$.

Solution

To illustrate the pattern rectification capability of the AA, from (3.4.3), we have

$$\mathbf{Y}_t(n) = G_h\{c[(\mathbf{X}_1\mathbf{X}_1\mathbf{Y}_t(n-1)^T)/8 + (\mathbf{X}_2^T\mathbf{X}_2\mathbf{Y}_t(n-1)^T)/8]\} \quad (\text{E3.3.1})$$

Hence

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Y}_t(2) = G_h\{c[(\mathbf{X}_1\mathbf{X}_1\mathbf{Y}_t(1)^T)/8 + (\mathbf{X}_2^T\mathbf{X}_2\mathbf{Y}_t(1)^T)/8]\}$$

$$= G_h\{c[0.75\mathbf{X}_1 + 0.25\mathbf{X}_2]\}$$

$$= G_h\{[1 \quad 0.5 \quad 1 \quad 0.5 - 0.5 - 1 - 0.5 - 1]\} = \mathbf{X}_1$$

$$\mathbf{Y}_t(3) = \mathbf{X}_1$$

$$\mathbf{Y}_t(4) = \mathbf{X}_1$$

...

$\mathbf{Y}_t(n)$ does converge to $\mathbf{X}_1$ for $n \geq 2$.

3.4.2 Pattern completion

Pattern completion is an ability to retrieve the original pattern from an external input vector which contains one or more missing elements.

Example 3.4 (Pattern completion property of Hebb-rule trained AA)
Matlab

For the Example 3.3, given an incomplete version of $\mathbf{X}_2$ to be used as an external input test vector, such that

$$\mathbf{X}_t = [1 \quad -1 \quad 1 \quad -1 \quad 0 \quad 0 \quad 0 \quad 0]$$

Assuming that the bipolar step function is used, show how the AA can be used to retrieve the original pattern $\mathbf{X}_2$.

Solution

From (3.9), we obtain

$$\begin{aligned} \mathbf{Y}_t(n) \\ = G_h\{c[(\mathbf{X}_1\mathbf{X}_1\mathbf{Y}_t(n-1)^T)/8 + (\mathbf{X}_2^T\mathbf{X}_2\mathbf{Y}_t(n-1)^T)/8]\} \end{aligned} \quad (\text{E3.4.1})$$

Hence

$$\begin{aligned} \mathbf{Y}_t(1) &= \mathbf{X}_t \\ \mathbf{Y}_t(2)^T &= G_h\{c[(\mathbf{X}_1\mathbf{X}_1\mathbf{Y}_t(1)^T)/8 + (\mathbf{X}_2^T\mathbf{X}_2\mathbf{Y}_t(1)^T)/8]\} \\ &= G_h\{c[0.0\mathbf{X}_1 + 0.5\mathbf{X}_2]\} \\ &= G_h\{[0.5 - 0.5 \quad 0.5 - 0.5 \quad 0.5 - 0.5 \quad 0.5 - 0.5]\} = \mathbf{X}_2 \end{aligned}$$

$$\mathbf{Y}_t(3) = \mathbf{X}_2$$

$$\mathbf{Y}_t(4) = \mathbf{X}_2$$

...

$\mathbf{X}_t$ does converge to $\mathbf{X}_2$ for $n \geq 2$.

3.4.3 Central tendency

An AA has an ability to learn to respond better to the prototype of a set of distorted exemplars of a category than to any individual distortion.

This is because the characteristics of any individual exemplar tend to average out as more and more exemplars are presented.

Example 3.5 (Central tendency property of Hebb-rule trained AA)
Matlab

Suppose we want to store a pattern $\mathbf{X} = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$ in an AA using the Hebb rule. However, the pattern itself is not available but we have some noisy (or distorted) versions of $\mathbf{X}$ at hand. For example,

assuming that we have the following eight distorted patterns of $\mathbf{X}$ as given by

$$\mathbf{X}_1 = [-1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_2 = [1 \ -1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_3 = [1 \ 1 \ -1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_4 = [1 \ 1 \ 1 \ -1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_5 = [1 \ 1 \ 1 \ 1 \ -1 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_6 = [1 \ 1 \ 1 \ 1 \ 1 \ -1 \ 1 \ 1]$$

$$\mathbf{X}_7 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1 \ 1]$$

$$\mathbf{X}_8 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1]$$

- (a) Determine the weight matrix of the AA in order to store $\mathbf{X}$.
- (b) Assuming that the bipolar step function is used, check whether the stored pattern $\mathbf{X}$ can be recalled from all of the eight distorted forms of $\mathbf{X}$.
- (c) What property of the Hebb-rule trained AA has this example demonstrated?

Solution

- (a) We train the AA with these 8 distorted patterns by using Hebb rule and obtain the weight matrix as

$$\mathbf{W} = \varepsilon \sum_{k=1}^8 \mathbf{X}_k^T \mathbf{X}_k = \varepsilon \begin{bmatrix} 8 & 4 & 4 & 4 & 4 & 4 & 4 & 4 \\ 4 & 8 & 4 & 4 & 4 & 4 & 4 & 4 \\ 4 & 4 & 8 & 4 & 4 & 4 & 4 & 4 \\ 4 & 4 & 4 & 8 & 4 & 4 & 4 & 4 \\ 4 & 4 & 4 & 4 & 8 & 4 & 4 & 4 \\ 4 & 4 & 4 & 4 & 4 & 8 & 4 & 4 \\ 4 & 4 & 4 & 4 & 4 & 4 & 8 & 4 \\ 4 & 4 & 4 & 4 & 4 & 4 & 4 & 8 \end{bmatrix} \quad (\text{E3.5.1})$$

- (b) Now we try to recall $\mathbf{X}$ by using $\mathbf{X}_1$ through $\mathbf{X}_8$ as

For $k=1$:

$$\mathbf{X}_t = \mathbf{X}_1 = [-1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[20 \ 28 \ 28 \ 28 \ 28 \ 28 \ 28 \ 28]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

...

For $k=2$:

$$\mathbf{X}_t = \mathbf{X}_2 = [1 \ -1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[28 \ 20 \ 28 \ 28 \ 28 \ 28 \ 28 \ 28]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

...

For $k=3$:

$$\mathbf{X}_t = \mathbf{X}_3 = [1 \ 1 \ -1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[28 \ 28 \ 20 \ 28 \ 28 \ 28 \ 28 \ 28]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

...

For $k=4$:

$$\mathbf{X}_t = \mathbf{X}_4 = [1 \ 1 \ 1 \ -1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[28 \ 28 \ 28 \ 20 \ 28 \ 28 \ 28 \ 28]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

...

For $k=5$:

$$\mathbf{X}_t = \mathbf{X}_1 = [1 \ 1 \ 1 \ 1 \ -1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[28 \ 28 \ 28 \ 28 \ 20 \ 28 \ 28 \ 28]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

...

For $k=6$:

$$\mathbf{X}_t = \mathbf{X}_2 = [1 \ 1 \ 1 \ 1 \ 1 \ -1 \ 1 \ 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[28 \ 28 \ 28 \ 28 \ 28 \ 20 \ 28 \ 28]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

...

For $k=7$:

$$\mathbf{X}_t = \mathbf{X}_3 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1 \ 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[28 \ 28 \ 28 \ 28 \ 28 \ 28 \ 20 \ 28]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

...

For $k=8$:

$$\mathbf{X}_t = \mathbf{X}_4 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[28 \ 28 \ 28 \ 28 \ 28 \ 28 \ 28 \ 20]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36 \ 36]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}$$

...

The stored pattern $\mathbf{X}$ can be recalled from all of its eight distorted patterns.

(c) We can see that each distorted pattern, instead of recalling itself, can recall the original pattern $\mathbf{X}$. That means the prototype of $\mathbf{X}_1$ to $\mathbf{X}_8$, which is $\mathbf{X}$, has been stored instead of their distorted versions. This demonstrates the property of "central tendency" in Hebb-rule trained AAs.

Example 3.6 (Hebb-rule trained AA for pattern rectification and completion) Matlab

Given two patterns as

$$\mathbf{X}_1 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_2 = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1]$$

which are to be stored in a Hebb-rule trained AA. Assuming $\varepsilon = 0.125$, and the bipolar step function is used,

(a) determine the weight matrix $\mathbf{W}$;

(b) recall $\mathbf{X}_1$ and $\mathbf{X}_2$;

(c) illustrate the pattern rectification capability of the AA by recalling the corrupted version of each of $\mathbf{X}_1$ and $\mathbf{X}_2$ as given below:

$$\mathbf{X}_{t1} = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1]$$

$$\mathbf{X}_{t2} = [-1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1]$$

(d) illustrate the pattern completion capability of the AA by recalling the incomplete version of $\mathbf{X}_1$ as given below:

$$\mathbf{X}_{t3} = [1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0]$$

and by recalling the incomplete versions of $\mathbf{X}_2$ as given below:

$$\mathbf{X}_{t4} = [0 \ 0 \ 0 \ 0 \ 1 \ 1 \ -1 \ -1]$$

$$\mathbf{X}_{t5} = [1 \ 1 \ 0 \ 0 \ 0 \ 0 \ -1 \ -1]$$

Solution

(a)

$$\begin{aligned} \mathbf{W} &= \varepsilon \sum_{k=1}^2 \mathbf{X}_k^T \mathbf{X}_k = \varepsilon \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \\ &+ \varepsilon \begin{bmatrix} 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \end{bmatrix} \end{aligned} \quad (\text{E3.6.1})$$

Hence,

$$\mathbf{W} = 0.125 \begin{bmatrix} 2 & 2 & 0 & 0 & 2 & 2 & 0 & 0 \\ 2 & 2 & 0 & 0 & 2 & 2 & 0 & 0 \\ 0 & 0 & 2 & 2 & 0 & 0 & 2 & 2 \\ 0 & 0 & 2 & 2 & 0 & 0 & 2 & 2 \\ 2 & 2 & 0 & 0 & 2 & 2 & 0 & 0 \\ 0 & 0 & 2 & 2 & 0 & 0 & 2 & 2 \\ 0 & 0 & 2 & 2 & 0 & 0 & 2 & 2 \end{bmatrix} \quad (\text{E3.6.2})$$

(b)

Recall $\mathbf{X}_1$:

$$\mathbf{X}_t = \mathbf{X}_1 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

...

Recall $\mathbf{X}_2$:

$$\mathbf{X}_t = \mathbf{X}_2 = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[8 \ 8 \ -8 \ -8 \ 8 \ 8 \ -8 \ -8]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1] = \mathbf{X}_2$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[8 \ 8 \ -8 \ -8 \ 8 \ 8 \ -8 \ -8]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1] = \mathbf{X}_2$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[8 \ 8 - 8 - 8 \ 8 \ 8 - 8 - 8]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 - 1 - 1 \ 1 \ 1 - 1 - 1] = \mathbf{X}_2$$

...

The outputs of $\mathbf{X}_1$ and $\mathbf{X}_2$ can be recalled correctly.

(c)

Recall $\mathbf{X}_{t1}$:

$$\mathbf{X}_t = \mathbf{X}_{t1} = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 - 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[8 \ 8 \ 4 \ 4 \ 8 \ 8 \ 4 \ 4]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

...

Recall $\mathbf{X}_{t2}$:

$$\mathbf{X}_t = \mathbf{X}_{t2} = [1 \ 1 - 1 - 1 \ 1 \ 1 - 1 - 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[4 \ 4 - 8 - 8 \ 4 \ 4 - 8 - 8]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 - 1 - 1 \ 1 \ 1 - 1 - 1] = \mathbf{X}_2$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[8 \ 8 - 8 - 8 \ 8 \ 8 - 8 - 8]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 - 1 - 1 \ 1 \ 1 - 1 - 1] = \mathbf{X}_2$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[8 \ 8 - 8 - 8 \ 8 \ 8 - 8 - 8]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 -1 -1 \ 1 \ 1 -1 -1] = \mathbf{X}_2$$

...

Corrupted versions of $\mathbf{X}_1$ and $\mathbf{X}_2$ can be recalled correctly.

(d)

Recall $\mathbf{X}_{t3}$:

$$\mathbf{X}_t = \mathbf{X}_{t3} = [1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[4 \ 4 \ 4 \ 4 \ 4 \ 4 \ 4 \ 4]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

...

Recall $\mathbf{X}_{t4}$:

$$\mathbf{X}_t = \mathbf{X}_{t4} = [0 \ 0 \ 0 \ 0 \ 1 \ 1 -1 -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[4 \ 4 -4 -4 \ 4 \ 4 -4 -4]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 -1 -1 \ 1 \ 1 -1 -1] = \mathbf{X}_2$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[8 \ 8 -8 -8 \ 8 \ 8 -8 -8]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 -1 -1 \ 1 \ 1 -1 -1] = \mathbf{X}_2$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[8 \ 8 -8 -8 \ 8 \ 8 -8 -8]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 -1 -1 \ 1 \ 1 -1 -1] = \mathbf{X}_2$$

...

Recall $\mathbf{X}_{t5}$:

$$\mathbf{X}_t = \mathbf{X}_{t5} = [1 \ 1 \ 0 \ 0 \ 0 \ 0 \ -1 \ -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \varepsilon[4 \ 4 \ -4 \ -4 \ 4 \ 4 \ -4 \ -4]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1] = \mathbf{X}_2$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \varepsilon[8 \ 8 \ -8 \ -8 \ 8 \ 8 \ -8 \ -8]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1] = \mathbf{X}_2$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \varepsilon[8 \ 8 \ -8 \ -8 \ 8 \ 8 \ -8 \ -8]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1] = \mathbf{X}_2$$

...

Incomplete versions of $\mathbf{X}_1$ and $\mathbf{X}_2$ can be recalled correctly.

3.5 Delta Rule

3.5.1 Learning

Assuming all initial weights are zero. Starting from $k = 1$, we obtain:

$$z_{jk} = \sum_{i=1}^N x_{ik} w_{ij} \quad \text{for } j = 1 \text{ to } N \quad (3.5.1a)$$

or

$$\mathbf{Z}_k = \mathbf{X}_k \mathbf{W} \quad (3.5.1b)$$

and

$$y_{jk} = G[z_{jk}] \quad \text{for } j = 1 \text{ to } N \quad (3.5.2a)$$

or

$$\mathbf{Y}_k = G[\mathbf{Z}_k] \quad (3.5.2b)$$

For bipolar patterns, the bipolar ramp function is a preferred choice as compared with the bipolar step function to reduce quantization error in training.

However, the use of the bipolar step function will likely increase the speed of convergence in training.

The output error is

$$e_{jk} = x_{jk} - y_{jk} \quad \text{for } j = 1 \text{ to } N \quad (3.5.3a)$$

or

$$\mathbf{E}_k = \mathbf{X}_k - \mathbf{Y}_k \quad (3.5.3b)$$

The change of weight is

$$\delta w_{ijk} = \varepsilon x_{ik} e_{jk} \quad \text{for } i, j = 1 \text{ to } N \quad (3.5.3a)$$

or

$$\delta \mathbf{W}_k = \varepsilon \mathbf{X}_k^T \mathbf{E}_k \quad (3.5.3b)$$

where ε is the learning rate parameter.

For an AA with a small number of neurons, the diagonal elements of $\delta \mathbf{W}_k$ could be set to zero for stability purpose. If the diagonal elements are set to 0, we have

$$\delta w_{ijk} = 0 \quad \text{for } i = 1 \text{ to } N \quad (3.5.4a)$$

or

$$\delta \mathbf{W}_k \text{ diagonal} = 0 \quad (3.5.4b)$$

The above procedure is repeated for the next pattern $\mathbf{X}_{k+1}$ until all the remaining patterns $k=1$ to K have been presented. This finishes one iteration. Weights are updated at the end of each iteration as

$$\mathbf{W} = \mathbf{W} + \sum_{k=1}^K \delta \mathbf{W}_k \quad (3.5.5)$$

There are totally N^2 number of w_{ij} forming a $N \times N$ weight matrix, $\mathbf{W}$ as

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & \dots & w_{1N} \\ w_{21} & w_{22} & w_{23} & \dots & w_{2N} \\ w_{31} & w_{32} & w_{33} & \dots & w_{3N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w_{N1} & w_{N2} & w_{N3} & \dots & w_{NN} \end{bmatrix} \quad (3.5.6)$$

The training is repeated for more iterations until reaching zero in the SSE (sum of squared output errors) of all the patterns in an iteration. In practice, a zero SSE limit is preferred to ensure recall correctness.

In (3.5.3), ε could range from $(1/KN)$ to $1/N$ for bipolar patterns. In the delta rule, ε acts as a learning rate parameter to control the speed of convergence.

3.5.2 Recall

The recall algorithm is the same as that in Section 3.3.2. For bipolar input and output patterns, either the bipolar ramp function or the bipolar step function may be used for recall.

Similar to the Hebb-rule trained AAs, the use of the bipolar step function will usually recall the correct output after one iteration.

Example 3.7 (Calculate weight matrix and recalls of delta-rule trained AA – 2 pattern-pairs) Matlab

Assuming the bipolar ramp function (with $L = 1$) is used in both training and recall, $\varepsilon = 0.5$, and initial weights are zero, the delta rule is used to store the following two patterns in an AA.

$$\mathbf{X}_1 = [1 \ -1 \ 1 \ -1] \text{ and } \mathbf{X}_2 = [1 \ -1 \ -1 \ 1]$$

For illustration, round all values to four decimal places.

- Determine the weight matrix with diagonal elements set to 0.
- Check the recall correctness of the AA.

Solution

(a) For computational convenience, let

$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \end{bmatrix}, \mathbf{Z} = \begin{bmatrix} \mathbf{Z}_1 \\ \mathbf{Z}_2 \end{bmatrix}, \mathbf{Y} = \begin{bmatrix} \mathbf{Y}_1 \\ \mathbf{Y}_2 \end{bmatrix}, \mathbf{E} = \begin{bmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{bmatrix} \quad (\text{E3.7.1})$$

Iteration 1

$$\mathbf{W} = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1.0000 & -1.0000 & 1.0000 & -1.0000 \\ 1.0000 & -1.0000 & -1.0000 & 1.0000 \end{bmatrix}$$

$$\mathbf{Z} = \mathbf{XW} = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}$$

$$\mathbf{Y} = G_r[\mathbf{Z}] = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}$$

$$\mathbf{E} = \mathbf{X} - \mathbf{Y} = \begin{bmatrix} 1.0000 & -1.0000 & 1.0000 & -1.0000 \\ 1.0000 & -1.0000 & -1.0000 & 1.0000 \end{bmatrix}$$

$$\text{SSE} = 8.000000$$

$$\delta \mathbf{W} = \varepsilon \mathbf{X}^T \mathbf{E} = \begin{bmatrix} 1.0000 & -1.0000 & 0.0000 & 0.0000 \\ -1.0000 & 1.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 1.0000 & -1.0000 \\ 0.0000 & 0.0000 & -1.0000 & 1.0000 \end{bmatrix}$$

$$\mathbf{W} = \mathbf{W} + \delta \mathbf{W} = \begin{bmatrix} 0.0000 & -1.0000 & 0.0000 & 0.0000 \\ -1.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & -1.0000 \\ 0.0000 & 0.0000 & -1.0000 & 0.0000 \end{bmatrix}$$

Iteration 2

$$\mathbf{W} = \begin{bmatrix} 0.0000 & -1.0000 & 0.0000 & 0.0000 \\ -1.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & -1.0000 \\ 0.0000 & 0.0000 & -1.0000 & 0.0000 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1.0000 & -1.0000 & 1.0000 & -1.0000 \\ 1.0000 & -1.0000 & -1.0000 & 1.0000 \end{bmatrix}$$

$$\mathbf{Z} = \mathbf{X}\mathbf{W} = \begin{bmatrix} 1.0000 & -1.0000 & 1.0000 & -1.0000 \\ 1.0000 & -1.0000 & -1.0000 & 1.0000 \end{bmatrix}$$

$$\mathbf{Y} = G_r[\mathbf{Z}] = \begin{bmatrix} 1.0000 & -1.0000 & 1.0000 & -1.0000 \\ 1.0000 & -1.0000 & -1.0000 & 1.0000 \end{bmatrix}$$

$$\mathbf{E} = \mathbf{X} - \mathbf{Y} = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}$$

$$\text{SSE} = 0.000000\text{E}+00$$

(b)

$$\mathbf{W} = \begin{bmatrix} 0.0000 & -1.0000 & 0.0000 & 0.0000 \\ -1.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & -1.0000 \\ 0.0000 & 0.0000 & -1.0000 & 0.0000 \end{bmatrix} \quad (\text{E3.7.2})$$

For $k=1$:

$$\mathbf{X}_t(1) = \mathbf{X}_{1t}$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t(1) = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{Y}_t(2) = G_r[\mathbf{Z}_t(2)] = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{Y}_t(3) = G_r[\mathbf{Z}_t(3)] = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{Y}_t(4) = G_r[\mathbf{Z}_t(4)] = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}$$

...

For $k=2$:

$$\mathbf{X}_t(1) = \mathbf{X}_{2t}$$

$$\mathbf{Y}_t(1) = \mathbf{X}_t(1) = \begin{bmatrix} 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)\mathbf{W} = \begin{bmatrix} 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{Y}_t(2) = G_r[\mathbf{Z}_t(2)] = \begin{bmatrix} 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2)\mathbf{W} = \begin{bmatrix} 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{Y}_t(3) = G_r[\mathbf{Z}_t(3)] = \begin{bmatrix} 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3)\mathbf{W} = \begin{bmatrix} 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{Y}_t(4) = G_r[\mathbf{Z}_t(4)] = \begin{bmatrix} 1 & -1 & -1 & 1 \end{bmatrix}$$

...

Both inputs can be recalled correctly after one iteration.

Example 3.8 (Calculate weight matrix and recalls of delta-rule trained AA - 5 pattern-pairs) Matlab

Assuming the bipolar ramp function is used for both training and recall, and $\varepsilon = 0.2$, the delta rule is used to store the following five patterns in an AA.

$$\mathbf{X}_1 = \begin{bmatrix} -1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{X}_2 = \begin{bmatrix} 1 & -1 & 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{X}_3 = \begin{bmatrix} 1 & 1 & -1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{X}_4 = \begin{bmatrix} 1 & 1 & 1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{X}_5 = \begin{bmatrix} 1 & 1 & 1 & 1 & -1 \end{bmatrix}$$

For illustration, round all values to four decimal places.

- Determine the weight matrix with nonzero diagonal elements.
- Check the recall correctness.

Solution

- For computational convenience, let

$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \mathbf{X}_3 \\ \mathbf{X}_4 \\ \mathbf{X}_5 \end{bmatrix}, \mathbf{Z} = \begin{bmatrix} \mathbf{Z}_1 \\ \mathbf{Z}_2 \\ \mathbf{Z}_3 \\ \mathbf{Z}_4 \\ \mathbf{Z}_5 \end{bmatrix}, \mathbf{Y} = \begin{bmatrix} \mathbf{Y}_1 \\ \mathbf{Y}_2 \\ \mathbf{Y}_3 \\ \mathbf{Y}_4 \\ \mathbf{Y}_5 \end{bmatrix}, \mathbf{E} = \begin{bmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \\ \mathbf{E}_3 \\ \mathbf{E}_4 \\ \mathbf{E}_5 \end{bmatrix} \quad (\text{E3.8.1})$$

Iteration 1

$$\mathbf{W} = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & -1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z} = \mathbf{XW} = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}$$

$$\mathbf{Y} = G_r[\mathbf{Z}] = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}$$

$$\mathbf{E} = \mathbf{X} - \mathbf{Y}$$

$$= \begin{bmatrix} -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & -1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 \end{bmatrix}$$

$$\text{SSE} = 25.000000$$

$$\delta \mathbf{W} = \varepsilon \mathbf{X}^T \mathbf{E} = \begin{bmatrix} 1.0000 & 0.2000 & 0.2000 & 0.2000 & 0.2000 \\ 0.2000 & 1.0000 & 0.2000 & 0.2000 & 0.2000 \\ 0.2000 & 0.2000 & 1.0000 & 0.2000 & 0.2000 \\ 0.2000 & 0.2000 & 0.2000 & 1.0000 & 0.2000 \\ 0.2000 & 0.2000 & 0.2000 & 0.2000 & 1.0000 \end{bmatrix}$$

$$\mathbf{W} = \mathbf{W} + \delta \mathbf{W} = \begin{bmatrix} 1.0000 & 0.2000 & 0.2000 & 0.2000 & 0.2000 \\ 0.2000 & 1.0000 & 0.2000 & 0.2000 & 0.2000 \\ 0.2000 & 0.2000 & 1.0000 & 0.2000 & 0.2000 \\ 0.2000 & 0.2000 & 0.2000 & 1.0000 & 0.2000 \\ 0.2000 & 0.2000 & 0.2000 & 0.2000 & 1.0000 \end{bmatrix}$$

Iteration 2

$$\mathbf{W} = \begin{bmatrix} 1.0000 & 0.2000 & 0.2000 & 0.2000 & 0.2000 \\ 0.2000 & 1.0000 & 0.2000 & 0.2000 & 0.2000 \\ 0.2000 & 0.2000 & 1.0000 & 0.2000 & 0.2000 \\ 0.2000 & 0.2000 & 0.2000 & 1.0000 & 0.2000 \\ 0.2000 & 0.2000 & 0.2000 & 0.2000 & 1.0000 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & -1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z} = \mathbf{XW} = \begin{bmatrix} -0.2000 & 1.4000 & 1.4000 & 1.4000 & 1.4000 \\ 1.4000 & -0.2000 & 1.4000 & 1.4000 & 1.4000 \\ 1.4000 & 1.4000 & -0.2000 & 1.4000 & 1.4000 \\ 1.4000 & 1.4000 & 1.4000 & -0.2000 & 1.4000 \\ 1.4000 & 1.4000 & 1.4000 & 1.4000 & -0.2000 \end{bmatrix}$$

$$\mathbf{Y} = G_r[\mathbf{Z}]$$

$$= \begin{bmatrix} -0.2000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -0.2000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & -0.2000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & -0.2000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -0.2000 \end{bmatrix}$$

$$\mathbf{E} = \mathbf{X} - \mathbf{Y}$$

$$= \begin{bmatrix} -0.8000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & -0.8000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & -0.8000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & -0.8000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & -0.8000 \end{bmatrix}$$

$$\text{SSE} = 3.200000$$

$$\delta \mathbf{W} = \varepsilon \mathbf{X}^T \mathbf{E}$$

$$= \begin{bmatrix} 0.1600 & -0.1600 & -0.1600 & -0.1600 & -0.1600 \\ -0.1600 & 0.1600 & -0.1600 & -0.1600 & -0.1600 \\ -0.1600 & -0.1600 & 0.1600 & -0.1600 & -0.1600 \\ -0.1600 & -0.1600 & -0.1600 & 0.1600 & -0.1600 \\ -0.1600 & -0.1600 & -0.1600 & -0.1600 & 0.1600 \end{bmatrix}$$

$$\mathbf{W} = \mathbf{W} + \delta \mathbf{W} = \begin{bmatrix} 1.1600 & 0.0400 & 0.0400 & 0.0400 & 0.0400 \\ 0.0400 & 1.1600 & 0.0400 & 0.0400 & 0.0400 \\ 0.0400 & 0.0400 & 1.1600 & 0.0400 & 0.0400 \\ 0.0400 & 0.0400 & 0.0400 & 1.1600 & 0.0400 \\ 0.0400 & 0.0400 & 0.0400 & 0.0400 & 1.1600 \end{bmatrix}$$

Iteration 3

$$\mathbf{W} = \begin{bmatrix} 1.1600 & 0.0400 & 0.0400 & 0.0400 & 0.0400 \\ 0.0400 & 1.1600 & 0.0400 & 0.0400 & 0.0400 \\ 0.0400 & 0.0400 & 1.1600 & 0.0400 & 0.0400 \\ 0.0400 & 0.0400 & 0.0400 & 1.1600 & 0.0400 \\ 0.0400 & 0.0400 & 0.0400 & 0.0400 & 1.1600 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & -1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z} = \mathbf{XW} = \begin{bmatrix} -1.0000 & 1.2400 & 1.2400 & 1.2400 & 1.2400 \\ 1.2400 & -1.0000 & 1.2400 & 1.2400 & 1.2400 \\ 1.2400 & 1.2400 & -1.0000 & 1.2400 & 1.2400 \\ 1.2400 & 1.2400 & 1.2400 & -1.0000 & 1.2400 \\ 1.2400 & 1.2400 & 1.2400 & 1.2400 & -1.0000 \end{bmatrix}$$

$$\begin{aligned}
\mathbf{Y} &= G_r[\mathbf{Z}] \\
&= \begin{bmatrix} -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & -1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 \end{bmatrix} \\
\mathbf{E} = \mathbf{X} - \mathbf{Y} &= \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}
\end{aligned}$$

$$\text{SSE} = 0.000000\text{E}+00$$

(b) Iteration 1

$$\begin{aligned}
\mathbf{W} &= \begin{bmatrix} 1.1600 & 0.0400 & 0.0400 & 0.0400 & 0.0400 \\ 0.0400 & 1.1600 & 0.0400 & 0.0400 & 0.0400 \\ 0.0400 & 0.0400 & 1.1600 & 0.0400 & 0.0400 \\ 0.0400 & 0.0400 & 0.0400 & 1.1600 & 0.0400 \\ 0.0400 & 0.0400 & 0.0400 & 0.0400 & 1.1600 \end{bmatrix} \\
\mathbf{X}_t &= \begin{bmatrix} -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & -1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 \end{bmatrix}
\end{aligned}$$

$$\begin{aligned}
\mathbf{Z}_t &= \mathbf{X}_t \mathbf{W} \\
&= \begin{bmatrix} -1.0000 & 1.2400 & 1.2400 & 1.2400 & 1.2400 \\ 1.2400 & -1.0000 & 1.2400 & 1.2400 & 1.2400 \\ 1.2400 & 1.2400 & -1.0000 & 1.2400 & 1.2400 \\ 1.2400 & 1.2400 & 1.2400 & -1.0000 & 1.2400 \\ 1.2400 & 1.2400 & 1.2400 & 1.2400 & -1.0000 \end{bmatrix}
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_t &= G_r[\mathbf{Z}_t] \\
&= \begin{bmatrix} -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & -1.0000 & 1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 \end{bmatrix}
\end{aligned}$$

$$\mathbf{E}_t = \mathbf{X}_t - \mathbf{Y}_t = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 0.0000 & 0.0000 \end{bmatrix}$$

$$\text{SSE} = 0.000000\text{E}+00$$

3.6 Properties of Delta-Rule Trained AAs

The pattern rectification, pattern completion, and central tendency properties of the Hebb-rule trained AAs as described in Section 3.4 exist in the delta-rule trained AAs as illustrated by the following examples.

Example 3.9 (Pattern rectification and completion properties of delta-rule trained AA) Matlab

Given two patterns as

$$\mathbf{X}_1 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_2 = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1]$$

which are to be stored in a delta-rule trained AA.

Assuming $\varepsilon = 0.125$, and the bipolar ramp function is used in training and the bipolar step function is used in recall. For illustration, round all values to four decimal places.

- Determine all w_{ij} with nonzero diagonal elements.
- Recall $\mathbf{X}_1$ and $\mathbf{X}_2$.
- To illustrate the pattern rectification capability of the AA, recall the corrupted version of each of $\mathbf{X}_1$ and $\mathbf{X}_2$ as given by

$$\mathbf{X}_{tn1} = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1]$$

$$\mathbf{X}_{tn2} = [-1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1]$$

- To illustrate the pattern completion capability of the AA, recall the incomplete version of each of $\mathbf{X}_1$ and $\mathbf{X}_2$ as given by

$$\mathbf{X}_{tn1} = [1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0]$$

$$\mathbf{X}_{tn2} = [1 \ 1 \ 0 \ 0 \ 0 \ 0 \ -1 \ -1]$$

Solution

The solution of this example can be obtained by writing a Matlab program to implement (a) to (d) and to verify the pattern rectification and the pattern completion properties of delta-rule trained AA.

Example 3.10 (Central tendency property of delta-rule trained AA)
Matlab

Suppose we want to store the following four patterns in an AA using the delta rule.

$$\mathbf{X}_1 = [-1 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_2 = [1 \ -1 \ 1 \ 1]$$

$$\mathbf{X}_3 = [1 \ 1 \ -1 \ 1]$$

$$\mathbf{X}_4 = [1 \ 1 \ 1 \ -1]$$

However, the four patterns themselves are not available but we have some noisy versions of each of the four patterns.

Assuming the bipolar ramp function is used for training and the bipolar step function is used for recall. For illustration, all computations are rounded to four decimal places.

(a) Determine the weight matrix of the AA.

(b) To illustrate the central tendency property of the AA, check the recall correctness of the AA using the four original patterns and their noisy versions.

Solution

(a) To simulate the noisy versions of the given patterns, up to ± 0.25 random noise is added to each element of each of the four prototype patterns.

We train the AA using a different set of the noisy versions of the four prototype patterns in each iteration by the delta rule. After ten iterations, a 4x4 weight matrix as

$$\mathbf{W} = \begin{bmatrix} 1.3971 & -0.2535 & 0.1358 & 0.0971 \\ 0.0988 & 1.1922 & -0.0222 & 0.0544 \\ -0.1857 & -0.0618 & 1.4659 & 0.0808 \\ 0.2189 & -0.1071 & 0.1404 & 1.3303 \end{bmatrix} \quad (\text{E3.10.1})$$

(b) When using the four prototype patterns, correct recalls can be obtained. Correct recalls can also be obtained by corrupting each element of the four prototype patterns with up to ± 0.25 random noise. These noisy patterns are

$$\mathbf{X}_{1t} = [-1.2120 \quad 0.9700 \quad 1.0891 \quad 1.2366]$$

$$\mathbf{X}_{2t} = [0.7796 \quad -0.8055 \quad 0.8178 \quad 0.8436]$$

$$\mathbf{X}_{3t} = [1.0797 \quad 0.9309 \quad -0.8015 \quad 0.9318]$$

$$\mathbf{X}_{4t} = [1.1234 \quad 0.8739 \quad 1.1468 \quad -1.2452]$$

3.7 Comparisons

Under zero initial weights, both the Hebb rule and the delta rule (without setting diagonal elements to zero, $G_r[.]$ is used, and weight-updates at the iteration end) produce identical weights immediately after the first iteration.

For the case of the Hebb rule, the weights will keep on increasing linearly without bound for more and more training iterations for both orthogonal and non-orthogonal patterns trainings.

For orthogonal training patterns, the delta rule (without setting diagonal elements to zero, $G_r[.]$ is used) will stop learning after the first time presentation of all patterns as all obtained outputs will be exactly the same as the ideal outputs.

After the first iteration, the delta rule (with diagonal elements set to zero) will keep on adapting weights and hence the output so as to

produce exact target patterns for both orthogonal and non-orthogonal patterns trainings.

For non-orthogonal patterns training, the delta rule (with or without setting diagonal elements to zero) always adapts the weights and hence the output so as to produce exact target patterns.

The pattern rectification, pattern completion, and central tendency properties of the Hebb-rule trained AAs as described in Section 3.4 exist in the delta-rule trained AAs.

3.8 Autoassociators with Increased Storage Capacity

In [Lee9006], two new learning algorithms for storing data in an autoassociator with full storage capacity are presented. The algorithm 1 can teach a neural network to store short-bit length data with full storage capacity (as compared to the Hebb rule) and the algorithm 2 can do the same job for long-bit-length data at a higher speed (as compared to the delta rule).

These two algorithms eliminate the problem of limited storage capacity associated with the Hebb rule and their learning speeds are much faster than that of the corresponding delta rule.

Simulations had been done to compare the two learning algorithms to those of the Hebb rule and the delta rule. Results show that the two algorithms are much better than the Hebb rule in terms of storage capacity, and are much better than the delta rule in terms of learning speed. Details can be found in [Lee9006].

Chapter 4

Multilayer Neural Networks

4.1 Introduction

Backpropagation learning algorithms are based on gradient descent. Based on [Tan9205], [Tan9301], this chapter presents the standard backpropagation learning algorithm and some of its enhanced algorithms which include the adaptive bias of a neuron, the adaptive slope of an activation function, and the backpropagation with momentum.

In particular, a systematic derivation of the backpropagation algorithms for training multilayer feedforward neural networks is presented.

Some observations on the convergence speed of learning and the generalization capability properties of the standard backpropagation algorithm and its enhanced algorithms are also given.

In this chapter, design examples are described which include real-valued two-layer feedforward neural networks [Kwa8906], [Kwa9301] and real-valued two-layer recurrent neural networks [Kwa0008a], [Kwa0105]. The design of multiplierless neural networks is described in Chapter 18.

The standard backpropagation algorithm and its enhanced algorithms had also been used to design multilayer feedbackward neural networks [Kwa9105], high-order feedbackward neural networks [Kwa9106], cascaded recursive artificial neural networks [Kwa9109b], and multilayer feedforward and feedbackward neural networks [Kwa9209b]. A method for generating partially correlated vector sets for neural network simulations is also described in [Kwa9304].

4.2 Backpropagation Learning Algorithm

A L -layer feedforward neural network and the internal structure of a neuron are shown in Figures 4.2.1-4.2.2. The major notations adopted in this Chapter are explained in Table 4.2.1.

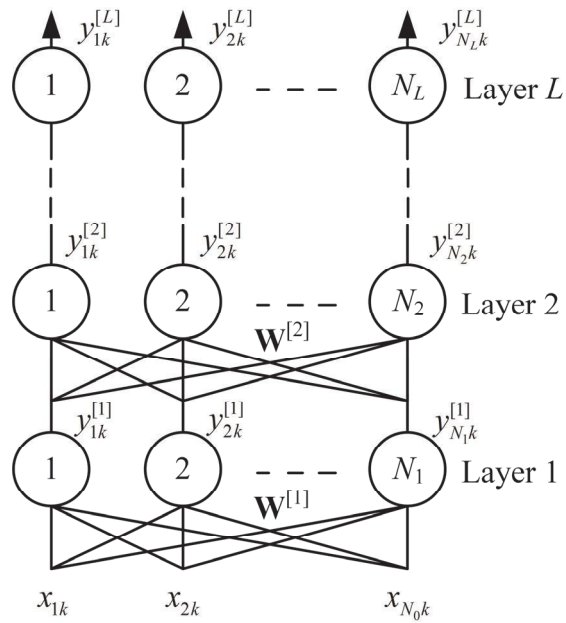


Figure 4.2.1 L -layer feedforward neural network.

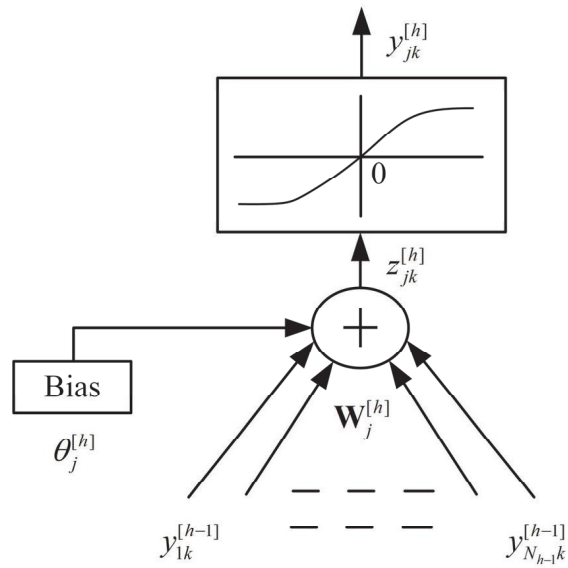


Figure 4.2.2 Structure of a neuron.

Table 4.2.1 Notations for backpropagation learning algorithms.

Notation	Description
N_h	Number of neurons at layer h for $0 \leq h \leq L$
x_{ik}	Element i of input pattern k for $1 \leq i \leq N_0$ and $1 \leq k \leq K$
$z_{jk}^{[h]}$	Internal input $z_{jk}^{[h]}$ to neuron j at layer h due to pattern k for $1 \leq j \leq N_h$
$w_{ij}^{[h]}$	Connection weight between neuron i of layer $[h - 1]$ and neuron j of layer h
$y_{ik}^{[h-1]}$	Activation of neuron i of layer $[h - 1]$ due to pattern k
$\theta_j^{[h]}$	Threshold value of neuron j of layer h
t_{jk}	Element j of target output pattern k
e_{jk}	Output error at neuron j due to pattern k
e_k	Total output error due to a pattern k

$\delta_k w_{ij}^{[h]}$	Weight update between neuron i of layer $[h - 1]$ and neuron j of layer h due to pattern k
$\delta_k \theta_j^{[h]}$	Bias update at neuron j of layer h due to pattern k
$\delta_k \alpha_j^{[h]}$	Update in slope α at neuron j of layer h due to pattern k

As shown in Figure 4.2.2, the internal input $z_{jk}^{[h]}$ to a neuron j at a layer h due to a pattern k is

$$z_{jk}^{[h]} = \theta_j^{[h]} + \sum_{i=1}^{N_{h-1}} w_{ij}^{[h]} y_{ik}^{[h-1]} \text{ for } h = 1, 2, \dots, L \quad (4.2.1)$$

$$y_{ik}^{[0]} = x_{ik} \quad (4.2.2)$$

Applying a nonlinear activation function $F(z)$ to the internal input yields the output of a neuron j at a layer h of a pattern k as

$$y_{jk}^{[h]} = F(z_{jk}^{[h]}) \quad (4.2.3)$$

In the output layer in which h is equal to L , the activation becomes

$$y_{jk}^{[L]} = F(z_{jk}^{[L]}) \quad (4.2.4)$$

Unless the network has been perfectly trained, this activation does not match the target output, and the output error at a neuron j due to a pattern k in the output layer L is defined as

$$e_{jk} = (1/2) (t_{jk} - y_{jk}^{[L]})^2 \quad (4.2.5)$$

where t_{jk} is the element j of a target output pattern k and the factor $1/2$ serves to normalize the derivative of (4.2.5) to 1. The total output error due to a pattern k is the sum of all the output errors at N_L neurons as

$$e_k = (1/2) \sum_{j=1}^{N_L} (t_{jk} - y_{jk}^{[L]})^2 \quad (4.2.6)$$

The backpropagation adopts a gradient descent algorithm. The change in weight between a neuron i of a layer $(h - 1)$ and a neuron j of a layer h is

$$\delta_k w_{ij}^{[h]} = -\varepsilon \frac{\partial e_k}{\partial w_{ij}^{[h]}} \quad \text{for } 1 \leq h \leq L \quad (4.2.7)$$

where ε is a learning rate parameter.

From (4.2.1), the derivative part of (4.2.7) can be expressed as

$$\begin{aligned} \frac{\partial e_k}{\partial w_{ij}^{[h]}} &= \frac{\partial e_k}{\partial z_{jk}^{[h]}} \frac{\partial z_{jk}^{[h]}}{\partial w_{ij}^{[h]}} \\ &= \frac{\partial e_k}{\partial z_{jk}^{[h]}} \frac{\partial}{\partial w_{ij}^{[h]}} \left(\sum_{i=1}^{N_{h-1}} w_{ij}^{[h]} y_{ik}^{[h-1]} + \theta_j^{[h]} \right) = -\delta_{jk}^{[h]} y_{ik}^{[h-1]} \end{aligned} \quad (4.2.8)$$

where

$$\delta_{jk}^{[h]} = -\frac{\partial e_k}{\partial z_{jk}^{[h]}} \quad (4.2.9)$$

Substituting (4.2.8) into (4.2.7), we have

$$\delta_k w_{ij}^{[h]} = \varepsilon \delta_{jk}^{[h]} y_{ik}^{[h-1]} \quad (4.2.10)$$

Considering the weight-update at the output layer L , from (4.2.9) and (4.2.6), we obtain

$$\delta_{jk}^{[L]} = -\frac{\partial e_k}{\partial y_{jk}^{[L]}} \frac{\partial y_{jk}^{[L]}}{\partial z_{jk}^{[L]}} = (t_{jk} - y_{jk}^{[L]}) F'(z_{jk}^{[L]}) \quad (4.2.11)$$

where $F'(z_{jk}^{[L]})$ is the partial derivative of $F(z_{jk}^{[L]})$ with respect to $z_{jk}^{[L]}$.

Substituting (4.2.11) into (4.2.10), the weight-update at the output layer L is

$$\delta_k w_{ij}^{[L]} = \varepsilon \delta_{jk}^{[L]} y_{ik}^{[L-1]} = \varepsilon (t_{jk} - y_{jk}^{[L]}) F'(z_{jk}^{[L]}) y_{ik}^{[L-1]} \quad (4.2.12)$$

From (4.2.9) and for $h = L - 1$, we have

$$\begin{aligned} \delta_{jk}^{[L-1]} &= -\frac{\partial e_k}{\partial z_{jk}^{[L-1]}} \\ &= -\sum_{i=1}^{N_L} \frac{\partial e_k}{\partial z_{ik}^{[L]}} \frac{\partial z_{ik}^{[L]}}{\partial z_{jk}^{[L-1]}} = -\sum_{i=1}^{N_L} \frac{\partial e_k}{\partial z_{ik}^{[L]}} \frac{\partial z_{ik}^{[L]}}{\partial y_{jk}^{[L-1]}} \frac{\partial y_{jk}^{[L-1]}}{\partial z_{jk}^{[L-1]}} \end{aligned} \quad (4.2.13)$$

Substituting (4.2.3) and (4.2.1) into (4.2.13), we have

$$\delta_{jk}^{[L-1]} = -F'(z_{jk}^{[L-1]}) \sum_{i=1}^{N_L} \frac{\partial e_k}{\partial z_{ik}^{[L]}} \frac{\partial}{\partial y_{jk}^{[L-1]}} \left(\sum_{l=1}^{N_{L-1}} w_{li}^{[L]} y_{lk}^{[L-1]} + \theta_i^{[L]} \right) \quad (4.2.14)$$

Based on the notation defined in (4.2.9), (4.2.14) becomes

$$\delta_{jk}^{[L-1]} = F'(z_{jk}^{[L-1]}) \sum_{i=1}^{N_L} \delta_{ik}^{[L]} w_{ji}^{[L]} \quad (4.2.15)$$

Similar to (4.2.15), for $1 \leq h \leq L - 1$, we obtain

$$\delta_{jk}^{[h]} = F'(z_{jk}^{[h]}) \sum_{i=1}^{N_{h+1}} \delta_{ik}^{[h+1]} w_{ji}^{[h+1]} \text{ for } 1 \leq h \leq L - 1 \quad (4.2.16)$$

In summary, from (4.2.16) and (4.2.10), we have

$$\delta_{jk}^{[h]} = F'(z_{jk}^{[h]}) \sum_{i=1}^{N_{h+1}} \delta_{ik}^{[h+1]} w_{ji}^{[h+1]} \text{ for } 1 \leq h \leq L - 1 \quad (4.2.17)$$

$$\delta_k w_{ij}^{[h]} = \varepsilon \delta_{jk}^{[h]} y_{ik}^{[h-1]} \text{ for } 1 \leq h \leq L - 1 \quad (4.2.18)$$

To improve learning by averaging the instantaneous changes, we update the weights only when all the training patterns have been presented at the end of each epoch. Thus we have

$$w_{ij}^{[h]}(t+1) = w_{ij}^{[h]}(t) + \sum_{k=1}^K \delta_k w_{ij}^{[h]} \quad (4.2.19)$$

where K represents the total number of patterns to be trained.

The convergence speed of the standard backpropagation algorithm could be slow, especially when the input to output training set contains complex pattern-pairs.

A number of enhanced schemes are presented below to speed up the standard backpropagation algorithm.

4.3 Backpropagation with Adaptive Bias

The idea of gradient descent can be extended to the bias of each neuron in Figure 4.2.2 so that the biases can be adjusted. We define the bias update at a neuron j of a layer h during the training of a pattern k as

$$\delta_k \theta_j^{[h]} = -\varepsilon_b \frac{\partial e_k}{\partial \theta_j^{[h]}} \text{ for } 1 \leq h \leq L \quad (4.3.1)$$

where ε_b is the step size for bias adjustment.

Expressing $z_{jk}^{[h]}$ in term of (4.2.1) and based on the notation defined in (4.2.9), the partial derivative on the right hand side of (4.3.1) can be expressed as

$$\begin{aligned} \frac{\partial e_k}{\partial \theta_j^{[h]}} &= \frac{\partial e_k}{\partial z_{jk}^{[h]}} \frac{\partial z_{jk}^{[h]}}{\partial \theta_j^{[h]}} \\ &= \frac{\partial e_k}{\partial z_{jk}^{[h]}} \frac{\partial}{\partial \theta_j^{[h]}} \left(\sum_{i=1}^{N_{h-1}} w_{ij}^{[h]} y_{ik}^{[h-1]} + \theta_j^{[h]} \right) = \frac{\partial e_k}{\partial z_{jk}^{[h]}} = -\delta_{jk}^{[h]} \end{aligned} \quad (4.3.2)$$

Substituting (4.3.2) into (4.3.1), we have

$$\delta_k \theta_j^{[h]} = \varepsilon_b \delta_{jk}^{[h]} \quad (4.3.3)$$

Thus, the update of the bias $\theta_j^{[h]}(t + 1)$ at a neuron j can be carried out as

$$\theta_j^{[h]}(t + 1) = \theta_j^{[h]}(t) + \varepsilon_b \sum_{k=1}^K \delta_{jk}^{[h]} \quad (4.3.4)$$

4.4 Backpropagation with Adaptive Slope

The sigmoid function is the most widely used activation function which can be expressed in the binary form as

$$F(z) = \frac{1}{1 + e^{-\alpha z}} \quad (4.4.1)$$

or expressed in the bipolar form as

$$F(z) = \frac{1 - e^{-\alpha z}}{1 + e^{-\alpha z}} \quad (4.4.2)$$

By adjusting the parameter α in (4.4.1)-(4.4.2), we can adjust the slope of the corresponding sigmoid function.

We define the update in slope α at a neuron j of a layer h during the training of a pattern k as

$$\delta_k \alpha_j^{[h]} = -\frac{\partial e_k}{\partial \alpha_j^{[h]}} \quad \text{for } 1 \leq h \leq L \quad (4.4.3)$$

From (4.4.3) and (4.2.6), for $h = L$,

$$\delta_k \alpha_j^{[L]} = -\frac{\partial e_k}{\partial y_{jk}^{[L]}} \frac{\partial y_{jk}^{[L]}}{\partial \alpha_j^{[L]}} = (t_{jk} - y_{jk}^{[L]}) \frac{\partial y_{jk}^{[L]}}{\partial \alpha_j^{[L]}} \quad (4.4.4)$$

Let $F_{\alpha}'(z_{jk}^{[L]}, \alpha_j^{[L]})$ be the partial derivative of the nonlinear function $F_{\alpha}(z_{jk}^{[L]}, \alpha_j^{[L]})$ with respect to $\alpha_j^{[L]}$ such that

$$F_{\alpha}'(z_{jk}^{[L]}, \alpha_j^{[L]}) = \frac{\partial y_{jk}^{[L]}}{\partial \alpha_j^{[L]}} = \frac{F_{\alpha}(z_{jk}^{[L]}, \alpha_j^{[L]})}{\partial \alpha_j^{[L]}} \quad (4.4.5)$$

Substituting (4.4.5) into (4.4.4), we obtain

$$\delta_k \alpha_j^{[L]} = (t_{jk} - y_{jk}^{[L]}) F_{\alpha}'(z_{jk}^{[L]}, \alpha_j^{[L]}) \quad (4.4.6)$$

From (4.4.3), for $1 \leq h \leq L - 1$,

$$\begin{aligned} \delta_k \alpha_j^{[h]} &= -\frac{\partial e_k}{\partial \alpha_j^{[h]}} = -\sum_{i=1}^{N_{h+1}} \frac{\partial e_k}{\partial z_{ik}^{[h+1]}} \frac{\partial z_{ik}^{[h+1]}}{\partial \alpha_j^{[h]}} \\ &= -\sum_{i=1}^{N_{h+1}} \frac{\partial e_k}{\partial z_{ik}^{[h+1]}} \frac{\partial z_{ik}^{[h+1]}}{\partial y_{jk}^{[h]}} \frac{\partial y_{jk}^{[h]}}{\partial \alpha_j^{[h]}} \end{aligned} \quad (4.4.7)$$

From (4.4.5) and then (4.2.1), (4.4.7) can be expressed as

$$\begin{aligned} \delta_k \alpha_j^{[h]} &= -F_{\alpha}'(z_{jk}^{[h]}, \alpha_j^{[h]}) \sum_{i=1}^{N_{h+1}} \frac{\partial e_k}{\partial z_{ik}^{[h+1]}} \frac{\partial}{\partial y_{jk}^{[h]}} \left(\sum_{l=1}^{N_h} w_{li}^{[h+1]} y_{lk}^{[h]} + \theta_i^{[h+1]} \right) \end{aligned} \quad (4.4.8)$$

Based on the notation defined in (4.2.9), (4.4.8) can be expressed as

$$\delta_k \alpha_j^{[h]} = F_{\alpha}'(z_{jk}^{[h]}, \alpha_j^{[h]}) \sum_{i=1}^{N_{h+1}} \delta_{ik}^{[h+1]} w_{ji}^{[h+1]} \quad (4.4.9)$$

Let ε_{α} be the learning rate parameter for the slope, the update of the bias $\alpha_j^{[h]}(t + 1)$ at a neuron j can be carried out as

$$\alpha_j^{[h]}(t+1) = \alpha_j^{[h]}(t) + \varepsilon_\alpha \sum_{k=1}^K \delta_k \alpha_j^{[h]} \quad (4.4.10)$$

4.5 Backpropagation with Momentum

By incorporating a momentum term, the parameter change will be in a direction that is a combination of the current gradient and the previous gradient. For weight change with a momentum, (4.2.19) becomes

$$w_{ij}^{[h]}(t+1) = w_{ij}^{[h]}(t) + \sum_{k=1}^K \delta_k w_{ij}^{[h]} + \mu_w \{w_{ij}^{[h]}(t) - w_{ij}^{[h]}(t-1)\} \quad (4.5.1)$$

where μ_w denotes the momentum constant for weight change.

If momentum is also added to slope change, (4.4.10) becomes

$$\alpha_j^{[h]}(t+1) = \alpha_j^{[h]}(t) + \varepsilon_\alpha \sum_{k=1}^K \delta_k \alpha_j^{[h]} + \mu_\alpha \{\alpha_j^{[h]}(t) - \alpha_j^{[h]}(t-1)\} \quad (4.5.2)$$

where μ_α denotes the momentum constant for slope change.

4.6 Performances

In general, the convergence speed and generalization ability of the standard backpropagation and its enhanced learning algorithms could be affected by:

- The initial weights of the network.
- The complexity of the network (i.e., the number of layers and the number of neurons at each layer).
- The characteristic of the training patterns (i.e., the kind of problem being dealt with).

d. The number of pattern-pairs to be trained (i.e., especially when the number exceeds the storage capacity of the network).

The adaptive bias method which has an advantage in the improvement of the generalization capability but not in learning speed.

The adaptive slope method which can speed up the learning process considerably and also result in a good generalization.

Incorporating momentum has a speed advantage which arises mainly when some training pattern-pairs are different from the majority of the pattern-pairs.

Incorporating momentum may reduce the likelihood of being trapped in a local minimum.

4.7 Multilayer Feedforward Neural Network Design

The backpropagation algorithm as described in Sections 4.2 to 4.6 is an effective algorithm for design multilayer feedforward neural networks.

For example, it has been used to design a 2-layer feedforward neural network for pulse radar detection [Kwa8906], [Kwa9301] as described in this paragraph.

In the design, the 13-element Barker code and the maximum-length sequences (m-sequences) with lengths 15, 31, and 63 bits were used as the signal codes, and four networks were implemented, respectively.

In each of these networks, the number of inputs was the same as the signal length while the number of hidden neurons was three and the number of output neuron was one.

The backpropagation algorithm was used for training each of these networks and the number of training epochs was 500.

A more than 40 dB output peak signal-to-sidelobe ratio can be achieved.

The XOR problem defined in Table 4.7.1 is a linearly non-separable problem which has often been used to test the learning capability of a nonlinear classifier.

For illustration, a 2-layer feedforward neural network shown in Figure 4.7.1 is to be used as a nonlinear classifier to learn the XOR problem as explained in Example 4.2.

Table 4.7.1 XOR problem.

k	$\mathbf{X}_k$	y_k
1	$[0 \ 0]$	0
2	$[0 \ 1]$	1
3	$[1 \ 0]$	1
4	$[1 \ 1]$	0

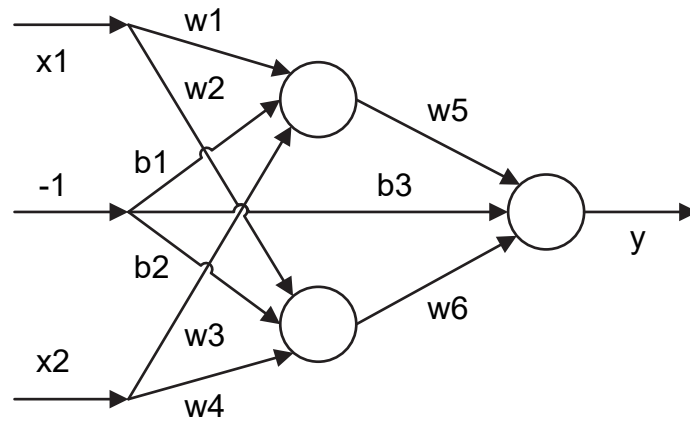


Figure 4.7.1 Two-layer neural network for XOR problem.

Design Example 4.1 (XOR problem learning using two-layer feedforward neural network) Matlab

In this example, a 2-layer feedforward neural network shown in Figure 4.7.1 is to be used to learn the XOR problem defined in Table 4.7.1.

The neural network consists of two inputs (denoted by x_1 and x_2); one bias of value -1 scaled by b_1 and b_2 to the input of each of the two hidden neurons; two hidden neurons (with outputs denoted by $y_1^{[1]}$ and $y_2^{[1]}$); and one output neuron (with output denoted by $y_1^{[2]}$).

The sum of the scaled inputs of each of the three neurons $z_i^{[h]}$ for $i = 1, 2; h = 1, 2$ is passed through a binary sigmoidal activation function $y[z_i^{[h]}]$ with a slope parameter r_n defined by

$$y_i^{[h]} = \frac{1}{1 + e^{-r_n z_i^{[h]}}} \quad \text{for } i = 1, 2; h = 1, 2 \quad (\text{E4.1.1})$$

The slope parameter r_n determines the slope of the transition region within the range $-L \leq z \leq L$, as well as determining the value of $\pm L$ in which the function saturates to 1 or 0.

For having a unity slope for (E4.1.1) at $z_i^{[h]} = 0$, an appropriate choice for the slope parameter is $r_n = 2$.

The inputs x_1, x_2 and the output $y_1^{[2]}$ take either 0.05 or 0.95 corresponding to false (0) or true (1), respectively, to speed up learning.

The parameter vector $\mathbf{c}$ has 9 parameters which can be grouped as

$$\mathbf{c} = [w_1, w_2, w_3, w_4, w_5, w_6, b_1, b_2, b_3] \quad (\text{E4.1.2})$$

The objective function to be minimized is equal to the sum of the absolute output differences over the four XOR patterns.

$$e(\mathbf{c}) = \sum_{i=1}^4 |y_1^{[2]} - y_i| \quad (\text{E4.1.3})$$

Assuming a search space as

$$-s < w_1, w_2, w_3, w_4, w_5, w_6, b_1, b_2, b_3 < s \quad (\text{E4.1.4})$$

Initial coefficients are generated using any of the parameter initialization methods as described in Chapter 17.

This problem can be learned using the backpropagation algorithm explained in this chapter or any evolutionary algorithm explained in Chapters 13-16.

4.8 Two-layer Recurrent Neural Networks

A recurrent neural network (RNN) has feedback connections among neurons. One consequence of these connections is that dynamical behaviors, such as limit cycles and chaos, not possible with strictly feedforward networks can be generated in a recurrent network.

A 1st-order recurrent neural network has only 1st-order feedbacks thus bounding its recall capability and learning speed.

Existing recurrent neural networks normally adopt a fixed structure in most applications. A fixed structure simplifies the design procedure but it considerably limits the performance of the resulting network.

In addition to adopting a 1st-order recurrent network, a 2nd-order recurrent network can be adopted with which its 2nd-order and 1st-order recurrent units together provide more memory context for the network.

4.8.1 Network structure

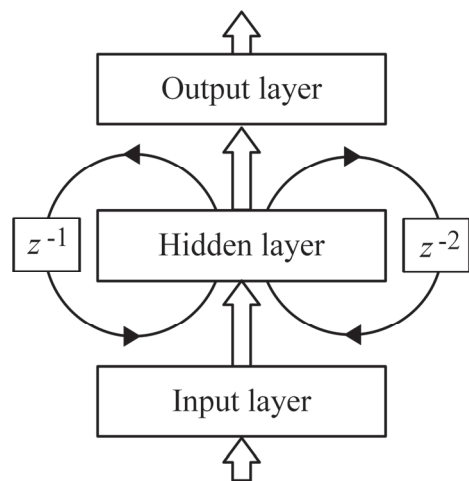


Figure 4.8.1 Two-layer recurrent neural network.

In this section, a two-layer RNN as shown in Figure 4.8.1 for learning of word sequences [Kwa0008a], [Kwa0105] is described.

The RNN adopts a recurrent structure, and makes use of backpropagation (BP) and genetic algorithm (GA) to arrive at an efficient design.

4.8.2 Gradient-based learning algorithm

Let the input pattern $\mathbf{X}(t)$ at time t , and the output patterns $\mathbf{Y}^{[h]}(t)$ at time t of layer 1 and 2 (i.e., $h = 1, 2$) be denoted, respectively, as

$$\mathbf{X}(t) = [x_1(t), x_2(t), x_3(t), \dots, x_{N_0}(t)] \quad (4.8.1)$$

$$\mathbf{Y}^{[h]}(t) = [y_1^{[h]}(t), y_2^{[h]}(t), y_3^{[h]}(t), \dots, y_{N_h}^{[h]}(t)] \text{ for } h = 1, 2 \quad (4.8.2)$$

where $i = 1, 2, 3, \dots, N_0$ denote the indexes of input elements at the input; $j = 1, 2, 3, \dots, N_h$ denote the indexes of the output elements at the output of layer h .

The output at the layer 1, $\mathbf{Y}^{[1]}(t)$, and the output at the layer 2, $\mathbf{Y}^{[2]}(t)$, can then be represented, respectively, by

$$\begin{aligned} & y_k^{[1]}(t) \\ &= F \left[\sum_{s=1}^{N_0} w_{sk}^{[1]} x_s(t) + \sum_{s=1}^{N_1} w_{sk}^{[r1]} y_s^{[1]}(t-1) + w_k^{[r2]} y_k^{[1]}(t-2) \right] \\ & \text{for } k = 1 \text{ to } N_1 \end{aligned} \quad (4.8.3)$$

$$y_l^{[2]}(t) = F \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right] \text{ for } l = 1 \text{ to } N_2 \quad (4.8.4)$$

where $F[(z)]$ denotes the activation function at the output of a neuron; $w_{sk}^{[1]}$ denotes the connection weight from input neuron s to hidden neuron k ; $w_{kl}^{[2]}$ denotes the connection weight from hidden neuron k to output neuron l ; $w_{sk}^{[r1]}$ denotes the connection weight from hidden neuron s to hidden neuron k ; $w_{kk}^{[r1]}$ denotes the first-order feedback weight of hidden neuron k ; and $w_k^{[r2]}$ denotes the second-order feedback weight of hidden neuron k .

The cost function to be minimized is

$$e(t) = \frac{1}{2} \sum_{l=1}^{N_2} e_l^2(t) \quad (4.8.5)$$

where

$$e_l(t) = d_l(t) - y_l^{[2]}(t) \quad (4.8.6)$$

Define the change of weight at the output layer as

$$\Delta w_{ij}^{[2]} = -\varepsilon \frac{\partial e(t)}{\partial w_{ij}^{[2]}} \text{ for } i = 1 \text{ to } N_1 \text{ and } j = 1 \text{ to } N_2 \quad (4.8.7)$$

From (4.8.4)-(4.8.5), (4.8.7) yields the change of weights in the second-layer as

$$\begin{aligned} \Delta w_{ij}^{[2]} &= -\varepsilon e_l(t) \left[\frac{\partial e_l(t)}{\partial w_{ij}^{[2]}} \right] = \varepsilon e_l(t) \left[\frac{\partial y_l^{[2]}(t)}{\partial w_{ij}^{[2]}} \right] \\ &= \varepsilon e_l(t) \left[\frac{\partial F \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right]}{\partial \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right]} \right] \frac{\partial \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right]}{\partial w_{ij}^{[2]}} \\ &= \varepsilon e_l(t) F' \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right] y_i^{[1]}(t) \end{aligned} \quad (4.8.8)$$

for $i = 1 \text{ to } N_1$ and $j = 1 \text{ to } N_2$

Define the change of weight at the input layer as

$$\Delta w_{ij}^{[1]} = -\varepsilon \frac{\partial e(t)}{\partial w_{ij}^{[1]}} \text{ for } i = 1 \text{ to } N_0 \text{ and } j = 1 \text{ to } N_1 \quad (4.8.9)$$

From (4.8.4)-(4.8.5), (4.8.9) yields the change of weights in the first-layer as

$$\begin{aligned} \Delta w_{ij}^{[1]} &= -\varepsilon e_l(t) \left[\frac{\partial e_l(t)}{\partial w_{ij}^{[1]}} \right] = \varepsilon e_l(t) \left[\frac{\partial y_l^{[2]}(t)}{\partial w_{ij}^{[1]}} \right] \\ &= \varepsilon e_l(t) \left[\frac{\partial F \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right]}{\partial \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right]} \right] \sum_{k=1}^{N_1} w_{kl}^{[2]} \frac{\partial y_k^{[1]}(t)}{\partial w_{ij}^{[1]}} \end{aligned}$$

$$= \varepsilon e_l(t) F' \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right] \sum_{k=1}^{N_1} w_{kl}^{[2]} \frac{\partial y_k^{[1]}(t)}{\partial w_{ij}^{[1]}}$$

for $i = 1$ to N_0 and $j = 1$ to N_1 (4.8.10)

From (4.8.3), we obtain

$$\begin{aligned} & \frac{\partial y_k^{[1]}(t)}{\partial w_{ij}^{[1]}} \\ &= F' \left[\sum_{s=1}^{N_0} w_{sk}^{[1]} x_s(t) + \sum_{s=1}^{N_1} w_{sk}^{[r1]} y_s^{[1]}(t-1) + w_k^{[r2]} y_k^{[1]}(t-2) \right] \\ & \times \left[x_i(t) + \sum_{s=1}^{N_1} w_{sk}^{[r1]} \frac{\partial y_s^{[1]}(t-1)}{\partial w_{ij}^{[1]}} + w_k^{[r2]} \frac{\partial y_k^{[1]}(t-2)}{\partial w_{ij}^{[1]}} \right] \end{aligned}$$

for $i = 1$ to N_0 and $j = 1$ to N_1 (4.8.11)

Define the change of weight $\Delta w_{ij}^{[r1]}$ at the hidden layer as

$$\Delta w_{ij}^{[r1]} = -\varepsilon \frac{\partial e(t)}{\partial w_{ij}^{[r1]}} \text{ for } i, j = 1 \text{ to } N_1 \quad (4.8.12)$$

From (4.8.4)-(4.8.5), (4.8.12) yields the change of weights in the first-layer as

$$\begin{aligned} \Delta w_{ij}^{[r1]} &= -\varepsilon e_l(t) \left[\frac{\partial e_l(t)}{\partial w_{ij}^{[r1]}} \right] = \varepsilon e_l(t) \left[\frac{\partial y_l^{[2]}(t)}{\partial w_{ij}^{[r1]}} \right] \\ &= \varepsilon e_l(t) \left[\frac{\partial F \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right]}{\partial \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right]} \right] \sum_{k=1}^{N_1} w_{kl}^{[2]} \frac{\partial y_k^{[1]}(t)}{\partial w_{ij}^{[r1]}} \\ &= \varepsilon e_l(t) F' \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right] \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} \frac{\partial y_k^{[1]}(t)}{\partial w_{ij}^{[r1]}} \right] \end{aligned}$$

$$\text{for } i, j = 1 \text{ to } N_1 \quad (4.8.13)$$

From (4.8.3), we obtain

$$\begin{aligned} & \frac{\partial y_k^{[1]}(t)}{\partial w_{ij}^{[r1]}} \\ &= F' \left[\sum_{s=1}^{N_0} w_{sk}^{[1]} x_s(t) + \sum_{s=1}^{N_1} w_{sk}^{[r1]} y_s^{[1]}(t-1) + w_k^{[r2]} y_k^{[1]}(t-2) \right] \\ & \times \left[\delta_{jk} y_i^{[1]}(t-1) + \sum_{s=1}^{N_1} w_{sk}^{[r1]} \frac{\partial y_s^{[1]}(t-1)}{\partial w_{ij}^{[r1]}} + w_k^{[r2]} \frac{\partial y_k^{[1]}(t-2)}{\partial w_{ij}^{[r1]}} \right] \\ & \text{for } \delta_{jk} = 1 \text{ for } j = k \text{ and } \delta_{ij} = 0 \text{ for } j \neq k \end{aligned} \quad (4.8.14)$$

Define the change of weight $\Delta w_i^{[r2]}$ at the hidden layer as

$$\Delta w_i^{[r2]} = -\varepsilon \frac{\partial e(t)}{\partial w_i^{[r2]}} \text{ for } i = 1 \text{ to } N_1 \quad (4.8.15)$$

From (4.8.4)-(4.8.5), (4.8.15) yields the change of weights in the first-layer as

$$\begin{aligned} \Delta w_i^{[r2]} &= -\varepsilon e_l(t) \left[\frac{\partial e_l(t)}{\partial w_i^{[r2]}} \right] = \varepsilon e_l(t) \left[\frac{\partial y_l^{[2]}(t)}{\partial w_i^{[r2]}} \right] \\ &= \varepsilon e_l(t) \left[\frac{\partial F \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right]}{\partial \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right]} \right] \sum_{k=1}^{N_1} w_{kl}^{[2]} \frac{\partial y_k^{[1]}(t)}{\partial w_i^{[r2]}} \\ &= \varepsilon e_l(t) F' \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} y_k^{[1]}(t) \right] \left[\sum_{k=1}^{N_1} w_{kl}^{[2]} \frac{\partial y_k^{[1]}(t)}{\partial w_i^{[r2]}} \right] \\ & \text{for } i = 1 \text{ to } N_1 \end{aligned} \quad (4.8.16)$$

From (4.8.3), we obtain

$$\begin{aligned}
& \frac{\partial y_k^{[1]}(t)}{\partial w_i^{[r2]}} \\
&= F' \left[\sum_{s=1}^{N_0} w_{sk}^{[1]} x_s(t) + \sum_{s=1}^{N_1} w_{sk}^{[r1]} y_s^{[1]}(t-1) + w_k^{[r2]} y_k^{[1]}(t-2) \right] \\
&\times \left[\delta_{ik} y_k^{[1]}(t-2) + \sum_{s=1}^{N_1} w_{sk}^{[r1]} \frac{\partial y_s^{[1]}(t-1)}{\partial w_i^{[r2]}} + w_k^{[r2]} \frac{\partial y_k^{[1]}(t-2)}{\partial w_i^{[r2]}} \right] \\
&\text{for } \delta_{ik} = 1 \text{ for } i = k \text{ and } \delta_{ik} = 0 \text{ for } i \neq k
\end{aligned} \tag{4.8.17}$$

4.8.3 Genetic-based learning algorithm

In this learning algorithm, both the weights and connections in a chromosome are learned dynamically to minimize the cost function.

4.8.3.1 Encoding

Figure 4.8.2 shows the structure of a chromosome which consists of the four sets of values of weight-connection pairs.

The four sets of weight-connection pairs are ordered in sequence as the input weight-connection pairs $(w_{ij}^{[1]}, c_{ij}^{[1]})$ for $i = 1$ to N_0 and $j = 1$ to N_1 , the output weight-connection pairs $(w_{ij}^{[2]}, c_{ij}^{[2]})$ for $i = 1$ to N_1 and $j = 1$ to N_2 , the inter-lateral hidden weight-connection pairs $(w_{ij}^{[h1]}, c_{ij}^{[h1]})$ for $i, j = 1$ to N_1 , and second-order feedback hidden weight-connection pairs $(w_j^{[1]}, c_j^{[1]})$ for $j = 1$ to N_1 .

The symbol w denotes weight value and the symbol c denotes the corresponding connection value of “1” or “0”.

In the simulations, $c_{ij}^{[h1]}$ and $c_j^{[1]}$ are always set to “1” so that the 1st-order and the 2nd-order self-feedbacks are effective in all the cases.

$$\left[\left(w_{11}^{[1]}, c_{11}^{[1]} \right), \dots, \left(w_{N_0 N_1}^{[1]}, c_{N_0 N_1}^{[1]} \right) \right], \left[\left(w_{11}^{[2]}, c_{11}^{[2]} \right), \dots, \left(w_{N_1 N_2}^{[2]}, c_{N_1 N_2}^{[2]} \right) \right], \\ \left[\left(w_{11}^{[h1]}, c_{11}^{[h1]} \right), \dots, \left(w_{N_1 N_1}^{[h1]}, c_{N_1 N_1}^{[h1]} \right) \right], \left[\left(w_1^{[h2]}, c_1^{[h2]} \right), \dots, \left(w_{N_1}^{[h2]}, c_{N_1}^{[h2]} \right) \right]$$

Figure 4.8.2 Chromosome structure containing weight-connection pairs.

4.8.3.2 Crossover

The best set of individuals based on the fitness values of their cost functions calculated by (4.8.5) are selected for crossover. Two parents produce two children.

Two selected chromosomes are cut at the point between $\left(w_{N_1 N_2}^{[2]}, c_{N_1 N_2}^{[2]} \right)$ and $\left(w_{11}^{[h1]}, c_{11}^{[h1]} \right)$ so that the first half contains the weights and connections of the input layer and the output layer; and the second half contains the weights and connections of the first-order feedbacks and second-order feedbacks in the hidden layer.

4.8.3.3 Mutation

We randomly select a given number of genes to make mutation: for the selected weight we add a random real value between 0 and 1 and for the selected connection we undertake $1 \rightarrow 0$ and $0 \rightarrow 1$.

4.8.3.4 Reproduction

The top two chromosomes among two parents and their two children are selected as two chromosomes for the next generation.

Design Example 4.2 (50 commands and 24 phone numbers learning using two-layer recurrent neural networks) Matlab

In this example [Kwa0008a], [Kwa0105], simulations are undertaken on the neural network learning of 50 commands of up to 3 words and the 24 phone numbers of 10 digits as listed in Table E4.2.1.

The types of neural networks to be used for learning are genetic algorithm (GA)-based recurrent neural network (GRNN), 1st-order fixed recurrent neural network (1st-order FRNN), 2nd-order fixed recurrent

neural network (2nd-order FRNN), and fixed feedforward neural network (FFNN).

1st-order recurrent backpropagation (RBP) algorithm, 2nd-order recurrent backpropagation (2RBP) algorithm, and GA are used in the training.

In the simulations, the 50 commands are represented by 50-element binary vectors and the 44 input words used in the 50 commands are represented by 44-element binary vectors (See Table E4.2.1). The 24 phone numbers are represented by 24-element binary vectors and each phone digit is represented by 10-element binary vector.

Table E4.2.1 List of 50 commands consisting of 44 words.

open left-front window	turn-on airconditioning	call Mary	AM-1
open right-front window	turn-off airconditioning	call Jane	AM-2
open left-back window	turn-on heater	call Paul	AM-3
open right-back window	turn-off heater	call John	AM-4
close left-front window	turn-on map-light	turn-off phone	AM-5
close right-front window	turn-off map-light	yes	AM-6
close left-back window	turn-on head-lamps	no	AM-7
close right-back window	turn-off head-lamps		FM-1
higher	turn-on front-lamps	turn-on AM-radio	FM-2
lower	turn-off front-lamps	turn-off AM-radio	FM-3
perfect	turn-on wipers	turn-on FM-radio	FM-4
	turn-off wipers	turn-off FM-radio	FM-5
	faster		FM-6
	slower		FM-7

As summarized in Table E4.2.2, the neural network for the learning of 50 commands is a 2nd-order recurrent neural network with 44 neurons at the input layer, 50 neurons at the output layer, and a dynamic number of neurons at the hidden layer (for the FFNN and the FRNN, the number of hidden neurons is 6).

The neural network for the 24 phone numbers consists of 10 input neurons, 24 output neurons and a dynamic number of hidden neurons (for the FFNN and the FRNN, the number of hidden neurons is 4).

Table E4.2.2 Input, hidden, and output configurations of different neural networks.

		GRNN	FRNN	FFNN
50 commands	N_0	44	44	44
	N_1	Dynamic	6	6
	N_2	50	50	50
24 phone numbers	N_0	10	10	10
	N_1	Dynamic	4	4
	N_2	24	24	24

For the GA learning, the population size is 50, the crossover probability is 50%, and mutation rate is 10%. In the 2nd-order recurrent network part, we use average sum of squared errors (ASSE) for the error measure. 1/ASSE is used as the fitness measure.

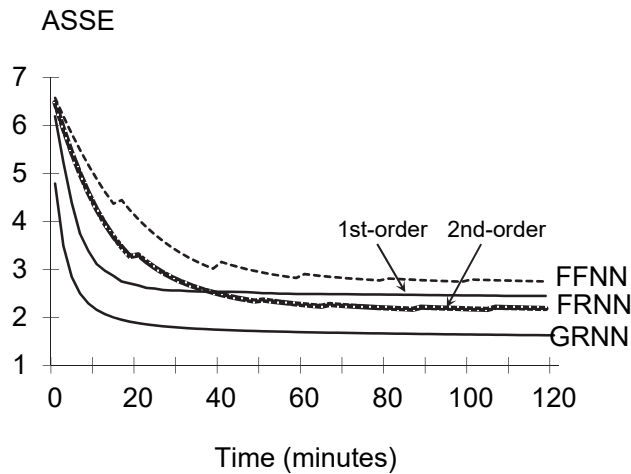


Figure E4.2.1 ASSE of different neural networks for 50 commands learning.

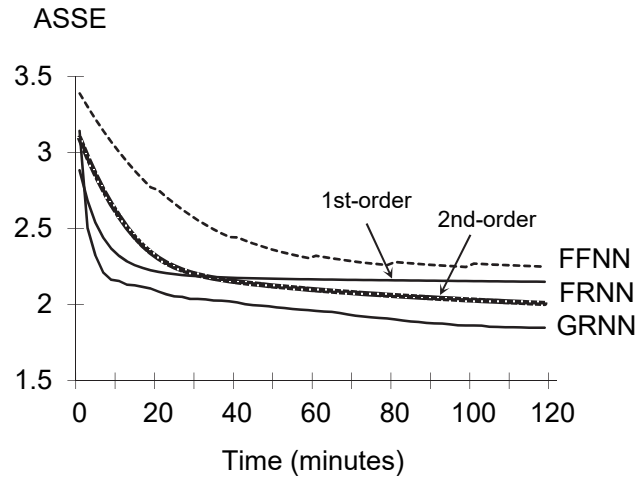


Figure E4.2.2 ASSE of different neural networks for 24 phone numbers learning.

Figures E4.2.1 and E4.2.2 show the average sum of squared errors decrease with time in the learning of the 50 commands and the 24 phone numbers.

The curves in both Figures E4.2.1 and E4.2.2 indicate that the GRNN has the best performance in learning, followed by the 2nd-order FRNN and then the 1st-order FRNN, and finally the FFNN. The convergence of the 1st-order FRNN is faster than that of the 2nd-order FRNN during the initial period.

Benefiting from the memory mechanism of the 1st-order and 2nd-order recurrent neurons, the word sequences can be better learned by the 2nd-order FRNN than those of the 1st-order FRNN and the FFNN.

As learning progresses, this memory mechanism enhances the convergence performance of the 2nd-order FRNN.

In the simulations, the number of hidden neurons in the GRNN is allowed to change dynamically. By adopting a dynamic structure based on genetic algorithm, the GRNN is more capable to escape from local minimum points than those of the fixed-structure schemes where stagnation prevails at a later period.

The advantage of a dynamic neural network is also observed by studying the number of connections among the network neurons in Figures E4.2.3 and E4.2.4.

The connection of the FFNN is too simple to study the current task well while that of the FRNN is complex enough to cause over-fitting in the learning problems.

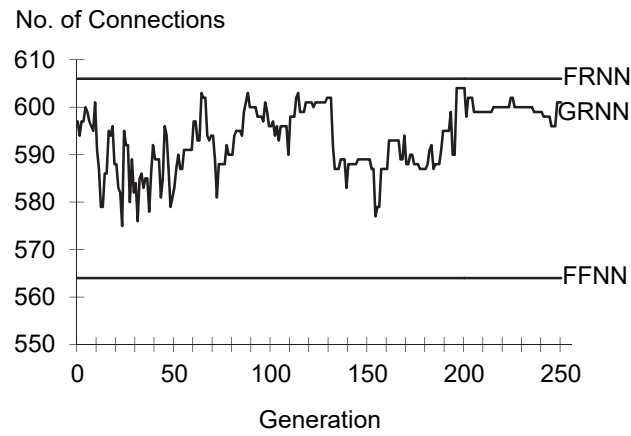


Figure E4.2.3 Number of weight connections in different neural networks for 50 commands.

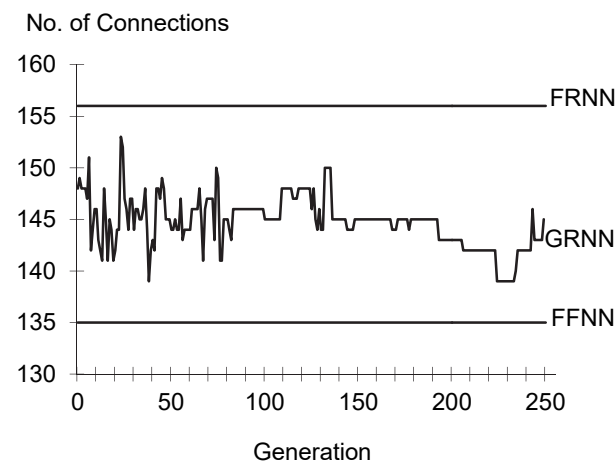


Figure E4.2.4 Number of weight connections in different networks for 24 phone numbers.

To demonstrate the time efficiency of the GRNN, we summarize in Table E4.2.3 the computational time of each scheme in reaching a target ASSE of 2.5 for the learning of 50 commands.

With an Intel Celeron 500 MHz PC, for the GRNN to reach the ASSE of 2.5, the CPU time required is 6.04 minutes by the GRNN. The FRNN is faster (49.26 minutes for the 2nd-order FRNN and 52.12 minutes for the 1st-order FRNN) due to the memory mechanism of the recurrent units.

The FFNN has no memory unit and requires the longest time (205.05 minutes) among the four networks.

By avoiding local minima through the GA and maintaining memory context through recurrent units, the GRNN is the fastest among all the schemes.

Table E4.2.3 Time requirements of different neural networks reaching ASSE=2.5 for 50 commands learning.

Network	GRNN	2 nd -order FRNN	1 st -order FRNN	FFNN
Learning	GA	2RBP	RBP	BP
CPU (Mins.)	6.04	49.26	52.12	205.05

Table E4.2.4 Recall accuracies of different neural networks for 50 commands (not bracketed) and 24 phone numbers (bracketed).

Set	GRNN	2 nd -order	1 st -order	FFNN
1	0.990	0.970 (0.971)	0.953 (0.956)	0.925
2	0.981	0.968 (0.970)	0.949 (0.950)	0.922
3	0.968	0.965 (0.968)	0.944 (0.947)	0.921
4	0.963	0.962 (0.966)	0.942 (0.945)	0.915
5	0.962	0.958 (0.964)	0.937 (0.940)	0.912
6	0.958	0.955 (0.962)	0.935 (0.938)	0.907
7	0.952	0.952 (0.960)	0.934 (0.937)	0.902
8	0.949	0.945 (0.957)	0.931 (0.935)	0.898
9	0.945	0.940 (0.953)	0.930 (0.932)	0.895

We also tested the recall accuracies of these neural networks for both the 50 commands and the 24 phone numbers and the results are summarized in Table E4.2.4.

In Table E4.2.4, we use 9 testing sets constructed by adding 10%, 20%, and 30% of random noise to 10%, 20% and 30% of the randomly selected inputs, respectively.

Table E4.2.4 shows that the GRNN achieves the best recall accuracy in every testing set, followed by the FRNN (2nd-order then 1st-order) and finally the FFNN.

The testing results for the 24 phone numbers are better than those of the 50 commands. It is due in part to smaller ASSE in the training phrase and in part to the longer time sequence of the 24 phone numbers.

To illustrate that these parameters are good choices, we have experimented different population sizes, crossover probabilities, and mutation rates with the learning task of the 50 commands.

The results are summarized in Table E4.2.5, and Figures E4.2.5 and E4.2.6.

In general, the ASSE reduces with an increase of the population size, we observe from Table E4.2.5 that a 9% reduction from 2.31 to 2.09 in the ASSE could only be realized at the cost of nearly ten times CPU time from 8.62 minutes to 85.01 minutes.

The population size of 50 turns out to be a reasonable choice requiring a small amount of CPU time.

Table E4.2.5 Performance effects of population size.

Population size	20	50	100	500
ASSE at 240 generation	2.66	2.31	2.17	2.09
CPU (mins.) reaching generation 240	3.85	8.62	17.55	85.01

Figures E4.2.5 and E4.2.6 show that the crossover probability of 0.5 (50%) and mutation rate of 0.1 (10%) achieve the smallest ASSE, respectively, which indicate that they are good choices.

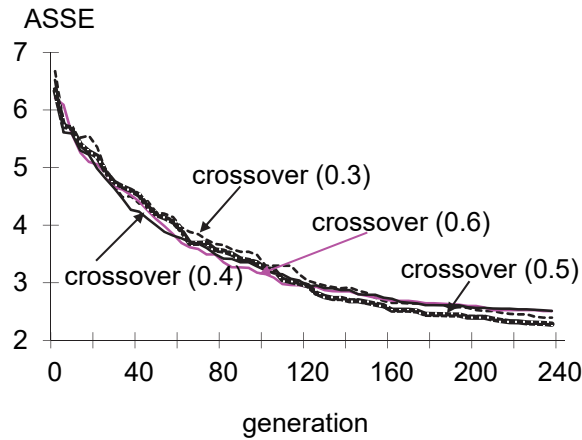


Figure E4.2.5 ASSE for GRNN with different crossover probabilities.

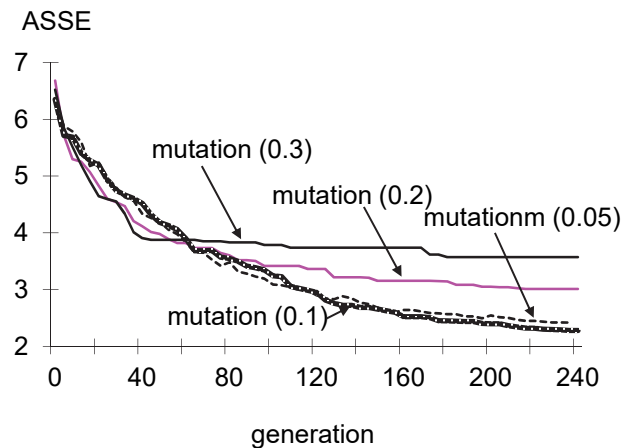


Figure E4.2.6 ASSE for GRNN with different mutation rates.

Based on the results shown in in Table E4.2.5, and Figures E4.2.5 and E4.2.6, we have chosen 50 as the population size, 50% as the crossover probability, and 10% as the mutation rate in the GRNN simulations.

Figure E4.2.7 shows the ASSE performance of the GRNN (of the 50 commands) with (1, -1, 0) weights.

Such a network is faster to hit a local minimum and reaches a lower level of ASSE in earlier due to fewer updating possibilities and larger updating values at each learning step for the weights.

However, fewer updating possibilities are easier to cause a stagnation at late generations for the (1, -1, 0) weights as compared with that of the continuous weights.

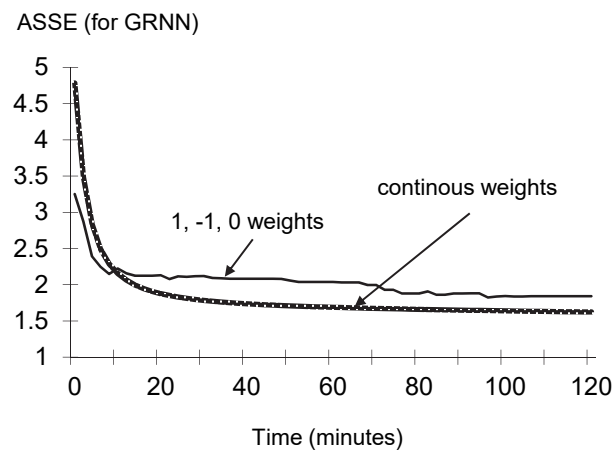


Figure E4.2.7 ASSE for GRNN with discrete and continuous weights.

4.8.4 Summary

This section has described a genetic algorithm GA-based 2nd-order recurrent neural network (GRNN).

Feedbacks in the structure enable the network to remember cues from the recent past of a word sequence.

The GA is used to design an improved network by evolving weights and connections dynamically.

The GRNN has been applied to the learning of the 50 and the 24 phone numbers.

The results indicate that the GRNN has the advantages of shortest learning time, smallest convergence error, highest recall accuracy, and reduced number of connection weights.

The effects of population size, crossover probability, and mutation rate on the performances of the GRNN are also presented.

Chapter 5

Associative Memories

5.1 Introduction

A discrete bidirectional associative memories (BAM) [Kos8712], [Kos8801] can be used to store K discrete pattern pairs $(\mathbf{X}_k, \mathbf{Y}_k)$ in the form of a weight matrix $\mathbf{W}$ such that any one of $\mathbf{X}_k$ or $\mathbf{Y}_k$ (with or without noise) can be used to recall its partner and itself after a number of forward and backward iterations.

An $N \times M$ real matrix $\mathbf{W}$ can be interpreted as a matrix of synapses between two fields of neurons.

Information passes forward from one neuron field to the other by passing through the connection matrix $\mathbf{W}$ while information passes backward through the transpose matrix $\mathbf{W}^T$.

A bidirectional associative memories can also be used as a temporal associative memories (TAM) [Kos1988] to store a sequence of discrete patterns; for example, $\mathbf{X}_1 \rightarrow \mathbf{X}_2 \rightarrow \mathbf{X}_3 \rightarrow \mathbf{X}_1$.

In this chapter, the structure, the weight matrix for learning and recall, the storage, and the properties of bidirectional associative memories and temporal associative memories with zero threshold, bipolar patterns, and bipolar weight matrix are described.

In addition, cascaded and ring associative memories, and multilayer symmetric and asymmetrical associative memories are introduced.

Cascaded and ring associative memories [Kwa8906b], [Kwa8909b], [Kwa9205] can increase capacities for storing bipolar temporal patterns.

Multilayer symmetric and asymmetrical associative memories [Kwa8909a] [Kwa0208a], [Kwa0305a], [Kwa0505] can store discrete-valued as well as real-valued pattern-pairs.

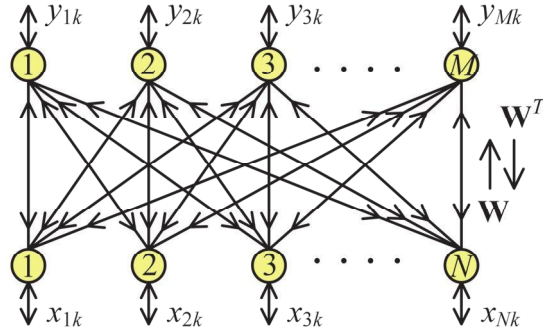


Figure 5.2.1 Structure of BAM.

5.2 Bidirectional Associative Memories

Figure 5.2.1 shows the structure of a BAM which can be used for realizing bidirectional associative memories and temporal associative memories.

5.2.1 Weight matrix

The bidirectional associative memories can learn associations of K discrete-valued pattern pairs using the Hebb learning rule.

The weight matrix is constructed by summing the outer products of all the pattern pairs $\mathbf{X}_k$ and $\mathbf{Y}_k$, $k = 1, 2, \dots, K$, as

$$\mathbf{W} = \sum_{k=1}^K \mathbf{X}_k^T \mathbf{Y}_k \quad (5.2.1)$$

where $\mathbf{X}_k$ and $\mathbf{Y}_k$ are vectors of dimensions $1 \times N$ and $1 \times M$, respectively, so $\mathbf{W}$ is an $N \times M$ matrix. For $N = M = 3$ and $K = 2$, we have

$$\begin{aligned} \mathbf{W} &= \begin{bmatrix} x_{11} \\ x_{21} \\ x_{31} \end{bmatrix} \begin{bmatrix} y_{11} & y_{21} & y_{31} \end{bmatrix} + \begin{bmatrix} x_{12} \\ x_{22} \\ x_{32} \end{bmatrix} \begin{bmatrix} y_{12} & y_{22} & y_{32} \end{bmatrix} \\ &= \begin{bmatrix} x_{11}y_{11} + x_{12}y_{12} & x_{11}y_{21} + x_{12}y_{22} & x_{11}y_{31} + x_{12}y_{32} \\ x_{21}y_{11} + x_{22}y_{12} & x_{21}y_{21} + x_{22}y_{22} & x_{21}y_{31} + x_{22}y_{32} \\ x_{31}y_{11} + x_{32}y_{12} & x_{31}y_{21} + x_{32}y_{22} & x_{31}y_{31} + x_{32}y_{32} \end{bmatrix} \end{aligned}$$

5.2.2 Recall procedure

Taking into account the existence of a nonlinear quantizer $G[.]$ at the output of each neuron, the operations of forward and backward recalls can be expressed as

$$\mathbf{Y}_k = G[\mathbf{X}_k \mathbf{W}] \quad (5.2.2)$$

$$\mathbf{X}_k = G[\mathbf{Y}_k \mathbf{W}^T] \quad (5.2.3)$$

For bipolar patterns, the bipolar step function $G_h[.]$ provides an appropriate choice for the nonlinear quantizer in which $G_h[.]$ is defined as

$$G_h(z) = \begin{cases} +1 & z \geq 0 \\ -1 & z < 0 \end{cases} \quad (5.2.4)$$

Orthonormal vector plays an important role in the recall procedure of a BAM. An orthonormal vector $\mathbf{X}_k$ is defined as

$$\mathbf{X}_k \mathbf{X}_j^T = \begin{cases} 1 & k = j \\ 0 & k \neq j \end{cases} \quad (5.2.5)$$

For example, the product of two orthonormal row vectors $\mathbf{X}_k$ and $\mathbf{X}_j$ each consisting of three elements exhibits the following property as

$$\begin{aligned} \mathbf{X}_k \mathbf{X}_j^T &= [x_{1k} \quad x_{2k} \quad x_{3k}] \begin{bmatrix} x_{1j} \\ x_{2j} \\ x_{3j} \end{bmatrix} = x_{1k}x_{1j} + x_{2k}x_{2j} + x_{3k}x_{3j} \\ &= \begin{cases} 1 & \text{for } k = j \\ 0 & \text{for } k \neq j \end{cases} \end{aligned}$$

In general, orthogonal vectors are unnormalized orthonormal vectors. Therefore, when orthogonal vectors are used, for forward recall, we have

$$G_h[\mathbf{X}_k \mathbf{W}] = G_h \left[\overset{\text{Information term}}{\mathbf{X}_k \mathbf{X}_k^T \mathbf{Y}_k} + \sum_{j=1, j \neq k}^K \overset{\text{Noisy term}}{\mathbf{X}_k \mathbf{X}_j^T \mathbf{Y}_j} \right] = \mathbf{Y}_k \quad (5.2.6)$$

for orthogonal vectors

Similarly, for backward recall, we have

$$G_h[\mathbf{Y}_k \mathbf{W}^T] = G_h \left[\overset{\text{Information term}}{\mathbf{Y}_k \mathbf{Y}_k^T \mathbf{X}_k} + \sum_{j=1, j \neq k}^K \overset{\text{Noisy term}}{\mathbf{Y}_k \mathbf{Y}_j^T \mathbf{X}_j} \right] = \mathbf{X}_k \quad (5.2.7)$$

for orthogonal vectors

5.2.3 Properties of BAMs

The properties of BAMs can be summarized as

(a) A bipolar BAM stores $\mathbf{X}_k$ and $\mathbf{Y}_k$ as well as its complementary pair $\mathbf{X}_k^c$ and $\mathbf{Y}_k^c$. This implies

$$\mathbf{Y}_k^c = G_h[\mathbf{X}_k^c \mathbf{W}] \quad (5.2.8)$$

$$\mathbf{X}_k^c = G_h[\mathbf{Y}_k^c \mathbf{W}^T] \quad (5.2.9)$$

(b) In general, the number of pattern pairs K that can be stored and accurately recalled is less than the minimum between the two vector dimensions (N, M) , which is

$$K < \min(N, M) \quad (5.2.10)$$

(c) Every real-valued matrix is bidirectionally stable.

Example 5.1 (Learning and recalls of BAM) Matlab

A BAM is to be used to store the following 2 pattern pairs given by

$$\mathbf{X}_1 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] \text{ and } \mathbf{Y}_1 = [1 \ -1 \ 1 \ -1 \ 1 \ -1]$$

$$\mathbf{X}_2 = [1 \ 1 \ 1 \ 1 \ -1 \ -1 \ -1 \ -1] \text{ and } \mathbf{Y}_2 = [1 \ 1 \ -1 \ -1 \ 1 \ 1]$$

Assuming that the bipolar step function is used at the output of each of the neurons,

(a) Determine the weight matrix $\mathbf{W}$ of the BAM.

- (b) Check the correctness of recalls of the original pattern pairs.
- (c) Check the correctness of recalls of the complementary pattern pairs.
- (d) Determine the steady-state recall pattern pair of the following noisy vector, $\mathbf{X}_t = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ 1 \ 1]$.

Solution

(a)

$$\mathbf{X}_1^T \mathbf{Y}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1]$$

$$= \begin{bmatrix} 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{X}_2^T \mathbf{Y}_2 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ -1 \\ -1 \\ -1 \\ -1 \end{bmatrix} [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1]$$

$$\begin{aligned}
&= \begin{bmatrix} 1 & 1 & -1 & -1 & 1 & 1 \\ 1 & 1 & -1 & -1 & 1 & 1 \\ 1 & 1 & -1 & -1 & 1 & 1 \\ 1 & 1 & -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 \end{bmatrix} \\
\mathbf{W} = \mathbf{X}_1^T \mathbf{Y}_1 + \mathbf{X}_2^T \mathbf{Y}_2 &= \begin{bmatrix} 2 & 0 & 0 & -2 & 2 & 0 \\ 2 & 0 & 0 & -2 & 2 & 0 \\ 2 & 0 & 0 & -2 & 2 & 0 \\ 2 & 0 & 0 & -2 & 2 & 0 \\ 0 & -2 & 2 & 0 & 0 & -2 \\ 0 & -2 & 2 & 0 & 0 & -2 \\ 0 & -2 & 2 & 0 & 0 & -2 \\ 0 & -2 & 2 & 0 & 0 & -2 \end{bmatrix}
\end{aligned}$$

(b) For original pattern pairs recalls

Forward recalls:

$$\begin{aligned}
&G_h[\mathbf{X}_1 \mathbf{W}] \\
&= G_h[[1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] \mathbf{W}] \\
&= G_h[[8 \quad -8 \quad 8 \quad -8 \quad 8 \quad -8]] \\
&= [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] = \mathbf{Y}_1
\end{aligned}$$

$$\begin{aligned}
&G_h[\mathbf{X}_2 \mathbf{W}] \\
&= G_h[[1 \quad 1 \quad 1 \quad 1 \quad -1 \quad -1 \quad -1 \quad -1] \mathbf{W}] \\
&= G_h[[8 \quad 8 \quad -8 \quad -8 \quad 8 \quad 8]] \\
&= [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1] = \mathbf{Y}_2
\end{aligned}$$

Backward recalls:

$$\begin{aligned}
 & G_h[\mathbf{Y}_1 \mathbf{W}^T] \\
 &= G_h[[1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \mathbf{W}^T] \\
 &= G_h[[6 \quad 6 \quad 6 \quad 6 \quad 6 \quad 6 \quad 6 \quad 6]] \\
 &= [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1
 \end{aligned}$$

$$\begin{aligned}
 & G_h[\mathbf{Y}_2 \mathbf{W}^T] \\
 &= G_h[[1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1] \mathbf{W}^T] \\
 &= G_h[[6 \quad 6 \quad 6 \quad 6 \quad -6 \quad -6 \quad -6 \quad -6]] \\
 &= [1 \quad 1 \quad 1 \quad 1 \quad -1 \quad -1 \quad -1 \quad -1] = \mathbf{X}_2
 \end{aligned}$$

(c) For complementary pattern pairs recalls

Forward recalls:

$$\begin{aligned}
 & G_h[\mathbf{X}_1^c \mathbf{W}] \\
 &= G_h[[-1 \quad -1 \quad -1 \quad -1 \quad -1 \quad -1 \quad -1 \quad -1] \mathbf{W}] \\
 &= G_h[[-8 \quad 8 \quad -8 \quad 8 \quad -8 \quad 8]] \\
 &= [-1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1] = \mathbf{Y}_1^c
 \end{aligned}$$

$$\begin{aligned}
 & G_h[\mathbf{X}_2^c \mathbf{W}] \\
 &= G_h[[-1 \quad -1 \quad -1 \quad -1 \quad 1 \quad 1 \quad 1 \quad 1] \mathbf{W}] \\
 &= G_h[[-8 \quad -8 \quad 8 \quad 8 \quad -8 \quad -8]] \\
 &= [-1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] = \mathbf{Y}_2^c
 \end{aligned}$$

Backward recalls:

$$\begin{aligned}
 & G_h[\mathbf{Y}_1^c \mathbf{W}^T] \\
 &= G_h[[-1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1] \mathbf{W}^T] \\
 &= G_h[[-6 \quad -6 \quad -6 \quad -6 \quad -6 \quad -6 \quad -6 \quad -6]] \\
 &= [-1 \quad -1 \quad -1 \quad -1 \quad -1 \quad -1 \quad -1 \quad -1] = \mathbf{X}_1^c
 \end{aligned}$$

$$\begin{aligned}
 & G_h[\mathbf{Y}_2^c \mathbf{W}^T] \\
 &= G_h[[-1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \mathbf{W}^T] \\
 &= G_h[[-6 \quad -6 \quad -6 \quad -6 \quad 6 \quad 6 \quad 6 \quad 6]] \\
 &= [-1 \quad -1 \quad -1 \quad -1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_2^c
 \end{aligned}$$

(d) For the recall of the noisy vector,

$$\begin{aligned}
 \mathbf{X}_t &= [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad 1 \quad 1] \\
 & G_h[\mathbf{X}_t \mathbf{W}] \\
 &= G_h[[1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad 1 \quad 1] \mathbf{W}] \\
 &= G_h[[0 \quad -8 \quad 8 \quad 0 \quad 0 \quad -8]] \\
 &= [1 \quad -1 \quad 1 \quad 1 \quad 1 \quad -1] = \mathbf{Y}_t \\
 & G_h[\mathbf{Y}_t \mathbf{W}^T] \\
 &= G_h[[1 \quad -1 \quad 1 \quad 1 \quad 1 \quad -1] \mathbf{W}^T] \\
 &= G_h[[2 \quad 2 \quad 2 \quad 2 \quad 6 \quad 6 \quad 6 \quad 6]] \\
 &= [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1
 \end{aligned}$$

$$G_h[\mathbf{X}_1 \mathbf{W}] = \mathbf{Y}_1$$

$$G_h[\mathbf{Y}_1 \mathbf{W}^T] = \mathbf{X}_1$$

The steady-state recall pattern pair is $\mathbf{X}_1$ and $\mathbf{Y}_1$.

5.3 Temporal Associative Memories

In a temporal associative memories (TAM), temporal patterns are ordered vectors which can be represented as a sequence of vectors.

For example, the four vectors ($\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \mathbf{X}_1$) are temporal patterns in the form of a sequence of vectors as

$$\mathbf{X}_1 = [1 \ -1 \ -1 \ 1 \ -1 \ -1 \ 1 \ -1 \ -1 \ 1]$$

$$\mathbf{X}_2 = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1]$$

$$\mathbf{X}_3 = [1 \ -1 \ 1 \ -1 \ 1 \ 1 \ 1 \ -1 \ 1 \ -1].$$

$\mathbf{X}_1$ is appended to the sequence ($\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3$) to convert the sequence into an infinite loop in the sense that $\mathbf{X}_k$ is always followed by $\mathbf{X}_{k+1}$ for $k = 1, 2$ and $\mathbf{X}_3$ is followed by $\mathbf{X}_1$.

The temporal patterns ($\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_K$) can be memorized by storing the local relationships $\mathbf{X}_1 - \mathbf{X}_2, \mathbf{X}_2 - \mathbf{X}_3, \dots, \mathbf{X}_{K-1} - \mathbf{X}_K$, and $\mathbf{X}_K - \mathbf{X}_1$.

5.3.1 Weight matrix

The local relationship $\mathbf{X}_k - \mathbf{X}_{k+1}$ is memorized as if it were the heteroassociative pair $(\mathbf{X}_k, \mathbf{X}_{k+1})$ by forming the correlation matrix $\mathbf{X}_k^T \mathbf{X}_{k+1}$.

In order to store the temporal discrete patterns $\mathbf{X}_k$ for $k = 1, 2, \dots, K$, all the local relationships are added to obtain the weight matrix $\mathbf{W}$ as

$$\mathbf{W} = \mathbf{X}_K^T \mathbf{X}_1 + \sum_{k=1}^{K-1} \mathbf{X}_k^T \mathbf{X}_{k+1} \quad (5.3.1)$$

5.3.2 Recall procedure

Forward recall for $k \neq K$:

$$\begin{aligned}
 G_h[\mathbf{X}_k \mathbf{W}] &= G_h \left[\overset{\text{Noisy term}}{\mathbf{X}_k \mathbf{X}_K^T \mathbf{X}_1} + \overset{\text{Information term}}{\mathbf{X}_k \mathbf{X}_k^T \mathbf{X}_{k+1}} + \overset{\text{Noisy term}}{\sum_{j=1, j \neq k}^{K-1} \mathbf{X}_k \mathbf{X}_j^T \mathbf{X}_{j+1}} \right] \\
 &= \mathbf{X}_{k+1} \text{ for correct recall}
 \end{aligned} \tag{5.3.2}$$

Forward recall for $k = K$:

$$\begin{aligned}
 G_h[\mathbf{X}_K \mathbf{W}] &= G_h \left[\overset{\text{Information term}}{\mathbf{X}_K \mathbf{X}_K^T \mathbf{X}_1} + \overset{\text{Noisy term}}{\sum_{j=1}^{K-1} \mathbf{X}_K \mathbf{X}_j^T \mathbf{X}_{j+1}} \right] \\
 &= \mathbf{X}_1 \text{ for correct recall}
 \end{aligned} \tag{5.3.3}$$

Taking the transpose of the weight matrix (5.3.1), we obtain

$$\mathbf{W}^T = \mathbf{X}_1^T \mathbf{X}_K + \sum_{k=1}^{K-1} \mathbf{X}_{k+1}^T \mathbf{X}_k \tag{5.3.4}$$

Substituting $j = k + 1$ into the second term of (5.3.4), we obtain

$$\mathbf{W}^T = \mathbf{X}_1^T \mathbf{X}_K + \sum_{j=2}^K \mathbf{X}_j^T \mathbf{X}_{j-1} \tag{5.3.5}$$

Backward recall for $k \neq 1$:

$$G_h[\mathbf{X}_k \mathbf{W}^T] = G_h \left[\overset{\text{Noisy term}}{\mathbf{X}_k \mathbf{X}_1^T \mathbf{X}_K} + \overset{\text{Information term}}{\mathbf{X}_k \mathbf{X}_k^T \mathbf{X}_{k-1}} + \overset{\text{Noisy term}}{\sum_{j=2, j \neq k}^K \mathbf{X}_k \mathbf{X}_j^T \mathbf{X}_{j-1}} \right]$$

$$= \mathbf{X}_{k-1} \text{ for correct recall} \quad (5.3.6)$$

Backward recall for $k = 1$:

$$G_h[\mathbf{X}_1 \mathbf{W}^T] = G_h \left[\overset{\text{Information term}}{\mathbf{X}_1 \mathbf{X}_1^T \mathbf{X}_K} + \overset{\text{Noisy term}}{\sum_{j=2}^K \mathbf{X}_1 \mathbf{X}_j^T \mathbf{X}_{j-1}} \right]$$

$$= \mathbf{X}_K \text{ for correct recall} \quad (5.3.7)$$

The operations of the forward recall and the backward recall are illustrated by Figures 5.3.1 and 5.3.2.

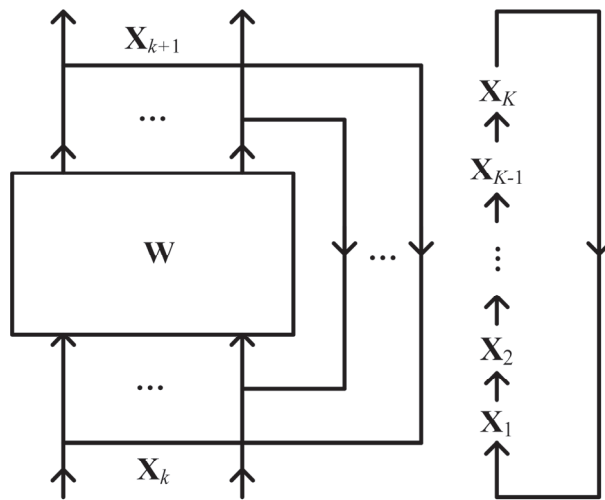


Figure 5.3.1 Forward recall of a TAM using a looped BAM.

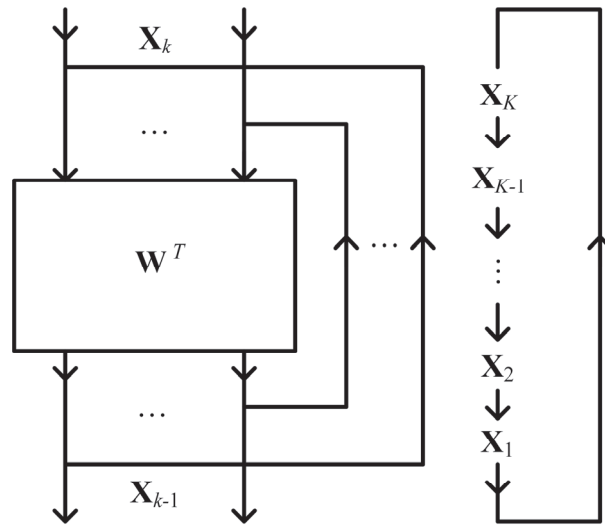


Figure 5.3.2 Backward recall of a TAM using a looped BAM.

5.3.3 Storage capacity

The approximate maximum storage capacity bound for a K -length temporal patterns is $K < N$, where N is the number of elements in each of the vectors $\mathbf{X}_k$ for $k = 1$ to K .

Example 5.2 (Learning and recalls of TAM) Matlab

Consider a sequence of temporal patterns $\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \mathbf{X}_1$ given below are to be stored in a TAM,

$$\mathbf{X}_1 = [1 \ -1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1]$$

$$\mathbf{X}_2 = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1]$$

$$\mathbf{X}_3 = [1 \ 1 \ 1 \ 1 \ -1 \ -1 \ -1 \ -1].$$

- Determine the memory matrix $\mathbf{W}$ of the TAM.
- Check the correctness of forward recalls and backward recalls.
- Check the correctness of complementary forward recalls and complementary backward recalls.

(d) Given the following noisy vector as $\mathbf{X}_t = [1 \ 1 \ -1 \ -1 \ 1 \ -1 \ -1 \ -1]$, identify the original vector of $\mathbf{X}_t$ by performing forward and backward recalls.

Solution

(a)

$$\mathbf{W} = \mathbf{X}_1^T \mathbf{X}_2 + \mathbf{X}_2^T \mathbf{X}_3 + \mathbf{X}_3^T \mathbf{X}_1$$

$$= \begin{bmatrix} 3 & 1 & 1 & -1 & 1 & -1 & -1 & -3 \\ 1 & -1 & 3 & 1 & -1 & -3 & 1 & -1 \\ 1 & -1 & -1 & -3 & 3 & 1 & 1 & -1 \\ -1 & -3 & 1 & -1 & 1 & -1 & 3 & 1 \\ 1 & 3 & -1 & 1 & -1 & 1 & -3 & -1 \\ -1 & 1 & 1 & 3 & -3 & -1 & -1 & 1 \\ -1 & 1 & -3 & -1 & 1 & 3 & -1 & 1 \\ -3 & -1 & -1 & 1 & -1 & 1 & 1 & 3 \end{bmatrix}$$

(b) For the original pattern sequence recall

Forward recalls:

$$\begin{aligned} G_h[\mathbf{X}_1 \mathbf{W}] &= G_h[[1 \ -1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1] \mathbf{W}] \\ &= G_h[[8 \ 8 \ -8 \ -8 \ 8 \ 8 \ -8 \ -8]] \\ &= [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1] = \mathbf{X}_2 \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_2 \mathbf{W}] &= G_h[[1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1] \mathbf{W}] \\ &= G_h[[8 \ 8 \ 8 \ 8 \ -8 \ -8 \ -8 \ -8]] \\ &= [1 \ 1 \ 1 \ 1 \ -1 \ -1 \ -1 \ -1] = \mathbf{X}_3 \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_3 \mathbf{W}] &= G_h[[1 \ 1 \ 1 \ 1 \ -1 \ -1 \ -1 \ -1] \mathbf{W}] \\ &= G_h[[8 \ -8 \ 8 \ -8 \ 8 \ -8 \ 8 \ -8]] \\ &= [1 \ -1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1] = \mathbf{X}_1 \end{aligned}$$

Backward recalls:

$$\begin{aligned} G_h[\mathbf{X}_1 \mathbf{W}^T] &= G_h[[1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \mathbf{W}^T] \\ &= G_h[[8 \quad 8 \quad 8 \quad 8 \quad -8 \quad -8 \quad -8 \quad -8]] \\ &= [1 \quad 1 \quad 1 \quad 1 \quad -1 \quad -1 \quad -1 \quad -1] = \mathbf{X}_3 \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_3 \mathbf{W}^T] &= G_h[[1 \quad 1 \quad 1 \quad 1 \quad -1 \quad -1 \quad -1 \quad -1] \mathbf{W}^T] \\ &= G_h[[8 \quad 8 \quad -8 \quad -8 \quad 8 \quad 8 \quad -8 \quad -8]] \\ &= [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] = \mathbf{X}_2 \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_2 \mathbf{W}^T] &= G_h[[1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \mathbf{W}^T] \\ &= G_h[[8 \quad -8 \quad 8 \quad -8 \quad 8 \quad -8 \quad 8 \quad -8]] \\ &= [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] = \mathbf{X}_1 \end{aligned}$$

(c) For the complementary pattern sequence recalls

Forward recalls:

$$\begin{aligned} G_h[\mathbf{X}_1^c \mathbf{W}] &= G_h[[-1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1] \mathbf{W}] \\ &= G_h[[-8 \quad -8 \quad 8 \quad 8 \quad -8 \quad -8 \quad 8 \quad 8]] \\ &= [-1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1] = \mathbf{X}_2^c \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_2^c \mathbf{W}] &= G_h[[-1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1] \mathbf{W}] \\ &= G_h[[-8 \quad -8 \quad -8 \quad -8 \quad 8 \quad 8 \quad 8 \quad 8]] \\ &= [-1 \quad -1 \quad -1 \quad -1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_3^c \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_3^c \mathbf{W}] &= G_h[[-1 \quad -1 \quad -1 \quad -1 \quad 1 \quad 1 \quad 1 \quad 1] \mathbf{W}] \\ &= G_h[[-8 \quad 8 \quad -8 \quad 8 \quad -8 \quad 8 \quad -8 \quad 8]] \\ &= [-1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1] = \mathbf{X}_1^c \end{aligned}$$

Backward recalls:

$$\begin{aligned} G_h[\mathbf{X}_1^c \mathbf{W}^T] &= G_h[[-1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1 \ 1] \mathbf{W}^T] \\ &= G_h[[-8 \ -8 \ -8 \ -8 \ 8 \ 8 \ 8 \ 8]] \\ &= [-1 \ -1 \ -1 \ -1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_3^c \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_3^c \mathbf{W}^T] &= G_h[[-1 \ -1 \ -1 \ -1 \ 1 \ 1 \ 1 \ 1] \mathbf{W}^T] \\ &= G_h[[-8 \ -8 \ 8 \ 8 \ -8 \ -8 \ 8 \ 8]] \\ &= [-1 \ -1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1] = \mathbf{X}_2^c \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_2^c \mathbf{W}^T] &= G_h[[-1 \ -1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1] \mathbf{W}^T] \\ &= G_h[[-8 \ 8 \ -8 \ 8 \ -8 \ 8 \ -8 \ 8]] \\ &= [-1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1 \ 1] = \mathbf{X}_1^c \end{aligned}$$

(d) For the noisy pattern recalls

Forward recall:

$$\begin{aligned} G_h[\mathbf{X}_t \mathbf{W}] &= G_h[[1 \ 1 \ -1 \ -1 \ 1 \ -1 \ -1 \ -1] \mathbf{W}] \\ &= G_h[[10 \ 6 \ 6 \ 2 \ -2 \ -6 \ -6 \ -10]] \\ &= [1 \ 1 \ 1 \ 1 \ -1 \ -1 \ -1 \ -1] = \mathbf{X}_3 \end{aligned}$$

Backward recall:

$$\begin{aligned} G_h[\mathbf{X}_t \mathbf{W}^T] &= G_h[[1 \ 1 \ -1 \ -1 \ 1 \ -1 \ -1 \ -1] \mathbf{W}^T] \\ &= G_h[[10 \ -2 \ 6 \ -6 \ 6 \ -6 \ 2 \ -10]] \\ &= [1 \ -1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1] = \mathbf{X}_1 \end{aligned}$$

Hence, $\mathbf{X}_t$ is a corrupted version of $\mathbf{X}_2$.

5.4 Use of Bipolar Step Function

So far, for simplicity, the bipolar step function defined in (5.2.4) was used (like the Hebb-rule trained pattern associators and the autoassociators in Chapters 2 and 3) in both the BAM and the TAM for

storing discrete patterns such that the activation output for $z = 0$ is $G_h(0) = 1$.

It is noted that in the BAM and the TAM described in [Kos8712], [Kos8801], the activation output for $z = 0$ remains the same as its previous activation output.

The difference does not pose a problem as $z = 0$ rarely occurs but if it does occur, the use of bipolar step function defined in (5.2.4) can obtain correct recalls. If not, correct recalls can still be obtained by applying two or more iterations.

5.5 Asymmetric, Cascaded, and Ring Associative Memories

Three improved methods [Kwa8909b], [Kwa9205] to store temporal associative patterns are described.

The first method is to use a single bidirectional associative memories with non-symmetric weights which shall be called an asymmetric BAM (ABAM), the second method is to use a cascade bidirectional associative memories (CBAM), and the third method is to use a ring unidirectional associative memories (RUAM).

These three associate memories have an improvement over the classical BAM described in (5.5.1)-(5.5.4) such that they can store two identical patterns located in both odd and even positions such as ($\mathbf{X}_1, \mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3$).

These three networks can store multiple temporal associative sequences. The stability of these three networks is also described and proved.

Their performances are compared by simulations which use words of four alphabets as the temporal associative patterns. These words can be recalled by inputting any one of the alphabets at its designated input.

5.5.1 Classical BAM

In the classical bidirectional associative memories (BAM) network [Kos1988], a number of pairs of data $(\mathbf{X}_k, \mathbf{Y}_k)$ can be stored up to the storage capacity of the network.

Given K bipolar row-vector pairs $(\mathbf{X}_1, \mathbf{Y}_1), (\mathbf{X}_2, \mathbf{Y}_2), (\mathbf{X}_3, \mathbf{Y}_3), \dots, (\mathbf{X}_K, \mathbf{Y}_K)$ where $\mathbf{X}_k \in (-1, 1)^N$ and $\mathbf{Y}_k \in (-1, 1)^M$, which are to be stored in a BAM by a weight matrix $\mathbf{W}$ obtained from a summation of the vector outer products, $\mathbf{X}_k^T \mathbf{Y}_k$ as

$$\mathbf{W} = \sum_{k=1}^K \mathbf{X}_k^T \mathbf{Y}_k \quad (5.5.1)$$

Information stored in such a BAM is recalled by the following transfer function equations as

$$\mathbf{Y}_k = H\{\mathbf{X}_k \mathbf{W}\} \quad (5.5.2)$$

$$\mathbf{X}_k = H\{\mathbf{Y}_k \mathbf{W}^T\} \quad (5.5.3)$$

where $H\{z\}$ represents the operation of a hardlimit function defined by

$$H\{z\} = \begin{cases} 1 & \text{if } z > h \\ \text{unchanged} & \text{if } z = h \\ -1 & \text{if } z < h \end{cases} \quad (5.5.4)$$

where h is a predetermined threshold value. In the following discussion, h is assumed to be zero for all neurons.

The operations of this BAM simply involve a recall of $\mathbf{Y}_k$ by $\mathbf{X}_k$ in one cycle and a recall of $\mathbf{X}_k$ by $\mathbf{Y}_k$ in another cycle until the two vectors are stable.

To store temporal patterns which are ordered vectors as a temporal associative memories (TAM), for example, $(\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \mathbf{X}_4)$, we can store $(\mathbf{X}_1, \mathbf{X}_2)$, and $(\mathbf{X}_3, \mathbf{X}_4)$ in one direction and $(\mathbf{X}_2, \mathbf{X}_3)$ and $(\mathbf{X}_4, \mathbf{X}_1)$ in the opposition direction in a BAM and then recall the whole sequence by inputting $\mathbf{X}_1$ to forward recall $\mathbf{X}_2$ at the output; then feedback recall $\mathbf{X}_3$ at the input and forward recall $\mathbf{X}_4$ at the output; and finally feedback recall $\mathbf{X}_1$ at the input to form a complete loop which can be repeated as needed.

In this method, the length of a sequence will be limited by the storage capacity of the BAM and also we cannot store a sequence which has identical row vectors in different positions. For example, we cannot store $(\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_1, \mathbf{X}_3)$ because $\mathbf{X}_1$ can only forward recall either $\mathbf{X}_2$ or $\mathbf{X}_3$; for another example, we cannot store $(\mathbf{X}_1, \mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3)$ because $\mathbf{X}_1$ can only backward recall either $\mathbf{X}_1$ or $\mathbf{X}_2$.

5.5.2 Asymmetric BAM

In order to store temporal associative patterns, we can use two different weight matrices for the forward and backward directions in a BAM to form an asymmetric BAM [Kwa8909b].

For example, if we want to store $(\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \mathbf{X}_4)$, we can store $(\mathbf{X}_1, \mathbf{X}_2)$ and $(\mathbf{X}_3, \mathbf{X}_4)$ in one direction (say forward direction), and $(\mathbf{X}_2, \mathbf{X}_3)$ and $(\mathbf{X}_4, \mathbf{X}_1)$ in the other direction (say backward direction).

Then we can recall $\mathbf{X}_2$ by inputting $\mathbf{X}_1$ in the forward direction and $\mathbf{X}_2$ will then recall $\mathbf{X}_3$ in the backward direction and so on as shown in Figure 5.5.1.

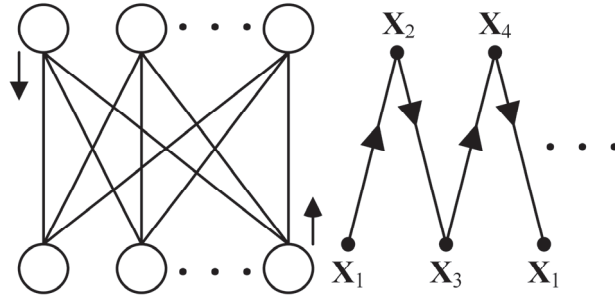


Figure 5.5.1 Temporal associative memory using single BAM.

In general, the weight matrices (i.e., $\mathbf{W}_f$ for the forward direction and $\mathbf{W}_b$ for the backward direction) for the storage of even K vectors can be calculated as

$$\mathbf{W}_f = (\mathbf{X}_1)^T \mathbf{X}_2 + (\mathbf{X}_3)^T \mathbf{X}_4 + \cdots + (\mathbf{X}_{K-1})^T \mathbf{X}_K \quad (5.5.5)$$

where

$$\mathbf{X}_{2k} = G_h\{\mathbf{X}_{2k-1}\mathbf{W}_f\} \text{ for } k = 1, 2, \dots, K/2 \quad (5.5.6)$$

and

$$\mathbf{W}_b = (\mathbf{X}_2)^T \mathbf{X}_3 + (\mathbf{X}_4)^T \mathbf{X}_5 + \dots + (\mathbf{X}_K)^T \mathbf{X}_1 \quad (5.5.7)$$

Where

$$\mathbf{X}_1 = G_h\{\mathbf{X}_K \mathbf{W}_b\} \quad (5.5.8)$$

$$\mathbf{X}_{2k+1} = G_h\{\mathbf{X}_{2k} \mathbf{W}_b\} \text{ for } k = 1, 2, \dots, (K/2 - 1) \quad (5.5.9)$$

For recalls, the bipolar hardlimit function defined in (5.2.4) is used in (5.5.6), (5.5.8)-(5.5.9).

In order to store S (>2) such sequences of temporal associative patterns, $(\mathbf{X}_{j1}, \mathbf{X}_{j2}, \dots, \mathbf{X}_{js}, \dots, \mathbf{X}_{jS})$, where $\mathbf{X}_{js}$ is one set of temporal pattern with element row vectors of $(\mathbf{X}_{1,s}, \mathbf{X}_{2,s}, \dots, \mathbf{X}_{K,s})$, we can modify the weight matrices by the following equations as

$$\mathbf{W}_f = \sum_{s=1}^S [(\mathbf{X}_{1,s})^T \mathbf{X}_{2,s} + (\mathbf{X}_{3,s})^T \mathbf{X}_{4,s} + \dots + (\mathbf{X}_{K-1,s})^T \mathbf{X}_{K,s}] \quad (5.5.10)$$

$$\mathbf{W}_b = \sum_{s=1}^S [(\mathbf{X}_{2,s})^T \mathbf{X}_{3,s} + (\mathbf{X}_{4,s})^T \mathbf{X}_{5,s} + \dots + (\mathbf{X}_{K,s})^T \mathbf{X}_{1,s}] \quad (5.5.11)$$

Hence, we can store a number of temporal associative patterns using this method.

Since we distinguish the two directions in recalling, the same data can be used when one is in odd position and the other is in even position. For example, we can now store $(\mathbf{X}_1, \mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3)$ because according to (5.5.10)-(5.5.11), the BAM is asymmetric such that $\mathbf{W}_b \neq (\mathbf{W}_f)^T$.

However, in this method, the length of temporal associative patterns is still limited by the storage capacity of the BAM.

In order to overcome this storage capacity problem, the CBAM and the RUAM described below should be used.

5.5.3 Cascade bidirectional associative memories

A CBAM [Kwa8909b] is constructed by connecting several BAMs to form a TAM as shown in Figure 5.5.2.

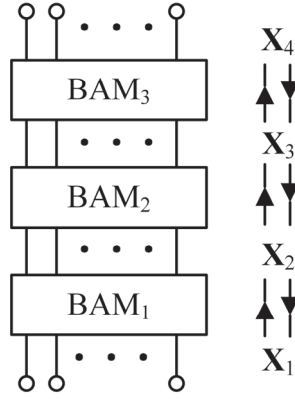


Figure 5.5.2 Temporal associative memory using CBAM.

Given the s th sequence of temporal associative patterns $(\mathbf{X}_{1,s}, \mathbf{X}_{2,s}, \dots, \mathbf{X}_{K,s})$ for $s = 1, 2, \dots, S$, we can use $K - 1$ BAMs to respectively store pattern-pairs $(\mathbf{X}_{1,s}, \mathbf{X}_{2,s}), (\mathbf{X}_{2,s}, \mathbf{X}_{3,s}), \dots, (\mathbf{X}_{K-1,s}, \mathbf{X}_{K,s})$ for $s = 1, 2, \dots, S$. The $K - 1$ weight matrices are calculated by

$$\mathbf{W}_1 = (\mathbf{X}_{1,1})^T \mathbf{X}_{2,1} + (\mathbf{X}_{1,2})^T \mathbf{X}_{2,2} + \dots + (\mathbf{X}_{1,S})^T \mathbf{X}_{2,S} \quad (5.5.12)$$

$$\mathbf{W}_2 = (\mathbf{X}_{2,1})^T \mathbf{X}_{3,1} + (\mathbf{X}_{2,2})^T \mathbf{X}_{3,2} + \dots + (\mathbf{X}_{2,S})^T \mathbf{X}_{3,S} \quad (5.5.13)$$

$$\vdots$$

$$\mathbf{W}_{K-1} = (\mathbf{X}_{K-1,1})^T \mathbf{X}_{K,1} + (\mathbf{X}_{K-1,2})^T \mathbf{X}_{K,2} + \dots + (\mathbf{X}_{K-1,S})^T \mathbf{X}_{K,S} \quad (5.5.14)$$

The recalling procedure is that firstly, vector $\mathbf{X}_{1,s}$ is input into BAM_1 and $\mathbf{X}_{2,s} (= G_h\{\mathbf{X}_{1,s} \mathbf{W}_1\})$ will be recalled. $\mathbf{X}_{2,s}$ will recall $\mathbf{X}_{3,s} (= G_h\{\mathbf{X}_{2,s} \mathbf{W}_2\})$ through BAM_2 at the next iteration and so on until $\mathbf{X}_{K,s} (= G_h\{\mathbf{X}_{K-1,s} \mathbf{W}_{K-1}\})$ through BAM_{K-1} is recalled.

Then this vector $\mathbf{X}_{K,s}$ will be passed back to BAM_{K-1} so that $\mathbf{X}_{K-1,s}$ ($= G_h\{\mathbf{X}_{K,s}(\mathbf{W}_1)^T\}$) through BAM_{K-1} will be renewed. This renewal will be carried on until $\mathbf{X}_{1,s}$ ($= G_h\{\mathbf{X}_{2,s}(\mathbf{W}_1)^T\}$) BAM_1 is renewed.

Then the above procedure is repeated until all neuron states are stable. Note that one complete loop of recalls involves six vector and matrix multiplications and also, the bipolar hardlimit function defined by (5.2.4) is used for recalls.

5.5.4 Ring unidirectional associative memories

A RUAM [Kwa8909b], [Kwa9205] is constructed by connecting several unidirectional associative memories (UAMs) in a ring and it can do the same job as the CBAM.

A UAM is a BAM in which the signal flow is restricted to be in a fixed direction and is not allowed in the reversed direction.

Here, the number of UAMs used is K instead of $K - 1$ as for the case of the CBAM. The structure of a RUAM is shown in Figure 5.5.3.

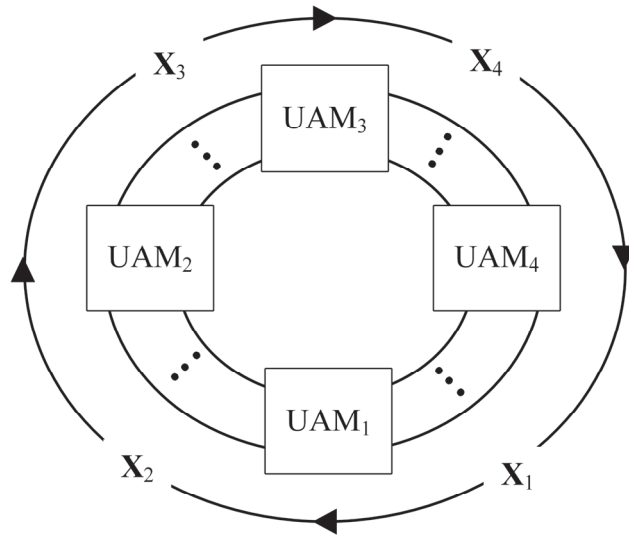


Figure 5.5.3 Temporal associative memory using RUAM.

The weight matrices are the same as in (5.5.12)-(5.5.14) but in addition there is one more weight matrix $\mathbf{W}_K$ for the storing of $(\mathbf{X}_{K,S}, \mathbf{X}_{1,S})$, hence,

$$\mathbf{W}_K = (\mathbf{X}_{K,1})^T \mathbf{X}_{1,1} + (\mathbf{X}_{K,2})^T \mathbf{X}_{1,2} + \cdots + (\mathbf{X}_{K,S})^T \mathbf{X}_{1,S} \quad (5.5.15)$$

The recalling procedure is similar to that of the CBAM but when $\mathbf{X}_{K,S}$ has been recalled, it will recall $\mathbf{X}_{1,S}$ instead of passing backward to recall $\mathbf{X}_{K-1,S}$ as in the case of CBAM. The recalling is thus unidirectional. In practice, both forward and backward recalls can be used in a RUAM. Also, the bipolar hardlimit function defined by (5.2.4) is used for recalls.

5.5.5 Stability

According to [Kos8712], [Kos8801], every $N \times M$ matrix is bidirectionally stable heteroassociative content addressable memory with binary, bipolar, and continuous neurons.

Since the classical BAM is consisted of a single BAM, each of the CBAM and the RUAM is consisted of, respectively, a number of BAMs and UAMs, each of these three networks will converge to a stable state if the local energy of each respective BAM or UAM is decreased after one iteration and this energy is bounded. A stability proof [Kwa8909b] is shown below.

An input to output pattern pair $(\mathbf{X}_{k-1}, \mathbf{X}_k)$ defines the state of a BAM. Suppose at time k , $\mathbf{X}_{k-1}$ is updated to $\mathbf{X}_k$ through a BAM_{k-1} , define the energy (or Lyapunov function) of a BAM (or UAM) as

$$E(\mathbf{X}_{k-1}, \mathbf{X}_k) = -\mathbf{X}_{k-1} \mathbf{W}_{k-1} (\mathbf{X}_k)^T \quad (5.5.16)$$

where $\mathbf{X}_{k-1}$ is the input vector and $\mathbf{X}_k$ is the output vector.

At time k , the change of energy $\Delta E(\mathbf{X}_{k-1}, \mathbf{X}_k)$ of this network due to a change of $\Delta \mathbf{X}_k = \mathbf{X}_k - \mathbf{X}_{k-1}$ is

$$\begin{aligned} \Delta E(\mathbf{X}_{k-1}, \mathbf{X}_k) &= E(\mathbf{X}_{k-1}, \mathbf{X}_k) - E(\mathbf{X}_{k-1}, \mathbf{X}_{k-1}) \\ &= -\mathbf{X}_{k-1} \mathbf{W}_{k-1} (\mathbf{X}_k)^T + \mathbf{X}_{k-1} \mathbf{W}_{k-1} (\mathbf{X}_{k-1})^T \end{aligned}$$

$$= -\mathbf{X}_{k-1} \mathbf{W}_{k-1} (\mathbf{X}_k - \mathbf{X}_{k-1})^T \quad (5.5.17)$$

Since $\mathbf{X}_{k-1} \mathbf{W}_{k-1} = \mathbf{X}_k$, (5.5.17) becomes

$$\Delta E(\mathbf{X}_{k-1}, \mathbf{X}_k) = -\mathbf{X}_k (\mathbf{X}_k - \mathbf{X}_{k-1})^T = -\mathbf{X}_k (\mathbf{X}_k)^T + \mathbf{X}_k (\mathbf{X}_{k-1})^T \leq 0 \quad (5.5.18)$$

In (5.5.18), for bipolar vectors, the first term $-\mathbf{X}_k (\mathbf{X}_k)^T = -K^2$ but the second term $\mathbf{X}_k (\mathbf{X}_{k-1})^T \leq K^2$, hence $\Delta E(\mathbf{X}_{k-1}, \mathbf{X}_k) \leq 0$. Thus, the local energy is always decreasing.

For bipolar $\mathbf{X}_{k-1}$ and $\mathbf{X}_k$, since $\mathbf{X}_{k-1} \mathbf{W}_{k-1} = \mathbf{X}_k$, from (5.5.17), the energy $E(\mathbf{X}_{k-1}, \mathbf{X}_k)$ is actually bounded by

$$E(\mathbf{X}_{k-1}, \mathbf{X}_k) = -\mathbf{X}_{k-1} \mathbf{W}_{k-1} (\mathbf{X}_k)^T = -\mathbf{X}_k (\mathbf{X}_k)^T = -K^2 \quad (5.5.19)$$

Based on (5.5.17)-(5.5.18), a similar energy decreasing argument applies to each of the associative memory networks consisting of binary or continuous neurons, thus an associative memory network will always converge to a stable state.

5.5.6 Pattern partitioning

One can use several AMs to store patterns by pattern partitioning. Suppose we want to store patterns of size $n \times m$ pixels, we can first partition each pattern into m “ n -pixel” patterns.

These patterns can be stored in m UAMs with each UAM consisting of n neurons at its input layer and n neurons at its output layer and these m UAMs can be connected as a RUAM as shown in Figure 5.5.3.

For a RUAM, each UAM actually has only n neurons since the output neurons of one UAM is the same as the input neurons in the next UAM.

In general, it is possible to have different numbers of input and output neurons in each UAM so long as the number of input neurons of the next UAM and the number of output neurons of the previous UAM are the same.

Thus, we can recall the whole pattern using any of the m “ n -pixel” patterns. For example, if we want to store 6×9 patterns $\{x_{ij}\}$, where

$i = 1, 2, \dots, 6$ and $j = 1, 2, \dots, 9$, we can store them as $(\mathbf{X}_{i1}, \mathbf{X}_{i2}), (\mathbf{X}_{i2}, \mathbf{X}_{i3}), (\mathbf{X}_{i3}, \mathbf{X}_{i4}), (\mathbf{X}_{i4}, \mathbf{X}_{i5}), (\mathbf{X}_{i5}, \mathbf{X}_{i6}), (\mathbf{X}_{i6}, \mathbf{X}_{i7}), (\mathbf{X}_{i7}, \mathbf{X}_{i8}), (\mathbf{X}_{i8}, \mathbf{X}_{i9}), (\mathbf{X}_{i9}, \mathbf{X}_{i1})$ in 9 UAMs.

This partitioning approach for RUAM has an advantage that the total number of summations for recalling is about m times smaller as compared to that of the one-layer Hopfield network.

In each UAM of the RUAM, each of the n neurons is connected to $n - 1$ neurons, there are $n - 1$ summations and the total number of summations is $(m \times n) \times (n - 1)$. For the one-layer Hopfield network, each of the $n \times m$ neurons is connected to $n \times m - 1$ neurons and so the total number of summations is $(m \times n) \times (n \times m - 1)$.

However, in the RUAM, we cannot have the identical row vectors among different patterns as the input or as the output in one UAM.

For example, if we want to store 4×2 $\mathbf{X}_1 = (1, -1; \mathbf{1}, -\mathbf{1}; 1, 1; 1, 1)$ and 4×2 $\mathbf{X}_2 = (1, -1; -\mathbf{1}, \mathbf{1}; -1, 1; 1, -1)$, we cannot partition each of these two vectors into four sub-vectors, each having two elements. It is because the network cannot recall both $(1, -1)$ and $(-1, 1)$ from $(1, -1)$ in the first UAM.

In this case, we could partition each of these two vectors into two sub-vectors each having four elements. Hence, in partitioning patterns in the RUAM, this design constraint should be included.

Design Example 5.3 (Learning, recall and comparisons of ABAM, CBAM, and RUAM) Matlab

Four pattern sequences each consisting of four alphabets are used as temporal associative patterns. The four alphabet sequences are “RING”, “WEAK”, “BOLD” and “THIN”. Each alphabet is represented by 54 (i.e., 6×9) pixels of ± 1 bipolar values.

For the ABAM, only the first two-word sequences (i.e., “RING” and “WEAK”) are to be stored as it is limited by its storage capacity. For the CBAM and the RUAM, all the four-word sequences are to be stored.

Design, recall, and compare the performances of ABAM, CBAM, and RUAM as temporal associative memories.

Solution

To check the performance of each designed network under noise conditions, the pixels of the first alphabet (R, W, B, T) of each of the four four-word sequences corrupted with noise of different error probabilities, P_e (0.1, 0.3, 0.5, 0.7, 0.9), are used to form the input into each of the three networks.

The recall results are summarized in Table E5.3.1 and Table E5.3.2. In Table E5.3.1, all the ABAM, the CBAM, and the RUAM stored only the first two-word sequences (i.e., “RING” and “WEAK”).

In Table E5.3.2, both the CBAM and the RUAM stored all the four-word sequences.

In Tables E5.3.1 and E5.3.2, “ x ” means incorrect recalls, “ c ” means complement recalls of the desired words, “1” means that only one loop ($\mathbf{X}_1 \rightarrow \mathbf{X}_2 \rightarrow \dots \rightarrow \mathbf{X}_L$) was required for recalling the desired words while “2” means that two loops were required for recalling the desired words.

For each P_e , 60 trials were done to calculate the four individual probabilities of occurrences of “1”, “2”, “ c ” and “ x ” with a total probability of 1.

Table E5.3.1 Recall results for storing two-word sequences.

	ABAM					CBAM					RUAM				
P_e	0.1	0.3	0.5	0.7	0.9	0.1	0.3	0.5	0.7	0.9	0.1	0.3	0.5	0.7	0.9
1	.97	.70	0	0	0	1	.92	.18	0	0	1	.89	.15	0	0
2	.03	.15	.03	0	0	0	0	0	0	0	0	.03	.03	0	0
c	0	0	.22	.78	1	0	0	.47	.97	1	0	0	.47	.97	1
x	0	.15	.75	.22	0	0	.08	.35	.03	0	0	.08	.35	.03	0

Table E5.3.2 Recall results for storing four-word sequences.

	CBAM					RUAM				
Pe	0.1	0.3	0.5	0.7	0.9	0.1	0.3	0.5	0.7	0.9
1	1	.78	.08	0	0	.93	.46	.03	0	0
2	0	.05	.03	0	0	.07	.37	.08	0	0
c	0	0	.08	.87	1	0	0	.08	.87	1
x	0	.17	.81	.13	0	0	.17	.81	.13	0

From Tables E5.3.1 and E5.3.2, it can be noted that all the three networks performed well in recalls with 0.1 error probability but they performed worst in recalls with 0.5 error probability.

It is because with 0.5 error probability, all the stable states will have the same probability to be recalled.

With error probability greater than 0.5, the complements of the patterns will most likely be recalled as the Hamming distance of the corrupted input patterns will be closer to their complement patterns.

Comparing the three networks based on Table E5.3.1, we can observe that the ABAM based on one single BAM has the lowest correct recalls.

It is because the storage capacity is likely reached first in the case of the ABAM based on one single BAM while the storage capacity is likely not yet reached in the case of the CBAM (or the RUAM) based on multiple BAMs (or UAMs).

From Table E5.3.1 and Table E5.3.2, we can observe that the recall performances of the CBAM and the RUAM are similar but the CBAM requires reduced number of loops for recalls.

This is because the number of iterations in one loop is the highest in the CBAM (six iterations per loop are required for the CBAM but only four iterations per loop are required for the ABAM and the RUAM).

If we normalize the error probability by the number of iterations in one loop, then the RUAM is better than the CBAM for 0.1 error probability.

In practice, unidirectional recalls in the RUAM is better than bidirectional recalls in the CBAM since unidirectional recalls use a direct path with all vectors recalled evenly and reduced number of additions and multiplications (for storing four words, the CBAM requires six layers of summation of products of vector and matrix for recalls while the RUAM only requires four layers of summation of products of vector and matrix for recalls).

Finally, since different BAMs (or UAMs) are used to store the pattern-pairs of a temporal pattern in the CBAM (or the RUAM), a sequence of patterns which consists of two or more identical row vectors are stored in two or more different BAMs (or UAMs).

This enables both the CBAM and the RUAM to recall identical row vectors in two or more different patterns of a temporal sequence which cannot be otherwise be performed by the ABAM using one single BAM.

5.5.7 Summary

The CBAM and the RUAM are constructed, respectively, by connecting a number of BAMs and UAMs (a UAM is equivalent to a BAM with only one direction for signal flow), their storage capacities are proportional to the number of AMs used as well as the storage capacity of each AM.

However, since sequences of patterns are stored in these memories, the storage capacity will be defined as the maximum number of sequences that can be stored, instead of the maximum number of pattern pairs that can be stored in each AM.

Three methods to store temporal patterns have been described and explained, and their performances have also been compared.

The CBAM and the RUAM have similar performance. However, if the number of iterations in one loop and the hardware complexity are taken into account, the RUAM is more efficient.

The ABAM based on one single BAM has two limitations. The first one is a limitation on the pattern length due to the storage capacity and the second one is that identical row vectors is not allowed in either odd positions or even positions of a sequence of patterns.

5.5.8 BP-trained ring unidirectional associative memories

In [Kwa9205], the RUAM trained by both Hebb rule and the backpropagation algorithm are described and their performances are compared.

In general, the Hebb rule is simple and fast for training orthogonal pattern sequences.

The backpropagation algorithm has a stronger noisy pattern sequence recall capability and a larger storage capacity than those of the Hebb rule when used to train a RUAM for storing non-orthogonal pattern sequences. Details are described in [Kwa9205].

5.6 Multilayer Symmetrical and Asymmetrical Associative Memories

In this section, three-layer bidirectional symmetrical and asymmetrical associative memories [Kwa8909a], [Kwa0208a], [Kwa0305a], [Kwa0505] are described and a least mean square (LMS) type of delta rule learning algorithm is used for training the networks.

The networks possess desired structural features of a bidirectional associative memory and a three-layer feedforward neural network and are able to learn and recall bipolar alpha-numeral patterns and gray level images.

The network allows two modes of memory recall, namely, recalling by a pattern-pair from both the input and the output layers, and recalling by single-pattern from either the input layer or the output layer.

Simulation results on alpha-numeral and face image types of pattern learning and recall problems are presented.

5.6.1 Network Structures

As shown in Figures 5.6.1 and 5.6.2, the three-layer bidirectional symmetrical (or asymmetrical) associative memory consists of three layers, from the layer 0 to the layer 2.

The layer 0 is the input layer and the layer 2 is the output layer. The layer k has N_k neurons for $k = 0, 1, 2$. The layer 1 is connected with the layer 2 above and the layer 0 below through bidirectional symmetrical (or asymmetrical) interconnections.

Define the output vector of the layer k as

$$\mathbf{O}^{[k]} = G[\mathbf{Z}^{[k]}] \text{ for } k = 0 \text{ to } 2 \quad (5.6.1)$$

where $\mathbf{Z}^{[k]}$ is the linear output vector of the layer k and $G[.]$ represents the operation of applying nonlinear activation function.

5.6.1.1 Symmetrical network

For the layer 1,

$$\mathbf{Z}^{[1]} = \mathbf{O}^{[0]} \cdot \mathbf{W}^{[1]} + \mathbf{O}^{[2]} \cdot [\mathbf{W}^{[2]}]^T \quad (5.6.2)$$

For the layer 2,

$$\mathbf{Z}^{[2]} = \mathbf{O}^{[1]} \cdot \mathbf{W}^{[2]} \quad (5.6.3)$$

For the layer 0,

$$\mathbf{Z}^{[0]} = \mathbf{O}^{[1]} \cdot [\mathbf{W}^{[1]}]^T \quad (5.6.4)$$

The superscript T represents the transpose of a matrix. $\mathbf{W}^{[k]}$ represents the N_{k-1} by N_k feedforward weight connections from the layer $(k - 1)$ to the layer k for $k = 1$ and 2 .

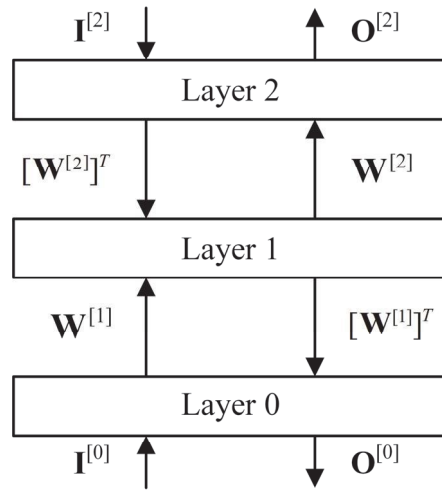


Figure 5.6.1 Three-layer bidirectional symmetrical associative memories.

5.6.1.2 Asymmetrical network

In an asymmetrical network, the two feedbackward weights $[\mathbf{W}^{[k]}]^T$ are different from the two feedforward weights $\mathbf{W}^{[k]}$ for $k = 1$ and 2 as shown below.

For the layer 1,

$$\mathbf{Z}^{[1]} = \mathbf{O}^{[0]} \mathbf{W}^{[1]} + \mathbf{O}^{[2]} [\mathbf{W}^{[4]}]^T \quad (5.6.5)$$

For the layer 2,

$$\mathbf{Z}^{[2]} = \mathbf{O}^{[1]} \mathbf{W}^{[2]} \quad (5.6.6)$$

For the layer 0,

$$\mathbf{Z}^{[0]} = \mathbf{O}^{[1]} [\mathbf{W}^{[3]}]^T \quad (5.6.7)$$

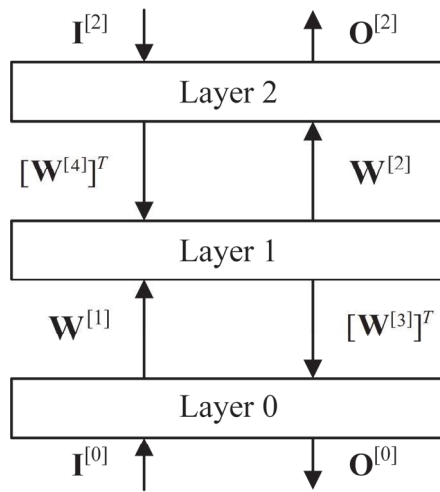


Figure 5.6.2 Three-layer bidirectional asymmetrical associative memories.

5.6.2 Learning algorithm

A key issue in the training of a multilayer neural network is the absence of desired outputs in its hidden layers. In the training of both the symmetrical and asymmetrical networks, this issue is solved by adopting pre-determined patterns as the desired output patterns in the hidden layer.

The training of each network is consisted of two phases. In the first phase, the hidden layer accepts the input patterns from the layers 0 and 2 and computes its output pattern. The output errors in the hidden layer are used to adaptively change all the interconnecting weights from the layers 0 and 2.

In the second phase, each of the layers 0 and 2 accepts the input pattern from the hidden layer and computes its output pattern. In each of the layers 0 and 2, the output errors are used to adaptively changing all the interconnecting weights from the hidden layer.

5.6.2.1 Symmetrical network

The delta rule is used in the training. Define the error vector at a layer k (for $k = 0, 1, 2$), $\mathbf{E}^{[k]}$, as the difference between the desired output vector, $\mathbf{O}_d^{[k]}$, and the obtained output vector, $\mathbf{O}_o^{[k]}$ as

$$\mathbf{E}^{[k]} = \mathbf{O}_d^{[k]} - \mathbf{O}_o^{[k]} \quad (5.6.8)$$

For the feedbackward weight connections, the change in the $N_k \times N_{k-1}$ weight matrix, $[\Delta \mathbf{W}^{[k]}]^T$, is defined by

$$[\Delta \mathbf{W}^{[k]}]^T = \alpha [\mathbf{O}_d^{[k]}]^T \mathbf{E}^{[k-1]} \text{ for } k = 1, 2 \quad (5.6.9)$$

α is the learning rate parameter; $\mathbf{O}_d^{[2]}$ can be any of the P output training patterns $\mathbf{O}_p$ and $\mathbf{O}_d^{[1]}$ can be any of the desired hidden patterns.

For the feedforward weight connections, the change of the $N_{k-1} \times N_k$ weight matrix, $\Delta \mathbf{W}^{[k]}$, is defined by

$$\Delta \mathbf{W}^{[k]} = \alpha [\mathbf{O}_d^{[k-1]}]^T \mathbf{E}^{[k]} \text{ for } k = 1, 2 \quad (5.6.10)$$

$\mathbf{O}_d^{[0]}$ can be any of the P input training patterns $\mathbf{I}_p$ corresponding to $\mathbf{O}_p$ and $\mathbf{O}_d^{[1]}$ can be any of the desired hidden patterns.

5.6.2.2 Asymmetrical network

In the first phase of learning, the error vector, $\mathbf{E}^{[1]}$, at the hidden layer is

$$\mathbf{E}^{[1]} = \mathbf{O}_d^{[1]} - \mathbf{O}_o^{[1]} \quad (5.6.11)$$

The change of the $N_0 \times N_1$ weight matrix, $\Delta \mathbf{W}^{[1]}$, is calculated by

$$\Delta \mathbf{W}^{[1]} = \alpha [\mathbf{I}_p]^T \mathbf{E}^{[1]} \quad (5.6.12)$$

The change of the $N_1 \times N_2$ weight matrix, $[\Delta \mathbf{W}^{[4]}]^T$, is calculated by

$$[\Delta \mathbf{W}^{[4]}]^T = \alpha [\mathbf{O}_p]^T \mathbf{E}^{[1]} \quad (5.6.13)$$

In the second phase of learning, the error vector, $\mathbf{E}^{[0]}$, at the layer 0 is

$$\mathbf{E}^{[0]} = \mathbf{I}_p - \mathbf{O}_o^{[0]} \quad (5.6.14)$$

The change of the $N_0 \times N_1$ weight matrix, $[\Delta \mathbf{W}^{[3]}]^T$, is calculated by

$$[\Delta \mathbf{W}^{[3]}]^T = \alpha [\mathbf{O}_d^{[1]}]^T \mathbf{E}^{[0]} \quad (5.6.15)$$

The error vector, $\mathbf{E}^{[2]}$, at the layer 2 is

$$\mathbf{E}^{[2]} = \mathbf{O}_p - \mathbf{O}_o^{[2]} \quad (5.6.16)$$

The change of the $N_1 \times N_2$ weight matrix, $\Delta \mathbf{W}^{[2]}$, is calculated by

$$\Delta \mathbf{W}^{[2]} = \alpha [\mathbf{O}_d^{[1]}]^T \mathbf{E}^{[2]} \quad (5.6.17)$$

In the symmetrical (or asymmetrical) network, given a set of P input to output pattern-pairs for training, the first pattern-pair is first presented to the inputs of the layers 0 and 2 and compute the weight changes according to (5.6.8)-(5.6.10) or (5.6.11)-(5.6.17) for asymmetrical network), from which weight-updates are carried out accordingly.

The whole procedure is repeated for the next pattern-pair presentation until all the P pattern-pairs have been presented and this forms an epoch of training.

More epochs are carried out in a repeated manner. The training process is terminated when all the obtained output patterns of a network converge to their corresponding desired training patterns.

5.6.3 Memory recalls

A recall iteration is consisted of two phases. In the first phase, the layer 1 accepts the input patterns from the layers 0 and 2 and compute an output pattern. In the second phase, the layers 0 and 2 accept the input vector from the layer 1 and calculate their output vectors.

Before each recall of a test pattern, the outputs of all the neurons, except those of the input neurons, should be set to zero because the recalled pattern depends on the initial conditions of each of the networks.

In both the symmetrical and asymmetrical networks, two modes of memory recall, namely single pattern recall and pattern pair recall, are allowed.

In the single pattern recall, only one test pattern is applied to either the layer 0 or the layer 2 input of the network. The input of the remaining layer 0 or layer 2 input of a network is set to zero.

In pattern-pair recall, a pair of test patterns is applied to the inputs of the layers 0 and 2 of a network.

Weight-updates can be carried out either at the end of each pattern-pair training or at the end of each epoch. In this simulation, weight-updates are carried out at the end of each pattern-pair training.

If the two networks are being used as an autoassociator, for the case of the symmetrical network, $\mathbf{W}^{[1]} = [\mathbf{W}^{[2]}]^T$, for the case of the asymmetrical network, $\mathbf{W}^{[1]} = [\mathbf{W}^{[4]}]^T$ and $\mathbf{W}^{[2]} = [\mathbf{W}^{[3]}]^T$.

If the two networks are being used as a pattern associator, this symmetrical properties of weight matrices do not exist.

Design Example 5.4 (Symmetrical and asymmetrical associative memories for alpha-numeral pattern problem) Matlab

Design and show the performances of the three-layer bidirectional symmetrical and asymmetrical associative memories as an autoassociator and as a pattern associator.

In the case of the autoassociator, the network is required to learn and recall the mapping of each of the 0 to 9 numerals at the layer 0 to the corresponding 0 to 9 numerals at the layer 2.

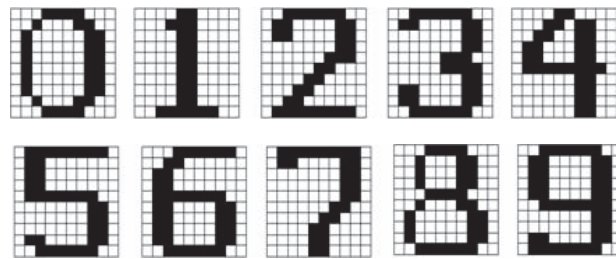


Figure E5.4.1 0 to 9 numerals of 10x10 bipolar pixels.

The number of ± 1 pixels of each of the 0 to 9 numerals is 10x10 and the 10 numerals are displayed as shown in Figure E5.4.1.

In the case of the pattern associator, the network is used to learn and recall the mapping of each of the 26 small alphabets a to z at the layer 0 to each of the corresponding 26 capital alphabets A to Z at the layer 2.

The number of pixels of each of the 26 small and 26 capital alphabets is 16x16 as displayed in Figures E5.4.2 and E5.4.3.

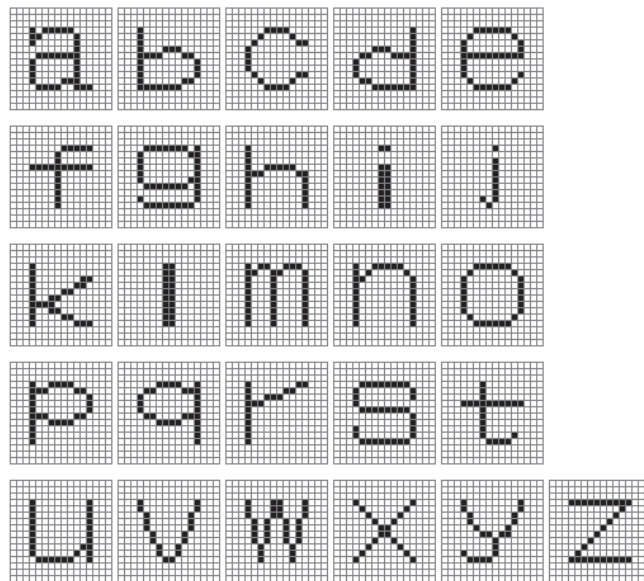


Figure E5.4.2 Samples of 26 small alphabets of 16x16 bipolar pixels.

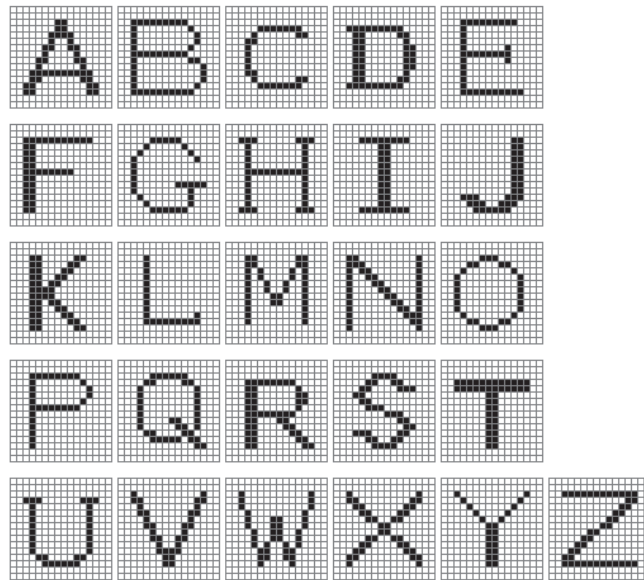


Figure E5.4.3 Samples of 26 capital alphabets of 16x16 bipolar pixels.

Solution

In both cases, the memories are trained with desired patterns at the layers 0 and 2. Orthogonal vectors are used as the desired patterns in the layer 1. The number of interconnected weights of the symmetrical network is half of that of the asymmetrical network if the same number of hidden neurons are used.

After the training, the recall capability of each network is tested using one hundred noisy versions of each pattern. A noisy version of a pattern is constructed by inverting a specified percentage, P_e (from 5% to 45%), of its randomly selected elements. The maximum number of allowed iterations for the convergence of each recall is 10.

The recognition results of both the three-layer symmetrical and asymmetrical networks with different specified numbers of hidden neurons in the forms of generalization capability and average number of iterations for reaching convergence are summarized in Tables E5.4.1-E5.4.9.

Each of Tables E5.4.1-E5.4.9 shows the results of pattern-pair recalls from both the layers 0 and 2 and from each of the single-pattern recalls. In general, the results of single-pattern recalls in terms of recall accuracy and number of iterations are slightly inferior to those of the pattern-pair recalls.

Tables E5.4.1-E5.4.3 summarizes the results of the symmetrical network (32 hidden neurons) and the asymmetrical network (16 hidden neurons) as the autoassociator of learning and recalls of the 0 to 9 numerals.

Table E5.4.4-E5.4.6 summarizes the results of the symmetrical network (64 hidden neurons) and the asymmetrical network (32 hidden neurons) as the pattern associator of learning and recalls of the a-z to A-Z alphabet-pairs.

Table E5.4.7-E5.4.9 summarizes the corresponding results of the asymmetrical network as both the autoassociator and the pattern associator with 12 and 28 hidden neurons, respectively.

Table E5.4.1 Autoassociator: Pattern-pair recalls.

Pe	Symmetrical Network (32 hidden neurons)		Asymmetrical Network (16 hidden neurons)	
	Recall (%)	Iterations	Recall (%)	Iterations
0%	100.00	1.00	100.00	1.00
5%	99.70	1.00	100.00	1.00
10%	98.70	1.03	100.00	1.00
15%	96.80	1.08	100.00	1.00
20%	91.70	1.18	100.00	1.02
25%	86.80	1.29	99.50	1.14
30%	78.30	1.48	97.90	1.41
35%	63.40	1.77	90.00	2.03
40%	37.20	2.33	63.10	2.85
45%	8.00	3.33	14.20	4.44

Table E5.4.2 Autoassociator: Single-pattern recall from layer 0.

Pe	Symmetrical Network (32 hidden neurons)		Asymmetrical Network (16 hidden neurons)	
	Recall (%)	Iterations	Recall (%)	Iterations
0%	100.00	1.00	100.00	1.00
5%	100.00	1.00	100.00	1.00
10%	99.90	1.00	100.00	1.01
15%	99.40	1.01	99.90	1.08
20%	98.80	1.02	99.10	1.29
25%	97.80	1.02	97.20	1.56
30%	96.40	1.06	90.80	2.06
35%	95.50	1.06	68.30	2.62
40%	93.30	1.11	35.20	3.55
45%	92.90	1.12	5.60	4.98

Table E5.4.3 Autoassociator: Single-pattern recall from layer 2.

Pe	Symmetrical Network (32 hidden neurons)		Asymmetrical Network (16 hidden neurons)	
	Recall (%)	Iterations	Recall (%)	Iterations
0%	100.00	1.00	100.00	1.00
5%	99.10	1.01	100.00	1.00
10%	96.00	1.09	100.00	1.01
15%	92.50	1.18	100.00	1.06
20%	85.10	1.36	99.50	1.27
25%	77.00	1.47	96.50	1.63
30%	63.80	1.84	89.40	2.04
35%	42.70	2.26	70.60	2.63
40%	18.30	2.91	38.60	3.55
45%	3.20	4.00	7.00	5.10

Table E5.4.4 Pattern associator: Pattern-pair recalls.

Pe	Symmetrical Network (64 hidden neurons)		Asymmetrical Network (32 hidden neurons)	
	Recall (%)	Iterations	Recall (%)	Iterations
0%	100.00	1.00	100.00	1.00
5%	100.00	1.00	100.00	1.00
10%	99.85	1.03	100.00	1.00
15%	99.19	1.17	100.00	1.04
20%	95.96	1.49	100.00	1.26
25%	88.42	1.94	99.85	1.74
30%	71.31	2.50	97.19	2.31
35%	43.04	3.24	81.62	3.27
40%	16.23	4.24	45.19	4.55
45%	2.08	5.11	8.35	6.24

Table E5.4.5 Pattern associator: Single-pattern recall from layer 0.

Pe	Symmetrical Network (64 hidden neurons)		Asymmetrical Network (32 hidden neurons)	
	Recall (%)	Iterations	Recall (%)	Iterations
0%	100.00	1.00	96.15	2.12
5%	100.00	1.00	95.58	2.35
10%	100.00	1.00	93.69	2.57
15%	100.00	1.00	90.81	2.83
20%	99.92	1.02	85.31	3.20
25%	99.46	1.04	74.54	3.65
30%	98.00	1.11	57.42	4.15
35%	95.38	1.21	36.50	4.85
40%	90.69	1.33	15.23	5.63
45%	84.42	1.49	3.81	6.93

Table E5.4.6 Pattern associator: Single-pattern recall from layer 2.

Pe	Symmetrical Network (64 hidden neurons)		Asymmetrical Network (32 hidden neurons)	
	Recall (%)	Iterations	Recall (%)	Iterations
0%	88.46	1.96	100.00	1.54
5%	87.19	2.19	99.96	1.69
10%	83.73	2.30	99.77	1.82
15%	78.23	2.48	99.31	2.02
20%	68.54	2.69	98.08	2.29
25%	54.04	2.99	93.96	2.75
30%	39.35	3.51	81.96	3.27
35%	19.81	4.13	57.58	4.15
40%	6.62	4.85	23.04	5.35
45%	0.46	6.08	3.31	6.99

Table E5.4.7 Asymmetrical network: Pattern-pair recalls.

Pe	Autoassociator (12 hidden neurons)		Pattern Associator (28 hidden neurons)	
	Recall (%)	Iterations	Recall (%)	Iterations
0%	100.00	1.00	100.00	1.00
5%	100.00	1.00	100.00	1.00
10%	100.00	1.00	100.00	1.00
15%	100.00	1.00	100.00	1.05
20%	99.90	1.02	100.00	1.24
25%	99.30	1.16	99.65	1.71
30%	97.60	1.44	96.46	2.31
35%	86.70	2.01	79.46	3.34
40%	57.90	2.84	38.58	4.82
45%	12.50	4.23	6.19	6.23

Table E5.4.8 Asymmetrical net: Single-pattern recall from layer 0.

Pe	Autoassociator (12 hidden neurons)		Pattern Associator (28 hidden neurons)	
	Recall (%)	Iterations	Recall (%)	Iterations
0%	100.00	1.00	96.15	2.00
5%	100.00	1.00	94.69	2.23
10%	100.00	1.00	92.04	2.45
15%	100.00	1.09	88.62	2.69
20%	99.50	1.31	81.31	3.08
25%	97.70	1.60	70.62	3.54
30%	87.90	2.06	53.46	4.21
35%	66.60	2.63	30.81	4.96
40%	33.00	3.45	12.77	5.85
45%	4.00	4.93	3.00	6.88

Table E5.4.9 Asymmetrical net: Single-pattern recall from layer 2.

Pe	Autoassociator (12 hidden neurons)		Pattern Associator (28 hidden neurons)	
	Recall (%)	Iterations	Recall (%)	Iterations
0%	100.00	1.00	100.00	1.46
5%	100.00	1.00	99.31	1.69
10%	100.00	1.01	99.46	1.90
15%	100.00	1.08	98.19	2.11
20%	99.40	1.28	95.38	2.39
25%	95.30	1.63	90.08	2.88
30%	87.70	2.04	74.35	3.57
35%	64.90	2.59	47.73	4.42
40%	33.40	3.36	17.92	5.62
45%	4.80	4.90	3.04	6.80

Design Example 5.5 (Symmetrical and asymmetrical associative memories for face image problem) Matlab

In this example, the face images of two males and two females as shown in Figures E5.5.1-E5.5.3 from the UMIST Face Database [Gra9800] are used to aid the design and test the performances of symmetrical and asymmetrical associative memories.

Besides the original set of face images, two additional sets of face images each consisting of two side-face images for each of the four persons are provided for training as shown in Figure E5.5.2 and for testing as shown in Figure E5.5.3. The format of the face images is of 8-bit gray level with 46x56 pixels.

Four orthogonal vectors of dimensions 1x16 were used as the ideal hidden vectors for all the three face images of each of the four persons.

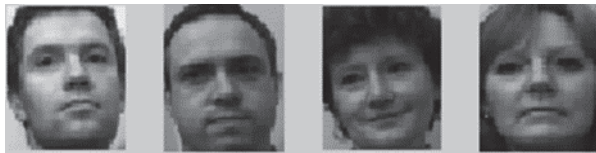


Figure E5.5.1 Four original front-face images.



Figure E5.5.2 Four sets of 2 side-face images for training.



Figure E5.5.3 Four sets of 2 side-face images for testing.

Solution

Each of the symmetrical network and the asymmetrical network is trained as a pattern associator to map each of the 4 front-face images as an ideal output to the corresponding two side-face images of the same person from the training set plus his/her front-face image.

In other words, each of 4 persons has 3 image-pairs for training. After training, each of the 2 side-face images of each person from the testing set is used for pattern-pair recalls.

After the design, the recall results are shown in Figure E5.5.4 for the symmetrical network and in Figure E5.5.5 for the asymmetric network. The recall of each test-face image can be correctly converged within a few iterations to the original front-face image.



Figure E5.5.4 Symmetrical network recalls by test-face images of Figure E5.5.3.



Figure E5.5.5 Asymmetrical network recalls by test-face images of Figure E5.5.3.

5.6.4 Summary

The structure, learning, recall, and performance of a three-layer associative memory network, with bidirectional symmetrical or asymmetrical interconnections have been presented.

In general, with the same number of weights, the asymmetrical network requires fewer epochs to train.

As shown from the results obtained in Tables E5.4.1-E5.4.9, the asymmetrical network has better generalization and recognition performance than those of the corresponding symmetrical network due to a more flexible network structure.

Increasing the number of hidden neurons could enhance the noise immunity of these networks. The asymmetrical network requires two times the number of weights as the symmetrical network, but the number of interconnection weights is much less than that required by the BAM for $N_1 < N_0 N_2 (N_0 + N_2)$ as listed in Table E5.5.1.

Table E5.5.1 Numbers of weights of different associative memory networks.

			BAM	Symmetrical Network		Asymmetrical Network	
P	N_0	N_2	$N_0 \times N_2$	N_1	$N_1(N_0 + N_2)$	N_1	$2N_1(N_0 + N_2)$
10	100	100	10000	32	6400	16	6400
10	256	256	65536	64	32768	32	32768
12	2576	2576	6635776	16	82432	16	164864

Chapter 6

Self-Organizing Maps

6.1 Introduction

In the mammalian brain, the neurons have a spatial order which usually corresponds to some characteristic of perceived signals.

This ordering can be formed automatically and soon after a stimulus is perceived. Such ordering found in neurons is at least partially due to self-organization.

Self-organization of neurons allows different events of the same concept to be clustered at an orderly manner to facilitate classification process.

The complete process of self-organization consists of two distinct phases:

- a. Competitive and cooperative clustering: Formation of an activity cluster as demonstrated by localized responses in the area around the neuron that receives the largest initial stimulus.

The winning neuron determines the topological neighborhood of excited neurons realized through lateral interactions among a set of excited neurons.

- b. Adaptation of self-organizing map (SOM): Adaptive changes in the input weights of those neurons where activity was confined within the neighborhood.

These two distinct phases can be realized and explained in terms of lateral feedback weights and feedforward weights of a neural network.

During the learning process, lateral feedback weights are not modified as they are considered a part of the permanent structure of the network, while feedforward weights are modified to self-organize the network.

The SOM is widely applied to clustering problems and data exploration in engineering, natural sciences, industry, finance, and linguistics.

There are two distinct phases of operations in SOM, phase 1 involves the use of SOM (one-dimensional or two-dimensional) learning algorithms to organize neurons through a feedforward neural network to form memory clusters [Kob9009]; and phase 2 involves the use of the Mexican hat function through lateral interconnections to simulate competitive and cooperative clustering [Hay9400]. Starting from an external input, phase 1 operation begins and then followed by phase 2 operation.

The causes and effects of parameters on the operations of adaptive SOM are also described and explained.

6.2 Competitive and Cooperative Clustering through Feedback Connections

In this section, the mechanism of the second phase is analyzed. It will be shown that lateral feedbacks play a crucial role in this phase.

There are both anatomical and physiological evidences for the existence of lateral interactions among neurons.

This type of feedback in a single layer results in the formation of clusters (or bubbles of activity).

Clustering phenomenon can be interpreted as instability in a feedback network, which results in high activities in some neurons and cutoffs in other neurons.

If the feedback factor is less than a critical value, the feedback network will not operate under instability and bubble formation will not emerge.

Short range lateral feedback excitations exist which can reach laterally up to a radius of 50 to 100 micrometers.

This excitatory area is surrounded by an area of inhibitory action reaching up to 200 to 500 micrometers.

There exist weaker excitatory actions surrounding the inhibitory area

reaching to a radius of several centimeters.

The feedback function looks like the shape of a Mexican hat as shown in Figure 6.2.1, and for this reason, it is called the Mexican hat function.

6.2.1 Mexican hat function

A Gaussian function can be expressed as

$$f(x) = ae^{\frac{-(x-b)^2}{2c^2}} \quad (6.2.1)$$

where the parameters “ a ”, “ b ”, and “ c ” control respectively the amplitude, the center, and the width of the Gaussian function.

The Mexican hat wavelet is defined as the negative normalized second derivative of a Gaussian function.

A wavelet is a wave-like brief oscillation with an amplitude that begins at zero, increases, and then decreases back to zero (like electrocardiogram). Generally, a wavelet is purposefully crafted to have specific properties that make it useful in signal processing.

A mathematical form of the Mexican hat function can be constructed by modifying the Mexican hat wavelet.

By taking the negative normalized second derivative of (6.2.1), the Mexican hat wavelet can be obtained [See Problem 6.1] as

$$f''(x) = \frac{2a}{c^2} e^{\frac{-(x-b)^2}{2c^2}} \left\{ 1 - \frac{(x-b)^2}{c^2} \right\} \quad (6.2.2)$$

A firing neuron tends to excite neurons in its immediate neighborhood more than neurons which are farther away from it and also it tends to exert inhibition upon neurons surrounding the most excited neurons.

The Mexican hat wavelet can further be modified by adding a small positive value d representing a small positive excitation in area surrounding the most excited neurons and the surrounding inhibitory neurons to give a closed-form mathematical expression for the one-dimensional Mexican hat function $h(x)$ as shown in (6.2.3).

$$h(x) = d - \frac{2a}{c^2} e^{\frac{-(x-b)^2}{2c^2}} \left\{ 1 - \frac{(x-b)^2}{c^2} \right\} \quad (6.2.3)$$

The one-dimensional Mexican hat function in (6.2.3) can be generalized to form a two-dimensional Mexican hat function $h(x, y)$ with inputs x and y as in (6.2.4).

$$h(x, y) = d - \frac{2a}{c^2} e^{\frac{-[(x-b)^2 + (y-b)^2]}{2c^2}} \left\{ 1 - \left[\frac{(x-b)^2}{c^2} + \frac{(y-b)^2}{c^2} \right] \right\} \quad (6.2.4)$$

A plot of (6.2.3)-(6.2.4) are shown respectively in Figures 6.2.1-6.2.2.

In (6.2.3)-(6.2.4), “ a ” controls the amplitude of the cone; “ b ” controls the location of the cone; “ c ” controls the width of the cone; and “ d ” specifies the level of weak excitatory region.

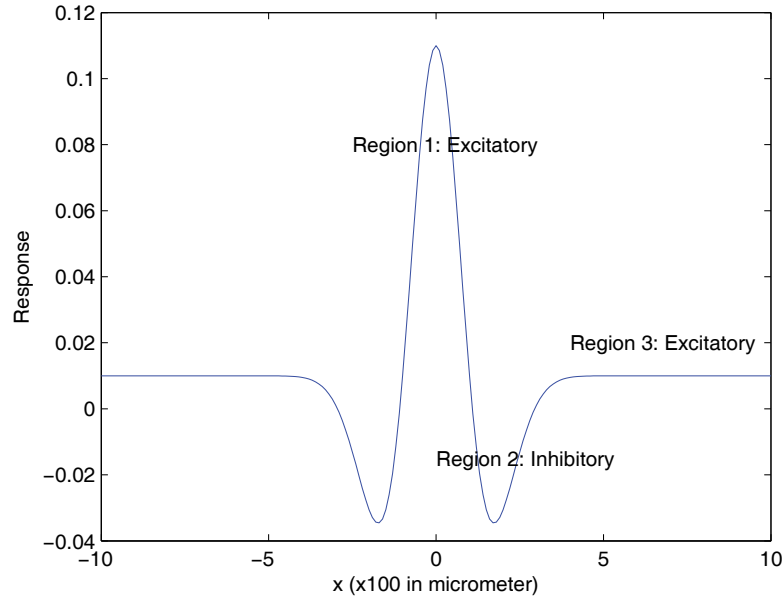


Figure 6.2.1 One-dimensional Mexican hat function ($a = -0.05$; $b = 0$; $c = 1$; $d = 0.01$).

As shown in Figures 6.2.1-6.2.2, the tip of the Mexican hat function represents the location of the winning neuron.

The cone at the center represents the winning neuron's excitatory influence on its immediate neighboring neurons located in the region 1, followed by its inhibitory influence on the neurons located in the region 2 surrounding the central excitatory region, and finally by its weak excitatory influence on further away neurons located in the region 3.

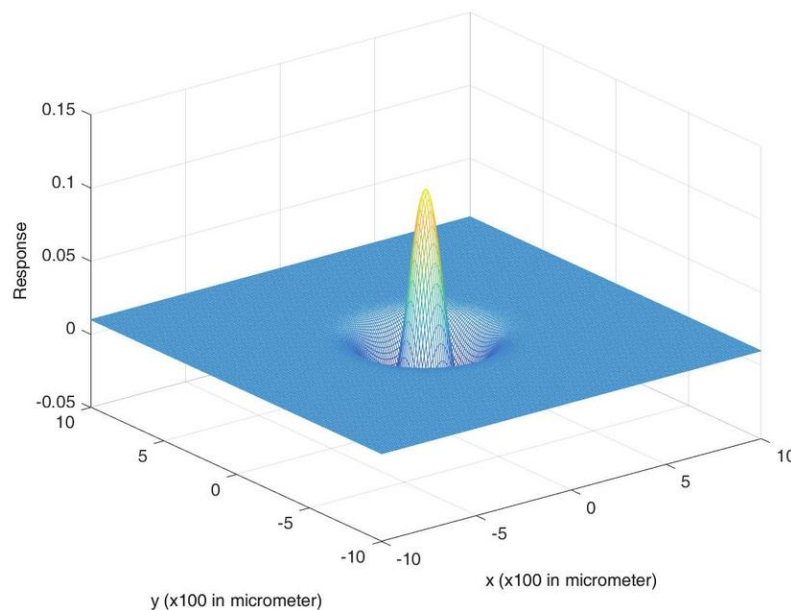


Figure 6.2.2 Two-dimensional Mexican hat function ($a = -0.05$; $b = 0$; $c = 1$; $d = 0.01$).

Each of the three regions of the Mexican hat function specifies the corresponding weight value for each type of lateral feedback connections.

The region 1 specifies a positive feedback connection weight value between two nearby neurons in the excitatory region; the region 2 specifies a negative feedback connection weight value between two further-away neurons in the inhibitory region; and the region 3 specifies a negligible positive (approximately zero) feedback connection value

between two far-away neurons surrounding the inhibitory region.

6.2.2 Clustering phenomenon

The one-dimensional discrete approximation of Figure 6.2.1 is shown in Figure 6.2.3 which defines the nonzero lateral weights f_{mj} for $m = -M$ to M .

In Figure 6.2.3, all the lateral weights in the excitatory region 1 within the excitation width $m = -a$ to a are represented by a constant discrete positive value b (called the excitatory height); all the lateral weights in the inhibitory region 2 are represented by a constant discrete negative value c (called the inhibitory height); and all the lateral weights in the excitatory region 3 (for $m \leq -M$ and $m \geq M$) are approximated by zero value.

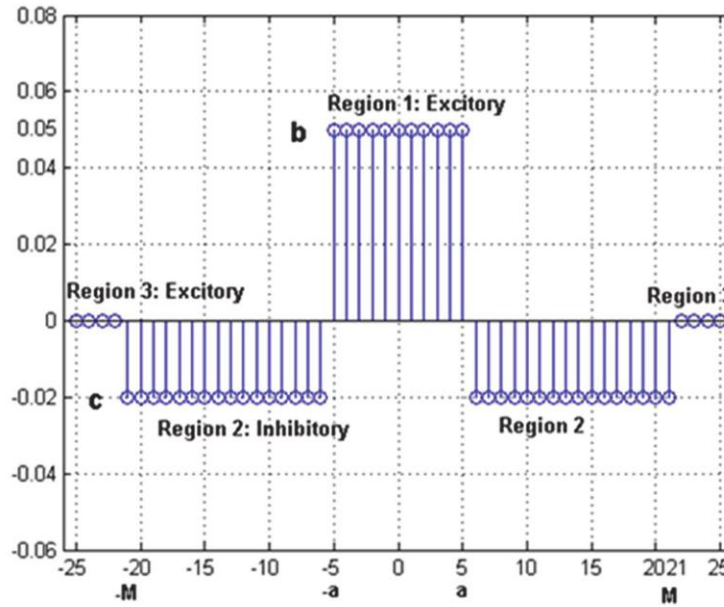


Figure 6.2.3 Discrete approximation of one-dimensional Mexican hat function ($a = 5, M = 21; b = 0.05, c = -0.02$).

A two-dimensional discrete approximation of Figure 6.2.3 is shown in Figure 6.2.4 where a_1 and a_2 , and M_1 and M_2 are respectively the

two-dimensional extensions of a and M .

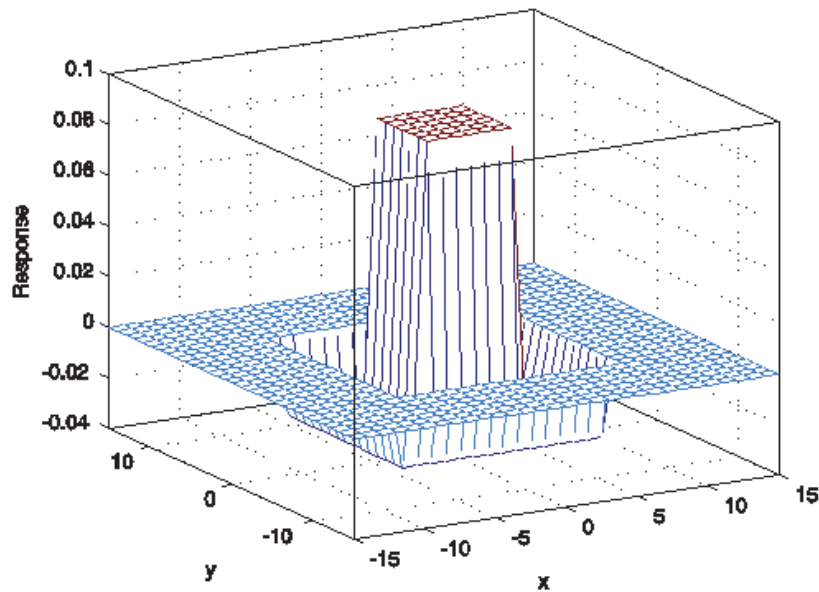


Figure 6.2.4 Discrete approximation of two-dimensional Mexican hat function ($a_i = 3, M_i = 2a_i + 1$, for $i = 1, 2$; $b = 0.09, c = -b/3$).

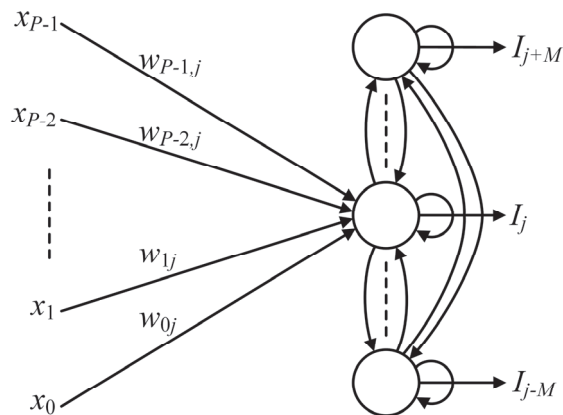


Figure 6.2.5 P feedforward external inputs to neuron j for feature detection and one self-feedback and $2M$ lateral feedbacks of neuron j for clustering.

Consider an array of N neurons shown in Figure 6.2.5. The total external control exerted by the weighted inputs $x_i w_{ij}$ for $i = 0$ to $P - 1$ at a neuron j is

$$I_j = \sum_{i=0}^{P-1} x_i w_{ij} \quad (6.2.5)$$

where x_i for $i = 0$ to $P - 1$ represents the i th input signal to the network; and w_{ij} represents the weight connected a neuron i (for $i = 0$ to $P - 1$) to the neuron j (for $j = 0$ to $N - 1$); and $N > 2M + 1$.

In Figure 6.2.5, the inputs to the neuron j consist of P forward connections from the primary sources of excitation x_i (for $i = 0$ to $P - 1$), and the inputs that are internal to the network which consist of a self-feedback of I_j and the lateral feedbacks of I_{j+m} (for $m = -M$ to M excluding j) from all other outputs of neighboring neurons.

The weighted sum of the input signals at each neuron j is designed to perform feature detection.

The feedback connections produce excitatory or inhibitory effects, depending on individual distance from the neuron j .

The output at a neuron $j = 0$ to $N - 1$ due to the effect of all its feedforward and feedback connections can be represented by

$$y_j(0) = F[I_j(0)] \quad (6.2.6)$$

$$y_j(k) = F \left[\alpha \sum_{m=-M}^M f_{mj} y_{j+m}(k-1) \right] \text{ for } k > 0 \quad (6.2.7)$$

where $y_{j+m}(k-1)$ represents a neuron output within the same layer; f_{mj} represents a lateral feedback weight connecting a neuron m to a neuron j (See Figure 6.2.6); α is the feedback factor; and $F[z]$ defines a characteristic function which is usually taken to be sigmoidal.

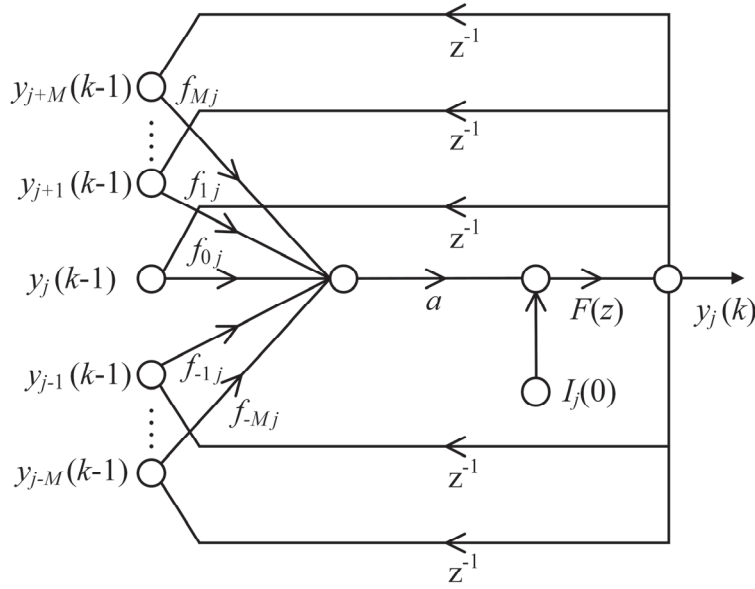


Figure 6.2.6 One self-feedback and $2M$ lateral feedbacks connections of a neuron j with excitatory and inhibitory effects for clustering (with the total external input to neuron j denoted by $I_j(0)$).

For simplicity, $F[z]$ can be approximated by a binary ramp function with a 45° slope such that

$$F[z] = \max(\min(z, S), 0) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{if } 0 \leq z < S \\ S & \text{if } S \leq z \end{cases} \quad (6.2.8)$$

The operations of (6.2.6)-(6.2.8) are illustrated as shown in Figure 6.2.6.

For a two-dimensional SOM of size $N \times N$, the corresponding (6.2.6)-(6.2.7) can be generalized as in (6.2.9) and (6.2.10) such that

$$y_{ij}(0) = F[I_{ij}(0)] \quad \text{for } i, j = 1 \text{ to } N \quad (6.2.9)$$

$$y_{ij}(k) = F \left[\alpha \sum_{m=-M}^M \sum_{n=-M}^M f_{i+m, j+n} y_{i+m, j+n}(k-1) \right] \quad \text{for } k > 0 \quad (6.2.10)$$

Example 6.1 (Bubble formation by SOM clustering) Matlab

Compute the clustering effect in a one-dimensional array consisting of $N = 11$ neurons. Assuming that the total external input is applied only at the beginning of the first iteration.

$$\begin{cases} I_j(0) = A \sin^2\left(\frac{j\pi}{N-1}\right) & \text{for } j = 0 \text{ to } N-1 \\ I_j(k) = 0 & \text{for } k > 0 \end{cases} \quad (\text{E6.1.1})$$

Given that the approximated Mexican hat function as shown in Figure 6.2.3 is used.

The external feedback parameters are $\alpha = 7$; $a = 2$; $b = 0.06$; $c = -0.04$. For $A = 0.1$, $M = 2a+1$, $S = 10$ and with all the computed values rounded to 2 decimal places, compute, tabulate, and plot (using a Matlab program) the outputs at the N neurons from $k = 0$ to 10 using (6.2.6)-(6.2.8).

Identify the locations of visible clustering of activity at $k = 0$ and $k = 10$.

Solution

Due to the shape of the Mexican hat function, each neuron tries to amplify its output and those of its immediate left and right neighbors.

Meanwhile, each neuron also tries to attenuate the output of the neurons which are located farther away on the left and right sides.

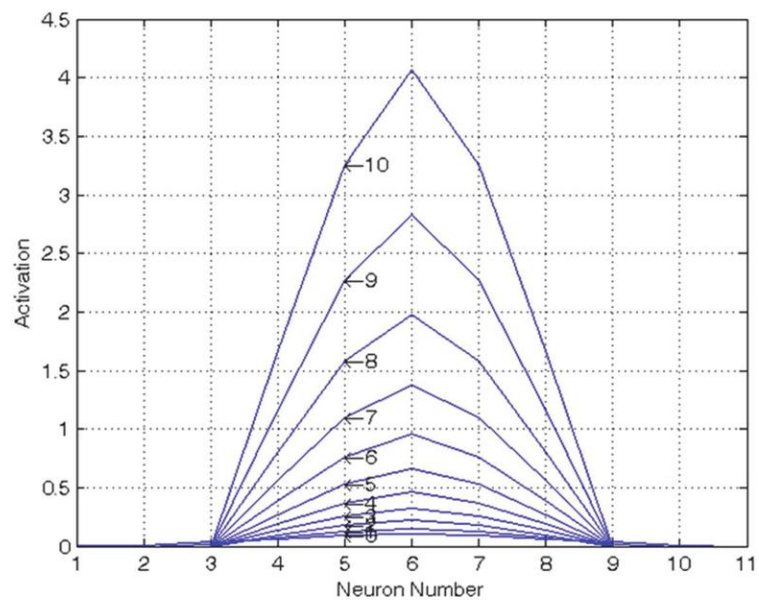
Neurons which have higher output values at the start are going to remain higher during the competitive process.

The clustering of activity around the sixth neuron $y_5(k)$ had the largest amount of activation at $k = 0$, and remains so for $k > 0$, are clearly visible.

The computed outputs at the 11 neurons for $k = 0$ to 10 are listed in Table E6.1.1 and shown in Figure E6.1.1.

Table E6.1.1 Bubble formation by clustering at outputs of neurons.

k	y_0	y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8	y_9	y_{10}
0	0.00	0.01	0.03	0.07	0.09	0.10	0.09	0.07	0.03	0.01	0.00
1	0.00	0.00	0.01	0.07	0.13	0.15	0.13	0.07	0.01	0.00	0.00
2	0.00	0.00	0.00	0.09	0.18	0.22	0.18	0.09	0.00	0.00	0.00
3	0.00	0.00	0.00	0.13	0.26	0.32	0.26	0.13	0.00	0.00	0.00
4	0.00	0.00	0.00	0.19	0.37	0.46	0.37	0.19	0.00	0.00	0.00
5	0.00	0.00	0.00	0.27	0.53	0.67	0.53	0.27	0.00	0.00	0.00
6	0.00	0.00	0.00	0.39	0.76	0.96	0.76	0.39	0.00	0.00	0.00
7	0.00	0.00	0.00	0.56	1.10	1.37	1.10	0.56	0.00	0.00	0.00
8	0.00	0.00	0.00	0.81	1.58	1.97	1.58	0.81	0.00	0.00	0.00
9	0.00	0.00	0.00	1.16	2.27	2.83	2.27	1.16	0.00	0.00	0.00
10	0.00	0.00	0.00	1.67	3.26	4.07	3.26	1.67	0.00	0.00	0.00

**Figure E6.1.1** Bubble formation by clustering.

The step-by-step computations of the activated outputs at two neurons

$j = 2$ and 5 at $k = 1$ are shown below. To start with, the lateral feedback weights of the Mexican hat function are given by

$$f_{-2j} = f_{-1j} = f_{0j} = f_{1j} = f_{2j} = 0.06 \text{ for } j = 0 \text{ to } 10$$

$$f_{-5j} = f_{-4j} = f_{-3j} = f_{3j} = f_{4j} = f_{5j} = -0.04 \text{ for } j = 0 \text{ to } 10$$

$y_j(0)$ (at $k=0$) and any j is given as

$$y_j(0) = F \left[A \sin^2 \left(\frac{j\pi}{N-1} \right) \right] \text{ for } j = 0 \text{ to } N-1$$

Hence, for $j = 0$ to $N-1 = 10$, we have

$$y_0(0) = F \left[0.1 \sin^2 \left(\frac{0 \times \pi}{11-1} \right) \right] = 0.00$$

$$y_1(0) = F \left[0.1 \sin^2 \left(\frac{\pi}{11-1} \right) \right] = 0.01$$

$$y_2(0) = F \left[0.1 \sin^2 \left(\frac{2 \times \pi}{11-1} \right) \right] = 0.0345 \approx 0.03$$

$$y_3(0) = F \left[0.1 \sin^2 \left(\frac{3 \times \pi}{11-1} \right) \right] = 0.07$$

$$y_4(0) = F \left[0.1 \sin^2 \left(\frac{4 \times \pi}{11-1} \right) \right] = 0.09$$

$$y_5(0) = F \left[0.1 \sin^2 \left(\frac{5 \times \pi}{11-1} \right) \right] = 0.10$$

$$y_6(0) = F \left[0.1 \sin^2 \left(\frac{6 \times \pi}{11-1} \right) \right] = 0.09$$

$$y_7(0) = F \left[0.1 \sin^2 \left(\frac{7 \times \pi}{11-1} \right) \right] = 0.07$$

$$y_8(0) = F \left[0.1 \sin^2 \left(\frac{8 \times \pi}{11-1} \right) \right] = 0.0345 \approx 0.03$$

$$y_9(0) = F \left[0.1 \sin^2 \left(\frac{9 \times \pi}{11-1} \right) \right] = 0.01$$

$$y_{10}(0) = F \left[0.1 \sin^2 \left(\frac{10 \times \pi}{11 - 1} \right) \right] = 0.00$$

Computation of $y_j(k)$ for $j = 0$ to 10 at any $k \geq 1$ is

$$\begin{aligned} y_j(k) &= F \left[7 \sum_{m=-5}^5 f_{mj} y_{j+m}(k-1) \right] \\ &= F [7 \{ f_{-5,j} y_{j-5}(k-1) + f_{-4,j} y_{j-4}(k-1) + f_{-3,j} y_{j-3}(k-1) \\ &\quad + f_{-2,j} y_{j-2}(k-1) + f_{-1,j} y_{j-1}(k-1) + f_{0,j} y_j(k-1) \\ &\quad + f_{1,j} y_{j+1}(k-1) + f_{2,j} y_{j+2}(k-1) + f_{3,j} y_{j+3}(k-1) \\ &\quad + f_{4,j} y_{j+4}(k-1) + f_{5,j} y_{j+5}(k-1) \}] \end{aligned}$$

For $k = 1$ and $j = 2$, we have

$$\begin{aligned} y_2(1) &= F \left[7 \sum_{m=-5}^5 f_{m2} y_{2+m}(0) \right] \\ &= F [7 \{ f_{-5,2} y_{2-5}(0) + f_{-4,2} y_{2-4}(0) + f_{-3,2} y_{2-3}(0) \\ &\quad + f_{-2,2} y_{2-2}(0) + f_{-1,2} y_{2-1}(0) + f_{0,2} y_2(0) + f_{1,2} y_{2+1}(0) \\ &\quad + f_{2,2} y_{2+2}(0) + f_{3,2} y_{2+3}(0) + f_{4,2} y_{2+4}(0) + f_{5,2} y_{2+5}(0) \}] \\ &= F [7 \{ f_{-5,2} y_{-3}(0) + f_{-4,2} y_{-2}(0) + f_{-3,2} y_{-1}(0) \\ &\quad + f_{-2,2} y_0(0) + f_{-1,2} y_1(0) + f_{0,2} y_2(0) + f_{1,2} y_3(0) \\ &\quad + f_{2,2} y_4(0) + f_{3,2} y_5(0) + f_{4,2} y_6(0) + f_{5,2} y_7(0) \}] \\ &= F [7 \{ f_{-5,2} \times (0.0) + f_{-4,2} \times (0.0) + f_{-3,2} \times (0.0) \\ &\quad + f_{-2,2} \times (0.0) + f_{-1,2} \times (0.01) + f_{0,2} \times (0.03) + f_{1,2} \times (0.07) \\ &\quad + f_{2,2} \times (0.09) + f_{3,2} \times (0.10) + f_{4,2} \times (0.09) + f_{5,2} \times (0.07) \}] \\ &= F [7 \{ -0.04 \times (0.0) - 0.04 \times (0.0) - 0.04 \times (0.0) \\ &\quad + 0.06 \times (0.0) + 0.06 \times (0.01) + 0.06 \times (0.03) + 0.06 \times (0.07) \}] \end{aligned}$$

$$\begin{aligned}
& +0.06 \times (0.09) - 0.04 \times (0.1) - 0.04 \times (0.09) - 0.04 \times (0.07)\} \\
& = F[0.0122] = 0.0122 \cong 0.01
\end{aligned}$$

For $k=1$ and $j=5$, we have

$$\begin{aligned}
y_5(1) &= F\left[7 \sum_{m=-5}^5 f_{m5} y_{5+m}(0)\right] \\
&= F\{7[f_{-5,5}y_{5-5}(0) + f_{-4,5}y_{5-4}(0) + f_{-3,5}y_{5-3}(0) \\
&\quad + f_{-2,5}y_{5-2}(0) + f_{-1,5}y_{5-1}(0) + f_{0,5}y_{5-0}(0) + f_{1,5}y_{5+1}(0) \\
&\quad + f_{2,5}y_{5+2}(0) + f_{3,5}y_{5+3}(0) + f_{4,5}y_{5+4}(0) + f_{5,5}y_{5+5}(0)]\} \\
&= F\{7[f_{-5,5}y_0(0) + f_{-4,5}y_1(0) + f_{-3,5}y_2(0) \\
&\quad + f_{-2,5}y_3(0) + f_{-1,5}y_4(0) + f_{0,5}y_5(0) + f_{1,5}y_6(0) \\
&\quad + f_{2,5}y_7(0) + f_{3,5}y_8(0) + f_{4,5}y_9(0) + f_{5,5}y_{10}(0)]\} \\
&= F\{-0.04 \times (0.0) - 0.04 \times (0.01) - 0.04 \times (0.03) \\
&\quad + 0.06 \times (0.07) + 0.06 \times (0.09) + 0.06 \times (0.1) + 0.06 \times (0.09) \\
&\quad + 0.06 \times (0.07) - 0.04 \times (0.03) - 0.04 \times (0.01) - 0.04 \times (0.0)\} \\
&= F[0.1540] = 0.154 \cong 0.15
\end{aligned}$$

Example 6.2 (Effect of feedback factor on bubble formation) Matlab

Show the mechanism of the formation of clusters using a one-dimensional array of N neurons.

Assuming the external inputs $I_j(0)$ to be applied initially is a smooth function defined by

$$I_j(0) = A \sin\left(\frac{j\pi}{N-1}\right) \text{ for } j = 0 \text{ to } N-1 \quad (\text{E6.2.1})$$

Assuming that the one-dimensional discrete approximated Mexican hat function as shown in Figure 6.2.3 is used; with $A = 0.1$, $b = 0.4$, $c = -b/2$, $a = 10$, $M = 4a + 1$, $N = 2M + 1$ ($= 83$), $S = 6$, compute and

plot (using a Matlab program) the outputs of all the N neurons using (6.2.6)-(6.2.8) with an appropriate choice of the feedback factor α to illustrate respectively:

- (a) no formation of a bubble;
- (b) formation of a bubble; and
- (c) formation of a saturated bubble.

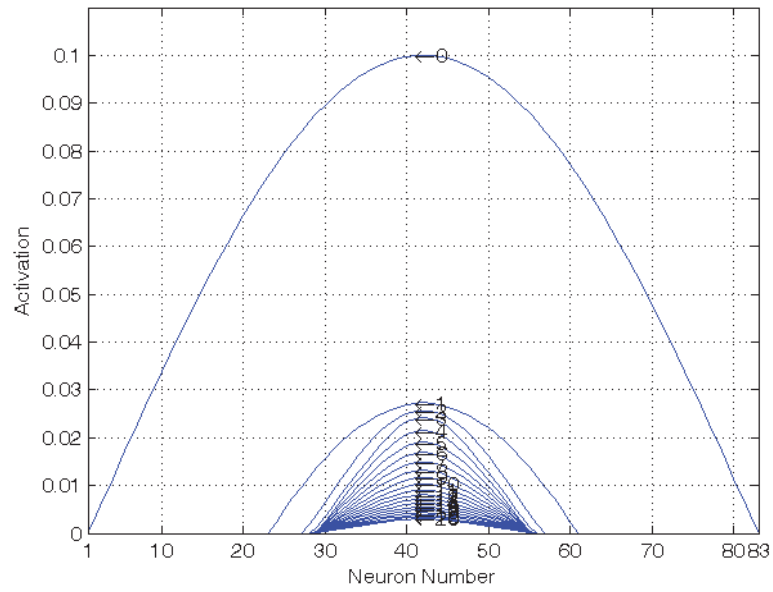
Solution

Using (6.2.6)-(6.2.8), the results are computed and plotted as shown in Figures E6.2.1(a)-(c).

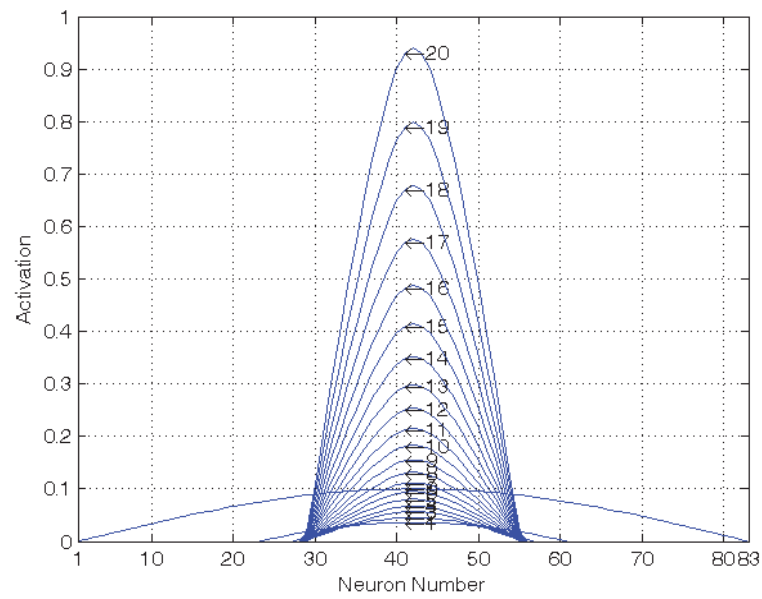
In Figure E6.2.1(a) (with $\alpha = 0.15$), no formation of a bubble occurs. This experiment shows that a critical value for the feedback factor α exists.

If the strength of the feedback is less than a critical value, bubbles of activity will not appear.

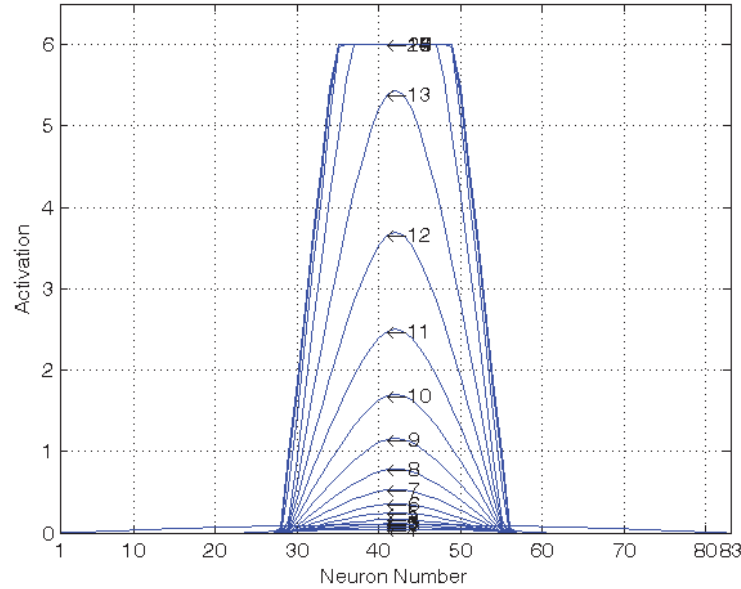
In Figure E6.2.1(b) (with $\alpha = 0.20$), formation of a bubble occurs. The outputs of the neurons stabilize to a bubble shape roughly similar to that of the initial input.



(a)



(b)



(c)

Figure E6.2.1 Bubble formation of neurons 1 to 83 at $k = 0$ to 20 using different α : (a) No (or decreasing) bubble formation for $\alpha = 0.15$; (b) Bubble formation for $\alpha = 0.2$; (c) Saturated bubble formation for $\alpha = 0.25$.

In Figure E6.2.1(c) (with $\alpha = 0.25$), formation of a saturated bubble occurs. The outputs of the neurons converge to a stable (or saturated) cluster of height equal to S ($=6$).

Note that the saturated outputs of the neurons have a magnitude ($= 6$) same as the maximum height ($S = 6$) of the characteristic function $F(z)$.

The width of the cluster (13 at the top and 28 at the bottom) is roughly of the same order of the width of the Mexican hat ($2a+1 = 21$).

Example 6.3 (Bubble formation occurs at initial maximum location)
Matlab

In this example, we demonstrate that the clustering does not always form at the middle, and that the activity bubble is located around where the initial value has a maximum. The external input is set to

$$I_j(0) = A \sin^2 \left(\pi \left(\frac{j}{N-1} \right)^{0.5} \right) \text{ for } j = 0 \text{ to } N - 1 \quad (\text{E6.3.1})$$

For $A = 0.1$, $N = 11$, $\alpha = 6.5$; $a = 2$, $b = 0.06$, $c = -0.04$; $S = 10$, $M = 2a+1$, compute, tabulate, and plot (using a Matlab program with all the computed values rounded to four decimal places) the outputs at the N neurons from $k = 0$ to 10 using (6.2.6)-(6.2.8). Identify the locations of visible clustering of activity at $k = 0$ and $k = 10$.

Solution

Using (E6.3.1), at $k = 0$, $\mathbf{Y}(0) = [0.0000, 0.0702, 0.0973, 0.0978, 0.0837, 0.0633, 0.0423, 0.0241, 0.0106, 0.0026, 0.0000]$, the initial center of activity is located at the fourth neuron ($y_3(k)$).

At $k = 10$, $\mathbf{Y}(10) = [0.0000, 0.7462, 1.4984, 1.8978, 1.5385, 0.8111, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000]$, the center of activity is still located at the fourth neuron ($y_3(k)$).

The outputs of the eleven neurons at $k = 0$ to 10 are listed in Table E6.3.1 and its plot is shown in Figure E6.3.1.

Table E6.3.1 Outputs of neurons showing activity center.

k	$\mathbf{Y}(k)$
0	[0.0000,0.0702,0.0973,0.0978,0.0837,0.0633,0.0423,0.0241,0.0106,0.0026,0.0000]
1	[0.0017,0.0542,0.1024,0.1407,0.1219,0.0744,0.0177,0.0000,0.0000,0.0000,0.0000]
2	[0.0000,0.0610,0.1402,0.1875,0.1637,0.0972,0.0062,0.0000,0.0000,0.0000,0.0000]
3	[0.0000,0.0821,0.1886,0.2518,0.2161,0.1250,0.0031,0.0000,0.0000,0.0000,0.0000]
4	[0.0000,0.1143,0.2547,0.3360,0.2846,0.1620,0.0000,0.0000,0.0000,0.0000,0.0000]
5	[0.0000,0.1588,0.3438,0.4491,0.3749,0.2093,0.0000,0.0000,0.0000,0.0000,0.0000]
6	[0.0000,0.2193,0.4630,0.5990,0.4958,0.2723,0.0000,0.0000,0.0000,0.0000,0.0000]
7	[0.0000,0.3000,0.6223,0.7993,0.6567,0.3558,0.0000,0.0000,0.0000,0.0000,0.0000]
8	[0.0000,0.4081,0.8350,1.0663,0.8713,0.4668,0.0000,0.0000,0.0000,0.0000,0.0000]
9	[0.0000,0.5528,1.1191,1.4225,1.1572,0.6145,0.0000,0.0000,0.0000,0.0000,0.0000]
10	[0.0000,0.7462,1.4984,1.8978,1.5385,0.8111,0.0000,0.0000,0.0000,0.0000,0.0000]

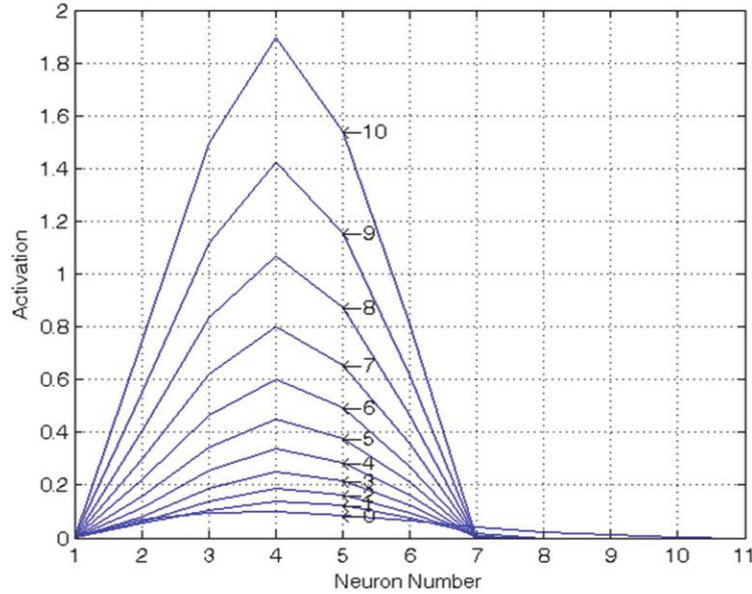


Figure E6.3.1 Activity center remains at initial maximum location.

As observed from Table E6.3.1 and Figure E6.3.1, if the initial center of activity is located near a border, a slight compression of the center of activity away from the border end could happen.

Example 6.4 (Two-dimensional bubble formation by clustering)
Matlab

Show the mechanism of the formation of clusters using a two-dimensional array of $N \times N$ neurons.

Assuming the two-dimensional external inputs $I_{ij}(0)$ to be applied initially is a smooth function defined by

$$I_{ij}(0) = A \sin\left(\frac{i\pi}{N-1}\right) \sin\left(\frac{j\pi}{N-1}\right) \text{ for } i, j = 0 \text{ to } N-1 \quad (\text{E6.4.1})$$

Given the parameters of the approximated two-dimensional Mexican hat function as $a = 3$, $M = 3a$, $b = 0.06$, and $c = -b/3$; also $N = 4M + 1$, $A = 1$ and $S = 100$, use a Matlab program to plot the approximated two-dimensional Mexican hat function and to compute the input and the

corresponding outputs of the $N \times N$ neurons using (6.2.6)-(6.2.8) with an appropriate choice of the feedback factor α to illustrate respectively:

- (a) formation of no bubble;
- (b) formation of bubble; and
- (c) formation of saturated bubble.

Solution

A plot of the discrete approximation of the two-dimensional Mexican hat function is shown in Figure E6.4.1.

With the use of an appropriate feedback factor α , it is shown in Figure E6.4.2 that any one of the three cases can be achieved for (a) formation of no bubble, (b) formation of bubble, and (c) formation of saturated bubble.

The value of the feedback factor α plays a critical role in the occurrence of bubble formation.

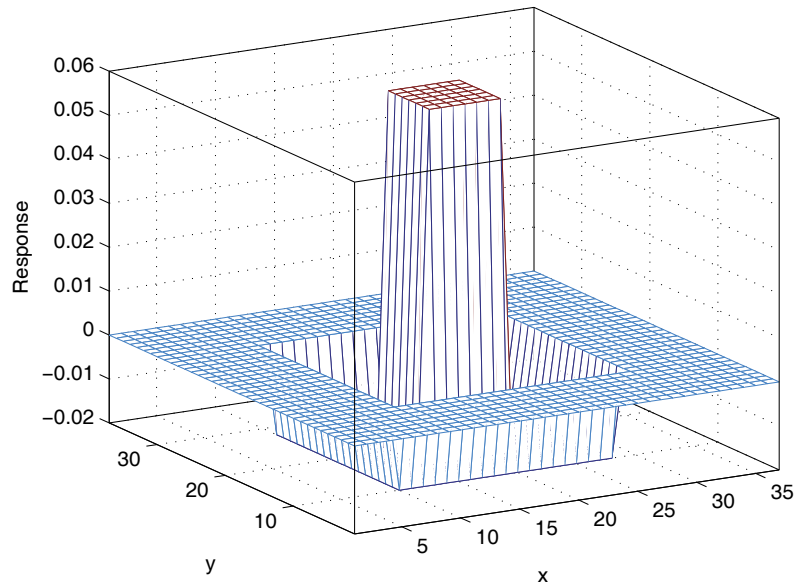
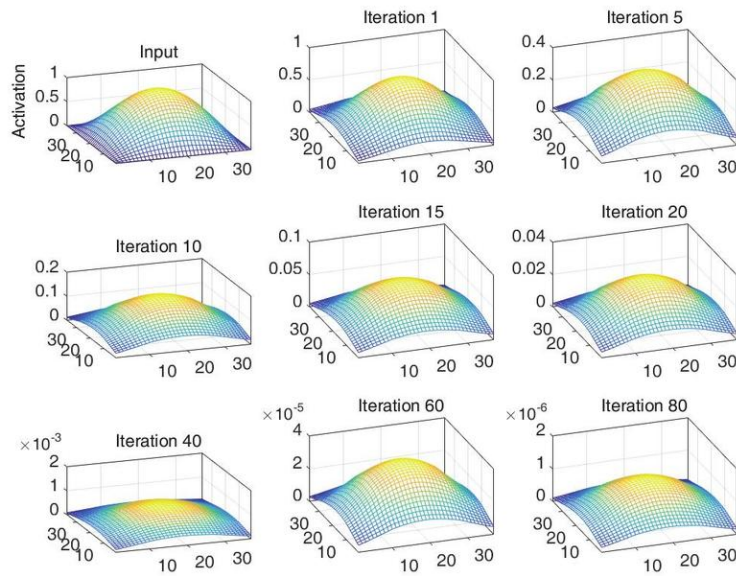
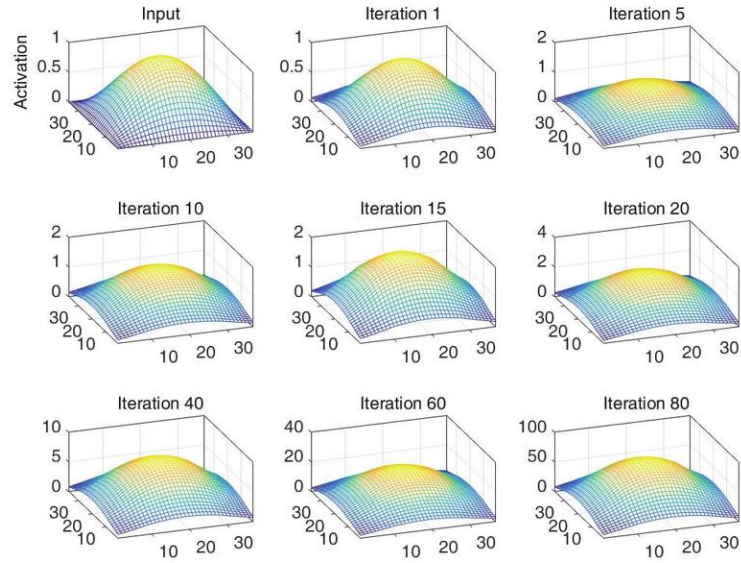


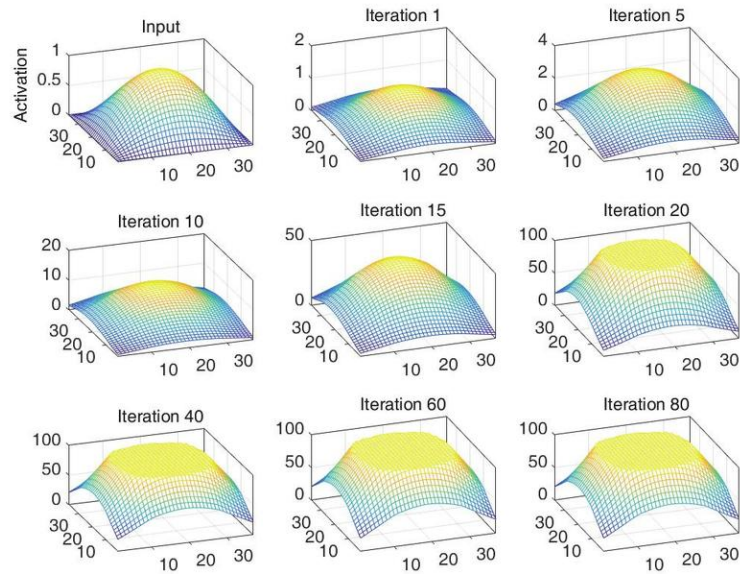
Figure E6.4.1 Discrete two-dimensional Mexican hat function ($a = 3$, $M = 3a$, $b = 0.06$, and $c = -b/3$) and $N = 4M + 1$.



(a)



(b)



(c)

Figure E6.4.2 Effect of feedback factor on bubble formation: (a) No (or decreasing) bubble formation for $\alpha = 0.05$; (b) Bubble formation for $\alpha = 0.056$; (c) Saturated bubble formation for $\alpha = 0.062$.

6.3 Self-Organization Map Adaptation through Feedforward Connections

In this section, the first phase of self-organization in neurons which involves the adaptation of the feedforward weights connecting input stimulus received to those activated neurons is described.

Consider an N -input M -output SOM as shown in Figure 6.3.1, we have

$$y_j = \sum_{i=1}^N x_i w_{ij} \quad \text{for } j = 1 \text{ to } M \quad (6.3.1)$$

Each ordered set $\mathbf{W}_j = [w_{1j}, w_{2j}, \dots, w_{ij}, \dots, w_{Nj}]$ can be considered as a pattern which should be matched with the input $\mathbf{X} = [x_1, x_2, \dots, x_i, \dots, x_N]$.

In general, $\mathbf{W}_j$ represents time-varying adaptive weights $\mathbf{W}_j(k)$, $\mathbf{X}$ represents time-varying input vector $\mathbf{X}(k)$, y_j represents time-varying output $y_j(k)$ at neuron j , and k is the discrete time index.

The largest value among all the M output values indicates the best match as given by

$$y_j(k) = \max_{j=1 \text{ to } M} y_j(k) \quad (6.3.2)$$

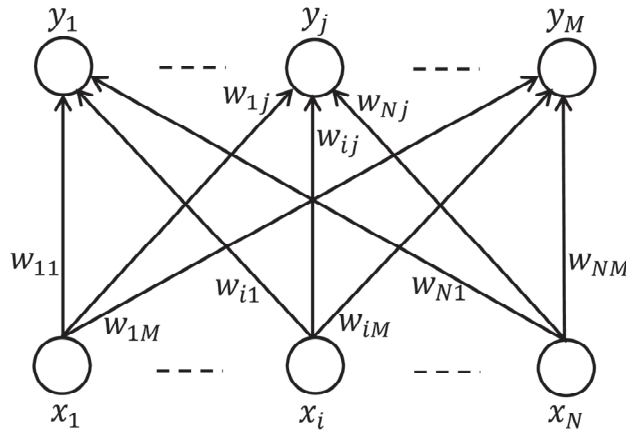


Figure 6.3.1 N -input M -output SOM.

Hence, the maximum value obtained at $y_j(k)$ can be defined as the best match between $\mathbf{W}_j(k) = [w_{1j}(k), w_{2j}(k), \dots, w_{ij}(k), \dots, w_{Nj}(k)]$ and $\mathbf{X}(k) = [x_1(k), x_2(k), \dots, x_i(k), \dots, x_N(k)]$.

The SOM principle applies to $\mathbf{X}(k)$ and $\mathbf{W}_j(k)$ expressed in either bipolar value or real value.

For any unit j and all its nearest neighbors, the following adaptive process is assumed to be active.

$$\mathbf{W}_j(k+1) = \frac{\mathbf{W}_j(k) + g(k)\mathbf{X}(k)}{\|\mathbf{W}_j(k) + g(k)\mathbf{X}(k)\|} \quad (6.3.3)$$

In (6.3.3), $g(k)$ is the learning rate parameter, and the denominator is the Euclidean norm of the numerator to be used for normalization. Normalization improves sensitivity in discrimination but is not always necessary.

Equation (6.3.3) is similar to the learning rule of the perceptron (where $w_{ij}(k+1) = w_{ij}(k)$ for zero output error, and $w_{ij}(k+1) = w_{ij}(k) + g(k)[x_i(k)y_j(k)]$ for non-zero output error) except that the direction of corrections in (6.3.3) is always that of $\mathbf{X}(k)$ and the weight vectors $\mathbf{W}(k)$ are normalized.

A simpler version of the learning rule of (6.3.3) is given by

$$\mathbf{W}_j(k+1) = \mathbf{W}_j(k) + g(k)[\mathbf{X}(k) - \mathbf{W}_j(k)] \quad (6.3.4)$$

Three examples of two-dimensional topological neighborhoods are shown in Figure 6.3.2. The corresponding one-dimensional topological neighborhoods can be seen along either the horizontal or vertical axis.

The learning rate parameter $g(k)$ at iteration k is usually a slowly decreasing function with an appropriate initial value $g(0)$ chosen to be a fraction of $\mathbf{X}(k)$.

Experience has shown that the best results in self-organization are obtained if the neighborhood is selected fairly wide in the beginning and then allowed to shrink with time.

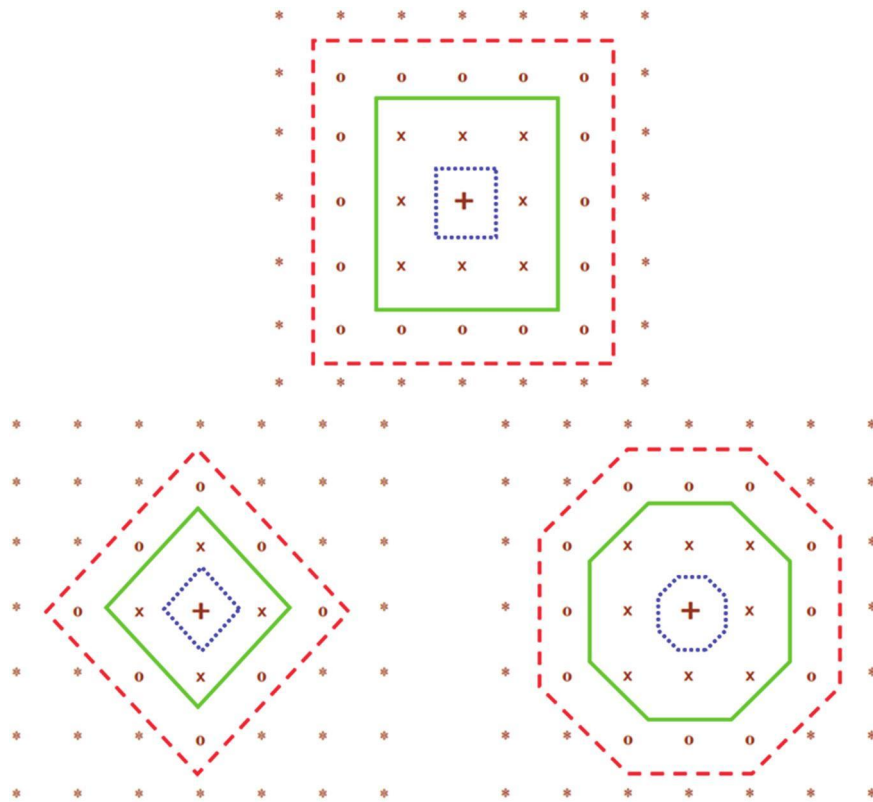


Figure 6.3.2 Two-dimensional topological neighborhoods ($R=0$: dot line; $R=1$: solid line; $R=2$: dash line): (a) Square neighborhood (Top); (b) Diamond neighborhood (Left); (c) Octagonal neighborhood (Right).

6.3.1 One-dimensional SOM algorithm

Step 1. Initialize weights.

Initialize weights from N input nodes to M output nodes to small random values. Set initial radius of the neighborhood to the widest and $g(0)$ to the largest.

Step 2. Present new random input.

Feed a randomly generated input vector $\mathbf{X}(k)$ (or one of the input vectors $\mathbf{X}(k)$ for clustering) to the input nodes.

Step 3. Compute distances to all output nodes.

Compute the distance $d_j(k)$ between the input vector $\mathbf{X}(k)$ and weight vector $\mathbf{W}_j(k)$ for each output node j using the squared Euclidean distance as

$$d_j(k) = \sum_{i=1}^N [x_i(k) - w_{ij}(k)]^2 \text{ for } j = 1 \text{ to } M \quad (6.3.5)$$

Step 4. Select the output node with the minimum distance.

Denote node J as the output node with the minimum distance $d_J(k)$.

Step 5. Update.

Weights of all the nodes in the neighborhood of this node J will be updated according to

$$w_{i,J+r}(k+1) = w_{i,J+r}(k) + g(k)[x_i(k) - w_{i,J+r}(k)] \quad (6.3.6)$$

for $-R \leq r \leq R$

Step 6. Repeat by going to Step 2.

For a one-dimensional input with a neighborhood of two, the following five nodes with node numbers $J-2, J-1, J, J+1, J+2$ will be updated.

In general, the algorithm can be applied to any N (for $N > 1$) inputs. For $N = 2$, we have $\mathbf{X}(k) = [x_1(k), x_2(k)]$, which corresponds to the case of a two-dimensional input.

Example 6.5 (Weight order formation depends on initial random input)
Matlab

Consider a 1-input and 11-output SOM as shown in Figure E6.5.1. The input is a random sequence between 0.0 and 1.0. The neighborhood width and the learning rate parameter are to be updated according to Table E6.5.1.

Show the step-by-step calculation within one complete iteration loop of the SOM.

Plot the SOM at a random start and at iterations 101, 201, 301, and 1001.

Comment on the results.

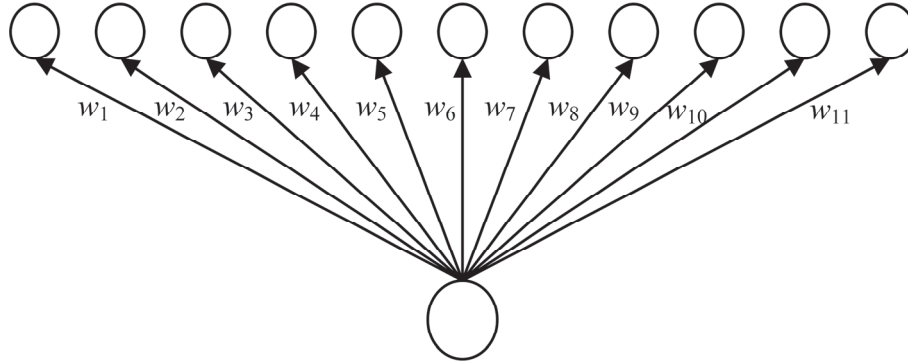


Figure E6.5.1 1-input and 11-output SOM.

Table E6.5.1 Choices of neighborhood width and learning rate parameter.

Iteration k	Neighborhood width, R	Learning rate, $g(k)$
1	4	0.4
101	3	0.3
201	2	0.2
301	1	0.1
1001	1	0.01

Solution

Step 1: Initialize weights with random numbers.

$w_1(0)=0.8900, w_2(0)=0.3900, w_3(0)=0.3500, w_4(0)=0.7100, w_5(0)=0.1600, w_6(0)=0.2800, w_7(0)=0.5800, w_8(0)=0.2500, w_9(0)=0.8800, w_{10}(0)=0.3200, w_{11}(0)=0.8600$.

Step 2: Present a randomized input, $x(0)=0.6$.

Step 3: Compute the squared Euclidean distances to all the 11 nodes.

$d_1=(-0.29)^2, d_2=(0.21)^2, d_3=(0.25)^2, d_4=(-0.11)^2, d_5=(0.44)^2,$

$$d_6=(0.32)^2, d_7=(0.02)^2, d_8=(0.35)^2, d_9=(-0.28)^2, d_{10}=(0.28)^2, \\ d_{11}=(-0.26)^2.$$

Step 4: Select the output node with the minimum distance.

Output node 7.

Step 5: Update.

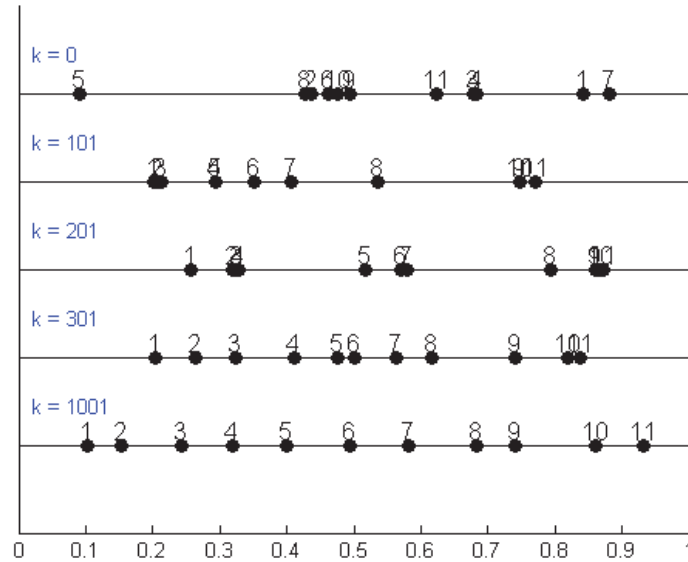
Using $w_j(k+1) = w_j(k) + g(k)[x_i(k) - w_j(k)]$ for weights $j = 3, 4, 5, 6, 7, 8, 9, 10$, and 11 which are within a neighborhood of 4 from node 7.

$$w_1(1)=0.8900, w_2(1)=0.3900, w_3(1)=0.4500, w_4(1)=0.6660, w_5(1)=0.3360, \\ w_6(1)=0.4080, w_7(1)=0.5880, w_8(1)=0.3900, w_9(1)=0.7680, w_{10}(1)=0.4320, \\ w_{11}(1)=0.7560.$$

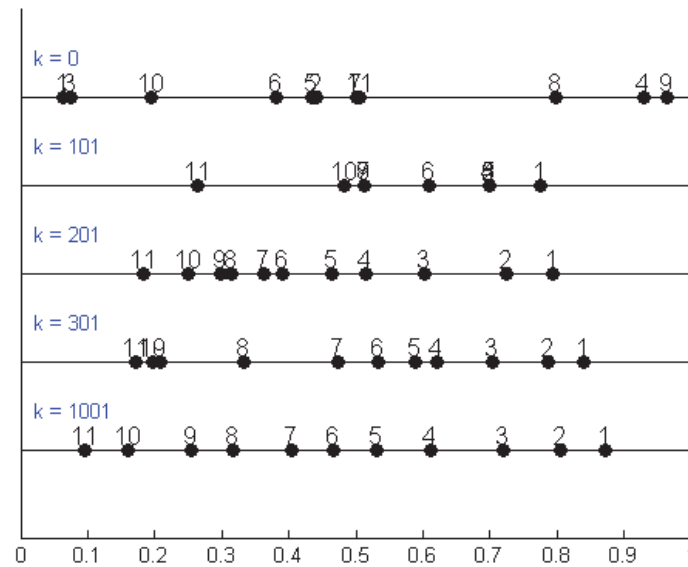
Step 6: Go to Step 2.

As the neighboring width and the learning rate decrease, the weights would be gradually organized.

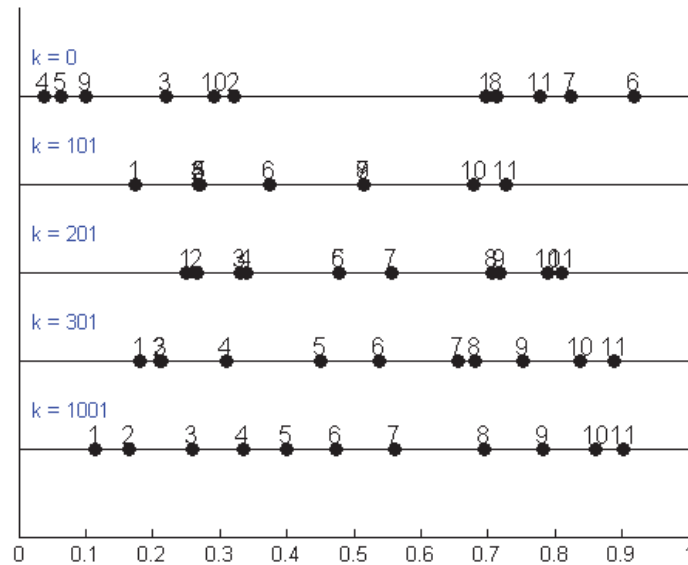
The plots of the organized weights of three different random starts at $k=0, 101, 201, 301$, and 1001 are shown in Figures E6.5.2(a)-(c).



(a)



(b)



(c)

Figure E6.5.2 SOM results: (a) Forward order; (b) Backward order; (c) Two compressed ends.

It can be seen that the weights tend to distribute evenly in either forward order or backward order as in Figures E6.5.2 (a) and (b).

The values of the initial random input shall determine which side of the one-dimensional map will be organized first, whether it is 1 and its neighbors or N and its neighbors, and eventually determine whether the organization will be settled in forward order or in backward order.

Also, at the borders, it is common to observe that the weights are slightly compressed as in Figure E6.5.2(c) due to non-even neighbor updates at the ends.

6.3.2 Causes of self-organization

In a SOM, the change in each value of the weights is proportional to its distance with respect to a random input.

The central neuron representing the winner of the whole neighborhood will therefore change the least while all other neighboring neurons of the winner in the region 1 of the Mexican hat function will be changed in a manner to move closer toward the central neuron.

Neighboring neurons in the region 2 of the Mexican hat function will be changed in a manner to move farther away from the central neuron.

Neighboring neurons in the region 3 of the Mexican hat function will be changed in a manner to move closer toward the central neuron.

With a sufficient number of iterations of random inputs, all the neurons and their neighboring neurons will be organized in a global manner.

6.3.3 Effect of parameters

Appropriate choices of the neighborhood width R and the learning rate parameter $g(k)$ are critical to the success of the SOM algorithm. To obtain the best results:

- (a) The neighborhood width R should be chosen to be wide at the beginning and shrinks gradually.
- (b) The learning rate parameter $g(k)$ should be chosen to be large at the beginning and decreases monotonically.
- (c) The initial weights (or neurons) need not be evenly distributed. In fact, a pre-ordering of initial weights (or neurons) may slow down self-organization.

In the following examples, the effects of unsuitable neighborhood width, learning rate parameter, and initial distribution on a one-dimensional array are examined.

Example 6.6 (SOM requires gradually decreasing neighborhood width)
Matlab

Using the same one-dimensional array and parameters adopted in Example 6.4 except the neighborhood width is set to four and two, respectively, for all k , the results of the self-organized weights are shown respectively in Figures E6.6.1 and E6.6.2.

Due to a poor resolution, it is observed that some of the points are clustered together.

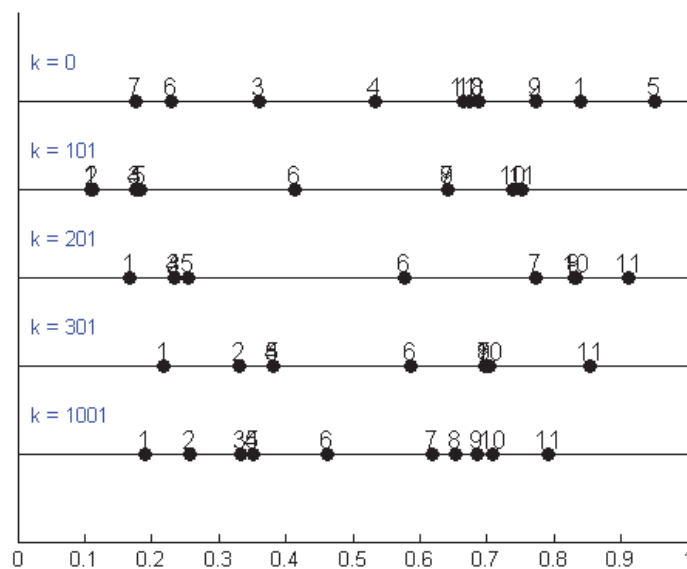


Figure E6.6.1 SOM results with fixed neighborhood width of 4.

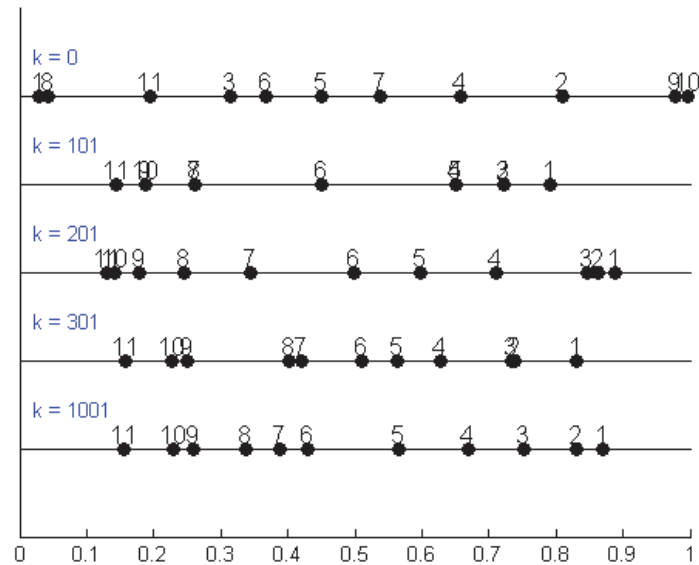


Figure E6.6.2 SOM results with fixed neighborhood width of 2.

Example 6.7 (SOM requires monotonically decreasing learning rate)
Matlab

Using the same one-dimensional array and parameters adopted in Example 6.4 except the learning rate $g(k)$ is set to 0.4 for all k , the self-organized weights are plotted in Figure E6.7.1.

In this example, a high learning rate parameter prevents the weights from gracefully settling down to their final values by causing them to oscillate.

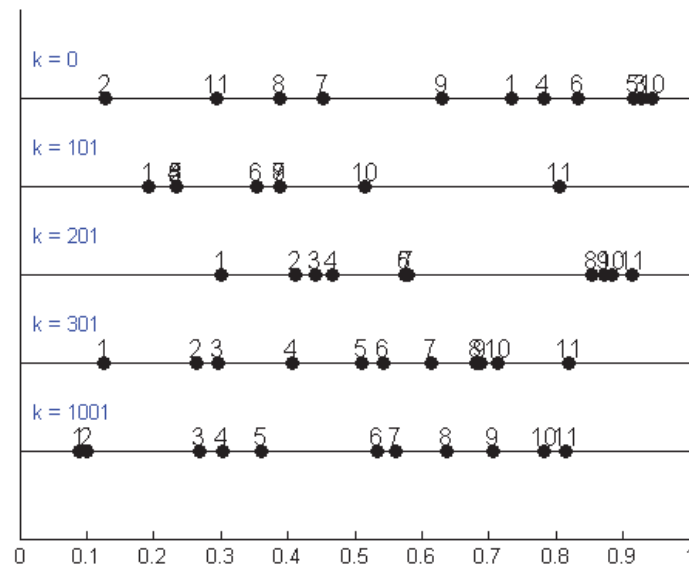


Figure E6.7.1 SOM results with fixed learning rate of 0.4.

Example 6.8 (SOM performs negatively with pre-ordered initial vector)
Matlab

Using the same one-dimensional array and parameters adopted in Example 6.4 except an evenly spread initial weight vector, the result plotted in Figure E6.8.1 shows that an attempt to define a pre-ordered initial weight vector may have an adverse effect on convergence resulting a slowdown but the self-organization process may eventually be recovered with more iterations.

Hence, it is useful to keep the initial values of weights random.

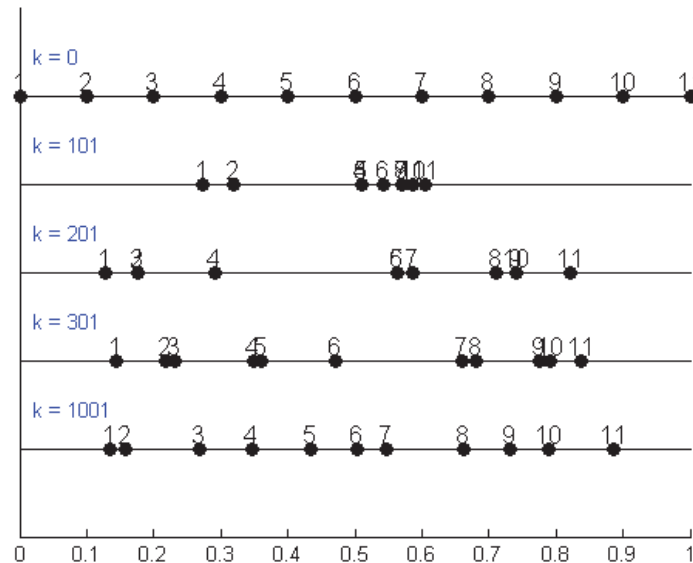


Figure E6.8.1 SOM results with evenly spread initial weights.

The self-organization process of neighboring neurons through random iterations shown by 1-dimensional SOM in Examples 6.5 to 6.8 (as well as 2-dimensional SOM in Design Examples 6.10) demonstrates how scattered memory instants can be linked and unfolded into an organized memory event.

6.4 Clustering by SOM

SOM can be used to cluster input patterns into a set of clusters each represented by a cluster center. The resultant number of cluster centers is usually smaller than the number of input patterns.

SOM is generally regarded as an unsupervised learning algorithm. When it is used for data clustering, the number of clusters that will emerge after training is unknown.

In practice, we could set the number of initial clusters to be higher. After convergence, weight vectors that are sufficiently close to each other can

be combined to reduce the number of clusters.

The above procedure of merging clusters is repeated until each cluster is spread sufficiently away from its neighboring clusters. The remaining clusters shall form the final set of clusters.

SOM clustering can be used to determine the basis centers of a radial basis function network described in Chapter 8.

Design Example 6.9 (SOM for clustering) Matlab

A self-organizing map is used to cluster $P = 4$ vectors $\mathbf{X}_p$ for $p = 1$ to P as given by

$$\mathbf{X}_1 = [1, 0, 0, 0], \mathbf{X}_2 = [0, 1, 0, 0]$$

$$\mathbf{X}_3 = [0, 0, 1, 0], \mathbf{X}_4 = [0, 0, 0, 1]$$

The maximum number of clusters to be formed is $C = 2$. In this example, the number of inputs is $N = 4$ and the maximum number of outputs is $M = 2$. Suppose the learning rate parameter is $g(0) = 0.5$ and $g(k + 1) = 0.99 g(k)$, and the initial weight matrix $\mathbf{W}_j(0)$ for $j = 1$ to M is given as

$$\mathbf{W}_1(0) = [1, 0, 0, 0], \mathbf{W}_2(0) = [0, 0, 0, 1]$$

Compute the weight vectors $\mathbf{W}_1(2)$ and $\mathbf{W}_2(2)$ after 1000 iterations.

Solution

As the maximum number of output classes is $C = 2$, we shall assume that the number of outputs $M = 2$ with no neighborhood update ($R = 0$) due to a small M .

At iteration k , for each of the input vector $\mathbf{X}_p$ for $p = 1$ to P , perform the following steps:

Step 1: Compute the sum of its N squared distances from each of the M weight vectors $\mathbf{W}_j$ for $j = 1$ to M as

$$d_{jp}(k) = \sum_{i=1}^N [x_{ip} - w_{ij}(k)]^2 \text{ for } j = 1 \text{ to } M$$

Step 2: Among $j = 1$ to M , let say $j = J$ which has a minimum value of d_{Jp} for $d_{Jp} \leq d_{jp}$ for $j \neq J$, update the values of $w_{ij}(k)$ connected to the J -output as

$$w_{ij}(k+1) = w_{ij}(k) + g(k)[x_{ip}(k) - w_{ij}(k)] \quad \text{for } i = 1 \text{ to } N$$

The numerical calculations are listed below:

Iteration 1: $g(0) = 0.5$

Present the input vector $\mathbf{X}_1 = [1 \ 0 \ 0 \ 0]$

$$d_1 = (1-1.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 = 0.00$$

$$d_2 = (1-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-1.00)^2 = 2.00$$

d_1 is the smallest, hence, the weight vector $\mathbf{W}_1$ will be updated,

$$\begin{aligned} \mathbf{W}_1 &= [1.00 \ 0.00 \ 0.00 \ 0.00] + 0.5*([1 \ 0 \ 0 \ 0] - [1.00 \ 0.00 \ 0.00 \ 0.00]) \\ &= [1.00 \ 0.00 \ 0.00 \ 0.00] \end{aligned}$$

Present the input vector $\mathbf{X}_2 = [0 \ 1 \ 0 \ 0]$

$$d_1 = (0-1.00)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 = 2.00$$

$$d_2 = (0-0.00)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-1.00)^2 = 2.00$$

d_1 is the smallest, hence, the weight vector $\mathbf{W}_1$ will be updated,

d_2 is also the smallest, hence, the weight vector $\mathbf{W}_2$ will also be updated,

$$\begin{aligned} \mathbf{W}_1 &= [1.00 \ 0.00 \ 0.00 \ 0.00] + 0.5*([0 \ 1 \ 0 \ 0] - [1.00 \ 0.00 \ 0.00 \ 0.00]) \\ &= [0.50 \ 0.50 \ 0.00 \ 0.00] \end{aligned}$$

$$\begin{aligned} \mathbf{W}_2 &= [0.00 \ 0.00 \ 0.00 \ 1.00] + 0.5*([0 \ 1 \ 0 \ 0] - [0.00 \ 0.00 \ 0.00 \ 1.00]) \\ &= [0.00 \ 0.50 \ 0.00 \ 0.50] \end{aligned}$$

Present the input vector $\mathbf{X}_3 = [0 \ 0 \ 1 \ 0]$

$$d_1 = (0-0.50)^2 + (0-0.50)^2 + (1-0.00)^2 + (0-0.00)^2 = 1.50$$

$$d_2 = (0-0.00)^2 + (0-0.50)^2 + (1-0.00)^2 + (0-0.50)^2 = 1.50$$

d_1 is the smallest, hence, the weight vector $\mathbf{W}_1$ will be updated,

d_2 is also the smallest, hence, the weight vector $\mathbf{W}_2$ will also be updated,

$$\begin{aligned}\mathbf{W}_1 &= [0.50 \ 0.50 \ 0.00 \ 0.00] + 0.5*([0 \ 0 \ 1 \ 0] - [0.50 \ 0.50 \ 0.00 \ 0.00]) \\ &= [0.25 \ 0.25 \ 0.50 \ 0.00]\end{aligned}$$

$$\begin{aligned}\mathbf{W}_2 &= [0.00 \ 0.50 \ 0.00 \ 0.50] + 0.5*([0 \ 0 \ 1 \ 0] - [0.00 \ 0.50 \ 0.00 \ 0.50]) \\ &= [0.00 \ 0.25 \ 0.50 \ 0.25]\end{aligned}$$

Present the input vector $\mathbf{X}_4 = [0 \ 0 \ 0 \ 1]$

$$d_1 = (0-0.25)^2 + (0-0.25)^2 + (0-0.50)^2 + (1-0.00)^2 = 1.38$$

$$d_2 = (0-0.00)^2 + (0-0.25)^2 + (0-0.50)^2 + (1-0.25)^2 = 0.88$$

d_2 is the smallest, hence, the weight vector $\mathbf{W}_2$ will be updated,

$$\begin{aligned}\mathbf{W}_2 &= [0.00 \ 0.25 \ 0.50 \ 0.25] + 0.5*([0 \ 0 \ 0 \ 1] - [0.00 \ 0.25 \ 0.50 \ 0.25]) \\ &= [0.00 \ 0.13 \ 0.25 \ 0.63]\end{aligned}$$

After Iteration 1, the weight matrix is

$$\mathbf{W}(1) = \begin{bmatrix} 0.25 & 0.00 \\ 0.25 & 0.13 \\ 0.50 & 0.25 \\ 0.00 & 0.63 \end{bmatrix}$$

Compute $g(1) = 0.99$ $g(0)$ for Iteration 2, $g(2) = 0.99$ $g(1)$ for Iteration 3, and so on.

After iterations 2, 3, 4, ..., we obtain

$$\mathbf{W}(2) = \begin{bmatrix} 0.31 & 0.00 \\ 0.56 & 0.03 \\ 0.13 & 0.31 \\ 0.00 & 0.65 \end{bmatrix}, \mathbf{W}(3) = \begin{bmatrix} 0.33 & 0.00 \\ 0.64 & 0.01 \\ 0.03 & 0.33 \\ 0.00 & 0.66 \end{bmatrix}, \dots$$

Repeat the above for more iterations until after 1000 iterations, we obtain

$$\mathbf{W}(1000) = \begin{bmatrix} 0.4979 & 0.0000 \\ 0.5021 & 0.0000 \\ 0.0000 & 0.4979 \\ 0.0000 & 0.5021 \end{bmatrix}$$

To verify the clustering results, we compute

$$\mathbf{Y}_p = \mathbf{X}_p \mathbf{W}_j(1000) \text{ for } p = 1 \text{ to } 4 \text{ and } j = 1 \text{ to } 2$$

and obtain $\mathbf{Y}_1 = [0.4979, 0.0000]$, $\mathbf{Y}_2 = [0.5021, 0.0000]$, $\mathbf{Y}_3 = [0.0000, 0.4979]$, $\mathbf{Y}_4 = [0.0000, 0.5021]$.

From the above, it can be seen that “ $\mathbf{X}_1$ and $\mathbf{X}_2$ ” form cluster 1 and “ $\mathbf{X}_3$ and $\mathbf{X}_4$ ” form cluster 2.

6.5 Two-Dimensional Self-Organizing Map

In a two-dimensional self-organizing map, there are two inputs $x_1(k)$ and $x_2(k)$ in which their values are random and are assumed to have a uniform probability density function.

A two-dimensional self-organizing map connects two inputs $[x_1(k), x_2(k)]$ to its $N \times M$ outputs $[u_{ij}(k), v_{ij}(k)]$ for $i = 1$ to N and $j = 1$ to M . For a square map, this implies $N = M$.

Each of the $N \times M$ outputs with an index (i, j) is connected directly to each of the two inputs $[x_1(k), x_2(k)]$ through two weights $[u_{ij}(k), v_{ij}(k)]$.

In a two-dimensional display, $u_{ij}(k)$ represents the (i, j) th weight value along the horizontal axis, and $v_{ij}(k)$ represents the (i, j) th weight value along the vertical axis.

At iteration k , the weight vector $\mathbf{W}_{ij}(k) = [u_{ij}(k), v_{ij}(k)]$ is shown as one point (i, j) in the same two-dimensional coordinate system where the input $\mathbf{X}(k) = [x_1(k), x_2(k)]$ is represented.

To show to which unit $\mathbf{W}_{ij}(k)$ belongs, each of the $N \times M$ points $\mathbf{W}_{ij}(k)$ has been connected by a lattice of lines to the weight vectors of its immediate neighbors.

As shown in Figure E6.10.1, a line connecting $\mathbf{W}_{ij}(k)$ to each of its four immediate neighbors, $\mathbf{W}_{ij}(k)$ to $\mathbf{W}_{i-1,j}(k)$, $\mathbf{W}_{ij}(k)$ to $\mathbf{W}_{i+1,j}(k)$, $\mathbf{W}_{ij}(k)$ to $\mathbf{W}_{i,j-1}(k)$, $\mathbf{W}_{ij}(k)$ to $\mathbf{W}_{i,j+1}(k)$ indicates that these four points are adjacent in the physical array.

During the self-organizing process, the values of all the weight outputs $[u_{ij}(k), v_{ij}(k)]$ evolve while keeping their identifying indexes (i, j) the same. If there is more than one winner in iteration k , a random choice among the winners is adopted.

Adopting two-dimensional input vectors, two-dimensional weight vectors, and a two-dimensional square grid neighborhood for update, the self-organization of a set of $5 \times 5 (=N \times M)$ weight vectors $\mathbf{W}_{ij}(k)$ (with a two-dimensional lattice interconnection) during an ordering process is shown in Figures E6.10.1-E6.10.7 of Example 6.10.

6.5.1 Two-dimensional SOM algorithm

In each iteration k , we have two random inputs $[x_1(k), x_2(k)]$ and $N \times M$ self-organizing outputs $[u_{ij}(k), v_{ij}(k)]$ for $i = 1$ to $N, j = 1$ to M .

The distance-computations are

$$d_{ij}(k) = [x_1(k) - u_{ij}(k)]^2 + [x_2(k) - v_{ij}(k)]^2 \quad (6.5.1)$$

The weight-updates within a defined neighborhood $(-R \leq r \leq R)$ are

$$u_{ij}(k+1) = u_{ij}(k) + g(k)[x_1(k) - u_{ij}(k)] \quad (6.5.2)$$

$$v_{ij}(k+1) = v_{ij}(k) + g(k)[x_2(k) - v_{ij}(k)] \quad (6.5.3)$$

To display the two-dimensional self-organized map, we compute

$$r_{ij}(k) = [u_{ij}^2(k) + v_{ij}^2(k)]^{1/2} \quad (6.5.4)$$

$$\theta_{ij}(k) = \tan^{-1}[v_{ij}(k)/u_{ij}(k)] \quad (6.5.5)$$

$r_{ij}(k) \angle \theta_{ij}(k)$ for $1 \leq i \leq N$ and $1 \leq j \leq M$ specifies all the $N \times M$ points of a two-dimensional map in the two-dimensional coordinate

system with respect to the origin.

At iteration k , $[x_1(k), x_2(k)]$ forms a new random pair of inputs to self-organize the two-dimensional map.

In such a network, each of the $N \times M$ weight vectors is represented by $[u_{ij}(k), v_{ij}(k)]$, both weight values are evolving dynamically during the self-organization process.

Initially, the $N \times M$ random pairs of $[u_{ij}(k), v_{ij}(k)]$ are not organized according to their locations at (i, j) as shown in Figure E6.10.1.

After a sufficient number of iterations, a two-dimensional self-organized map with a starting index (1, 1) from any of the four corners (usually the top two corners for this simulation-example) will emerge as shown in Figure E6.10.7.

The final location of the index (1, 1) of the organized map depends on the initial locations of all the indexes (i, j) and the initial sets of winners.

The steps of the two-dimensional SOM are summarized in Table 6.5.1.

Table 6.5.1 Two-dimensional SOM algorithm.

Step	Description
1	Initialize $N \times M$ weight matrix with random values $[u_{ij}(0), v_{ij}(0)]$ for each element. Select initial neighborhood width and initial learning rate $g(0)$.
2	At iteration k , present new random input $[x_1(k), x_2(k)]$.
3	Compute the distance between input nodes and output nodes using $d_{ij}(k) = [x_1(k) - u_{ij}(k)]^2 + [x_2(k) - v_{ij}(k)]^2$
4	The winning neuron is computed by finding the minimum distance and its index.
5	The winning neuron and its neighbors are updated using $u_{ij}(k+1) = u_{ij}(k) + g(k)[x_1(k) - u_{ij}(k)]$ $v_{ij}(k+1) = v_{ij}(k) + g(k)[x_2(k) - v_{ij}(k)]$

6	Learning rate $g(k)$ and neighborhood width are adjusted as iteration progresses.
7	Steps 2 to 6 are repeated until the 2-D SOM has converged.

Design Example 6.10 (Two-dimensional self-organization map)
Matlab

Consider a two-dimensional self-organization network consisting of 5×5 outputs, with two random inputs $[x_1(k), x_2(k)]$ and 5×5 initial random outputs $[u_{ij}(k), v_{ij}(k)]$.

The range of each of the input and output values is 0.0 to 1.0. The learning rate parameter $g(k)$ and the square neighborhood width are given in Table E6.10.1. The maximum number of iterations is 5000.

Show the self-organization results at $k = 0, 100, 200, 500, 1000, 2000$, and 5000.

Table E6.10.1 Choice of learning rate parameter and neighborhood width.

Iteration, k	Learning rate $g(k)$	Neighborhood width
0	0.700	2
100	0.660	2
200	0.633	1
500	0.540	1
1000	0.420	1
2000	0.250	1
5000	0.050	1

Solution

$$k = 0: x_1(0) = 0.54, x_2(0) = 0.58$$

$$[u_{ij}(0), v_{ij}(0)] =$$

[0.07, 0.91]	[0.08, 0.97]	[0.92, 0.72]	[0.32, 0.83]	[0.32, 0.39]
[0.43, 0.15]	[0.12, 0.48]	[0.43, 0.83]	[0.49, 0.24]	[0.49, 0.61]
[0.33, 0.40]	[0.97, 0.05]	[0.41, 0.74]	[0.16, 0.64]	[0.54, 0.35]
[0.47, 0.28]	[0.46, 0.56]	[0.14, 0.82]	[0.58, 0.86]	[0.43, 0.82]
[0.07, 0.90]	[0.36, 0.86]	[0.64, 0.63]	[0.89, 0.43]	[0.09, 0.23]

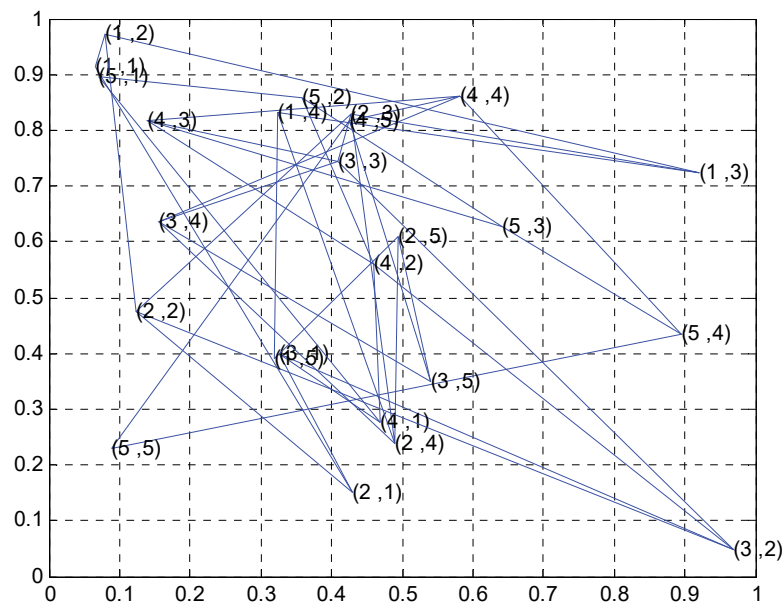


Figure E6.10.1 Two-dimensional self-organization map at $k = 0$.

$$k = 100: x_1(k) = 0.21, x_2(k) = 0.90$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.92, 0.01]	[0.80, 0.25]	[0.71, 0.31]	[0.39, 0.12]	[0.18, 0.10]
[0.92, 0.12]	[0.79, 0.62]	[0.56, 0.32]	[0.31, 0.32]	[0.11, 0.16]
[0.85, 0.35]	[0.65, 0.62]	[0.53, 0.54]	[0.28, 0.45]	[0.11, 0.53]
[0.85, 0.71]	[0.65, 0.73]	[0.45, 0.57]	[0.22, 0.71]	[0.08, 0.74]
[0.81, 0.90]	[0.61, 0.90]	[0.38, 0.89]	[0.10, 0.76]	[0.03, 0.89]

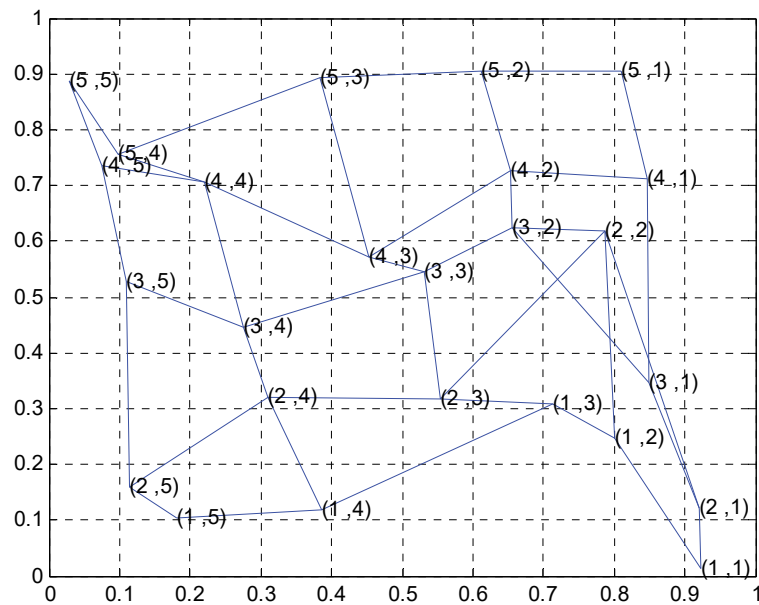


Figure E6.10.2 Two-dimensional self-organization map at $k = 100$.

$$k = 200: x_1(k) = 0.84, x_2(k) = 0.09$$

$$[u_{ij}(k), v_{ij}(k)] =$$

$[0.96, 0.06]$	$[0.94, 0.20]$	$[0.59, 0.02]$	$[0.36, 0.10]$	$[0.19, 0.14]$
$[0.92, 0.16]$	$[0.75, 0.38]$	$[0.57, 0.29]$	$[0.33, 0.15]$	$[0.12, 0.57]$
$[0.84, 0.66]$	$[0.72, 0.54]$	$[0.50, 0.50]$	$[0.33, 0.26]$	$[0.05, 0.70]$
$[0.84, 0.73]$	$[0.67, 0.67]$	$[0.49, 0.66]$	$[0.22, 0.50]$	$[0.05, 0.82]$
$[0.82, 0.90]$	$[0.66, 0.91]$	$[0.47, 0.98]$	$[0.07, 0.78]$	$[0.02, 0.92]$

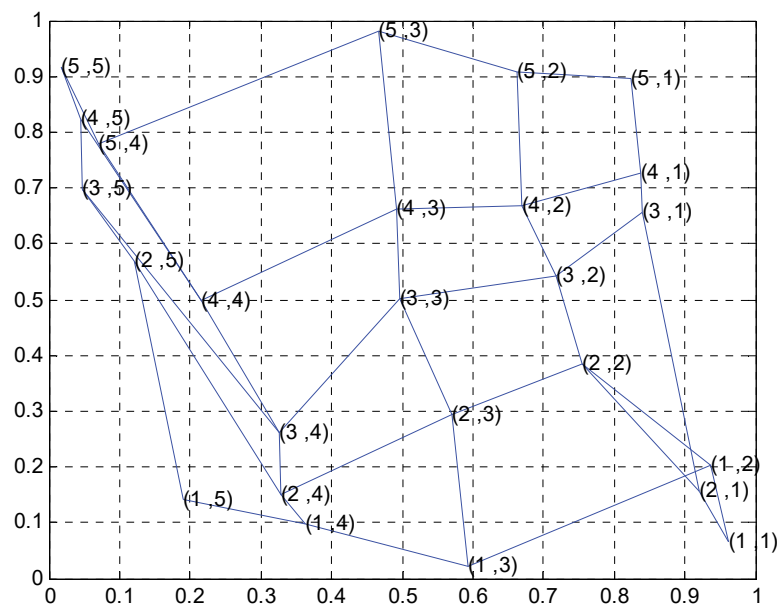


Figure E6.10.3 Two-dimensional self-organization map at $k = 200$.

$$k = 500: x_1(k) = 0.03, x_2(k) = 0.43$$

$$[u_{ij}(k), v_{ij}(k)] =$$

$$\begin{bmatrix} [0.94, 0.03] & [0.91, 0.06] & [0.66, 0.15] & [0.49, 0.08] & [0.18, 0.12] \\ [0.92, 0.52] & [0.80, 0.25] & [0.62, 0.38] & [0.40, 0.53] & [0.07, 0.26] \\ [0.88, 0.65] & [0.74, 0.32] & [0.55, 0.56] & [0.36, 0.75] & \mathbf{[0.05, 0.46]} \\ [0.84, 0.74] & [0.65, 0.44] & [0.54, 0.60] & [0.34, 0.92] & [0.05, 0.72] \\ [0.82, 0.93] & [0.63, 0.91] & [0.41, 0.81] & [0.22, 0.94] & [0.04, 0.98] \end{bmatrix}$$

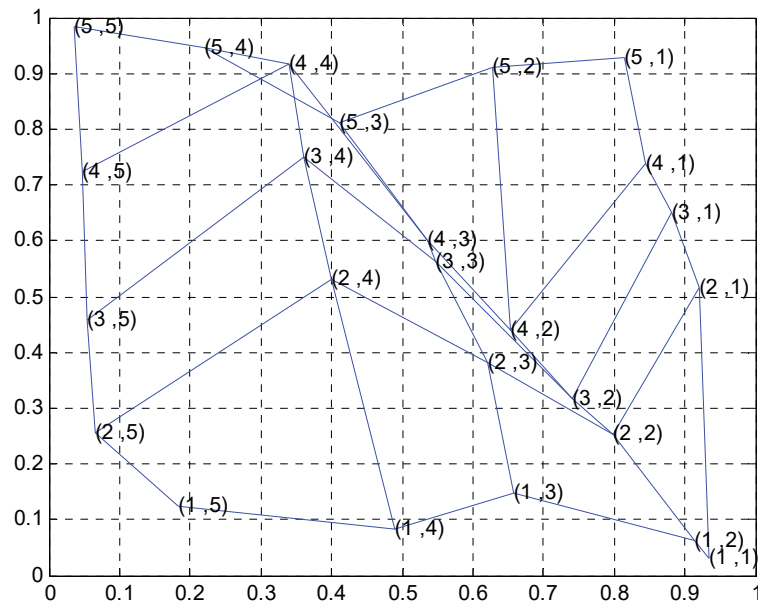


Figure E6.10.4 Two-dimensional self-organization map at $k = 500$.

$$k = 1000: x_1(k) = 0.92, x_2(k) = 0.79$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.93, 0.12]	[0.72, 0.04]	[0.73, 0.39]	[0.53, 0.12]	[0.15, 0.05]
[0.93, 0.21]	[0.71, 0.24]	[0.57, 0.62]	[0.33, 0.19]	[0.12, 0.26]
[0.92, 0.38]	[0.68, 0.45]	[0.54, 0.65]	[0.30, 0.40]	[0.09, 0.55]
[0.89, 0.74]	[0.66, 0.76]	[0.50, 0.76]	[0.27, 0.61]	[0.07, 0.72]
[0.79, 0.94]	[0.66, 0.93]	[0.45, 0.89]	[0.21, 0.94]	[0.04, 0.81]

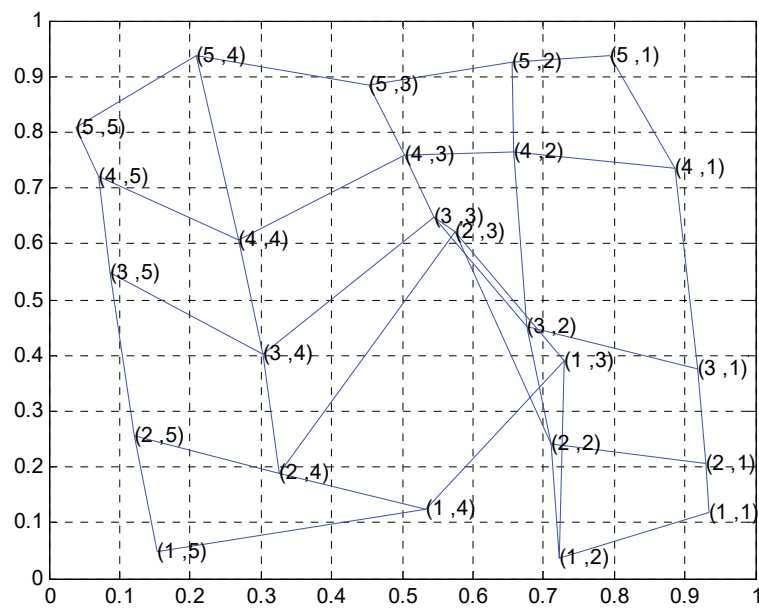


Figure E6.10.5 Two-dimensional self-organization map at $k = 1000$.

$$k = 2000: x_1(k) = 0.89, x_2(k) = 0.07$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.95, 0.08]	[0.80, 0.27]	[0.58, 0.08]	[0.44, 0.06]	[0.28, 0.16]
[0.94, 0.18]	[0.79, 0.62]	[0.54, 0.12]	[0.41, 0.30]	[0.15, 0.43]
[0.94, 0.40]	[0.75, 0.75]	[0.53, 0.33]	[0.35, 0.38]	[0.13, 0.53]
[0.82, 0.52]	[0.74, 0.86]	[0.52, 0.53]	[0.23, 0.76]	[0.11, 0.70]
[0.78, 0.91]	[0.61, 0.90]	[0.42, 0.72]	[0.23, 0.91]	[0.07, 0.88]

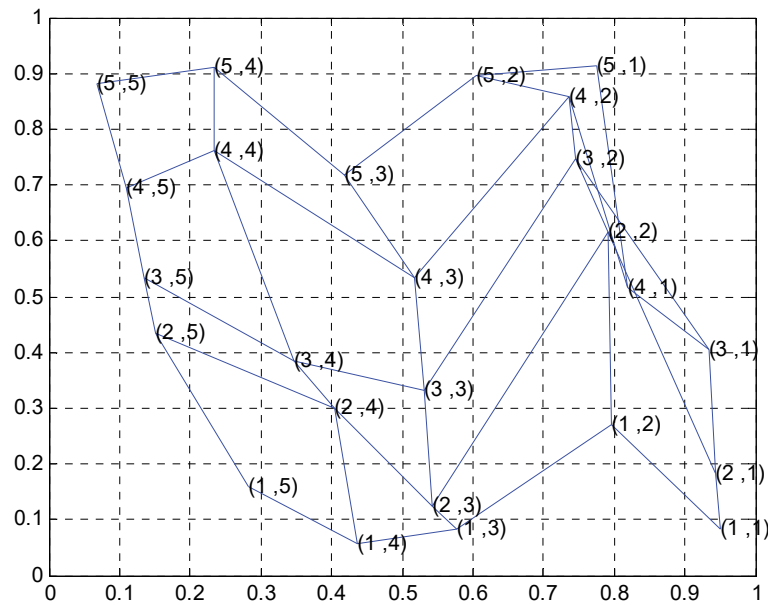


Figure E6.10.6 Two-dimensional self-organization map at $k = 2000$.

$$k = 5000: x_1(k) = 0.49, x_2(k) = 0.97$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.92, 0.08]	[0.77, 0.09]	[0.57, 0.13]	[0.40, 0.10]	[0.12, 0.07]
[0.92, 0.21]	[0.76, 0.33]	[0.55, 0.35]	[0.32, 0.32]	[0.12, 0.26]
[0.92, 0.42]	[0.69, 0.50]	[0.49, 0.55]	[0.29, 0.56]	[0.10, 0.49]
[0.90, 0.65]	[0.67, 0.70]	[0.49, 0.71]	[0.27, 0.76]	[0.09, 0.68]
[0.88, 0.89]	[0.66, 0.92]	[0.46, 0.89]	[0.23, 0.91]	[0.07, 0.91]

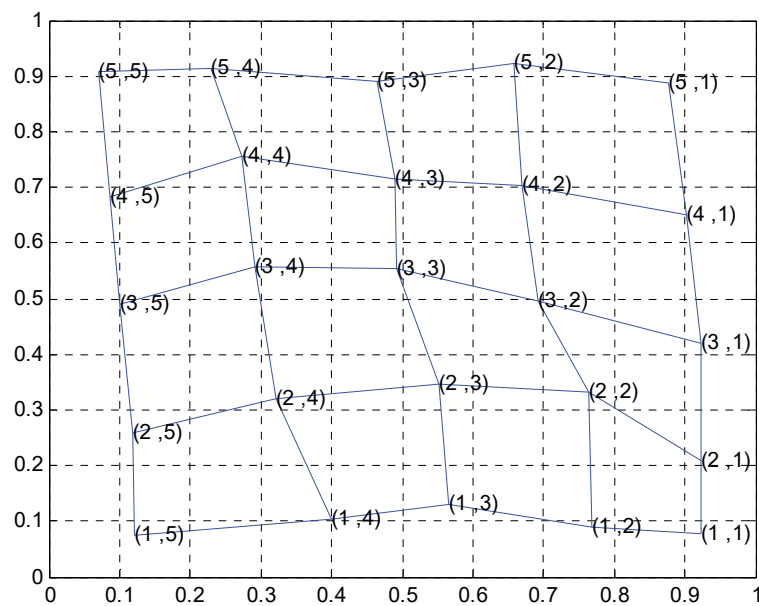


Figure E6.10.7 Two-dimensional self-organization map at $k = 5000$.

Chapter 7

Learning Vector Quantization

7.1 Introduction

Unlike the self-organizing map (SOM) [Koh9009] which is an unsupervised learning network, the learning vector quantization (LVQ) [Koh9009], [Koh9006] is a supervised learning network.

Supervised learning algorithms include support vector machine (SVM) [Cor9509], and newer algorithms include L2 soft margin support vector machine [Gu0405], and distriected matching [Che0410]. Support vector machine (SVM) is a supervised machine learning algorithm which can be used for classification or regression problems but is mostly used in classification problems.

Both types of SOM and LVQ networks are of one-layer and differ from each other in the number of outputs of their network structures; and both are linear networks.

The learning vector quantization is a pattern classification method with each output unit representing a particular class.

The weight vector for an output unit which represents the center of a class, is often referred to as a reference vector (or codebook) for the class that the unit represents.

During training, the output units are positioned by adjusting their weights through supervised training to approximate the decision boundary of the optimal classifier.

An initial set of reference vectors is either given or randomly chosen. It is assumed that a set of training patterns with their known classifications is provided.

After training, a learning vector quantization network classifies an input vector by assigning it to the same class as the output unit that has a weight vector closest to the input vector.

The topics covered in this chapter include LVQ architecture, learning algorithm, weight initialization methods, and improved LVQ algorithms. Examples are given to illustrate the capabilities and properties of learning vector quantization networks.

7.2 LVQ Network

The structure of an N -input and M -output learning vector quantization network is shown in Figure 7.2.1, the learning vector quantization network is essentially the same as that of a self-organizing map network.

However, the self-organizing map does not have a topological structure being assumed for the output units before training, a final set of output units will emerge after training.

This makes the SOM an unsupervised learning network. In the learning vector quantization, each output unit has a known class that it represents, which makes the LVQ a supervised learning network.

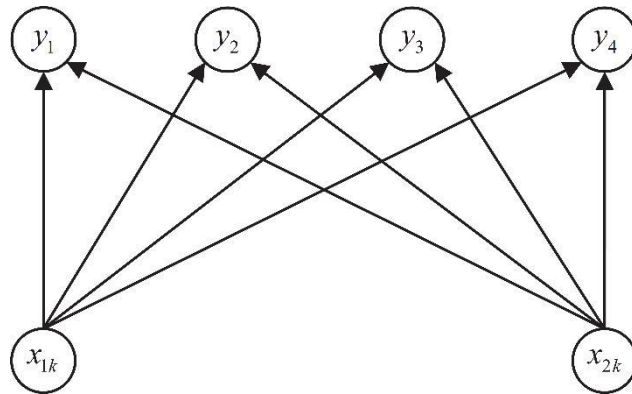


Figure 7.2.1 2-input 4-output LVQ network.

7.3 LVQ Learning Algorithm

The motivation of the learning vector quantization algorithm is to find the output unit j represented by a weight vector $\mathbf{W}_j$ that is closest to an input vector $\mathbf{X}_i$.

To achieve this, if $\mathbf{X}_i$ and $\mathbf{W}_j$ belong to the same class, then the weight vector is moved toward the new input vector; if $\mathbf{X}_i$ and $\mathbf{W}_j$ belong to two different classes, then the weight vector moved away from this input vector.

The adopted notations for LVQ are defined in Table 7.3.1.

Table 7.3.1 Notations for LVQ.

Symbol	Description
$\mathbf{X}_i$	Training vector $[x_{1i}, \dots, x_{ni}, \dots, x_{Ni}]$.
T_i	Correct class for the i th training vector.
$\mathbf{W}_j(t)$	Weight vector (reference vector) $[w_{1j}, \dots, w_{nj}, \dots, w_{Nj}]$ for j th output unit.
C_j	Class represented by j th output unit for $j = 1$ to M .
C_J	Class represented by the winning output unit J .
d_{ij}	Squared l_2 -norm between two vectors $\mathbf{X}_i$ and $\mathbf{W}_j$ is defined as $\ \mathbf{X}_i - \mathbf{W}_j(t)\ _2^2 = (x_{1i} - w_{1j})^2 + (x_{2i} - w_{2j})^2 + \dots + (x_{Ni} - w_{Nj})^2$. Other distance measurements between two vectors such as l_2 -norm or sum of absolute differences can also be used.
$g(t)$	Learning rate parameter

Learning algorithm

Step 0: Set iteration index $t = 0$, initialize all the weight vectors $\mathbf{W}_j(0)$ for $j = 1$ to M and $g(0)$.

Step 1: While stopping condition is false, do Steps 2-6.

Step 2: For each training input vector $\mathbf{X}_i$, do Steps 3-4.

Step 3: Find J so that the squared l_2 -norm $\|\mathbf{X}_i - \mathbf{W}_J(t)\|_2^2$ is a minimum.

Step 4: Update $\mathbf{W}_J(t)$ as follows:

If $T_i = C_J$, then

$$\mathbf{W}_J(t+1) = \mathbf{W}_J(t) + g(t)[\mathbf{X}_i - \mathbf{W}_J(t)] \quad (7.3.1)$$

If $T_i \neq C_J$, then

$$\mathbf{W}_J(t+1) = \mathbf{W}_J(t) - g(t)[\mathbf{X}_i - \mathbf{W}_J(t)] \quad (7.3.2)$$

Step 5: Set $t = t + 1$, reduce the learning rate parameter $g(t+1)$.

Step 6: Test stopping condition: The stopping condition can be one of the followings: (a) All the training input vectors have been learned and classified correctly or (b) a fixed number of iterations has been reached or (c) when $g(t)$ reaches a sufficiently small value.

Note that within each iteration, the latest set of weights:

(a) can be updated immediately before the next input vector presentation which is applicable for a smaller learning problem consisting of a small number of input-output training pairs; or

(b) can be updated immediately before the next iteration starts which is applicable for a bigger learning problem consisting of a large number of input-output training pairs.

7.4 Weight Initialization Methods

Three methods for initializing the weights of a learning vector quantization network are listed in Table 7.4.1.

Table 7.4.1 LVQ weight initialization methods.

Method	Description
1	The simplest method for initializing the weight vectors is to take the first M training vectors with distinct classes to be used as M initial weight vectors with corresponding classes.
2	Another simple method for initializing the weight vectors is to assign randomly the weights within specified range and their corresponding classes obtained as M initial weight vectors with classes.
3	The third method for initializing the weights is to use the self-organizing map to determine the weights. Each of the weight vectors is then compared with all the input patterns to find the class that has the largest number of winning input patterns, and to assign that class to the weight vector.
	If there is more number of classes than what are required by the input vectors, choose the top winners.
	If there is less number of classes than required, assign the remaining classes “with random weights or with arbitrary chosen training vectors” belonging to the remaining classes.

For simple problems, the method 1 or 2 above is suitable. For more complex problems, the method 3 provides a systematic approach for LVQ weight initialization.

Example 7.1 (LVQ for clustering vectors) Matlab

A learning vector quantization network as shown in Figure E7.1.1 is used to cluster 4 vectors into 2 classes as defined by Table E7.1.1.

Assuming $\mathbf{X}_1$ and $\mathbf{X}_3$ are selected as the initial weights of two different clusters as shown in Table E7.1.2; and the updated weights are used immediately for each input vector presentation.

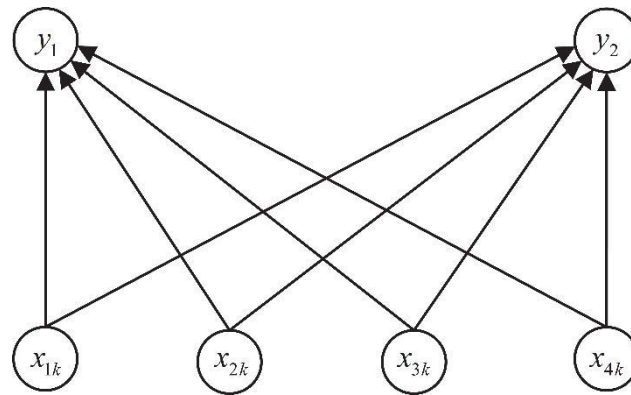


Figure E7.1.1 4-input 2-output LVQ network.

Table E7.1.1 Clustering vectors using LVQ.

k	$\mathbf{X}_k$	T_k
1	[1 0 0 0]	1
2	[0 1 0 0]	1
3	[0 0 1 0]	2
4	[0 0 0 1]	2

Table E7.1.2 Initial weights for LVQ.

j	$\mathbf{W}_j$	C_j
1	[1 0 0 0]	1
2	[0 0 1 0]	2

(a) With $g(1) = 0.5$, $g(t+1) = 0.5 g(t)$, show the step-by-step calculations up to the end of iteration 2.

(b) With $g(1) = 0.5$, $g(t+1) = 0.99 g(t)$, write a Matlab program to compute the weight vectors and verify the recall correctness at the outputs after 100 iterations.

Compare the recalled outputs with those obtained in Example 6.9 using the SOM.

Solution

(a) Iteration 1

Presenting the input $\mathbf{X}_1 = [1\ 0\ 0\ 0]$ of class T_1 :

$$d_1 = (1 - 1.00)^2 + (0 - 0.00)^2 + (0 - 0.00)^2 + (0 - 0.00)^2 \\ = 0.00$$

$$d_2 = (1 - 0.00)^2 + (0 - 0.00)^2 + (0 - 1.00)^2 + (0 - 0.00)^2 \\ = 2.00$$

 d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C_1 will be updated.The input $\mathbf{X}_1$ and the weight $\mathbf{W}_1$ are of the same class,

$$\mathbf{W}_1 = [1.00\ 0.00\ 0.00\ 0.00] \\ + 0.5 \times ([1\ 0\ 0\ 0] - [1.00\ 0.00\ 0.00\ 0.00]) \\ = [1.00\ 0.00\ 0.00\ 0.00]$$

Presenting the input $\mathbf{X}_2 = [0\ 1\ 0\ 0]$ of class T_1 :

$$d_1 = (0 - 1.00)^2 + (1 - 0.00)^2 + (0 - 0.00)^2 + (0 - 0.00)^2 \\ = 2.00$$

$$d_2 = (0 - 0.00)^2 + (1 - 0.00)^2 + (0 - 1.00)^2 + (0 - 0.00)^2 \\ = 2.00$$

 $d_1 = d_2$, both the weight vector $\mathbf{W}_1$ of class C_1 and the weight vector $\mathbf{W}_2$ of class C_2 will be updated.The input $\mathbf{X}_2$ and the weight $\mathbf{W}_1$ are of the same class,

$$\mathbf{W}_1 = [1.00\ 0.00\ 0.00\ 0.00] \\ + 0.5 \times ([0\ 1\ 0\ 0] - [1.00\ 0.00\ 0.00\ 0.00]) \\ = [0.50\ 0.50\ 0.00\ 0.00]$$

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_2$ are of different classes,

$$\mathbf{W}_2 = [0.00\ 0.00\ 1.00\ 0.00]$$

$$\begin{aligned}
 & -0.5 \times ([0 \ 1 \ 0 \ 0] - [0.00 \ 0.00 \ 1.00 \ 0.00]) \\
 & = [0.00 \ -0.50 \ 1.50 \ 0.00]
 \end{aligned}$$

Presenting the input $\mathbf{X}_3 = [0 \ 0 \ 1 \ 0]$ of class T_2 :

$$\begin{aligned}
 d_1 &= (0 - 0.50)^2 + (0 - 0.50)^2 + (1 - 0.00)^2 + (0 - 0.00)^2 \\
 &= 1.50
 \end{aligned}$$

$$\begin{aligned}
 d_2 &= (0 - 0.00)^2 + (0 + 0.50)^2 + (1 - 1.50)^2 + (0 - 0.00)^2 \\
 &= 0.50
 \end{aligned}$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C_2 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_2$ are of the same class,

$$\begin{aligned}
 \mathbf{W}_2 &= [0.00 \ -0.50 \ 1.50 \ 0.00] \\
 &+ 0.5 \times ([0 \ 0 \ 1 \ 0] - [0.00 \ -0.50 \ 1.50 \ 0.00]) \\
 &= [0.00 \ -0.25 \ 1.25 \ 0.00]
 \end{aligned}$$

Presenting the input $\mathbf{X}_4 = [0 \ 0 \ 0 \ 1]$ of class T_2 :

$$\begin{aligned}
 d_1 &= (0 - 0.50)^2 + (0 - 0.50)^2 + (0 - 0.00)^2 + (1 - 0.00)^2 \\
 &= 1.50
 \end{aligned}$$

$$\begin{aligned}
 d_2 &= (0 - 0.00)^2 + (0 + 0.25)^2 + (0 - 1.25)^2 + (1 - 0.00)^2 \\
 &= 2.63
 \end{aligned}$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C_1 will be updated.

The input $\mathbf{X}_4$ and the weight $\mathbf{W}_1$ are of different classes,

$$\begin{aligned}
 \mathbf{W}_1 &= [0.50 \ 0.50 \ 0.00 \ 0.00] \\
 &- 0.5 \times ([0 \ 0 \ 0 \ 1] - [0.50 \ 0.50 \ 0.00 \ 0.00]) \\
 &= [0.75 \ 0.75 \ 0.00 \ -0.50]
 \end{aligned}$$

After Iteration 1: The weight vectors are

$$\mathbf{W}_1 = [0.75, 0.75, 0.00, -0.50]$$

$$\mathbf{W}_2 = [0.00, -0.25, 1.25, 0.00]$$

Compute $g(2) = 0.5 \times g(1) = 0.25$.

Iteration 2

Presenting the input $\mathbf{X}_1 = [1 \ 0 \ 0 \ 0]$ of class T_1 :

$$\begin{aligned} d_1 &= (1 - 0.75)^2 + (0 - 0.75)^2 + (0 - 0.00)^2 + (0 + 0.50)^2 \\ &= 0.88 \end{aligned}$$

$$\begin{aligned} d_2 &= (1 - 0.00)^2 + (0 + 0.25)^2 + (0 - 1.25)^2 + (0 - 0.00)^2 \\ &= 2.63 \end{aligned}$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C_1 will be updated.

The input $\mathbf{X}_1$ and the weight $\mathbf{W}_1$ are of the same class,

$$\begin{aligned} \mathbf{W}_1 &= [0.75 \ 0.75 \ 0.00 \ -0.50] \\ &+ 0.25 \times ([1 \ 0 \ 0 \ 0] - [0.75 \ 0.75 \ 0.00 \ -0.50]) \\ &= [0.81 \ 0.56 \ 0.00 \ -0.38] \end{aligned}$$

Presenting the input $\mathbf{X}_2 = [0 \ 1 \ 0 \ 0]$ of class T_1 :

$$\begin{aligned} d_1 &= (0 - 0.81)^2 + (1 - 0.56)^2 + (0 - 0.00)^2 + (0 + 0.38)^2 \\ &= 0.99 \end{aligned}$$

$$\begin{aligned} d_2 &= (0 - 0.00)^2 + (1 + 0.25)^2 + (0 - 1.25)^2 + (0 - 0.00)^2 \\ &= 3.13 \end{aligned}$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C_1 will be updated.

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_1$ are of the same class,

$$\begin{aligned} \mathbf{W}_1 &= [0.81 \ 0.56 \ 0.00 \ -0.38] \\ &+ 0.25 \times ([0 \ 1 \ 0 \ 0] - [0.81 \ 0.56 \ 0.00 \ -0.38]) \\ &= [0.61 \ 0.67 \ 0.00 \ -0.28] \end{aligned}$$

Presenting the input $\mathbf{X}_3 = [0 \ 0 \ 1 \ 0]$ of class T_2 :

$$\begin{aligned} d_1 &= (0 - 0.61)^2 + (0 - 0.67)^2 + (1 - 0.00)^2 + (0 + 0.28)^2 \\ &= 1.90 \end{aligned}$$

$$d_2 = (0 - 0.00)^2 + (0 + 0.25)^2 + (1 - 1.25)^2 + (0 - 0.00)^2 \\ = 0.13$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C_2 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_2$ are of the same class,

$$\mathbf{W}_2 = [0.00 \ -0.25 \ 1.25 \ 0.00] \\ + 0.25 \times ([0 \ 0 \ 1 \ 0] - [0.00 \ -0.25 \ 1.25 \ 0.00]) \\ = [0.00 \ -0.19 \ 1.19 \ 0.00]$$

Presenting the input $\mathbf{X}_4 = [0 \ 0 \ 0 \ 1]$ of class T_2 :

$$d_1 = (0 - 0.61)^2 + (0 - 0.67)^2 + (0 - 0.00)^2 + (1 + 0.28)^2 \\ = 2.46$$

$$d_2 = (0 - 0.00)^2 + (0 + 0.19)^2 + (0 - 1.19)^2 + (1 - 0.00)^2 \\ = 2.45$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C_2 will be updated.

The input $\mathbf{X}_4$ and the weight $\mathbf{W}_2$ are of the same class,

$$\mathbf{W}_2 = [0.00 \ -0.19 \ 1.19 \ 0.00] \\ + 0.25 \times ([0 \ 0 \ 0 \ 1] - [0.00 \ -0.19 \ 1.19 \ 0.00]) \\ = [0.00 \ -0.14 \ 0.89 \ 0.25]$$

After Iteration 2: The weight vectors are

$$\mathbf{W}_1 = [0.61, 0.67, 0.00, -0.28]$$

$$\mathbf{W}_2 = [0.00, -0.14, 0.89, 0.25]$$

Compute $g(3) = 0.5 \times g(2) = 0.125$.

(b) With $g(0) = 0.5$, $g(t+1) = 0.99 \times g(t)$, after iteration 100, the weight vectors are

$$\mathbf{W}_1 = [0.4480 \ 0.5520 \ -0.0000 \ 0.0000]$$

$$\mathbf{W}_2 = [0.0000 \ -0.0000 \ 0.4480 \ 0.5520]$$

To verify the classification results, compute the output as

$$\mathbf{Y}_k = \mathbf{X}_k \mathbf{W}_j(100) \text{ for } k = 1 \text{ to } 4 \text{ and } j = 1 \text{ to } 2$$

and obtain the results shown in Table E7.1.3.

From the above table, it can be seen that “ $\mathbf{X}_1$ and $\mathbf{X}_2$ ” form cluster 1 and “ $\mathbf{X}_3$ and $\mathbf{X}_4$ ” form cluster 2, and correct input to output classifications are obtained.

The LVQ results of this Example 7.1 are similar to those of the SOM results of Example 6.9.

Table E7.1.3 Classification results obtained by LVQ.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$	T_k
1	[1 0 0 0]	[0.4480, 0.0000]	1
2	[0 1 0 0]	[0.5520, 0.0000]	1
3	[0 0 1 0]	[0.0000, 0.4480]	2
4	[0 0 0 1]	[0.0000, 0.5520]	2

Example 7.2 (Weight movements after LVQ learning) Matlab

Consider the 4 initial centers on a 2-dimensional plane defined by $[-1 \leq x \leq +1$ and $-1 \leq y \leq +1]$ as shown in Table E7.2.1. Assuming that the updated weights are used only when the next iteration starts.

(a) Given the learning rate parameter $g(1) = 0.5$, show the calculations to illustrate what will happen after presenting the four input vectors as listed in Table E7.2.2.

(b) On the 2-dimensional plane, plot the movement of each of the four centers by joining the initial center to the final center by an arrow.

Table E7.2.1 Initial centers for LVQ.

c	$[x_c, y_c]$	C_c
1	[0.5, 0.5]	1
2	[-0.5, 0.5]	2
3	[-0.5, -0.5]	3
4	[0.5, -0.5]	4

Table E7.2.2 Clustering 2-input vectors using LVQ.

k	$[x_k, y_k]$	T_k
$\mathbf{X}_1$	[0.4, 0.2]	1
$\mathbf{X}_2$	[-0.3, 0.7]	4
$\mathbf{X}_3$	[-0.1, -0.9]	3
$\mathbf{X}_4$	[0.3, -0.2]	2

Solution

(a) Presenting the first input vector $\mathbf{X}_1 = [0.40 \ 0.20]$ of class T_1 :

$$d_1 = (0.4 - 0.5)^2 + (0.2 - 0.5)^2 = 0.1$$

$$d_2 = (0.4 + 0.5)^2 + (0.2 - 0.5)^2 = 0.9$$

$$d_3 = (0.4 + 0.5)^2 + (0.2 + 0.5)^2 = 1.3$$

$$d_4 = (0.4 - 0.5)^2 + (0.2 + 0.5)^2 = 0.5$$

d_1 is the smallest, the weight vector $\mathbf{W}_1$ of class C_1 will be updated.

The input $\mathbf{X}_1$ and the weight $\mathbf{W}_1$ are of the same class,

$$\begin{aligned}\mathbf{W}_1 &= [0.5 \ 0.5] + 0.5 \times ([0.4 \ 0.2] - [0.5 \ 0.5]) \\ &= [0.45 \ 0.35]\end{aligned}$$

Presenting the second input vector $\mathbf{X}_2 = [-0.30 \ 0.70]$ of class T_4 :

$$d_1 = (-0.3 - 0.5)^2 + (0.7 - 0.5)^2 = 0.68$$

$$d_2 = (-0.3 + 0.5)^2 + (0.7 - 0.5)^2 = 0.08$$

$$d_3 = (-0.3 + 0.5)^2 + (0.7 + 0.5)^2 = 1.48$$

$$d_4 = (-0.3 - 0.5)^2 + (0.7 + 0.5)^2 = 2.08$$

d_2 is the smallest, the weight vector $\mathbf{W}_2$ of class C_2 will be updated.

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_2$ are of different classes,

$$\begin{aligned}\mathbf{W}_2 &= [-0.5 \ 0.5] - 0.5 \times ([-0.3 \ 0.7] - [-0.5 \ 0.5]) \\ &= [-0.60 \ 0.40]\end{aligned}$$

Presenting the third input vector $\mathbf{X}_3 = [-0.10 \ -0.90]$ of class T_3 :

$$d_1 = (-0.1 - 0.5)^2 + (-0.9 - 0.5)^2 = 2.32$$

$$d_2 = (-0.1 + 0.5)^2 + (-0.9 - 0.5)^2 = 2.12$$

$$d_3 = (-0.1 + 0.5)^2 + (-0.9 + 0.5)^2 = 0.32$$

$$d_4 = (-0.1 - 0.5)^2 + (-0.9 + 0.5)^2 = 0.52$$

d_3 is the smallest, the weight vector $\mathbf{W}_3$ of class C_3 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_3$ are of the same class,

$$\begin{aligned}\mathbf{W}_3 &= [-0.5 \ -0.5] + 0.5 \times ([-0.1 \ -0.9] - [-0.5 \ -0.5]) \\ &= [-0.30 \ -0.70]\end{aligned}$$

Presenting the fourth input vector $\mathbf{X}_4 = [0.30 \ -0.20]$ of class T_2 :

$$d_1 = (0.3 - 0.5)^2 + (-0.2 - 0.5)^2 = 0.53$$

$$d_2 = (0.3 + 0.5)^2 + (-0.2 - 0.5)^2 = 1.13$$

$$d_3 = (0.3 + 0.5)^2 + (-0.2 + 0.5)^2 = 0.73$$

$$d_4 = (0.3 - 0.5)^2 + (-0.2 + 0.5)^2 = 0.13$$

d_4 is the smallest, the weight vector $\mathbf{W}_4$ of class C_4 will be updated.

The input $\mathbf{X}_4$ and the weight $\mathbf{W}_4$ are of different classes,

$$\begin{aligned}\mathbf{W}_4 &= [0.5 \quad -0.5] - 0.5 \times ([0.3 \quad -0.2] - [0.5 \quad -0.5]) \\ &= [0.60 \quad -0.65]\end{aligned}$$

After one iteration, the new weight vectors become:

$$\mathbf{W}_1 = [0.45 \quad 0.35]$$

$$\mathbf{W}_2 = [-0.60 \quad 0.40]$$

$$\mathbf{W}_3 = [-0.30 \quad -0.70]$$

$$\mathbf{W}_4 = [0.60 \quad -0.65]$$

(b) Figure E7.2 shows the movements of the weights after presenting the four input vectors.

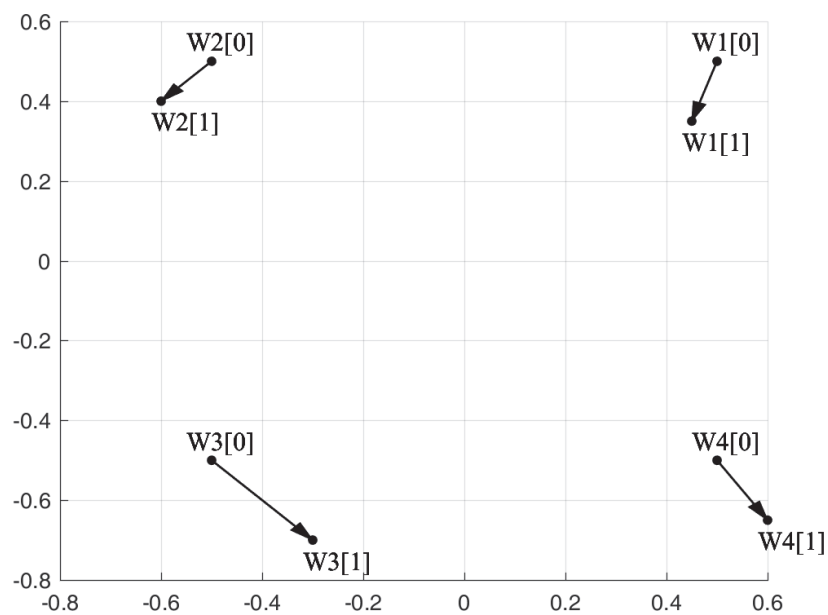


Figure E7.2.1 Movements of original weights after four 2-input vector learning.

Example 7.3 (XOR problem cannot be learnt by LVQ classifier) Matlab

A learning vector quantization network with a structure of 2 inputs with 2 output classes “0” and “1” is given.

Consider the XOR problem shown in Table E7.3.1, assume $\mathbf{X}_1$ and $\mathbf{X}_2$ are chosen as the two initial weight vectors as shown in Table E7.3.2; the latest weight vectors are used within an iteration; the learning rate $g(1) = 0.5$ and $g(t + 1) = 0.5 \times g(t)$:

- Show the step-by-step calculations to obtain the weights of the LVQ.
- After learning, verify the classification correctness of the LVQ and tabulate them in Table E7.3.3.

Table E7.3.1 XOR problem.

k	$\mathbf{X}_k$	T_k
1	[1, 1]	1
2	[0, 1]	2
3	[0, 0]	1
4	[1, 0]	2

Table E7.3.2 Initial weights for XOR problem.

j	$\mathbf{W}_j$	C_j
1	[1, 1]	1
2	[0, 1]	2

Solution

(a) Iteration 1

Since $\mathbf{X}_1$ and $\mathbf{X}_2$ have been chosen as the initial weight vectors, these two vectors are not required to be presented at Iteration 1.

Presenting the input $\mathbf{X}_3 = [0, 0]$ of class T_1 :

$$d_1 = (0 - 1)^2 + (0 - 1)^2 = 2$$

$$d_2 = (0 - 0)^2 + (0 - 1)^2 = 1$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C_2 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_2$ are of different classes,

$$\mathbf{W}_2 = [0 \ 1] - 0.5 \times ([0 \ 0] - [0 \ 1]) = [0.00 \ 1.50]$$

Presenting the input $\mathbf{X}_4 = [1, 0]$ of class T_2 :

$$d_1 = (1 - 1)^2 + (0 - 1)^2 = 1$$

$$d_2 = (1 - 0)^2 + (0 - 1.5)^2 = 3.25$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C_1 will be updated.

The input $\mathbf{X}_4$ and the weight $\mathbf{W}_1$ are of different classes,

$$\mathbf{W}_1 = [1 \ 1] - 0.5 \times ([1 \ 0] - [1 \ 1]) = [1.00 \ 1.50]$$

After Iteration 1: The weight vectors are:

$$\mathbf{W}_1 = [1.00 \ 1.50]$$

$$\mathbf{W}_2 = [0.00 \ 1.50]$$

Compute $g(2) = 0.5 \times g(1) = 0.25$.

Iteration 2

Presenting the input $\mathbf{X}_1 = [1, 1]$ of class T_1 :

$$d_1 = (1 - 1)^2 + (1 - 1.5)^2 = 0.25$$

$$d_2 = (1 - 0)^2 + (1 - 1.5)^2 = 1.25$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C_1 will be updated.

The input $\mathbf{X}_1$ and the weight $\mathbf{W}_1$ are of the same class,

$$\mathbf{W}_1 = [1 \ 1.5] + 0.25 \times ([1 \ 1] - [1 \ 1.5]) = [1.00 \ 1.375]$$

Presenting the input $\mathbf{X}_2 = [0, 1]$ of class T_2 :

$$d_1 = (0 - 1)^2 + (1 - 1.375)^2 = 1.14063$$

$$d_2 = (0 - 0)^2 + (1 - 1.5)^2 = 0.25$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C_2 will be updated.

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_2$ are of the same class,

$$\mathbf{W}_2 = [0 \ 1.5] + 0.25 \times ([0 \ 1] - [0 \ 1.5]) = [0.00 \ 1.375]$$

Presenting the input $\mathbf{X}_3 = [0, 0]$ of class T_1 :

$$d_1 = (0 - 1)^2 + (0 - 1.375)^2 = 2.89063$$

$$d_2 = (0 - 0)^2 + (0 - 1.375)^2 = 1.89063$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C_2 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_2$ are of different classes,

$$\begin{aligned} \mathbf{W}_2 &= [0 \ 1.375] - 0.25 \times ([0 \ 0] - [0 \ 1.375]) \\ &= [0.00 \ 1.71875] \end{aligned}$$

Presenting the input $\mathbf{X}_4 = [1, 0]$ of class T_2 :

$$d_1 = (1 - 1)^2 + (0 - 1.375)^2 = 1.89063$$

$$d_2 = (1 - 0)^2 + (0 - 1.71875)^2 = 3.9541$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C_1 will be updated.

The input $\mathbf{X}_4$ and the weight $\mathbf{W}_1$ are of different classes,

$$\begin{aligned} \mathbf{W}_1 &= [1 \ 1.375] - 0.25 \times ([1 \ 0] - [1 \ 1.375]) \\ &= [1.00 \ 1.71875] \end{aligned}$$

After Iteration 2: The weight vectors are:

$$\mathbf{W}_1 = [1.00 \ 1.71875]$$

$$\mathbf{W}_2 = [0.00 \ 1.71875]$$

Compute $g(3) = 0.5 \times g(2) = 0.125$.

(b) After Iteration 100: The weight vectors are:

$$\mathbf{W}_1 = [1.00 \ 1.95]$$

$$\mathbf{W}_2 = [0.00 \ 1.95]$$

To verify the clustering results, compute the outputs as

$$\mathbf{Y}_k = \mathbf{X}_k \mathbf{W}_j(100) \text{ for } k = 1 \text{ to } 4 \text{ and } j = 1 \text{ to } 2$$

and obtain the results shown in Table E7.3.3.

From the Table E7.3.3, it can be observed that no two distinct clusters can be found for correct classification.

This demonstrates that the XOR problem is a highly nonlinear problem in which no linear classifier can be obtained even by the LVQ learning.

Table E7.3.3 XOR classification results obtained by LVQ.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$	T_k
1	[1, 1]	[2.95, 1.95]	1
2	[0, 1]	[1.95, 1.95]	2
3	[0, 0]	[0.00, 0.00]	1
4	[1, 0]	[1.00, 0.00]	2

7.5 Improved LVQ Algorithms

In the original algorithm of the learning vector quantization, only the reference vector closest to the input vector is to be updated.

In practice, it is possible to have an input vector of approximately equal distance to two other vectors. If this happens, both vectors should be updated.

In the improved algorithms of the learning vector quantization introduced in [Koh9009], [Koh9006] and summarized in [Fau9400], the winner and the runner-up vectors are updated.

Chapter 8

Radial Basis Function Networks

8.1 Introduction

In this chapter, the design of a supervised neural network called the radial basis function (RBF) network is described by treating it as a curve-fitting problem in a high-dimensional space.

Applications of the radial basis function networks can be found in diverse fields including signal processing, speech and image processing, time-series analysis, adaptive equalization, radar point-source location, medical diagnosis, and others.

The topics covered in this chapter include pattern separability, nonlinear transformation for converting a linearly nonseparable problem into a linearly separable problem; Gaussian mapping function for nonlinear transformation; interpolation RBF network; generalized RBF network; and approaches to determine radial basis function centers.

Examples are given to illustrate the design procedures. A general reference for the radial basis function network is given by [Hay9900].

8.2 Radial Basis Function Networks

The basic radial basis function network consists of three different layers: The input layer consisting of N linear input elements; the second layer consisted of P hidden units; and the output layer formed by M output units.

The transformation from the input space to the hidden-unit space is nonlinear, whereas the transformation from the hidden-unit space to the output space is linear.

The dimensions, N and M respectively, of the input layer and the output layer are specified by a given problem; whereas the dimension P

of the hidden-unit space needs to be sufficiently high for the given problem.

The number of hidden neurons P depends on the complexity of the input to output mapping, which can be less than or equal to the number of input training patterns K to be presented to the input layer.

In a radial basis function network, learning is equivalent to finding a surface in a multidimensional space that provides a best fit to a set of given training data, and generalization is equivalent to the use of this multidimensional surface to interpolate a set of untrained test data.

The radial basis function networks and the multilayer feedforward neural networks (MLFNNs) are examples of nonlinear layered feedforward neural networks. Both of them are universal approximators.

However these two networks differ from each other in a number of aspects as listed in Table 8.2.1.

Table 8.2.1 Comparison between RBF networks and MLFNNs.

1	An RBF network has a single hidden layer, whereas a MLFNN can have one or more hidden layers.
2	The hidden layer of an RBF network is nonlinear, whereas the output layer is linear. Both the hidden layer(s) and the output layer of a MLFNN are usually nonlinear.
3	MLFNNs construct global approximations capable of generalization where few training data are available.
4	Each basis function of a RBF represents a prototype and plays a distinct role as compared to the hidden neurons of a MLFNN.
5	RBF networks construct local approximations capable of fast learning and reduced sensitivity to the presentation order of training data. However, it often requires a sufficiently large number of radial basis neurons to yield a smooth mapping.

8.3 Separability of Patterns

A well designed radial basis function network is a demonstration of the Cover's theorem which indicates that a complex pattern-classification problem is more likely to be linearly separable if cast nonlinearly and in a higher dimensional space.

When a radial basis function network is used to perform a linearly non-separable pattern-classification problem, the problem can usually be solved by first converting it into a linearly separable problem through nonlinear transformation and/or increasing the number of hidden units.

The underlying justification is based on the Cover's theorem regarding separability of patterns. Once we have linearly separable patterns, then the classification problem is relatively easy to solve.

Examples of the type of separating surfaces are illustrated in Figures 8.3.1-8.3.2 for a configuration of six points located in a 2-dimensional input space.

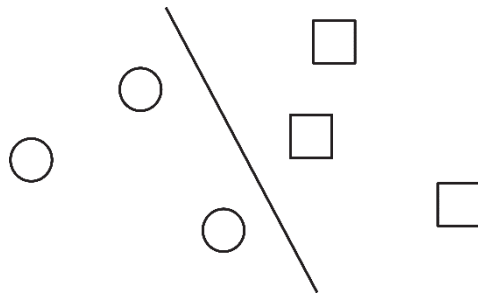


Figure 8.3.1 Linearly separable problem.

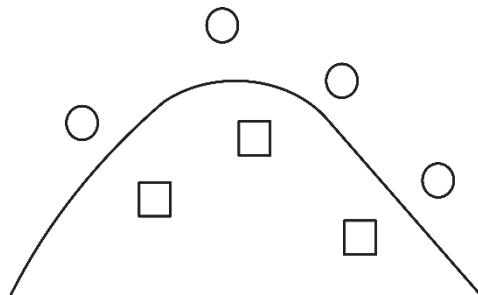


Figure 8.3.2 Linearly nonseparable problem.

Given a set of patterns $\mathbf{X}$ in an input space of an arbitrary dimension N , a nonlinear transformation of $\mathbf{X}$ to $\varphi(\mathbf{X})$ of a high enough dimension P can usually be found such that linear separability can be achieved in the φ space with

$$[\varphi_1(\mathbf{X}), \varphi_2(\mathbf{X}), \varphi_3(\mathbf{X}), \dots, \varphi_P(\mathbf{X})] \quad (8.3.1)$$

The Cover's theorem on separability of patterns consists of two components:

- (a) Nonlinear transformation: Transformation of an input vector $\mathbf{X}$ of dimension N to a hidden layer consisting of P nonlinear functions defined by $\varphi_p(\mathbf{X})$ for $p = 1, 2, \dots, P$.
- (b) Depending on the complexity of a classification problem, P should be sufficiently high.

It is noted that in some cases the use of nonlinear transformation is sufficient to produce linear separability without having to increase the dimensionality P of the hidden layer.

8.4 Gaussian Mapping Function

Using a Gaussian function, the output of the p th nonlinear hidden function $\varphi_p(\mathbf{X}_k)$ due to an input vector $\mathbf{X}_k$ can be obtained as

$$\begin{aligned} \varphi_{kp} &= \varphi_p(\mathbf{X}_k) = \varphi(\|\mathbf{X}_k - \mathbf{T}_p\|) \\ &= \exp\left(-\|\mathbf{X}_k - \mathbf{T}_p\|_2^2\right) = \exp\left(-[\mathbf{X}_k - \mathbf{T}_p][\mathbf{X}_k - \mathbf{T}_p]^T\right) \end{aligned} \quad (8.4.1)$$

for $k = 1$ to K and $p = 1$ to P

In (8.4.1), $\|\mathbf{X}\|_2 = \sqrt{\mathbf{X}\mathbf{X}^T}$ denotes the l_2 -norm of a row vector $\mathbf{X}$ which is equal to the positive square root of the scalar product of $\mathbf{X}$ with itself. For convenience, squared l_2 -norm of a row vector $\mathbf{X}$ as shown in (8.4.1) can be used.

For simplicity, the l_2 -norm of a row vector $\mathbf{X}$ is represented by $\|\mathbf{X}\|$ and the squared l_2 -norm of a row vector $\mathbf{X}$ is represented by $\|\mathbf{X}\|^2$.

Example 8.1 (XOR problem solved by nonlinear transform)

In the XOR problem as shown in Table E8.1.1 and Figure E8.1.1, there are four patterns (or points), $[0, 0]$, $[1, 1]$, $[0, 1]$, and $[1, 0]$, in a 2-dimensional input space.

The XOR problem requirement is to construct a pattern classifier that produces the binary output 0 in response to the input pattern $[0, 0]$ or $[1, 1]$ with identical elements; and the binary output 1 in response to the input pattern $[0, 1]$ or $[1, 0]$ with different elements.

Table E8.1.1 XOR input to output mapping.

k	$\mathbf{X}_k$	y_k
1	$[0 \ 0]$	0
2	$[0 \ 1]$	1
3	$[1 \ 0]$	1
4	$[1 \ 1]$	0

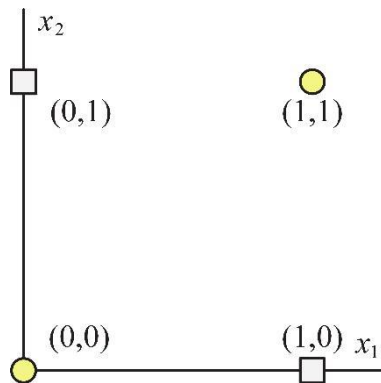


Figure E8.1.1 Linearly nonseparable XOR problem shown on 2-dimensional plane.

Solution

Adopt a pair of Gaussian hidden functions as follows

$$\varphi_1(\mathbf{X}_k) = \exp(-\|\mathbf{X}_k - \mathbf{T}_1\|^2) \text{ at } \mathbf{T}_1 = [1,1]$$

$$\varphi_2(\mathbf{X}_k) = \exp(-\|\mathbf{X}_k - \mathbf{T}_2\|^2) \text{ at } \mathbf{T}_2 = [0,0]$$

By applying an input pattern $\mathbf{X}_k$ for $k = 1$ to 4 of Table E8.1.1 to each of the above Gaussian hidden functions with center $\mathbf{T}_p$ for $p = 1, 2$ such that

$$\varphi_p(\mathbf{X}_k) = \exp(-\|\mathbf{X}_k - \mathbf{T}_p\|^2) \text{ for } k = 1, 2, 3, 4, p = 1, 2$$

For $\mathbf{X}_k = [x_{k1}, x_{k2}]$ for $k = 1, 2, 3, 4$ and $\mathbf{T}_p = [t_{p1}, t_{p2}]$:

$$\varphi_p(\mathbf{X}_k) = \exp\left\{-\left[(x_{k1} - t_{p1})^2 + (x_{k2} - t_{p2})^2\right]\right\} \text{ for } p = 1, 2$$

The results obtained are summarized in Table E8.1.2 and plotted in Figure E8.1.2.

Table E8.1.2 XOR classification using nonlinear transform.

k	$\mathbf{X}_k$	$\varphi_1(\mathbf{X}_k), \mathbf{T}_1 = [1, 1]$		$\varphi_2(\mathbf{X}_k), \mathbf{T}_2 = [0, 0]$		y_k
1	[0, 0]	$\exp(-\ [0,0] - [1,1]\ ^2)$	0.135335	$\exp(-\ [0,0] - [0,0]\ ^2)$	1	0
2	[0, 1]	$\exp(-\ [0,1] - [1,1]\ ^2)$	0.367879	$\exp(-\ [0,1] - [0,0]\ ^2)$	0.367879	1
3	[1, 0]	$\exp(-\ [1,0] - [1,1]\ ^2)$	0.367879	$\exp(-\ [1,0] - [0,0]\ ^2)$	0.367879	1
4	[1, 1]	$\exp(-\ [1,1] - [1,1]\ ^2)$	1	$\exp(-\ [1,1] - [0,0]\ ^2)$	0.135335	0

As seen from Figure E8.1.2, the transformed input patterns [0, 0] and [1, 1] are now linearly separable from the remaining input patterns [0, 1] and [1, 0].

Thereafter, the XOR problem can be solved by using the functions $\varphi_1(\mathbf{X}_k)$ and $\varphi_2(\mathbf{X}_k)$ as the inputs to a linear classifier.

In the XOR example, the nonlinearity of the Gaussian hidden function is sufficient to transform the problem into a linearly separable one, and

there is no need for an increase in the dimensionality of the hidden-unit space.

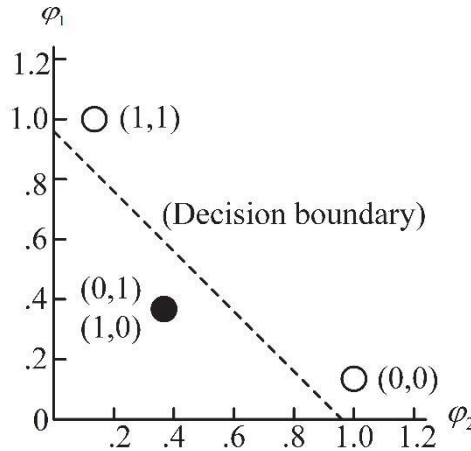


Figure E8.1.2 Linearly separable XOR problem obtained by nonlinear transform.

8.5 Interpolation RBF Network

In the interpolation RBF (IRBF) network shown in Figure 8.5.1, the input is composed of elements correspond to an input vector $\mathbf{X}_k$ of dimensions $N \times 1$.

The hidden layer is composed of nonlinear neurons that are fully connected to all of the input elements. The activation functions of the individual hidden neurons are nonlinear functions.

There is one hidden neuron for each input vector $\mathbf{X}_k$, $k = 1, 2, \dots, K$, where K is the number of training patterns.

The one-to-one correspondence between a training input pattern $\mathbf{X}_k$ and the nonlinear function $\varphi_{kp} = \varphi_p(\mathbf{X}_k)$ for $k, p = 1, 2, \dots, K$ results in an interpolation network that is prohibitively expensive to compute for a large K .

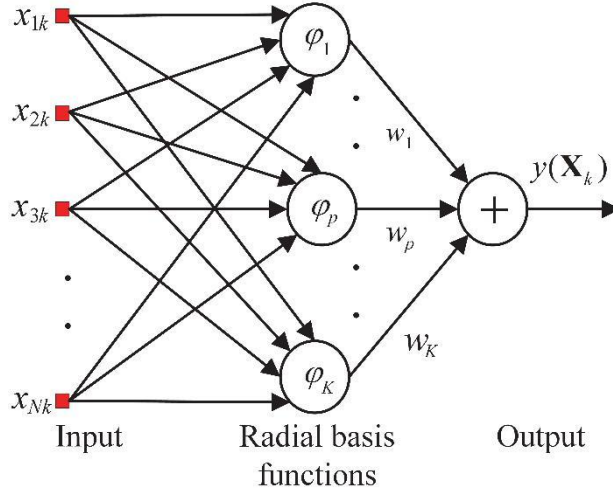


Figure 8.5.1 Interpolation RBF network ($P = K$).

Given a set of K different inputs represented by K vectors each $\mathbf{X}_k$ consisting of N real-value elements as

$$\{\mathbf{X}_k \in R^N | k = 1, 2, \dots, K\} \quad (8.5.1)$$

and a corresponding set of K outputs each represented by a real-value element d_k as

$$\{d_k \in R^1 | k = 1, 2, \dots, K\} \quad (8.5.2)$$

The problem is to find a function that satisfies the interpolation condition such that

$$F(\mathbf{X}_k) = d_k \text{ for } k = 1, 2, \dots, K \quad (8.5.3)$$

In a RBF network, F has the form given by

$$F(\mathbf{X}_k) = \sum_{p=1}^K w_p \varphi_p(\mathbf{X}_k) \quad (8.5.4)$$

$$\varphi_p(\mathbf{X}_k) = \varphi(\|\mathbf{X}_k - \mathbf{X}_p\|) \text{ for } p = 1, 2, \dots, K \quad (8.5.5)$$

where $\varphi_p(\mathbf{X}_k)$ is a set of K nonlinear functions known as radial basis functions. $\mathbf{X}_p \in \mathbf{R}^N \mid p = 1, 2, \dots, K$ are taken to be the centers of P radial basis functions, and $\mathbf{X}_k$ is the k th input vector.

Inserting the interpolation conditions of (8.5.3) into (8.5.4)-(8.5.5), we obtain

$$\begin{bmatrix} \varphi_{11} & \varphi_{12} & \dots & \varphi_{1P} \\ \varphi_{21} & \varphi_{22} & \dots & \varphi_{2P} \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_{K1} & \varphi_{K2} & \dots & \varphi_{KP} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_P \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_K \end{bmatrix} \quad \text{for } P = K \quad (8.5.6)$$

where

$$\varphi_{kp} = \varphi_p(\mathbf{X}_k) = \varphi(\|\mathbf{X}_k - \mathbf{X}_p\|) \quad \text{for } k, p = 1, 2, \dots, K \quad (8.5.7)$$

In φ_{kp} , k is the index of an input vector and p is the index of a radial basis function. Let

$$\mathbf{D} = [d_1, d_2, \dots, d_K]^T \quad (8.5.8)$$

$$\mathbf{W} = [w_1, w_2, \dots, w_K]^T \quad (8.5.9)$$

$$\Phi = \{\varphi_{kp} \mid k, p = 1, 2, \dots, K\} \quad (8.5.10)$$

where $\mathbf{D}$ denotes the desired output response vector, $\mathbf{W}$ denotes the linear weight vector connecting the hidden layer and the output layer, and Φ denotes the interpolation matrix. (8.5.6) can be rewritten in a compact matrix form as

$$\Phi \mathbf{W} = \mathbf{D} \quad (8.5.11)$$

Suppose that the data points are all distinct, the interpolation matrix Φ is positive definite, and so we can solve the above equation and obtain the weight vector $\mathbf{W}$ as

$$\mathbf{W} = \Phi^{-1} \mathbf{D} \quad (8.5.12)$$

For a large $K \times P$ matrix Φ , it could be arbitrarily close to singular, $\mathbf{W}$ cannot be solved using the set of equations given in (8.5.12).

Under this circumstance, regularization theory is applied to solve the problem by perturbing the matrix Φ and replace it by $\Phi + \lambda \mathbf{I}$.

Example 8.2 (XOR problem solved by interpolation RBF network)
Matlab

In this example, we design an interpolation RBF network to map the four input-output pattern-pairs of the XOR problem as defined in Table E8.1.1.

Using the Gaussian function as the nonlinear basis function, an exact solution is considered using an RBF network with four hidden units, with each radial basis function center set to one of the input patterns.

From Table E8.1.1, the four input patterns are $[0, 0]$, $[0, 1]$, $[1, 0]$, and $[1, 1]$, which represent the corners of a square. Illustrate the design procedures by

- (a) Constructing the interpolation matrix Φ for the resulting network to compute the inverse matrix of Φ^{-1} ; and then
- (b) Calculating the linear weights $\mathbf{W}$ for the output layer of the network.

Solution

Based on the four input patterns, the four RBF centers are obtained as

$$\mathbf{T}_1 = \mathbf{X}_1 = [0, 0], \mathbf{T}_2 = \mathbf{X}_2 = [0, 1],$$

$$\mathbf{T}_3 = \mathbf{X}_3 = [1, 0], \mathbf{T}_4 = \mathbf{X}_4 = [1, 1].$$

Using the Gaussian function consisting of squared l_2 -norm operation as the nonlinear basis function, the elements of the interpolation matrix can be calculated as

$$\begin{aligned} \varphi_{11} &= \exp(-\|\mathbf{X}_1 - \mathbf{T}_1\|^2) = \exp(-\|[0, 0] - [0, 0]\|^2) = \exp(-(0)^2) \\ &= 1.0000 \end{aligned}$$

$$\begin{aligned}\varphi_{12} &= \exp(-\|\mathbf{X}_1 - \mathbf{T}_2\|^2) = \exp(-\|[0, 0] - [0, 1]\|^2) = \exp(-(1)^2) \\ &= 0.3679\end{aligned}$$

$$\begin{aligned}\varphi_{13} &= \exp(-\|\mathbf{X}_1 - \mathbf{T}_3\|^2) = \exp(-\|[0, 0] - [1, 0]\|^2) \\ &= \exp(-(1)^2) = 0.3679\end{aligned}$$

$$\begin{aligned}\varphi_{14} &= \exp(-\|\mathbf{X}_1 - \mathbf{T}_4\|^2) = \exp(-\|[0, 0] - [1, 1]\|^2) \\ &= \exp(-(1.414)^2) = 0.1353\end{aligned}$$

$$\begin{aligned}\varphi_{21} &= \exp(-\|\mathbf{X}_2 - \mathbf{T}_1\|^2) = \exp(-\|[0, 1] - [0, 0]\|^2) \\ &= \exp(-(1)^2) = 0.3679\end{aligned}$$

$$\begin{aligned}\varphi_{22} &= \exp(-\|\mathbf{X}_2 - \mathbf{T}_2\|^2) = \exp(-\|[0, 1] - [0, 1]\|^2) \\ &= \exp(-(0)^2) = 1.0000\end{aligned}$$

$$\begin{aligned}\varphi_{23} &= \exp(-\|\mathbf{X}_2 - \mathbf{T}_3\|^2) = \exp(-\|[0, 1] - [1, 0]\|^2) \\ &= \exp(-(1.414)^2) = 0.1353\end{aligned}$$

$$\begin{aligned}\varphi_{24} &= \exp(-\|\mathbf{X}_2 - \mathbf{T}_4\|^2) = \exp(-\|[0, 1] - [1, 1]\|^2) \\ &= \exp(-(1)^2) = 0.3679\end{aligned}$$

$$\begin{aligned}\varphi_{31} &= \exp(-\|\mathbf{X}_3 - \mathbf{T}_1\|^2) = \exp(-\|[1, 0] - [0, 0]\|^2) \\ &= \exp(-(1)^2) = 0.3679\end{aligned}$$

$$\begin{aligned}\varphi_{32} &= \exp(-\|\mathbf{X}_3 - \mathbf{T}_2\|^2) = \exp(-\|[1, 0] - [0, 1]\|^2) \\ &= \exp(-(1.414)^2) = 0.1353\end{aligned}$$

$$\begin{aligned}\varphi_{33} &= \exp(-\|\mathbf{X}_3 - \mathbf{T}_3\|^2) = \exp(-\|[1, 0] - [1, 0]\|^2) \\ &= \exp(-(0)^2) = 1.0000\end{aligned}$$

$$\begin{aligned}\varphi_{34} &= \exp(-\|\mathbf{X}_3 - \mathbf{T}_4\|^2) = \exp(-\|[1, 0] - [1, 1]\|^2) \\ &= \exp(-(1)^2) = 0.3679\end{aligned}$$

$$\begin{aligned}\varphi_{41} &= \exp(-\|\mathbf{X}_4 - \mathbf{T}_1\|^2) = \exp(-\|[1, 1] - [0, 0]\|^2) \\ &= \exp(-(1.414)^2) = 0.1353\end{aligned}$$

$$\begin{aligned}\varphi_{42} &= \exp(-\|\mathbf{X}_4 - \mathbf{T}_2\|^2) = \exp(-\|[1, 1] - [0, 1]\|^2) \\ &= \exp(-(1)^2) = 0.3679\end{aligned}$$

$$\begin{aligned}\varphi_{43} &= \exp(-\|\mathbf{X}_4 - \mathbf{T}_3\|^2) = \exp(-\|[1, 1] - [1, 0]\|^2) \\ &= \exp(-(1)^2) = 0.3679\end{aligned}$$

$$\begin{aligned}\varphi_{44} &= \exp(-\|\mathbf{X}_4 - \mathbf{T}_4\|^2) = \exp(-\|[1, 1] - [1, 1]\|^2) \\ &= \exp(-(0)^2) = 1.0000\end{aligned}$$

The interpolation matrix is obtained as

$$\Phi = \begin{bmatrix} 1.0000 & 0.3679 & 0.3679 & 0.1353 \\ 0.3679 & 1.0000 & 0.1353 & 0.3679 \\ 0.3679 & 0.1353 & 1.0000 & 0.3679 \\ 0.1353 & 0.3679 & 0.3679 & 1.0000 \end{bmatrix}$$

The inverse matrix of Φ is

$$\Phi^{-1} = \begin{bmatrix} 1.3376 & -0.4922 & -0.4922 & 0.1812 \\ -0.4922 & 1.3376 & 0.1812 & -0.4922 \\ -0.4922 & 0.1812 & 1.3376 & -0.4922 \\ 0.1812 & -0.4922 & -0.4922 & 1.3376 \end{bmatrix}$$

From (8.5.12), $\mathbf{W} = \Phi^{-1}\mathbf{D}$, and $\mathbf{D} = [0 \ 1 \ 1 \ 0]^T$, the weight matrix is obtained as $\mathbf{W} = [-0.9843 \ 1.5188 \ 1.5188 \ -0.9843]^T$.

It can be verified that the obtained $\mathbf{W}$ can recall $\mathbf{D}$ correctly from $\mathbf{D} = \Phi \mathbf{W}$ of (8.5.11).

Ill-conditioning matrix: The numerical conditioning of an approximation matrix is defined as the ratio of the largest eigenvalue to the smallest eigenvalue of the matrix. If the value of the ratio increases, the degree of ill-conditioning increases.

Linearly independent functions: A set of radial basis functions $\varphi_1(\mathbf{X})$, $\varphi_2(\mathbf{X})$, ..., $\varphi_n(\mathbf{X})$ is said to be linearly independent if none of the functions in the set can be expressed as a linear combination of one or more of the other functions in the set.

Therefore a necessary and sufficient condition for the set of functions to be linearly independent is that $c_1\varphi_1(\mathbf{X}) + c_2\varphi_2(\mathbf{X}) + \dots + c_n\varphi_n(\mathbf{X}) = 0$ only when all the scalars c_i are zero.

8.6 Generalized RBF Network

The generalized RBF (GRBF) network shown in Figure 8.6.1 is an approximation of the regularized solution of the interpolation problem of the nonlinear mapping of (8.5.6)-(8.5.11), which can be used to avoid a tedious computation and likelihood of ill-conditioning for a large matrix.

The generalized RBF method provides a suboptimal solution in a reduced-dimensional space as

$$F^*(\mathbf{X}_k) = \sum_{p=1}^P w_p \varphi_p(\mathbf{X}_k) + w_0 \quad (8.6.1)$$

where $\varphi_p(\mathbf{X}_k)$ for $p = 1, 2, \dots, P$ is a set of linearly independent radial basis functions; the number of radial basis functions P is usually less than the number of input vectors ($P < K$); and w_p (for $p = 1, 2, \dots, P$) constitutes a new set of weights.

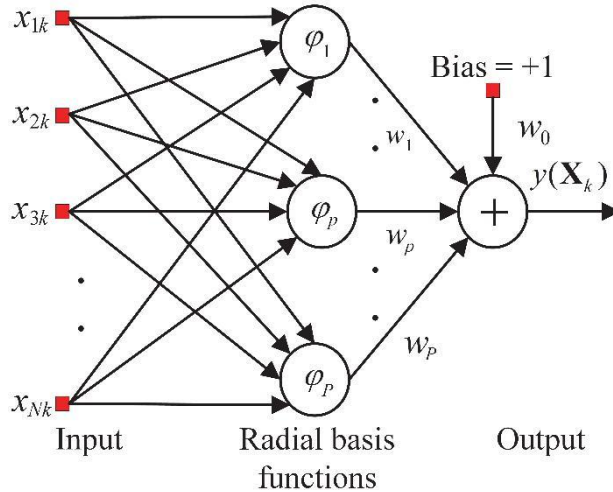


Figure 8.6.1 Generalized RBF network ($P < K$).

In this network, a bias (i.e., data-independent variable) can be applied to the output unit by inserting a linear weight w_0 to the output

layer equal to the bias with an associated radial basis function equal to 1.

8.6.1 For $w_0 = 0$

Let

$$\varphi_p(\mathbf{X}_k) = G(\mathbf{X}_k, \mathbf{T}_p) = G(\|\mathbf{X}_k - \mathbf{T}_p\|) \quad (8.6.2)$$

for $k = 1$ to K and $p = 1$ to P

where the set of centers $\{\mathbf{T}_p | p = 1, 2, \dots, P\}$ is to be determined.

Substituting (8.6.2) into (8.6.1), and with $w_0 = 0$, $F^*(\mathbf{X}_k)$ can be expressed as

$$F^*(\mathbf{X}_k) = \sum_{p=1}^P w_p G(\mathbf{X}_k, \mathbf{T}_p) = \sum_{p=1}^P w_p G(\|\mathbf{X}_k - \mathbf{T}_p\|) \quad (8.6.3)$$

(8.6.3) can be represented as

$$\mathbf{G}\mathbf{W} = \mathbf{D} \text{ or } \mathbf{W} = \mathbf{G}^{-1}\mathbf{D} \quad (8.6.4)$$

and

$$\mathbf{D} = [d_1, d_2, \dots, d_P]^T \quad (8.6.5)$$

$$\mathbf{W} = [w_1, w_2, \dots, w_P]^T \quad (8.6.6)$$

$$\mathbf{G} = \begin{bmatrix} G(\mathbf{X}_1, \mathbf{T}_1) & G(\mathbf{X}_1, \mathbf{T}_2) & \dots & G(\mathbf{X}_1, \mathbf{T}_P) \\ G(\mathbf{X}_2, \mathbf{T}_1) & G(\mathbf{X}_2, \mathbf{T}_2) & \dots & G(\mathbf{X}_2, \mathbf{T}_P) \\ \vdots & \vdots & \ddots & \vdots \\ G(\mathbf{X}_K, \mathbf{T}_1) & G(\mathbf{X}_K, \mathbf{T}_2) & \dots & G(\mathbf{X}_K, \mathbf{T}_P) \end{bmatrix} \quad (8.6.7)$$

Since $\mathbf{G}$ is a $K \times P$ matrix which is a non-square matrix, (8.6.4) represents an over-determined problem in which there are K equations and P unknowns for $K > P$, the resultant solution is not unique.

Using regularization theory, it can be shown that the solution of the (8.6.4) can be derived as

$$(\mathbf{G}^T \mathbf{G} + \lambda \mathbf{G}_0) \mathbf{W} = \mathbf{G}^T \mathbf{D} \quad (8.6.8)$$

$$\mathbf{G}_0 = \begin{bmatrix} G(\mathbf{X}_1, \mathbf{T}_1) & G(\mathbf{X}_1, \mathbf{T}_2) & \cdots & G(\mathbf{X}_1, \mathbf{T}_P) \\ G(\mathbf{X}_2, \mathbf{T}_1) & G(\mathbf{X}_2, \mathbf{T}_2) & \cdots & G(\mathbf{X}_2, \mathbf{T}_P) \\ \vdots & \vdots & \ddots & \vdots \\ G(\mathbf{X}_P, \mathbf{T}_1) & G(\mathbf{X}_P, \mathbf{T}_2) & \cdots & G(\mathbf{X}_P, \mathbf{T}_P) \end{bmatrix} \quad (8.6.9)$$

Note that as the regularization parameter λ approaches zero, the weight vector $\mathbf{W}$ converges to the pseudo-inverse (minimum-norm) solution as

$$\mathbf{W} = \mathbf{G}^+ \mathbf{D} \text{ for } \lambda = 0 \quad (8.6.10)$$

where $\mathbf{G}^+$ is the pseudo-inverse of the matrix $\mathbf{G}$ which can be expressed as

$$\mathbf{G}^+ = (\mathbf{G}^T \mathbf{G})^{-1} \mathbf{G}^T \quad (8.6.11)$$

The generalized RBF network differs from the interpolation RBF network in having a reduced number of hidden units.

The number of units in the hidden layer of the generalized RBF network is P , which is smaller than K (the number of training patterns).

On the other hand, the number of hidden units in the interpolation RBF network is K .

8.6.2 For $w_0 \neq 0$

Assuming that a non-zero bias w_0 is added as shown in Figure 8.6.1, (8.6.3) can be expressed as

$$\begin{aligned} F^*(\mathbf{X}_k) &= \sum_{p=1}^P w_p G(\mathbf{X}_k, \mathbf{T}_p) + w_0 \\ &= \sum_{p=1}^P w_p G(\|\mathbf{X}_k - \mathbf{T}_p\|) + w_0 \end{aligned} \quad (8.6.12)$$

Combining w_0 with $\mathbf{W}$, we obtain

$$\mathbf{W}_c = [w_0, w_1, w_2, \dots, w_P]^T \quad (8.6.13)$$

From (8.6.12)-(8.6.13), we obtain

$$\mathbf{H}\mathbf{W}_c = \mathbf{D} \text{ or } \mathbf{W}_c = \mathbf{H}^{-1}\mathbf{D} \quad (8.6.14)$$

and

$$\mathbf{H} = \begin{bmatrix} 1 & G(\mathbf{X}_1, \mathbf{T}_1) & G(\mathbf{X}_1, \mathbf{T}_2) & \cdots & G(\mathbf{X}_1, \mathbf{T}_P) \\ 1 & G(\mathbf{X}_2, \mathbf{T}_1) & G(\mathbf{X}_2, \mathbf{T}_2) & \cdots & G(\mathbf{X}_2, \mathbf{T}_P) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & G(\mathbf{X}_K, \mathbf{T}_1) & G(\mathbf{X}_K, \mathbf{T}_2) & \cdots & G(\mathbf{X}_K, \mathbf{T}_P) \end{bmatrix} \quad (8.6.15)$$

Since $\mathbf{H}$ is a $K \times (P + 1)$ matrix which is not a square matrix, (8.6.14) represents an over-determined problem in which there are K equations and $P + 1$ unknowns for $K > (P + 1)$, the resultant solution is not unique.

Using regularization theory and in a similar manner as (8.6.8)-(8.6.11), it can be shown that the solution of the (8.6.14) can be derived as

$$\mathbf{W}_c = \mathbf{H}^+\mathbf{D} \quad (8.6.16)$$

$\mathbf{H}^+$ represents the pseudo-inverse of a matrix $\mathbf{H}$ which is given by

$$\mathbf{H}^+ = (\mathbf{H}^T\mathbf{H})^{-1}\mathbf{H}^T \quad (8.6.17)$$

Equation (8.6.17) is a system of over-determined linear equations in which $\mathbf{H}^+$ is the coefficient matrix, $\mathbf{D}$ is the known output vector, and $\mathbf{W}_c$ is the unknown weight vector.

Since the number of equations K is greater than the number of unknowns $P + 1$, there is no unique solution for $\mathbf{W}_c$.

So far, we have discussed N -input and 1-output RBF networks. In practice, the number of output nodes M of a RBF network can be greater than one ($M > 1$).

In general, with a zero bias at each of the M output nodes, the dimensions of $\mathbf{D}$ are $K \times M$, the dimensions of $\mathbf{W}$ are $P \times M$, and the dimensions of $\mathbf{G}^+$ are $P \times K$.

With a non-zero bias at each of the M output nodes, the dimensions of $\mathbf{D}$ remain $K \times M$, whereas the dimensions of $\mathbf{W}_c$ are $(P + 1) \times M$ and the dimensions of $\mathbf{H}^+$ are $(P + 1) \times K$.

Example 8.3 (XOR problem solved by generalized RBF network)
Matlab

In this example, we shall illustrate the design procedures of the generalized RBF network for the XOR problem using one bias and two RBF centers instead of four RBF centers as required in the interpolation RBF network of Example 8.2.

The input-output pattern-pairs of the XOR problem are shown in Table E8.3.1. Using the Gaussian function consisting of squared l_2 -norm operator as the nonlinear function, a pair of Gaussian functions is defined as

$$G(\|\mathbf{X}_k - \mathbf{T}_p\|) = \exp(-\|\mathbf{X}_k - \mathbf{T}_p\|^2) \text{ for } k = 1, 2, 3, 4 \text{ and } p = 1, 2$$

where the two RBF centers are represented by $\mathbf{T}_p$ for $p = 1, 2$ are $\mathbf{T}_1 = [1, 1]$, $\mathbf{T}_2 = [0, 0]$.

Table E8.3.1 XOR input to output mapping.

k	$\mathbf{X}_k$	d_k
1	[1, 1]	0
2	[0, 1]	1
3	[0, 0]	0
4	[1, 0]	1

Solution

The output of the generalized RBF network can be expressed as

$$y(\mathbf{X}_k) = \sum_{p=1}^2 w_p \varphi(\|\mathbf{X}_k - \mathbf{T}_p\|) + b$$

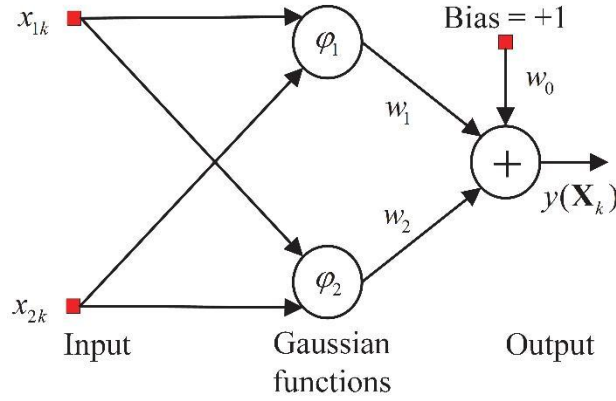


Figure E8.3.1 Generalized RBF network for XOR problem.

Using the Gaussian functions, $y(\mathbf{X}_k)$ becomes

$$y(\mathbf{X}_k) = \sum_{p=1}^2 w_p \exp(-\|\mathbf{X}_k - \mathbf{T}_p\|^2) + w_0$$

To map the input to output patterns of the XOR problem, we require

$$y(\mathbf{X}_k) = d_k \text{ for } k = 1, 2, 3, 4$$

where $\mathbf{X}_k$ is an input vector and d_k is the corresponding desired output.

The radial basis function value at the p hidden unit due to a k th input pattern is equal to

$$g_{kp} = \exp(-\|\mathbf{X}_k - \mathbf{T}_p\|^2) \text{ for } k = 1, 2, 3, 4; p = 1, 2$$

For example, $k = 2, p = 1$:

$$g_{21} = \exp(-\|\mathbf{X}_2 - \mathbf{T}_1\|^2) = \exp(-\|[0,1] - [1,1]\|^2) = 0.3679$$

Repeating the above procedures, the entries of the $\mathbf{G}$ matrix can be obtained as

$$\mathbf{G} = \begin{bmatrix} g_{11} & g_{12} \\ g_{21} & g_{22} \\ g_{31} & g_{32} \\ g_{41} & g_{42} \end{bmatrix} = \begin{bmatrix} 1 & 0.1353 \\ 0.3679 & 0.3679 \\ 0.1353 & 1 \\ 0.3679 & 0.3679 \end{bmatrix}$$

Define the combined weight vector as

$$\mathbf{W}_c = [w_0, w_1, w_2]^T$$

The following matrix equation can be obtained

$$\mathbf{W}_c = \mathbf{H}^+ \mathbf{D}$$

where $\mathbf{H}^+$ represents the pseudo-inverse of the matrix $\mathbf{H}$ which is defined as

$$\mathbf{H}^+ = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T$$

and

$$\mathbf{D} = [0 \ 1 \ 0 \ 1]^T$$

From $\mathbf{G}$, the combined matrix $\mathbf{H}$ can be expressed as

$$\mathbf{H} = \begin{bmatrix} 1 & g_{11} & g_{12} \\ 1 & g_{21} & g_{22} \\ 1 & g_{31} & g_{32} \\ 1 & g_{41} & g_{42} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0.1353 \\ 1 & 0.3679 & 0.3679 \\ 1 & 0.1353 & 1 \\ 1 & 0.3679 & 0.3679 \end{bmatrix}$$

Hence,

$$\mathbf{H}^+ = \begin{bmatrix} -0.9207 & 1.4207 & -0.9207 & 1.4207 \\ 1.8296 & -1.2513 & 0.6731 & -1.2513 \\ 0.6731 & -1.2513 & 1.8296 & -1.2513 \end{bmatrix}$$

Solving $\mathbf{W}_c = \mathbf{H}^+ \mathbf{D}$, $\mathbf{W}_c$ can be obtained as

$$\mathbf{W}_c = \begin{bmatrix} 2.8413 \\ -2.5027 \\ -2.5027 \end{bmatrix}$$

Using the designed values of weights and bias, the actual values of the output vector, $\mathbf{Y} = [y_1, y_2, \dots, y_{K-1}, y_K]^T$ can be verified using $\mathbf{Y} =$

$\mathbf{H} \mathbf{W}_c$. The obtained output values, y_k for $k = 1$ to 4 are listed under the last column of Table E8.3.2.

From the results, it can be observed that the two sets of output weights are equal. This is due to the symmetry of the problem which results having the output unit uses weight sharing, hence there is only a single weight w to determine.

The output unit includes a nonzero bias w_0 because the output values of the XOR function have a nonzero mean of 0.5.

Table E8.3.2 XOR problem classification using GRBF network.

k	$\mathbf{X}_k$	d_k	y_k
1	[1, 1]	0	0
2	[0, 1]	1	1
3	[0, 0]	0	0
4	[1, 0]	1	1

8.7 Determination of Radial Basis Function Centers

Between the interpolation RBF network and the generalized RBF network, the generalized RBF function network with a non-zero bias is a more efficient choice.

If a generalized RBF network is being used to perform more than strict interpolation, such as more general function approximation or classification, the training of the generalized RBF network will become more complex due to the lack of an obvious choice for the radial basis function centers.

Typically, the training of a generalized RBF network is performed in two stages, the first stage is to determine the centers and the width of the radial basis functions, and the second stage is to determine (by the pseudo-inverse method explained above) the values of the linear weights and bias between the hidden layer and the output layer. So far, the second stage has been covered.

Now, the first stage shall be briefly described. The radial basis function centers can be randomly selected among the input patterns or obtained by orthogonal least square learning algorithm or determined by clustering the available input patterns and choosing the cluster means as the centers.

There are different clustering methods, such as the self-organizing map [Cha0608], [Faq1704] (See Example 6.9 as adopted in Example 8.4), the differential evolution [Zho1012], the particle swarm optimization [Esm0905], [Qas0905], the ant colony optimization [Wu0510], [Man0712], and others. The widths of the RBF centers are usually fixed to same value which is proportional to the maximum distance between any two adjacent centers.

The parameters to be determined in an N -input 1-output generalized RBF network are P radial basis function centers ($\mathbf{T}_p$ for $p = 1, 2, 3, \dots, P$) and their widths in the first stage; and a weight vector including a bias $\mathbf{W}_c = [w_0, w_1, w_2, \dots, w_P]$ in the second stage. For a M -output generalized RBF network, there are M sets of $\mathbf{W}_c$.

Design Example 8.4 (Generalized RBF network design for input to output mapping) Matlab

Given the problem on 4-input to 1-output mapping as shown in Table E8.4.1.

Assuming that the Gaussian function consisting of squared l_2 -norm operator are used as the nonlinear function defined by

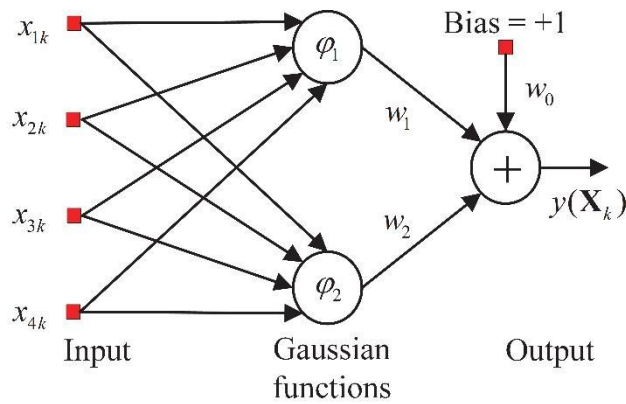
$$G(\|\mathbf{X}_k - \mathbf{T}_p\|) = \exp(-\|\mathbf{X}_k - \mathbf{T}_p\|^2) \text{ for } k = 1, 2, 3, 4 \text{ and } p = 1, 2$$

Two radial basis functions with centers $\mathbf{T}_p$ for $p = 1, 2$ are $\mathbf{T}_1 = [0.4979, 0.5021, 0, 0]$, $\mathbf{T}_2 = [0, 0, 0.4979, 0.5021]$ as determined by clustering in Example 6.9.

Show the design procedures of the generalized RBF network for the above input to output mapping problem using one bias and two RBF centers.

Table E8.4.1 Input (1x4) to output mapping.

k	$\mathbf{X}_k$	d_k
1	[1, 0, 0, 0]	1
2	[0, 1, 0, 0]	1
3	[0, 0, 1, 0]	0
4	[0, 0, 0, 1]	0

Solution**Figure E8.4.1** Generalized RBF network for 4-input to 1-output mapping problem.

The k th output of the generalized RBF network can be expressed as

$$y(\mathbf{X}_k) = \sum_{p=1}^2 w_p \varphi(\|\mathbf{X}_k - \mathbf{T}_p\|) + b$$

Using the Gaussian functions, the k th output $y(\mathbf{X}_k)$ becomes

$$y(\mathbf{X}_k) = \sum_{p=1}^2 w_p \exp(-\|\mathbf{X}_k - \mathbf{T}_p\|^2) + b$$

To map the input to output patterns of the problem, we require

$$y(\mathbf{X}_k) = d_k \text{ for } k = 1, 2, 3, 4$$

where $\mathbf{X}_k$ is an input vector and d_k is the corresponding desired output.

The radial basis function value at the p hidden unit due to a k th input pattern is equal to

$$g_{kp} = \exp\left(-\|\mathbf{X}_k - \mathbf{T}_p\|^2\right) \text{ for } k = 1, 2, 3, 4 \text{ and } p = 1, 2$$

For example, $k = 3, p = 1$:

$$\begin{aligned} g_{31} &= \exp(-\|\mathbf{X}_3 - \mathbf{T}_1\|^2) \\ &= \exp(-\|[0, 0, 1, 0] - [0.4979, 0.5021, 0, 0]\|^2) = 0.2231 \end{aligned}$$

Repeating the above procedures, all the entries of the $\mathbf{G}$ matrix can be obtained as

$$\mathbf{G} = \begin{bmatrix} g_{11} & g_{12} \\ g_{21} & g_{22} \\ g_{31} & g_{32} \\ g_{41} & g_{42} \end{bmatrix} = \begin{bmatrix} 0.6040 & 0.2231 \\ 0.6091 & 0.2231 \\ 0.2231 & 0.6040 \\ 0.2231 & 0.6091 \end{bmatrix}$$

Define the combined weight vector as

$$\mathbf{W}_c = [w_0, w_1, w_2]^T$$

The following matrix equation can be obtained

$$\mathbf{W}_c = \mathbf{H}^+ \mathbf{D}$$

where $\mathbf{H}^+$ represents the pseudo-inverse of the matrix $\mathbf{H}$ which is defined as

$$\mathbf{H}^+ = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T$$

And the desired output vector is

$$\mathbf{D} = [1 \ 1 \ 0 \ 0]^T$$

From $\mathbf{G}$, the combined matrix $\mathbf{H}$ can be expressed as

$$\mathbf{H} = \begin{bmatrix} 1 & g_{11} & g_{12} \\ 1 & g_{21} & g_{22} \\ 1 & g_{31} & g_{32} \\ 1 & g_{41} & g_{42} \end{bmatrix} = \begin{bmatrix} 1 & 0.6040 & 0.2231 \\ 1 & 0.6091 & 0.2231 \\ 1 & 0.2231 & 0.6040 \\ 1 & 0.2231 & 0.6091 \end{bmatrix}$$

Hence,

$$\mathbf{H}^+ = \begin{bmatrix} 81.6717 & -81.1717 & 81.6717 & -81.1717 \\ -97.4911 & 98.7951 & -98.7864 & 97.4824 \\ -98.7864 & 97.4824 & -97.4911 & 98.7951 \end{bmatrix}$$

Solving $\mathbf{W}_c = \mathbf{H}^+ \mathbf{D}$, $\mathbf{W}_c$ can be obtained as

$$\mathbf{W}_c = \begin{bmatrix} 0.5000 \\ 1.3041 \\ -1.3041 \end{bmatrix}$$

Using the designed values of weights and bias, the actual values of the output vector, $\mathbf{Y} = [y_1, y_2, \dots, y_{K-1}, y_K]^T$ can be verified using $\mathbf{Y} = \mathbf{H} \mathbf{W}_c$.

The obtained output values, y_k for $k = 1$ to 4 are listed under the last column of Table E8.4.2.

Table E8.4.2 GRBF network for (1x4) input to output mapping.

k	$\mathbf{X}_k$	d_k	y_k
1	[1, 0, 0, 0]	1	0.9967
2	[0, 1, 0, 0]	1	1.0033
3	[0, 0, 1, 0]	0	0.0033
4	[0, 0, 0, 1]	0	-0.0033

From the results, it can be observed that the recalled outputs are correct and closed to the desired outputs.

Due to the symmetry of the problem, the output unit uses weight sharing, hence there is only one single weight $w_2 = 1.3041$ to determine (differs from $w_3 = -1.3041$ by the sign).

The output unit includes a nonzero bias $w_0 = 0.5$ because the output values of the mapping problem have a nonzero mean of 0.5.

Part III

Fuzzy Neural Computing

III.1 Overview

A fuzzy system processes analog input values expressed in the form of fuzzy sets, and apply fuzzy logic and fuzzy reasoning to produce analog output values in a way that mimics the human thinking process.

Fuzzy sets are sets containing variables with each of them expressed in term of a membership function with a value represented in the real unit interval $[0, 1]$.

Fuzzy logic and reasoning is the driving engine for fuzzy systems, fuzzy neural networks, natural language processing, and other applications. It is a soft-computing technique which processes imprecise or incomplete information expressed in fuzzy sets to produce quick, simple, and sufficiently good decisions or solutions.

A fuzzy system can take either fuzzy inputs or crisp inputs, but the outputs produced are almost always fuzzy. Sometimes it is necessary to have crisp outputs in which defuzzification is required to extract a crisp value that appropriately represents a fuzzy output.

A fuzzy system involves three key operations, namely applying fuzzy if-then rules obtained from expert knowledge or training in which membership functions are used to represent fuzzy sets and fuzzy rules; fuzzy aggregation to induce a fuzzy conclusion from the consequences of all the rules; and defuzzification of the fuzzy conclusion to a crisp output.

A fuzzy system can be designed using expert knowledge in the form of fuzzy if-then rules or through optimization of membership functions and parameters to learn available input to output relationship to acquire fuzzy if-then rules.

The Mamdani [Mam7501], the Tsukamoto [Tsu7900], and the TSK [Tak8501], [Sug8810] fuzzy systems are three popular fuzzy systems with applications in a variety of fields. The differences between these three fuzzy systems lie in the consequents of their fuzzy rules, and thus their aggregation and defuzzification procedures are different.

The Mamdani fuzzy system requires defuzzification whereas the Tsukamoto fuzzy system and the TSK fuzzy system do not require defuzzification.

A fuzzy neural network [Kwa9408], [Cai9806] combines the structure of a neural network with the operations of fuzzy logic to form a new class of intelligent networks. The features of fuzzy neural networks include parallel structure, allowing quick learning, and processing information using fuzzy logic.

Multilayer neural networks, fuzzy systems, and fuzzy neural networks can be trained using given input to output data to solve a problem without requiring a mathematical model of the given problem.

A special feature of fuzzy neural networks is that they have the ability to interpret if-then rules learned from input to output data, which is not straightforward to do so for the case of multilayer neural networks.

Part III on fuzzy neural computing covers four chapters including fuzzy sets, fuzzy logic and reasoning, fuzzy systems, and fuzzy neural networks as presented in Chapters 9 to 12.

Fuzzy systems are constructed based on fuzzy sets, fuzzy logic and reasoning to mimic the human-like thinking.

Fuzzy neural networks form a relative new class of intelligent computing networks by incorporating fuzzy logic and reasoning with the parallel structure of neural networks to process information through the use of fuzzy neurons.

Chapter 9

Fuzzy Sets

9.1 Introduction

Fuzzy sets are sets containing variables with each of them expressed in term of a membership function with a value represented in the real unit interval $[0, 1]$.

This Chapter on fuzzy sets and the Chapter 10 on fuzzy logic and reasoning covers the fundamentals of Chapters 11 and 12 on fuzzy systems and fuzzy neural networks.

In this chapter, a review of crisp sets is first given in Section 9.2. The notations and the operations of fuzzy sets in discrete and continuous universes are explained in Section 9.3.

The terminologies of membership functions are defined in Section 9.4. One-dimensional and two-dimensional parameterized membership functions (MFs) are formulated in Sections 9.5 and 9.6. The derivatives of these parameterized membership functions are given in Section 9.7.

In Section 9.8, linguistic variables, primary and composite linguistic values are used to express meanings of words. A similarity measure is used to measure the degree of similarity between any two fuzzy sets.

In Section 9.9, extension principle is then used to transform one fuzzy set in a multi-dimensional universe (including one-dimensional universe) to another fuzzy set in a multi-dimensional universe through a pre-defined mapping function.

Reference [Zad6506] provides a seminal reference on fuzzy sets. [Jan9700], [Lin9600] provide two other earlier references for fuzzy sets.

9.2 Crisp Sets

In this section, the notations and definitions of crisp sets (classical or nonfuzzy sets) [Kli9500] are introduced and listed in Table 9.2.1.

Table 9.2.1 Notations and definitions of crisp sets.

Notation	Definition
Iff	If and only if.
$x \in A$	An individual object x is a member (or an element) of a set A .
$x \notin A$	x is not a member of a set A .
$\mathbf{Z}$	$\{\dots, -2, -1, 0, 1, 2, \dots\}$, the set of all integers.
$\mathbf{N}$	$\{1, 2, 3, \dots\}$, the set of all positive integers (or natural numbers).
$\mathbf{N}_n$	$\{1, 2, \dots, n\}$, the set of all positive integers up to n .
$\mathbf{N}_0$	$\{0, 1, 2, \dots\}$, the set of all non-negative integers.
$\mathbf{N}_{0,n}$	$\{0, 1, 2, \dots, n\}$, the set of all non-negative integers up to n .
$\mathbf{R}$	The set of all real numbers.
$\mathbf{R}^+$	The set of all non-negative real numbers.
$[a, b] = \{x: a \leq x \leq b\}$ where $a < b$	Closed interval of real numbers between a and b .
$(a, b] = \{x: a < x \leq b\}$ where $a < b$	Left-open interval of real numbers between a and b .
$[a, b) = \{x: a \leq x < b\}$ where $a < b$	Right-open interval of real numbers between a and b .
$(a, b) = \{x: a < x < b\}$	Open interval of real numbers between a and b .

where $a < b$	
$\langle x_1, x_2, \dots, x_n \rangle$	Ordered n -tuple of elements $x_1, x_2, \dots, x_n$.
$\exists$	Existential quantifier, means “there exists a/an”.
$\forall$	Universal quantifier, means “for all”.
$X, Y, Z, \dots$	Sets are denoted by upper-case letters.
$x, y, z, \dots$	Members of a set are denoted by lower-case letters.
$\emptyset$	The empty set.
$ $	Such that.
$A = \{a_1, a_2, \dots, a_n\}$	A set is defined by naming all its finite number of members.
$A = \{x \mid P(x)\}$	A is defined as the set of all the members of X such that the proposition $P(x)$ is true.
$\chi_A(x) = \begin{cases} 1 & x \in A \\ 0 & x \notin A \end{cases}$	Set A is defined by its characteristic function, χ_A , that maps the elements of X to the elements of the set $\{0, 1\}$. For each $x \in X$, when $\chi_A(x) = 1$, x is a member of the set A .
$\{A_i \mid i \in I\}$	A is a family of sets A_i , where i and I are called the set index and the index set.
$A \subseteq B$	A is a subset of B if every member of the set A is also a member of the set B .
$A \supseteq B$	B is a subset of A if every member of the set B is also a member of the set A .
$A = B$	A and B contain the same set of members are called equal sets.
$A \neq B$	A and B are not equal sets.
$A \subset B$	A is called a proper subset of B , where $A \subseteq B$ and $A \neq B$, then the set B contains at least one individual that is not a member of the set A .
$A \supset B$	B is called a proper subset of A , where $A \supseteq B$ and $A \neq B$, then the set A contains at least one

	individual that is not a member of the set B .
$\wp(A)$	The family of all the subsets of a given set A is called the power set of A .
$\wp^2(A)$	The family of all the subsets of $\wp(A)$ is called a second order power set of A , which stands for $\wp(\wp(A))$.
$B - A$	The relative complement of a set A with respect to a set B is the set containing all the members of the set B that are not the members of the set A , thus, $B - A = \{x \mid x \in B \text{ and } x \notin A\}$.
$\bar{A}$	The complement of a set A , in which all the members are not included in the set A .
$ A $	The number of members of a finite set A is called the cardinality of A .
$A \cup B$	The set containing all the elements that belong either to a set A alone, to a set B alone, or to both set A and set B is called the union of the sets A and B , thus, $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$.
$A \cap B$	The set containing all the elements belonging to both a set A and a set B is called the intersection of the sets A and B , thus, $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$.
$r = \sup R$	A real number r is the supremum of R iff (a) r is an upper bound of R ; and (b) no number less than r is an upper bound of R .
$s = \inf R$	A real number s is the infimum of R iff (a) s is a lower bound of R ; and (b) no number greater than s is a lower bound of R .

9.3 Fuzzy Sets

Suppose that A is the set of hot water, in a conventional (or crisp) set,

an element is either belongs to or not belong to a set; there is nothing in between.

Therefore to define a crisp set A , a number say 85 degree Celsius needs to be found, as shown in the Figure 9.3.1(a), such that for water's temperature greater than 85 degree Celsius, the water is in the set of "*hot water*".

This is not meaningful and is an unnatural way of representing the concept of "*hot water*".

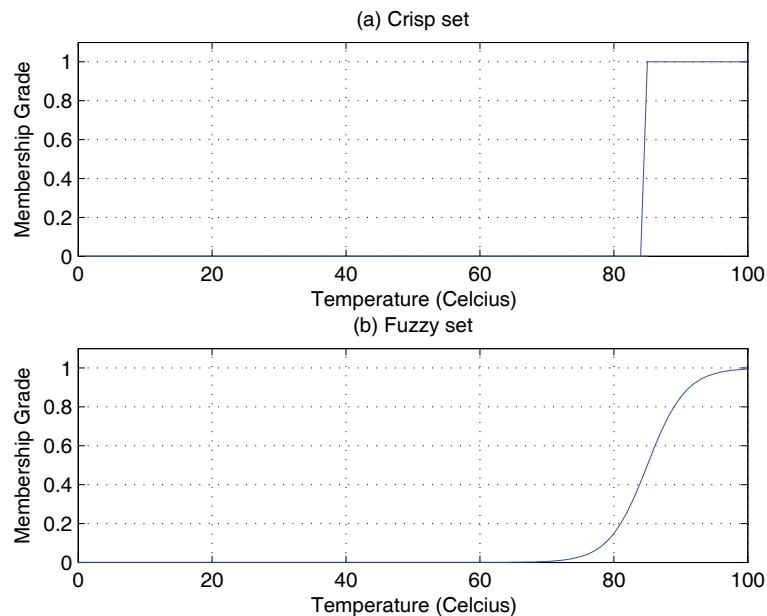


Figure 9.3.1 Crisp set and fuzzy set of hot water: (a) Crisp set; (b) Fuzzy set.

For a fuzzy version of set A called fuzzy set A , the degree of belonging is allowed to vary between 0 and 1. So how hot is "*hot water*" is a matter of degree which offers a more natural way to express the concept of "*hot water*".

As shown in the Figure 9.3.1(b), for water with the temperature of 85 degree Celsius, the water is hot to the degree of 0.5; for 90 degree Celsius, the water is hot to the degree of 0.852; for 80 degree Celsius, the water is hot to the degree of 0.148.

A plot of the degree of belonging (membership grade) to hot water with respect to temperature (a continuous universe of discourse) is a curve called a membership function.

A fuzzy set A in X is defined as a set of ordered pairs such that

$$A = \{[x, \mu_A(x)] | x \in X\} \quad (9.3.1)$$

where X is a collection of members denoted generally by x , A is a fuzzy set, $\mu_A(x)$ is called the membership function (MF) for the fuzzy set A .

The membership function maps each member of X to a membership grade between 0 and 1. X is called the universe (or the universe of discourse).

Due to its smooth transition, a fuzzy set is a better representation of thinking process. A fuzzy set with a step-function-like membership function is a crisp set.

9.3.1 Fuzzy sets with a discrete universe

Fuzzy set $A = \text{"desirable city to work"}$,

$X = \{\text{City 1, City 2, City 3}\}$ discrete and non-ordered

$A = \{[\text{City 1, 0.9}], [\text{City 2, 0.8}], [\text{City 3, 0.7}]\}$

Fuzzy set $B = \text{"number of televisions in an average family"}$,

$X = \{0, 1, 2, 3, 4, 5\}$ discrete universe

$B = \{[0, 0.1], [1, 0.8], [2, 1.0], [3, 0.6], [4, 0.2], [5, 0.1]\}$

Figure 9.3.2(a) plots this fuzzy set B in the discrete universe.

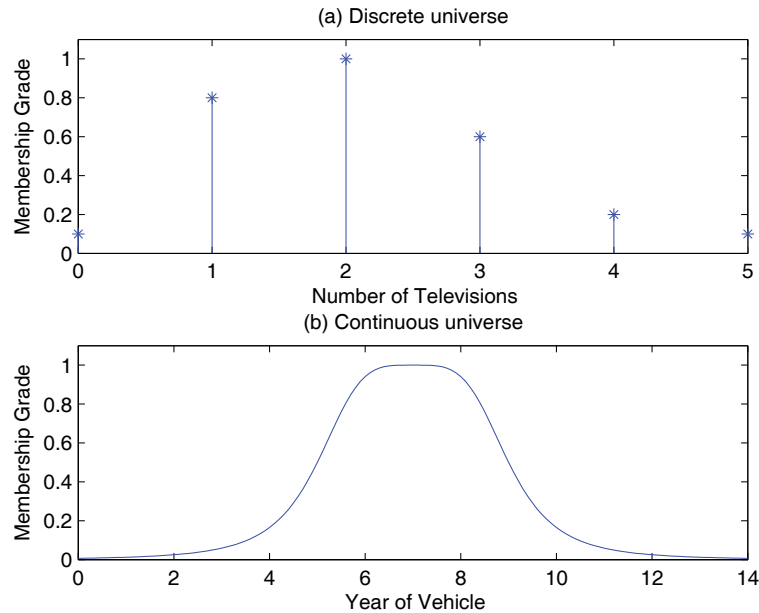


Figure 9.3.2 Membership grades: (a) Discrete universe; (b) Continuous universe.

9.3.2 Fuzzy sets with a continuous universe

Fuzzy set C = “*about 7 year old vehicle*”,

X = Set of positive real numbers (continuous)

$$C = \{[x, \mu_C(x)] | x \in X\}$$

where

$$\mu_C(x) = \frac{1}{1 + \left(\frac{x-7}{2}\right)^4}$$

Figure 9.3.2(b) plots this fuzzy set C in the continuous universe.

9.3.3 Notations for fuzzy sets

A discrete or continuous fuzzy set A can be denoted using the following notations:

For X is discrete:

$$A = \sum_{x_i \in X} \mu_A(x_i) / x_i \quad (9.3.2)$$

For X is continuous:

$$A = \int_X \mu_A(x) / x \quad (9.3.3)$$

In (9.3.2)-(9.3.3), each of the sum sign and the integral sign stands for the union of membership grades; and “/” stands for a marker (not a division sign). Using the above notations, the examples of Section 9.3.2 become

$$A = 0.9 / \text{City 1} + 0.8 / \text{City 2} + 0.7 / \text{City 3}$$

$$B = 0.1/0 + 0.8/1 + 1/2 + 0.6/3 + 0.2/4 + 0.1/5$$

$$C = \int_{R^+} \frac{1}{1 + \left(\frac{x-7}{2}\right)^4} / x$$

9.3.4 Subset

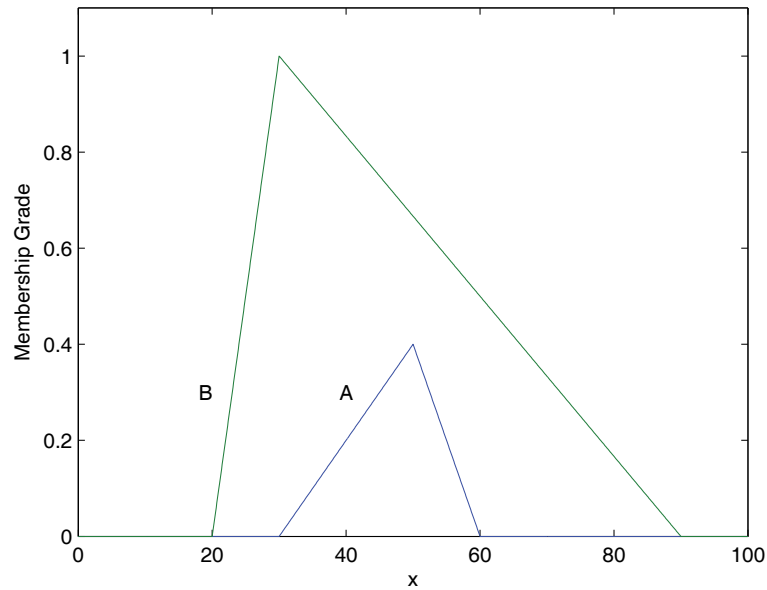


Figure 9.3.3 Fuzzy set A is contained in fuzzy set B ($A \subseteq B$).

$$A \subseteq B \text{ implies } \mu_A(x) \leq \mu_B(x) \quad (9.3.4)$$

Fuzzy set A is contained in fuzzy set B (or A is a subset of B , or A is smaller than or equal to B) if and only if $\mu_A(x) \leq \mu_B(x)$, as shown in Figure 9.3.3.

9.3.5 Complement, union, and intersection

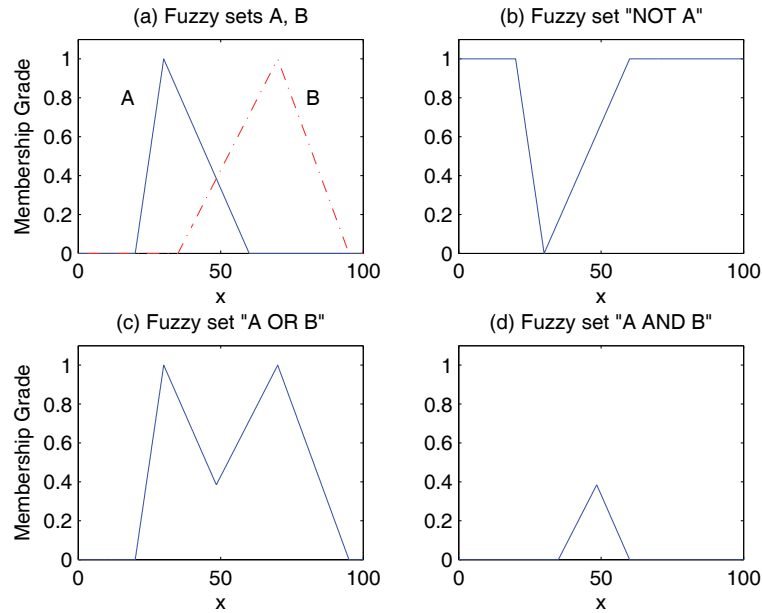


Figure 9.3.4 Fuzzy set operations: (a) Fuzzy sets A, B ; (b) Fuzzy set "NOT A "; (c) Fuzzy set " A OR B "; (d) Fuzzy set " A AND B ".

9.3.5.1 Complement (Negation)

$$\bar{A} = X - A \text{ implies } \mu_{\bar{A}}(x) = 1 - \mu_A(x) \quad (9.3.5)$$

9.3.5.2 Union (Disjunction)

$C = A \cup B$ implies

$$\mu_C(x) = \max[\mu_A(x), \mu_B(x)] = \mu_A(x) \vee \mu_B(x) \quad (9.3.6)$$

Equation (9.3.6) is equivalent to A OR B which represents the smallest (or minimum) fuzzy set containing both A and B .

9.3.5.3 Intersection (Conjunction)

$C = A \cap B$ implies

$$\mu_C(x) = \min[\mu_A(x), \mu_B(x)] = \mu_A(x) \wedge \mu_B(x) \quad (9.3.7)$$

Equation (9.3.7) is equivalent to $A \text{ AND } B$ which represents the largest (or maximum) fuzzy set which is contained in both A and B .

The complement, max, and min operators are referred to as the classical fuzzy operators for complement, union, and intersection respectively on fuzzy sets. The three fuzzy set operations are illustrated in Figure 9.3.4.

9.3.6 Fuzzy partition

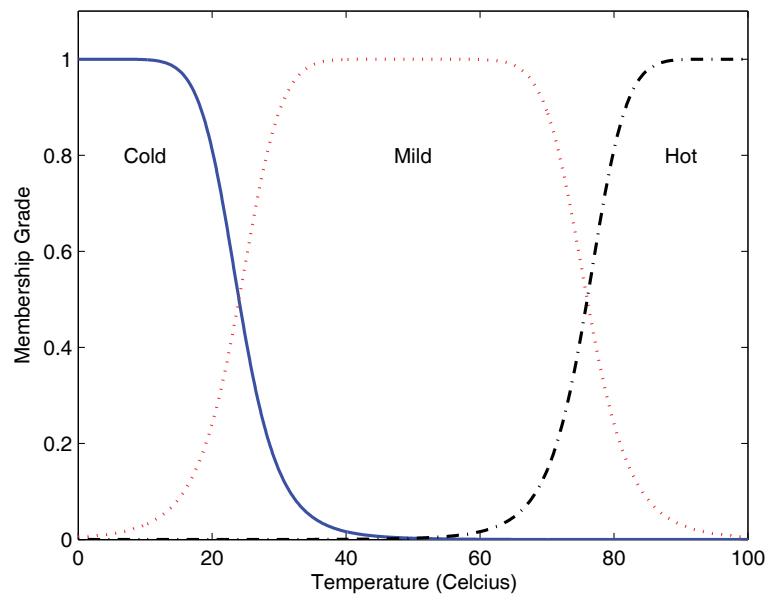


Figure 9.3.5 Typical membership functions.

A continuous universe of discourse can be partitioned and described by a number of fuzzy sets. Fuzzy partitions formed by the linguistic values “Cold”, “Mild”, and “Hot”, are illustrated in Figure 9.3.5.

9.4 Terminologies of Membership Functions

In this section, two important properties of membership functions, subjectivity and non-randomness, will be emphasized. These properties of a fuzzy set distinguish itself from the randomness of a probability function.

First of all, a membership function is a subjective measure. One's membership function of “*hot*” is likely to be different from others.

For example, as shown on Figure 9.4.1, 35°C is considered “*hot*” in Toronto. In Hong Kong, 35°C is only considered “*medium hot*”, and may be hot to the degree of 0.5. However, in Singapore, 35°C is considered “*not hot*”.

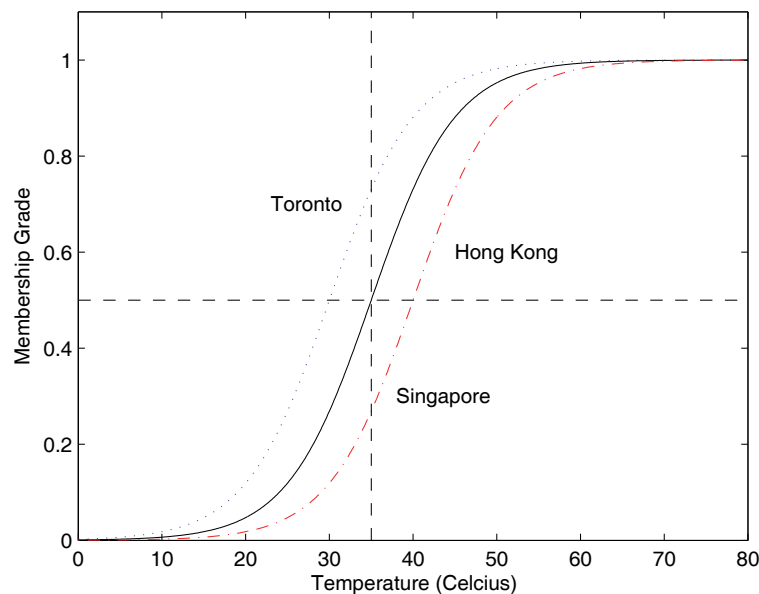


Figure 9.4.1 Subjective membership functions.

In Figure 9.4.1, there are three different membership functions in different contexts but they do share some common characteristics such

as they are all monotonically increasing from 0 to 1.

9.4.1 Symmetry

To describe membership functions more specifically, some terminologies are defined, as shown in Figure 9.4.2.

A symmetric fuzzy set A has a membership function which is symmetric around a certain point $x = c$ as

$$\mu_A(c + x) = \mu_A(c - x) \quad \text{for all } x \in X \quad (9.4.1)$$

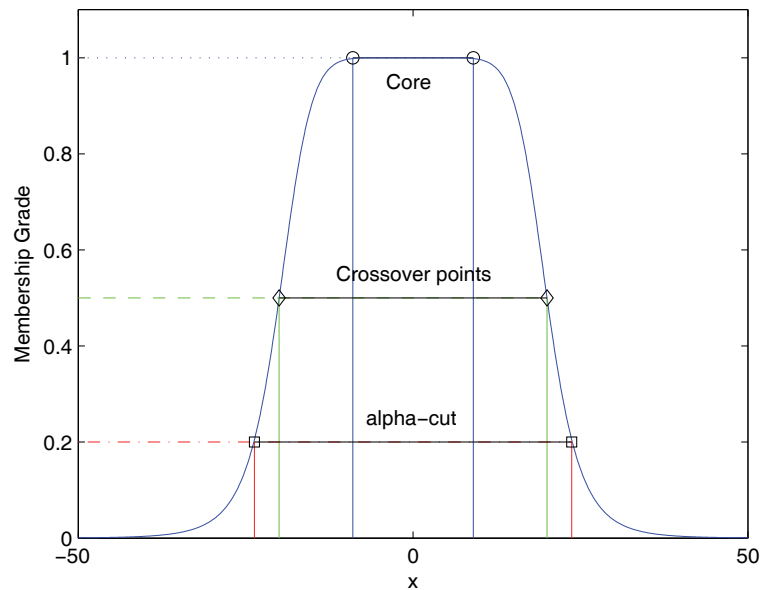


Figure 9.4.2 Membership function terminologies.

9.4.2 Open left, open right, closed

Table 9.4.1 Definitions of open left, open right, and closed fuzzy sets.

Fuzzy set A	Membership function
Open left	If $\lim_{x \rightarrow -\infty} \mu_A(x) = 1$ and $\lim_{x \rightarrow \infty} \mu_A(x) = 0$
Open right	If $\lim_{x \rightarrow -\infty} \mu_A(x) = 0$ and $\lim_{x \rightarrow \infty} \mu_A(x) = 1$
Closed	If $\lim_{x \rightarrow -\infty} \mu_A(x) = 0$ and $\lim_{x \rightarrow \infty} \mu_A(x) = 0$

Membership functions of open left, open right, and closed fuzzy sets are defined in Table 9.4.1 and shown in Figure 9.3.5.

9.4.3 Normality

A normal fuzzy set A has a nonempty core such that a point $x \in X$ can always be found in which $\mu_A(x) = 1$.

9.4.4 Support

A set of all points x in X such that $\mu_A(x) > 0$ refers to the support of a fuzzy set A which can be expressed as

$$\text{Support}(A) = \{x | \mu_A(x) > 0\} \quad (9.4.2)$$

9.4.5 Fuzzy singleton

If the support of a fuzzy set consists of only a single point in X with $\mu_A(x) = 1$, it is called a fuzzy singleton.

9.4.6 Core

The core of a fuzzy set A is defined as the set of all points x in X such that $\mu_A(x) = 1$ which can be expressed as

$$\text{Core}(A) = \{x | \mu_A(x) = 1\} \quad (9.4.3)$$

9.4.7 Crossover points

The crossover points of a fuzzy set A are defined as points $x \in X$ with $\mu_A(x) = 0.5$ as

$$\text{Crossover}(A) = \{x | \mu_A(x) = 0.5\} \quad (9.4.4)$$

9.4.8 α -cut and strong α -cut

The α -cut (or α -level) of a fuzzy set A is a crisp set defined by

$$A_\alpha = \{x | \mu_A(x) \geq \alpha\} \quad (9.4.5)$$

From the definitions of α -cut and core, $A_1 \rightarrow \text{Core}(A)$ for $\alpha = 1$. The strong α -cut of a fuzzy set A is a crisp set defined by

$$A'_\alpha = \{x | \mu_A(x) > \alpha\} \quad (9.4.6)$$

From the definitions of strong α -cut and support, $A'_0 = \text{Support}(A)$.

9.4.9 Convexity

For any $x_1, x_2 \in X$ and any $\lambda \in [0, 1]$, a fuzzy set A is said to be convex if and only if the following condition applies

$$\mu_A(\lambda x_1 + (1 - \lambda)x_2) \geq \min[\mu_A(x_1), \mu_A(x_2)] \quad (9.4.7)$$

A simple way to define a convex fuzzy set A is to consider its α -cuts such that if all its α -cuts are convex, then the fuzzy set A is convex.

Figure 9.4.3 shows examples on convex and nonconvex membership functions.

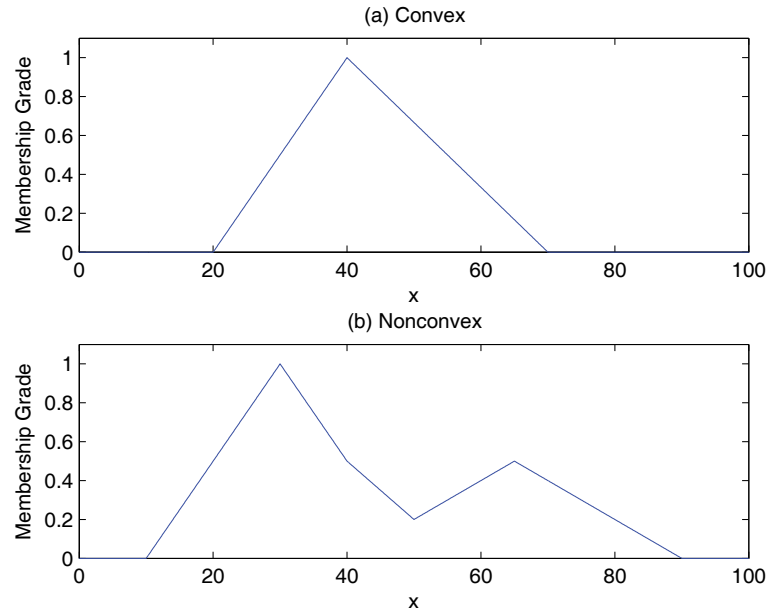


Figure 9.4.3 Convex and nonconvex membership functions: (a) Convex; (b) Nonconvex.

9.4.10 Widths of normal and convex fuzzy sets

The width of a normal and convex fuzzy set is defined as the distance between the two unique crossover points at $\mu_A(x_1) = \mu_A(x_2) = 0.5$ such that

$$\text{width}(A) = |x_2 - x_1| \quad (9.4.8)$$

9.4.11 Fuzzy number

In contrast to exactness as is the case with a single-valued number, a fuzzy number is a quantity whose value is not precise.

Each possible value of a fuzzy number has a membership function grade between 0 and 1.

A fuzzy number can be thought of as a fuzzy set whose domain is a

specified set (usually the set of real numbers) that satisfies the conditions of normality and convexity and therefore forms the most basic type of fuzzy set.

9.5 One-Dimensional Membership Functions

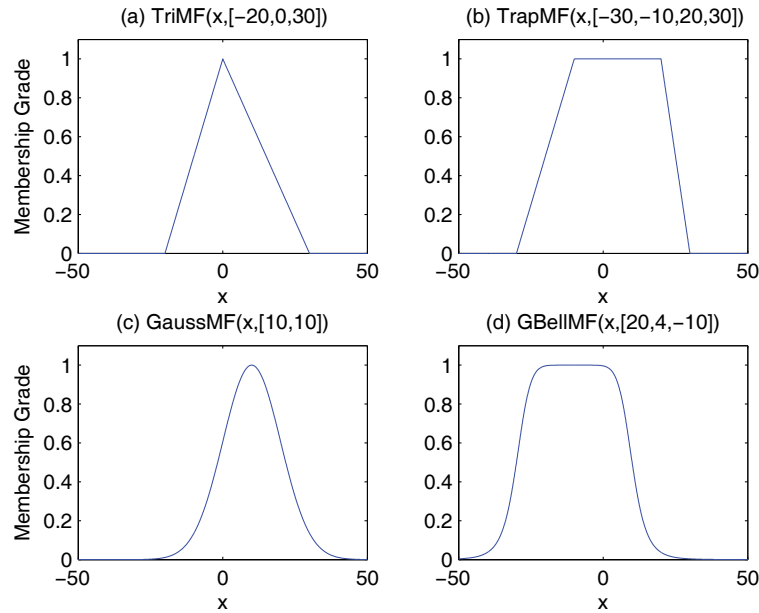


Figure 9.5.1 Parameterized membership functions: (a) Triangular; (b) Trapezoidal; (c) Gaussian; (d) Generalized bell.

9.5.1 Triangular membership function

A triangular membership function is defined in (9.5.1). As shown on the top-left of Figure 9.5.1(a), $\text{TriMF}(x, [a, b, c]) = \text{TriMF}(x, [-20, 0, 30])$, where a , b , and c specify the locations of the three corners.

The TriMF is symmetrical if b is located at the centre of a and c , otherwise, it is non-symmetrical.

$$\text{TriMF}(x, [a, b, c]) = \max \left[\min \left(\frac{x-a}{b-a}, \frac{c-x}{c-b} \right), 0 \right] \quad (9.5.1)$$

9.5.2 Trapezoidal membership function

A trapezoidal membership function is defined in (9.5.2). As shown on the top-right of Figure 9.5.1(b), $\text{TrapMF}(x, [a, b, c, d]) = \text{TrapMF}(x, [-30, -10, 20, 30])$, where a, b, c and d specify the locations of the four corners.

$$\text{TrapMF}(x, [a, b, c, d]) = \max \left[\min \left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c} \right), 0 \right] \quad (9.5.2)$$

9.5.3 Gaussian membership function

A Gaussian membership function is defined in (9.5.3). As shown on the bottom-left of Figure 9.5.1(c), $\text{GaussMF}(x, [c, \sigma]) = \text{GaussMF}(x, [10, 10])$ where c and σ represent, respectively, membership function's center and width.

$$\text{GaussMF}(x, [c, \sigma]) = e^{-\frac{1}{2} \left(\frac{x-c}{\sigma} \right)^2} \quad (9.5.3)$$

9.5.4 Generalized bell membership function

A generalized bell membership function is defined in (9.5.4) and shown in Figure 9.5.1(d), $\text{GBellMF}(x, [a, b, c]) = \text{GBellMF}(x, [20, 4, -10])$, where $b > 0$.

In general, the parameter a controls the width (larger a larger width), the parameter b controls the slopes of the crossover points (larger b steeper slope), and the parameter c controls the center of the generalized bell membership function.

$$\text{GBellMF}(x, [a, b, c]) = \frac{1}{1 + \left[\left(\frac{x-c}{a} \right)^2 \right]^b} \quad (9.5.4)$$

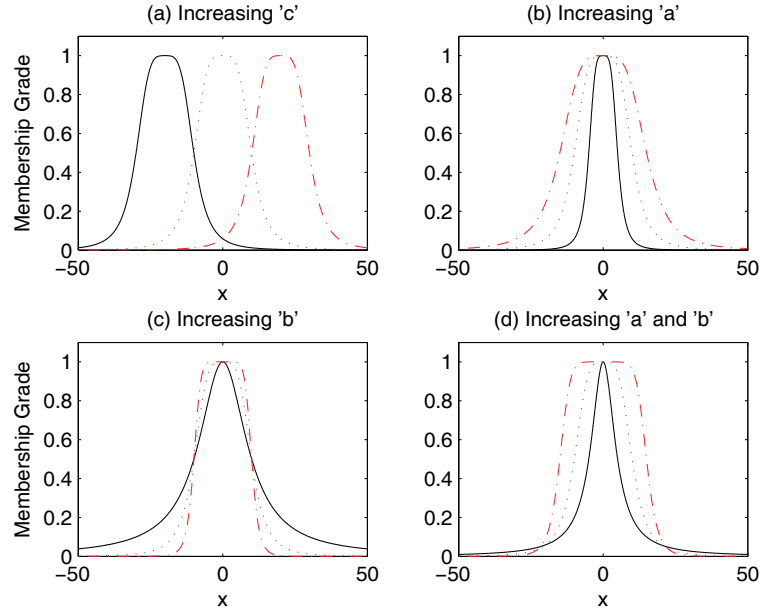


Figure 9.5.2 Parameter effects on generalized bell membership function: (a) Increasing $c = [-20, 0, 20]$, $a = 10$, $b = 2$; (b) Increasing $a = [5, 10, 15]$, $b = 2$, $c = 0$; (c) Increasing $b = [1, 2, 4]$, $a = 10$, $c = 0$; (d) Increasing $a = [5, 10, 15]$, $b = [1, 2, 4]$, $c = 0$.

Figure 9.5.2 shows the effects of changing parameters in generalized bell membership functions.

A generalized bell membership function is regarded as the most versatile membership function in which its width, slope, and center can be changed to suit a variety of applications.

9.5.5 Sigmoidal membership function

A sigmoidal membership function is defined in (9.5.5).

$$\text{SigMF}(x, [a, c]) = \frac{1}{1 + e^{-a(x-c)}} \quad (9.5.5)$$

In (9.5.5), “ a ” represents the slope and “ c ” represents the crossover point.

Open left (for $a < 0$) and open right ($a > 0$) membership functions can be easily constructed by changing the sign of a controlling the slope as shown in Figure 9.5.3(a) and (c).

9.5.6 Asymmetric and close membership functions

Triangular and trapezoidal membership functions offer asymmetric shape but they are piecewise linear functions.

Gaussian and generalized bell membership functions offer continuous functions but they are of symmetric shape.

In practice, a class of asymmetric and continuous membership functions can be built from sigmoidal membership function.

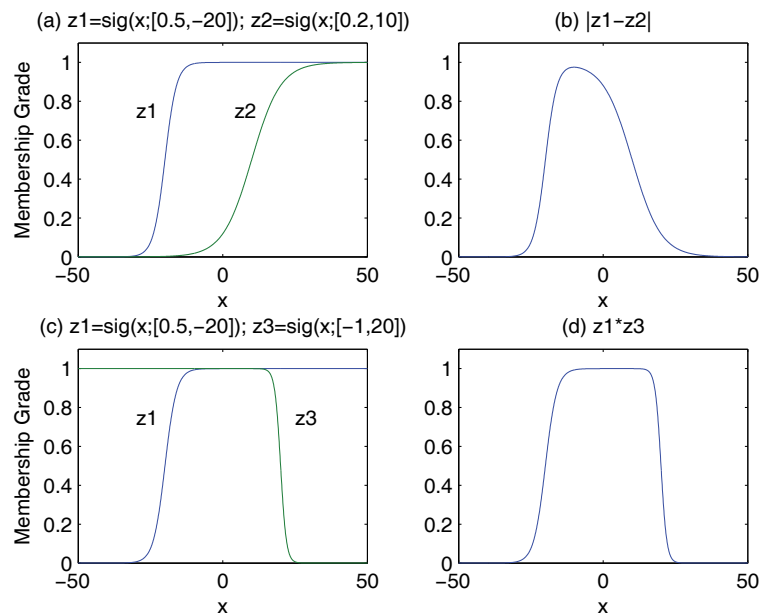


Figure 9.5.3 Construction of close and asymmetric membership functions: (a) Sigmoidal functions $z1, z2$; (b) $|z1 - z2|$; (c) Sigmoidal

functions z_1, z_3 ; (d) $z_1 * z_3$.

Figure 9.5.3 illustrates two ways in (b) and (d) to obtain a close and asymmetric membership function using two sigmoid membership functions.

9.6 Two-Dimensional Membership Functions

A two-dimensional membership function refers to a membership function with two inputs, X and Y , each in a different universe of discourse.

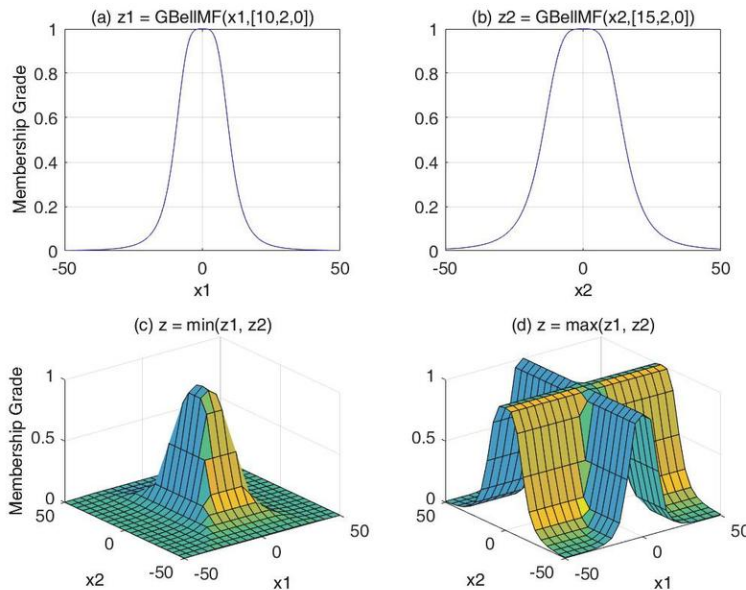


Figure 9.6.1 Min and Max operators for two-dimensional membership function construction: (a) z_1 ; (b) z_2 ; (c) $\min(z_1, z_2)$; (d) $\max(z_1, z_2)$.

A two-dimensional membership function can be composite and non-composite.

A composite two-dimensional membership function can be expressed

analytically as two one-dimensional membership functions, whereas non-composite membership function cannot.

A composite two-dimensional membership function can be defined as the AND or OR aggregation of its two constituent membership functions based on Min and Max operators.

Figure 9.6.1 shows two examples of two-dimensional composite membership functions based on Min and Max operators.

9.7 Derivatives of Parameterized Membership Functions

The derivatives of a membership function with respect to its inputs and parameters are useful for the gradient and adaptive learning of a fuzzy system and a fuzzy neural network.

The derivatives of both the one-dimensional and two-dimensional Gaussian and generalized bell membership functions are listed in Tables 9.7.1 to 9.7.4.

Table 9.7.1 One-dimensional Gaussian membership function and its derivatives.

$y = \text{GaussMF}(x, c, \sigma) = e^{-\frac{1}{2} \left(\frac{x - c}{\sigma} \right)^2}$
$\frac{\partial y}{\partial x} = -\frac{x - c}{\sigma^2} y$
$\frac{\partial y}{\partial c} = \frac{x - c}{\sigma^2} y$
$\frac{\partial y}{\partial \sigma} = \frac{(x - c)^2}{\sigma^3} y$

Table 9.7.2 One-dimensional generalized bell membership function and its derivatives.

$y = \text{GBellMF}(x, a, b, c) = \frac{1}{1 + \left[\left(\frac{x - c}{a} \right)^2 \right]^b}$
$\frac{\partial y}{\partial x} = \begin{cases} -\frac{2b}{x - c} y(1 - y) & \text{if } x \neq c \\ 0 & \text{if } x = c \end{cases}$
$\frac{\partial y}{\partial a} = \frac{2b}{a} y(1 - y)$
$\frac{\partial y}{\partial b} = \begin{cases} -2 \ln \left \frac{2b}{x - c} \right y(1 - y) & \text{if } x \neq c \\ 0 & \text{if } x = c \end{cases}$
$\frac{\partial y}{\partial c} = \begin{cases} \frac{2b}{x - c} y(1 - y) & \text{if } x \neq c \\ 0 & \text{if } x = c \end{cases}$

Table 9.7.3 Two-dimensional Gaussian (composite) membership function and its derivatives.

$\mu_A(x, y) = e^{-\frac{1}{2} \left[\left(\frac{x - c_x}{\sigma_x} \right)^2 + \left(\frac{y - c_y}{\sigma_y} \right)^2 \right]}$
$\frac{\partial \mu_A(x, y)}{\partial x} = -\frac{x - c_x}{\sigma_x^2} \mu_A(x, y)$
$\frac{\partial \mu_A(x, y)}{\partial c_x} = \frac{x - c_x}{\sigma_x^2} \mu_A(x, y)$
$\frac{\partial \mu_A(x, y)}{\partial \sigma_x} = \frac{(x - c_x)^2}{\sigma_x^3} \mu_A(x, y)$

$\frac{\partial \mu_A(x, y)}{\partial y} = -\frac{y - c_y}{\sigma_y^2} \mu_A(x, y)$
$\frac{\partial \mu_A(x, y)}{\partial c_y} = \frac{y - c_y}{\sigma_y^2} \mu_A(x, y)$
$\frac{\partial \mu_A(x, y)}{\partial \sigma_y} = \frac{(y - c_y)^2}{\sigma_y^3} \mu_A(x, y)$

Table 9.7.4 Two-dimensional generalized bell (non-composite) membership function and its derivatives.

$\mu_A(x, y) = \frac{1}{1 + \left[\left(\frac{x - c_x}{a_x} \right)^2 + \left(\frac{y - c_y}{a_y} \right)^2 \right]^b}$	
$\frac{\partial \mu_A(x, y)}{\partial x} = \frac{-2b(x - c_x)}{a_x^2}$ $\times \left[\left(\frac{x - c_x}{a_x} \right)^2 + \left(\frac{y - c_y}{a_y} \right)^2 \right]^{b-1} (\mu_A(x, y))^2$	for $x \neq c_x$
$\frac{\partial \mu_A(x, y)}{\partial x} = 0$	for $x = c_x$
$\frac{\partial \mu_A(x, y)}{\partial y} = \frac{-2b(y - c_y)}{a_y^2}$ $\times \left[\left(\frac{x - c_x}{a_x} \right)^2 + \left(\frac{y - c_y}{a_y} \right)^2 \right]^{b-1} (\mu_A(x, y))^2$	for $y \neq c_y$
$\frac{\partial \mu_A(x, y)}{\partial y} = 0$	For $y = c_y$

$\frac{\partial \mu_A(x, y)}{\partial c_x} = \frac{2b(x - c_x)}{a_x^2}$ $\times \left[\left(\frac{x - c_x}{a_x} \right)^2 + \left(\frac{y - c_y}{a_y} \right)^2 \right]^{b-1} (\mu_A(x, y))^2$	for $x \neq c_x$
$\frac{\partial \mu_A(x, y)}{\partial c_x} = 0$	for $x = c_x$
$\frac{\partial \mu_A(x, y)}{\partial c_y} = \frac{2b(y - c_y)}{a_y^2} \times \left[\left(\frac{x - c_x}{a_x} \right)^2 + \left(\frac{y - c_y}{a_y} \right)^2 \right]^{b-1} (\mu_A(x, y))^2$	for $y \neq c_y$
$\frac{\partial \mu_A(x, y)}{\partial c_y} = 0$	for $y = c_y$
$\frac{\partial \mu_A(x, y)}{\partial a_x} = \frac{2b(x - c_x)}{a_x^3}$ $\times \left[\left(\frac{x - c_x}{a_x} \right)^2 + \left(\frac{y - c_y}{a_y} \right)^2 \right]^{b-1} (\mu_A(x, y))^2$	for $x \neq c_x$
$\frac{\partial \mu_A(x, y)}{\partial a_x} = 0$	for $x = c_x$
$\frac{\partial \mu_A(x, y)}{\partial a_y} = \frac{2b(y - c_y)}{a_y^3}$ $\times \left[\left(\frac{x - c_x}{a_x} \right)^2 + \left(\frac{y - c_y}{a_y} \right)^2 \right]^{b-1} (\mu_A(x, y))^2$	for $y \neq c_y$

$\frac{\partial \mu_A(x, y)}{\partial a_y} = 0$	for $y = c_y$
$\frac{\partial \mu_A(x, y)}{\partial b} = -b \ln \left[\left(\frac{x - c_x}{a_x} \right)^2 + \left(\frac{y - c_y}{a_y} \right)^2 \right]$ $\times \left[\left(\frac{x - c_x}{a_x} \right)^2 + \left(\frac{y - c_y}{a_y} \right)^2 \right]^{b-1} (\mu_A(x, y))^2$	for $x \neq c_x$ and $y \neq c_y$
$\frac{\partial \mu_A(x, y)}{\partial b} = -b \ln \left[\left(\frac{y - c_y}{a_y} \right)^2 \right]$ $\times \left[\left(\frac{y - c_y}{a_y} \right)^2 \right]^{b-1} (\mu_A(x, y))^2$	for $x = c_x$ or $y \neq c_y$
$\frac{\partial \mu_A(x, y)}{\partial b} = -b \ln \left[\left(\frac{x - c_x}{a_x} \right)^2 \right]$ $\times \left[\left(\frac{x - c_x}{a_x} \right)^2 \right]^{b-1} (\mu_A(x, y))^2$	for $x \neq c_x$ or $y = c_y$
$\frac{\partial \mu_A(x, y)}{\partial b} = 0$	for $x = c_x$ and $y = c_y$

9.8 Linguistic Variables

The concept of linguistic variables was introduced in [Zad7104a], [Zad7500a], [Zad7500b], [Zad7500c]. A numerical variable takes numerical value: Temperature = 100°C. A linguistic variable takes linguistic value: Temperature is “hot”.

Three elements of interest in this topic that characterize a linguistic

variable are: the name of the linguistic variable x ; the set of its linguistic values (or the term set of x) $T(x)$; and the universe of discourse X .

A linguistic value is a fuzzy set. Using this representation, primary and composite linguistic values can be expressed in term of membership functions which can be used as an input to a fuzzy system for numerical computation and analysis.

9.8.1 Term set and its components

Let “*temperature*” be a linguistic variable, an example for its term set $T(\text{temperature})$ could be expressed as

$T(\text{temperature}) =$
 $\{\text{cold, not cold, quite cold, very cold, too cold, extremely cold, ... ,}$
 $\text{warm, not warm, quite warm, very warm, extremely warm, ... ,}$
 $\text{hot, not hot, quite hot, very hot, extremely hot, ... ,}$
 $\text{more or less hot, ... ,}$
 $\text{not very cold and not very hot, ... ,}$
 $\text{not cold but not too hot, ... ,}$
 $\text{either hot or cold, ... ,}$
 $\text{neither hot nor cold, ... }\}$

Each term in $T(\text{temperature})$ is characterized by a fuzzy set of a continuous universe of discourse X .

For example, “*temperature is hot*” is used to denote the assignment of the linguistic value “*hot*” to the linguistic variable “*temperature*”. In this example, the term set is consisted of

primary terms:	(<i>cold, warm, hot</i>)
negation:	(<i>not</i>)
hedges:	(<i>very, too, quite, extremely, more or less</i>)
connectives:	(<i>and, but, or, nor, either, neither</i>)

Each of the negation, the hedges, and the connectives is an operator that can modify independently the meaning of its operand.

9.8.2 Concentration and dilation

For a linguistic value A characterized by a fuzzy set with a membership function $\mu_A(x)$, a modified linguistic value can be obtained by taking the k th power of A as

$$A^k = \int_X [\mu_A(x)]^k / x \quad (9.8.1)$$

With a different choice of k , A^k can express a different meaning. In particular, the concentration operator (meaning “*very*”) can be obtained by choosing $k = 2$ as

$$\text{CON}(A) = A^2 \quad (9.8.2)$$

Choosing $k = 0.5$, the dilation operator (meaning “*more or less*”) can be obtained by

$$\text{DIL}(A) = A^{0.5} \quad (9.8.3)$$

9.8.3 Contrast intensifier INT(A) and contrast diminisher DIM(A)

Contrast intensifier (or contrast diminisher) is an operation on linguistic fuzzy sets based on concentration (or dilation). The operation of the contrast intensifier on a linguistic value A is defined by

$$\text{INT}(A) = \begin{cases} 2A^2 & 0 \leq \mu_A(x) \leq 0.5 \\ \neg 2(\neg A)^2 & 0.5 \leq \mu_A(x) \leq 1 \end{cases} \quad (9.8.4)$$

For the contrast intensifier INT, it increases the values of $\mu_A(x)$ above the point 0.5 and diminishes those below the point 0.5.

Basically, the contrast intensifier reduces the fuzziness of a linguistic value A by making the transitions of the membership function steeper.

The contrast diminisher offers the reverse operation of the contrast

intensifier INT such that

$$\text{DIM}(\text{INT}(A)) \approx A \quad (9.8.5)$$

$$\text{DIM}(A) = \begin{cases} (A/2)^{0.5} & 0 \leq \mu_A(x) \leq 0.5 \\ \neg((\neg A)/2)^{0.5} & 0.5 \leq \mu_A(x) \leq 1 \end{cases} \quad (9.8.6)$$

Note that the contrast diminisher and the contrast intensifier are different and do not form a perfect reconstruction pair.

Example 9.1 (Contrast intensifier and contrast diminisher) Matlab

Given a generalized bell membership function $\mu_A(x)$ specified below.

$$\mu_A(x) = \text{GBellMF}(x, [30, 2, 100]) = \frac{1}{1 + \left(\frac{x - 100}{30}\right)^4}$$

Plot $\mu_A(x)$ after one time, two times, and three times applications of (a) the contrast intensifier; and (b) the contrast diminisher.

Discuss observations on each of the two cases.

Solution

(a) A plot on the applications of the contrast intensifier is shown in Figure E9.1.1. It can be observed that the effect of the contrast intensifier is to cause any value of the original membership function that is greater than 0.5 to approach 1.0 after several intensifications.

On the lower side, any value less than 0.5 will approach 0 after several intensifications. In other words, the intensifier operation produces a more rectangular-like membership function, eventually approaching a crisp set with transitions located at the crossover points ($\alpha = 0.5$, see Figure 9.4.2) of the original membership function.

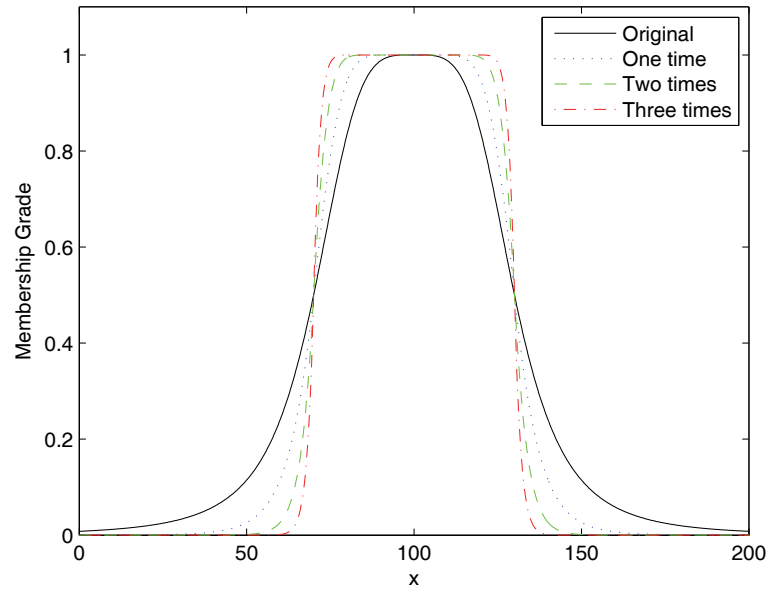


Figure E9.1.1 Effects of applying contrast intensifier.

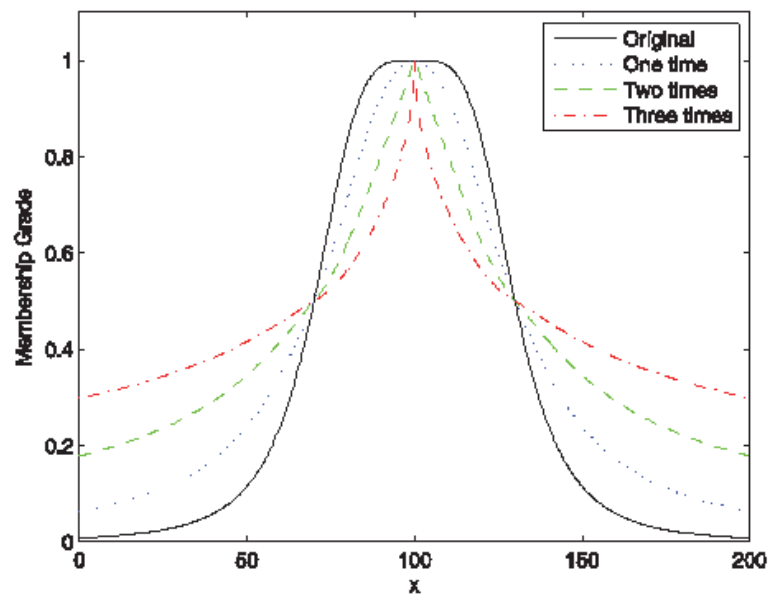


Figure E9.1.2 Effects of applying contrast diminisher.

(b) A plot on the applications of the contrast diminisher is shown in Figure E9.1.2.

It can be observed that the effect of the contrast diminisher is to cause any value of the original membership function that is greater than 0.5 to approach 0.5 after several diminishing operations.

On the lower side, any value less than 0.5 will also approach 0.5 after several diminishing operations.

The diminishing operation tends to cause the membership function to become a fuzzy set with a peak located where the membership function initially equals to 1 and cross-over points remain the same.

9.8.4 Negation (NOT) and connectives (AND and OR)

For two linguistic values A and B whose meanings are characterized by $\mu_A(x)$ and $\mu_B(x)$, the operations of NOT, AND, and OR are defined as

$$\text{NOT}(A) = \neg A = \int_X [1 - \mu_A(x)] / x \quad (9.8.7)$$

$$A \text{ OR } B = A \cup B = \int_X [\mu_A(x) \vee \mu_B(x)] / x \quad (9.8.8)$$

$$A \text{ AND } B = A \cap B = \int_X [\mu_A(x) \wedge \mu_B(x)] / x \quad (9.8.9)$$

Example 9.2 (Membership function constructions of composite linguistic terms)

The linguistic values “*cold*” and “*hot*” of a linguistic variable “*temperature*” in °C (continuous universe of discourse) are defined by the following general membership functions.

$$\mu_{cold}(x) = \text{GBellMF}(x, [a_1, b_1, c_1]) = \frac{1}{1 + \left(\frac{x - c_1}{a_1}\right)^{2b_1}} \quad (\text{E9.2.1})$$

$$\mu_{hot}(x) = \text{GBellMF}(x, [a_2, b_2, c_2]) = \frac{1}{1 + \left(\frac{x - c_2}{a_2}\right)^{2b_2}} \quad (\text{E9.2.2})$$

Assume that the meaning of the hedge “*extremely*” is the same as that of “*very very*” and the connective “*but*” is the same as that of “*and*”.

Construct the membership function for each of the following composite linguistic terms: “*extremely cold*”, “*more or less cold*”, “*not cold but not hot*”, and “*very hot*”.

Solution

$$\textit{extremely cold} = \text{CON}(\text{CON}(\textit{cold}))$$

$$\text{CON}(\text{CON}(\textit{cold})) = (\textit{cold}^2)^2 = \int_x \left[\frac{1}{1 + \left(\frac{x - c_1}{a_1}\right)^{2b_1}} \right]^4 / x \quad (\text{E9.2.3})$$

$$\textit{more or less cold} = \text{DIL}(\textit{cold})$$

$$\text{DIL}(\textit{cold}) = \textit{cold}^{0.5} = \int_x \left[\frac{1}{1 + \left(\frac{x - c_1}{a_1}\right)^{2b_1}} \right]^{1/2} / x \quad (\text{E9.2.4})$$

$$\textit{not cold but not hot} = \neg \textit{cold} \cap \neg \textit{hot}$$

$$\neg cold \cap \neg hot$$

$$= \int_x \left[1 - \frac{1}{1 + \left(\frac{x - c_1}{a_1} \right)^{2b_1}} \right] \wedge \left[1 - \frac{1}{1 + \left(\frac{x - c_2}{a_2} \right)^{2b_2}} \right] / x \quad (\text{E9.2.5})$$

$$very\ hot = CON(hot)$$

$$CON(hot) = hot^2 = \int_x \left[\frac{1}{1 + \left(\frac{x - c_2}{a_2} \right)^{2b_2}} \right]^2 / x \quad (\text{E9.2.6})$$

Example 9.3 (Membership function plots of composite linguistic terms)
Matlab

Given the membership functions for the primary linguistic terms “*cold*” and “*hot*”, plot the membership functions of (a) the primary linguistic terms “*cold*” and “*hot*”; and (b) the composite linguistic terms “*extremely cold*”, “*more or less cold*”, “*not cold but not hot*”, and “*very hot*”.

$$\mu_{cold}(x) = \text{GBellMF}(x, [40, 2, -50]) = \frac{1}{1 + \left(\frac{x + 50}{40} \right)^4}$$

$$\mu_{hot}(x) = \text{GBellMF}(x, [40, 2, 50]) = \frac{1}{1 + \left(\frac{x - 50}{40} \right)^4}$$

Solution

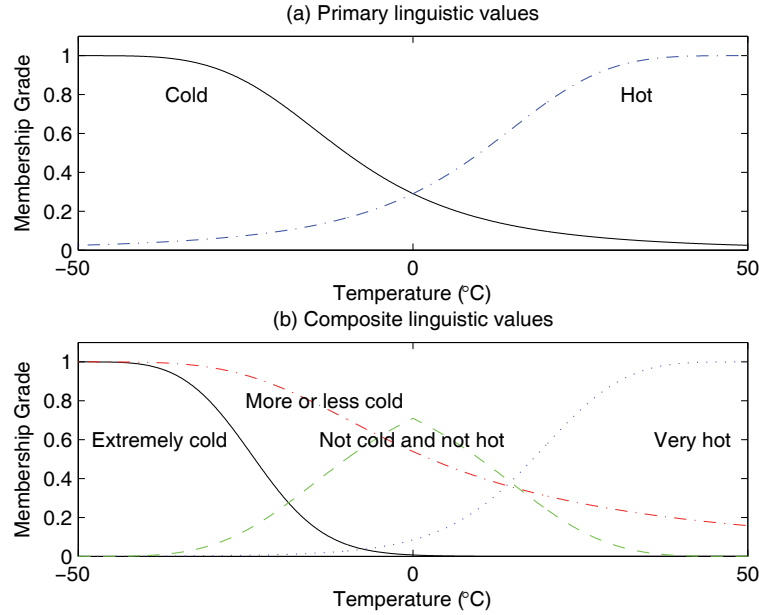


Figure E9.3.1 Membership functions of primary and composite linguistic values: (a) Primary; (b) Composite.

The membership functions for the primary linguistic terms “cold” and “hot” are plotted in Figure E9.3.1(a) and the membership functions of the composite linguistic terms “*extremely cold*”, “*more or less cold*”, “*not cold but not hot*”, and “*very hot*” are plotted as shown in Figure E9.3.1(b).

9.8.5 Orthogonality

A linguistic variable x on a universe of discourse X is described as orthogonal if the membership functions of its set of linguistic values $T = \{t_1, t_2, t_3, \dots, t_n\}$ satisfy the following requirement as

$$\sum_{i=1}^n \mu_{t_i}(x) = 1 \quad \forall x \in X \quad (9.8.1)$$

The orthogonality requirement should be satisfied to some extent for arriving at intuitively reasonable membership functions within a term

set.

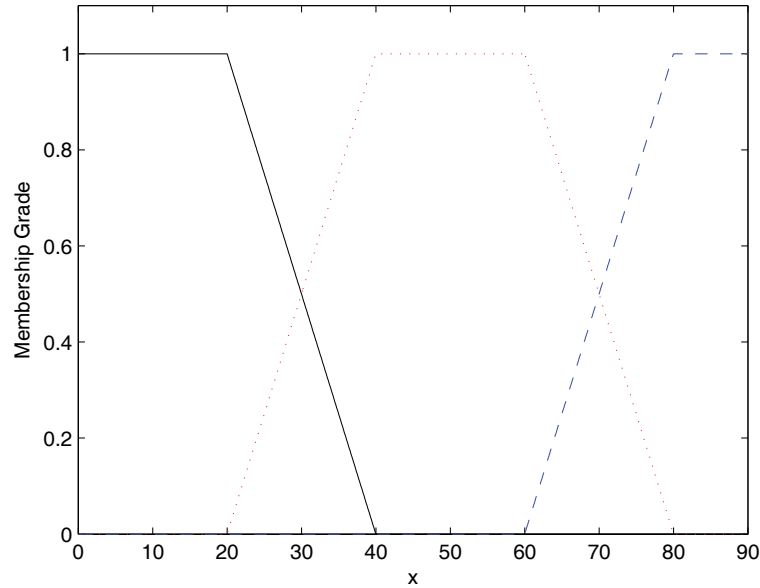


Figure 9.8.1 Orthogonal membership functions.

An example of orthogonal membership functions is shown in Figure 9.8.1.

9.8.6 Similarity measure

The similarity measure of two fuzzy sets $E(A, B)$ is defined as

$$E(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (9.8.11)$$

where a fuzzy set F (A or B) in a universe of discourse X is denoted as

$$F = \sum_{x \in X} \mu_F(x) \quad (9.8.12)$$

If $A = B$, (9.8.11) gives $E(A, B) = 1$. If $A \neq B$, (9.37) becomes 0 as A and B are not similar.

Example 9.4 (Linguistic meaning of fuzzy set)

Given the term set $\{A, B, C\}$ of the linguistic variable “*true*” in a discrete universe of discourse $X = \{0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$ such that

$$A = \text{“true”} = 0.7/0.8 + 1.0/0.9 + 1.0/1.0,$$

$$B = \text{“more or less true”} = 0.5/0.6 + 0.7/0.7 + 1.0/0.8 + 1.0/0.9 + 1.0/1.0,$$

$$C = \text{“almost true”} = 0.9/0.7 + 1.0/0.9 + 0.6/1.0.$$

Determine the approximated linguistic meaning of the fuzzy set D given by

$$D = 0.6/0.8 + 1.0/0.9 + 1.0/1.0.$$

Solution

$$D = 0.6/0.8 + 1/0.9 + 1/1$$

$$A = 0.7/0.8 + 1/0.9 + 1/1$$

$$\begin{aligned} E(D, A) &= \frac{|D \cap A|}{|D \cup A|} = \frac{\left| \frac{0.6 \wedge 0.7}{0.8} + \frac{1 \wedge 1}{0.9} + \frac{1 \wedge 1}{1} \right|}{\left| \frac{0.6 \vee 0.7}{0.8} + \frac{1 \vee 1}{0.9} + \frac{1 \vee 1}{1} \right|} \\ &= \frac{|0.6/0.8 + 1/0.9 + 1/1|}{|0.7/0.8 + 1/0.9 + 1/1|} = \frac{0.6 + 1 + 1}{0.7 + 1 + 1} = \frac{2.6}{2.7} = 0.96 \end{aligned}$$

$$D = 0.6/0.8 + 1/0.9 + 1/1$$

$$B = 0.5/0.6 + 0.7/0.7 + 1/0.8 + 1/0.9 + 1/1$$

$$E(D, B) = \frac{|D \cap B|}{|D \cup B|} = \frac{\left| \frac{0.0 \wedge 0.5}{0.6} + \frac{0.0 \wedge 0.7}{0.7} + \frac{0.6 \wedge 1}{0.8} + \frac{1 \wedge 1}{0.9} + \frac{1 \wedge 1}{1} \right|}{\left| \frac{0.0 \vee 0.5}{0.6} + \frac{0.0 \vee 0.7}{0.7} + \frac{0.6 \vee 1}{0.8} + \frac{1 \vee 1}{0.9} + \frac{1 \vee 1}{1} \right|}$$

$$= \frac{|0.6/0.8 + 1/0.9 + 1/1|}{|0.5/0.6 + 0.7/0.7 + 1/0.8 + 1/0.9 + 1/1|} = \frac{2.6}{4.2} = 0.62$$

$$D = 0.6/0.8 + 1/0.9 + 1/1$$

$$C = 0.9/0.7 + 1/0.9 + 0.6/1$$

$$E(D, C) = \frac{|D \cap C|}{|D \cup C|} = \frac{\left| \frac{0.0 \wedge 0.9}{0.7} + \frac{0.6 \wedge 0.0}{0.8} + \frac{1 \wedge 1}{0.9} + \frac{1 \wedge 0.6}{1} \right|}{\left| \frac{0.0 \vee 0.9}{0.7} + \frac{0.6 \vee 0.0}{0.8} + \frac{1 \vee 1}{0.9} + \frac{1 \vee 0.6}{1} \right|}$$

$$= \frac{\left| \frac{1}{0.9} + \frac{0.6}{1} \right|}{\left| \frac{0.9}{0.7} + \frac{0.6}{0.8} + \frac{1}{0.9} + \frac{1}{1} \right|} = \frac{1 + 0.6}{0.9 + 0.6 + 1 + 1} = \frac{1.6}{3.5} = 0.457$$

Since,

$$E(D, A) = 0.96$$

$$E(D, B) = 0.62$$

$$E(D, C) = 0.457$$

$E(D, A)$ has a higher value than $E(D, B)$ and $E(D, C)$, the fuzzy set D is most similar to set A and hence the linguistic approximation of fuzzy set D is the term A “*true*”.

9.9 Extension Principle

The extension principle [Zad7801] is a concept of fuzzy set theory that offers a way to map a M -tuple $(x_1, x_2, x_3, \dots, x_M)$ in a crisp set A to a point in the crisp set B and to generalize it to map M fuzzy subsets in A to a fuzzy subset in B .

Consequently, a mathematical relationship between non-fuzzy elements can be extended to fuzzy elements.

The extension principle is also useful for high-order fuzzy set operations.

The extension principle can be regarded as a special case of compositional rule of inference such that a fuzzy set B on Y can be deduced from a fuzzy set A on X through a fuzzy relation in the form of $y = f(x)$.

Suppose that $y = f(x)$ is a function from X to Y , and A is a fuzzy set on X , we have

$$A = \mu_A(x_1)/x_1 + \mu_A(x_2)/x_2 + \cdots + \mu_A(x_n)/x_n \quad (9.9.1)$$

Then by the extension principle, the image of A under $f(A)$ is a fuzzy set B such that

$$B = f(A) = \mu_A(x_1)/y_1 + \mu_A(x_2)/y_2 + \cdots + \mu_A(x_n)/y_n \quad (9.9.2)$$

where $y_i = f(x_i)$ for $i = 1$ to n .

$$B = f(A) = \mu_A[f^{-1}(y_1)]/y_1 + \mu_A[f^{-1}(y_2)]/y_2 + \cdots + \mu_A[f^{-1}(y_n)]/y_n \quad (9.9.3)$$

In other words, the fuzzy set B can be defined through the values of $y_i = f(x_i)$ for $x = \{x_1, x_2, \dots, x_n\}$.

Example 9.5 (Fuzzy set construction by extension principle)

Given $X = \{0.25, 0.50, 0.75, 1.00, 1.25, 1.50, 1.75\}$ and a fuzzy set $A = \text{small} = 1/0.25 + 0.9/0.50 + 0.7/0.75$, a fuzzy set $B = \text{small}^2$ can be constructed by the extension principle through the function $y = x^2$ as

$$B = \text{small}^2 = 1/0.0625 + 0.9/0.2500 + 0.7/0.5625$$

9.9.1 Extension principle of universe X to universe Y

Suppose that a function $y = f(x)$ is an m -to-1 mapping and there exists unequal values of $x_1, x_2, \dots, x_m \in X$, such that $f(x_1) =$

$f(x_2) = \dots = f(x_m) = y^*, y^* \in Y$, then the membership value of B at $y = y^*$ is the maximum of the membership values of A at $x = x_1, x_2, \dots, x_m$.

$$\mu_B(y) = \max_{x=f^{-1}(y)} \mu_A(x) \quad (9.9.4)$$

Example 9.6 (Fuzzy set mapping using quadratic function by extension principle)

Let a fuzzy set A with a discrete universe of discourse $X = \{-3, -2, -1, 0, 1, 2, 3\}$ be defined as $A = 0.5/-3 + 0.7/-2 + 0.9/-1 + 1.0/0 + 0.8/1 + 0.4/2 + 0.2/3$ and $y = f(x) = x^2 - 5$.

Apply the extension principle to find the fuzzy set B which is obtained from the image of the fuzzy set A under the mapping $y = f(x)$.

Draw a sagittal diagram to show the mapping.

Solution

Given $A = \mu_A(x_1)/x_1 + \mu_A(x_2)/x_2 + \dots + \mu_A(x_n)/x_n$ as

$$A = 0.5/-3 + 0.7/-2 + 0.9/-1 + 1.0/0 + 0.8/1 + 0.4/2 + 0.2/3$$

and $y = f(x) = x^2 - 5$,

then $B = f(A) = \mu_A(x_1)/y_1 + \mu_A(x_2)/y_2 + \dots + \mu_A(x_n)/y_n$

can be obtained as

$$\begin{aligned}
B &= 0.5/[-3]^2 - 5] + 0.7/[-2]^2 - 5] + 0.9/[-1]^2 - 5] \\
&\quad + 1/[0]^2 - 5] + 0.8/[1]^2 - 5] + 0.4/[2]^2 - 5] \\
&\quad + 0.2/[3]^2 - 5] \\
&= 0.5/4 + 0.7/-1 + 0.9/-4 + 1/-5 + 0.8/-4 \\
&\quad + 0.4/-1 + 0.2/4 \\
&= 1/-5 + (0.9 \vee 0.8)/-4 + (0.7 \vee 0.4)/-1 \\
&\quad + (0.5 \vee 0.2)/4 \\
&= 1/-5 + 0.9/-4 + 0.7/-1 + 0.5/4
\end{aligned}$$

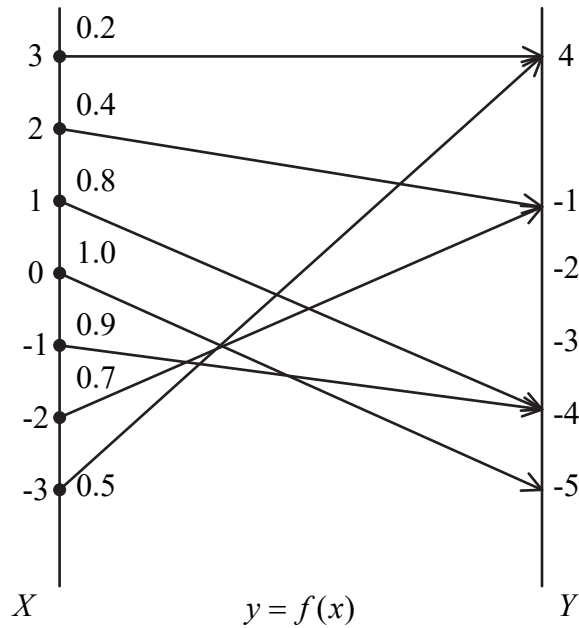


Figure E9.6.1 Extension principle on fuzzy sets with discrete universes.

The calculation steps to yield the sagittal diagram shown in Figure E9.6.1 are summarized in the Table E9.6.1.

Table E9.6.1 Fuzzy set mapping using quadratic function by extension principle.

x	$\mu_A(x)$	$y = f(x) = x^2 - 5$	$\mu_B(y)$
-3	0.5	4	$\max[0.5, 0.2]=0.5$
-2	0.7	-1	$\max[0.7, 0.4]=0.7$
-1	0.9	-4	$\max[0.9, 0.8]=0.9$
0	1.0	-5	$\max[1.0] = 1.0$
1	0.8	-4	$\max[0.9, 0.8]=0.9$
2	0.4	-1	$\max[0.7, 0.4]=0.7$
3	0.2	4	$\max[0.5, 0.2]=0.5$

Example 9.7 (Fuzzy set mapping using cubic polynomial by extension principle) Matlab

Given the membership function of a fuzzy set A defined as

$$\mu_A(x) = \text{TrapMF}(x, [-5, -4, -1, 3])$$

and $y = f(x) = (x^3 + 3x^2 - 4x + 30)/6$ for $x \in [-5, 3]$:

(a) Plot $\mu_A(x)$.

(b) Plot $y = f(x)$.

(c) Apply the extension principle to calculate the fuzzy set B which is the image of A under $y = f(x)$ and plot $\mu_B(y)$.

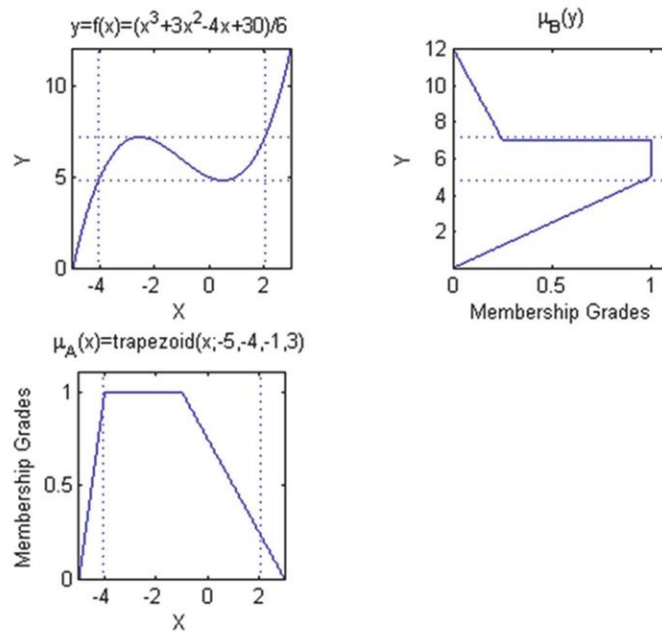
Solution

The fuzzy set mapping is shown in Figure E9.7.1.

For $-5 \leq x < -4.0534$ and $2.0549 < x \leq 3$, $f(x)$ is a one-to-one mapping, the fuzzy set B which is the image of A under $f(x)$ can be mapped according to Table E9.7.1.

Table E9.7.1 One-to-one mapping using cubic polynomial.

One-to-one mapping, $-5 \leq x < -4.0534$ and $2.0549 < x \leq 3$					
k	x	$\mu_A(x)$	y	$A \rightarrow B$	
				$A = \mu_A(x)/x$	$B = \mu_A(x)/y$
1	-5	0.0	0.0	0.0/-5	0.0/0
2	-4.5	0.5	2.9275	0.5/-4.5	0.5/2.9275
3	3	0.0	12	0.0/3	0.0/12

**Figure E9.7.1** Extension principle on fuzzy sets with continuous universes.

For the two points $x = [-4.0534, 2.0549]$, $f(x)$ is a two-to-one mapping, the max operator is used to obtain the membership functions of B when $y = [4.8125, 7.1875]$. The mapping of the two points $x = [-2.5, 2.0549]$ to $y = 7.1875$ results in a discontinuity of $\mu_B(y)$

at $y = 7.1875$. The mapping calculations are shown in Table E9.7.2.

Table E9.7.2 Two-to-one mapping using cubic polynomial.

Two-to-one mapping, $x = [-4.0534, 2.0549]$					
k	x	$\mu_A(x)$	y	$\mu_B(y)$	$B = \mu_B(y)/y$
1	-4.0534	0.9466	4.8125	$\max[0.9466, 0.625]=0.9466$	$0.9466/4.8125$
2	0.5	0.625	4.8125	$\max[0.9466, 0.625]=0.9466$	$0.9466/4.8125$
3	-2.5	1.0	7.1875	$\max[1.0, 0.236275]=1.0$	$1.0/7.1875$
4	2.0549	0.236275	7.1875	$\max[1.0, 0.236275]=1.0$	$1.0/7.1875$

As shown in Table E9.7.2, each pair of the two points in x are mapped to the same point in y as

$A \rightarrow B$	
For points $k = 1, 2$:	
$A = 0.9466/-4.0534 + 0.625/0.5$	
$\rightarrow B = (0.9466 \vee 0.625)/4.8125 = 0.9466/4.8125$	
For points $k = 3, 4$:	
$A = 1.0/-2.5 + 0.236275/2.0549$	
$\rightarrow B = (1.0 \vee 0.236275)/7.1875 = 1.0/7.1875$	

For $x \in [-4.0534, 2.0549]$, $f(x)$ is a three-to-one mapping, the max operator is used to obtain the membership functions of B when $y = 6$. The mapping calculations are shown in Table E9.7.3.

Table E9.7.3 Three-to-one mapping using cubic polynomial.

Three to one mapping, $x \in [-4.0534, 2.0549]$					
k	x	$\mu_A(y)$	y	$\mu_B(y)$	$B = \mu_B(y)/y$
1	-3.5	1.0	6.0	$\max[1.0, 1.0, 0.32]=1.0$	$1.0/6.0$
2	-1.0	1.0	6.0	$\max[1.0, 1.0, 0.32]=1.0$	$1.0/6.0$

3	1.5	0.32	6.0	$\max[1.0, 1.0, 0.32] = 1.0$	1.0/6.0
---	-----	------	-----	------------------------------	---------

As shown in Table E9.7.3, the three points in x are mapped to the same point in y as

$A \rightarrow B$
For points $k = 1, 2, 3$: $A = 1.0/-3.5 + 1.0/-1.0 + 0.32/1.5$ $\rightarrow B = (1.0 \vee 1.0 \vee 0.32)/6.0 = 1.0/6.0$

Similar mappings apply to any multiple x values map to the same $y \in [4.8125, 7.1875]$ where $\max [\mu_A(x)] = 1.0$. This gives rise to the plot of the whole region with $\mu_B(y) = 1$.

According to the extension principle as stated in (9.9.4), $\mu_B(y) = 1$ for $y \in [4.8125, 7.1875]$ as this y range includes $x \in [-4, -1]$ which has $\mu_A(x) = 1$.

9.9.2 Extension principle of universe $X_1 \times X_2 \times \dots \times X_M$ to universe Y

Suppose that a function $y = f(x_1, x_2, x_3, \dots, x_M)$ is a mapping from a M -dimensional Cartesian product of universes $X = X_1 \times X_2 \times \dots \times X_M$ to a one-dimensional universe Y such that $y_k = f(x_{1k}, x_{2k}, x_{3k}, \dots, x_{Mk})$ for one or more sets of $(x_{1k}, x_{2k}, x_{3k}, \dots, x_{Mk})$ in k ; and suppose $A_1, A_2, \dots, A_M$ are M fuzzy sets in $X_1, X_2, \dots, X_M$, the fuzzy set B induced is

$$\mu_B(y) = \max_{\substack{(x_{1k}, x_{2k}, \dots, x_{Mk}) = f^{-1}(y) \\ (x_{1k}, x_{2k}, \dots, x_{Mk}) \in X}} \left\{ \min_k [\mu_{A_1}(x_1), \mu_{A_2}(x_2), \dots, \mu_{A_M}(x_M)] \right\} \quad (9.9.5)$$

Example 9.8 (Two-dimensional to one-dimensional universal mapping using linear function by extension principle)

Suppose f is a function mapping ordered pairs from $X_1 = \{1, 3, 5\}$ and $X_2 = \{2, 4\}$ to $Y = \{3, 5, 7, 9\}$ and $y = f(x_1, x_2) = x_1 + x_2$.

Let A_1 and A_2 be fuzzy sets defined on X_1 and X_2 , respectively, such that $A_1 = 0.1/1 + 0.3/3 + 0.5/5$ and $A_2 = 0.2/2 + 0.4/4$.

Apply the extension principle to derive $B = f(A_1, A_2)$.

Solution

The calculations for the two-dimensional to one-dimensional universal mapping is shown in Table E9.8.1.

In Table E9.8.1, since there are two input membership functions, $\mu_{A_1}(x_1)$ and $\mu_{A_2}(x_2)$ their minimum membership values is taken as the input membership value.

Table E9.8.1 Two-dimensional to one-dimensional universal mapping using linear function by extension principle.

k	x_1	$\mu_{A_1}(x_1)$	x_2	$\mu_{A_2}(x_2)$	y	$\min\{\mu_{A_1}(x_1), \mu_{A_2}(x_2)\}$	$\mu_B(y)$
1	1	0.1	2	0.2	3	$\min\{0.1, 0.2\}=0.1$	$\max[0.1]=0.1$
2	1	0.1	4	0.4	5	$\min\{0.1, 0.4\}=0.1$	$\max[0.1, 0.2]=0.2$
3	3	0.3	2	0.2	5	$\min\{0.3, 0.2\}=0.2$	$\max[0.1, 0.2]=0.2$
4	3	0.3	4	0.4	7	$\min\{0.3, 0.4\}=0.3$	$\max[0.3, 0.2]=0.3$
5	5	0.5	2	0.2	7	$\min\{0.5, 0.2\}=0.2$	$\max[0.3, 0.2]=0.3$
6	5	0.5	4	0.4	9	$\min\{0.5, 0.4\}=0.4$	$\max[0.4]=0.4$

From the above calculations, it is clear that there are two ordered pairs given by

(a) $[x_1, x_2] = [1, 4]$ with membership functions $[\mu_{A_1}(x_1), \mu_{A_2}(x_2)] =$

$[0.1, 0.4]$, and (b) $[x_1, x_2] = [3, 2]$ with membership functions $[\mu_{A_1}(x_1), \mu_{A_2}(x_2)] = [0.3, 0.2]$, both are mapped to the same point $y = 5$.

(b) $[x_1, x_2] = [3, 4]$ with membership functions $[\mu_{A_1}(x_1), \mu_{A_2}(x_2)] = [0.3, 0.4]$, and (d) $[x_1, x_2] = [5, 2]$ with membership functions $[\mu_{A_1}(x_1), \mu_{A_2}(x_2)] = [0.5, 0.2]$, both are mapped to the same point $y = 7$.

Thus, their respective maximum membership values must be taken. The fuzzy set B can be obtained from the extension principle as

$$B = 0.1/3 + 0.2/5 + 0.3/7 + 0.4/9$$

Chapter 10

Fuzzy Logic and Reasoning

10.1 Introduction

This chapter covers the basics of fuzzy logic and reasoning in which the fuzzy relations between fuzzy sets A and B are first described in Section 10.2.

In Section 10.3, a fuzzy if-then rule is formulated as a binary fuzzy relation $R(X, Y)$ [Zad6506], [Zad7104b] on the product space $X \times Y$.

By interpreting fuzzy if-then rules as appropriate fuzzy relations, inference procedures based on the compositional rule of inference [Zad7301] are used in Section 10.4 to derive conclusions from a set of fuzzy if-then rules and known facts.

In Section 10.5, fuzzy combination of conclusions as an aggregated membership function is then explained.

Methods for defuzzification of an aggregated output membership function to give a crisp output are described in Section 10.6. In particular, a fast method for computing centroid is given.

The above procedures form the basics of fuzzy reasoning as described in Section 10.7 in which the computational aspects of the generalized modus ponens based fuzzy reasoning are presented and illustrated using an example on multiple rules with multiple antecedents.

An earlier reference for fuzzy logic and reasoning is given by [Jan9700]. In addition to the notations introduced in Table 9.2.1, some more notations used in this chapter are given in Table 10.1.1.

Table 10.1.1 Notations for fuzzy logic and reasoning.

Symbol	Description
$R(X, Y)$	Binary fuzzy relation between a fuzzy set X and a fuzzy set Y .
$R^{-1}(Y, X)$	Inverse of binary fuzzy relation between a fuzzy set X and a fuzzy set Y .
$\mathbf{R}$	$\mathbf{R} = [R(x, y)]$ is the membership matrix of the binary fuzzy relation $R(X, Y)$.
$\wedge$	Minimum operation.
$\vee$	Maximum operation.
$\vee_x$	Maximum operation along x .

10.2 Fuzzy Relations

Fuzzy relation generalizes relation in the classical sense into one that expresses a partial relation (or membership value) between two or more fuzzy sets.

In other words, fuzzy relations represent nonzero degrees of association between the elements of two or more fuzzy sets.

A fuzzy relation describes the degree of connection for example between two persons which is more meaningful as compared to “related” or “unrelated” in the classical sense.

Unary fuzzy relations are fuzzy sets in X that maps each element in X to a membership value between 0 and 1. In other words, unary fuzzy relations are fuzzy sets with one-dimensional membership functions.

Binary fuzzy relations [Zad7301], [Jan9700] are fuzzy sets in $X \times Y$ that maps each element in $X \times Y$ to a membership value between 0 and 1.

Binary fuzzy relations are fuzzy sets with two-dimensional membership functions.

10.2.1 Binary fuzzy relations

A binary fuzzy relation (BFR) $R(X, Y)$ in $X \times Y$, in which X and Y denote two universes of discourse, can be expressed in term of a two-dimensional membership function $\mu_R(x, y)$ such that

$$R(X, Y) = \{[(x, y), \mu_R(x, y)] | (x, y) \in X \times Y\} \quad (10.2.1)$$

Equivalently, (10.2.1) can be expressed as

$$R(X, Y) = \begin{bmatrix} \mu_R(x_1, y_1) & \mu_R(x_1, y_2) & \cdots & \mu_R(x_1, y_M) \\ \mu_R(x_2, y_1) & \mu_R(x_2, y_2) & \cdots & \mu_R(x_2, y_M) \\ \vdots & \vdots & \ddots & \vdots \\ \mu_R(x_N, y_1) & \mu_R(x_N, y_2) & \cdots & \mu_R(x_N, y_M) \end{bmatrix} = \mathbf{R} \quad (10.2.2)$$

A binary fuzzy relation $R(X, Y)$ can be expressed in term of a binary fuzzy relation matrix $\mathbf{R}$ as shown in (10.2.3) or by a sagittal diagram as shown in Figure 10.2.1.

$$\mathbf{R} = \begin{bmatrix} 0.7 & 0.4 & 0.1 \\ 0.5 & 0.9 & 0.4 \\ 0.2 & 0.5 & 0.8 \end{bmatrix} \quad (10.2.3)$$

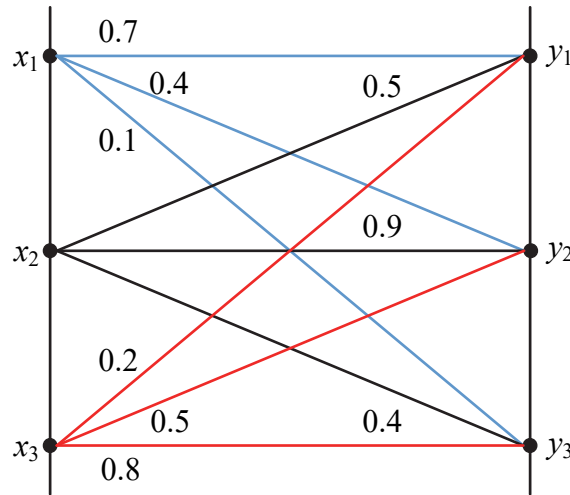


Figure 10.2.1 Sagittal diagram of a binary fuzzy relation $R(X, Y)$.

Some examples of binary fuzzy relations are

- x is smaller than y .
- x changes with y .
- x and y looks the same.
- If x is high then y is low.

Example 10.1 (Compute the binary fuzzy relation matrix)

Let $R(X, Y) = “x \text{ is close to } y”$ and $X = Y = \mathbf{R}^+$, the membership function of the fuzzy relation $R(X, Y)$ is defined as

$$\mu_R(x, y) = e^{-|y-x|}$$

Given $X = \{1, 2, 3, 4\}$ and $Y = \{1, 2, 3, 4, 5, 6\}$, compute the binary fuzzy relation matrix $\mathbf{R} = R(X, Y) = \{[(x, y), \mu_R(x, y)] | (x, y) \in X \times Y\}$.

Solution

From $\mu_R(x, y)$, the binary fuzzy relation matrix $\mathbf{R}$ can be computed as

$$\mathbf{R} = \begin{bmatrix} 1.0000 & 0.3679 & 0.1353 & 0.0498 & 0.0183 & 0.0067 \\ 0.3679 & 1.0000 & 0.3679 & 0.1353 & 0.0498 & 0.0183 \\ 0.1353 & 0.3679 & 1.0000 & 0.3679 & 0.1353 & 0.0498 \\ 0.0498 & 0.1353 & 0.3679 & 1.0000 & 0.3679 & 0.1353 \end{bmatrix}$$

The (i, j) th element for $1 \leq i \leq 4$ and $1 \leq j \leq 6$ of the fuzzy relation matrix $\mathbf{R}$ expresses the degree of membership of $R(X, Y) = “x \text{ is close to } y”$.

10.2.2 Composition of two fuzzy relations in same product space $X \times Y$

The operation $R_1 \circ R_2$ means a composition of 2 fuzzy relations $R_1(X, Y)$ and $R_2(X, Y)$. Fuzzy relations in the same product space can be combined through a compositional operator.

The two common compositional operators are intersection ($\cap$) and union ($\cup$) which can be defined as

$$\mu_{R_1 \cap R_2}(x, y) = \mu_{R_1}(x, y) * \mu_{R_2}(x, y) \quad (10.2.4)$$

$$\mu_{R_1 \cup R_2}(x, y) = \mu_{R_1}(x, y) \oplus \mu_{R_2}(x, y) \quad (10.2.5)$$

where $*$ is any T -norm operator (such as $\min$) and $\oplus$ is any T -conorm operator (such as $\max$).

10.2.3 Composition of two fuzzy relations in different product spaces $X \times Y$ and $Y \times Z$

Consider the composition of two fuzzy relations from different product spaces that share a common set, such as $R_1(X, Y)$ and $R_2(Y, Z)$.

If $R_1(X, Y)$ and $R_2(Y, Z)$ are represented in discrete universes of discourse, then $R_1 \circ R_2$ can be represented either by a fuzzy relational matrix or a sagittal diagram.

Max-Star composition

In general, $R_1 \circ R_2$ means Max-Star composition of two fuzzy relations $R_1(X, Y)$ and $R_2(Y, Z)$ in different product spaces. Max-star can be max-min or max-product.

Max-star (or Max- $*$) composition is defined as follows

$$\mu_{R_1 \circ R_2}(x, z) = \max_y [\mu_{R_1}(x, y) * \mu_{R_2}(y, z)] \quad (10.2.6)$$

where $*$ is a T -norm operator.

Minimum and algebraic product are common T -norm operators. In other words, max-min composition and max-product composition are subsets of max-star composition.

10.2.3.1 Max-Min composition

Fuzzy relations in different product spaces can be combined through a compositional operation. Different compositional operations have been suggested for fuzzy relations.

The best known one is max-min composition. The max-min composition of $R_1(X, Y)$ and $R_2(Y, Z)$ is a fuzzy set, in which

$R_1(X, Y)$ and $R_2(Y, Z)$ are two fuzzy relations defined on $X \times Y$ and $Y \times Z$ respectively, which is defined by

$$\mu_{R_1 \circ R_2}(x, z) = \max_y \left\{ \min[\mu_{R_1}(x, y), \mu_{R_2}(y, z)] \right\} \quad (10.2.7)$$

Let $P(X, Y)$, $Q(Y, Z)$, and $R(Z, W)$ be binary fuzzy relations on $X \times Y$, $Y \times Z$, and $Z \times W$, the following properties listed in Table 10.2.1 apply for max-min composition.

Table 10.2.1 Properties of Max-Min composition.

Name	Property	Equation
Associativity	$P \circ (Q \circ R) = (P \circ Q) \circ R$	(10.2.8)
Distributivity over union	$P \circ (Q \cup R) = (P \circ Q) \cup (P \circ R)$	(10.2.9)
Monotonicity	$Q \supseteq R \Rightarrow (P \circ Q) \supseteq (P \circ R)$	(10.2.10)
Weak distributivity over intersection	$P \circ (Q \cap R) \subseteq (P \circ Q) \cap (P \circ R)$	(10.2.11)

10.2.3.2 Max-Product composition

Although max-min composition is commonly used, it is not amenable to mathematical analysis. For mathematical tractability, max-product composition offers an alternative to max-min composition.

Given two fuzzy relations $R_1(X, Y)$ and $R_2(Y, Z)$ defined on $X \times Y$ and $Y \times Z$ respectively, the max-product composition of $R_1(X, Y)$ and $R_2(Y, Z)$ is a fuzzy set defined by

$$\mu_{R_1 \circ R_2}(x, z) = \max_y [\mu_{R_1}(x, y) \times \mu_{R_2}(y, z)] \quad (10.2.12)$$

Let $P(X, Y)$, $Q(Y, Z)$, and $R(Z, W)$ be binary fuzzy relations on $X \times Y$, $Y \times Z$, and $Z \times W$, the max-product composition shares the same set of properties as the max-min composition in associativity as (10.2.8), distributivity over union as (10.2.9), and monotonicity as (10.2.10).

The weak distributivity over intersection property defined in (10.2.11)

holds for max-product composition subject to certain condition.

Example 10.2 (Choice between programs using fuzzy relation matrices)

Two high-school students, Student 1 and Student 2, who have different interests for subjects Languages, Mathematics, and Sciences are required to choose one of the following two degree programs, Business or Engineering. Let

$$X = \{x_1, x_2\} = \{\text{Student 1, Student 2}\},$$

$$Y = \{y_1, y_2, y_3\} = \{\text{Languages, Mathematics, Sciences}\},$$

$$Z = \{z_1, z_2\} = \{\text{Business, Engineering}\},$$

the “students to subjects” are represented by the fuzzy relation $R_1(X, Y)$, and the “subjects to programs” are represented by the fuzzy relation $R_2(Y, Z)$.

Assume that $R_1(X, Y)$ and $R_2(Y, Z)$ can be expressed as the following fuzzy relation matrices

$$R_1 = \begin{bmatrix} 0.9 & 0.7 & 0.5 \\ 0.6 & 0.8 & 0.9 \end{bmatrix}, \quad R_2 = \begin{bmatrix} 1.0 & 0.6 \\ 0.6 & 0.8 \\ 0.3 & 1.0 \end{bmatrix}$$

(a) Explain how to determine the relation between the element x_2 in X and the element z_1 in Z . Draw a sagittal diagram to illustrate the composition of such fuzzy relations.

(b) Compute $R_1 \circ R_2$ using

(i) Max-min composition

(ii) Max-product composition

(iii) Max-average composition

(c) Based on each of the results of (b)(i), (ii), and (iii), derive a conclusion for recommending an appropriate program for each of the two students. Compare the composition results among (b)(i), (ii), and (iii).

Solution

(a) For simplicity, suppose that we are only interested in the degree of relevance between $x_2 (\in X)$ and $z_1 (\in Z)$.

$x_2 (\in X)$ refers to the 2nd row in $\mu_{R_1}(x, y)$, which means $\mu_R(x_2, y)$.

$z_1 (\in Z)$ refers to the 1st column in $\mu_{R_2}(y, z)$, which means $\mu_R(y, z_1)$.

The relation between the element x_2 in X and the element z_1 in Z is built up via the three possible paths connecting these 2 elements as shown in Figure E10.2.1.

The degree of relevance between x_2 and z_1 is the maximum of these three paths' strengths.

Depending on either max-min or max-product or max-average composition, each path's strength is the minimum or product or average of the strengths of its constituent links.

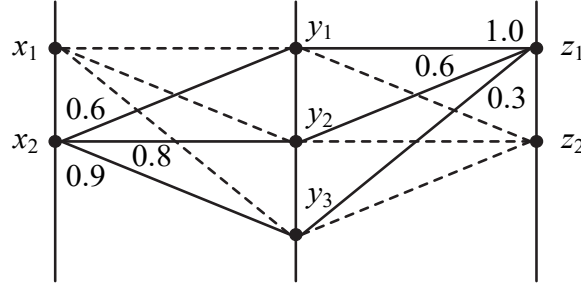


Figure E10.2.1 Composition of fuzzy relations $\mu_{R_1 \circ R_2}(x_2, z_1)$.

(b) (i) Max-min composition

$$\begin{aligned}
 \mu_{R_1 \circ R_2}(x, z) &= \max_y \left\{ \min[\mu_{R_1}(x, y), \mu_{R_2}(y, z)] \right\} \\
 &= \begin{bmatrix} \max\{0.9 \wedge 1.0, 0.7 \wedge 0.6, 0.5 \wedge 0.3\} & \max\{0.9 \wedge 0.6, 0.7 \wedge 0.8, 0.5 \wedge 1.0\} \\ \max\{0.6 \wedge 1.0, 0.8 \wedge 0.6, 0.9 \wedge 0.3\} & \max\{0.6 \wedge 0.6, 0.8 \wedge 0.8, 0.9 \wedge 1.0\} \end{bmatrix} \\
 &= \begin{bmatrix} \max\{0.9, 0.6, 0.3\} & \max\{0.6, 0.7, 0.5\} \\ \max\{0.6, 0.6, 0.3\} & \max\{0.6, 0.8, 0.9\} \end{bmatrix} = \begin{bmatrix} 0.9 & 0.7 \\ 0.6 & 0.9 \end{bmatrix}
 \end{aligned}$$

(c) (ii) Max-product composition

$$\begin{aligned}
 \mu_{R_1 \circ R_2}(x, z) &= \max_y [\mu_{R_1}(x, y) \times \mu_{R_2}(y, z)] \\
 &= \begin{bmatrix} \max\{0.9 \times 1.0, 0.7 \times 0.6, 0.5 \times 0.3\} & \max\{0.9 \times 0.6, 0.7 \times 0.8, 0.5 \times 1.0\} \\ \max\{0.6 \times 1.0, 0.8 \times 0.6, 0.9 \times 0.3\} & \max\{0.6 \times 0.6, 0.8 \times 0.8, 0.9 \times 1.0\} \end{bmatrix} \\
 &= \begin{bmatrix} \max\{0.90, 0.42, 0.15\} & \max\{0.54, 0.56, 0.50\} \\ \max\{0.60, 0.48, 0.27\} & \max\{0.36, 0.64, 0.90\} \end{bmatrix} = \begin{bmatrix} 0.90 & 0.56 \\ 0.60 & 0.90 \end{bmatrix}
 \end{aligned}$$

(b) (iii) Max-Average composition

$$\begin{aligned}
 \mu_{R_1 \circ R_2}(x, z) &= \max_y \left[\frac{\mu_{R_1}(x, y) + \mu_{R_2}(y, z)}{2} \right] \\
 &= \begin{bmatrix} \max \left[\frac{0.9 + 1.0}{2}, \frac{0.7 + 0.6}{2}, \frac{0.5 + 0.3}{2} \right] & \max \left[\frac{0.9 + 0.6}{2}, \frac{0.7 + 0.8}{2}, \frac{0.5 + 1.0}{2} \right] \\ \max \left[\frac{0.6 + 1.0}{2}, \frac{0.8 + 0.6}{2}, \frac{0.9 + 0.3}{2} \right] & \max \left[\frac{0.6 + 0.6}{2}, \frac{0.8 + 0.8}{2}, \frac{0.9 + 1.0}{2} \right] \end{bmatrix} \\
 &= \begin{bmatrix} \max[0.95, 0.65, 0.40] & \max[0.75, 0.75, 0.75] \\ \max[0.80, 0.70, 0.60] & \max[0.60, 0.80, 0.95] \end{bmatrix} = \begin{bmatrix} 0.95 & 0.75 \\ 0.80 & 0.95 \end{bmatrix}
 \end{aligned}$$

(c) Based on the conclusions obtained in (b)(i)-(iii), the Student 1 is recommended to take the Business program and the Student 2 is recommended to take the Engineering program.

Comparing the results among the three compositions, the result of the max-average composition is the most fuzzy (with most similar output values) and that of the max-product composition is the least fuzzy (with least similar output values).

10.3 Fuzzy If-Then Rules

Let A and B are linguistic values defined respectively by fuzzy sets on universes of discourse X and Y , a fuzzy if-then rule takes the following form

If x is A then y is B

In the above if-then rule: “ x is A ” is called the antecedent and “ y is B ” is called the consequence. Some examples of the fuzzy if-then rules are

- a. If rain is heavy, then water level in reservoir is high.
- b. If street light is dim, then walking is difficult.
- c. If an orange's skin is sticky, then the orange is juicy.

Before fuzzy if-then rules can be applied, the meaning of the expression “if x is A then y is B ” or $A \rightarrow B$ for $x \in A$ and $y \in B$ has to be formalized.

The expression has a membership function $\mu_{A \rightarrow B}(x, y)$ which measures the degree of truth of the implication relation between x and y .

A number of induction methods can be formulated to calculate the fuzzy relation $R = A \rightarrow B$. Viewing R as a fuzzy set with a two-dimensional membership function

$$\mu_R(x, y) = f[\mu_A(x), \mu_B(y)] \quad (10.3.1)$$

where the fuzzy implication function $f[.]$ performs the task of transforming the membership values, $\mu_A(x)$ of x in A and $\mu_B(y)$ of y in B , into those of (x, y) in $R = A \rightarrow B$.

The two widely used fuzzy implication functions, which preserve cause and effect, are given below.

The Mamdani's minimum implication function [Mam7501]

$$\mu_R(x, y) = \min[\mu_A(x), \mu_B(y)] \quad (10.3.2)$$

The Larsen's algebraic product implication function [Lar8001]

$$\mu_R(x, y) = \mu_A(x) \times \mu_B(y) \quad (10.3.3)$$

Each of the expressions (10.3.2) and (10.3.3) describes a relation between two variables x and y .

This implies that a fuzzy if-then rule can be defined as a binary fuzzy relation $R(X, Y)$ on the product space $X \times Y$.

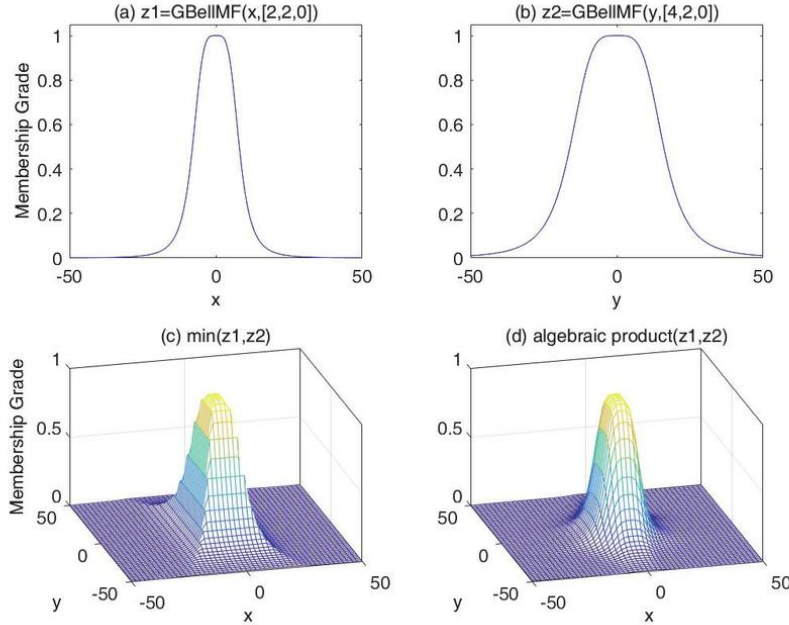


Figure 10.3.1 Transformation by fuzzy implication functions: (a) z_1 ; (b) z_2 ; (c) $\min(z_1, z_2)$; (d) $z_1 * z_2$.

Figure 10.3.1 shows the transformed binary fuzzy relations $R_1(X, Y)$ and $R_2(X, Y)$ ($R_1, R_2 = A \rightarrow B$), with $z_1 = \mu_A(X) = \text{GBellMF}(x, [2, 2, 0])$ and $z_2 = \mu_B(y) = \text{GBellMF}(y, [4, 2, 0])$, where $x \in X$ for $-50 \leq x \leq 50$ and $y \in Y$ for $-50 \leq y \leq 50$.

10.4 Compositional Rule of Inference

In Section 10.2.3, the idea of compositional rule of inference has been employed to explain the general max-* composition, and in particular the max-min and max-product compositions of fuzzy relation matrices.

Let $R(X, Y)$ be a fuzzy relation on $X \times Y$ (Figure 10.4.1(a)) and A is a fuzzy set of X . To find the resulting fuzzy set B , a cylindrical extension $c(A)$ with the base A can be constructed (Figure 10.4.1(b)).

The intersection of $c(A)$ and $R(X, Y)$ forms the region of intersection (Figure 10.4.1(c)). By projecting $c(A) \cap R(X, Y)$ onto the y -axis, a

fuzzy set B on the y -axis can be obtained (Figure 10.4.1(d)).

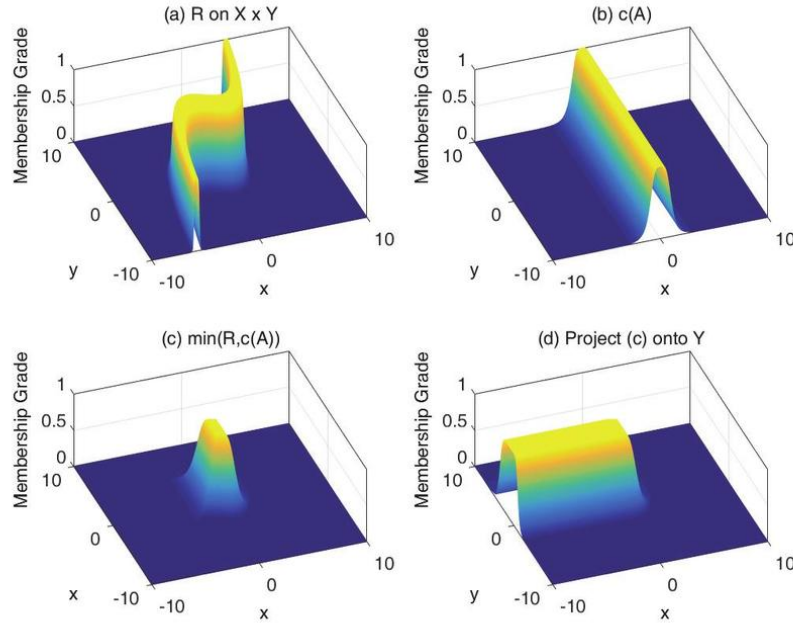


Figure 10.4.1 Compositional rule of inference: (a) $R(X, Y)$; (b) $c(A)$; (c) $\min(R(X, Y), c(A))$; (d) $\mu_B(y)$.

Let fuzzy sets A, B , and the cylindrical extension of $c(A)$, and fuzzy relation $R(X, Y)$, have the respective membership functions $\mu_A(x)$, $\mu_B(y)$, $\mu_{c(A)}(x, y)$ and $\mu_R(x, y)$, $\mu_{c(A)}(x, y)$ is related to $\mu_A(x)$ by

$$\mu_{c(A)}(x, y) = \mu_A(x) \quad (10.4.1)$$

The intersection of $c(A)$ and $R(X, Y)$ gives

$$\begin{aligned} \mu_{c(A) \cap R(X, Y)}(x, y) \\ = \min[\mu_{c(A)}(x, y), \mu_R(x, y)] = \min[\mu_A(x), \mu_R(x, y)] \end{aligned} \quad (10.4.2)$$

Projecting $c(A) \cap R(X, Y)$ onto the y -axis gives

$$\mu_B(y) = \max_x \{ \min[\mu_A(x), \mu_R(x, y)] \} \quad (10.4.3)$$

The above mathematical operations are illustrated in Figure 10.4.1. If both A (a unary fuzzy relation) and $R(X, Y)$ (a binary fuzzy relation) have finite universes of discourse, (10.4.3) reduces to the Max-Min composition of two fuzzy relation matrices.

From (10.4.3), B can be expressed as the composition of A and $R(X, Y)$

$$B = A \circ R \quad (10.4.4)$$

Using the compositional rule of inference, an inference procedure, generally called fuzzy reasoning, can be formalized on a set of fuzzy if-then rules.

Example 10.3 (Linguistic variables and interpretations)

Let the discrete universe of discourse of $X, Y = \{1, 2, 3, 4, 5\}$ and the term set $\{A, B, C\}$ of the linguistic variable “*large*”. Given the facts that

$$A = \text{“large”} = 0.1/1 + 0.2/2 + 0.6/3 + 0.8/4 + 0.9/5$$

and a fuzzy relation

$R(X, Y) = X$ and Y are “*more or less the same*” as

$$R(X, Y) = 1/(1,1) + 1/(2,2) + 1/(3,3) + 1/(4,4) + 1/(5,5) + 0.7/(1,2) + 0.7/(2,3) + 0.7/(3,4) + 0.7/(4,5) + 0.7/(2,1) + 0.7/(3,2) + 0.7/(4,3) + 0.7/(5,4) \text{ such that}$$

$$R(X, Y) = \begin{bmatrix} 1 & 0.7 & 0.0 & 0.0 & 0.0 \\ 0.7 & 1 & 0.7 & 0.0 & 0.0 \\ 0.0 & 0.7 & 1 & 0.7 & 0.0 \\ 0.0 & 0.0 & 0.7 & 1 & 0.7 \\ 0.0 & 0.0 & 0.0 & 0.7 & 1 \end{bmatrix}$$

(a) Derive the fuzzy set $B = \text{“very large”}$ and the fuzzy set $C = \text{“more or less large”}$.

(b) Derive the conclusion $A \circ R$ using

(i) Max-min composition rule of inference;

- (ii) Max-product composition rule of inference; and
 (iii) Interpret the linguistic meaning of each derived conclusion.

Solution

(a) For $B = \text{"very large"}$

$$\begin{aligned} B &= \text{CON}(X) = X^2 \\ &= 0.1^2/1 + 0.2^2/2 + 0.6^2/3 + 0.8^2/4 + 0.9^2/5 \\ &= 0.01/1 + 0.04/2 + 0.36/3 + 0.64/4 + 0.81/5 \end{aligned}$$

For $C = \text{"more or less large"}$

$$\begin{aligned} C &= \text{DIL}(X) = X^{0.5} \\ &= 0.1^{0.5}/1 + 0.2^{0.5}/2 + 0.6^{0.5}/3 + 0.8^{0.5}/4 + 0.9^{0.5}/5 \\ &= 0.3162/1 + 0.4472/2 + 0.7746/3 + 0.8944/4 + 0.9487/5 \end{aligned}$$

(b) (i) According to the composition rule of inference, the conclusion is $Y_1 = A \circ R$, where $A \circ R$ refers to the max-min composition of a fuzzy set A and a fuzzy relation $R(X, Y)$. Hence, we obtain

$$\begin{aligned} Y_1 &= A \circ R \\ &= [0.1 \quad 0.2 \quad 0.6 \quad 0.8 \quad 0.9] \circ \begin{bmatrix} 1 & 0.7 & 0.0 & 0.0 & 0.0 \\ 0.7 & 1 & 0.7 & 0.0 & 0.0 \\ 0.0 & 0.7 & 1 & 0.7 & 0.0 \\ 0.0 & 0.0 & 0.7 & 1 & 0.7 \\ 0.0 & 0.0 & 0.0 & 0.7 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \vee_x [0.1 \wedge 1.0 & 0.2 \wedge 0.7 & 0.6 \wedge 0.0 & 0.8 \wedge 0.0 & 0.9 \wedge 0.0] \\ \vee_x [0.1 \wedge 0.7 & 0.2 \wedge 1.0 & 0.6 \wedge 0.7 & 0.8 \wedge 0.0 & 0.9 \wedge 0.0] \\ \vee_x [0.1 \wedge 0.0 & 0.2 \wedge 0.7 & 0.6 \wedge 1.0 & 0.8 \wedge 0.7 & 0.9 \wedge 0.0] \\ \vee_x [0.1 \wedge 0.0 & 0.2 \wedge 0.0 & 0.6 \wedge 0.7 & 0.8 \wedge 1.0 & 0.9 \wedge 0.7] \\ \vee_x [0.1 \wedge 0.0 & 0.2 \wedge 0.0 & 0.6 \wedge 0.0 & 0.8 \wedge 0.7 & 0.9 \wedge 1.0] \end{bmatrix}^T \\ &= [0.2 \quad 0.6 \quad 0.7 \quad 0.8 \quad 0.9] \end{aligned}$$

Using the similarity measure of (9.8.11), we obtain

$$\begin{aligned}
E(Y_1, X) &= \frac{|Y_1 \cap X|}{|Y_1 \cup X|} \\
&= \frac{|(0.2 \wedge 0.1)/1 + (0.6 \wedge 0.2)/2 + (0.7 \wedge 0.6)/3 + (0.8 \wedge 0.8)/4 + (0.9 \wedge 0.9)/5|}{|(0.2 \vee 0.1)/1 + (0.6 \vee 0.2)/2 + (0.7 \vee 0.6)/3 + (0.8 \vee 0.8)/4 + (0.9 \vee 0.9)/5|} \\
&= \frac{|0.1/1 + 0.2/2 + 0.6/3 + 0.8/4 + 0.9/5|}{|0.2/1 + 0.6/2 + 0.7/3 + 0.8/4 + 0.9/5|} = \frac{0.1 + 0.2 + 0.6 + 0.8 + 0.9}{0.2 + 0.6 + 0.7 + 0.8 + 0.9} \\
&= \frac{2.6}{3.2} = 0.8125
\end{aligned}$$

$$\begin{aligned}
E(Y_1, X^2) &= \frac{|Y_1 \cap X^2|}{|Y_1 \cup X^2|} \\
&= \frac{|(0.20 \wedge 0.01)/1 + (0.60 \wedge 0.04)/2 + (0.70 \wedge 0.36)/3 + (0.80 \wedge 0.64)/4 + (0.90 \wedge 0.81)/5|}{|(0.20 \vee 0.01)/1 + (0.60 \vee 0.04)/2 + (0.70 \vee 0.36)/3 + (0.80 \vee 0.64)/4 + (0.90 \vee 0.81)/5|} \\
&= \frac{|0.01/1 + 0.04/2 + 0.36/3 + 0.64/4 + 0.81/5|}{|0.20/1 + 0.60/2 + 0.70/3 + 0.80/4 + 0.90/5|} \\
&= \frac{0.01 + 0.04 + 0.36 + 0.64 + 0.81}{0.20 + 0.60 + 0.70 + 0.80 + 0.90} = \frac{1.86}{3.20} = 0.58125
\end{aligned}$$

$$\begin{aligned}
E(Y_1, X^{0.5}) &= \frac{|Y_1 \cap X^{0.5}|}{|Y_1 \cup X^{0.5}|} \\
&= \frac{|(0.2 \wedge 0.3162)/1 + (0.6 \wedge 0.4472)/2 + (0.7 \wedge 0.7746)/3 + (0.8 \wedge 0.8367)/4 + (0.9 \wedge 0.9487)/5|}{|(0.2 \vee 0.3162)/1 + (0.6 \vee 0.4472)/2 + (0.7 \vee 0.7746)/3 + (0.8 \vee 0.8367)/4 + (0.9 \vee 0.9487)/5|} \\
&= \frac{|0.2/1 + 0.4472/2 + 0.7/3 + 0.8/4 + 0.9/5|}{|0.3162/1 + 0.6/2 + 0.7746/3 + 0.8944/4 + 0.9487/5|} \\
&= \frac{0.2 + 0.4472 + 0.7 + 0.8 + 0.9}{0.3162 + 0.6 + 0.7746 + 0.8944 + 0.9487} \\
&= \frac{3.0472}{3.5339} = 0.86228
\end{aligned}$$

Based on the similarity measures between the linguistic terms, and Y_1 is close to X by 0.8125, Y_1 is close to X^2 by 0.58125, and Y_1 is close to $X^{0.5}$ by 0.86228, therefore the linguistic meaning of Y_1 is $X^{0.5} = \text{"more or less large"}$ and followed closely by $X = \text{"large"}$.

(b) (ii) According to the composition rule of inference, the conclusion is $Y_2 = A \circ R$, where $A \circ R$ refers to the max-product composition of a fuzzy set A and a fuzzy relation $R(X, Y)$. Hence, we obtain

$$\begin{aligned}
 Y_2 &= A \circ R \\
 &= [0.1 \quad 0.2 \quad 0.6 \quad 0.8 \quad 0.9] \circ \begin{bmatrix} 1 & 0.7 & 0.0 & 0.0 & 0.0 \\ 0.7 & 1 & 0.7 & 0.0 & 0.0 \\ 0.0 & 0.7 & 1 & 0.7 & 0.0 \\ 0.0 & 0.0 & 0.7 & 1 & 0.7 \\ 0.0 & 0.0 & 0.0 & 0.7 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \vee_x [0.1 \times 1.0 & 0.2 \times 0.7 & 0.6 \times 0.0 & 0.8 \times 0.0 & 0.9 \times 0.0] \\ \vee_x [0.1 \times 0.7 & 0.2 \times 1.0 & 0.6 \times 0.7 & 0.8 \times 0.0 & 0.9 \times 0.0] \\ \vee_x [0.1 \times 0.0 & 0.2 \times 0.7 & 0.6 \times 1.0 & 0.8 \times 0.7 & 0.9 \times 0.0] \\ \vee_x [0.1 \times 0.0 & 0.2 \times 0.0 & 0.6 \times 0.7 & 0.8 \times 1.0 & 0.9 \times 0.7] \\ \vee_x [0.1 \times 0.0 & 0.2 \times 0.0 & 0.6 \times 0.0 & 0.8 \times 0.7 & 0.9 \times 1.0] \end{bmatrix}^T \\
 &= [0.14 \quad 0.42 \quad 0.60 \quad 0.80 \quad 0.90]
 \end{aligned}$$

Using the similarity measure of (9.8.11), we obtain

$$\begin{aligned}
 E(Y_2, X) &= \frac{|Y_2 \cap X|}{|Y_2 \cup X|} \\
 &= \frac{|(0.14 \wedge 0.10) / 1 + (0.42 \wedge 0.20) / 2 + (0.60 \wedge 0.60) / 3 + (0.80 \wedge 0.80) / 4 + (0.90 \wedge 0.90) / 5|}{|(0.14 \vee 0.10) / 1 + (0.42 \vee 0.20) / 2 + (0.60 \vee 0.60) / 3 + (0.80 \vee 0.80) / 4 + (0.90 \vee 0.90) / 5|} \\
 &= \frac{|0.10 / 1 + 0.20 / 2 + 0.60 / 3 + 0.80 / 4 + 0.90 / 5|}{|0.14 / 1 + 0.42 / 2 + 0.60 / 3 + 0.80 / 4 + 0.90 / 5|} \\
 &= \frac{0.1 + 0.2 + 0.6 + 0.8 + 0.9}{0.14 + 0.42 + 0.6 + 0.8 + 0.9} = \frac{2.60}{2.86} = 0.90909 \\
 E(Y_2, X^2) &= \frac{|Y_2 \cap X^2|}{|Y_2 \cup X^2|}
 \end{aligned}$$

$$\begin{aligned}
&= \frac{|(0.14 \wedge 0.01) / 1 + (0.42 \wedge 0.04) / 2 + (0.60 \wedge 0.36) / 3 + (0.80 \wedge 0.64) / 4 + (0.90 \wedge 0.81) / 5|}{|(0.14 \vee 0.01) / 1 + (0.42 \vee 0.04) / 2 + (0.60 \vee 0.36) / 3 + (0.80 \vee 0.64) / 4 + (0.90 \vee 0.81) / 5|} \\
&= \frac{|0.01 / 1 + 0.04 / 2 + 0.36 / 3 + 0.64 / 4 + 0.81 / 5|}{|0.14 / 1 + 0.42 / 2 + 0.60 / 3 + 0.80 / 4 + 0.90 / 5|} \\
&= \frac{0.01 + 0.04 + 0.36 + 0.64 + 0.81}{0.14 + 0.42 + 0.60 + 0.80 + 0.90} = \frac{1.86}{2.86} = 0.65035
\end{aligned}$$

$$\begin{aligned}
E(Y_2, X^{0.5}) &= \frac{|Y_2 \cap X^{0.5}|}{|Y_2 \cup X^{0.5}|} \\
&= \frac{|(0.14 \wedge 0.3162) / 1 + (0.42 \wedge 0.4472) / 2 + (0.6 \wedge 0.7746) / 3 + (0.8 \wedge 0.8944) / 4 + (0.9 \wedge 0.9487) / 5|}{|(0.14 \vee 0.3162) / 1 + (0.42 \vee 0.4472) / 2 + (0.6 \vee 0.7746) / 3 + (0.8 \vee 0.8944) / 4 + (0.9 \vee 0.9487) / 5|} \\
&= \frac{|0.14 / 1 + 0.42 / 2 + 0.6 / 3 + 0.8 / 4 + 0.9 / 5|}{|0.3162 / 1 + 0.4472 / 2 + 0.7746 / 3 + 0.8944 / 4 + 0.9487 / 5|} \\
&= \frac{0.14 + 0.42 + 0.6 + 0.8 + 0.9}{0.3162 + 0.4472 + 0.7746 + 0.8944 + 0.9487} = \frac{2.8600}{3.3811} = 0.84588
\end{aligned}$$

Based on the similarity measures between the linguistic terms, Y_2 is close to X by 0.90909, Y_2 is close to X^2 by 0.65035, and Y_2 is close to $X^{0.5}$ by 0.84588, therefore the linguistic meaning of Y_2 is $X =$ “*large*” and followed closely by $X^{0.5} =$ “*more or less large*”.

10.5 Fuzzy Aggregation

Consider a R -rule M -antecedent (or M -input) and single-output fuzzy system, let $\mu_{r_1}(x_1), \mu_{r_2}(x_2), \mu_{r_3}(x_3), \dots, \mu_{r_M}(x_M)$ be the M antecedent membership functions of the r th rule for $r = 1$ to R .

Given any input $[x_1, x_2, x_3, \dots, x_M]$, there are two common operators to define the output weight w_r of the r th rule of the fuzzy system.

Minimum (min) operator

$$w_r = \min[\mu_{r_1}(x_1), \mu_{r_2}(x_2), \mu_{r_3}(x_3), \dots, \mu_{r_M}(x_M)] \quad (10.5.1)$$

Algebraic product (product) operator

$$w_r = \mu_{r_1}(x_1) \times \mu_{r_2}(x_2) \times \mu_{r_3}(x_3) \times \cdots \times \mu_{r_M}(x_M) \quad (10.5.2)$$

Given the r th output weight w_r and the r th consequent output $\mu_{C_r}(z)$ for $r = 1$ to R , the defuzzified or crisp output y of a fuzzy system can be computed according to either of the following two definitions.

The weighted average operator is defined in (10.5.3) and the weighted sum operator is defined in (10.5.4).

$$y = \frac{w_1\mu_{C_1}(y) + w_2\mu_{C_2}(y) + w_3\mu_{C_3}(y) + \cdots + w_R\mu_{C_R}(y)}{w_1 + w_2 + w_3 + \cdots + w_R} \quad (10.5.3)$$

In practice, the weighted average operator in (10.5.3) is sometimes replaced by a weighted sum operator defined in (10.5.4) to reduce computation in training.

$$w_1\mu_{C_1}(y) + w_2\mu_{C_2}(y) + w_3\mu_{C_3}(y) + \cdots + w_R\mu_{C_R}(y) \quad (10.5.4)$$

However, this simplification in the weighted sum operator could lead to the loss of membership function linguistic meanings.

10.6 Defuzzification

Defuzzification refers to the conversion of a fuzzy value to a crisp value. In general, there are a number of methods for defuzzifying an aggregated output membership function $\mu_{B'}(y)$ in a universe of discourse Y .

The defuzzified output y can be (a) the smallest, the mean, and the largest value of y at which $\mu_{B'}(y)$ is maximum; or (b) the value of y at which $\mu_{B'}(y)$ is divided into equal left and right areas; or (c) the centroid of $\mu_{B'}(y)$.

Among these defuzzification methods, the centroid is the most commonly adopted method, its definitions for continuous and discrete universes Y are given below.

10.6.1 Centroid of area – Continuous universe of discourse

For an aggregated output membership function $\mu_{B'}(y)$ in a continuous

universe of discourse represented by $\{[y, \mu_{B'}(y)] | y \in Y\}$, the centroid y_C of the aggregated output membership function is defined by

$$y_C = \frac{\int_Y \mu_{B'}(y) y dy}{\int_Y \mu_{B'}(y) dy} \quad (10.6.1)$$

The corresponding area of the aggregated output membership function is defined by

$$a(B') = \int_Y \mu_{B'}(y) dy \quad (10.6.2)$$

10.6.2 Centroid of area – Discrete universe of discourse

For an aggregated output membership function, $\mu_{B'}(y)$, in a discrete universe of discourse with equally spaced values, $Y = \{y_1, y_2, y_3, \dots, y_N\}$, such that $y_1 < y_2 < y_3 < \dots < y_N$, the centroid of the aggregated output membership function is defined by

$$y_C = \frac{\sum_{n=1}^N y_n \mu_{B'}(y_n)}{\sum_{n=1}^N \mu_{B'}(y_n)} \quad (10.6.3)$$

The corresponding area of the aggregated output membership function is defined by

$$a(B') = \sum_{n=1}^N \mu_{B'}(y_n) \quad (10.6.4)$$

10.6.3 Centroid computation

With sum-product composition, the output of a centroid defuzzification system (10.6.1) can be expressed as the weight-area average of the centroids of consequent membership functions (10.6.9) [Zad7104b].

Consider a fuzzy system with R rules, by using product for implication operation and sum for aggregate operation, respectively, the aggregated output membership function $\mu_{B'}(y)$ can be expressed as

$$\mu_{B'}(y) = w_1 \mu_{B_1}(y) + w_2 \mu_{B_2}(y) + w_3 \mu_{B_3}(y) + \dots + w_R \mu_{B_R}(y) \quad (10.6.5)$$

where $\mu_{B_r}(y)$ is the consequent membership function of the r th rule.

Substituting (10.6.5) into (10.6.1), the crisp output under centroid defuzzification is

$$\begin{aligned} y_c &= \frac{\int_Y \mu_{B'}(y) y dy}{\int_Y \mu_{B'}(y) dy} \\ &= \frac{w_1 \int_Y \mu_{B_1}(y) y dy + w_2 \int_Y \mu_{B_2}(y) y dy + \cdots + w_R \int_Y \mu_{B_R}(y) y dy}{w_1 \int_Y \mu_{B_1}(y) dy + w_2 \int_Y \mu_{B_2}(y) dy + \cdots + w_R \int_Y \mu_{B_R}(y) dy} \end{aligned} \quad (10.6.6)$$

Let the area and centroid, respectively, of the consequent membership function $\mu_{B_i}(y)$ of the i th rule be expressed as

$$a_i = a(B_i) = \int_Y \mu_{B_i}(y) dy \quad (10.6.7)$$

$$y_i = \frac{\int_Y \mu_{B_i}(y) y dy}{\int_Y \mu_{B_i}(y) dy} \quad (10.6.8)$$

Multiplying each i th pair of numerator and denominator terms of (10.6.6) by its corresponding term $a_i / \int_Y \mu_{B_i}(y) dy$, and then substituting (10.6.8) into (10.6.6), we obtain

$$y_c = \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + \cdots + w_R a_R y_R}{w_1 a_1 + w_2 a_2 + w_3 a_3 + \cdots + w_R a_R} \quad (10.6.9)$$

With the above, the area a_i and the centroid y_i of each consequent membership function can be computed in advance.

Consequently, the computation of the centroid of an aggregated output membership function $\mu_{B'_i}(y)$ due to an input i is reduced to the computation of a simple weighted average, which saves time as compared to that of the direct centroid computation of the aggregated output membership function which could be of an arbitrary and varying shape depending on its inputs.

10.7 Fuzzy reasoning

In traditional propositional logic, modus ponens is important especially for engineering applications due to cause and effect characteristics.

Modus ponens

Fact: x_1 is A_1 and x_2 is A_2

Rule: If x_1 is A_1 and x_2 is A_2 then y is B

Conclusion: y is B

Generalized modus ponens

Fact: x_1 is A'_1 and x_2 is A'_2

Rule: If x_1 is A_1 and x_2 is A_2 then y is B

Conclusion: y is B'

In general, A'_i is not necessarily equal to A_i for $i = 1, 2$ and B' is not necessarily equal to B .

However, as long as there is a nonzero degree of similarity between the fact and the antecedent of a rule, the rule is fired.

The result of such a rule-firing is a consequent that has a nonzero degree of similarity to the rule's consequent.

Let A'_i , A_i for $i = 1, 2$, and B be fuzzy sets of universes of discourses X, X , and Y , respectively.

Assume that the fuzzy implication $A_1 \times A_2 \rightarrow B$ is expressed as a fuzzy relation $R(X, Y)$ on $X \times Y$, then the fuzzy set B' induced by $A'_1 \times A'_2$ and the fuzzy rule if x_1 is A_1 and x_2 is A_2 then y is B can be expressed as

$$B' = (A'_1 \times A'_2) \circ R = (A'_1 \times A'_2) \circ (A_1 \times A_2 \rightarrow B) \quad (10.7.1)$$

Using the minimum implication function for fuzzy relation and the max-

product for compositional rule of inference, (10.7.1) becomes

$$\begin{aligned}
 \mu_{B'}(y) &= \max_{x_1, x_2} \left\{ \min[\mu_{A'_1}(x_1), \mu_{A'_2}(x_2)] \right. \\
 &\quad \left. \times \min[\mu_{A_1}(x_1), \mu_{A_2}(x_2), \mu_B(y)] \right\} \\
 &= \min \left\{ \left[\max_{x_1} \left(\min[\mu_{A'_1}(x_1), \mu_{A_1}(x_1)] \right) \right. \right. \\
 &\quad \left. \left. \times \max_{x_2} \left(\min[\mu_{A'_2}(x_2), \mu_{A_2}(x_2)] \right) \right], \mu_B(y) \right\} \\
 &= \min\{[w_1 \times w_2], \mu_B(y)\}
 \end{aligned}
 \tag{10.7.2}$$

In (10.7.2), algebraic product of $w_1 \times w_2$ (instead of minimum of w_1 and w_2) is used as the T -norm operator. In general, T -norm operators include both algebraic product and minimum.

Example 10.4 (2-rule 2-input 1-output fuzzy reasoning system)

Consider the case of two-rule two-input one-output fuzzy reasoning system.

Fact: x_1 is A'_1 and x_2 is A'_2

Rule 1: If x_1 is A_{11} and x_2 is A_{12} then y is B_1

Rule 2: If x_1 is A_{21} and x_2 is A_{22} then y is B_2

Conclusion: y is B'

Given

$$A_{11} = \text{TriMF}(x_1, [10, 40, 70])$$

$$A_{21} = \text{TriMF}(x_1, [30, 60, 90])$$

$$A'_1 = \text{TriMF}(x_1, [20, 30, 40])$$

$$A_{12} = \text{TriMF}(x_2, [-20, 10, 40])$$

$$A_{22} = \text{TriMF}(x_2, [-40, -10, 20])$$

$$A'_2 = \text{TriMF}(x_2, [-25, -15, -5])$$

$$B_1 = \text{TrapMF}(y, [-6, -2, 2])$$

$$B_2 = \text{TrapMF}(y, [0, 4, 8])$$

Using the minimum implication function for fuzzy relation and the max-product compositional rule of inference, show the computational aspects of the generalized modus ponens based fuzzy reasoning by

- graphical plot to obtain the output fuzzy set B' ; and
- derivation to arrive at the fuzzy output $\mu_{B'}(y)$ and its crisp output y_o .

Solution

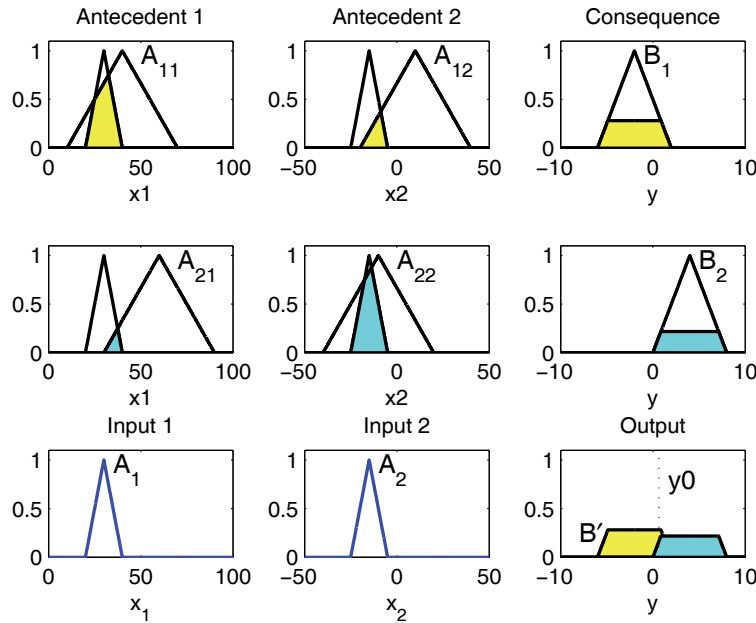


Figure E10.4.1 2-rule 2-input 1-output fuzzy reasoning (Algebraic product for weights, $w_1 = 0.27998$, $w_2 = 0.21832$; centroid defuzzified output, $y_o = 0.68096$).

- The output fuzzy set B' can be determined graphically as shown in

Figure E10.4.1.

(b) Since $B'_r = (A'_1 \times A'_2) \circ R_r$, the fuzzy output $\mu_{B'_r}(y)$ for each of the two rules $r = 1, 2$ can be obtained as

$$\begin{aligned}
 & \mu_{B'_r}(y) \\
 &= \max_{x_1, x_2} \left\{ \min[\mu_{A'_1}(x_1), \mu_{A'_2}(x_2)] \times \min[\mu_{A_{r1}}(x_1), \mu_{A_{r2}}(x_2), \mu_{B_r}(y)] \right\} \\
 &= \min \left\{ \left[\max_{x_1} \left(\min[\mu_{A'_1}(x_1), \mu_{A_{r1}}(x_1)] \right) \right. \right. \\
 & \quad \left. \left. \times \max_{x_2} \left(\min[\mu_{A'_2}(x_2), \mu_{A_{r2}}(x_2)] \right) \right], \mu_{B_r}(y) \right\} \\
 &= \min\{[w_{r1} \times w_{r2}], \mu_{B_r}(y)\} = \min\{w_r, \mu_{B_r}(y)\} \quad (\text{E10.4.1})
 \end{aligned}$$

Since there are two rules, $R_1 = A_{11} \times A_{12} \rightarrow B_1$ and $R_2 = A_{21} \times A_{22} \rightarrow B_2$, we have

$$\begin{aligned}
 B' &= (A'_1 \times A'_2) \circ (R_1 \cup R_2) \\
 &= [(A'_1 \times A'_2) \circ R_1] \cup [(A'_1 \times A'_2) \circ R_2] = B'_1 \cup B'_2 \quad (\text{E10.4.2})
 \end{aligned}$$

Hence, the aggregated fuzzy output is

$$\mu_{B'}(y) = \mu_{B'_1}(y) \cup \mu_{B'_2}(y) \quad (\text{E10.4.3})$$

Using the given membership functions, the weights for the two rules two inputs can be computed as

$$w_{11} = 0.75, \quad w_{12} = 0.3733$$

$$w_{21} = 0.25, \quad w_{22} = 0.8733$$

Therefore, taking the algebraic product operator for the firing strength, the weights for the two rules are

$$\text{Rule 1: } w_1 = w_{11}w_{12} = 0.27998$$

$$\text{Rule 2: } w_2 = w_{21}w_{22} = 0.21832$$

Substituting the above firing strengths w_1 and w_2 into the consequent triangular membership functions $\mu_{B_1}(y)$ and $\mu_{B_2}(y)$ in (E10.4.1), the

inferred areas $\mu_{B'_1}(y)$ and $\mu_{B'_2}(y)$ are obtained.

Using the maximum operator for aggregation, the consequence at the output $\mu_{B'}(y)$ is the maximum of $\mu_{B'_1}(y)$ and $\mu_{B'_2}(y)$ as shown in (E10.4.3).

By taking the centroid (discrete universe) of aggregated output membership function as the defuzzification scheme defined in (10.6.3), the centroid of the defuzzified output y_o is computed as 0.68096.

By taking the weight-area average of centroids of consequent membership functions as the defuzziation scheme as defined in (10.6.9), the centroid of the defuzzified output y_o is computed as 0.62884.

Their difference in the centroid computation is negligible with respect to the entire range of y from -6 to 8 for the aggregated output membership function.

Chapter 11

Fuzzy Systems

11.1 Introduction

The block diagram of a fuzzy system (FS) with one crisp (or fuzzy) input and one crisp output is shown in Figure 11.1.1.

A fuzzy system consists of three core operations, namely applying fuzzy if-then rules obtained from training (or expert knowledge) in which membership functions are used to represent fuzzy rules; fuzzy aggregation to induce a fuzzy conclusion from the consequences of all the rules; and defuzzification of the fuzzy conclusion to a crisp output.

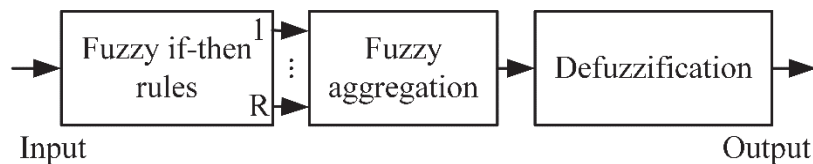


Figure 11.1.1 Block diagram of a fuzzy system.

A fuzzy system can take either crisp inputs or fuzzy inputs, but the outputs produced are often fuzzy.

For practical applications, it is often required to have crisp outputs in which defuzzification is required to extract a crisp value that appropriately represents a fuzzy output.

In this chapter, three fuzzy systems are described and each of them is illustrated by two design examples.

The differences between these three fuzzy systems lie in the

consequents of their fuzzy rules, and consequently their aggregation and defuzzification procedures are different.

The Mamdani fuzzy system requires a defuzzification whereas the TSK and Tsukamoto fuzzy systems do not require a defuzzification.

Fuzzy systems have applications in a variety of fields. An earlier reference for fuzzy systems is given by [Jan9700].

11.2 Mamdani Fuzzy System

The Mamdani fuzzy system [Mam7501] is the earliest fuzzy system among the three fuzzy systems described in this chapter.

Different T -norm operators (such as minimum, algebraic product) and S -norm operators (such as maximum, algebraic sum) can be used for fuzzy reasoning.

Typical examples are min-max and product-max in which the latter has been used due to its amenability to mathematical analysis.

For illustration, an example is shown in Figure 11.2.1 to explain how a two-rule two-input one-output Mamdani fuzzy system derives an overall output y when subjected to two fuzzy inputs x_1 and x_2 .

Rule 1: If A_{11} is low and A_{12} is high then B_1 is low

Rule 2: If A_{21} is high and A_{22} is low then B_2 is high

The inferred output of each rule B_r (for $r = 1$ to 2) is a membership function $\mu_{B_r}(y)$ cropped by the rule's firing strength w_r obtained using (a) the minimum operator (as $w_r = \min(w_{r1}, w_{r2})$) or (b) the algebraic product operator (as $w_r = w_{r1}w_{r2}$), where w_{ri} (for $i = 1, 2$) denotes the peak of the intersection of the input j with the antecedent j of the r th rule.

The Mamdani fuzzy system aggregates all its rule's membership function outputs by taking the maximum operator before applying defuzzification.

The Mamdani fuzzy system can take either crisp inputs or fuzzy inputs (for example, fuzzy inputs A_1 and A_2 are used in Figure 11.2.1).

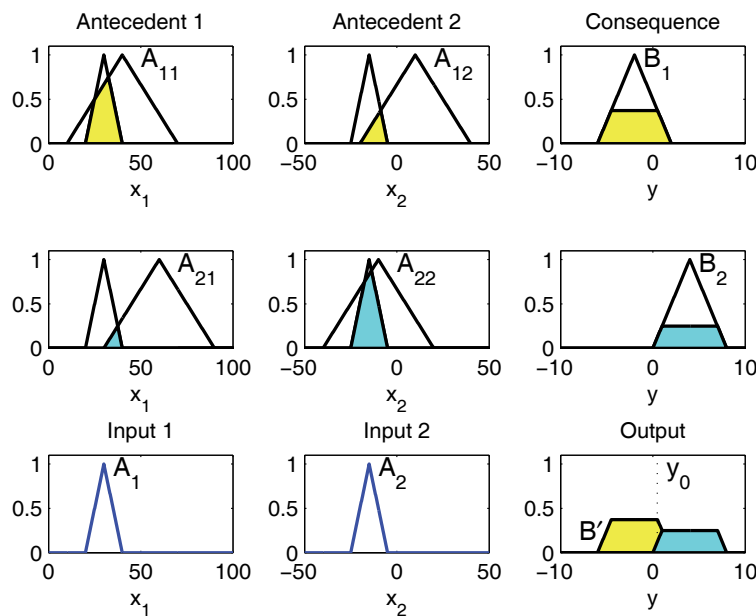


Figure 11.2.1 Mamdani fuzzy system (2-rule 2-input 1-output; minimum operator for weights $w_1 = 0.3733$, $w_2 = 0.2500$; centroid defuzzified output $y_o = 0.4815$).

Design Example 11.1 (4-rule 1-input 1-output Mamdani fuzzy system)
Matlab

For a four-rule one-input one-output Mamdani fuzzy system, let X be the fuzzy input and Y be the fuzzy output of the system. The four rules of the fuzzy system are defined by

Rule 1: If X is very young then Y implies fair life experience

Rule 2: If X is young then Y implies good life experience

Rule 3: If X is old then Y implies very good life experience

Rule 4: If X is very old then Y implies excellent life experience

The antecedent membership functions are defined as

$$\mu_{\text{very young}}(x) = \text{GBellMF}(x, [10, 3, 0])$$

$$\mu_{\text{young}}(x) = \text{GBellMF}(x, [20, 5, 30])$$

$$\mu_{\text{old}}(x) = \text{GBellMF}(x, [20, 5, 70])$$

$$\mu_{\text{very old}}(x) = \text{GBellMF}(x, [10, 3, 100])$$

where $x \in [0, 100]$.

The consequent membership functions are defined as

$$\mu_{\text{fair}}(y) = \text{GBellMF}(y, [1, 8, 0])$$

$$\mu_{\text{good}}(y) = \text{GBellMF}(y, [1, 6, 2])$$

$$\mu_{\text{very good}}(y) = \text{GBellMF}(y, [2, 6, 5])$$

$$\mu_{\text{excellent}}(y) = \text{GBellMF}(y, [2, 8, 9])$$

where $y \in [0, 10]$.

Use the max-min composition and the centroid defuzzification, determine the overall input to output curve with a crisp input x .

Solution

Figure E11.1.1(a)-(b) plots the membership function of input X and output Y , where the input and output universes are $[0, 100]$ and $[0, 10]$, respectively.

With max-min composition and the centroid defuzzification, the overall input-output curve can be obtained as shown in Figure E11.1.1(c).

For examples,

- a. At $x = 0$, only the Rule 1 wins with a membership value of 1, the defuzzified output is equal to the centroid of the output membership function of the Rule 1 which is 0.6.

$$y_c = \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_1 a_1 y_1}{w_1 a_1} = y_1 = 0.6$$

- b. At $x = 30$, only the Rule 2 wins with a membership value of 1, the defuzzified output is equal to the centroid of the output membership function of the Rule 2 which is 2.0.

$$y_c = \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_2 a_2 y_2}{w_2 a_2} = y_2 = 2.0$$

- c. At $x = 70$, only the Rule 3 wins with a membership value of 1, the defuzzified output is equal to the centroid of the output membership function of the Rule 3 which is 5.0.

$$y_c = \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_3 a_3 y_3}{w_3 a_3} = y_3 = 5.0$$

- d. At $x = 100$, only the Rule 4 wins with a membership value of 1, the defuzzified output is equal to the centroid of the output membership function of the Rule 4 which is 8.4.

$$y_c = \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_4 a_4 y_4}{w_4 a_4} = y_4 = 8.4$$

e. At any x where multiple input membership functions overlap, multiple rules win each with a membership value of less than 1, the output is then given by the centroid of the multiple overlapping output membership functions each weighted by its input membership value. In this example, one rule can win as shown in the cases a to d above or two rules can win in the case f below.

f. At $x = 10$, the Rule 1 wins with a membership value of $w_1 = 0.5$ in which the area and the centroid of its output membership function are, respectively, $a_1 = 1$ and $y_1 = 0.6$; also, the Rule 2 wins with a membership of $w_2 = 0.5$ in which the area and centroid of its output membership function are, respectively, $a_2 = 2$ and $y_2 = 2$. According to the (10.6.9), the overall defuzzified centroid output is

$$\begin{aligned} y_c &= \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_1 a_1 y_1 + w_2 a_2 y_2}{w_1 a_1 + w_2 a_2} \\ &= \frac{0.5 \times 1 \times 0.6 + 0.5 \times 2 \times 2}{0.5 \times 1 + 0.5 \times 2} = 1.5333 \end{aligned}$$

Note that the output variable never reaches the minimum value 0 and the maximum value 10 of the output universe.

Instead, the reachable minimum and maximum of the output variable are determined by the centroids of the leftmost and rightmost consequent membership functions, respectively (as can be seen from the equation used to compute weight average).

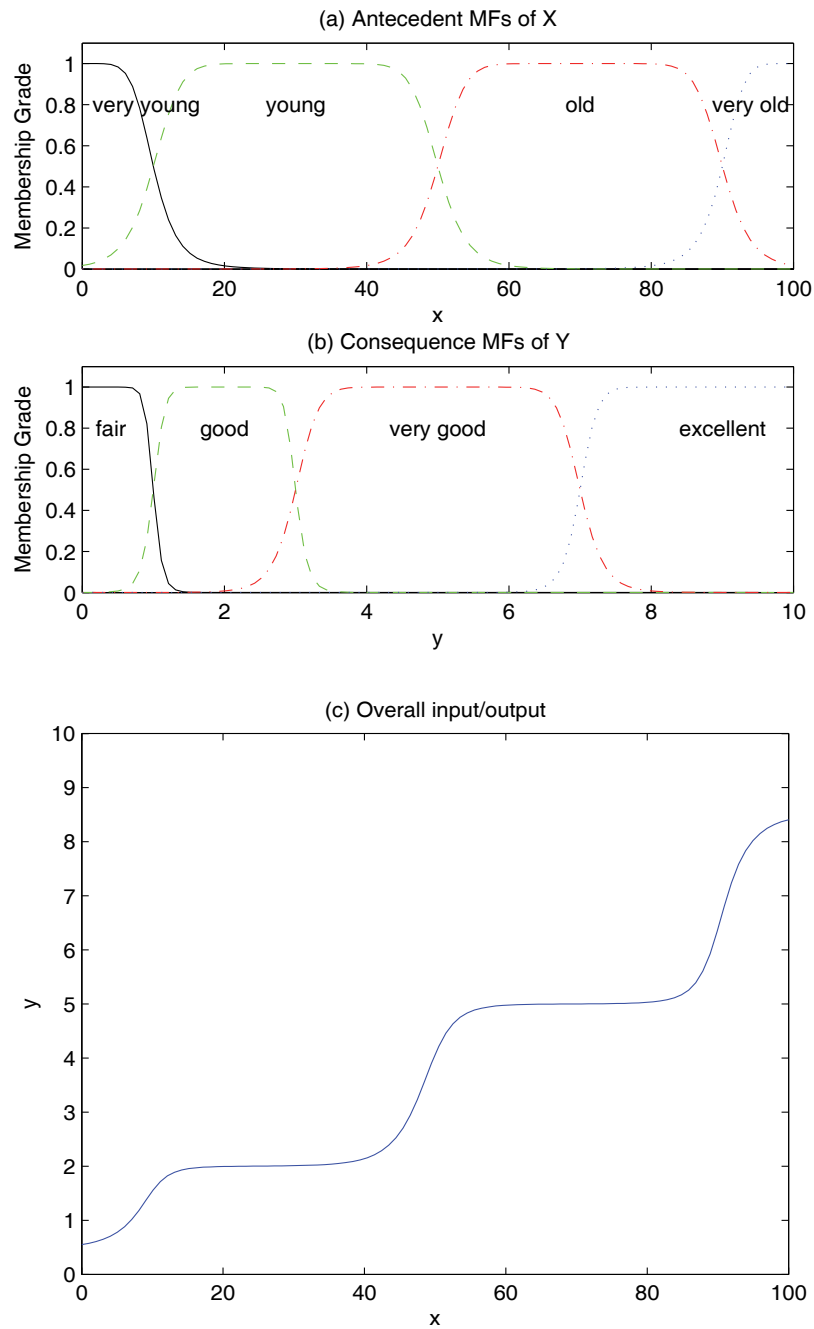


Figure E11.1.1 Mamdani fuzzy system (4-rule 1-input 1-output).

Design Example 11.2 (4-rule 2-input 1-output Mamdani fuzzy system)

Matlab

For a four-rule two-input one-output Mamdani fuzzy system, let X_1 and X_2 be the inputs and Y be the output of the system.

The four rules of the fuzzy system are defined by

Rule 1: If X_1 is cold and X_2 is cold then Y is very cold

Rule 2: If X_1 is cold and X_2 is warm then Y is cold

Rule 3: If X_1 is warm and X_2 is cold then Y is warm

Rule 4: If X_1 is warm and X_2 is warm then Y is very warm

The antecedent membership functions of X_1 are defined as

$$\mu_{\text{cold}}(x_1) = \text{GBellMF}(x_1, [20, 8, -5])$$

$$\mu_{\text{warm}}(x_1) = \text{GBellMF}(x_1, [15, 8, 30])$$

where $x_1 \in [-20, 30]$.

The antecedent membership functions of X_2 are defined as

$$\mu_{\text{cold}}(x_2) = \text{GBellMF}(x_2, [20, 10, -15])$$

$$\mu_{\text{warm}}(x_2) = \text{GBellMF}(x_2, [25, 10, 30])$$

where $x_2 \in [-20, 30]$.

The consequent membership functions of Y are defined as

$$\mu_{\text{very cold}}(y) = \text{GBellMF}(y, [10, 8, -20])$$

$$\mu_{\text{cold}}(y) = \text{GBellMF}(y, [10, 8, 0])$$

$$\mu_{\text{warm}}(y) = \text{GBellMF}(y, [7, 8, 17])$$

$$\mu_{\text{very warm}}(y) = \text{GBellMF}(y, [6, 8, 30])$$

where $y \in [-20, 30]$.

Using the max-min composition and the centroid defuzzification, determine the overall input to output relationship with two crisp inputs x_1 and x_2 .

Solution

Figure E11.2.1 (a)-(c) plots the membership function of inputs X_1 and X_2 , and output Y , where the input and output universes are all equal to $[-20, 30]$.

With the max-min composition and the centroid defuzzification, the overall input-output curve can be obtained as shown in Figure E11.2.1 (d).

For examples,

a. At $x_1 = -15$ and $x_2 = -15$, only the Rule 1 wins with a membership value of 1, the defuzzified output is equal to the centroid of the output membership function of the Rule 1 which is -15 .

$$y_C = \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_1 a_1 y_1}{w_1 a_1} = y_1 = -15$$

b. At $x_1 = 10$ and $x_2 = 10$, only the Rule 2 wins with a membership value of 1, the defuzzified output is equal to the centroid of the output membership function of the Rule 2 which is 0.0 .

$$y_C = \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_2 a_2 y_2}{w_2 a_2} = y_2 = 0$$

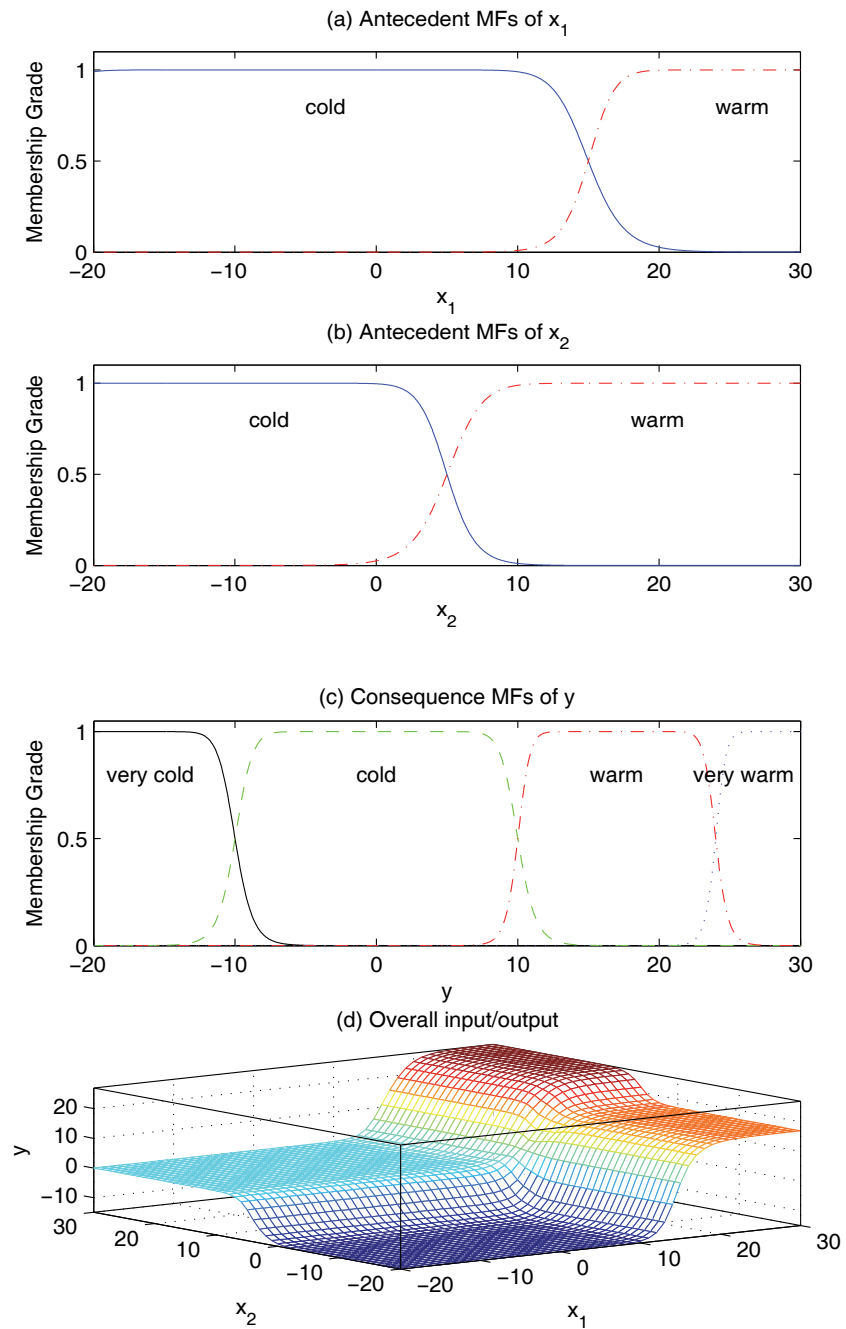


Figure E11.2.1 Mamdani fuzzy system (4 rule 2-input 1-output).

c. At $x_1 = 25$ and $x_2 = -10$, only the Rule 3 wins with a membership value of 1, the defuzzified output is equal to the centroid of the output membership function of the Rule 3 which is 17.

$$y_c = \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_3 a_3 y_3}{w_3 a_3} = y_3 = 17$$

d. At $x_1 = 25$ and $x_2 = 15$, only the Rule 4 wins with a membership value of 1, the defuzzified output is equal to the centroid of the output membership function of the Rule 4 which is 27.

$$y_c = \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_4 a_4 y_4}{w_4 a_4} = y_4 = 27$$

e. At any x_1 and x_2 where multiple input membership functions overlap, multiple rules win each with a membership value of less than 1, the output is then given by the centroid of the multiple overlapping output membership functions each weighted by its input membership value. In this example, one, two, and four rules can win.

f. At $x_1 = 15$ and $x_2 = -10$, the Rule 1 wins with a membership value of $w_1 = \min(0.5, 1.0) = 0.5$ in which the area and centroid of its output membership function are respectively, $a_1 = 10$ and $y_1 = -15$; also, the Rule 3 wins with a membership value of $w_3 = \min(0.5, 1.0) = 0.5$ in which the area and centroid of its output membership function are respectively, $a_3 = 14$ and $y_3 = 17$. According to (10.6.9), the overall defuzzified output can be expressed as

$$\begin{aligned} y_c &= \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} = \frac{w_1 a_1 y_1 + w_3 a_3 y_3}{w_1 a_1 + w_3 a_3} \\ &= \frac{0.5 \times 10 \times (-15) + 0.5 \times 14 \times 17}{0.5 \times 10 + 0.5 \times 14} = 3.6667 \end{aligned}$$

g. At $x_1 = 15$ and $x_2 = 2.5$, the Rule 1 wins with a membership value of $w_1 = \min(0.5, 0.85) = 0.5$ in which the area and centroid of

its output membership function are respectively, $a_1 = 10$ and $y_1 = -15$; the Rule 2 wins with a membership value of $\min(0.5, 0.15) = 0.15$ in which the area and centroid of its output membership function are respectively, $a_2 = 20$ and $y_2 = 0$; the Rule 3 wins with a membership value of $w_3 = \min(0.5, 0.85) = 0.5$ in which the centroid of its output membership function are respectively, $a_3 = 14$ and $y_3 = 17$; the Rule 4 wins with a membership value of $w_4 = \min(0.5, 0.15) = 0.15$ in which the area and centroid of its output membership function are respectively, $a_4 = 6$ and $y_4 = 27$. According to the (10.6.9), the overall defuzzified output is

$$\begin{aligned}
 y_c &= \frac{w_1 a_1 y_1 + w_2 a_2 y_2 + w_3 a_3 y_3 + w_4 a_4 y_4}{w_1 a_1 + w_2 a_2 + w_3 a_3 + w_4 a_4} \\
 &= \frac{0.5 \times 10 \times (-15) + 0.15 \times 20 \times 0 + 0.5 \times 14 \times 17 + 0.15 \times 6 \times 27}{0.5 \times 10 + 0.15 \times 20 + 0.5 \times 14 + 0.15 \times 6} \\
 &= 4.2956
 \end{aligned}$$

Note that the output variable never reaches the minimum (-20) and the maximum (30) and of the output universe.

Instead, the reachable minimum and maximum of the output variable are equal to the centroids of the leftmost and rightmost consequent membership functions, respectively.

11.3 Tsukamoto Fuzzy System

In the Mamdani fuzzy system, the output requires defuzzification to arrive at a crisp output.

In the Tsukamoto fuzzy system [Tsu7900] , no defuzzification is required in which the consequent of each fuzzy if-then rule is represented by a fuzzy set with a piecewise linear monotonic membership function as shown in Figure 11.3.1.

For formulating derivatives required in gradient learning, smooth output

membership functions such as sigmoidal membership functions defined in (9.5.5) could be used to replace monotonic membership functions.

In the Tsukamoto fuzzy system, the output membership function of each rule works like an input to output function with an input equal to the rule's firing strength w_i and the output equal to the inferred output y_i .

Figure 11.3.1 gives an example to show the reasoning procedure for a two-rule two-input one-output Tsukamoto fuzzy system with two crisp inputs x_1 and x_2 .

Rule 1: If X_1 is high and X_2 is low then Y is B_1 .

Rule 2: If X_1 is low and X_2 is high then Y is B_2 .

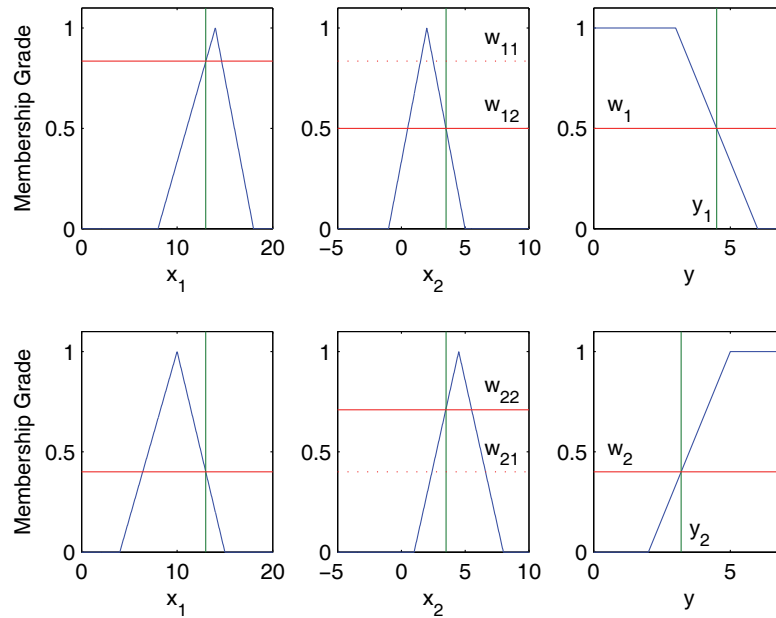


Figure 11.3.1 Tsukamoto fuzzy system (2-rule 2-input 1-output, minimum operator for weights).

The inferred output of each rule y_i is a crisp value induced by the rule's firing strength (w_i for $i = 1$ to $R=2$) obtained using the minimum operator or the algebraic product operator.

The Tsukamoto fuzzy system aggregates each rule's output simply by the method of weighted average as

$$y = \frac{\sum_{i=1}^R w_i y_i}{\sum_{i=1}^R w_i} \quad (11.3.1)$$

where y_i is the output of the i th rule induced by the firing strength w_i and membership function for B_i for $i=1$ to R .

Similar to the TSK fuzzy system to be described next, the reasoning procedure of the Tsukamoto fuzzy system does not follow strictly the compositional rule of inference; the output is always crisp even when the inputs are fuzzy.

Design Example 11.3 (4-rule 1-input 1-output Tsukamoto fuzzy system)

For a four-rule one-input one-output Tsukamoto fuzzy system, let X be the input and Y be the output of the system. Define the four-rule of the fuzzy system as

Rule 1: If X is very light then Y is B_1

Rule 2: If X is light then Y is B_2

Rule 3: If X is heavy then Y is B_3

Rule 4: If X is very heavy then Y is B_4

The antecedent membership functions are defined as

$$\mu_{\text{very light}}(x) = \text{GBellMF}(x, [4, 10, -10])$$

$$\mu_{\text{light}}(x) = \text{GBellMF}(x, [3, 10, -3])$$

$$\mu_{\text{heavy}}(x) = \text{GBellMF}(x, [3, 10, 3])$$

$$\mu_{\text{very heavy}}(x) = \text{GBellMF}(x, [4, 10, 10])$$

where $x \in [-10, 10]$.

The consequent membership functions are defined as

$$\mu_{B_1}(y) = \min \left[1, \max \left(\frac{y}{2}, 0 \right) \right]$$

$$\mu_{B_2}(y) = 1 - \min \left[1, \max \left(\frac{y-4}{2}, 0 \right) \right]$$

$$\mu_{B_3}(y) = \min \left[1, \max \left(\frac{y-7}{3}, 0 \right) \right]$$

$$\mu_{B_4}(y) = 1 - \min \left[1, \max \left(\frac{y}{3} - 2, 0 \right) \right]$$

where $y \in [0, 10]$.

Given the centroids of the four consequent membership functions as 0.6, 2.0, 5.0, and 8.4 respectively, use the weighted average as the defuzzification scheme, determine the overall input-output curve with a crisp input x .

Solution

Figure E11.3.1 (a)-(b) plot the membership functions of input X and output Y , where the input and output universes are $[-10, 10]$ and $[0, 10]$, respectively.

With the weighted average for defuzzification, the rule output and the

overall input-output curve can be obtained as plotted in Figure E11.3.1 (c)-(d).

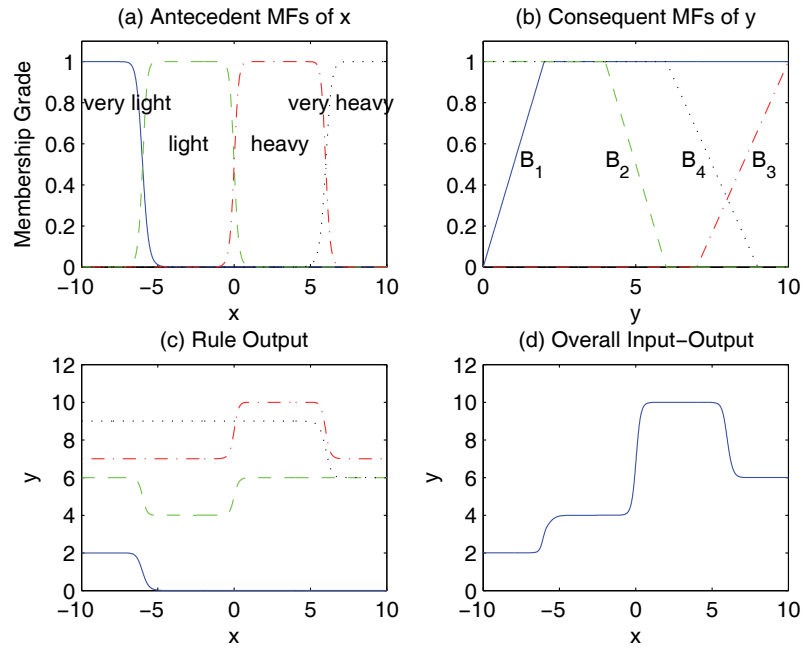


Figure E11.3.1 Tsukamoto fuzzy system (4-rule 1-input 1-output).

For examples,

At $x = -8$:

Rule	Antecedent	Consequent
1	$\mu_{very\ light}(-8) = 1 \Rightarrow w_1 = 1$	$\mu_{B1}(y_1) = w_1 \Rightarrow y_1 = 2$
2	$\mu_{light}(-8) = 0 \Rightarrow w_2 = 0$	$\mu_{B2}(y_2) = w_2 \Rightarrow y_2 = 6$
3	$\mu_{heavy}(-8) = 0 \Rightarrow w_3 = 0$	$\mu_{B3}(y_3) = w_3 \Rightarrow y_3 = 7$
4	$\mu_{very\ heavy}(-8) = 0 \Rightarrow w_4 = 0$	$\mu_{B4}(y_4) = w_4 \Rightarrow y_4 = 9$

then

$$y = \frac{w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4}{w_1 + w_2 + w_3 + w_4} = \frac{(1)(2) + 0 + 0 + 0}{1 + 0 + 0 + 0} = 2$$

At $x = 0.0$:

Rule	Antecedent	Consequent
1	$\mu_{very\ light}(0) = 0 \Rightarrow w_1 = 0.0$	$\mu_{B1}(y_1) = w_1 \Rightarrow y_1 = 0.0$
2	$\mu_{light}(0) = 0.5 \Rightarrow w_2 = 0.5$	$\mu_{B2}(y_2) = w_2 \Rightarrow y_2 = 5.0$
3	$\mu_{heavy}(0) = 0.5 \Rightarrow w_3 = 0.5$	$\mu_{B3}(y_3) = w_3 \Rightarrow y_3 = 8.5$
4	$\mu_{very\ heavy}(0) = 0 \Rightarrow w_4 = 0.0$	$\mu_{B4}(y_4) = w_4 \Rightarrow y_4 = 9.0$

Then

$$y = \frac{w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4}{w_1 + w_2 + w_3 + w_4} = \frac{0 + (0.5)(5) + (0.5)(8.5) + 0}{0 + 0.5 + 0.5 + 0} = 6.75$$

At $x = 6.0$:

Rule	Antecedent	Consequent
1	$\mu_{very\ light}(6) = 0.0 \Rightarrow w_1 = 0.0$	$\mu_{B1}(y_1) = w_1 \Rightarrow y_1 = 0.0$
2	$\mu_{light}(6) = 0.0 \Rightarrow w_2 = 0.0$	$\mu_{B2}(y_2) = w_2 \Rightarrow y_2 = 6.0$
3	$\mu_{heavy}(6) = 0.75 \Rightarrow w_3 = 0.75$	$\mu_{B3}(y_3) = w_3 \Rightarrow y_3 = 9.0$
4	$\mu_{very\ heavy}(6) = 0.25 \Rightarrow w_4 = 0.25$	$\mu_{B4}(y_4) = w_4 \Rightarrow y_4 = 8.0$

then

$$y = \frac{w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4}{w_1 + w_2 + w_3 + w_4} = \frac{0 + 0 + (0.75)(9) + (0.25)(8)}{0 + 0 + 0.75 + 0.25} = 8.75$$

Design Example 11.4 (4-rule 2-input 1-output Tsukamoto fuzzy system)

For a four-rule two-input one-output Tsukamoto fuzzy system, let X_1 and X_2 be the input and Y be the output of the system.

Define the four-rule of the system as

If X_1 is dry and X_2 is slow then Y is B_1

If X_1 is dry and X_2 is fast then Y is B_2

If X_1 is wet and X_2 is slow then Y is B_3

If X_1 is wet and X_2 is fast then Y is B_4

The antecedent membership functions of X_1 are defined as

$$\mu_{\text{dry}}(x_1) = \text{GBellMF}(x_1, [10, 8, -12])$$

$$\mu_{\text{wet}}(x_1) = \text{GBellMF}(x_1, [10, 8, 8])$$

where $x_1 \in [-10, 10]$.

The antecedent membership functions of X_2 are defined as

$$\mu_{\text{slow}}(x_2) = \text{GBellMF}(x_2, [4, 10, 2])$$

$$\mu_{\text{fast}}(x_2) = \text{GBellMF}(x_2, [4, 10, 10])$$

where $x_2 \in [0, 10]$.

The consequent membership functions are defined by

$$\mu_{B_1}(y) = \min \left[1, \max \left(\frac{y}{2}, 0 \right) \right]$$

$$\mu_{B_2}(y) = 1 - \min \left[1, \max \left(\frac{y-4}{2}, 0 \right) \right]$$

$$\mu_{B_3}(y) = \min \left[1, \max \left(\frac{y}{4} - 1, 0 \right) \right]$$

$$\mu_{B_4}(y) = 1 - \min \left[1, \max \left(\frac{y-7}{3}, 0 \right) \right]$$

where $y \in [0, 20]$.

Using the minimum operator for weights, and the weighted average as the defuzzification scheme, determine the overall input-output curve with two crisp inputs x_1 and x_2 and the corresponding output y .

Solution

Figure E11.4.1 (a)-(c) plots the membership function of inputs X_1 and X_2 , and output Y , with universes respectively equal to $[-10, 10]$, $[0, 10]$, and $[0, 10]$.

With the minimum operator for weights, and the weighted average for defuzzification, the overall input-output curve can be obtained as shown in Figure E11.4.1 (d).

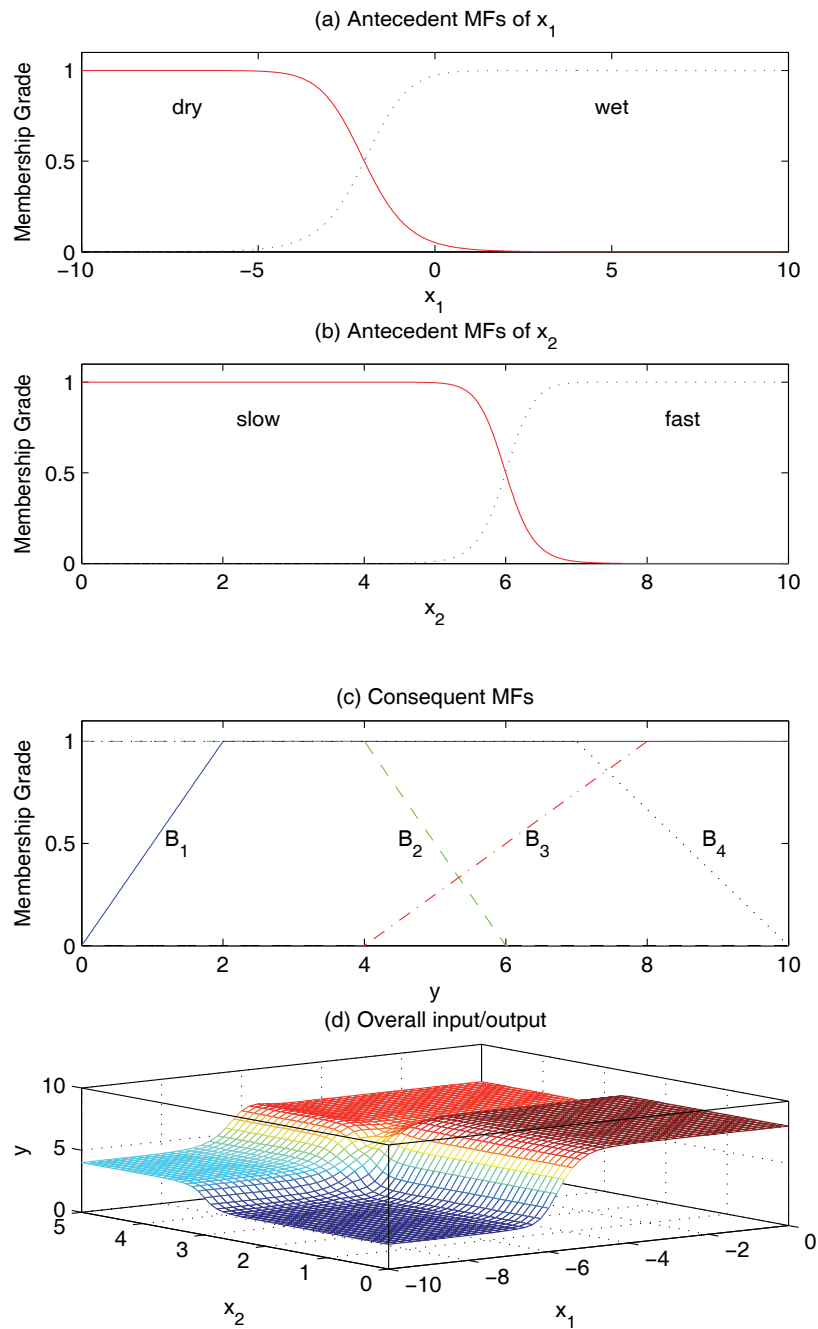


Figure E11.4.1 Tsukamoto fuzzy system (4-rule 2-input 1-output).

For examples,

At $x_1 = -8$ and $x_2 = 2$, using the minimum operator for weights, we obtain

Rule	Antecedent	Consequent
1	$\mu_{\text{dry}}(-8)=1.0, \mu_{\text{slow}}(2)=1.0$ $\Rightarrow w_1=\min[\mu_{\text{dry}}(-8), \mu_{\text{slow}}(2)]=1.0$	$\mu_{B1}(y_1) = w_1 \Rightarrow y_1 = 2$
2	$\mu_{\text{dry}}(-8)=1.0, \mu_{\text{fast}}(2)=0.0$ $\Rightarrow w_2=\min[\mu_{\text{dry}}(-8), \mu_{\text{fast}}(2)]=0.0$	$\mu_{B2}(y_2) = w_2 \Rightarrow y_2 = 6$
3	$\mu_{\text{wet}}(-8)=0.0, \mu_{\text{slow}}(2)=1.0$ $\Rightarrow w_3=\min[\mu_{\text{wet}}(-8), \mu_{\text{slow}}(2)]=0.0$	$\mu_{B3}(y_3) = w_3 \Rightarrow y_3 = 4$
4	$\mu_{\text{wet}}(-8)=0.0, \mu_{\text{fast}}(2)=0.0$ $\Rightarrow w_4=\min[\mu_{\text{wet}}(-8), \mu_{\text{fast}}(2)]=0.0$	$\mu_{B4}(y_4) = w_4 \Rightarrow y_4 = 10$

then

$$y = \frac{w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4}{w_1 + w_2 + w_3 + w_4} = \frac{(1)(2) + 0(6) + 0(4) + 0(10)}{1 + 0 + 0 + 0} = 2$$

At $x_1 = -2$ and $x_2 = 6$, using the minimum operator for weights, we obtain

Rule	Antecedent	Consequent
1	$\mu_{\text{dry}}(-2)=0.5, \mu_{\text{slow}}(6)=0.5$ $\Rightarrow w_1=\min[\mu_{\text{dry}}(-2), \mu_{\text{slow}}(6)]=0.5$	$\mu_{B1}(y_1) = w_1 \Rightarrow y_1 = 1$
2	$\mu_{\text{dry}}(-2)=0.5, \mu_{\text{fast}}(6)=0.5$ $\Rightarrow w_2=\min[\mu_{\text{dry}}(-2), \mu_{\text{fast}}(6)]=0.5$	$\mu_{B2}(y_2) = w_2 \Rightarrow y_2 = 5$
3	$\mu_{\text{wet}}(-2)=0.5, \mu_{\text{slow}}(6)=0.5$ $\Rightarrow w_3=\min[\mu_{\text{wet}}(-2), \mu_{\text{slow}}(6)]=0.5$	$\mu_{B3}(y_3) = w_3 \Rightarrow y_3 = 6$

4	$\mu_{\text{wet}}(-2)=0.5, \mu_{\text{fast}}(6)=0.5$ $\Rightarrow w_4=\min[\mu_{\text{wet}}(-2), \mu_{\text{fast}}(6)]=0.5$	$\mu_{B4}(y_4) = w_4 \Rightarrow y_4 = 8.5$
---	--	---

then

$$y = \frac{w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4}{w_1 + w_2 + w_3 + w_4}$$

$$= \frac{(0.5)(1) + (0.5)(5) + (0.5)(6) + (0.5)(8.5)}{0.5 + 0.5 + 0.5 + 0.5} = 5.125$$

At $x_1 = -2$ and $x_2 = 8$, using the minimum operator for weights, we obtain

Rule	Antecedent	Consequent
1	$\mu_{\text{dry}}(-2)=0.5, \mu_{\text{slow}}(8)=0.0$ $\Rightarrow w_1=\min[\mu_{\text{dry}}(-2), \mu_{\text{slow}}(8)]=0.0$	$\mu_{B1}(y_1) = w_1 \Rightarrow y_1 = 0$
2	$\mu_{\text{dry}}(-2)=0.5, \mu_{\text{fast}}(8)=1.0$ $\Rightarrow w_2=\min[\mu_{\text{dry}}(-2), \mu_{\text{fast}}(8)]=0.5$	$\mu_{B2}(y_2) = w_2 \Rightarrow y_2 = 5$
3	$\mu_{\text{wet}}(-2)=0.5, \mu_{\text{slow}}(8)=0.0$ $\Rightarrow w_3=\min[\mu_{\text{wet}}(-2), \mu_{\text{slow}}(8)]=0.0$	$\mu_{B3}(y_3) = w_3 \Rightarrow y_3 = 4$
4	$\mu_{\text{wet}}(-2)=0.5, \mu_{\text{fast}}(8)=1.0$ $\Rightarrow w_4=\min[\mu_{\text{wet}}(-2), \mu_{\text{fast}}(8)]=0.5$	$\mu_{B4}(y_4) = w_4 \Rightarrow y_4 = 8.5$

then

$$y = \frac{w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4}{w_1 + w_2 + w_3 + w_4}$$

$$= \frac{(0.0)(0) + (0.5)(5) + (0.0)(4) + (0.5)(8.5)}{0.0 + 0.5 + 0.0 + 0.5} = 6.75$$

11.4 TSK Fuzzy System

In the Mamdani fuzzy system, output defuzzification is required. In the Takagi-Sugeno-Kang (TSK) fuzzy system [Tak8501], [Sug8810], the consequent of fuzzy rules are directly generated from given input-output data without requiring output defuzzification.

A fuzzy rule of a two-input (x_1, x_2) one-output (y) TSK fuzzy system has the following equivalent forms as

$$\text{If } X_1 \text{ is } A_1 \text{ and } X_2 \text{ is } A_2 \text{ then } Y = f(X_1, X_2) \quad (11.4.1a)$$

$$\text{If } x_1 \text{ is } \mu_{A_1}(x_1) \text{ and } x_2 \text{ is } \mu_{A_2}(x_2) \text{ then } y = f(x_1, x_2) \quad (11.4.1b)$$

In (11.4.1a-b), $x_1 \in X_1$, $x_2 \in X_2$; $y \in Y$; A_1 and A_2 are the antecedent fuzzy sets; $\mu_{A_1}(x_1)$ is the membership function of A_1 at x_1 ; $\mu_{A_2}(x_2)$ is the membership function of A_2 at x_2 ; and $y = f(x_1, x_2)$ in the consequent is a crisp function.

In fact, $f(x_1, x_2)$ can be a polynomial function of the input variables x_1 and x_2 and can be of any order to appropriately describe the output of the system.

If $f(x_1, x_2)$ is a N th-order polynomial ($N \geq 1$), the resulting fuzzy inference system is called a N th-order TSK fuzzy system.

For $N = 0$, $f(x_1, x_2)$ is a constant, this results in a zero-order TSK fuzzy system, which can be viewed as a special case of (a) the Mamdani fuzzy system in which each rule's consequent is specified by a fuzzy singleton; or (b) the Tsukamoto fuzzy system in which each rule's consequent is specified by a step function with the step located at the constant.

Figure 11.4.1 shows the fuzzy reasoning procedure for a two-rule two-input one-output first-order TSK fuzzy system with two crisp inputs x_1 and x_2 .

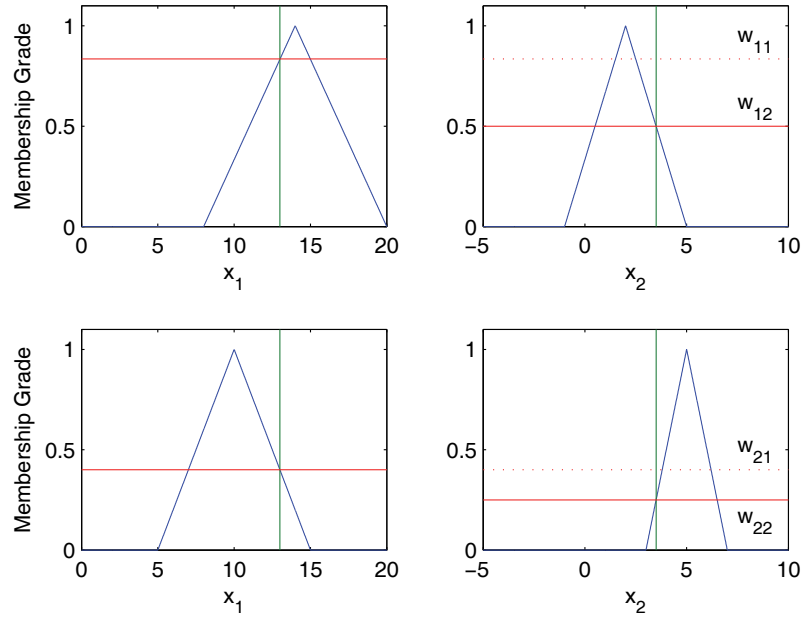


Figure 11.4.1 TSK fuzzy system (2-rule 2-input 1-output).

Rule 1: If X_1 is high and X_2 is low then y_1 is $f_1(x_1, x_2)$.

Rule 2: If X_1 is low and X_2 is high then y_2 is $f_2(x_1, x_2)$.

Each rule in the consequent has a crisp output y_r is equal to $f_r(x_1, x_2)$ for $r = 1, 2$ as

$$y_1 = a_{11}x_1 + a_{12}x_2 + c_1 \quad (11.4.2)$$

$$y_2 = a_{21}x_1 + a_{22}x_2 + c_2 \quad (11.4.3)$$

Given inputs x_1 and x_2 , the weights obtained by the antecedent membership functions $\mu_{11}(x_1)$ and $\mu_{12}(x_2)$ of the Rule 1 are w_{11} and w_{12} , and the corresponding weights obtained by the antecedent membership functions $\mu_{21}(x_1)$ and $\mu_{22}(x_2)$ of the Rule 2 are w_{21} and w_{22} . w_{ri} (for $i = 1, 2$) denotes the peak of the intersection of the input

i with the antecedent i of the r th rule.

The inferred output of each rule y_i (for $i = 1$ to R ($R=2$)) is a crisp value induced by the rule's firing strength w_r obtained using the minimum operator (as $w_r = \min(w_{r1}, w_{r2})$) or the algebraic product operator (as $w_r = w_{r1} w_{r2}$).

The TSK fuzzy system avoids the centroid defuzzification as required in the Mamdani fuzzy system by aggregating each rule's output through weighted average as

$$y = \frac{\sum_{r=1}^R w_r y_r}{\sum_{r=1}^R w_r} \quad (11.4.4)$$

In practice, weighted average is sometimes replaced by a weighted sum to reduce computation, especially in the case of training a fuzzy system with limited computing resource.

However, this simplification could lead to the loss of membership function linguistic meanings unless the sum of firing strengths is close to unity.

$$y = \sum_{r=1}^R w_r y_r \quad (11.4.5)$$

Design Example 11.5 (4-rule 1-input 1-output TSK fuzzy system)
Matlab

For a four-rule one-input one-output first-order TSK fuzzy system, let X be the input and Y be the output of the system, and the four rules of the fuzzy system are defined by

Rule 1: If X is very wet then $y = -2x + 5$

Rule 2: If X is wet then $y = 5x + 4$

Rule 3: If X is dry then $y = -10x + 4$

Rule 4: If X is very dry then $y = 3x - 10$

Define the antecedent membership functions as

$$\mu_{\text{very wet}}(x) = \text{GBellMF}(x, [3, 10, -10])$$

$$\mu_{\text{wet}}(x) = \text{GBellMF}(x, [3.5, 10, -3.5])$$

$$\mu_{\text{dry}}(x) = \text{GBellMF}(x, [3.5, 10, 3.5])$$

$$\mu_{\text{very dry}}(x) = \text{GBellMF}(x, [3, 10, 10])$$

where $x \in [-10, 10]$.

Using the weighted average for defuzzification, determine the input-output curve of the TSK fuzzy system with a crisp input x .

Solution

Figure E11.5.1 (top) plots the membership function of input X with a universe of $[-10, 10]$. With the weighted average for defuzzification, the input-output curve can be obtained as shown in Figure E11.5.1 (bottom). For examples,

At $x = -9$:

Rule	Antecedent	Consequent
1	$\mu_{\text{very wet}}(-9) = 1.0 \Rightarrow w_1 = 1.0$	$y_1 = -2(-9) + 5 = 23$
2	$\mu_{\text{wet}}(-9) = 0.0 \Rightarrow w_2 = 0.0$	$y_2 = 5(-9) + 4 = -41$
3	$\mu_{\text{dry}}(-9) = 0.0 \Rightarrow w_3 = 0.0$	$y_3 = -10(-9) + 4 = 94$
4	$\mu_{\text{very dry}}(-9) = 0.0 \Rightarrow w_4 = 0.0$	$y_4 = 3(-9) - 10 = -37$

Using weighted average output, we have

$$\begin{aligned}
 y &= (w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4) / (w_1 + w_2 + w_3 + w_4) \\
 &= [1.0(23) + 0.0(-41) + 0.0(94) + 0.0(-37)] / (1.0 + 0.0 + 0.0 + 0.0) \\
 &= 23
 \end{aligned}$$

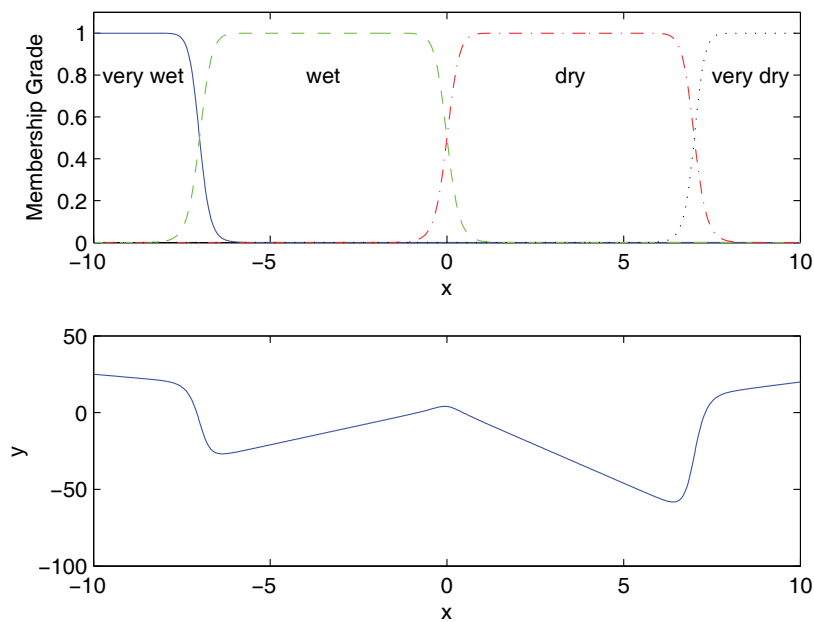


Figure E11.5.1 TSK fuzzy system (4-rule 1-input 1-output).

At $x = -4$:

Rule	Antecedent	Consequent
1	$\mu_{\text{very wet}}(-4) = 0.0 \Rightarrow w_1 = 0.0$	$y_1 = -2(-4) + 5 = 13$
2	$\mu_{\text{wet}}(-4) = 1.0 \Rightarrow w_2 = 1.0$	$y_2 = 5(-4) + 4 = -16$
3	$\mu_{\text{dry}}(-4) = 0.0 \Rightarrow w_3 = 0.0$	$y_3 = -10(-4) + 4 = 44$
4	$\mu_{\text{very dry}}(-4) = 0.0 \Rightarrow w_4 = 0.0$	$y_4 = 3(-4) - 10 = -22$

Using weighted average output, we have

$$\begin{aligned}
 y &= (w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4) / (w_1 + w_2 + w_3 + w_4) \\
 &= [0.0(13) + 1.0(-16) + 0.0(44) + 0.0(-22)] / (0.0 + 1.0 + 0.0 + 0.0) \\
 &= -16
 \end{aligned}$$

At $x = 0$:

Rule	Antecedent	Consequent
1	$\mu_{\text{very wet}}(0) = 0.0 \Rightarrow w_1 = 0.0$	$y_1 = -2(0) + 5 = 5$
2	$\mu_{\text{wet}}(0) = 0.5 \Rightarrow w_2 = 0.5$	$y_2 = 5(0) + 4 = 4$
3	$\mu_{\text{dry}}(0) = 0.5 \Rightarrow w_3 = 0.5$	$y_3 = -10(0) + 4 = 4$
4	$\mu_{\text{very dry}}(0) = 0.0 \Rightarrow w_4 = 0.0$	$y_4 = 3(0) - 10 = -10$

Using weighted average output, we have

$$\begin{aligned}
 y &= (w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4) / (w_1 + w_2 + w_3 + w_4) \\
 &= [0.0(5) + 0.5(4) + 0.5(4) + 0.0(-10)] / (0.0 + 0.5 + 0.5 + 0.0) = 4
 \end{aligned}$$

Design Example 11.6 (4-rule 2-input 1-output TSK fuzzy system)

Matlab

For a four-rule two-input one-output first-order TSK fuzzy system, let X_1 and X_2 be the inputs and Y be the output of the system, and the four rules of the fuzzy system are defined by

Rule 1: If X_1 is wet and X_2 is cold then $y_1 = x_1 + x_2 - 1$

Rule 2: If X_1 is wet and X_2 is hot then $y_2 = 0.5 x_1 - x_2$

Rule 3: If X_1 is dry and X_2 is cold then $y_3 = 2 x_1 + x_2 - 2$

Rule 4: If X_1 is dry and X_2 is hot then $y_4 = -x_1 + 0.5 x_2 + 1$

Define the antecedent membership functions of X_1 as

$$\mu_{\text{wet}}(x_1) = \text{GBellMF}(x_1, [5, 10, -5])$$

$$\mu_{\text{dry}}(x_1) = \text{GBellMF}(x_1, [5, 10, 5])$$

where $x_1 \in [-5, 5]$.

Define the antecedent membership functions of X_2 as

$$\mu_{\text{cold}}(x_2) = \text{GBellMF}(x_2, [5, 20, -7])$$

$$\mu_{\text{hot}}(x_2) = \text{GBellMF}(x_2, [5, 20, 3])$$

where $x_2 \in [-5, 5]$.

Using the algebraic product operator for weights, and the weighted average for defuzzification, determine the input-output curve for the TSK fuzzy system with two crisp inputs x_1 and x_2 .

Solution

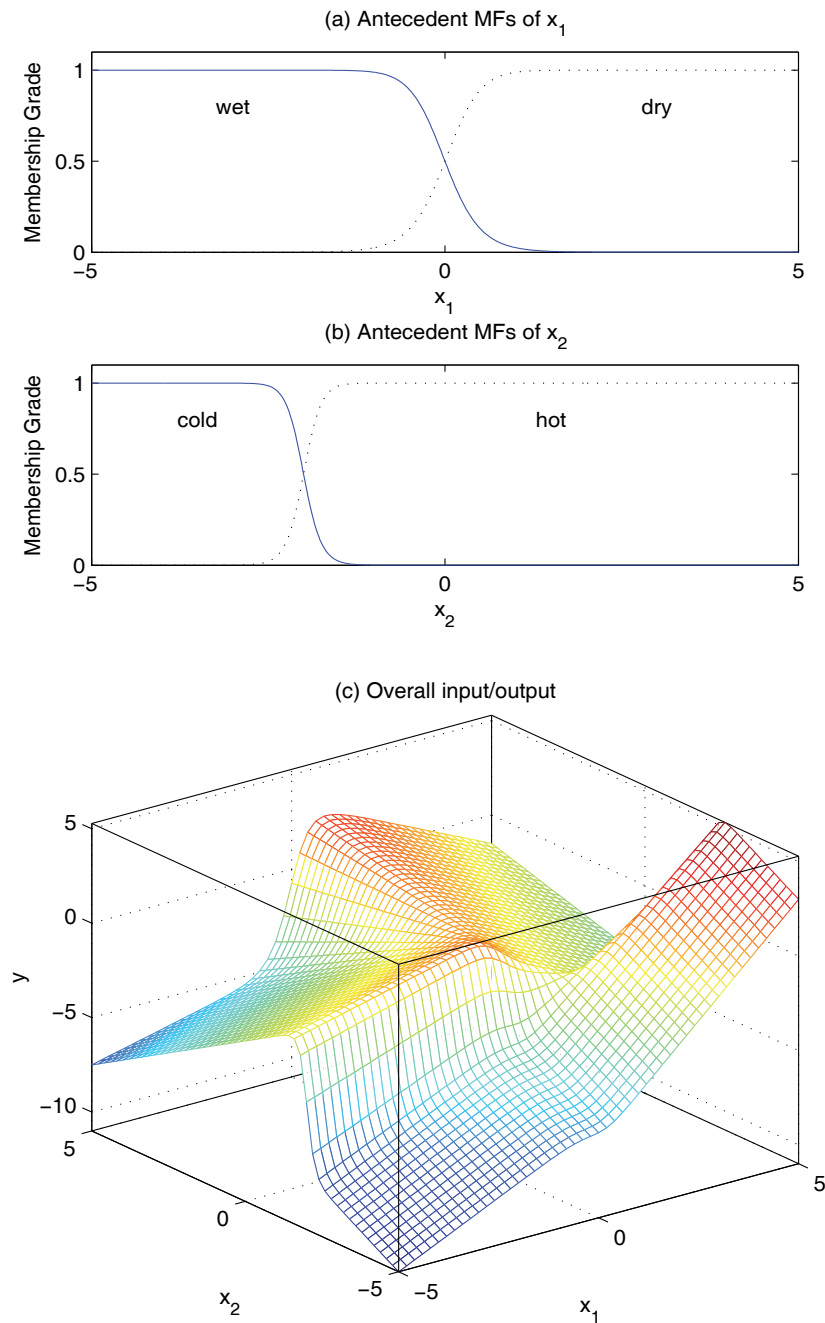


Figure E11.6.1 TSK fuzzy system (4-rule 2-input 1-output).

Figure E11.6.1 (a)-(b) plot the membership functions of inputs X_1 and X_2 , with universes both equal to $[-5, 5]$.

With the algebraic product operator for weights, and the weighted average for defuzzification, the overall input-output curve can be obtained as shown in Figure E11.6.1 (c).

For examples,

At $x_1 = -3, x_2 = -4$ and using algebraic product operator, we obtain

Rule	Antecedent	Consequent
1	$\mu_{\text{wet}}(-3) = 1.0, \mu_{\text{cold}}(-4) = 1.0$ $\Rightarrow w_1 = 1.0 \times 1.0 = 1.0$	$y_1 = (-3) + (-4) - 1 = -8$
2	$\mu_{\text{wet}}(-3) = 1.0, \mu_{\text{hot}}(-4) = 0.0$ $\Rightarrow w_2 = 1.0 \times 0.0 = 0.0$	$y_2 = 0.5(-3) - (-4) = 2.5$
3	$\mu_{\text{dry}}(-3) = 0.0, \mu_{\text{cold}}(-4) = 1.0$ $\Rightarrow w_3 = 0.0 \times 1.0 = 0.0$	$y_3 = 2(-3) + (-4) - 2 = -12$
4	$\mu_{\text{dry}}(-3) = 0.0, \mu_{\text{hot}}(-4) = 0.0$ $\Rightarrow w_4 = 0.0 \times 0.0 = 0.0$	$y_4 = -(-3) + 0.5(-4) + 1 = 2$

Using weighted average output, we have

$$y = (w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4) / (w_1 + w_2 + w_3 + w_4)$$

$$= [1.0(-8) + 0.0(2.5) + 0.0(-12) + 0.0(2)] / (1.0 + 0.0 + 0.0 + 0.0) = -8$$

At $x_1 = 0, x_2 = 2$ and using algebraic product operator, we obtain

Rule	Antecedent	Consequent
1	$\mu_{\text{wet}}(0) = 0.5, \mu_{\text{cold}}(2) = 0.0$ $\Rightarrow w_1 = 0.5 \times 0.0 = 0.0$	$y_1 = (0) + (2) - 1 = 1$
2	$\mu_{\text{wet}}(0) = 0.5, \mu_{\text{hot}}(2) = 1.0$ $\Rightarrow w_2 = 0.5 \times 1.0 = 0.5$	$y_2 = 0.5(0) - (2) = -2$
3	$\mu_{\text{dry}}(0) = 0.5, \mu_{\text{cold}}(2) = 0.0$ $\Rightarrow w_3 = 0.5 \times 0.0 = 0.0$	$y_3 = 2(0) + (2) - 2 = 0$
4	$\mu_{\text{dry}}(0) = 0.5, \mu_{\text{hot}}(2) = 1.0$ $\Rightarrow w_4 = 0.5 \times 1.0 = 0.5$	$y_4 = -(0) + 0.5(2) + 1 = 2$

Using weighted average output, we have

$$y = (w_1 y_1 + w_2 y_2 + w_3 y_3 + w_4 y_4) / (w_1 + w_2 + w_3 + w_4)$$

$$= [0.0(1) + 0.5(-2) + 0.0(0) + 0.5(2)] / (0.0 + 0.5 + 0.0 + 0.5) = 0$$

Unlike the Mamdani fuzzy system, the TSK fuzzy system does not follow the compositional rule of inference strictly in its fuzzy reasoning procedure.

The resultant overall system output via either weighted average or weighted sum is always crisp, and unable to preserve fuzziness from its input(s) to an output.

This creates a difficulty when the inputs to a TSK fuzzy system are fuzzy and a fuzzy output is required. However, due to its simplicity, the TSK fuzzy system is a popular choice for fuzzy modeling.

11.5 Comparisons

Among the three fuzzy systems, differences exist in their consequent parts, in which the Mamdani fuzzy system adopts membership functions for outputs; the Tsukamoto fuzzy system adopts monotonic membership functions (or input to output functions) for outputs; and the TSK fuzzy system adopts polynomial functions for outputs.

The output membership functions of the three systems are differentiable which facilitate gradient-based learning.

All the three systems use either the minimum operator or algebraic product operator for determining firing strength of each rule.

The Mamdani fuzzy system uses the maximum operator for aggregation of fuzzy outputs, and then computes the centroid of aggregated output membership functions for defuzzification.

The Tsukamoto fuzzy system and the TSK fuzzy system both use a weighted average for output defuzzification.

In the Mamdani fuzzy system, it is possible to compute the areas and the centroids of the consequent membership functions of each rule in advance.

Hence, the direct computation of the centroid of aggregated output membership functions can be reduced to the computation of a simple weighted average as in the other two fuzzy systems.

Chapter 12

Fuzzy Neural Networks

12.1 Introduction

A fuzzy neural network combines the structure of a neural network with the operations of fuzzy logic to form a class of intelligent networks. The features of fuzzy neural networks include parallel structure, allowing quick learning, and processing information using fuzzy logic.

Multilayer neural networks, fuzzy systems, and fuzzy neural networks can be trained from given input to output data to solve a problem without requiring a mathematical model of the given problem.

A special feature of fuzzy neural networks is that they have the ability to interpret if-then rules learned from input to output data, which is not straightforward to do so for the case of multilayer neural networks.

In this chapter, the fuzzy neuron and five types of fuzzy neurons are introduced and defined in Section 12.2. Three fuzzy inference models and their developed fuzzy inference networks and learning algorithms are described in Sections 12.3-12.5.

The applications of the unsupervised and supervised fuzzy neural networks for invariant pattern recognition are described in Section 12.6. The applications of the fuzzy inference networks for fuzzy and non-fuzzy classifications are explained in Section 12.7. References for fuzzy neural networks are given by [Kwa9211], [Kwa9408], [Kwa0008b], [Cai9305], [Cai9410], [Cai9502], [Cai9505], [Cai9806]. Notations for fuzzy neural networks is listed in Table 12.1.1.

Table 12.1.1 Notations for fuzzy neural networks.

Symbol	Description
FN	Fuzzy neuron
FNN	Fuzzy neural network

MCFNN	Min-Comp FNN (3-layer)
MSFNN	Min-Sum FNN (3-layer)
MMFNN	Min-Max FNN (3-layer)
UMCFNN	Unsupervised Min-Comp FNN (4-layer)
SMCFNN	Supervised Min-Comp FNN (4-layer)
Input-FN	Input fuzzy neuron
Tran-FN	Transit fuzzy neuron
Comp-FN	Competitive fuzzy neuron
Sum-FN	Sum fuzzy neuron
Max-FN	Maximum fuzzy neuron
Min-FN	Minimum fuzzy neuron
P	Number of outputs of MCFNN, MSFNN, MMFNN, UMCNN, SMCNN
M	Number of minimum neurons of MCFNN, MSFNN, MMFNN, UMCNN, SMCNN
N	Number of Tran-FNs and Max-FNs
$N_1 \times N_2$	Number of input pixels, Tran-FNs, and Max-FNs for UMCNN and SMCNN.
$[l]$	The subscript $[l]$ denotes layer index. For a three-layer FNN, $l=1, 2, 3$; for a four-layer FNN, $l=1, 2, 3, 4$.

12.2 Fuzzy Neurons

A traditional neuron has N weighted inputs, x_1 to x_N , and one output y . The neuron sums all these weighted inputs and subtracts the sum by an activation threshold, t .

The obtained value is then passed through an activating function, $f(\cdot)$, to give the output, y .

$$y = f \left[\left(\sum_{i=1}^N w_i x_i \right) - t \right] \quad (12.2.1)$$

In other words, a traditional neuron is characterized by multiple linear weighted inputs, summation being used as an aggregation function of weighted inputs and a threshold; and one nonlinear output.

A fuzzy neuron shown in Figure 12.2.1 is a neuron that has an ability to process fuzzy information.

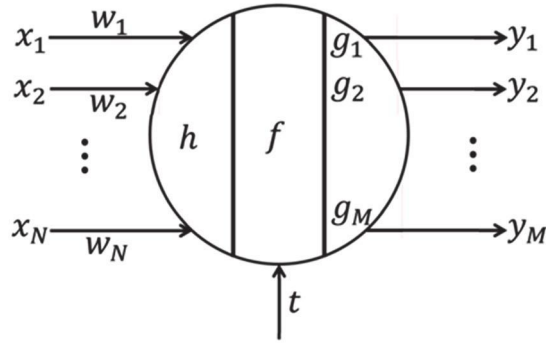


Figure 12.2.1 Diagram of a fuzzy neuron.

A fuzzy neuron can be defined by the following three equations as

$$z = h(w_1, x_1, w_2, x_2, \dots, w_N, x_N) \quad (12.2.2)$$

$$s = f(z, t) \quad (12.2.3)$$

$$y_j = g_j(s) \text{ for } j = 1, 2, \dots, M \quad (12.2.4)$$

A fuzzy neuron has N inputs (x_1 to x_N) and M outputs (y_1 to y_M); N input weights (w_1 to w_N); an input aggregation function $h(\cdot)$; a net input z ; an activation threshold t ; a nonlinear activation function $f(\cdot)$; a state s ; and M output nonlinear functions $g_1(\cdot)$ to $g_M(\cdot)$.

In general, a fuzzy neuron is characterized by multiple weighted inputs; an optional input aggregation function $h(\cdot)$; one state with a nonlinear activation function $f(\cdot)$; and multiple outputs with output nonlinear functions $g_j(\cdot)$.

The inputs of a fuzzy neuron can be fuzzy variables or non-fuzzy variables. The aggregation operation $h(\cdot)$ can be a fuzzy or non-fuzzy operation; fuzzy operations include min, max, and sum.

The outputs of a fuzzy neuron can be used to represent fuzzy concepts when the output nonlinear functions $g_j(.)$ for $j = 1, 2, \dots, M$ are membership functions of fuzzy sets.

Functionally, a fuzzy neuron has multiple inputs and multiple outputs in which its inputs (x_1 to x_N) are signals coming from other fuzzy neurons and its outputs (y_1 to y_M) are signals going to other fuzzy neurons.

The aggregation function $h(.)$ expresses the way on how a fuzzy neuron integrates the effects of all other fuzzy neurons (For a traditional neuron, the aggregation function is a simple summation).

The activation function $f(z, t)$ describes how a fuzzy neuron adjusts its membrane; and the state s corresponds to the membrane potential of a nerve cell.

The membrane potential is defined as the potential inside a cell membrane, measured relative to the fluid just outside; it is negative under resting conditions and becomes positive during an action potential.

The output functions, $g_j(s)$ for $j = 1$ to M , give a comprehensive description of the nonlinear interactions among fuzzy neurons.

The aggregation function $h(z)$ and the activation function $f(z, t)$ are the intrinsic features of a fuzzy neuron. Different types of fuzzy neurons can be defined by changing these functions.

The input weights, the output functions, and the activation threshold that describe the interactions between fuzzy neurons, can be adjusted during the learning process.

These parameters are called learning parameters. The process of adjusting the learning parameters is in fact a process of adjusting the structure of a fuzzy neural network.

Based on the definition of the fuzzy neuron, six types of fuzzy neurons are introduced next.

12.2.1 Input fuzzy neuron (Input-FN)

The Input-FN is a fuzzy neuron which is used in the input layer with one input x and one output z such that

$$z = x \quad (12.2.5)$$

12.2.2 Transit fuzzy neuron (Tran-FN)

The Transit fuzzy neuron is a fuzzy neuron that transfers an input into membership values.

The Tran-FN has only one input x , such that the state s is equal to the network input z , and both equal to the input x as

$$s = z = x \quad (12.2.6)$$

The outputs are obtained from the output functions of the states as

$$y_j = g_j(s) \text{ for } j = 1, 2, \dots, M \quad (12.2.7)$$

The functions $g_j(.)$ for $j = 1, 2, \dots, M$ are membership functions.

The outputs of the Tran-FN are fuzzy variables and Tran-FNs can be used to perform fuzzification in a network.

12.2.3 Competitive fuzzy neuron (Comp-FN)

The Comp-FN is a fuzzy neuron with competitive variable inputs $(c_1, c_2, \dots, c_K)$, a variable threshold T , and one output y such that

$$y = g(s - T) = \begin{cases} 1 & \text{if } s \geq T \\ 0 & \text{if } s < T \end{cases} \quad (12.2.8)$$

$$T = t(c_1, c_2, \dots, c_K) \quad (12.2.9)$$

In (12.2.8), s is the state of the Comp-FN, and $t(.)$ is the threshold function.

12.2.4 Sum fuzzy neuron (Sum-FN)

The Sum-FN is a fuzzy neuron with fuzzy variable inputs and a sum function as the aggregation function.

The output of the Sum-FN is a fuzzy variable which can be expressed as the sum of weight inputs as

$$z = \sum_{i=1}^N w_i x_i \quad (12.2.10)$$

12.2.5 Maximum fuzzy neuron (Max-FN)

The Max-FN is a fuzzy neuron with fuzzy variable inputs and a maximum function as the aggregation function. A maximum function is equivalent to a fuzzy OR operation.

The output (or the net input z) of the Max-FN is a fuzzy variable which can be expressed as the maximum of weight inputs as

$$z = \max_{i=1}^N w_i x_i \quad (12.2.11)$$

12.2.6 Minimum fuzzy neuron (Min-FN)

The Min-FN is a fuzzy neuron with fuzzy variable inputs and a minimum function as the aggregation function. A minimum function is equivalent to a fuzzy AND operation.

The output (or the net input z) of the Min-FN is a fuzzy variable which can be expressed as the minimum of weight inputs as

$$z = \min_{i=1}^N w_i x_i \quad (12.2.12)$$

12.3 Fuzzy Inference Models

Fuzzy rule-based systems have been successfully applied to expert systems and control systems.

A fuzzy rule-based system for pattern classification and recognition problems has been introduced in [Kwa9211], [Kwa9408], [Kwa0008b]. In this section, fuzzy inference models for pattern classification and recognition systems are explained.

A fuzzy inference model is expressed by fuzzy inference rules of the form shown below.

IF x_1 is $A_{1j}, \dots$, and x_N is A_{Nj} ,
 THEN y_1 is $y_{j1}, \dots$, and y_P is y_{jP} (12.3.1)

for $j = 1, 2, \dots, M$

where A_{ij} for $i = 1, 2, \dots, N$ are fuzzy subsets and y_{jp} for $j = 1, 2, \dots, M$ are real numbers in $[0, 1]$.

For simplicity, triangular functions can be used as membership functions of A_{ij} . In other words, the output is expressed in terms of a crisp variable instead of a linguistic variable.

Each output of the fuzzy inference model describes the membership value of the input pattern belonging to one fuzzy class. Each model deduces outputs according to its inference rules and its specified inference algorithm. Each fuzzy class is an aggregation of fuzzy hyperboxes. A hyperbox is a multidimensional box structure that defines a range of points.

Assuming B_j represents the j th hyperbox ($j = 1, 2, \dots, M$), $c_j(\mathbf{X})$ is the membership function of B_j , then each fuzzy class F_p ($p = 1$ to P) is an aggregation of B_j as

$$F_p = \bigcup_{j=1}^M B_j \quad (12.3.2)$$

Three fuzzy inference models are shown below to obtain outputs y_p for $p = 1, 2, 3, \dots, P$ from the inference rules and the current inputs x_i for $i = 1, 2, 3, \dots, N$ from the input layer.

The union operation is maximum for Min-Comp inference model, weighted sum for Min-Sum inference model, and weighted maximum for Min-Max inference model.

Let y_p represents the membership value of an input pattern belonging to the p th class F_p . The membership value y_p is determined by the most important prototype for the Min-Comp inference model and

the Min-Max inference model, and by all the important prototypes for Min-Sum inference model.

The most important prototype for an input pattern regarding the p th class is defined as $c_j(\mathbf{X})y_{jp} = \max_j(c_j(\mathbf{X})y_{jp})$.

An important prototype for an input pattern regarding the p th class is defined as $c_j(\mathbf{X}) \neq 0$ and $y_{jp} \neq 0$.

12.3.1 Min-Competitive inference model

The compatibility for the antecedent conditions of each rule is

$$c_j = \min_{i=1 \text{ to } N} (A_{ij}(x_i)) \text{ for } j = 1 \text{ to } M \quad (12.3.3)$$

The complete inference results are

$$t = \max_{j=1 \text{ to } M} (c_j) \quad (12.3.4)$$

$$y_p = w_{jp} \text{ if } c_j = t \text{ for } p = 1 \text{ to } P \quad (12.3.5)$$

12.3.2 Min-Sum inference model

The compatibility for the antecedent conditions of each rule is

$$c_j = \min_{i=1 \text{ to } N} (A_{ij}(x_i)) \text{ for } j = 1 \text{ to } M \quad (12.3.6)$$

The complete inference results are

$$y_p = \left\{ \begin{array}{ll} \frac{\sum_{j=1}^M (c_j y_{jp})}{\sum_{j=1}^M c_j} & \text{if } \sum_{j=1}^M c_j \neq 0 \\ 0 & \text{if } \sum_{j=1}^M c_j = 0 \end{array} \right\} \text{ for } p = 1 \text{ to } P \quad (12.3.7)$$

12.3.3 Min-Max inference model

The compatibility for the antecedent conditions of each rule is

$$c_j = \min_{i=1 \text{ to } N} (A_{ij}(x_i)) \text{ for } j = 1 \text{ to } M \quad (12.3.8)$$

The complete inference results are

$$y_p = \max_{j=1 \text{ to } M} (c_j y_{jp}) \text{ for } p = 1 \text{ to } P \quad (12.3.9)$$

12.4 Fuzzy Neural Networks

In this section, fuzzy neural networks for pattern classification and recognition representing mappings from input $U \subset R^N$ to the unit hypercube $[0, 1]^P$ of P -dimensional output spaces are to be realized in neural network structures as distributed fuzzy rule-based systems using the fuzzy neurons.

A hypercube is defined as a geometric figure in four or more dimensions.

Figure 12.4.1 shows a fuzzy neural network realized by a three-layer feedforward neural network structure.

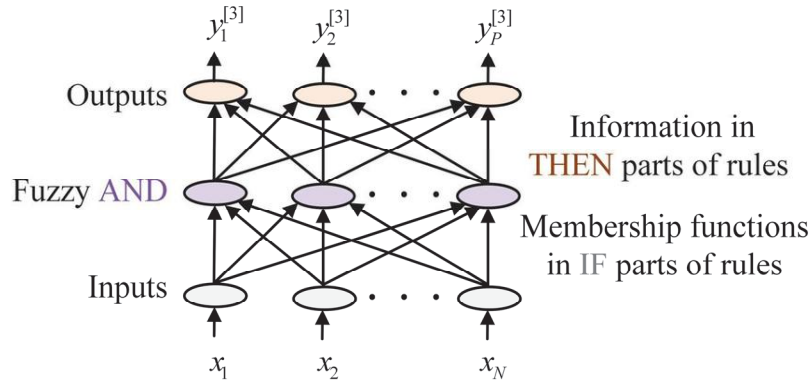


Figure 12.4.1 Three-layer fuzzy neural network.

The input layer accepts an input pattern into the fuzzy neural network.

The output functions of the FNs in the first layer represent the membership functions of fuzzy subsets $A_{1j}, A_{2j}, \dots, A_{Nj}$ ($j = 1, 2, \dots, M$) in the IF parts of the inference rules.

These output functions fuzzify the input data of an input pattern.

The second layer represents inference rules. Each FN in the second layer computes the compatibility of the input pattern to one of the inference rules.

The information in the THEN parts of the inference rules is stored in the weights from the FNs in the second layer to the FNs in the third layer.

The third layer produces outputs from the information of the input pattern and all the inference rules.

In summary, each layer of the FNNs has a different function. The input layer fuzzifies the input pattern and gives the membership values of the input data with respect to the defined fuzzy sets in each projection of U .

The second layer represents inference rules and computes the compatibility for the antecedent condition of each fuzzy inference rule.

The third layer combines the information of all the inference rules and the information of the input pattern and then gives outputs according to the inference method.

12.4.1 Min-Competitive FNN

The Min-Competitive Fuzzy Neural Network (MCFNN) as shown in Figure 12.4.2 has a three-layer feedforward neural network structure.

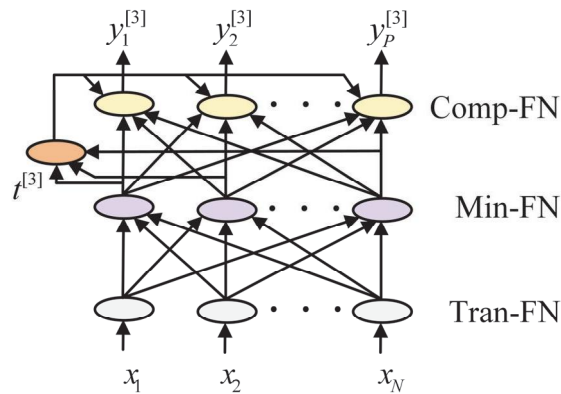


Figure 12.4.2 Min-Competitive Fuzzy Neural Network (MCFNN).

The MCFNN consists of N Tran-FNs, M Min-FNs, and P Comp-FNs, among which only M has to be determined by learning.

Tran-FNs are used in the first layer, Min-FNs are used in the second layer, and Comp-FNs are used in the third layer.

In the first layer, isosceles triangular membership functions of equal slopes α as shown in Figure 12.4.3 are used.

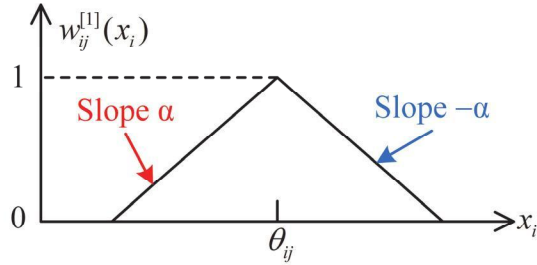


Figure 12.4.3 Weight function $w_{ij}^{[1]}(x_i)$ of MCFNN.

The operations of the first layer are

$$s_{ij}^{[1]} = w_{ij}^{[1]}(x_i) \quad \text{for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M \quad (12.4.1)$$

$$w_{ij}^{[1]}(x_i) = \begin{cases} 1 + \alpha(x_i - \theta_{ij}) & \text{if } 1 \geq 1 + \alpha(x_i - \theta_{ij}) \geq 0 \\ 1 - \alpha(x_i - \theta_{ij}) & \text{if } 1 \geq 1 - \alpha(x_i - \theta_{ij}) \geq 0 \\ 0 & \text{if otherwise} \end{cases} \quad (12.4.2)$$

for $i = 1, 2, 3, \dots, N$ and $j = 1, 2, 3, \dots, M$

The operation of the second layer is

$$s_j^{[2]} = \min_{i=1 \text{ to } N} (s_{ij}^{[1]}) \quad \text{for } j = 1 \text{ to } M \quad (12.4.3)$$

The operations of the third layer are

$$s_p^{[3]} = w_{jp}^{[2]} s_j^{[2]} \quad \text{if } s_j^{[2]} = t^{[3]} \quad \text{for } p = 1 \text{ to } P \quad (12.4.4)$$

$$t^{[3]} = \max_{j=1 \text{ to } M} (s_j^{[2]}) \quad \text{for } p = 1 \text{ to } P \quad (12.4.5)$$

The threshold represented by $t^{[3]}$ to be applied to all the Comp-FNs is taken as the maximum value among all the outputs of the FNs in the second layer.

$w_{jp}^{[2]}$ for $j = 1$ to M and $p = 1$ to P denotes the connection weight between the j th MIN-FN and the p th COMP-FN.

The output of $t^{[3]}$ controls each p th COMP-FN to allow only the output from the J th Min-FN to pass through to form the output $s_p^{[3]} = w_{jp}^{[2]} s_j^{[2]}$.

For a given input vector in which the output $s_j^{[2]}$ of the J th MIN-FN has a maximum value (or the J th rule wins), the output $s_j^{[2]}$ of the J th MIN-FN is equal to 1 for a perfect match between the input vector and the J th rule and a non-zero value up to 1 for a match of different degree.

Each layer of the MCFNN has a different function. The input layer fuzzifies the input pattern and gives the membership values of the input data with respect to the defined fuzzy sets in each projection of U .

The second layer represents inference rules and computes the compatibility for the antecedent condition of each fuzzy inference rule.

The third layer combines the information of all the inference rules and the information of the input pattern and then gives outputs according to the inference method.

Note that the learning algorithm of the three-layer MCFNN is the same for the four-layer SMCFNN (described in Section 12.6) for pattern recognition but without having the second layer (or without using fuzzification functions).

During learning, the parameters to be determined include the positive-value parameters α and θ_{ij} for $i = 1, 2, 3, \dots, N$ and $j = 1, 2, 3, \dots, M$ of the membership functions; the number M of Min-FNs; and all the connection weights $w_{jp}^{[2]}$ from the j th Min-FN in the second layer to the p th Comp-FN in the third layer for $j = 1$ to M and $p = 1$ to P .

12.4.1.1 One-input one-output problems

For a one-input one-output problem, all the parameters of a MCFNN can be grouped into a parameter vector $\mathbf{c}_c = [\alpha, \theta_{ij} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M, w_{jp}^{[2]} \text{ for } j = 1 \text{ to } M \text{ and } p = 1 \text{ to } P]$.

For example, for $N = 1, M = 5$, and $P = 1$, $\mathbf{c}_c = [\alpha, \theta_{11}, \theta_{12}, \theta_{13}, \theta_{14}, \theta_{15}, w_{11}^{[2]}, w_{21}^{[2]}, w_{31}^{[2]}, w_{41}^{[2]}, w_{51}^{[2]}]$.

12.4.1.2 Two-input one-output problems

For a two-input one-output problem, all the parameters of a MCFNN can be grouped into a parameter vector $\mathbf{c}_c = [\alpha, \theta_{ij} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M, w_{jp}^{[2]} \text{ for } j = 1 \text{ to } M \text{ and } p = 1 \text{ to } P]$.

For example, for $N = 2, M = 5$, and $P = 1$, $\mathbf{c}_c = [\alpha, \theta_{11}, \theta_{12}, \theta_{13}, \theta_{14}, \theta_{15}, \theta_{21}, \theta_{22}, \theta_{23}, \theta_{24}, \theta_{25}, w_{11}^{[2]}, w_{21}^{[2]}, w_{31}^{[2]}, w_{41}^{[2]}, w_{51}^{[2]}]$ with a total of 16 parameters.

12.4.2 Min-Sum FNN

The Min-Sum Fuzzy Neural Network (MSFNN) as shown in Figure 12.4.4 has a three-layer feedforward neural network structure.

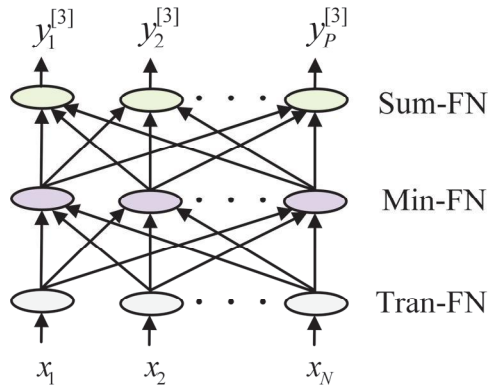


Figure 12.4.4 Min-Sum Fuzzy Neural Network.

The MSFNN consists of N Tran-FNs, M Min-FNs, and P Sum-FNs, among which only M has to be determined by learning.

Tran-FNs are used in the first layer, Min-FNs are used in the second layer, and Sum-FNs are used in the third layer.

In the first layer, non-isosceles triangular membership functions of unequal slopes as shown in Figure 12.4.5 are used between Tran-FNs and Min-FNs.

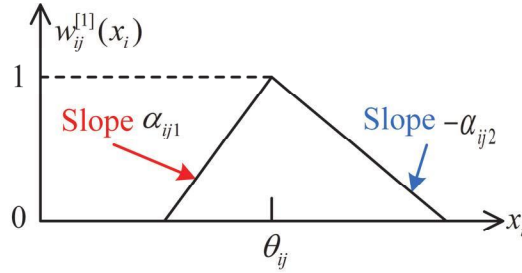


Figure 12.4.5 Weight function $w_{ij}^{[1]}(x_i)$ of MSFNN.

The operations of the first layer are

$$s_{ij}^{[1]} = w_{ij}^{[1]}(x_i) \quad \text{for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M \quad (12.4.6)$$

$$w_{ij}^{[1]}(x_i) = \begin{cases} 1 + \alpha_{ij1}(x_i - \theta_{ij}) & \text{if } 1 \geq 1 + \alpha_{ij1}(x_i - \theta_{ij}) \geq 0 \\ 1 - \alpha_{ij2}(x_i - \theta_{ij}) & \text{if } 1 \geq 1 - \alpha_{ij2}(x_i - \theta_{ij}) \geq 0 \\ 0 & \text{if otherwise} \end{cases} \quad (12.4.7)$$

for $i = 1, 2, 3, \dots, N$ and $j = 1, 2, 3, \dots, M$

The operation of the j th Min-FN in the second layer is

$$s_j^{[2]} = \min_{i=1 \text{ to } N} (s_{ij}^{[1]}) \quad \text{for } j = 1 \text{ to } M \quad (12.4.8)$$

In the third layer, the summation of the Sum-FNs combines the information of all the inference rules and the compatibilities of the input pattern; and weighted average is used as the inference output.

The operation of the third layer is

$$s_p^{[3]} = \begin{cases} \frac{\sum_{j=1}^M (w_{jp}^{[2]} s_j^{[2]})}{\sum_{j=1}^M s_j^{[2]}} & \text{if } \sum_{j=1}^M s_j^{[2]} \neq 0 \\ 0 & \text{if } \sum_{j=1}^M s_j^{[2]} = 0 \end{cases} \quad \text{for } p = 1 \text{ to } P \quad (12.4.9)$$

During learning, the parameters to be determined include the positive-value parameters α_{ij1} , α_{ij2} , and θ_{ij} of the weight (or membership) functions $w_{ij}^{[1]}(x_i)$ ($i = 1, 2, 3, \dots, N$ and $j = 1, 2, 3, \dots, M$); the number of Min-FNs, M ; and all the connection weights $w_{jp}^{[2]}$ from the j th Min-FN in the second layer to the p th Sum-FN in the third layer for $j = 1$ to M and $p = 1$ to P .

12.4.2.1 One-input one-output problems

For a one-input one-output problem, all the parameters of a MSFNN can be grouped into a parameter vector $\mathbf{c}_s = [\alpha_{ij1}, \alpha_{ij2}, \theta_{ij} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M, w_{jp}^{[2]} \text{ for } j = 1 \text{ to } M \text{ and } p = 1 \text{ to } P]$.

For example, for $N = 1$, $M = 5$, and $P = 1$, we have $\mathbf{c}_s = [\alpha_{111}, \alpha_{121}, \alpha_{131}, \alpha_{141}, \alpha_{151}, \alpha_{112}, \alpha_{122}, \alpha_{132}, \alpha_{142}, \alpha_{152}, \theta_{11}, \theta_{12}, \theta_{13}, \theta_{14}, \theta_{15}, w_{11}^{[2]}, w_{21}^{[2]}, w_{31}^{[2]}, w_{41}^{[2]}, w_{51}^{[2]}]$.

12.4.2.2 Two-input one-output problems

For a two-input one-output problem, all the parameters of a MSFNN can be grouped into a parameter vector $\mathbf{c}_s = [\alpha_{ij1}, \alpha_{ij2}, \theta_{ij} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M, w_{jp}^{[2]} \text{ for } j = 1 \text{ to } M \text{ and } p = 1 \text{ to } P]$.

For example, for $N = 2$, $M = 5$, and $P = 1$, we have $\mathbf{c}_s = [\alpha_{111}, \alpha_{112}, \alpha_{121}, \alpha_{122}, \alpha_{131}, \alpha_{132}, \alpha_{141}, \alpha_{142}, \alpha_{151}, \alpha_{152}, \theta_{11}, \theta_{12}, \theta_{13}, \theta_{14}, \theta_{15}, \alpha_{211}, \alpha_{212}, \alpha_{221}, \alpha_{222}, \alpha_{231}, \alpha_{232}, \alpha_{241}, \alpha_{242}, \alpha_{251}, \alpha_{252}, \theta_{21}, \theta_{22}, \theta_{23}, \theta_{24}, \theta_{25}, w_{11}^{[2]}, w_{21}^{[2]}, w_{31}^{[2]}, w_{41}^{[2]}, w_{51}^{[2]}]$ with a total of 35 parameters.

12.4.3 Min-Max FNN

The Min-Max Fuzzy Neural Network (MMFNN) as shown in Figure 12.4.6 has a three-layer feedforward neural network structure.

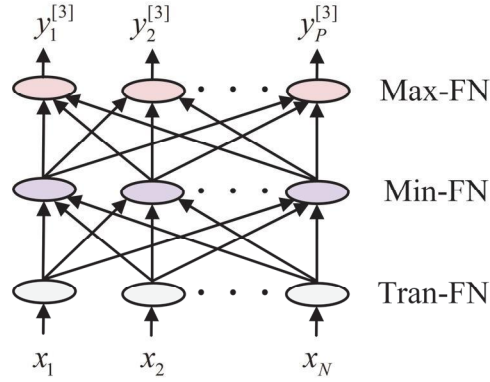


Figure 12.4.6 Min-Max Fuzzy Neural Network (MMFNN).

The MMFNN consists of N Tran-FNs, M Min-FNs, and P Max-FNs, among which M has to be determined by learning.

In the MMFNN, the first layer accepts input pattern x_i for $i = 1$ to N into the network in which each Tran-FN accepts one feature of an input pattern and transfers it to all the inputs of the M Min-FNs in the second layer.

Min-FNs are used in the second layer to represent inference rules, one Min-FN for each rule. The number of M rules is to be determined.

The weight functions $w_{ij}^{[1]}(x_i)$ connecting the first layer to the second layer are different among the MCFNN, the MSFNN, and the MMFNN.

In the MMFNN, x_i is the input to the i th Tran-FN in the first layer; and $w_{ij}^{[1]}(x_i)$ denotes the i th input to the j th Min-FN in the second layer. $w_{ij}^{[1]}(x_i)$ represents the membership function of the fuzzy subset A_{ij} of the fuzzy inference rule set.

In the first layer, a triangular membership function of equal slopes but different threshold on each side t_{ij1} and t_{ij2} are used for $w_{ij}^{[1]}(x_i)$.

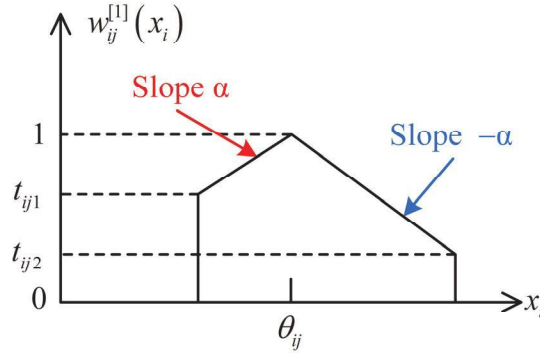


Figure 12.4.7 Weight function $w_{ij}^{[1]}(x_i)$ of MMFNN.

The operations of the first layer are

$$s_{ij}^{[1]} = w_{ij}^{[1]}(x_i) \quad \text{for } i = 1 \text{ to } N; j = 1 \text{ to } M \quad (12.4.10)$$

$$w_{ij}^{[1]}(x_i) = \begin{cases} 1 + \alpha(x_i - \theta_{ij}) & \text{if } 1 \geq 1 + \alpha(x_i - \theta_{ij}) > t_{ij1} \\ 1 - \alpha(x_i - \theta_{ij}) & \text{if } 1 \geq 1 - \alpha(x_i - \theta_{ij}) > t_{ij2} \\ 0 & \text{if otherwise} \end{cases} \quad (12.4.11)$$

for $i = 1, 2, 3, \dots, N$ and $j = 1, 2, 3, \dots, M$

The weights from the i th Tran-FN in the first layer compute and represent the membership values of the i th input with respect to the fuzzy sets defined in the i th projection of U .

The weight functions transform the features of an input pattern into membership values.

The information in the IF parts of the inference rules is contained in the membership functions $w_{ij}^{[1]}(x_i)$ which denotes the membership functions of the fuzzy subset A_{ij} in the first layer.

The second layer computes the compatibility for the antecedent condition of each fuzzy inference rule.

Each Min-FN in the second layer takes the minimum of all its inputs from the first layer. The operation of the j th Min-FN is

$$s_j^{[2]} = \min_{i=1 \text{ to } N} (s_{ij}^{[1]}) \text{ for } j = 1 \text{ to } M \quad (12.4.12)$$

The third layer gives the system outputs according to the fuzzy inference model. Max-FNs are used in the third layer.

The information in the THEN parts of the inference rules is contained in the weights from the Min-FNs in the second layer to the Max-FNs in the third layer.

Let $w_{jp}^{[2]}$ represent the weight from the j th Min-FN in the second layer to the p th Max-FN in the third layer; the operation of the p th Max-FN is

$$s_p^{[3]} = \max_{j=1 \text{ to } M} (w_{jp}^{[2]} s_j^{[2]}) \text{ for } p = 1 \text{ to } P \quad (12.4.13)$$

During learning, the parameters to be determined include the positive-value parameters, α , t_{ij1} , t_{ij2} , and θ_{ij} of the weight or membership functions $w_{ij}^{[1]}(x_i)$ for $i = 1, 2, 3, \dots, N$ and $j = 1, 2, 3, \dots, M$; the number of Min-FNs, M ; and all the connection weights $w_{jp}^{[2]}$ from the j th Min-FN in the second layer to the p th Max-FN in the third layer for $j = 1$ to M , and $p = 1$ to P .

12.4.3.1 One-input one-output problems

For a one-input one-output problem, all the parameters of a MMFNN can be grouped into a parameter vector as $\mathbf{c}_m = [\alpha, t_{ij1}, t_{ij2}, \theta_{ij} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M, w_{jp}^{[2]} \text{ for } j = 1 \text{ to } M \text{ and } p = 1 \text{ to } P]$.

For example, for $N = 1$, $M = 5$, and $P = 1$, we have $\mathbf{c}_m = [\alpha, t_{111}, t_{121}, t_{131}, t_{141}, t_{151}, t_{112}, t_{122}, t_{132}, t_{142}, t_{152}, \theta_{11}, \theta_{12}, \theta_{13}, \theta_{14}, \theta_{15}, w_{11}^{[2]}, w_{21}^{[2]}, w_{31}^{[2]}, w_{41}^{[2]}, w_{51}^{[2]}]$ with a total of 21 parameters.

12.4.3.2 Two-input one-output problems

For a two-input one-output problem, all the parameters of a MMFNN can be grouped into a parameter vector $\mathbf{c}_m = [\alpha, t_{ij1}, t_{ij2}, \theta_{ij} \text{ for } i = 1 \text{ to } N \text{ and } j = 1 \text{ to } M, w_{jp}^{[2]} \text{ for } j = 1 \text{ to } M \text{ and } p = 1 \text{ to } P]$.

For example, for $N = 2$, $M = 5$, and $P = 1$, we have $\mathbf{c}_m = [\alpha, t_{111}, t_{112}, t_{121}, t_{122}, t_{131}, t_{132}, t_{141}, t_{142}, t_{151}, t_{152}, \theta_{11}, \theta_{12}, \theta_{13}, \theta_{14}, \theta_{15}, t_{211}, t_{212}, t_{221}, t_{222}, t_{231}, t_{232}, t_{241}, t_{242}, t_{251}, t_{252}, \theta_{21}, \theta_{22}, \theta_{23}, \theta_{24}, \theta_{25}, w_{11}^{[2]}, w_{21}^{[2]}, w_{31}^{[2]}, w_{41}^{[2]}, w_{51}^{[2]}]$ with a total of 36 parameters.

12.5 Learning Algorithms

The realization of a Fuzzy Neural Network as a pattern classification and recognition system requires the determination of the appropriate fuzzy inference rules from given training data (i.e., to determine the number of inference rules, the membership functions of the fuzzy subsets in the antecedent part, and the outputs in the conclusion part in each rule).

The learning criterion adopted is to minimize “maximum output error of all the training patterns” $\leq E_t$ where E_t denotes error tolerance. The steps of the learning algorithm is listed in Table 12.5.1.

Table 12.5.1 Steps of learning algorithm.

Step	Description
1	Input training patterns and compute output errors.
2	Select prototypes from training patterns when maximum output error $> E_t$.
3	Establish Min-FNs in the layer 2 to represent the selected prototypes.
4	Adjust network parameters according to all the selected prototypes.

An example of learning a fuzzy inference rule is given: If a training sample $\{X_1 = 0, X_2 = 1, Y = 0\}$ is selected as a prototype, then an inference rule will be established as

IF X_1 is near 0 and X_2 is near 1, THEN Y is 0.

The membership functions for fuzzy terms “near 0” and “near 1” will be adjusted during the learning according to all the selected prototypes.

12.6 Min-Competitive Fuzzy neural Networks for Pattern Recognition

12.6.1 Unsupervised Min-Competitive Fuzzy Neural Network (UMCFNN)

The UMCFNN can be used to build a four-layer feedforward network as shown in Figure 12.6.1 for applications in 2-dimensional invariant pattern recognition problems.

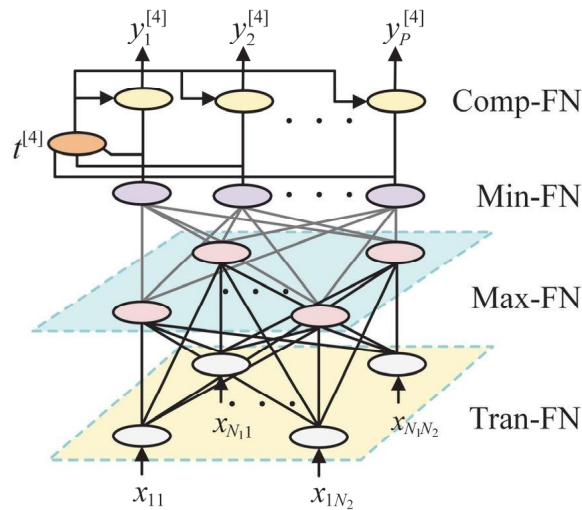


Figure 12.6.1 Unsupervised Min-Competitive Fuzzy Neural Network for pattern recognition.

Tran-FNs and Max-FNs are used to form two fuzzification layers to extract important features of an input pattern. The fuzzification allows

the UMCFNN to focus on the important features of the input patterns so that the networks have the ability to recognize noisy patterns.

Min-FNs and Comp-FNs are used to form the backbone of a Min-Comp FNN.

In the UMCFNN, the first layer is the input layer that accepts patterns into the network. Tran-FNs are used in this layer.

Each Tran-FN in the first layer corresponds to one pixel x_{ij} of an input pattern. The FNs are displayed and indexed in 2-dimension and the number of Tran-FNs in this layer is equal to the total number of pixels $N_1 \times N_2$ of an input pattern.

Let P_{vmax} be the maximum pixel value among all input patterns, the (i, j) th Tran-FN in the first layer performs

$$s_{ij}^{[1]} = z_{ij}^{[1]} = x_{ij} \quad \text{for } i = 1 \text{ to } N_1 \text{ and } j = 1 \text{ to } N_2 \quad (12.6.1)$$

$$y_{ij}^{[1]} = \frac{s_{ij}^{[1]}}{P_{vmax}} \quad \text{for } i = 1 \text{ to } N_1 \text{ and } j = 1 \text{ to } N_2 \quad (12.6.2)$$

The second layer is also displayed in 2-dimensional and consists of $N_1 \times N_2$ Max-FNs which serves to fuzzify input patterns through a weight function $w^{[1]}(m, n)$. The state of the (p, q) th Max-FN in the second layer performs

$$s_{pq}^{[2]} = \max_{i=p-m_1}^{p+m_1} \left\{ \max_{j=q-n_1}^{q+n_1} \left[w^{[1]}((p-i), (q-j)) y_{ij}^{[1]} \right] \right\} \quad (12.6.3)$$

for $p = 1 \text{ to } N_1$ and $q = 1 \text{ to } N_2$, $m_1 < N_1$ and $n_1 < N_2$

The first two layers serve to fuzzify and extract features from an input pattern which are based on a Gaussian fuzzification function defined in (12.6.4) as plotted in Figure 12.6.2.

The weight $w^{[1]}(p-i, q-j)$ connects the (i, j) th Tran-FN in the first layer to the (p, q) th Max-FN in the second-layer which is defined by

$$w^{[1]}(m, n) = \exp[-\beta^2(m^2 + n^2)] \quad (12.6.4)$$

for $m = -m_1$ to $m_1, n = -n_1$ to n_1

The choice of m_1 and n_1 should cover the expected maximum shift ranges of a pixel respectively along vertical and horizontal directions.

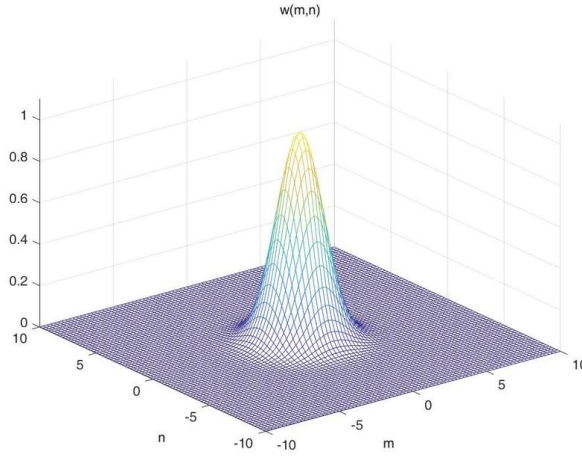


Figure 12.6.2 Fuzzification function ($\beta = 0.3, m_1 = 10, n_1 = 10$).

With the fuzzification function, each FN focuses on one center pixel while covering the surrounding pixels as a decreasing weight function from the center pixel with a width controlled by the value of β .

In the second layer, $y_{pqm}^{[2]}$ is the m th output of the (p, q) th Max-FN that is to be connected to the m th Min-FN in the third layer.

Each Max-FN in the second layer has M different outputs corresponding to the number of Min-FNs in the third layer.

The operation of the (p, q) th Max-FN is

$$y_{pqm}^{[2]} = g_{pqm}(s_{pq}^{[2]}) \quad (12.6.5)$$

for $p = 1$ to $N_1, q = 1$ to $N_2, m = 1$ to M

In the second layer, the output functions $g_{pqm}(s_{pq}^{[2]})$ are the membership functions in the IF parts of the inference rules.

For simplicity, isosceles triangle with height equal to 1 and slope equal to α as shown in Figure 12.6.3 is adopted as the output function of a Max-FN in the second layer such that

$$g_{pqm}(s_{pq}^{[2]}) = \begin{cases} 1 - \alpha |s_{pq}^{[2]} - \theta_{pqm}| & \text{if } 1 \geq 1 - \alpha |s_{pq}^{[2]} - \theta_{pqm}| \geq 0 \\ 0 & \text{if otherwise} \end{cases} \quad (12.6.6)$$

for $\alpha \geq 0, p = 1 \text{ to } N_1, q = 1 \text{ to } N_2, m = 1 \text{ to } M$.

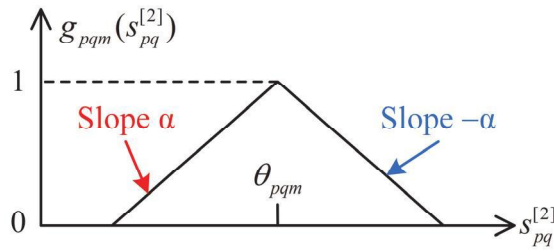


Figure 12.6.3 Output function of Max-FN in second layer of UMCNN.

The output function $g_{pqm}(s_{pq}^{[2]})$ for every set of p, q , and m and its slope α and base center location θ_{pqm} are to be determined by the learning algorithm.

Min-FNs are used in the third layer. Each Min-FN in the third layer represents one learned pattern. Hence, the number of Min-FNs in the third layer, M , is determined only after the learning procedure is finished.

Let $s_m^{[3]}$ represents the state of the m th Min-FN in the third layer; the output of the m th Min-FN in the third layer performs

$$y_m^{[3]} = s_m^{[3]} = \min_{p=1 \text{ to } N_1} \left[\min_{q=1 \text{ to } N_2} (y_{pqm}^{[2]}) \right] \text{ for } m = 1 \text{ to } M \quad (12.6.7)$$

The fourth layer is the output layer. Comp-FNs are used in this layer, each Comp-FN is connected to one Min-FN corresponding to one of the M learned patterns to provide nonfuzzy outputs.

If an input pattern is most similar to the m th learned pattern, then the output of the m th Comp-FN in the fourth layer is 1 while other outputs are 0. The number of Comp-FNs in the output layer is M .

Let $t^{[4]}$ denote the activation threshold of all the Comp-FNs in the fourth layer; the operations of the m th Comp-FN in the fourth layer are

$$s_m^{[4]} = z_m^{[4]} = y_m^{[3]} \quad \text{for } m = 1 \text{ to } M \quad (12.6.8)$$

$$y_m^{[4]} = g[s_m^{[4]} - t^{[4]}] = \begin{cases} 1 & \text{if } s_m^{[4]} = t^{[4]} \\ 0 & \text{if } s_m^{[4]} < t^{[4]} \end{cases} \quad \text{for } m = 1 \text{ to } M \quad (12.6.9)$$

$$t^{[4]} = \max_{m=1 \text{ to } M} (y_m^{[3]}) \quad \text{for } m = 1 \text{ to } M \quad (12.6.10)$$

Note that in the UMCFNN, the number of classes is unknown before training and can turn out to be less than the number of training patterns after training.

All the parameters of the UMCFNN can be grouped as a parameter vector $\mathbf{c}_{uc} = [\alpha, \beta, \theta_{pqm} \text{ for } p = 1 \text{ to } N_1, q = 1 \text{ to } N_2, \text{ and } m = 1 \text{ to } M]$.

For $N_1 = 2, N_2 = 2$, and $M = 3$, the parameter vector to be determined is $\mathbf{c}_{uc} = [\alpha, \beta, \theta_{111}, \theta_{121}, \theta_{211}, \theta_{221}, \theta_{112}, \theta_{122}, \theta_{212}, \theta_{222}, \theta_{113}, \theta_{123}, \theta_{213}, \theta_{223}]$. The UMCFNN learning algorithm is listed in Table 12.6.1.

Table 12.6.1 UMCFNN for pattern recognition learning algorithm.

Parameters to be determined		Initial parameters to be set	
Parameters α, θ_{pqm} ($p = 1$ to N_1 , $q = 1$ to $N_2, m = 1$ to M) for Max-FN output membership functions;		Output error limit, E_t ;	
		Total number of training patterns, K ;	
Fuzzification function parameter β ;		Input pattern $\mathbf{X}_k = \{x_{ijk}\}$ ($i = 1$ to $N_1, j = 1$ to $N_2, k = 1$ to K).	
M number of FNs in layer 3 and layer 4.			
	BEGIN		

1	Set E_t ; Create $N_1 \times N_2$ Tran-FNs in layer 1 & $N_1 \times N_2$ Max-FNs in layer 2; Choose appropriate values α for Max-FNs output membership functions & β for fuzzification function connecting layers 1 & 2; Create 1st Min-FN in layer 3 for training sample 1; Set the center point of the output triangular function of the (p, q) th Max-FN as $\theta_{pq1} = s_{pq1}^{[2]} \ (p = 1 \text{ to } N_1, q = 1 \text{ to } N_2)$; Initialize $m = 1, k = 2$;
2	DO UNTIL $(k \leq K)$ Input k th training pattern $\mathbf{X}_k$ & compute output errors: $e = 1 - \max_{j=1 \text{ to } m} y_{jk}^{[3]}$; IF $(e > E_t)$, THEN Create a new Min-FN in layer 3 and set $m = m + 1$; Set the center point of the output triangular function of the (p, q) th Max-FN as $\theta_{pqm} = s_{pqm}^{[2]} \ (p = 1 \text{ to } N_1, q = 1 \text{ to } N_2)$; END IF Set $k = k + 1$; END DO UNTIL
	END

12.6.2 Supervised Min-Competitive Fuzzy Neural Network (SMCFNN)

There are applications in which the number of output classes is known and therefore a supervised version of the MCFNN is required.

This results in the introduction of the four-layer Supervised MCFNN (SMCFNN) as shown in Figure 12.6.4 which can be used for pattern recognition. The first three layers of the UMCNN and those of the SMCFNN are the same.

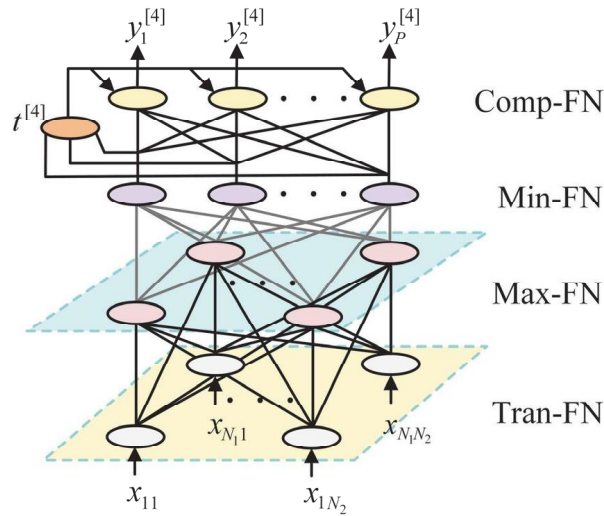


Figure 12.6.4 Supervised Min-Competitive Fuzzy Neural Network.

The differences between the UMCFNN and the SMCFNN can be summarized as follows:

- The UMCFNN is an unsupervised learning network while the SMCFNN is a supervised learning network and consequently their learning algorithms are different.
- For the UMCFNN, the number of Min-FNs and the number of Comp-FNs correspond to the same number of M learned patterns, and M is determined by the learning algorithm.
- For the SMCFNN, M is the number of Min-FNs which corresponds to the number of fuzzy inference rules (each contains the information of a prototype) to be determined by the learning algorithm and P is the number of Comp-FNs which is determined by the number of output classes.

The operations of the SMCFNN are the same as those of the UMCFNN defined (12.6.1)-(12.6.7) for the first three layers.

In the second-layer, the output functions $g_{pqm}(s_{pq}^{[2]})$ are the membership functions in the IF parts of the inference rules.

For simplicity, an isosceles triangle with height equal to 1 and a slope equal to α as shown in Figure 12.6.5 is adopted as the output function of a Max-FN in the second layer such that

$$y_{pqm}^{[2]} = g_{pqm}(s_{pq}^{[2]}) = \begin{cases} 1 - \alpha |s_{pq}^{[2]} - \theta_{pqm}| & \text{if } 1 - \alpha |s_{pq}^{[2]} - \theta_{pqm}| \geq 0 \\ 0 & \text{if otherwise} \end{cases} \quad (12.6.11)$$

for $\alpha \geq 0, p = 1$ to $N_1, q = 1$ to $N_2, m = 1$ to M .

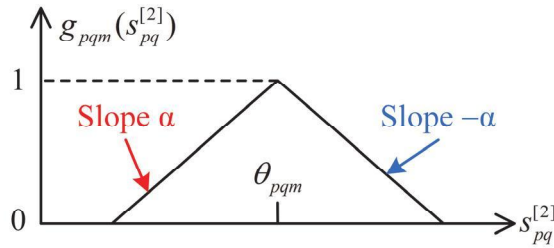


Figure 12.6.5 Output function of Max-FN in second layer of SMCFNN.

The output function $g_{pqm}(s_{pq}^{[2]})$ for every set of p, q , and m and the values of its slope α and base center location θ_{pqm} are to be determined by the learning algorithm.

Min-FNs are used in the third layer. Each Min-FN in the third layer represents one learned pattern. Hence, the number of Min-FNs in the third layer, M , is determined only after the learning procedure is completed. The output of the m th Min-FN in the third layer performs

$$y_m^{[3]} = s_m^{[3]} = \min_{p=1 \text{ to } N_1} \left[\min_{q=1 \text{ to } N_2} (y_{pqm}^{[2]}) \right] \text{ for } m = 1 \text{ to } M \quad (12.6.12)$$

In (12.6.12), $s_m^{[3]}$ represents the state of the m th Min-FN in the third layer.

The fourth layer of the SMCFFNN is the output layer. Comp-FNs are used in the fourth layer, one for each of the P output classes which is usually different from the number of prototypes M in the third layer.

The fourth layer computes the membership values of an input pattern to all the classes in the following manner,

$$y_r^{[4]} = s_r^{[4]} = w_{jr}^{[3]} \text{ if } y_j^{[3]} = t^{[4]} \text{ for } r = 1 \text{ to } P \quad (12.6.13)$$

$$t^{[4]} = \max_{m=1 \text{ to } M} (y_j^{[3]}) \text{ for } j = 1 \text{ to } M \quad (12.6.14)$$

$w_{jr}^{[3]}$ is the connection weight from the J th Min-FN in the third layer to the r th Comp-FN in the fourth layer for $r = 1$ to P . It contains the information of the J th prototype belonging to the r th class and it will be determined in the learning procedure. $t^{[4]}$ is the activation threshold of all the Comp-FNs in the fourth layer.

P is the number of classes, for 36 alpha-numeral pattern recognition problem, $P = 36$. For an input pattern k ($k = 1$ to K), the classified output $y_r^{[4]}$ from the winning Min-FN m is $y_r^{[4]} = w_{mr}^{[3]} d_{rk}$ ($r = 1$ to P) contains “ $P - 1$ ” zeros and “1” one (“1” representing the winning output).

All the parameters of the SMCFFNN can be grouped as a parameter vector $\mathbf{c}_{sc} = [\alpha, \beta, \theta_{pqm} \text{ for } p = 1 \text{ to } N_1, q = 1 \text{ to } N_2, \text{ and } m = 1 \text{ to } M]$.

For $N_1 = 2, N_2 = 2, M = 3$, and $P = 2$, $\mathbf{c}_{sc} = [\alpha, \beta, \theta_{111}, \theta_{121}, \theta_{211}, \theta_{221}, \theta_{112}, \theta_{122}, \theta_{212}, \theta_{222}, \theta_{113}, \theta_{123}, \theta_{213}, \theta_{223}, w_{11}^{[3]}, w_{21}^{[3]}, w_{31}^{[3]}, w_{12}^{[3]}, w_{22}^{[3]}, w_{32}^{[3]}]$. The SMCFFNN learning algorithm is listed in Table 12.6.2.

Table 12.6.2 SMCFNN for pattern recognition learning algorithm.

Parameters to be determined		Initial parameters to be set	
Parameters α, θ_{pqm} ($p = 1$ to $N_1, q = 1$ to $N_2, m = 1$ to M) for Max-FN output membership functions;		Output error limit, E_t .	
		Total number of training patterns, K ;	
Fuzzification function parameter β ;		Input pattern $\mathbf{X}_k = \{x_{ijk}\}$ ($i = 1$ to $N_1, j = 1$ to $N_2, k = 1$ to K);	
M number of Min-FNs in layer 3;			
Weights connecting layers 3 & 4, $w_{jr}^{[3]}$ ($j = 1$ to $M, r = 1$ to P).		Desired outputs d_{rk} ($r = 1$ to $P, k = 1$ to K).	
	BEGIN		
1	Set E_t ; Create $N_1 \times N_2$ Tran-FNs in layer 1, $N_1 \times N_2$ Max-FNs in layer 2 & P Comp-FNs in layer 4; Choose appropriate values α for Max-FNs output membership functions & β for fuzzification function connecting layers 1 & 2;		
2	DO UNTIL (largest output error $\sigma \leq E_t$); Create 1st Min-FNs in layer 3 for $k = 1$; Set the center point of the output triangular function & the output weight of the (p, q) th Max-FN as $\theta_{pq1} = s_{pq1}^{[2]}$ ($i = 1$ to $N_1, j = 1$ to N_2); $w_{1r}^{[3]} = d_{r1}$ ($r = 1$ to P); Initialize $m = 1, k = 2$; DO UNTIL ($k \leq K$) Input k th training pattern $\mathbf{X}_k$ & compute output errors: $e_r = y_r^{[4]} - d_{rk}$ ($r = 1$ to P); IF (there is one $ e_r > E_t$ ($r = 1$ to P), THEN Create a new Min-FN in layer 3 and set $m = m + 1$; Set the center point of the output triangular function & the output weight of the (p, q) th Max-FN as		

	$\theta_{pqm} = s_{pqm}^{[2]} \ (p = 1 \text{ to } N_1, q = 1 \text{ to } N_2);$ $w_{mr}^{[3]} = d_{rk} \ (r = 1 \text{ to } P);$ END IF Set $k = k + 1;$ END DO UNTIL Initialize $k = 1;$ DO UNTIL $(k \leq K)$ Input k th training pattern $\mathbf{X}_k$ & compute maximum output error: $\sigma_k = \max_{r=1 \text{ to } P} e_r ;$ Set $k = k + 1;$ END DO UNTIL $\sigma = \max_{k=1 \text{ to } K} \sigma_k ;$ IF $(\sigma > E_t),$ THEN set $\alpha = 0.9\alpha; \beta = 0.9\beta;$ END IF END DO UNTIL
	END

12.6.3 Application of MCFNN to pattern recognition

Figure 12.6.6 shows a scheme on the core operations of a pattern recognition system.

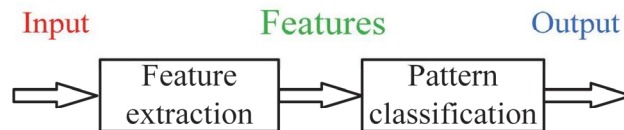


Figure 12.6.6 Core operations of pattern recognition.

There are two major steps involved in pattern recognition: feature extraction and pattern classification.

Traditional pattern recognition systems are sensitive to shifts and deformations of input patterns. Normalization of the position and the

shape of patterns are necessary which require a simple and effective normalization method.

Design Example 12.1 (Pattern recognition using FNNs)

In this example, the four-layer MCFNNs are used in which their first two layers are used to extract the important features of shifted and distorted patterns and then its last two layers are used to recognize the patterns.

To demonstrate the effectiveness of the UCMFNN for pattern recognition, 36 universal font patterns as shown in Figure E12.1.1 are used as training patterns.

Two training sets are used, the training set 1 consists of 36 universal font patterns; and the training set 2 has 3x36 font patterns consisting of 36 universal font patterns, 36 universal font patterns shifted upward 1 pixel, and 36 universal font patterns as shifted downward 1 pixel.



Figure E12.1.1 36 universal font patterns for training.

Bold patterns, Italic patterns, non-standard patterns, shifted patterns, and 10 kinds of distorted patterns as shown in Figures E12.1.2 to E12.1.6 can be used as testing patterns.



Figure E12.1.2 36 bold font patterns for testing.



Figure E12.1.3 36 Italic font patterns for testing.



Figure E12.1.4 36 non-standard font patterns for testing.

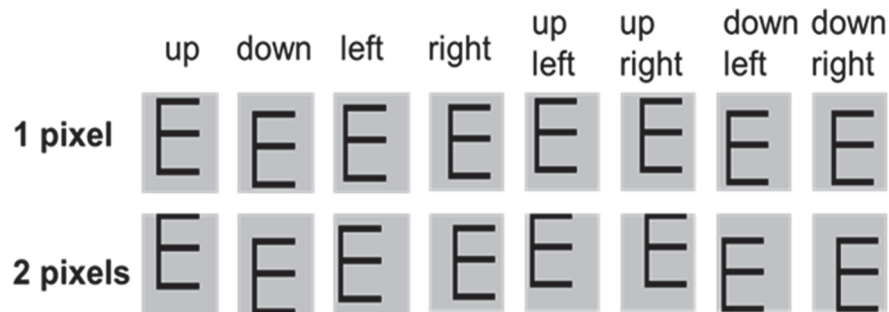


Figure E12.1.5 Shifted patterns of exemplar E.

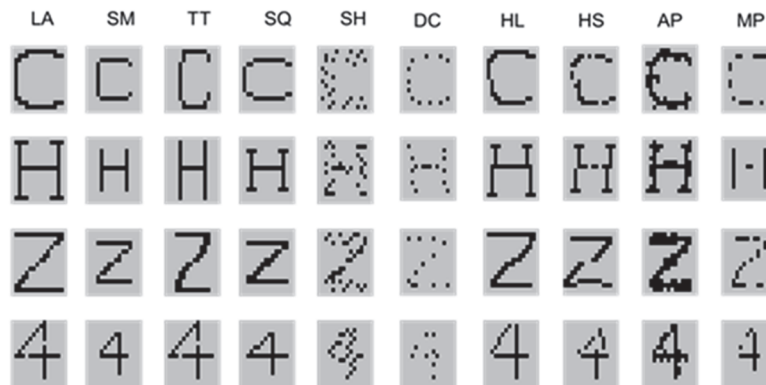


Figure E12.1.6 Distorted patterns for testing.

Solution

Table E12.1.1 summarizes the results of using 36 universal font patterns for training; and bold font patterns, Italic font patterns, non-standard font patterns, shifted patterns, and 10 kinds of distorted patterns as testing patterns.

Pattern recognition results of the four-layer MCFNNs are also compared to those of the Hamming network.

In practice, statistical data of the 36 exemplars (the number of black pixels of each pattern) can be also used to improve the recognition rates. Table E12.1.2 summarizes the results of training using the training set 1, the training set 2, and the statistical data when tested by patterns

shifted by 1 to 2 pixels. In general, more training patterns help to improve the recognition rates.

Table E12.1.1 Recognition rates when trained by training set 1.

	Bold	Italic	Non-standard	Distorted	Shift-1	Shift-2
UMCFNN	100	100	100	99.7	99.7	85.4
SMCFNN	100	100	100	99.7	99.7	85.4
Hamming	100	63.9	75.0	41.4	13.2	12.2

Table E12.1.2 Recognition rates when tested by shifted patterns.

		UMCFNN	SMCFNN	Hamming
Training set 1	Shift-1	99.7	99.7	13.2
	Shift-2	85.4	85.4	12.2
Training set 2	Shift-1	100	100	34.4
	Shift-2	93.8	94.1	13.9
Statistical data	Shift-1	100	100	42.0
	Shift-2	95.5	95.5	37.8

12.7 Fuzzy Neural Networks as Classifiers

The MSFNN and the MMFNN can be used as non-fuzzy and fuzzy classifiers with learning algorithms as listed respectively in Tables 12.7.1 and 12.7.2.

The input layer of each of the MSFNN and the MMFNN accepts and fuzzifies input patterns.

Fuzzification is performed by choosing triangular membership functions as the output functions of the Tran-FNs in the input layer.

These triangular functions give the membership values representing the features of the input x_i for $i = 1, 2, \dots, N$ with respect to the fuzzy subsets A_{ij} for $i = 1, 2, \dots, N$; $j = 1, 2, \dots, M$ in the inference rules.

The second layer computes the outputs of all the fuzzy inference rules in term of the distances or the similarities of the input patterns to the prototypes. Fuzzy similarities are used to compute the distances between patterns.

The third layers computes the membership values of an input pattern to all the output classes to determine the outputs according to the fuzzy inference rules and the corresponding fuzzy inference model.

Different fuzzy inference models can be employed in which Sum-FNs or Max-FNs can be used in the third layer.

Table 12.7.1 MSFNN classifier learning algorithm.

Parameters to be determined		Initial parameters to be set	
Parameters $\alpha_{ij1}, \alpha_{ij2}, \theta_{ij}$ of weight functions $w_{ij}^{[1]}(x_i)$ ($i = 1, 2, 3, \dots, N; j = 1, 2, 3, \dots, M$);		Output error limit, E_t ;	
		Total number of training patterns, K ;	
Number of Min-FNs in layer 2, M ;		Input pattern $\mathbf{X}_k = \{x_{ik}\}$ ($i = 1$ to $N, k = 1$ to K);	
Weights connecting layers 2 & 3, $w_{jr}^{[2]}$ ($j = 1$ to $M, r = 1$ to P).		Desired output d_{kr} ($k = 1$ to $K, r = 1$ to P);	
	BEGIN		
1	Set E_t ; Create N Tran-FNs in layer 1 & P Max-FNs in layer 3;		
2	Create 1 st Min-FNs in layer 2 for training sample 1; Adjust weight functions $w_{i1}^{[1]}(.)$ ($i = 1$ to N); Adjust weights $w_{1r}^{[2]}$ ($r = 1$ to P); Set $m = 1, k = 2$;		
3	DO UNTIL ($k \leq K$) Input k^{th} training pattern $\mathbf{X}_k$ & compute output errors: $e_r = y_r^{[3]} - d_{kr}$ ($r = 1$ to P); IF (there is one $ e_r > E_t$ ($r = 1$ to P), THEN Add a new Min-FN in layer 2 and set $m = m + 1$;		

	Adjust weight functions $w_{im}^{[1]}(.)$ ($i = 1$ to N); Adjust weights $w_{mr}^{[2]}$ ($r = 1$ to P); set $k = k + 1$; END DO UNTIL
4	Let $\sigma = \max_{k=1 \text{ to } K} (\max_{r=1 \text{ to } P} (e_r))$; DO UNTIL ($\sigma < E_t$) Initialize $k = 1$; DO UNTIL ($k \leq K$) Input k^{th} training pattern $\mathbf{X}_k$ & compute output error: $e_r = y_r^{[3]} - d_{kr}$ ($r = 1$ to P); IF (there is one $ e_r > E_t$ ($r = 1$ to P), THEN Add a new Min-FN in layer 2 and set $m = m + 1$; Adjust weight functions $w_{ij}^{[1]}(.)$ ($i = 1$ to $N, j = 1$ to m); Adjust weights $w_{jr}^{[2]}$ ($j = 1$ to $m, r = 1$ to P); set $k = k + 1$; END DO UNTIL END DO UNTIL
	END

Table 12.7.2 MMFNN classifier learning algorithm.

Parameters to be determined	Initial parameters to be set
Parameters $t_{ij1}, t_{ij2}, \alpha, \theta_{ij}$ of weight functions $w_{ij}^{[1]}(x_i)$ ($i = 1, 2, 3, \dots, N, j = 1, 2, 3, \dots, M$);	Output error limit, E_t ;
	Total number of training patterns, K ;
Number of Min-FNs in layer 2, M ;	Input pattern $\mathbf{X}_k = \{x_{ik}\}$ ($i = 1$ to $N, k = 1$ to K);
Weights connecting layers 2 & 3, $w_{jr}^{[2]}$ ($j = 1$ to $M, r = 1$ to P).	Desired output d_{kr} ($k = 1$ to $K, r = 1$ to P);
	Largest distance D between any 2 training patterns.

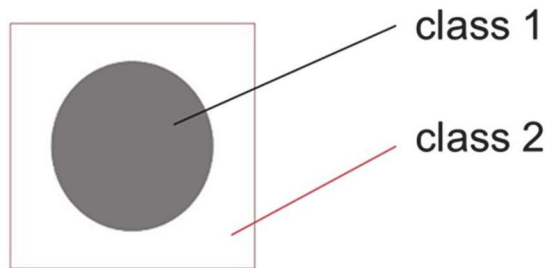
	BEGIN
1	Set E_t & $\alpha = 1/D$; Create N Tran-FNs in layer 1 & P Max-FNs in layer 3;
2	Create 1 st Min-FNs in layer 2 for training sample 1; Adjust weight functions $w_{i1}^{[1]}(.)$ ($i = 1$ to N) & weights $w_{1r}^{[2]}$ ($r = 1$ to P); Set $m = 1, k = 2$;
3	DO UNTIL ($k \leq K$) Input k^{th} training pattern X_k & compute output errors: $e_r = y_r^{[3]} - d_{kr}$ ($r = 1$ to P); IF (there is one $e_r > E_t$ ($r = 1$ to P)), THEN adjust t_{ij1} & t_{ij2} ($i = 1$ to $N, j = 1$ to m) until $e_r \leq E_t$ ($r = 1$ to P); IF (there is one $e_r < -E_t$ ($r = 1$ to P)), THEN add a new Min-FN in layer 2 and set $m = m + 1$; adjust weight functions $w_{im}^{[1]}(.)$ ($i = 1$ to N) and weights $w_{mr}^{[2]}$ ($r = 1$ to P); set $k = k + 1$; END DO UNTIL
4	Let $\sigma = \max_{k=1 \text{ to } K} (\max_{r=1 \text{ to } P} (e_r))$; DO UNTIL ($\sigma < E_t$) Initialize $k = 1$; DO UNTIL ($k \leq K$) Input k^{th} training pattern X_k & compute output error: $e_r = y_r^{[3]} - d_{kr}$ ($r = 1$ to P); IF (there is one $e_r > E_t$ ($r = 1$ to P)), THEN adjust t_{ij1} & t_{ij2} ($i = 1$ to $N, j = 1$ to m) until $e_r \leq E_t$ ($r = 1$ to P); IF (there is one $e_r < -E_t$ ($r = 1$ to P)), THEN add a new Min-FN in layer 2 and set $m = m + 1$; adjust weight functions $w_{im}^{[1]}(.)$ ($i = 1$ to N) and weights $w_{mr}^{[2]}$ ($r = 1$ to P);

	set $k = k + 1$; END DO UNTIL END DO UNTIL
	END

12.7.1 Application to non-fuzzy classification

In the circle-in-the-square problem shown in Figure 12.7.1, there are two output classes in the square, inside the circle (with output 1) and outside the circle (with output 0). The labels in these two sets are discrete and non-fuzzy.

The area inside the circle equals to the area outside the circle. This is a typical classification problem.



Area 1 = Area 2

Figure 12.7.1 Circle-in-the-square classification problem.

Design Example 12.2 (Circle-in-the-square problem using FNNs)
Matlab

This example serves to test the non-fuzzy classification ability of the MSFNN and MMFNN classifiers. The training set contains 100 and 1000 points randomly distributed in the square; and the testing data contains 10000 points regularly distributed in the square.

M denotes the number of Min-FNs in the second (or hidden) layer which represents the number of distinct features of all the input patterns for

forming the output class decision. The larger the value of M , the smoother will be the non-fuzzy boundaries.

Solution

The classification or testing results obtained by the MSFNN and the MMFNN for the two training sets in the circle-in-the-square problem are shown in Figures E12.2.1-E12.2.2.

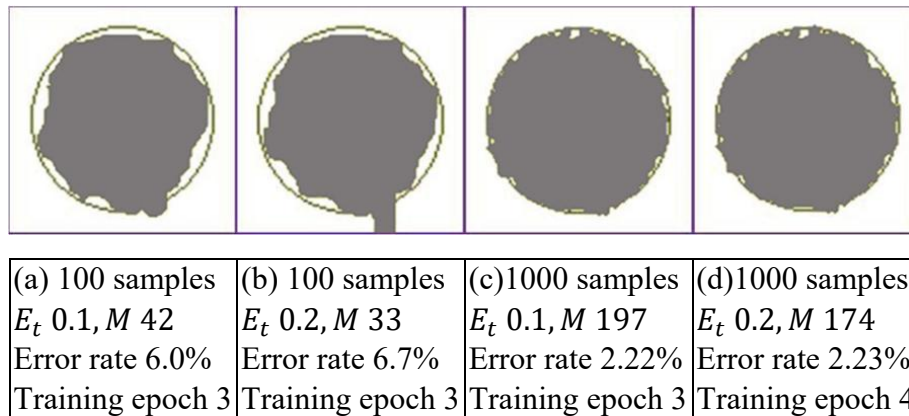


Figure E12.2.1 Classification results obtained by MSFNN.

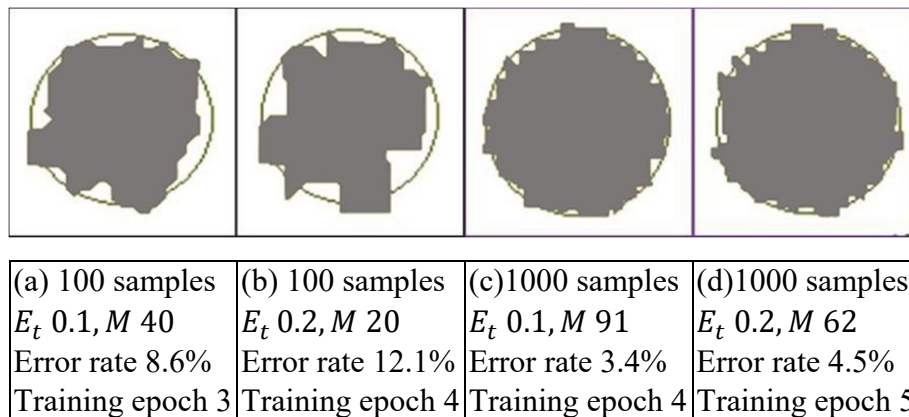


Figure E12.2.2 Classification results obtained by MMFNN.

12.7.2 Application to fuzzy classification

In a fuzzy classification problem, the membership values of the output fuzzy classes are continuous and depend on the inputs.

Design Example 12.3 (Fuzzy XOR problem using FNNs) Matlab

In the fuzzy XOR problem, there are two fuzzy classes Y^1 and Y^2 . The membership values of these output fuzzy classes are continuous depend on the inputs X_1 and X_2 .

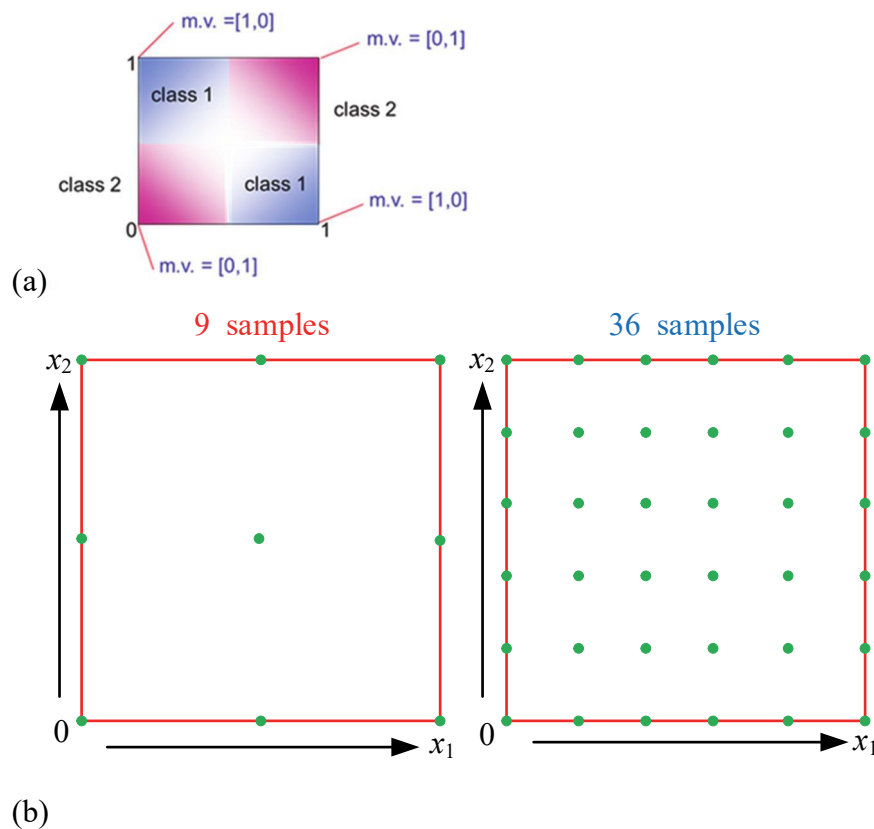


Figure E12.3.1 Fuzzy XOR classification problem and two training sets: (a) Fuzzy XOR problem (m.v.: membership value); (b) Training set 1 (9 samples), Training set 2 (36 samples).

Two sets of training data shown in Figure E12.3.1 (a-b) are used in the fuzzy XOR problem. The first set contains 9 samples and the second set

contains 36 samples and their respective membership values are listed in Tables E12.3.1 and E12.3.2.

The training samples are used to demonstrate the abilities of the MSFNN and the MMFNN to classify fuzzy patterns and to extract fuzzy inference rules from training samples.

Table E12.3.1 Fuzzy XOR problem (9 data-pairs).

Class	Input 1	x_1		
Input 2	Y^1Y^2	0.0	0.5	1.0
x_2	0.0	0, 1	0, 0	1, 0
	0.5	0, 0	0, 0	0, 0
	1.0	1, 0	0, 0	0, 1

Table E12.3.2 Fuzzy XOR problem (36 data-pairs).

Class	Input 1	x_1					
Input 2	Y^1Y^2	0.0	0.2	0.4	0.6	0.8	1.0
x_2	0.0	0,1.00	0,0.75	0,0.25	0.25,0	0.75,0	1.00,0
	0.2	0,0.75	0,0.75	0,0.25	0.25,0	0.75,0	0.75,0
	0.4	0,0.25	0,0.25	0,0.25	0.25,0	0.25,0	0.25,0
	0.6	0.25,0	0.25,0	0.25,0	0,0.25	0,0.25	0,0.25
	0.8	0.75,0	0.75,0	0.25,0	0,0.25	0,0.75	0,0.75
	1.0	1.00,0	0.75,0	0.25,0	0,0.25	0,0.75	0,1.00

Solution

Fuzzy inference rules are established during the learning procedure. The membership functions of the inference rules are automatically determined during the learning.

The fuzzy inference rules extracted by the MSFNN when trained by the 9-sample set are listed in Table E12.3.3. The corresponding learnt membership functions are shown in Figure E12.3.2.

The fuzzy boundaries obtained by the MSFNN for 9 samples and 36 samples of training data are shown respectively in Figures E12.3.3-E12.3.4.

M denotes the number of Min-FNs in the second layer which represents the number of distinct features of all the input patterns for making the output class decision. In general, the larger the M value, the smoother are the fuzzy boundaries.

Table E12.3.3 Fuzzy inference rules (from 9 samples) learnt by MSFMM.

Rule	IF	THEN
1	IF X_1 is small & X_2 is small	THEN Y^1 is 0 & Y^2 is 1
2	IF X_1 is small & X_2 is medium	THEN Y^1 is 0 & Y^2 is 0
3	IF X_1 is small & X_2 is large	THEN Y^1 is 1 & Y^2 is 0
4	IF X_1 is medium & X_2 is small	THEN Y^1 is 0 & Y^2 is 0
5	IF X_1 is medium & X_2 is large	THEN Y^1 is 0 & Y^2 is 0
6	IF X_1 is large & X_2 is small	THEN Y^1 is 1 & Y^2 is 0
7	IF X_1 is large & X_2 is medium	THEN Y^1 is 0 & Y^2 is 0
8	IF X_1 is large & X_2 is large	THEN Y^1 is 0 & Y^2 is 1

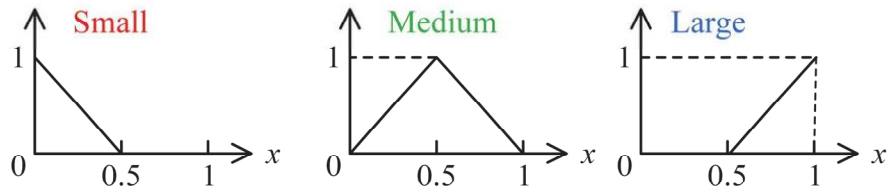
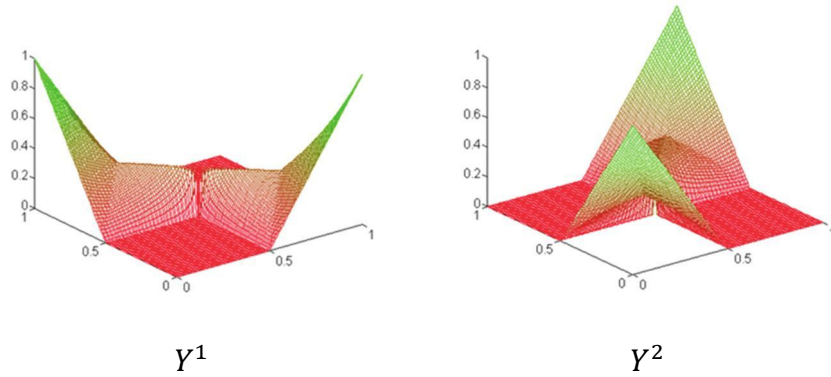
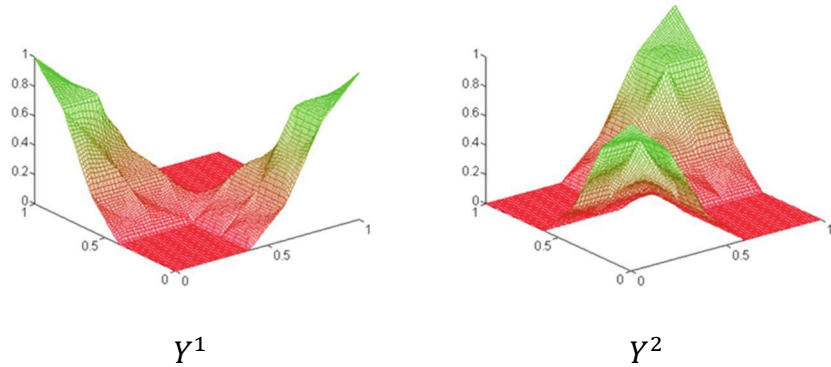


Figure E12.3.2 Membership functions learnt by MSFNN.



$M = 8, E_t = 0.1, 9$ training samples

Figure E12.3.3 Fuzzy boundaries obtained by MSFNN (9 samples).



$M = 31, E_t = 0.1, 36$ training samples

Figure E12.3.4 Fuzzy boundaries obtained by MSFNN (36 samples).

The fuzzy inference rules extracted by the MMFNN when trained by the 9-sample set are listed in Table E12.3.4. The corresponding learnt membership functions are shown in Figure E12.3.5.

The fuzzy boundaries obtained by the MMFNN for 9 samples and 36 samples of training data are shown respectively in Figures E12.3.6-E12.3.7.

Table E12.3.4 Fuzzy inference rules (from 9 samples) learnt by MMFNN.

Rule	IF	THEN
1	IF X_1 is small & X_2 is small	THEN Y^1 is 0 & Y^2 is 1
2	IF X_1 is small & X_2 is large	THEN Y^1 is 1 & Y^2 is 0
3	IF X_1 is large & X_2 is small	THEN Y^1 is 1 & Y^2 is 0
4	IF X_1 is large & X_2 is large	THEN Y^1 is 0 & Y^2 is 1

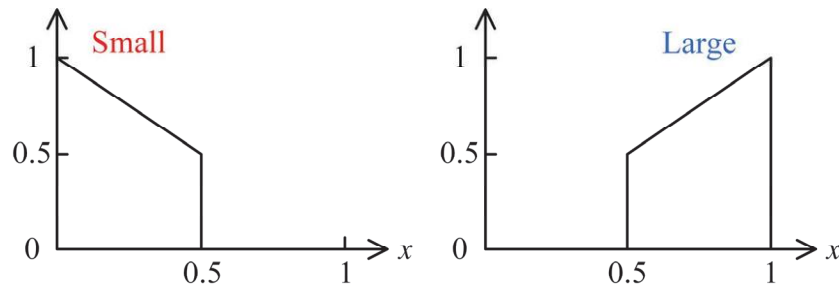
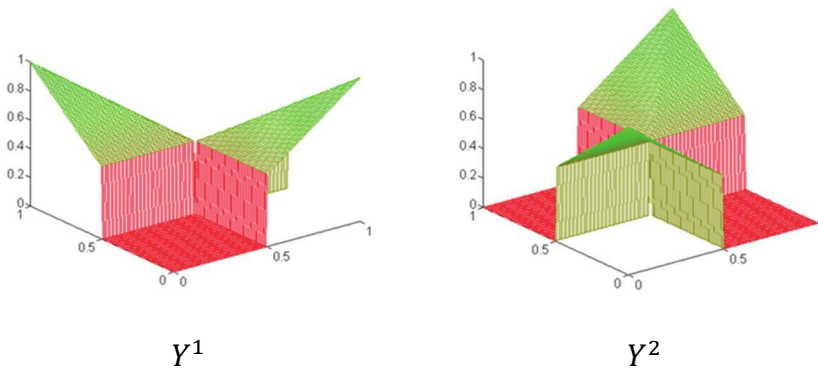
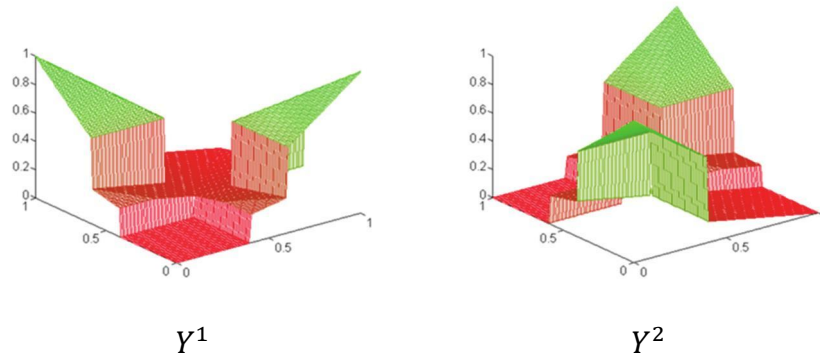


Figure E12.3.5 Membership functions learnt by MMFNN.



$M = 4, E_t = 0.1, 9$ training samples

Figure E12.3.6 Fuzzy boundaries obtained by MMFNN (9 samples).



$M = 14, E_t = 0.1, 36$ training samples

Figure E12.3.7 Fuzzy boundaries obtained by MMFNN (36 samples).

12.8 Summary

There are three types of fuzzy neural networks [Cai9305], [Cai9410], [Cai9502], [Cai9505], [Cai9806], namely the Min-Competitive Fuzzy Neural Network (MCFNN), the Min-Max Fuzzy Neural Network (MMFNN), and the Min-Sum Fuzzy Neural Network (MSFNN) which can be designed for pattern classification, recognition, interpolation, and other applications.

The MCFNN can be combined with the Maximum Fuzzy Neurons (Max-FNs) and the Input Fuzzy Neurons (Input-FNs) to form a fuzzy neural network for pattern recognition. The MCFNN, the MMFNN, and the MSFNN can be used for non-fuzzy and fuzzy pattern classification.

Efficient self-structure learning algorithms can be used for training. After being trained by labeled data, each of the fuzzy neural networks can find the fuzzy and hard partitions between the classes.

Each fuzzy neural network can build decision boundaries by creating subsets of the pattern space.

Fuzzy neural networks are model-free estimators and do not make assumptions of how outputs depend on inputs.

Instead, each of them adjust itself to a given training set by learning algorithms and decide the boundaries of classes.

When given an unknown pattern, each fuzzy neural network uses the learned knowledge to estimate the membership value [Kwa9306] of this pattern in each class and classify the input pattern according to the membership values.

Each of these fuzzy neural networks can be expressed by a set of fuzzy inference rules and these networks have been used for various applications.

The structure and the input to output relationship of each of these networks have also been explained.

Fuzzy neural networks and fuzzy concepts have found many applications and a few of them that we have tried are described in this paragraph. Fuzzy neural networks have been used in phoneme sequence recognition as described in [Kwa0205], [Don0306], [Kwa0312]. Adaptive neuro-fuzzy network can be used for speech enhancement in [The0512].

In practice, fuzzy concepts is an effective tool for filtering images contaminated by salt and pepper noise and the methodology and obtained results are described in [Kwa9308a], [Kwa0208b], [Kwa0303], [Kwa0305b], [Kwa1112]. In [Nil0505], an interactive content-based visual image retrieval system is described which uses linguistic terms to describe colors and textures to retrieve images from a large image database.

Part IV

Evolutionary Computing

IV.1 Overview

Evolutionary algorithms are metaheuristic (high-level heuristic) optimization algorithms that use operators to model biological evolution to arrive at a solution to solve an optimization problem.

The advantages of evolutionary algorithms include derivative-free and global search capability.

In this eBook, four common used evolutionary algorithms including genetic algorithm, differential evolution, particle swarm optimization, and simulated annealing are described; and coding, initialization, and constraints of these algorithms are also explained.

Matlab programs of these evolutionary algorithms are given. Evolutionary algorithms can be applied to design neural networks, fuzzy systems, and fuzzy neural networks, and have many other design applications. Additional evolutionary algorithms can be found in [Kwa1803] in which the uses of some of these algorithms for solving digital filter design problems are briefly described in the following paragraphs.

For digital filter design: artificial bee colony (ABC) algorithm has been applied to design linear phase FIR differentiators [Kwa1610], nonlinear phase FIR filters [Raj1704], sparse FIR filters [Raj1808], and IIR filters [Raj1805].

Cuckoo search algorithm (CSA) has been applied to design linear phase FIR filters [Kwa1610], nonlinear phase FIR filters [Lia1704], sparse FIR filters [Kwa1808], [Lai1811], 2-dimensional sparse FIR filters [Lai1811], and IIR filters [Lia1805].

Teaching-learning-based optimization (TLBO) has been applied to design linear phase FIR Hilbert transformers [Kwa1610], nonlinear phase FIR filters [Zha1704], IIR filters [Kwa1805], 2-dimensional linear phase FIR digital filters [Kwa1811], and 2-dimensional nonlinear phase FIR filters [Kwa1808];

Harmony search (HS) algorithm has been applied to design nonlinear phase FIR filters [Kwa1805], and allpass equalizers [Kwa1808], [Kwa1811]; and interactive-self-learning algorithm (ISLA) has been applied to design nonlinear phase FIR filters [Kwa1704].

Part IV on evolutionary computing covers five chapters including genetic algorithms, differential evolution, particle swarm optimization, simulated annealing, and coding, initialization, and constraints.

These four algorithms are covered in Chapters 13-16 and can be used to training shallow and deep neural networks, fuzzy systems, fuzzy neural networks, and other intelligent systems.

Chapter 17 covers common procedures such as coding, initialization, and constraints required for evolutionary optimization. Additional evolutionary algorithms are described and explained in [Kwa1803].

Chapter 13

Genetic Algorithms

13.1 Introduction

A genetic algorithm (GA) can be used to search for improved structures and parameters of a neural network, and to determine the fuzzy if-then rules and their membership functions of a fuzzy system or a fuzzy-neural network.

Genetic algorithms offer a general-purpose stochastic optimization method for solving a search problem inspired by biological mechanisms of evolution.

Genetic algorithms normally operate on a number of binary strings (called chromosomes). Each binary string (or chromosome) can be formed by concatenating a number of sub-strings (or parameters).

Each sub-string can be formed by applying binary coding of a parameter. Each bit (1 or 0) in a binary string is called an allele. Each binary string represents a possible solution to the search problem (or a point in the search space).

Special features of genetic algorithms include: (a) it searches in parallel to find the global minimum thus reducing the chance of being trapped by a local minimum; (b) it works with parameter coding and by evaluating its objective function (for example, error function for minimization or fitness function for maximization); and (c) its operations depend on probability.

In this chapter, we shall first introduce the three basic genetic operators of genetic algorithms, namely, reproduction, crossover, and mutation.

Definitions of schema, order, length are then given. Improved reproduction methods, namely, improved selection and linear scaling are described.

Fitness function mappings are described. Examples are given to explain the underlying concept in each topic.

Some earlier work on genetic algorithms can be found in [Hol6207], [Hol7500]. An earlier reference for genetic algorithms is [Siv0800].

13.2 Genetic Algorithms

A list of symbols to be used for genetic algorithms is shown in Table 13.2.1 and the steps of a basic genetic algorithm is shown in Table 13.2.2.

Table 13.2.1 Notations for genetic algorithms.

Symbol	Description
P	Population size or number of chromosomes
L_c	Length of chromosomes
x_i	Chromosome value of i th chromosome
$f(x_i)$	Fitness value of i th chromosome
$f_c(x_i)$	Cumulative fitness value up to i th chromosome
f_t	Total fitness value of all chromosomes
p_c	Probability of crossover
p_m	Probability of mutation
N_{cs}	Number of crossover sites
N_s	Crossover site location
G	Generation gap
p_v	Precision value

Table 13.2.2 Steps of a basic genetic algorithm.

Step	Description
1	Generate a population consisting of a number of chromosomes.
2	Select parents based on their fitness values.
3	Parent-pairing is carried out randomly. If selected, the probability that a parent-pair will undergo crossover at a probability p_c and mutation at a probability p_m .
4	Based on fitness values of chromosomes, select offsprings to reproduce a new population.
5	Stop if a feasible solution is obtained, otherwise go to step 2.

13.2.1 Reproduction

Reproduction is a process in which individual chromosomes (or strings) of a population are selected according to their fitness values.

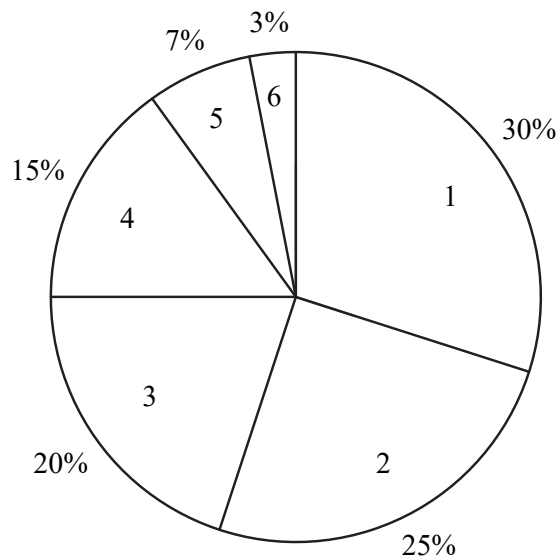


Figure 13.2.1 Roulette wheel with 6 chromosomes weighted with 30%, 25%, 20%, 15%, 7%, and 3%.

The selection process is carried out by spinning a roulette wheel with slots of different sizes proportional to the fitness value of each individual chromosome as shown in Figure 13.2.1. The roulette wheel reproduction is carried out as explained in Table 13.2.1.

Table 13.2.3 Steps of roulette wheel reproduction.

Step	Description
1	Compute the total fitness value (i.e., sum of the fitness values) of all the chromosomes in a population.
2	Generate a random number N_{rr} between 0 and the total fitness value.
3	Select the first chromosome with a cumulative fitness value of all the preceding chromosomes and itself greater than or equal to N_{rr} .

Example 13.1 (Roulette wheel reproduction of next population)

Given a population of six chromosomes with the fitness values specified by 30, 25, 20, 15, 7, and 3 as shown in Figure 13.2.1 and Table E13.1.1.

Six random numbers 53, 9, 64, 96, 28, and 82 are generated as shown in Table E13.1.2. Using the roulette wheel reproduction method, determine the chromosomes of the next population (or generation).

Solution

Table E13.1.1 Total fitness of 6 chromosomes.

Chromosome	Fitness	Total fitness	Percentage
1	30	30	30
2	25	55	25
3	20	75	20
4	15	90	15
5	7	97	7
6	3	100	3

Table E13.1.2 Roulette wheel parent selection.

Random number, N_{rr}	53	9	64	96	28	82
Chromosome for next generation	2	1	3	5	1	4

The next generation consists of one copy of chromosomes 2, 3, 4, and 5; and two copies of chromosome 1.

This example shows that chromosomes with higher fitness values have higher probabilities to be reproduced as offsprings in the next generation.

Over a number of generations, this reproduction operator will eliminate chromosomes with lower fitness values.

13.2.2 Crossover

Crossover operates on two selected parents to create two new children (or offsprings) which inherit genes (or traits) from both the parents.

Crossover is carried out by mating (or pairing) two selected parents with a crossover probability p_c at a crossover site N_s (i.e., a bit position) that is randomly selected.

In a multi-parameter chromosome, for each parameter on the child's chromosome, crossover operator can randomly select one of the parents and replace the parameter value of the child by that of the parameter from the same position of the selected parent.

Two-point crossover ($N_{cs} = 2$) can reduce positional bias of one-point crossover by taking two locations and swap the middle segments.

Two-point or multiple-point crossover can also retain quality schema that 1-point crossover cannot.

However, an increased number of points of crossover could lead to more mixing and less structure resulting a degraded performance. One-point crossover is illustrated by Example 13.2.

Example 13.2 (Crossover two parents to generate two children)

Given two selected parents A (1011010010) and B (0111100001), use the crossover operator with a crossover probability p_c of 1.0 to generate two new children.

Solution

Parent A	1011010010
Parent B	0111100001

Randomly select a crossover site after the bit position 4.

Parent A	1011	010010
Parent B	0111	100001

With a probability $p_c = 1$, recombine the two chromosomes to form two new children as

Child A	1011	100001
Child B	0111	010010

13.2.3 Mutation

Although reproduction and crossover operations can produce new chromosomes, they do not introduce new changes in the bit level of a population.

To introduce new bits on a chromosome, mutation can be applied to invert a randomly selected bit with a fixed but small probability p_m .

For the case of a multi-parameter chromosome, similar mutation can be applied to each of the parameters with a fixed but small probability p_m .

Though the mutated chromosome is usually similar to its parents, this will increase the diversity of genes in the population by introducing unexplored genetic bits.

Also it will prevent a genetic algorithm from converging prematurely to a sub-optimal solution.

Mutation should be used with a low probability otherwise the process will behave like a random search.

In a simple genetic algorithm, we need to specify the following four parameters: P = population size; p_c = crossover probability; p_m = mutation probability; and G = generation gap.

The generation gap G [Jon7500] is defined between 0 and 1 with $G = 1$ for non-overlapping population; and $0 < G < 1$ for overlapping population.

The pseudocode of a genetic algorithm using the three basic genetic operators described so far is given in Table 13.2.2.

Table 13.2.2 Pseudocode for genetic algorithm using three basic genetic operators (Start: S; End: E; Reproduction: Roulette Wheel; C: Crossover; M: Mutation).

Pseudocode	Comment
1 while (convergence criterion is not met)	S
2 for $p = 1:P$	select p th chromosome for next generation
3 $N_{rr} = f_t * rand;$	generate a random number $N_{rr} \in f_t[0,1]$
4 for $i = 1:P$	check i th cumulative fitness value
5 if $f_c(x_i) \geq N_{rr}$	check x_i eligibility for selection
6 Select x_i as p th parent for next generation; break	R
7 end	end x_i selection
8 end	end i th chromosome selected as p th chromosome
9 end	end p th chromosome selection
10 for $i = 1:P$	perform crossover for the i th parent
11 $N_{rc} = rand;$	generate a random number $N_{rc} \in [0,1]$
12 if $(p_c \geq N_{rc}) \& x_i$ not yet selected	C

13	Randomly select x_j with index $j \neq i$;		C
14	$N_S = \text{ceil}((L_c - 1) \times \text{rand})$;		C
15	i th and j th chromosomes perform crossover at site N_S ;		C
16	end	end x_i selection	C
17	end	end crossover	C
18	for $i = 1:P$	perform mutation for i th chromosome	M
19	for $k = 1:L_c$	perform mutation for k th bit	M
20	$N_{rm} = \text{rand}$;	generate a random number $N_{rm} \in [0,1]$	M
21	if ($P_m \geq N_{rm}$)	check k th bit eligibility for mutation	M
22	$x_i(k) = \text{XOR}(x_i(k), 1)$;		M
23	end	end k th bit selection	M
24	end	end mutation for k th bit	M
25	end	end mutation for i th chromosome	M
26	end	end while	E

Design Example 13.3 (Solving second-order polynomial function by GA) Matlab

Given a population size $P = 6$ with six arbitrarily selected initial positive values $\mathbf{X} = [0.500, 1.875, 2.125, 4.875, 5.500, 6.875]$; $p_c = 0.7$; and $p_m = 0.125$, write a Matlab program to solve the equation $e(x) = x^2 - 7x + 12.25 = 0$ with x defined within the interval $[0, 7]$ using the basic genetic algorithm consisting of three operators, reproduction, crossover, and mutation.

Solution

The equation of $e(x) = 0$ contains two double-roots at $x = 3.5$; and the function $e(x)$ is a positive function within the interval $[0, 7]$.

Solving $e(x)$ is equivalent to minimizing $e(x)$ with

$$e(x) = x^2 - 7x + 12.25 \quad (\text{E13.3.1})$$

Define the fitness function $f(x)$ such that

$$f(x) = \frac{1}{e(x) + \varepsilon} \quad (\text{E13.3.2})$$

The parameter ε is a small positive number such that $\varepsilon > 0$. Thus the problem of minimizing $e(x)$ is equivalent to maximizing $f(x)$ in which $f(x)$ is at its maximum when $e(x)$ is at its minimum.

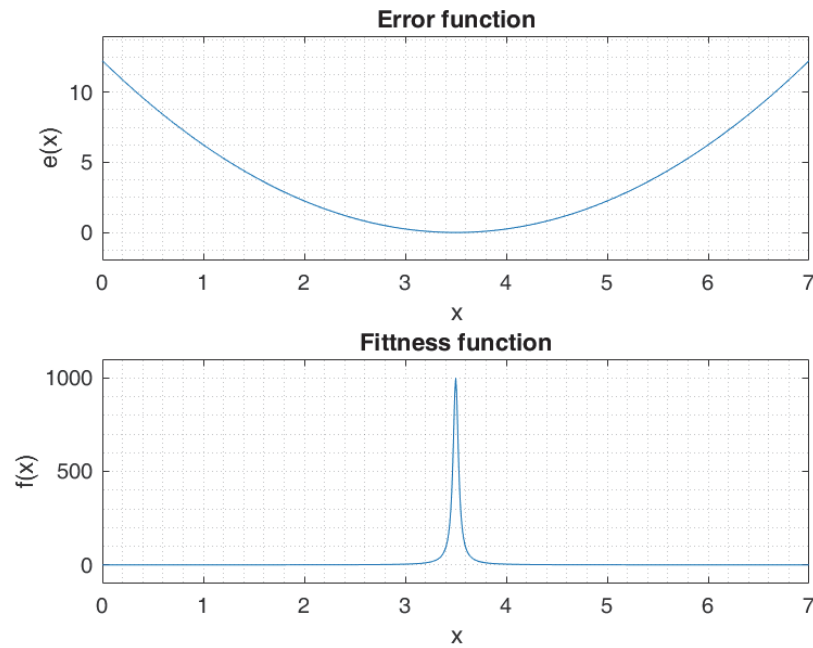


Figure E13.3.1 Minimizing $e(x)$ is equivalent to maximizing $f(x)$ (with $\varepsilon = 10^{-3}$).

With $\varepsilon = 10^{-3}$, the two functions are plotted as shown in Figure E13.3.1.

The results of reproduction, crossover, and mutation for the first three generations and the last generation are shown in Table E13.3.1.

Table E13.3.1 GA calculations involving reproduction, crossover, and mutation ($c1(t)$ to $c6(t)$ refer chromosomes 1 to 6 at generation t) (Hints: For both Crossover and Mutation Processes: View the Roulette Wheel Copies, the New index i , and the binary pattern of each chromosome i to keep track its movement).

Generation 1							
Reproduction Process (Fitness Sum = 1.8847; Fitness Average = 0.3141)							
i	Initial Population	Value x_i	Fitness $f(x_i)$	$\frac{f(x_i)}{\sum_{i=1}^6 f(x_i)}$	Roulette Wheel Copies	New Parents	New i
1	000.1000	0.500	0.1111	0.0589	1	0.500	1
2	001.1110	1.875	0.3786	0.2009	0	2.125	2
3	010.0010	2.125	0.5287	0.2805	2	2.125	3
4	100.1110	4.875	0.5287	0.2805	2	4.875	4
5	101.1000	5.500	0.2499	0.1326	0	4.875	5
6	110.1110	6.875	0.0878	0.0466	1	6.875	6
Crossover Process							
Assuming c1(1) and c2(1) have been randomly selected for pairing. Let c1(1)=c1(0) pairs c3(1)=c3(0) and c2(1)=c3(0) pairs c6(1)=c6(0). (Fitness Sum = 4.8426; Fitness Average = 0.8071)							
i	Mating Pool	Site	New Population	Value of New x_i'	Fitness $f(x_i')$	New i	
1	00 0.1000	2	000.0010	0.125	0.0878	1	
3	01 0.0010	2	010.1000	2.500	0.9990	3	
2	0 10.0010	1	010.1110	2.875	2.5535	2	
6	1 10.1110	1	110.0010	6.125	0.1451	6	

Mutation Process

c1(1), c2(1), c3(1), and c6(1) have crossover.

c4(1)=c4(0) and c5(1)=c4(0) have no crossover.

Copy c1(1) to c6(1) as updated population.

Assuming c2(1) and c3(1) have been randomly selected for mutation.

(Fitness Sum = 2.697; Fitness Average = 0.4495)

i	Population	Site	Mutated Population	Value x_i	Fitness $f(x_i)$	New i
1	000.0010	-	000.0010	0.125	0.0878	1
2	010.1110	5	010.1010	2.625	1.3044	2
3	010.1000	1 & 6	110.1010	6.625	0.1024	3
4	100.1110	-	100.1110	4.875	0.5287	4
5	100.1110	-	100.1110	4.875	0.5287	5
6	110.0010	-	110.0010	6.125	0.1451	6

Notes:

The expected mutation is $7 \times 6 \times 0.125 = 5.25$ bits per generation, and actually 3 mutations happen in this generation.

As shown above, population average fitness has improved from 0.3141 to 0.4495, and maximum fitness has increased from 0.5287 to 1.3044 during the same period.

Generation 2

Reproduction Process (Fitness Sum = 2.697; Fitness Average = 0.4495)

Copy the Mutated Population as the Initial Population.

i	Initial Population	Value x_i	Fitness $f(x_i)$	$\frac{f(x_i)}{\sum_{i=1}^6 f(x_i)}$	Roulette Wheel Copies	New Parents	New i
1	000.0010	0.125	0.0878	0.03255	0	2.625	1
2	010.1010	2.625	1.3044	0.48366	4	2.625	2
3	110.1010	6.625	0.1024	0.03796	0	2.625	3

4	100.1110	4.875	0.5287	0.19601	2	2.625	4
5	100.1110	4.875	0.5287	0.19601	0	4.875	5
6	110.0010	6.125	0.1451	0.05380	0	4.875	6

Crossover Process

Assuming $c1(2)$ and $c2(2)$ have been randomly selected for pairing.
 Let $c1(2)=c2(1)$ pairs $c6(2)=c4(1)$ and $c2(2)=c2(1)$ pairs $c5(2)=c4(1)$.
 (Fitness Sum = 7.7849; Fitness Average = 1.2974)

i	Mating Pool	Site	New Population	Value of New x_i'	Fitness $f(x_i')$	New i
1	010.10 10	5	010.1010	2.625	1.3044	1
6	100.11 10	5	100.1110	4.875	0.5286	6
2	010.1 010	4	010.1110	2.875	2.5535	2
5	100.1 110	4	100.1010	4.625	0.7895	5

Mutation Process

$c1(2)$, $c2(2)$, $c5(2)$, and $c6(2)$ have crossover.
 $c3(2)=c2(1)$ and $c4(2)=c2(1)$ have no crossover.
 Copy $c1(2)$ to $c6(2)$ as updated population.
 Assuming $c3(2)$, $c4(2)$, and $c6(2)$ have been randomly selected for mutation.
 (Fitness Sum = 5.8697; Fitness Average = 0.9783)

i	Population	Site	Mutated Population	Value x_i	Fitness $f(x_i)$	New i
1	010.1010	-	010.1010	2.625	1.3044	1
2	010.1110	-	010.1110	2.875	2.5535	2
3	010.1010	2	000.1010	0.625	0.1210	3
4	010.1010	6	010.1000	2.500	0.9990	4
5	100.1010	-	100.1010	4.625	0.7895	5
6	100.1110	1 & 4	000.0110	0.375	0.1024	6

Notes:

The expected mutation is $7 \times 6 \times 0.125 = 5.25$ bits per generation, and actually 4 mutations happen in this generation.

As shown above, population average fitness has improved from 0.4495 to 0.9783, and maximum fitness has increased from 1.3044 to 2.5535 in one generation.

Generation 3

Reproduction Process (Fitness Sum = 5.8697; Fitness Average = 0.9783)

Copy the Mutated Population as the Initial Population.

i	Initial Population	Value x_i	Fitness $f(x_i)$	$\frac{f(x_i)}{\sum_{i=1}^6 f(x_i)}$	Roulette Wheel Copies	New Parents	New i
1	010.1010	2.625	1.3044	0.22223	1	2.625	1
2	010.1110	2.875	2.5535	0.43502	3	2.875	2
3	000.1010	0.625	0.1210	0.02061	0	2.875	3
4	010.1000	2.500	0.9990	0.17020	2	2.875	4
5	100.1010	4.625	0.7895	0.13450	0	2.500	5
6	000.0110	0.375	0.1024	0.01744	0	2.500	6

Crossover Process

Assuming c1(3) and c4(3) have been randomly selected for pairing.

Let c1(3)=c1(2) pairs c6(3)=c4(2) and c4(3)=c2(2) pairs c5(3)=c4(2).

(Fitness Sum = 10.9628; Fitness Average = 1.8271)

i	Mating Pool	Site	New Population	Value of New x_i'	Fitness $f(x_i')$	New i
1	0 10.1010	1	010.1000	2.500	0.9990	1
6	0 10.1000	1	010.1010	2.625	1.3044	6
4	010.111 0	6	010.1110	2.875	2.5535	4
5	010.100 0	6	010.1000	2.500	0.9990	5

Mutation Process

c1(3), c4(3), c5(3), and c6(3) have crossover.

c2(3)=c2(2) and c3(3)=c2(2) have no crossover.

Copy c1(3) to c6(3) as updated population.

Assuming c1(3), c3(3), c4(3), and c6(3) have been randomly selected for mutation.

(Fitness Sum = 8.2209; Fitness Average = 1.3702)

i	Population	Site	Mutated Population	Value x_i	Fitness $f(x_i)$	New i
1	010.1000	2	000.1000	0.5000	0.1111	1
2	010.1110	-	010.1110	2.8750	2.5535	2
3	010.1110	7	010.1111	2.9375	3.1505	3
4	010.1110	5	010.1010	2.6250	1.3044	4
5	010.1000	-	010.1000	2.5000	0.9990	5
6	010.1010	1	110.1010	6.6250	0.1024	6

Notes:

The expected mutation is $7 \times 6 \times 0.125 = 5.25$ bits per generation, and actually 4 mutations happen in this generation.

As shown above, population average fitness has improved from 0.9783 to 1.3702, while maximum fitness has increased from 2.5535 to 3.1505 in one generation.

...

Generation 537 (GA converged)

Reproduction Process (Fitness Sum = 6000; Fitness Average = 1000)

i	Initial Population	Value x_i	Fitness $f(x_i)$	$\frac{f(x_i)}{\sum_{i=1}^6 f(x_i)}$	Roulette Wheel Copies	New Parents	New i
1	011.1000	3.500	1000	0.16667	1	3.500	1
2	011.1000	3.500	1000	0.16667	1	3.500	2
3	011.1000	3.500	1000	0.16667	2	3.500	3
4	011.1000	3.500	1000	0.16667	0	3.500	4

5	011.1000	3.500	1000	0.16667	1	3.500	5
6	011.1000	3.500	1000	0.16667	1	3.500	6

At the generation 537, the fitness and average fitness reach the maximum comparing to the generations 1, 2, 3, all the chromosomes reaches 3.5, which gives the optimal solution.

13.3 Schema, Order, Length

A schema [Glo8900] is defined as a similarity template describing a subset of strings which are identical at certain string positions.

Schema order $o(H)$ is defined as the number of fixed positions in a schema.

Schema length $d(H)$ is defined as the distance between the first and the last specified bit positions in a schema.

Examples for schema, schema order, and schema length are shown in Table 13.3.1.

Table 13.3.1 Examples for schema, order, length.

Term	Example (* means don't care)
Schema	Schema *101* describes a subset of 4 members [0 1 0 1 0, 0 1 0 1 1, 1 1 0 1 0, 1 1 0 1 1]
Schema order	$o(*101*) = 3$; $o(*1*0*) = 2$
Schema length	$d(010*1**) = 5-1 = 4$; $d(0*****) = 1-1 = 0$

Example 13.4 (Order, length and mutation immunity of schema)

Consider the following schemas consisting of 12 bits: (i) ***1*01***0*; (ii) ***1*****01**; (iii) *1*****1*****; (iv) 00**1*0**1**; (v) *010**110*10.

(a) What are the order and length of each of the schemas?

(b) Estimate the probability of surviving mutation for each schema if the probability of mutation p_m is 0.1 at a single bit position.

Solution

(a) From the definition of the order and the length of each of the schemas, the order and length can be summarized in Table E13.4.1.

Table E13.4.1 Order and length of schema.

Schema	(i)	(ii)	(iii)	(iv)	(v)
Order	4	3	2	5	8
Length	7	6	5	9	10

(b) If the probability of mutation p_m is 0.1 at a single bit position, the probability of surviving mutation for each schema is

(i) $(1 - 0.1)^4 = 0.6561$

(ii) $(1 - 0.1)^3 = 0.7290$

(iii) $(1 - 0.1)^2 = 0.8100$

(iv) $(1 - 0.1)^5 = 0.59049$

(v) $(1 - 0.1)^8 = 0.43046721$

Example 13.5 (Numbers of schemas and mutation immunities of binary and hex codes)

Consider one-mega-bit memory as a search space, compare a binary coded genetic algorithm to a hex-coded genetic algorithm by calculating the followings:

(a) Upper and lower bounds on the number of different schemas in a population of size $P = 50$.

(b) Calculate the probability of surviving mutation for each schema using both coding methods if the probability of mutation p_m is 0.1 at a single bit position for binary coding and at a single digit position for hex coding. State your conclusion from the obtained results.

Solution

(a) Since one-mega-bit = 1,048,576 = $2^{20} = 16^5$ bits, the length of a binary string is 20 and that for a hex string is 5.

The upper bound on the number of different schemas in a population of size $P = 50$ for a binary-coded genetic algorithm is $2^{20} + (2^{20} - 1) + (2^{20} - 2) + (2^{20} - 3) + \dots + (2^{20} - 48) + (2^{20} - 49)$ (for 50 different schemas).

In other words, 2^{20} implies there are 2^{20} different schemes, $(2^{20} - 1)$ implies there are 2^{20} schemes but there are only $(2^{20} - 1)$ different bit patterns (i.e., two of the 2^{20} schemes have identical bit patterns).

The lower bound on the number of different schemas in a population of size $P = 50$ for a binary-coded genetic algorithm is 2^{20} (for 50 identical schemas).

The upper bound on the number of different schemas in a population of size $P = 50$ for a hex-coded genetic algorithm is $16^5 + (16^5 - 1) + (16^5 - 2) + (16^5 - 3) + \dots + (16^5 - 48) + (16^5 - 49)$ (for 50 different schemas).

The lower bound on the number of different schemas in a population of size $P = 50$ for a hex-coded genetic algorithm is 16^5 (for 50 identical schemas).

(b) The probability of surviving mutation for each schema is

For binary coding: $(1 - 0.1)^{20} = 0.1216$

For hex coding: $(1 - 0.1)^5 = 0.81$

Hence, shorter schemas have higher chances to survive from one generation to the next.

13.4 Improved Reproduction Methods

Simple genetic algorithms are guided largely by the action of three operators: reproduction, crossover, and mutation. There are other

natural operators that can improve the robustness of simple genetic algorithms.

Under improved reproduction methods, an improved selection method and a macro operator are explained in this section.

Macro operators are genetic operators acting at a population level; and micro operators are genetic operators operating at a chromosomal level.

In this chapter, a number of natural operators are described which include two micro genetic operators, namely, multiple-point crossover (Problem 13.10) and inversion operator (Problem 13.11); and three macro genetic operators, namely, linear scaling (described below), ranking procedure (Problems 13.12 and 13.13), and sharing function (Problem 13.14).

13.4.1 Improved selection

The remainder stochastic sampling without replacement method [Lin9600] is an improved selection method based on elite strategy.

In this method, each chromosome $\mathbf{x}_i$ with a fitness value $f(\mathbf{x}_i)$ of a population of P chromosomes is allocated offsprings according to the integer part of C_i calculated from

$$C_i = P \times Pselect_i = P \times f(\mathbf{x}_i) / \left(\sum_{j=1}^P f(\mathbf{x}_j) \right) \quad (13.4.1)$$

The fractional part of C_i is used as a probability for choosing another offspring of the chromosome i up until a complete new population is formed.

This method could offer an improved performance over the expected value methods (like roulette wheel).

An example on remainder stochastic sampling without replacement method is shown in Example 13.6 and two other improved selection methods based on elite strategies are given and illustrated in Problems 13.7 and Problem 13.8.

Example 13.6 (Remainder stochastic sampling without replacement)

Consider ten chromosomes with the following fitness values: 70, 23, 9, 35, 55, 18, 46, 68, 87, and 94. The expected number of copies of each chromosome in the mating pool is summarized in Table E13.6.1.

Table E13.6.1 Calculations of remainder stochastic sampling without replacement.

Chromosome	Probability of reproduction	Copy	Probability of another copy
1	$10 \cdot 70 / 505 = 1.3861$	1	0.3861
2	$10 \cdot 23 / 505 = 0.4554$	0	0.4554
3	$10 \cdot 9 / 505 = 0.1782$	0	0.1782
4	$10 \cdot 35 / 505 = 0.6931$	0	0.6931
5	$10 \cdot 55 / 505 = 1.0891$	1	0.0891
6	$10 \cdot 18 / 505 = 0.3564$	0	0.3564
7	$10 \cdot 46 / 505 = 0.9109$	0	0.9109
8	$10 \cdot 68 / 505 = 1.3465$	1	0.3465
9	$10 \cdot 87 / 505 = 1.7228$	1	0.7228
10	$10 \cdot 94 / 505 = 1.8614$	1	0.8614

If the total number of offsprings is less than the population size P , the remaining offsprings can be selected based on the fractional part of C_i as a probability for choosing another offspring of the chromosome i up until a complete new population is obtained.

Alternatively, the remaining offsprings can be selected from those top chromosomes directly or from randomly selected chromosomes until the total number of offsprings is equal to the population size P .

13.4.2 Linear scaling

Linear scaling [Lin9600] is a macro genetic operator which operates in a population level.

Linear scaling is useful to control the number of offsprings that a chromosome can have to diversify the population.

This is especially critical in the initial generations as a number of superior chromosomes could occupy a considerable portion of the population, thereby reducing its diversity and converging to a local optimal solution.

Denoting the raw fitness as f and the scaled fitness as f' , linear scaling requires a linear relationship between f' and f as follows

$$f' = af + b \quad (13.4.2)$$

where the coefficients a and b are chosen such that

$$f'_{avg} = f_{avg} \quad (13.4.3)$$

$$f'_{max} = C_{mult} \times f_{avg} \quad (13.4.4)$$

$$f'_{min} \geq 0 \quad (13.4.5)$$

Equation (13.4.3) ensures that a population member with an average fitness value f_{avg} contributes one expected offspring in the next generation.

In (13.4.4), the parameter C_{mult} denotes the number of expected copies for the best population member. Equation (13.4.4) governs the number of offsprings given to a population member with the maximum raw fitness.

Equation (13.4.5) ensures negative scaled fitness value f' will not arise after scaling.

Example 13.7 (Linear scaling of fitness values) Matlab

Write a Matlab program to apply linear scaling to transform a given set of 10 raw fitness values $f = [3, 7, 9, 12, 16, 18, 21, 24, 88, 95]$ to 10 linearly scaled fitness values f' with the parameter C_{mult} to be selected as 2, to illustrate the advantages of linear scaling that include:

- (a) Each population member with average fitness contributes one expected offspring in the next generation (by (13.4.3));
- (b) The number of offspring given to the population member with a maximum raw fitness is controlled (by (13.4.4)); and
- (c) The minimum fitness function is non-negative (by (13.4.5)).

Plot the fitness values before and after linear scaling.

Solution

Summing the given fitness values f , we obtain

$$f_{total} = 3 + 7 + 9 + 12 + 16 + 18 + 21 + 24 + 88 + 95 = 293$$

$$f_{avg} = f_{total}/10 = 29.3 \quad (\text{E13.7.1})$$

Applying linear scaling of (13.4.2) to f ,

$$\begin{aligned} f'_1 &= 3a + b & f'_2 &= 7a + b & f'_3 &= 9a + b & f'_4 &= 12a + b \\ f'_5 &= 16a + b & f'_6 &= 18a + b & f'_7 &= 21a + b & f'_8 &= 24a + b \\ f'_9 &= 88a + b & f'_{10} &= 95a + b \end{aligned} \quad (\text{E13.7.2})$$

From (E13.7.2),

$$f'_{total} = 293a + 10b \quad (\text{E13.7.3})$$

$$f'_{avg} = f'_{total}/10 = 29.3a + b \quad (\text{E13.7.4})$$

From (E13.7.1) and (E13.7.4),

$$f_{avg} = f'_{avg} = 29.3a + b = 29.3 \quad (\text{E13.7.5})$$

With $C_{mult} = 2$, from (13.4.4) and (E13.7.5),

$$f'_{max} = C_{mult} \times f_{avg} = 2 \times 29.3 = 58.6 \quad (\text{E13.7.6})$$

From (E13.7.2) and (E13.7.6),

$$f'_{max} = 95a + b = 58.6 \quad (\text{E13.7.7})$$

Solving a and b from (E13.7.5) and (E13.7.7), we obtain

$$a = 0.446, b = 16.232 \quad (\text{E13.7.8})$$

From (E13.7.2) and (E13.7.8), the scaled fitness values f' can be obtained from a and b values as

$$f'_1 = 3(0.446) + 16.23 = 17.57 \quad f'_2 = 7(0.446) + 16.23 = 19.35$$

$$f'_3 = 9(0.446) + 16.23 = 20.25 \quad f'_4 = 12(0.446) + 16.2 = 21.58$$

$$f'_5 = 16(0.446) + 16.2 = 23.37 \quad f'_6 = 18(0.446) + 16.2 = 24.26$$

$$f'_7 = 21(0.446) + 16.2 = 25.60 \quad f'_8 = 24(0.446) + 16.2 = 26.94$$

$$f'_9 = 88(0.446) + 16.2 = 55.48 \quad f'_{10} = 95(0.446) + 16.2 = 58.60$$

A plot of the fitness values before (black) and after (blue) linear scaling is shown in Figure E13.7.1.

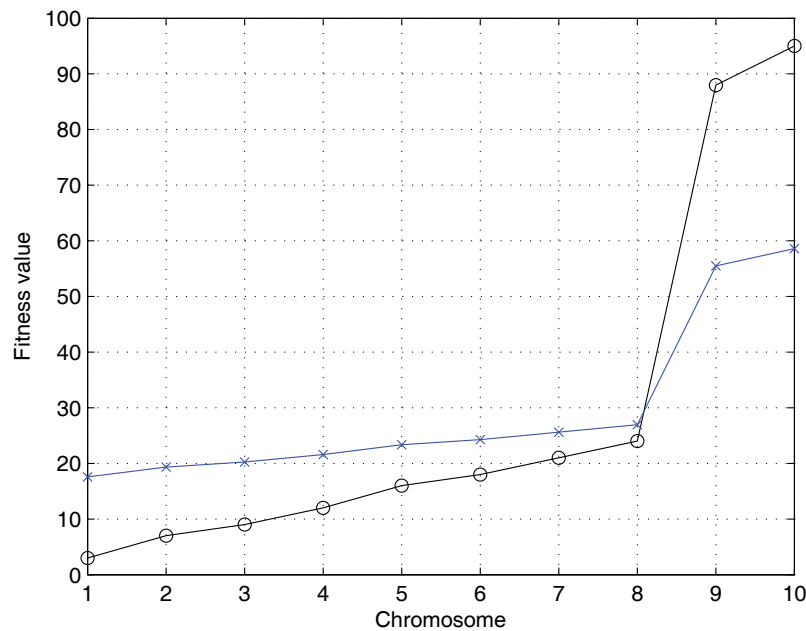


Figure E13.7.1 Fitness values before (black) and after (blue) linear scaling.

13.5 Fitness Function Mapping

In a genetic algorithm, an objective function is required to be appropriately mapped to a fitness function $f(\mathbf{x})$ of positive value where $\mathbf{x}$ represents a parameter vector to be optimized. In this section, we shall describe how this mapping can be carried out for maximization and minimization problems. Before that, we shall first introduce the 68-95-99.7 rule.

13.5.1 68-95-99.7 rule

The variance $var(\mathbf{x}) = \sigma^2$ and the standard deviation σ of a set of data $\mathbf{x} = [x_1, x_2, x_3, \dots, x_N]$ are measures for the dispersion of a set of data from its mean value μ .

For N equally likely values, the variance $var(\mathbf{x})$ can be written as

$$var(\mathbf{x}) = \sigma^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \mu)^2 \quad (13.5.1)$$

The standard deviation $std(\mathbf{x})$ is equal to the square root of variance such that

$$std(\mathbf{x}) = \sigma = \sqrt{\frac{1}{N} \sum_{n=1}^N (x_n - \mu)^2} \quad (13.5.2)$$

According to the 68-95-99.7 rule [Wik6895], for a normally distributed random variable $\mathbf{x}$, with its mean value denoted by μ and its standard deviation value denoted by σ , the fractions of values that lie within a band around the mean in a normal distribution with a width of two, four and six standard deviations are equal to

Fraction of $(\mu - \sigma \leq \mathbf{x} \leq \mu + \sigma) \approx 0.6827$

Fraction of $(\mu - 2\sigma \leq \mathbf{x} \leq \mu + 2\sigma) \approx 0.9545 \quad (13.5.3)$

Fraction of $(\mu - 3\sigma \leq \mathbf{x} \leq \mu + 3\sigma) \approx 0.9973$

13.5.2 Maximization problem

If the objective function is to maximize a profit function $p(x)$ with p_{min} being the absolute value of the expected smallest $p(x)$ value (such that $p(x) > -p_{min}$), then the profit-to-fitness function mapping can be defined as

$$f(x) = \begin{cases} p(x) + p_{min} & \text{if } p(x) + p_{min} > 0 \\ 0 & \text{otherwise} \end{cases} \quad (13.5.4)$$

Therefore, the fitness function $f(x)$ obtained is always positive as required.

p_{min} can be chosen from the absolute smallest $p(x)$ value “in the current population” or “in the last T populations” or “observed so far”.

Similar to minimization problems, p_{min} should be varied according to the variance $\sigma^2 = (1/N) \sum_{n=1}^N (x_n - \mu)^2$ of the profit function $p(x)$ in a population.

As explained in Section 13.5.1, an appropriate value for p_{min} is $(\mu - 3\sigma)$.

13.5.3 Minimization problem

Given the objective function is to minimize a real-value error function $e(x)$ with e_{max} being the expected largest $e(x)$ value, the error-to-fitness function mapping can be defined as

$$f(x) = \begin{cases} e_{max} - e(x) & \text{if } e(x) < e_{max} \\ 0 & \text{otherwise} \end{cases} \quad (13.5.5)$$

Therefore, the fitness function $f(x)$ obtained is always positive as required.

The parameter e_{max} can be taken as the largest $e(x)$ value “in the current population” or “in the last T populations” or “observed so far”.

More desirably, e_{max} should be varied according to the variance $\sigma^2 = (1/N) \sum_{n=1}^N (x_n - \mu)^2$ of the error function $e(x)$ in a population. As explained in Section 13.5.1, an appropriate value for e_{max} is $(\mu + 3\sigma)$.

In practice, for a minimization problem with an objective to minimize a real-value error function $e(x)$, a more common adopted error-to-fitness function mapping is

$$f(x) = \frac{1}{e(x) + \varepsilon} \quad (13.5.6)$$

where ε denotes a small positive constant.

13.6 Neural Network Design using Genetic Algorithm

Binary-coded (or integer-coded) genetic algorithms require processing entire strings even a few binary bits are involved in crossover and mutation, this occupies computer memory especially for problems involving a large number of parameters represented by long strings for a higher precision.

Under these circumstances, real-valued coded genetic algorithms can be used and one such representation method is described in [Li0712]. An example on integer-valued coded genetic algorithm is first given in Design Example 13.8. Example 13.9 explains how genetic algorithm can be used to solve mixed-valued coded problems; and Example 13.10 explains how genetic algorithm can be used to solve a real-valued coded problem.

Design Example 13.8 (2-layer feedforward neural network learning of integer-valued XOR problem using GA) Matlab

Write a Matlab program to design a 2-layer neural network with integer-valued parameters using the genetic algorithm to learn the 4 input to output pairs of the XOR problem as described according to Chapter 4, Example 4.1.

The search continues until the mean squared output error of a chromosome falls below δ .

With a small positive value for ε , the fitness function $f(\mathbf{c})$ to be maximized is defined as

$$f(\mathbf{c}) = \frac{1}{e(\mathbf{c}) + \varepsilon} \quad (\text{E13.8.1})$$

The coefficient search space is

$$-s < w_1, w_2, w_3, w_4, w_5, w_6, b_1, b_2, b_3 < s \quad (\text{E13.8.2})$$

The size of the population P is 90. Let the neural network parameters' range be $\pm s = \pm 16$. For $s = 16$, assign 1 sign bit and 5 integer bits ($B = 6$) is required to represent each neural network parameter.

Each chromosome has $N_c = 9$ parameters with a total length of $D = N_c B$ binary-bits encoded as a parameter vector $\mathbf{c}$.

The slope parameter of the binary sigmoidal function at each neuron's output is taken as $r_n = 2$ with a unity slope at zero input and an input range of $\pm L = \pm 2$ before saturating to 0 and 1 outputs [9].

Initial coefficients are generated randomly within ± 16 and then quantized to $B = 6$ binary bits.

The number of crossover positions N_{cs} is 1, the crossover probability p_c is 0.8, and the mutation probability p_m is 0.1.

The output mean absolute error limit is taken as $\delta = 0.0000001$ and $\varepsilon = 0.01$.

A summary of the suggested parameter values is given in Table E13.8.1.

Table E13.8.1 2-layer neural network parameters for GA.

P	N_c	s	B	D	r_n	L	N_{cs}	p_c	p_m	ε	δ
90	9	16	6	$N_c B$	2	2	1	0.8	0.1	0.01	0.0000001

Additional guidelines for using the genetic algorithm are given in Table E13.8.2.

Table E13.8.2 Additional guidelines for learning XOR problem using GA.

Additional guidelines	
1	Parent-pair is carried out randomly. N_{cs} -point crossover is adopted in which the positions are randomly chosen, and then alleles at their positions are exchanged with each other.
2	If selected, the probability that a parent-pair will undergo crossover at a probability p_c and mutation at a probability p_m .
3	Mutation occurs with a probability of 1.0 on selected parents who have not crossed over.
4	A bit of a chromosome, if selected, is mutated by inverting its binary bit from 1 to 0 or from 0 to 1.

Solution

The Matlab program for designing the XOR problem using genetic algorithm is listed in Table E13.8.3. The designed parameters and the mean absolute error of the 2-layer neural networks are listed in Table E13.8.4.

Table E13.8.3 Matlab program for integer-valued XOR design problem using genetic algorithm.

```
% nnet_xor_ga_mFun.m
% Use of Global Optimization Matlab Toolbox GA function for
% 2-layer neural network (with a bias at each neuron) design

clc; clear; close all;
format long;

%% XOR design problem
input = [0 0; 0 1; 1 0; 1 1];
dout = [0; 1; 1; 0];
nvar=9; % Number of variables
% Lower & upper bounds of variables
Lb=[-16,-16,-16,-16,-16,-16,-16,-16,-16];
```

```

Ub=[16,16,16,16,16,16,16,16,16];

%% GA (Set parameters for optimization)

% Default parameters
% PopulationSize = min(100,10*nvar); % P
% CrossoverFraction = 0.8000; % pc
% EliteCount: '0.05*PopulationSize'
% MaxGenerations = 100*nvar; % maxIter
% FunctionTolerance = 0.000001; % funTolerance

% Create mutated children using Gaussian distribution.
% scale = 1.0; shrink = 1.0;
% options = optimoptions ...
% ('ga','MutationFcn',{@mutationgaussian,scale,shrink});
% scale controls what fraction of the gene's range is searched
% shrink of 1 result in scale shrinking linearly to 0 as GA
progresses
% to the number of generations specified by the options structure.

% Choose parents using stochastic universal sampling
% options = optimoptions('ga', 'SelectionFcn', @selectionstochunif);

options = optimoptions('ga', 'Generations', 900);
options = optimoptions('ga', 'TolFun', 1e-12);
options = optimoptions('ga', 'Display', 'iter');

objfun = @nnet_xor_objFun; % objective function

% Design Example 13.8 - Integer-valued for parameters [1:9]
[W,fval,exitFlag,output,population,scores] = ...
ga(objfun,nvar,[],[],[],[],Lb,Ub,[],[1:9],options);

% Design Example 13.9 - Integer-valued for parameters [1:6]
% [W,fval,exitFlag,output,population,scores] = ...
% ga(objfun,nvar,[],[],[],[],Lb,Ub,[],[1:6],options);

% Design Example 13.9 - Integer-valued for parameters [7:9]
% [W,fval,exitFlag,output,population,scores] = ...

```

```

% ga(objfun,nvar,[],[],[],[],Lb,Ub,[],[7:9],options);

% Design Example 13.10 - Real-valued for parameters [1:9]
% [W,fval,exitFlag,output,population,scores] = ...
% ga(objfun,nvar,[],[],[],[],Lb,Ub,[],[],options);

%% Design results

mae = nnet_xor_objFun(W); % mean absolute error
yr = nnet_xor_verify(W,input); % recall output
fprintf('MAE: %18.15f\n', mae);
fprintf('Yr: %18.15f %18.15f %18.15f %18.15f\n', yr);
fprintf('W: %18.15f %18.15f %18.15f %18.15f %18.15f ...
%18.15f %18.15f %18.15f %18.15f \n', W);
figure; stem(W); grid; title('Parameters'); % designed parameters

function [mae] = nnet_xor_objFun(W)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);
input = [0,0; 0,1; 1,0; 1,1];
dout = [0; 1; 1; 0];

sae = 0; % set initial value for sum of absolute errors
alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 =w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z = y1*w5+(-1)*b3+y2*w6;
    y(j) = 1/(1+exp((-2)*z));
    ae(j)= (abs(y(j)-dout(j)));
    sae= sae+ ae(j);
end
mae= sae/4; % mean absolute error
end

```

```

function yr=nnet_xor_verify(W,input)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);

alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 =w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z = y1*w5+(-1)*b3+y2*w6;
    yr(j) = 1/(1+exp((-2)*z));
end
end

```

Table E13.8.4 Design parameters and result of integer-valued XOR problem using genetic algorithm.

Example 13.8 - Integer-valued for parameters [1:9] [W,fval,exitFlag,output,population,scores] = ... ga(objfun,nvar,[],[],[],[],Lb,Ub,[],[1:9],options);	
Weights (w1 to w6)	Biases (b1 to b3)
16	9
16	-8
-16	8
-16	
16	Mean absolute error
-16	0.000000112536242

Design Example 13.9 (2-layer feedforward neural network learning of mixed-valued XOR problem using GA) Matlab

Repeat the Design example 13.8 to design a 2-layer neural network with mixed-valued parameters using the genetic algorithm to learn the 4 input to output pairs of the XOR problem. In this problem, consider: (a)

the connection weights 1 to 6 are integer-valued and the bias connection weights 7 to 9 are real-valued; (b) the connection weights 1 to 6 are real-valued and the bias connection weights 7 to 9 are integer-valued;

Solution

Replacing the Matlab optimization function (ga.m) in the main program nnet_xor_ga_mFun.m for the Design Example 13.8 - Integer-valued parameters by that for the Design Example 13.9 - Mixed-valued for parameters, the design results obtained are listed in Table 13.9.1 and Table 13.9.2

Table E13.9.1 Design parameters and result of mixed-valued XOR problem using genetic algorithm (Case 1).

Design Example 13.9 - Integer-valued for parameters [1:6] [W,fval,exitFlag,output,population,scores] = ... ga(objfun,nvar,[],[],[],[],Lb,Ub,[],[1:6],options);	
Weights (w1 to w6)	Biases (b1 to b3)
-16	8.099986220098876
-16	-8.128374426400518
16	-8.000236511228797
16	
16	Mean absolute error
16	0.000000112535752

Table E13.9.2 Design parameters and result of mixed-valued XOR problem using genetic algorithm (Case 2).

Design Example 13.9 - Integer-valued for parameters [7:9] [W,fval,exitFlag,output,population,scores] = ... ga(objfun,nvar,[],[],[],[],Lb,Ub,[],[7:9],options);	
Weights (w1 to w6)	Biases (b1 to b3)
14.133625008584753	-10
15.793605630127409	7
-15.783664455769996	8

-15.748530427766456	
-15.999149390646943	Mean absolute error
15.999470241461024	0.000000112623090

Design Example 13.10 (2-layer feedforward neural network learning of real-valued XOR problem using GA) Matlab

Repeat the Design example 13.8 to design a 2-layer neural network with real-valued parameters using the genetic algorithm to learn the 4 input to output pairs of the XOR problem.

Solution

Replacing the Matlab optimization function (ga.m) in the main program nnet_xor_ga_mFun.m for the Design Example 13.8 - Integer-valued parameters by that for the Design Example 13.10 - Real-valued for parameters, the design results obtained are listed in Table 13.10.1.

Table E13.10.1 Design parameters and result of real-valued XOR problem using genetic algorithm.

Design Example 13.10 - Real-valued for parameters [1:9] [W,fval,exitFlag,output,population,scores] = ... ga(objfun,nvar,[],[],[],[],Lb,Ub,[],[],options);	
Weights (w1 to w6)	Biases (b1 to b3)
-15.895291114202422	7.868429453812528
15.306215407830916	7.856204270490352
15.466588446177816	7.995999727724460
-15.456302379622137	
15.992896196868639	Mean absolute error
15.982877339889427	0.0000001139114151918336

Chapter 14

Differential Evolution

14.1 Introduction

Differential evolution (DE) is a stochastic parallel search method that optimizes a population of parameter vectors at the same time to increase the chance of finding a global optimum.

As an evolutionary algorithm, it can be used to optimize non-differentiable type of nonlinear objective functions.

The function to be optimized is an objective function. In general, given an objective function $\varphi(\mathbf{x})$ defined in a D -dimensional space, the purpose is to find $\mathbf{x}_p = [x_{1p}, x_{2p}, \dots, x_{Dp}]$ for $p = 1$ to P such that the value of the objective function $\varphi(\mathbf{x}_p)$ is optimized among a population of $\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_P]$.

Differential evolution creates new members in the solution space by scaling and adding members from the existing population. This makes it a self-referential method.

Since scaling and addition are linear operators, the initial probability distribution of the members is kept the same. This property makes the differential evolution scheme completely self-organizing.

The major characteristics of differential evolution include inherently parallel; ability for effective global optimization; no predefined probability distribution is required; amendable to real, integer, and mixed-parameter optimization; precision limited only by floating-point format; and applicable to noisy objective functions.

Differential evolution requires only a few parameters to manage its operations.

Using differential evolution does not require knowledge of its underlining principles and other evolutionary optimization techniques.

In general, differential evolution is robust, fast, simple, and easy to use.

Three design examples are given to illustrate the applications of the differential evolution algorithms to optimization problems. A general reference for differential evolution is given in [Pri0500].

14.2 Algorithm

A list of symbols adopted for the differential evolution algorithm is shown in Table 14.2.1.

Table 14.2.1 Notations for DE algorithm.

Symbol	Description
D	Number of parameters in a parameter vector
P	Number of parameter vectors in a population
t	Generation (or iteration) index
T_{max}	Maximum number of generations (or iterations)
x_{dp}	d th parameter of p th parameter vector
$\mathbf{x}_p$	p th parameter vector, $\mathbf{x}_p = [x_{1p}, x_{2p}, \dots, x_{Dp}]$
$\mathbf{x}$	Present population, $\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_P]$
$\varphi(\mathbf{x}_p)$	Objective function of a parameter vector $\mathbf{x}_p$
$\mathbf{y}$	Mutated population, $\mathbf{y} = [\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_P]$ generated by a combination of randomly chosen members from the present population $\mathbf{x}$
$\mathbf{z}$	Child (or crossover) population, $\mathbf{z} = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_P]$ generated by combining members from the present population $\mathbf{x}$ and the mutated population $\mathbf{y}$
p_c	Crossover probability

Given a parameter vector $\mathbf{x}_p = [x_{1p}, x_{2p}, \dots, x_{Dp}]$ and its objective function $\varphi(\mathbf{x}_p)$, differential evolution iteratively changes $\mathbf{x}_p$ until the value of its objective function $\varphi(\mathbf{x}_p)$ converges to a desired level.

After all the parameter vectors are initialized, the processing of the first generation begins.

As described in the following sub-sections, each iteration consists of three steps, mutation, crossover, and selection; and at the end of a generation, a new population is created.

14.2.1 Initialization

Each of the P parameter vectors $\mathbf{x}_p$ for $p = 1, 2, \dots, P$ of a population is a member of the suspected solution space R^D and has a total of D parameters grouped as the D elements of a parameter vector.

Before starting, all the parameter vectors are randomly initialized following a uniform probability distribution within a bounded region where the minimum solution is suspected to be located.

This can be carried out by randomly assigning each parameter vector $\mathbf{x}_{initial}$ between the lower bound and upper bound vectors defined by $\mathbf{x}_{min}$ and $\mathbf{x}_{max}$.

14.2.2 Mutation

The key feature of differential evolution lies in its approach to mutation which gives differential evolution a self-referencing property.

During the mutation stage, a new population $\mathbf{y}$ consisting of population members $\mathbf{y}_p$ for $p = 1, 2, \dots, P$ is generated by a combination of randomly chosen members from the present population $\mathbf{x}$ consisting of population members $\mathbf{x}_p$ for $p = 1, 2, \dots, P$.

Each the P mutant members $\mathbf{y}_p$ of the population $\mathbf{y}$ is formed using a mutation scheme defined as

$$\mathbf{y}_p = \mathbf{x}_p + m_1(\mathbf{x}_{r1} - \mathbf{x}_{r2}) \text{ for } p = 1, 2, \dots, P \quad (14.2.1)$$

An alternative form for (14.2.1) is

$$\mathbf{y}_p = \mathbf{x}_p + m_1(\mathbf{x}_{r1} - \mathbf{x}_{r2}) + m_2(\mathbf{x}_{best} - \mathbf{x}_p) \quad (14.2.2)$$

for $p = 1, 2, \dots, P$

where $r1$ and $r2$ are two different randomly drawn indices from $(1, 2, \dots, P)$ which are distinct from p ; and m_1 and m_2 are mutation constants within the range of $[0.1, 1]$.

The last term in (14.2.2) provides a means to enhance the greediness by incorporating the difference between the present best parameter vector $\mathbf{x}_{best}$ and $\mathbf{x}_p$.

14.2.3 Crossover

The crossover stage creates a new (or child) population $\mathbf{z}$ consisting of members $\mathbf{z}_p$ for $p = 1, 2, \dots, P$ by combining members from the present population $\mathbf{x}$ and the mutated population $\mathbf{y}$.

One way to perform crossover is as follows: For each member $\mathbf{z}_p$ of the new population $\mathbf{z}$, a random index $randi_d$ is chosen within the interval $[1, D]$. Also, for each parameter z_{pd} of a member $\mathbf{z}_p$, a random value $rand_d$ is chosen within $[0, 1]$.

Then, the random index $randi_d$ is compared with the present index d and the random value $rand_d$ is compared with the given crossover probability p_c , such that if $d = randi_d$ or $rand_d \leq p_c$, z_{pd} is replaced by y_{pd} ; otherwise, z_{pd} is replaced by x_{pd} from the present member $\mathbf{x}_p$.

$$z_{pd} = \begin{cases} y_{pd} & \text{if } (d = randi_d \parallel rand_d \leq p_c) \\ x_{pd} & \text{otherwise} \end{cases} \quad \text{for } d = 1, \dots, D \quad (14.2.3)$$

In so doing $\mathbf{z}_p \neq \mathbf{x}_p$ can be ensured. In other words, the new member $\mathbf{z}_p$ will have at least one different parameter when compared to the present member $\mathbf{x}_p$.

14.2.4 Selection

Selection is performed by calculating the objective function values for the new member $\mathbf{z}_p$ and that of the present member $\mathbf{x}_p$.

14.2.4.1 Maximization problem

For a maximization problem, its objective (or profit) function is to be maximized. Between $\mathbf{z}_p$ and $\mathbf{x}_p$, the member which has the larger total objective (or profit) function value will be selected to become the next $\mathbf{x}_p$. Hence, $\mathbf{x}_p(t + 1)$ for the next generation is determined as

$$\mathbf{x}_p(t + 1) = \begin{cases} \mathbf{z}_p & \text{if } f(\mathbf{x}_p) \leq f(\mathbf{z}_p) \\ \mathbf{x}_p & \text{otherwise} \end{cases} \quad \text{for } p = 1 \text{ to } P \quad (14.2.4)$$

14.2.4.2 Minimization problem

For a minimization problem, its objective (or error) function is to be minimized. Between $\mathbf{z}_p$ and $\mathbf{x}_p$, the member which has the smaller objective (or error) function value will be selected to become the next $\mathbf{x}_p$. Hence, $\mathbf{x}_p(t + 1)$ for the next generation is determined as

$$\mathbf{x}_p(t + 1) = \begin{cases} \mathbf{z}_p & \text{if } f(\mathbf{x}_p) \geq f(\mathbf{z}_p) \\ \mathbf{x}_p & \text{otherwise} \end{cases} \quad (14.2.5)$$

Table 14.2.2 Pseudocode for DE algorithm (M: Mutation; C: Crossover; S: Selection).

Pseudocode		Comment	
1	Initialize $\mathbf{x}_p$ for $p = 1:P$, and other parameter values for $D, P, m_1, p_c, T_{max}, \delta$.		
2	for $t = 1:T_{max}$	Start iterations	
3	for $p = 1:P$		
4	$r1 = rand_{1p} \neq p$;		M
5	$r2 = rand_{2p} \neq p \neq r1$;		M
6	$\mathbf{y}_p = \mathbf{x}_p + m_1(\mathbf{x}_{r1} - \mathbf{x}_{r2})$;		M
7	$index = randi_d$;		C
8	For $d = 1:D$		C
9	if $d = index \parallel rand_{3p} \leq p_c$		C

10	$z_{pd} = y_{pd};$		C
11	else		C
12	$z_{pd} = x_{pd};$		C
13	end		C
14	end		C
15	if $f(\mathbf{x}_p) \geq f(\mathbf{z}_p)$	For minimization	S
16	$\mathbf{x}_p(t+1) = \mathbf{z}_p;$		S
17	else		S
18	$\mathbf{x}_p(t+1) = \mathbf{x}_p;$		S
19	end		S
20	end	for $p = 1:P$	
21	For $p = 1:P$		
22	if $f(\mathbf{x}_p) < \delta$ then stop		
23	end		
24	end	for $t = 1:T_{max}$	

A design example using the differential evolution algorithm described above is given in Design Example 14.1.

Design Example 14.1 (2-layer feedforward neural network learning of XOR problem using DE)

Write a Matlab program to design a two-layer neural network with real-valued parameters using the differential evolution to learn the 4 input to output pairs of the XOR problem as described in Chapter 4, Example 4.1.

The search continues until the output mean absolute error of a solution vector falls below the function tolerance of δ or when the maximum number of iterations (maxIter) is reached.

The coefficient search space is

$$-s < w1, w2, w3, w4, w5, w6, b1, b2, b3 < s \quad (\text{E14.1.1})$$

The size of the population P is 90. Let the range of the neural network parameters be $\pm s = \pm 16$. Each solution vector has $N_c = 9$ parameters with a length of $D = N_c$ grouped as a parameter vector $\mathbf{c}$.

The slope parameter of the binary sigmoidal function at each neuron's output is taken as $r_n = 2$ with a unity slope at zero input and an input range of $\pm L = \pm 2$ before saturating to 0 and 1 outputs [Kwa9207].

Initial coefficients are generated within $\pm s$ using a uniform distribution function.

The two mutation constants m_1 and m_2 are set within the range of $[0.1, 0.9]$.

The output mean absolute error (function tolerance) is taken as $\delta = 0.00000011254$. The maximum number of iterations (maxIter) is 900.

The algorithm for the differential evolution is described in (14.2.1)-(14.2.5).

A summary of the specified parameter values is given in Table E14.2.1.

Table E14.1.1 DE parameters for 2-layer neural network.

P	N_c	D	s	r_n	L	m_1, m_2	p_c	maxIter	δ
90	9	N_c	16	2	2	$\in [0.1, 0.9]$	0.5	900	0.00000011254

Solution

Use the IC Matlab Toolbox `de_ic` function to write a Matlab program to design the XOR problem using differential evolution as listed in Table E14.1.2. The designed parameters and the mean absolute error of the 2-layer neural networks are listed in Table E14.1.3. The convergence curve of the differential evolution algorithm is plotted in Figure E14.1.1

Table E14.1.2 Matlab program for XOR design problem using differential evolution.

```
% nnet_xor_de_ic.m
% Use of DE algorithm for 2-layer neural network (with a bias at
% each neuron) design

clc; clear; close all;
format long;

%% XOR design problem

input = [0 0; 0 1; 1 0; 1 1];
dout = [0; 1; 1; 0];
nvar=9; % Number of variables
% Lower & upper bounds of variables
Lb=[-16,-16,-16,-16,-16,-16,-16,-16,-16];
Ub=[16,16,16,16,16,16,16,16,16];

%% DE (Set parameters for optimization)

P = min(100,10*nvar); % Population size
mmin = 0.1; % Lower bound of mutation constant
mmax = 0.9; % Upper bound of mutation constant
pc = 0.5; % Crossover probability
maxIter = 100*nvar; % Maximum number of iterations
funTolerance = 0.00000011254;

[BestSol,BestCost] = ...
de_ic(nvar,Lb,Ub,P,mmin,mmax,pc,maxIter,funTolerance);

%% Design results

figure; semilogy(BestCost, 'LineWidth', 1); xlabel('Iteration');
ylabel('Best cost'); grid;

mae = nnet_xor_objFun(BestSol.Position); % mean absolute error
yr = nnet_xor_verify(BestSol.Position,input); % recall output
fprintf('MAE: %18.15f\n', mae);
```



```
fprintf('Yr: %18.15f %18.15f %18.15f %18.15f\n', yr);
fprintf('W: %18.15f %18.15f %18.15f %18.15f %18.15f %18.15f
%18.15f %18.15f %18.15f\n', BestSol.Position);
figure; stem(BestSol.Position); grid; title('Parameters');
```

```
function [mae] = nnet_xor_objFun(W)

w1= W(1); w2= W(2); w3= W(3); w4= W(4); w5= W(5); w6=
W(6);
b1= W(7); b2= W(8); b3= W(9);
input = [0,0; 0,1; 1,0; 1,1];
dout = [0; 1; 1; 0];

sae = 0;    % set initial value for sum of absolute errors
alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z  = y1*w5+(-1)*b3+y2*w6;
    y(j) = 1/(1+exp((-2)*z));
    ae(j)= (abs(y(j)-dout(j)));
    sae= sae+ ae(j);
end
mae= sae/4;    % mean sum of absolute errors
end
```

```
function yr=nnet_xor_verify(W,input)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1= W(7); b2= W(8); b3= W(9);

% recall using binary sigmoid activation function
alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
```

```

z2=w2*input(j,1)+(-1)*b2+w4*input(j,2);
y2 = 1/(1+exp((-alp)*z2));
z  = y1*w5+(-1)*b3+y2*w6;
yr(j) = 1/(1+exp((-2)*z));
end
end

```

Table E14.1.3 Design parameters and result of XOR problem using differential evolution.

Weights (w1 to w6)	Biases (b1 to b3)
-16	7.923429326033853
16	8.050712053181162
16	7.996365124773107
-16	
16	Mean absolute error
16	1.125387556957476e-07

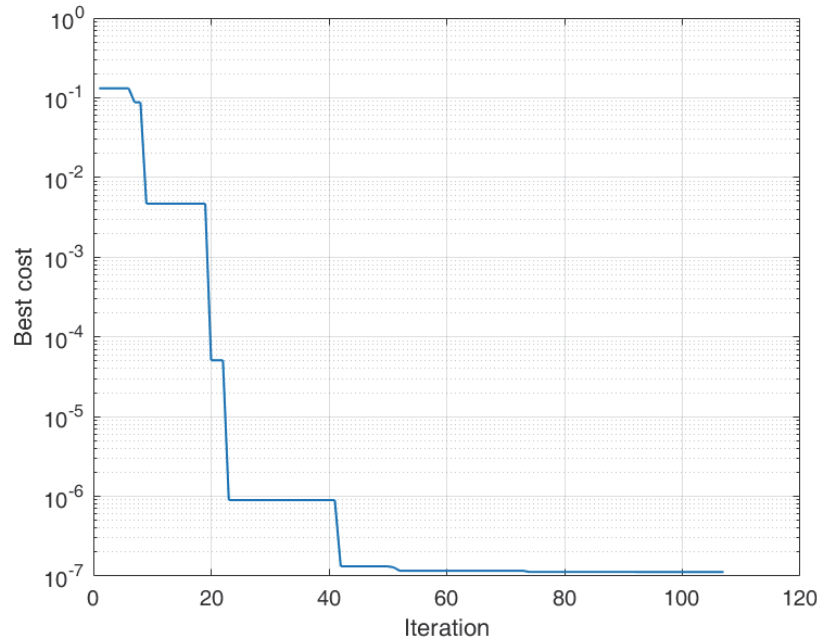


Figure E14.1.1 Convergence curve of XOR problem using differential evolution.

14.3 Population-based Crossover and Selection

In this section, a variant of the differential evolution using a population-based crossover and selection is described. The other two operators, initialization and mutation are identical to those of Sections 14.2.1 and 14.2.2.

14.3.1 Crossover

The crossover stage creates a new (or child or crossover) population $\mathbf{z}$ consisting of members $\mathbf{z}_p$ for $p = 1, 2, \dots, P$ by combining members from the present population $\mathbf{x}$ and the mutated population $\mathbf{y}$.

One way to perform crossover is as follows: For each member $\mathbf{z}_p$ of the new population $\mathbf{z}$, a real positive random number $rand_p$ is generated within the interval $[0, 1]$.

Then, this random number is compared with the given crossover probability p_c ; if $rand_p \leq p_c$, $\mathbf{z}_p$ is replaced by $\mathbf{y}_p$ from the muted population; otherwise, $\mathbf{z}_p$ is replaced by $\mathbf{x}_p$ from the present population.

$$\mathbf{z}_p = \begin{cases} \mathbf{y}_p & \text{if } (rand_p \leq p_c) \\ \mathbf{x}_p & \text{if } (rand_p > p_c) \end{cases} \text{ for } p = 1, \dots, P \quad (14.3.1)$$

The new population $\mathbf{z}$ is consisted of members from the present population $\mathbf{x}$ and the mutated population $\mathbf{y}$.

In order for $\mathbf{z}$ to have at least one different member than the present population $\mathbf{x}$, a member $\mathbf{y}_p$ can be randomly selected from the mutated population $\mathbf{y}$ to be included in $\mathbf{z}$.

14.3.2 Selection

Selection is performed by calculating the objective function values for the new population $\mathbf{z}$ and that of the present population $\mathbf{x}$.

14.3.2.1 Maximization problem

For a maximization problem in which its objective (or profit) function is to be maximized, the population which has the larger total objective (or profit) function value will be selected to form the next generation, and the remaining population will be removed.

The objective function is calculated by evaluating the sum of the profit functions for $p = 1, 2, \dots, P$ to be maximized for each of the two competing populations, $\mathbf{x}$ and $\mathbf{z}$. The new population $\mathbf{x}(t + 1)$ is then determined as

$$\mathbf{x}(t + 1) = \begin{cases} \mathbf{z} & \text{if } \sum_{p=1}^P f(\mathbf{x}_p) \leq \sum_{p=1}^P f(\mathbf{z}_p) \\ \mathbf{x} & \text{otherwise} \end{cases} \quad (14.3.2)$$

14.3.2.2 Minimization problem

For a minimization problem in which its objective (or error) function is to be minimized, the population which has the smaller total objective

(or error) function value will be selected to form the next generation, and the remaining population will be removed.

The objective function is calculated by evaluating the sum of error functions for $p = 1, 2, \dots, P$ to be minimized for each of the two competing populations, $\mathbf{x}$ and $\mathbf{z}$. The new population $\mathbf{x}(t + 1)$ is then determined as

$$\mathbf{x}(t + 1) = \begin{cases} \mathbf{z} & \text{if } \sum_{p=1}^P f(\mathbf{x}_p) \geq \sum_{p=1}^P f(\mathbf{z}_p) \\ \mathbf{x} & \text{otherwise} \end{cases} \quad (14.3.3)$$

Table 14.3.1 Pseudocode for DE algorithm using population-based crossover and selection (M: Mutation; C: Crossover; S: Selection).

Pseudocode		Comment	
1	Initialize $\mathbf{x}_p$ for $p = 1:P$; and other parameter values $D, P, m_1, p_c, T_{max}, \delta$.		
2	for $t = 1:T_{max}$	Start iterations	
3	for $p = 1:P$		
4	$r1 = rand_{1p} \neq p$;		M
5	$r2 = rand_{2p} \neq p \neq r1$;		M
6	$\mathbf{y}_p = \mathbf{x}_p + m_1(\mathbf{x}_{r1} - \mathbf{x}_{r2})$;		M
7	if $rand_{3p} \leq p_c$		C
8	$\mathbf{z}_p = \mathbf{y}_p$;		C
9	else		C
10	$\mathbf{z}_p = \mathbf{x}_p$;		C
11	end		C
12	end		
13	if $\sum_{p=1}^P f(\mathbf{x}_p) \geq \sum_{p=1}^P f(\mathbf{z}_p)$	For minimization	S
14	$\mathbf{x}(t + 1) = \mathbf{z}$;		S
15	else		S

16	$\mathbf{x}(t + 1) = \mathbf{x};$		S
17	end		S
18	For $p = 1:P$		
19	if $f(\mathbf{x}_p) < \delta$ then stop		
20	end		
21	end	End iterations	

Two design examples using the population-based crossover and selection in the differential evolution algorithm are given below.

Design Example 14.2 (Third-order polynomial interpolation by DE)

This example serves to illustrate the application of the differential evolution to polynomial interpolation.

Given N pairs of observed input $\mathbf{x} = [x_1, x_2, \dots, x_N]$ and output $\mathbf{y} = [y_1, y_2, \dots, y_N]$ data values, the optimization problem requires the differential evolution to search the optimal set of coefficients $\mathbf{a} = [a_0 \ a_1 \ a_2 \ a_3]$ of a third-order polynomial to fit a set of given data while able to interpolate satisfactorily using other arbitrary input values.

A general third-order polynomial can be expressed as

$$y = g(x) = a_0 + a_1x + a_2x^2 + a_3x^3$$

Assuming that the optimal set of coefficients is $\mathbf{a} = [1, -2.4, -3, 10]$; and $N(= 5)$ pairs of observed input and output data values are used for learning.

$$\mathbf{x} = [-0.5000 \ -0.2500 \ 0.0000 \ 0.2500 \ 0.5000]$$

$$\mathbf{y} = [0.2000 \ 1.2563 \ 1.0000 \ 0.3688 \ 0.3000]$$

Using the following set of differential evolution parameters $D = 4$; $P = 6$; $m_1 = 0.7$; $p_c = 0.5$; show the step-by-step calculations of the first three generations and the optimized results obtained after convergence.

Evaluate the interpolation error of the optimized solution using 40 uniformly distributed values of inputs within $-6 \leq x \leq 6$.

Solution

With a population of P solution vectors $[\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3, \dots, \mathbf{a}_P]$ such that the p th solution vector is represented by $\mathbf{a}_p = [a_{0p} \ a_{1p} \ a_{2p} \ a_{3p}]$, define the objective (or error) function to be minimized as

$$f(\mathbf{a}_p) = \frac{\sum_{n=1}^N (g_p(x_n) - y_n)^2}{N}$$

where

$$g_p(x_n) = a_{0p} + a_{1p}x_n + a_{2p}x_n^2 + a_{3p}x_n^3$$

Define the error limit (or convergence criterion) $\delta (= 10^{-5})$ such that if in a generation there is a population member $\mathbf{a}_p$ with $f(\mathbf{a}_p) \leq \delta$, then the differential evolution algorithm stops and a satisfactory optimized solution $\mathbf{a}_p$ is said to have been found.

If the optimal solution or its region in the search space of an optimization problem is known, one could randomly assign a population of initial solutions within the specific space.

In this example, given the optimal solution $[a_0 \ a_1 \ a_2 \ a_3] = [1, -2.4, -3, 10]$, random initial vectors can be assigned within an appropriate range of each coefficient.

Generation 1

Randomly assign 6 ($= P$) initial vectors as

$\mathbf{a}_1$	[0.7, -0.5, 0.6, 0.6]
$\mathbf{a}_2$	[0.9, 0.5, 0.9, -0.3]
$\mathbf{a}_3$	[-0.6, 0, 0.1, -0.6]
$\mathbf{a}_4$	[-0.3, -0.3, -0.8, -0.8]
$\mathbf{a}_5$	[0.8, 0.4, -0.2, -0.2]
$\mathbf{a}_6$	[-1, 0.7, 0.1, -0.8]

$$\sum_{p=1}^6 f(\mathbf{a}_p) = 6.6289$$

The objective function values of the generation 1 are

$$\begin{cases} f(\mathbf{a}_1) = 0.2276 \\ f(\mathbf{a}_2) = 0.4543 \\ f(\mathbf{a}_3) = 1.6570 \\ f(\mathbf{a}_4) = 1.1904 \\ f(\mathbf{a}_5) = 0.2314 \\ f(\mathbf{a}_6) = 2.8684 \end{cases}$$

The minimum objective function value in the generation 1 is $f(\mathbf{a}_1) = 0.2276 \geq \delta = 10^{-5}$, thus evolution has to be continued.

Compute the sum of fitness values as

$$\begin{aligned} \sum_{p=1}^6 f(\mathbf{a}_p) &= f(\mathbf{a}_1) + f(\mathbf{a}_2) + f(\mathbf{a}_3) + f(\mathbf{a}_4) + f(\mathbf{a}_5) + f(\mathbf{a}_6) \\ &= 6.6289 \end{aligned}$$

Mutation

Adopt two randomly chosen $\mathbf{a}_{r1}$ and $\mathbf{a}_{r2}$ to compute $\mathbf{b}_p = \mathbf{a}_p + m(\mathbf{a}_{r1} - \mathbf{a}_{r2})$ for p in $\{1, 2, \dots, 6\}$ as

$\mathbf{b}_1 = \mathbf{a}_1 + m(\mathbf{a}_2 - \mathbf{a}_4)$	[1.54, 0.06, 1.79, 0.95]
$\mathbf{b}_2 = \mathbf{a}_2 + m(\mathbf{a}_4 - \mathbf{a}_1)$	[0.2, 0.64, -0.08, -1.28]
$\mathbf{b}_3 = \mathbf{a}_3 + m(\mathbf{a}_4 - \mathbf{a}_2)$	[-1.44, -0.56, -1.09, -0.95]
$\mathbf{b}_4 = \mathbf{a}_4 + m(\mathbf{a}_3 - \mathbf{a}_1)$	[-1.21, 0.05, -1.15, -1.64]
$\mathbf{b}_5 = \mathbf{a}_5 + m(\mathbf{a}_1 - \mathbf{a}_2)$	[0.66, -0.3, -0.41, 0.43]
$\mathbf{b}_6 = \mathbf{a}_6 + m(\mathbf{a}_2 - \mathbf{a}_4)$	[-0.16, 1.26, 1.29, -0.45]

Crossover

Select $\mathbf{z}_p = \mathbf{b}_p$ if $\text{rand}_p \leq p_c$ or $\mathbf{z}_p = \mathbf{a}_p$ if $\text{rand}_p > p_c$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{z}_1 = \mathbf{b}_1$ ($\text{rand}_1 = 0.20242$)	[1.54, 0.06, 1.79, 0.95]
$\mathbf{z}_2 = \mathbf{a}_2$ ($\text{rand}_2 = 0.63505$)	[0.9, 0.5, 0.9, -0.3]
$\mathbf{z}_3 = \mathbf{a}_3$ ($\text{rand}_3 = 0.75862$)	[-0.6, 0, 0.1, -0.6]
$\mathbf{z}_4 = \mathbf{b}_4$ ($\text{rand}_4 = 0.26501$)	[-1.21, 0.05, -1.15, -1.64]
$\mathbf{z}_5 = \mathbf{b}_5$ ($\text{rand}_5 = 0.08510$)	[0.66, -0.3, -0.41, 0.43]
$\mathbf{z}_6 = \mathbf{a}_6$ ($\text{rand}_6 = 0.57142$)	[-1, 0.7, 0.1, -0.8]

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population

$$\sum_{p=1}^6 f(\mathbf{z}_p) = f(\mathbf{z}_1) + f(\mathbf{z}_2) + f(\mathbf{z}_3) + f(\mathbf{z}_4) + f(\mathbf{z}_5) + f(\mathbf{z}_6)$$

$$= 10.8077$$

As $\min \{ \sum_{p=1}^6 f(\mathbf{a}_p), \sum_{p=1}^6 f(\mathbf{z}_p) \} = \min \{ 6.6289, 10.8077 \} = 6.6289$, select $\mathbf{a}_p$ for $p=\{1, 2, \dots, 6\}$ for generation 2.

Generation 2

$\mathbf{a}_1$	[0.7, -0.5, 0.6, 0.6]
$\mathbf{a}_2$	[0.9, 0.5, 0.9, -0.3]
$\mathbf{a}_3$	[-0.6, 0, 0.1, -0.6]
$\mathbf{a}_4$	[-0.3, -0.3, -0.8, -0.8]
$\mathbf{a}_5$	[0.8, 0.4, -0.2, -0.2]
$\mathbf{a}_6$	[-1, 0.7, 0.1, -0.8]
$\sum_{p=1}^6 f(\mathbf{a}_p) = 6.6289$	

The minimum objective value of the generation 2 is $f(\mathbf{a}_1) = 0.2276 \geq \delta = 10^{-5}$, thus evolution has to be continued.

Mutation

Adopt two randomly chosen $\mathbf{a}_{r1}$ and $\mathbf{a}_{r2}$ to compute $\mathbf{b}_p = \mathbf{a}_p + m(\mathbf{a}_{r1} - \mathbf{a}_{r2})$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{b}_1 = \mathbf{a}_1 + m(\mathbf{a}_5 - \mathbf{a}_3)$	[1.68, -0.22, 0.39, 0.88]
$\mathbf{b}_2 = \mathbf{a}_2 + m(\mathbf{a}_3 - \mathbf{a}_5)$	[-0.08, 0.22, 1.11, -0.58]
$\mathbf{b}_3 = \mathbf{a}_3 + m(\mathbf{a}_1 - \mathbf{a}_6)$	[0.59, -0.84, 0.45, 0.38]
$\mathbf{b}_4 = \mathbf{a}_4 + m(\mathbf{a}_6 - \mathbf{a}_1)$	[-1.49, 0.54, -1.15, -1.78]
$\mathbf{b}_5 = \mathbf{a}_5 + m(\mathbf{a}_1 - \mathbf{a}_4)$	[1.5, 0.26, 0.78, 0.78]
$\mathbf{b}_6 = \mathbf{a}_6 + m(\mathbf{a}_2 - \mathbf{a}_1)$	[-0.86, 1.4, 0.31, -1.43]

Crossover

Select $\mathbf{z}_p = \mathbf{b}_p$ if $rand_p \leq p_c$ or $\mathbf{z}_p = \mathbf{a}_p$ if $rand_p > p_c$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{z}_1 = \mathbf{a}_1 (rand_1 = 0.65542)$	[0.7, -0.5, 0.6, 0.6]
$\mathbf{z}_2 = \mathbf{a}_2 (rand_2 = 0.59618)$	[0.9, 0.5, 0.9, -0.3]
$\mathbf{z}_3 = \mathbf{b}_3 (rand_3 = 0.11437)$	[0.59, -0.84, 0.45, 0.38]
$\mathbf{z}_4 = \mathbf{a}_4 (rand_4 = 0.76579)$	[-0.3, -0.3, -0.8, -0.8]
$\mathbf{z}_5 = \mathbf{a}_5 (rand_5 = 0.61286)$	[0.8, 0.4, -0.2, -0.2]
$\mathbf{z}_6 = \mathbf{a}_6 (rand_6 = 0.77541)$	[-1, 0.7, 0.1, -0.8]

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population as

$$\begin{aligned} \sum_{j=1}^4 f(\mathbf{z}_j) &= f(\mathbf{z}_1) + f(\mathbf{z}_2) + f(\mathbf{z}_3) + f(\mathbf{z}_4) + f(\mathbf{z}_5) + f(\mathbf{z}_6) \\ &= 5.1969 \end{aligned}$$

As $\min \{\sum_{p=1}^6 f(\mathbf{a}_p), \sum_{p=1}^6 f(\mathbf{z}_p)\} = \min \{6.6289, 5.1969\} = 5.1969$,
select $\mathbf{z}_p$ for $p = \{1, 2, \dots, 6\}$ for generation 3.

Generation 3

$\mathbf{a}_1$	[0.7, - 0.5, 0.6, 0.6]
$\mathbf{a}_2$	[0.9, 0.5, 0.9, - 0.3]
$\mathbf{a}_3$	[0.59, - 0.84, 0.45, 0.38]
$\mathbf{a}_4$	[-0.3, - 0.3, - 0.8, - 0.8]
$\mathbf{a}_5$	[0.8, 0.4, - 0.2, - 0.2]
$\mathbf{a}_6$	[-1, 0.7, 0.1, - 0.8]
$\sum_{p=1}^6 f(\mathbf{a}_p) = 5.1969$	

The objective values of the generation 3 are

$$\begin{cases} f(\mathbf{a}_1) = 0.2276 \\ f(\mathbf{a}_2) = 0.4543 \\ f(\mathbf{a}_3) = 0.2250 \\ f(\mathbf{a}_4) = 1.1904 \\ f(\mathbf{a}_5) = 0.2314 \\ f(\mathbf{a}_6) = 2.8684 \end{cases}$$

The minimum objective value of the generation 3 is $f(\mathbf{a}_3) = 0.2250 \geq \delta = 10^{-5}$, thus evolution has to be continued.

Mutation

Adopt two randomly chosen $\mathbf{a}_{r_1}$ and $\mathbf{a}_{r_2}$ to compute $\mathbf{b}_p = \mathbf{a}_p + m(\mathbf{a}_{r_1} - \mathbf{a}_{r_2})$ for $p=\{1, 2, \dots, 6\}$ as

$\mathbf{b}_1 = \mathbf{a}_1 + m(\mathbf{a}_6 - \mathbf{a}_4)$	[0.21, 0.2, 1.23, 0.6]
$\mathbf{b}_2 = \mathbf{a}_2 + m(\mathbf{a}_1 - \mathbf{a}_3)$	[0.977, 0.738, 1.005, - 0.146]
$\mathbf{b}_3 = \mathbf{a}_3 + m(\mathbf{a}_1 - \mathbf{a}_6)$	[1.78, - 1.68, 0.8, 1.36]
$\mathbf{b}_4 = \mathbf{a}_4 + m(\mathbf{a}_6 - \mathbf{a}_1)$	[-1.49, 0.54, - 1.15, - 1.78]

$\mathbf{b}_5 = \mathbf{a}_5 + m(\mathbf{a}_4 - \mathbf{a}_2)$	$[-0.04, \quad -0.16, \quad -1.39, \quad -0.55]$
$\mathbf{b}_6 = \mathbf{a}_6 + m(\mathbf{a}_2 - \mathbf{a}_4)$	$[-0.16, \quad 1.26, \quad 1.29, \quad -0.45]$

Crossover

Select $\mathbf{z}_p = \mathbf{b}_p$ if $rand_p \leq p_c$ or $\mathbf{z}_p = \mathbf{a}_p$ if $rand_p > p_c$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{z}_1 = \mathbf{b}_1 (rand_1 = 0.07010)$	$[0.21, \quad 0.2, \quad 1.23, \quad 0.6]$
$\mathbf{z}_2 = \mathbf{a}_2 (rand_2 = 0.53434)$	$[0.9, \quad 0.5, \quad 0.9, \quad -0.3]$
$\mathbf{z}_3 = \mathbf{b}_3 (rand_3 = 0.05816)$	$[1.78, \quad -1.68, \quad 0.8, \quad 1.36]$
$\mathbf{z}_4 = \mathbf{b}_4 (rand_4 = 0.46529)$	$[-1.49, \quad 0.54, \quad -1.15, \quad -1.78]$
$\mathbf{z}_5 = \mathbf{a}_5 (rand_5 = 0.75640)$	$[0.8, \quad 0.4, \quad -0.2, \quad -0.2]$
$\mathbf{z}_6 = \mathbf{b}_6 (rand_6 = 0.31743)$	$[-0.16, \quad 1.26, \quad 1.29, \quad -0.45]$

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population as

$$\sum_{j=1}^4 f(\mathbf{z}_j) = f(\mathbf{z}_1) + f(\mathbf{z}_2) + f(\mathbf{z}_3) + f(\mathbf{z}_4) + f(\mathbf{z}_5) + f(\mathbf{z}_6)$$

$$= 9.1853$$

As $\min \{\sum_{j=1}^4 f(\mathbf{a}_j), \sum_{j=1}^4 f(\mathbf{z}_j)\} = \min \{5.1969, 9.1853\} = 5.1969$, select $\mathbf{a}_p$ for $p=\{1,2, \dots, 6\}$ for generation 4.

...

Generation 4681 (DE optimization converged)

$\mathbf{a}_1$	$[0.9968, \quad -2.3410, \quad -2.9931, \quad 9.7629]$
$\mathbf{a}_2$	$[1.0159, \quad -2.4604, \quad -3.0532, \quad 10.1689]$
$\mathbf{a}_3$	$[0.9995, \quad -2.3419, \quad -3.0454, \quad 9.74976]$
$\mathbf{a}_4$	$[0.9996, \quad -2.3881, \quad -3.0015, \quad 9.92049]$

$\mathbf{a}_5$	[1.0029, -2.4160, -2.9978, 10.2288]
$\mathbf{a}_6$	[0.9919, -2.4102, -2.9553, 9.98854]
$\sum_{p=1}^6 f(\mathbf{a}_p) = 6.1093 \times 10^{-4}$	

The objective values of the generation 4681 are

$$\begin{cases} f(\mathbf{a}_1) = 5.5081 \times 10^{-5} \\ f(\mathbf{a}_2) = 2.1196 \times 10^{-4} \\ f(\mathbf{a}_3) = 1.0724 \times 10^{-4} \\ f(\mathbf{a}_4) = 7.9460 \times 10^{-6} \\ f(\mathbf{a}_5) = 1.8008 \times 10^{-4} \\ f(\mathbf{a}_6) = 4.8612 \times 10^{-5} \end{cases}$$

Since $\min_{p=1 \text{ to } 6} f(\mathbf{a}_p) = f(\mathbf{a}_4) = 7.9460 \times 10^{-6} \leq \delta = 10^{-5}$, the differential evolution algorithm stops, and the sum of objective values has been reduced from the initial value 6.6289 to the converged value 6.1093×10^{-4} .

It can be seen that, at last, all the 6 population members are approaching the optimal set of coefficients $[a_0, a_1, a_3, a_3] = [1, -2.4, -3, 10]$.

The best set of coefficients is

$$\mathbf{a}_4 = [0.9996, -2.3881, -3.0015, 9.92049].$$

The optimal function and the optimized function curves are plotted in Figure E14.1.1.

To evaluate the performance of the optimized function, forty uniformly spread values of x between $-0.6 \leq x \leq 0.6$ are used to calculate the mean squared error MSE_q between the optimal function $g_{optimal}(x_i)$ and the optimized function $g_{\mathbf{a}_4}(x_i)$ as

$$MSE_q = \frac{\sum_{i=1}^{40} (g_{optimal}(x_i) - g_{\mathbf{a}_4}(x_i))^2}{40} = 2.2605 \times 10^{-6}$$

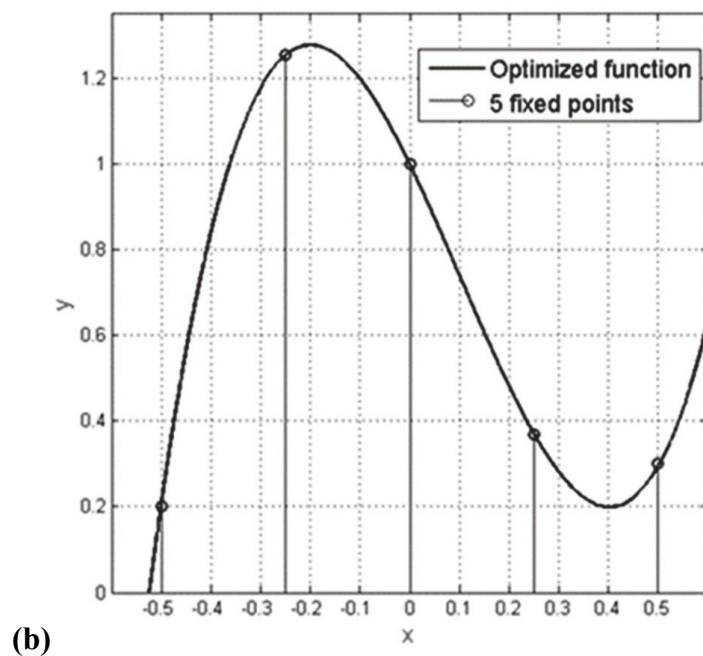
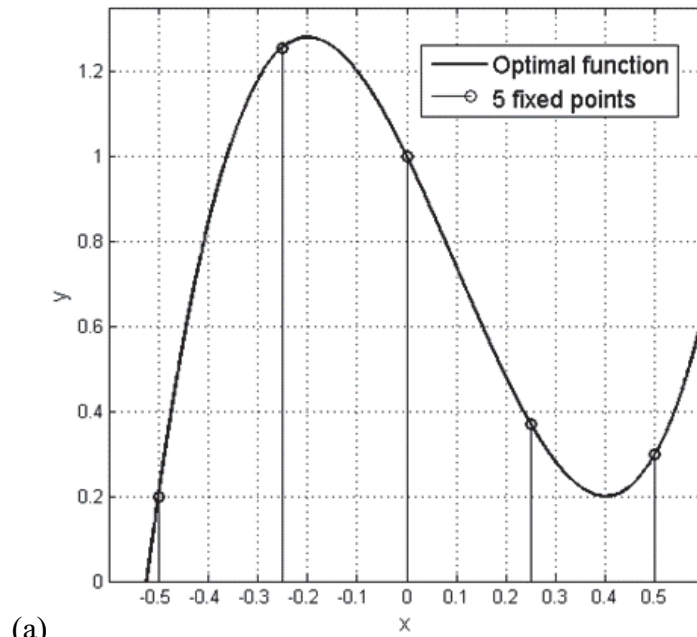


Figure E14.3.1 (a) Optimal function; (b) Optimized function at generation 4681.

The MSE_q value obtained indicates a satisfactory optimized solution for the interpolation problem has been obtained.

Depending on the requirements of an application, a smaller or larger value of the convergence criterion δ could be used.

Design Example 14.3 (Joint-objective 2-variable quadratic function minimization by DE)

Apply the differential evolution to search the optimal vector consisting of the two elements $\mathbf{x} = [x_1, x_2]$ with values that solve the following system of two-variable equations as given by

$$\begin{cases} 3x_1 + x_2 = 1 \\ x_1 + 2x_2 = 4 \end{cases}$$

Given the optimal (or ideal) solution of the system of two-variable equations is $\mathbf{x}^* = [x_1 \ x_2] = [-0.4, 2.2]$. Also, the differential evolution parameters to be used are given by

$$\mathbf{x} = [x_1, x_2]; D = 2; P = 4; m = 0.7; p_c = 0.5.$$

Solution

The system of two-variable equations can be written as

$$\begin{cases} 3x_1 + x_2 - 1 = 0 \\ x_1 + 2x_2 - 4 = 0 \end{cases}$$

Define an objective function as

$$f(\mathbf{x}) = f(x_1, x_2) = (3x_1 + x_2 - 1)^2 + (x_1 + 2x_2 - 4)^2$$

Solving the system of two-variable equations is equivalent to minimizing the objective function $f(x_1, x_2)$ such that

$$f(x_1, x_2) = 0$$

The defined joint objective function $f(\mathbf{x})$ is positive and can be used directly as a cost function to be minimized for arriving at a solution for the given system of two-variable equations.

Let the convergence criterion be set such that if $f(\mathbf{x}) \leq \delta$, iteration stops. The value of the parameter δ is chosen as 10^{-5} .

If the optimal solution or its region in the search space of a problem is known, one could randomly assign a population of initial solutions within the specific region. For example, given the optimal solution $\mathbf{x} = [x_1 \ x_2] = [-0.4, 2.2]$, random initial vectors can be assigned within the following ranges, $-1 < x_1 < 1$ and $-3 < x_2 < 3$.

Generation 1

Assume 4 ($= P$) initial vectors are randomly assigned as

$\mathbf{x}_1$	[0.5, 0.9]
$\mathbf{x}_2$	[-0.3, -0.8]
$\mathbf{x}_3$	[-0.9, -0.7]
$\mathbf{x}_4$	[-0.2, -0.1]
$\sum_{p=1}^4 f(\mathbf{x}_p)$	128.25

Compute the sum of objective function values as

$$\begin{aligned} \sum_{p=1}^4 f(\mathbf{x}_p) &= [3 \times (0.5) + (0.9) - 1]^2 + [(0.5) + 2 \times (0.9) - 4]^2 \\ &+ [3 \times (-0.3) + (-0.8) - 1]^2 + [(-0.3) + 2 \times (-0.8) - 4]^2 \\ &+ [3 \times (-0.9) + (-0.7) - 1]^2 + [(-0.9) + 2 \times (-0.7) - 4]^2 \\ &+ [3 \times (-0.2) + (-0.1) - 1]^2 + [(-0.2) + 2 \times (-0.1) - 4]^2 = 128.25 \end{aligned}$$

Mutation

Adopt two randomly chosen $\mathbf{x}_{r1}$ and $\mathbf{x}_{r2}$ to compute $\mathbf{y}_p = \mathbf{x}_p + m(\mathbf{x}_{r1} - \mathbf{x}_{r2})$ for p in $\{1, 2, 3, 4\}$ as

p	$r1$	$r2$	$\mathbf{y}_p$ equation	$\mathbf{y}_p$ value
1	2	4	$\mathbf{y}_1 = \mathbf{x}_1 + m(\mathbf{x}_2 - \mathbf{x}_4)$	[0.43, 0.41]
2	4	3	$\mathbf{y}_2 = \mathbf{x}_2 + m(\mathbf{x}_4 - \mathbf{x}_3)$	[0.19, -0.38]

3	1	2	$y_3 = x_3 + m(x_1 - x_2)$	$[-0.34, 0.49]$
4	3	1	$y_4 = x_4 + m(x_3 - x_1)$	$[-1.18, -1.22]$

Crossover

Select $z_p = y_p$ if $rand_p \leq p_c$ or $z_p = x_p$ if $rand_p > p_c$ for $p = \{1, 2, 3, 4\}$ as

p	Random value	$z_p = x_p$ or y_p	z_p value
1	$rand_1 = 0.07929$	$z_1 = y_1$	$[0.43, 0.41]$
2	$rand_2 = 0.92924$	$z_2 = x_2$	$[-0.3, -0.8]$
3	$rand_3 = 0.09393$	$z_3 = y_3$	$[-0.34, 0.49]$
4	$rand_4 = 0.83201$	$z_4 = x_4$	$[-0.2, -0.1]$

Selection

Compute the sum of the objective function values of the z_p population

$$\begin{aligned}
 \sum_{p=1}^4 f(z_p) &= [3 \times (0.43) + 0.41 - 1]^2 + [(0.43) + 2 \times (0.41) - 4]^2 \\
 &+ [3 \times (-0.3) + (-0.8) - 1]^2 + [(-0.3) + 2 \times (-0.8) - 4]^2 \\
 &+ [3 \times (-0.34) + (0.49) - 1]^2 + [(-0.34) + 2 \times (0.49) - 4]^2 \\
 &+ [3 \times (-0.2) + (-0.1) - 1]^2 + [(-0.2) + 2 \times (-0.1) - 4]^2 = 86.03
 \end{aligned}$$

Since $\min \{\sum_{p=1}^4 f(x_p), \sum_{p=1}^4 f(z_p)\} = \min \{128.25, 86.03\} = 86.03$, select z_p for p in $\{1, 2, 3, 4\}$ for generation 2.

Generation 2

x_1	$[0.43, 0.41]$
x_2	$[-0.3, -0.8]$
x_3	$[-0.34, 0.49]$
x_4	$[-0.2, -0.1]$

$\sum_{p=1}^4 f(\mathbf{x}_p)$	86.03
--------------------------------	-------

Mutation

Adopt two randomly chosen $\mathbf{x}_{r1}$ and $\mathbf{x}_{r2}$ to compute $\mathbf{y}_p = \mathbf{x}_p + m(\mathbf{x}_{r1} - \mathbf{x}_{r2})$ for p in $\{1, 2, 3, 4\}$ as

p	$r1$	$r2$	$\mathbf{y}_p$ equation	$\mathbf{y}_p$ value
1	3	2	$\mathbf{y}_1 = \mathbf{x}_1 + m(\mathbf{x}_3 - \mathbf{x}_2)$	[0.402, 1.313]
2	1	3	$\mathbf{y}_2 = \mathbf{x}_2 + m(\mathbf{x}_1 - \mathbf{x}_3)$	[0.239, -0.856]
3	1	2	$\mathbf{y}_3 = \mathbf{x}_3 + m(\mathbf{x}_1 - \mathbf{x}_2)$	[0.171, 1.337]
4	2	3	$\mathbf{y}_4 = \mathbf{x}_4 + m(\mathbf{x}_2 - \mathbf{x}_3)$	[-0.172, -1.003]

Crossover

Select $\mathbf{z}_p = \mathbf{y}_p$ if $rand_p \leq p_c$ or $\mathbf{z}_p = \mathbf{x}_p$ if $rand_p > p_c$ for $p = \{1, 2, 3, 4\}$ as

p	Random value	$\mathbf{z}_p = \mathbf{x}_p$ or $\mathbf{y}_p$	$\mathbf{z}_p$ value
1	$rand_1 = 0.77566$	$\mathbf{z}_1 = \mathbf{x}_1$	[0.43, 0.41]
2	$rand_2 = 0.52152$	$\mathbf{z}_2 = \mathbf{x}_2$	[-0.3, -0.8]
3	$rand_3 = 0.47374$	$\mathbf{z}_3 = \mathbf{y}_3$	[0.171, 1.337]
4	$rand_4 = 0.86317$	$\mathbf{z}_4 = \mathbf{x}_4$	[-0.2, -0.1]

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population

$$\begin{aligned}
 \sum_{p=1}^4 f(\mathbf{z}_p) &= [3 \times (0.43) + (0.41) - 1]^2 + [(0.43) + 2 \times (0.41) - 4]^2 \\
 &+ [3 \times (-0.3) + (-0.8) - 1]^2 + [(-0.3) + 2 \times (-0.8) - 4]^2 \\
 &+ [3 \times (0.171) + (1.337) - 1]^2 + [(0.171) + 2 \times (1.337) - 4]^2 \\
 &+ [3 \times (-0.2) + (-0.1) - 1]^2 + [(-0.2) + 2 \times (-0.1) - 4]^2 = 74.459
 \end{aligned}$$

Since $\min \left\{ \sum_{p=1}^4 f(\mathbf{x}_p), \sum_{p=1}^4 f(\mathbf{z}_p) \right\} = \min\{86.03, 74.459\} = 74.459$, select $\mathbf{z}_p$ for p in $\{1, 2, 3, 4\}$ for generation 3.

Generation 3

$\mathbf{x}_1$	[0.43, 0.41]
$\mathbf{x}_2$	[-0.3, -0.8]
$\mathbf{x}_3$	[0.171, 1.337]
$\mathbf{x}_4$	[-0.2, -0.1]
$\sum_{p=1}^4 f(\mathbf{x}_p)$	74.459

Mutation

Adopt two randomly chosen $\mathbf{x}_{r1}$ and $\mathbf{x}_{r2}$ to compute $\mathbf{y}_p = \mathbf{x}_p + m(\mathbf{x}_{r1} - \mathbf{x}_{r2})$ for p in $\{1, 2, 3, 4\}$ as

p	$r1$	$r2$	$\mathbf{y}_p$ equation	$\mathbf{y}_p$ value
1	4	2	$\mathbf{y}_1 = \mathbf{x}_1 + m(\mathbf{x}_4 - \mathbf{x}_2)$	[0.5, 0.9]
2	3	4	$\mathbf{y}_2 = \mathbf{x}_2 + m(\mathbf{x}_3 - \mathbf{x}_4)$	[-0.0403, 0.206]
3	4	2	$\mathbf{y}_3 = \mathbf{x}_3 + m(\mathbf{x}_4 - \mathbf{x}_2)$	[0.241, 1.827]
4	2	1	$\mathbf{y}_4 = \mathbf{x}_4 + m(\mathbf{x}_2 - \mathbf{x}_1)$	[-0.711, -0.947]

Crossover

Select $\mathbf{z}_p = \mathbf{y}_p$ if $\text{rand}_p \leq p_c$ or $\mathbf{z}_p = \mathbf{x}_p$ if $\text{rand}_p > p_c$ for $p = \{1, 2, 3, 4\}$ as

p	Random value	$\mathbf{z}_p = \mathbf{x}_p$ or $\mathbf{y}_p$	$\mathbf{z}_p$ value
1	$\text{rand}_1 = 0.24847$	$\mathbf{z}_1 = \mathbf{y}_1$	[0.5, 0.9]
2	$\text{rand}_2 = 0.25192$	$\mathbf{z}_2 = \mathbf{y}_2$	[-0.0403, 0.206]
3	$\text{rand}_3 = 0.96111$	$\mathbf{z}_3 = \mathbf{x}_3$	[0.171, 1.337]
4	$\text{rand}_4 = 0.85702$	$\mathbf{z}_4 = \mathbf{x}_4$	[-0.2, -0.1]

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population as

$$\begin{aligned} \sum_{p=1}^4 f(\mathbf{z}_p) &= [3 \times (0.5) + (0.9) - 1]^2 + [(0.5) + 2 \times (0.9) - 4]^2 \\ &+ [3 \times (-0.0403) + (0.206) - 1]^2 + [(-0.0403) + 2 \times (0.206) - 4]^2 \\ &+ [3 \times (0.171) + (1.337) - 1]^2 + [(0.171) + 2 \times (1.337) - 4]^2 \\ &+ [3 \times (-0.2) + (-0.1) - 1]^2 + [(-0.2) + 2 \times (-0.1) - 4]^2 = 43.160 \end{aligned}$$

Since $\min \{ \sum_{p=1}^4 f(\mathbf{x}_p), \sum_{p=1}^4 f(\mathbf{z}_p) \} = \min \{ 74.459, 43.160 \} = 43.160$, select $\mathbf{z}_p$ for p in $\{1, 2, 3, 4\}$ for generation 4.

...

Generation 1118 (DE optimization converged)

$\mathbf{x}_1$	$[-0.40012, 2.1989]$
$\mathbf{x}_2$	$[-0.40035, 2.2005]$
$\mathbf{x}_3$	$[-0.39989, 2.2001]$
$\mathbf{x}_4$	$[-0.40024, 2.2006]$
$\sum_{p=1}^4 f(\mathbf{x}_p)$	9.247×10^{-6}

As $\sum_{p=1}^4 f(\mathbf{x}_p) \leq \delta = 10^{-5}$, the algorithm stops. It is noted that the sum of fitness has been reduced from an initial value of 128.25 to 9.247×10^{-6} . It can also be seen that all the four population members are close to the optimal (or ideal) solution $\mathbf{x} = [x_1 \ x_2] = [-0.4, 2.2]$ of the system of two-variable equations.

Chapter 15

Particle Swarm Optimization

15.1 Introduction

Swarm intelligence is a study of collective behaviour in self-organized and decentralized systems which are intended to emulate artificial intelligence.

Swarm intelligence systems are typically made up of a population of simple particles interacting with one another locally and around their surroundings.

Examples of such systems can be found in nature, including ant colonies, animal herding, bird flocking, fish schooling, and bacteria molding.

Particle swarm optimization (PSO) is a type of swarm intelligence. In a swarm of particles, particles behave like social beings where they are independent and dependent at the same time.

Each of these particles tends to improve by imitating the better qualities of its neighbouring particles.

These particles are self-centred and don't care much about anything else. By conventional theories, this should not really work; but surprisingly this decentralized system works well for a population.

In this chapter, the principles of particle swarm optimization are explained in Sections 15.1 to 15.3.

In Sections 15.4 and 15.5, the continuous model and the binary model of the particle swarm optimization are respectively described and their pseudocodes are given.

Six different examples are given to show the applications of particle swarm optimization to both continuous and discrete problems.

15.2 Social Cognition and Particle Swarm

The particle swarm is an algorithm based on social cognitive theory.

To have an understanding on the idea of particle swarm optimization, it is necessary to have some knowledge of how the social cognitive theory works which is based on social and cognitive behaviour of a particle in a structure.

Social cognitive theory of any society is based on three principles: Evaluate, Compare, Imitate.

These three principles allow a particle to evaluate, compare, and imitate all particles (including itself) within its own and other environments.

Evaluate: A particle conceives a unique behavioural characteristic of having a tendency to evaluate stimuli and rate them as positive or negative, like or dislike.

Through this learning process, a particle improves its environment evaluation from observing the changes.

Compare: A particle in a society uses others as models for comparing itself in the society to serve as a motivation to learn and adjust.

Imitate: A particle comparing itself to its neighbouring particles can imitate a better being for improved performances.

True imitation only occurs if a particle is able to understand what is imitating or knows its purpose.

In particle swarm optimization, although these three principles are explained separately, these principles are connected to one another and behave like an interactive process.

In particle swarm optimization, an algorithm is implemented as a computer program to solve a problem or to realize a complex environment.

Different models of particle swarm optimization have been developed. Among which, three basic models are described below: The

continuous number model [Ken0100], the binary number model [Ken0100], [Ken9710], and hybrid models.

15.3 Choices of Local and Global Neighborhoods

Particles are connected to topological neighbours, and neighbours tend to cluster in the same region of the search space. Physical proximity indicates neighbouring particles share similar qualities.

These particles may move toward one another to influence agreement.

Constantly moving particles in a search space will dynamically interact toward a direction of movement as a function of current position, current velocity, location of previous success of a particle, and locations of previous successes of neighbouring particles.

Random numbers are used for transition, so for each generation, the exact location of a particle will change.

In the particle swarm optimization, to enable a particle to move in a desired direction, two pieces of information are available, one from its past performance and another one from other particles' past performances.

These information can be derived from the LBest $\mathbf{p}_p(t)$ for each particle within a local neighborhood, and the GBest $\mathbf{p}_g(t)$ for each particle within a global neighborhood.

A choice of local and global neighborhoods for particle updates can affect the performances of particle swarm optimization with continuous and discrete parameters and is likely problem-dependent.

In practice, there are different choices of local neighborhood and different choices of global neighborhood to be combined to form a set of local and global neighborhoods tailor-made for an optimization problem.

The choices apply to both the continuous particle swarm optimization and the discrete particle swarm optimization. A list of typical choices are shown in Table 15.3.1.

Table 15.3.1 Choices of local and global neighborhoods.

	Choice	Description
LBest	1	The local neighborhood of a particle p includes only itself and the LBest particle is selected from its best in all generations.
	2	The local neighborhood of a particle p includes itself and its nearest (P_L-1) -neighbouring particles, and the LBest particle is selected from the best among all involved particles in all generations. For $P_L - 1 = 1$, the LBest is selected from particles $p - 1, p, p + 1$. For $P_L - 1 = 0$, the choice 2 is the same as the choice 1.
GBest	1	The global neighborhood of a particle p includes itself and its nearest (P_G-1) -neighbouring particles, and the GBest is selected from the best among all the involved particles in all generations. For $P_G - 1 = 2$, the GBest is selected from particles $p - 2, p - 1, p, p + 1, p + 2$. Note that $(2P_G - 1) \leq P$.
	2	The global neighborhood of a particle p includes all the particles in a population, and the GBest is selected from the best among all the particles in all generations.
Note that best particle means the one with the largest fitness for maximization problems (or the smallest error for minimization problems).		

15.4 Continuous-valued Particle Swarm Optimization

15.4.1 Continuous number model

Table 15.4.1 summarizes the notations for the particle swarm optimization continuous number model.

Table 15.4.1 Notations for PSO continuous number model.

Symbol	Description
P	Number of particles in a population.
P_L	Number of particles in a local neighborhood of particle p including itself ($2P_L - 1 < P$).
P_G	Number of particles in a global neighborhood of particle p including itself ($2P_L - 1 < 2P_G - 1 \leq P$).
N_c	Number of distinct parameters.
D	Number of parameters in a particle, $D = N_c$.
N	Number of inputs for training.
T	Number of iterations, $t = 1$ to T .
t	Iteration index.
$x_{pd}(t)$	Parameter value of d th parameter (for $d = 1$ to D) of a particle p (for $p = 1$ to P).
$\mathbf{x}_p(t)$	Parameter vector of p th particle, $\mathbf{x}_p(t) = [x_{p1}(t), x_{p2}(t), x_{p3}(t), \dots, x_{pD}(t)]$.
$\varphi(\mathbf{x}_p(t))$	Objective function of p th particle $\mathbf{x}_p(t)$.
$v_{pd}(t)$	Velocity or incremental change ($= \Delta x_{pd}(t)$) of $x_{pd}(t)$.
$\mathbf{v}_p(t)$	Velocity of p th particle, $\mathbf{v}_p(t) = [v_{p1}(t), v_{p2}(t), v_{p3}(t), \dots, v_{pD}(t)]$.
$w(t)$	Inertia weight.
w_1, w_2	Maximum and minimum inertia weights.
$\rho_{pd}(t)$	Local best (LBest) within a small neighbour of a particle p of a generation (or all generations).
$\boldsymbol{\rho}_p(t)$	LBest of p th particle, $\boldsymbol{\rho}_p(t) = [\rho_{p1}(t), \rho_{p2}(t), \rho_{p3}(t), \dots, \rho_{pD}(t)]$.
$\rho_{ga}(t)$	Global best (GBest) among all the particles of a generation (or all generations).
$\boldsymbol{\rho}_g(t)$	GBest of p th particle,

	$\rho_g(t) = [\rho_{g1}(t), \rho_{g2}(t), \rho_{g3}(t), \dots, \rho_{gD}(t)]$.
φ_1, φ_2	Acceleration coefficients which are positive random numbers uniformly distributed with a predefined upper limit.
V_{max}	Maximum velocity limit of a particle.

For minimization problems the local best (LBest) and the global best (GBest) are updated at the start of each iteration as

$$\rho_p(t+1) = \begin{cases} \mathbf{x}_p(t) & \text{if } \varphi(\mathbf{x}_p(t)) \leq \varphi(\rho_p(t)) \\ \rho_p(t) & \text{otherwise} \end{cases} \quad (15.4.1)$$

$$\rho_g(t+1) = \rho_q(t+1) |_{q=\arg \min_{p=1 \text{ to } P} \varphi(\rho_p(t+1))} \quad (15.4.2)$$

Let the position of a particle be denoted by $x_{pd}(t)$ and the change of the position of a particle be represented by $\Delta x_{pd}(t)$; for simplicity, $\Delta x_{pd}(t)$ can be expressed as its velocity which is denoted by $v_{pd}(t)$.

Velocity is a vector that is added to the position coordinates in order to move the particle from one to another time step (or iteration or generation).

To adjust an individual's position toward the successes of the individual and its community, an update equation is constructed for each $v_{pd}(t+1)$ at time step (or iteration or generation) $t+1$ as a sum of its weighted current velocity $v_{pd}(t)$, and the weighted differences between the individual's current position $x_{pd}(t)$ and the respective local best $\rho_{pd}(t)$ and global best $\rho_{gd}(t)$.

If no knowledge is available on whether individual-learning or social-influence should be stronger, two positive random numbers (φ_1, φ_2) uniformly distributed with a predefined upper limit are used as weights.

$$\begin{aligned} \mathbf{v}_p(t+1) = & w(t) \mathbf{v}_p(t) + \varphi_1 [\rho_p(t) - \mathbf{x}_p(t)] \\ & + \varphi_2 [\rho_g(t) - \mathbf{x}_p(t)] \end{aligned} \quad (15.4.3)$$

$$\mathbf{x}_p(t+1) = \mathbf{x}_p(t) + \mathbf{v}_p(t+1) \quad (15.4.4)$$

where

$$\varphi_m = c_m \times rand \text{ for } m = 1, 2 \quad (15.4.5)$$

The parameters c_1 (self adjustment weight) and c_2 (social adjustment weight) are preset constants of about 1.49, and $rand$ is a random variable evenly distributed within $[0,1]$ such that $\varphi_1 + \varphi_2 < 3$.

The value of the inertia weight $w(t)$ [Shi9805] is linearly decreasing from w_1 to w_2 for t increasing from 1 to T such that for $t = 1$, $w(k) \rightarrow w_1$; for $t = T$, $w(k) \rightarrow w_2$; and $w_1 (= 1.1) > w_2 (= 0.1)$.

$$w(k) = w_1 - (w_1 - w_2) \times t/T \quad (15.4.6)$$

The system given so far could be an unstable one for having a tendency to oscillate and becomes unstable.

To achieve a somewhat controlled search, the maximum velocity of a particle is limited to a certain value of V_{max} in each of its parameters $d = 1$ to D as

$$\text{if } v_{pd}(t) > V_{max} \text{ then } v_{pd}(t) = s_c(v_{pd}(t)) = V_{max} \quad (15.4.7a)$$

$$\text{if } v_{pd}(t) < -V_{max} \text{ then } v_{pd}(t) = s_c(v_{pd}(t)) = -V_{max} \quad (15.4.7b)$$

If the objective function $\varphi(\mathbf{x}_p(t))$ is to be minimized, given a positive constant parameter ε , the alternative fitness function $F_p(t)$ to be maximized is defined as

$$F_p(t) = \frac{1}{\varepsilon + \varphi(\mathbf{x}_p(t))} \quad (15.4.8)$$

15.4.2 Bipolar sigmoid function as saturating function

A close-form expression for (15.4.7a) and (15.4.7b) can be obtained through the use of the bipolar sigmoid function as

$$s_g[v_{pd}(t)] = V_{max} \left[\frac{1 - e^{-r_g v_{pd}(t)}}{1 + e^{-r_g v_{pd}(t)}} \right] \quad (15.4.9)$$

for $v_{pd}(t) \in [-V_{max}, V_{max}]$

In (15.4.9), the parameter r_g determines the slope of the transition region between $-V_{max} < v_{pd}(t) < V_{max}$ in which the function $s_g[v_{pd}(t)]$ saturates to $\pm V_{max}$.

For having a unity slope at $v_{pd}(t) = 0$, $r_g = 2$ for (15.4.9).

By scaling the bipolar sigmoid function by a factor of V_{max} , the range of $s_g[v_{pd}(t)]$ becomes $\pm V_{max}$.

Population initialization may be done by choosing random positions within their lower and upper bounds and random velocities between $-V_{max} \leq v_{pd}(t) \leq V_{max}$.

The relationship between the slope parameter values r_g for $r_g = [2, 1, 0.5, 0.25, 0.125]$ and the ranges of velocity value $-V_{max} \leq v_{pd}(t) \leq V_{max}$ of the bipolar sigmoid function is listed in Table 15.4.2 and plotted in Figure 15.4.1.

Table 15.4.2 Relationship between slope parameter r_g and range of $v_{pd}(t)$ for bipolar sigmoid function s_g .

r_g	2	1	0.5	0.25	0.125
$s_g[v_{pd}(t)]$ $= \pm 0.9V_{max}$	$v_{pd}(t)$				
	± 1.472	± 2.944	± 5.888	± 11.780	± 23.555

From Table 15.4.2, for a range of $\pm V_{max} = \pm 1.472$, an appropriate slope value is $r_g = 2$.

In practice, a simple way to implement the bipolar sigmoid function is described in [Kwa9207] with a digital hardware complexity of an order of one binary multiplication.

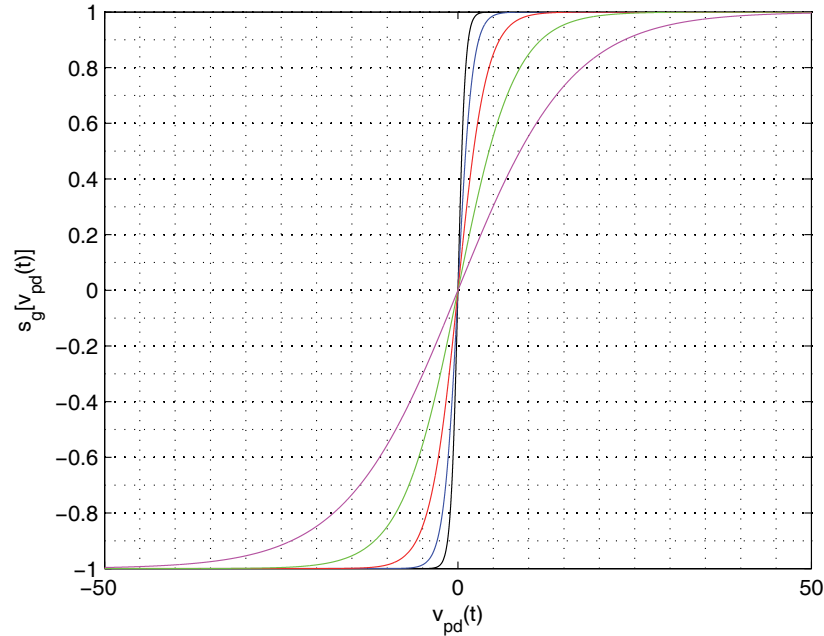


Figure 15.4.1 Bipolar sigmoid function with slope parameter $r_g = [2, 1, 0.5, 0.25, 0.125]$ correspond to black, blue, red, green, and magenta color curve respectively.

The pseudocode of a particle swarm optimization algorithm with the LBest of a particle selected from $(P_L - 1)$ -neighbouring particles, and the GBest of a particle selected from $(P_G - 1)$ -neighbouring particles is listed in Table 15.4.3.

Table 15.4.3 Pseudocode for PSO continuous number model (E: Evaluate; C: Compare; I: Inmitate).

Pseudocode		Comment	
1	Loop	Generation t	
2	for $p = 1$ to P	P is population size	
3	Compute output, $f_{p,t}(x_n)$ for $n = 1$ to N ;		E
4	Compute output $error_p(t+1)$;	Error function	E
5	Compute fitness, $F_p(t + 1)$;	Fitness function	E

6	end	Next p	E
7	for $p = 1$ to P		C
8	if $F_p(t+1) > F_{\rho_p}(t)$	$F_{\rho_p}(t)$ is LBest fitness	C
9	$F_{\rho_p}(t+1) = F_p(t+1)$		C
10	end		C
11	for $i = 1$ to P_L but $i \neq p$	$P_L \ll P$	C
12	if $F_i(t+1) > F_{\rho_p}(t+1)$	$F_{\rho_p}(t)$ is LBest fitness	C
13	for $d = 1$ to D		C
14	$\rho_{pd} = x_{id}(t);$	ρ_p is LBest particle	C
15	end	Next d	C
16	end		C
17	end	Next i	C
18	if $F_p(t+1) > F_{\rho_g}(t)$	$F_{\rho_g}(t)$ is GBest fitness	C
19	$F_{\rho_g}(t+1) = F_p(t+1)$		C
20	end		C
21	for $j = 1$ to P_G but $j \neq p$	$P_G \leq P$	C
22	if $F_j(t+1) > F_{\rho_g}(t+1)$	$F_{\rho_g}(t)$ is GBest fitness	C
23	for $d = 1$ to D		C
24	$\rho_{gd} = \rho_{jd};$	ρ_g is Gbest particle	C
25	end	Next d	C
26	end		C
27	end	Next j	C
28	if $\min_{p=1 \text{ to } P} error_p(t) < \delta$ break end		E
29	for $d = 1$ to D		I
30	$v_{pd}(t+1) =$ $w(t) v_{pd}(t) + \varphi_1[\rho_{pd} - x_{pd}(t)] + \varphi_2[\rho_{gd} - x_{pd}(t)];$		I

31	$v_{pd}(t + 1) = s_g[v_{pd}(t + 1)];$	I
32	$x_{pd}(t + 1) = x_{pd}(t) + v_{pd}(t + 1);$	I
33	end	Next d
34	end	Next p
35	end	Next generation

15.5 Discrete-valued Particle Swarm Optimization

15.5.1 Binary number model

In addition to Table 15.4.1, a number of additional abbreviations for the binary particle swarm optimization (BPSO) are listed in Table 15.5.1.

Table 15.5.1 Notations for PSO binary number model.

Symbol	Description
$x_{pd}(t + 1)$	Next state of the position of particle p at the d th bit in a bit string.
$x_{pd}(t)$	Current state of the position of particle p at the d th bit in a bit string.
ρ_{pd}	Best state found within the local neighborhood of a particle p (=1 if the best success attained by any member of the local neighborhood was when x_{pd} was 1 and 0 if it was 0).
ρ_{gd}	Best state found within the global neighborhood of a particle p (=1 if the best success attained by any member of the global neighborhood was when it was in the 1 state and 0 otherwise).
B	Number of binary bits to represent a parameter.
B_i	Number of integer bits to represent a parameter.
B_f	Number of fractional bits to represent a parameter.
N_c	Number of distinct parameters in a binary PSO problem.

D	Total number of binary bits to represent a particle ($D = N_c B$).
φ_1, φ_2	Arbitrary positive random numbers uniformly distributed but often set so that the sum of the two (φ_1, φ_2) limits is 4.

What are the required changes to transform the continuous PSO algorithm such that it can work as the binary PSO for discrete-valued (or binary) optimization problems? The answer is to represent each of the particles $\mathbf{x}_p(t+1)$ by a binary bit string of length d by first constraining each of the d th parameter of the transformed velocity vector $S_f(\mathbf{v}_p(t+1))$ to have a value within the interval $[0, 1]$. The d th parameter of the transformed velocity vector $S_f(v_{pd}(t+1))$ is then used as the probability to determine each of the d th particle bit $x_{pd}(t+1)$ to attain a value of 1 such that

$$x_{pd}(t+1) = \begin{cases} 1 & \text{if } rand \leq S_f(v_{pd}(t+1)) \\ 0 & \text{otherwise} \end{cases} \quad \text{for } d = 1 \text{ to } D \quad (15.5.1)$$

As listed in Table 15.5.2 and described below, the Steps 8 to 27 of both the PSO and the BPSO are identical except the latter performs LBest and GBest updates in bit level.

Table 15.5.2 Pseudocode for PSO binary number model (C: Compare; I: Inmitate).

Pseudocode		Comment	
1	Loop	Generation t	
2	for $p = 1$ to P	P is population size	
3	Compute output, $f_{p,t}(x_n)$ for $n = 1$ to N ;		E
4	Compute output $error_p(t+1)$;	Error function	E

5	Compute fitness, $F_p(t + 1)$;	Fitness function	E
6	end	Next p	E
7	for $p = 1$ to P		C
8 to 17	For $d = 1$ to D , the d th bit of the local best of a particle p , $\rho_{pd}(t + 1)$, is set to 1 if the best success of the particle q of the defined local neighborhood occurred when $x_{qd} = 1$ or is set to 0 if the best success of the particle q of the defined local neighborhood occurred when $x_{qd} = 0$.		C
18 to 27	For $d = 1$ to D , the d th bit of the global best of a particle p , $\rho_{gd}(t + 1)$, is set to 1 if the best success of any particle q of the defined global neighborhood occurred when $x_{qd} = 1$ or is set to 0 if the best success of the particle q of the defined global neighborhood occurred when $x_{qd} = 0$.		C
28	if $\min_{p=1 \text{ to } P} error_p(t) < \delta$ break end		E
29	for $d = 1$ to D		I
30	$v_{pd}(t + 1) =$ $w(t) v_{pd}(t) + \varphi_1[\rho_{pd} - x_{pd}(t)] + \varphi_2[\rho_{gd} - x_{pd}(t)];$		I
31	$v_{pd}(t + 1) = s_g[v_{pd}(t + 1)];$		I
32	if $r_{pd} < s_f[v_{pd}(t + 1)]$ $x_{pd}(t + 1) = 1;$ else $x_{pd}(t + 1) = 0;$ end		I
33	end	Next d	I
34	end	Next p	
35	end	Next generation	

Key: $r_{pd} \in [0,1]$ is a uniformly distributed random number.

During each iteration, the local best (LBest) $\rho_{pd}(t + 1)$ and the global best (GBest) $\rho_{gd}(t + 1)$ of each bit of each particle are updated as follows:

The d th bit of the local best of a particle p , $\rho_{pd}(t + 1)$, is set to 1 if the best success of the particle q of a defined local neighborhood occurred when $x_{qd} = 1$ or is set to 0 if the best success of the particle q of the defined local neighborhood occurred when $x_{qd} = 0$.

The d th bit of the global best of a particle p , $\rho_{gd}(t + 1)$, is set to 1 if the best success of any particle q of a defined global neighborhood occurred when $x_{qd} = 1$ or is set to 0 if the best success of the particle q of the defined global neighborhood occurred when $x_{qd} = 0$.

Using the notations listed in Table 15.5.1, a formula for binary decision is constructed for each $v_{pd}(t + 1)$ at time $t + 1$ as a function of its current velocity $v_{pd}(t)$, and the differences between the particle's current state $x_{pd}(t)$ and the respective best state found so far by itself ρ_{pd} and by its neighbours ρ_{gd} as

$$v_{pd}(t + 1) = w(t) v_{pd}(t) + \varphi_1 [\rho_{pd} - x_{pd}(t)] + \varphi_2 [\rho_{gd} - x_{pd}(t)]$$

$$\varphi_m = c_m \times rand \text{ for } m = 1, 2 \quad (15.5.2)$$

The parameters c_1 and c_2 are preset constants of about 1.49; $rand$ is a random variable evenly distributed within $[0, 1]$.

The upper limit of the two uniformly distributed positive random numbers φ_1, φ_2 is arbitrary but often set so that the sum of $\varphi_1 + \varphi_2 < 3$. The range of the decreasing inertia weight $w(t)$ is $[1.1, 0.1]$

To control the current probability of particle p at the d th bit in a bit string to decide 1 to be within an appropriate range $[-V_{max}, +V_{max}]$, the bipolar sigmoid function is applied such that

$$v_{pd}(t+1) = s_g[v_{pd}(t+1)] \quad (15.5.3)$$

and

$$s_g[v_{pd}(t+1)] = V_{max} \left[\frac{1 - e^{-r_g v_{pd}(t+1)}}{1 + e^{-r_g v_{pd}(t+1)}} \right]$$

for $v_{pd}(t+1) \in [-V_{max}, V_{max}]$ (15.5.4)

Also, a particle's probability of change $x_{pd}(k)$ for $d = 1$ to D is now updated using (15.5.5) as

$$x_{pd}(t+1) = \begin{cases} 1 & \text{if } r_{pd} < s_f[v_{pd}(t+1)] \\ 0 & \text{otherwise} \end{cases} \quad (15.5.5)$$

The parameter r_{pd} is a random variable evenly distributed within $[0, 1]$. The parameter $v_{pd}(t+1)$ measures a particle's probability of deciding 1 which is being used to determine a probability threshold and can be implemented by a binary sigmoid function $s_f[v_{pd}(t+1)]$ as

$$s_f[v_{pd}(t+1)] = \frac{1}{1 + e^{-r_f v_{pd}(t+1)}} \text{ for } v_{pd}(t) \in [-V_{max}, +V_{max}]$$

(15.5.6)

The function in (15.5.6) transforms the lower and upper limits $[-V_{max}, +V_{max}]$ of $v_{pd}(t)$ obtained from (15.5.3)-(15.5.4) to $[0, 1]$.

To allow a reasonable chance for a bit to flip, the binary sigmoid function should be operating between $s_f[v_{pd}(t+1)] = 0$ and $s_f[v_{pd}(t+1)] = 1$.

In (15.5.4), the slope parameter r_f determines the slope of the transition region between $-V_{max} < v_{pd}(t) < V_{max}$ in which the function saturates to 1 or 0.

If the objective function $\varphi(\mathbf{x}_p(t+1))$ is to be minimized, the alternative fitness function $F_p(t+1)$ to be maximized is defined as

$$F_p(t+1) = \frac{1}{\varepsilon + \varphi(\mathbf{x}_p(t+1))} \quad (15.5.7)$$

In (15.5.7), the parameter ε denotes a small positive constant. The pseudocode for the binary particle swarm algorithm is shown in Table 15.5.2.

15.5.2 Binary sigmoid function as saturating function

In (15.5.6), r_f determines the slope of the transition region between $-V_{max} < v_{pd}(t) < V_{max}$ in which the function $s_f[v_{pd}(t)]$ saturates to $[0, 1]$.

Particle initialization in a population could be carried out by choosing random 0 or 1 bit for $x_{pd}(t)$ and random velocities between $-V_{max} \leq v_{pd}(t) \leq V_{max}$.

The relationship between the slope parameter values r_f for $r_f = [2, 1, 0.5, 0.25, 0.125]$ and the ranges of velocity value $-V_{max} \leq v_{pd}(t) \leq V_{max}$ of the binary sigmoid function is listed in Table 15.5.3 and plotted in Figure 15.5.1.

Table 15.5.3 Relationship between slope parameter r_f and range of $v_{pd}(t)$ for binary sigmoid function s_f .

r_f	2	1	0.5	0.25	0.125
$s_f[v_{pd}(t)]$ $= \pm 0.95V_{max}$	$v_{pd}(t)$				
	± 1.472	± 2.944	± 5.888	± 11.780	± 23.555

From Table 15.5.3, for a range of $\pm V_{max} = \pm 1.472$, an appropriate slope parameter value is $r_f = 2$.

In practice, a simple way to implement the binary sigmoid function is described in [Kwa9207] with a digital hardware complexity of an order of one binary multiplication.

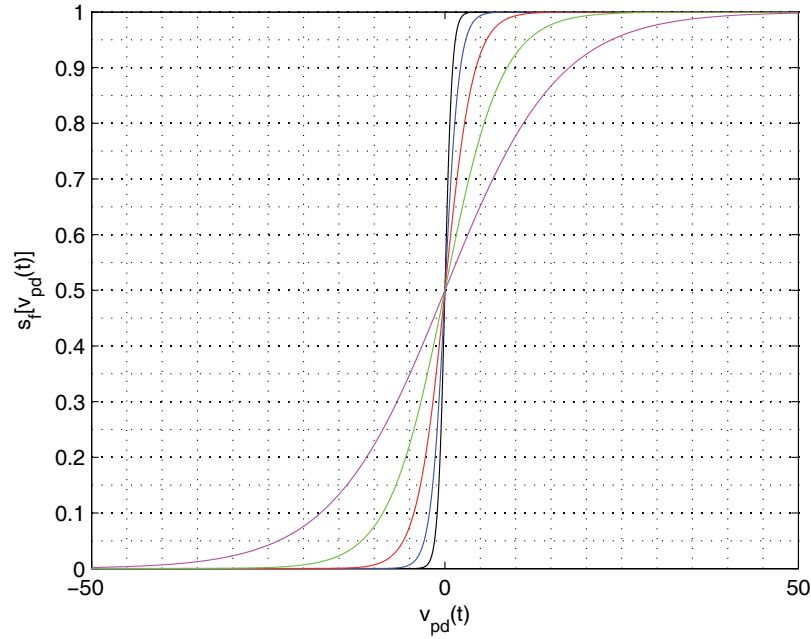


Figure 15.5.1 Binary sigmoid function with slope parameter $r_f = [2, 1, 0.5, 0.25, 0.125]$ correspond to black, blue, red, green, and magenta color curve respectively.

Design Example 15.1 (Exponential function interpolation by PSO)

Consider the use of the particle swarm optimization to determine the optimal set of continuous coefficients $a = 4.4$, $b = 0.25$, $c = -6.8$, of an exponential function

$$y(x) = ae^{bx} + c$$

to fit N number of input and output data pairs.

For simplicity, a population size of four (4) is taken and the calculation steps of first three generations plus the last generation are shown.

In the PSO, the parameters c_1 and c_2 are chosen to be 1 and the term ‘*rand*’ represents a random value between 0 and 1.

The error function is taken to be the mean squared error difference between the output of the target function and the output from each particle.

The fitness function is defined by the inverse of the sum of the error function and a small positive constant.

The local best (LBest) ρ_{pd} of a particle is taken as the best of the individual particle in all generations.

Due to the small size of the population, the global best (GBest) ρ_{gd} is taken as the global best which is the best of all the particles in all generations.

The particle velocity and position updates within each generation are carried out by implementing the following two equations in which $x_{pd}(t)$ denotes the position and $v_{pd}(t)$ denotes velocity of the parameter d of a particle p at iteration (or generation) t .

$$\begin{aligned}
 &v_{pd}(t+1) \\
 &= w(t) v_{pd}(t) + \varphi_1 [\rho_{pd}(t) - x_{pd}(t)] + \varphi_2 [\rho_{gd}(t) - x_{pd}(t)] \\
 &v_{pd}(t+1) = s_c [v_{pd}(t+1)] \in [-V_{max}, V_{max}] \\
 &x_{pd}(t+1) = x_{pd}(t) + v_{pd}(t+1)
 \end{aligned}$$

where $w(t) = 1$; $\varphi_1 = c_1 \times rand(1)$; and $\varphi_2 = c_2 \times rand(2)$.

For generation t , the error function of a particle p to be minimized is defined as the mean squared error (MSE) $error_p(t)$ between the desired output $\mathbf{d}_t(\mathbf{x})$ and the obtained output $\mathbf{f}_{p,t}(\mathbf{x})$ due to an input vector $\mathbf{x} (= [x_1, x_2, \dots, x_N])$ such that

$$error_p(t) = \frac{1}{N} \sum_{n=1}^N [d_t(x_n) - f_{p,t}(x_n)]^2$$

If the input $\mathbf{x}$ is kept the same in all generations, the desired output $\mathbf{d}_t(\mathbf{x})$ remains unchanged and therefore $\mathbf{d}_t(\mathbf{x}) = \mathbf{d}(\mathbf{x})$.

The particle's fitness function is defined as the inverse of the mean squared error plus a small positive constant ε to avoid a zero-valued denominator. Hence the fitness function $F_p(t)$ to be maximized becomes

$$F_p(t) = \frac{1}{\varepsilon + error_p(t)}$$

During the generation t , the best among all the particles GBest is defined as the particle in record that has the largest fitness or the smallest mean squared error as

$$GBest(t) = \max_{F_p} \{\mathbf{x}_p\} \text{ or } GBest(t) = \min_{error_p} \{\mathbf{x}_p\}$$

Define a small positive error limit (or convergence criterion) δ such that if $error_{GBest}(t) \leq \delta$, then the algorithm stops, and it could be assumed that a satisfying solution has been obtained in the generation t .

Solution

Initialization

$$c_1 = 1$$

$$c_2 = 1$$

$$\varepsilon = 0.01$$

$$\text{Convergence criterion: } \delta = 1 \times 10^{-4}$$

$$\text{Target function: } y(x) = ae^{bx} + c, \quad a = 4.4, \quad b = 0.25, \quad c = -6.8$$

$$\text{Number of particles (population size), } P = 4$$

$$\text{Number of parameters in a particle, } D = 3$$

$$\text{Initializing GBest particle} = \mathbf{0}$$

$$\text{Initializing GBest fitness} = 0$$

$$\text{Number of fixed inputs, } N = 3$$

$$V_{max} = 10$$

At the start, a set of three ($N = 3$) randomly selected fixed inputs $\mathbf{x} = [x_1, x_2, x_3] = [0.4, 7.6, 12.5]$ are to be used in all generations.

The corresponding three desired outputs $\mathbf{d} = [d(x_1), d(x_2), d(x_3)]$ are

$$d(0.4) = 4.4 \times e^{0.25 \times 0.4} - 6.8 = -1.9373$$

$$d(7.6) = 4.4 \times e^{0.25 \times 7.6} - 6.8 = 22.6179$$

$$d(12.5) = 4.4 \times e^{0.25 \times 12.5} - 6.8 = 93.3435$$

In other words, the desired outputs for all the particles are $\mathbf{d} = [-1.9373, 22.6179, 93.3435]$ under fixed inputs $\mathbf{x} = [0.4, 7.6, 12.5]$.

$$\text{Initializing LBest} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\text{Initializing LBest fitness} = [0 \ 0 \ 0 \ 0]^T$$

$$\text{Initializing random population} = \begin{bmatrix} 0.75 & 0.05 & 0.01 \\ -0.26 & 0.65 & 0.80 \\ -0.47 & 0.96 & 0.79 \\ 0.86 & 0.22 & -0.99 \end{bmatrix}$$

$$\text{Initializing velocity (for each particle in the population)} = [0 \ 0 \ 0 \ 0]^T$$

Generation 1

Particle 1 output:

$$f_{1,1}(0.4) = 0.75 \times e^{0.05 \times 0.4} + 0.01 = 0.77515$$

$$f_{1,1}(7.6) = 0.75 \times e^{0.05 \times 7.6} + 0.01 = 1.10671$$

$$f_{1,1}(12.5) = 0.75 \times e^{0.05 \times 12.5} + 0.01 = 1.41118$$

$$\text{In vector format: } \mathbf{f}_{1,1}(\mathbf{x}) = [0.77515 \ 1.10671 \ 1.41118]$$

Particle 1 error:

$$\begin{aligned} error_1(1) &= \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{1,1}(x_n)]^2 \\ &= \frac{1}{3} [(-1.9373 - 0.77515)^2 + (22.6179 - 1.10671)^2 \\ &\quad + (93.3435 - 1.41118)^2] = 2973.88 \end{aligned}$$

Particle 1 fitness:

$$F_1(1) = \frac{1}{\varepsilon + error_1(k)} = \frac{1}{0.01 + error_1(1)} = 3.3626 \times 10^{-4}$$

Similarly,

$$\text{Particle 2 output: } \mathbf{f}_{2,1}(\mathbf{x}) = [0.46280, -35.5403, -877.446]$$

$$\text{Particle 2 error: } error_2(1) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{2,1}(x_n)]^2 = 315273$$

$$\text{Particle 2 fitness: } F_2(1) = 1/(\varepsilon + error_2(1)) = 3.1719 \times 10^{-6}$$

$$\text{Particle 3 output: } \mathbf{f}_{3,1}(\mathbf{x}) = [0.09997, -692.173, -76493.96]$$

Particle 3 error:

$$error_3(1) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{3,1}(x_n)]^2 = 1.95538 \times 10^9$$

$$\text{Particle 3 fitness: } F_3(1) = 1/(\varepsilon + error_3(1)) = 5.1141 \times 10^{-10}$$

$$\text{Particle 4 output: } \mathbf{f}_{4,1}(\mathbf{x}) = [-0.05089, 3.58761, 12.46266]$$

$$\text{Particle 4 error: } error_4(1) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{4,1}(x_n)]^2 = 2302.476$$

$$\text{Particle 4 fitness: } F_4(1) = 1/(\varepsilon + error_4(1)) = 4.3431 \times 10^{-4}$$

As this is the first generation, the LBest is the current population and the fitness is the current fitness.

$$\text{LBest} = \begin{bmatrix} 0.75 & 0.05 & 0.01 \\ -0.26 & 0.65 & 0.80 \\ -0.47 & 0.96 & 0.79 \\ 0.86 & 0.22 & -0.99 \end{bmatrix}$$

LBest fitness

$$= [3.363 \times 10^{-4} \quad 3.172 \times 10^{-6} \quad 5.114 \times 10^{-10} \quad 4.343 \times 10^{-4}]^T$$

Among the 4 fitness values, the particle 4 has the highest fitness value 4.343×10^{-4} . Consequently, GBest fitness is equal to the particle 4's LBest fitness.

$$\text{GBest particle} = [0.86 \quad 0.22 \quad -0.99]$$

$$\text{GBest fitness} = 4.343 \times 10^{-4}$$

With GBest, $\min error_{1 \text{ to } P}(1) = error_3(1) = 2302.48 > \delta = 1 \times 10^{-4}$, so particle updates need to be continued and more generations are required.

Particle velocity update (with $rand(1) = 0.98$, $rand(2) = 0.28$):

$$\begin{aligned} \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \mathbf{v}_3 \\ \mathbf{v}_4 \end{bmatrix} &= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \\ &+ 1 \times 0.98 \times \left(\begin{bmatrix} 0.75 & 0.05 & 0.01 \\ -0.26 & 0.65 & 0.80 \\ -0.47 & 0.96 & 0.79 \\ 0.86 & 0.22 & -0.99 \end{bmatrix} - \begin{bmatrix} 0.75 & 0.05 & 0.01 \\ -0.26 & 0.65 & 0.80 \\ -0.47 & 0.96 & 0.79 \\ 0.86 & 0.22 & -0.99 \end{bmatrix} \right) \\ &+ 1 \times 0.28 \times \left(\begin{bmatrix} 0.86 & 0.22 & -0.99 \\ 0.86 & 0.22 & -0.99 \\ 0.86 & 0.22 & -0.99 \\ 0.86 & 0.22 & -0.99 \end{bmatrix} - \begin{bmatrix} 0.75 & 0.05 & 0.01 \\ -0.26 & 0.65 & 0.80 \\ -0.47 & 0.96 & 0.79 \\ 0.86 & 0.22 & -0.99 \end{bmatrix} \right) \\ &= \begin{bmatrix} 0.03099 & 0.04789 & -0.28171 \\ 0.31552 & -0.12114 & -0.50427 \\ 0.37468 & -0.20847 & -0.50145 \\ 0 & 0 & 0 \end{bmatrix} \end{aligned}$$

Particle position update:

$$\begin{aligned}
\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \mathbf{x}_4 \end{bmatrix} &= \begin{bmatrix} 0.75 & 0.05 & 0.01 \\ -0.26 & 0.65 & 0.80 \\ -0.47 & 0.96 & 0.79 \\ 0.86 & 0.22 & -0.99 \end{bmatrix} \\
&\quad + s_c \left\{ \begin{bmatrix} 0.03099 & 0.04789 & -0.28171 \\ 0.31552 & -0.12114 & -0.50427 \\ 0.37468 & -0.20847 & -0.50145 \\ 0 & 0 & 0 \end{bmatrix} \right\} \\
&= \begin{bmatrix} 0.78099 & 0.09789 & -0.27171 \\ 0.05552 & 0.52886 & 0.29573 \\ -0.09532 & 0.75153 & 0.28855 \\ 0.86 & 0.22 & -0.99 \end{bmatrix}
\end{aligned}$$

Generation 2

From generation 1, a new population is produced. So the new particle positions are

$$\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \mathbf{x}_4 \end{bmatrix} = \begin{bmatrix} 0.78099 & 0.09789 & -0.27171 \\ 0.05552 & 0.52886 & 0.29573 \\ -0.09532 & 0.75153 & 0.28855 \\ 0.86 & 0.22 & -0.99 \end{bmatrix}$$

Particle 1 output: $\mathbf{f}_{1,2}(\mathbf{x}) = [0.54046, 1.37170, 2.38329]$

Particle 1 error: $error_1(2) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{1,2}(x_n)]^2 = 2910.44$

Particle 1 fitness: $F_1(2) = 1/(\varepsilon + error_1(2)) = 3.4359 \times 10^{-4}$

Particle 2 output: $\mathbf{f}_{2,2}(\mathbf{x}) = [0.36433, 3.38619, 41.54982]$

Particle 2 error: $error_2(2) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{2,2}(x_n)]^2 = 1019.25$

Particle 2 fitness: $F_2(2) = 1/(\varepsilon + error_2(2)) = 9.811 \times 10^{-4}$

Particle 3 output: $\mathbf{f}_{3,2}(\mathbf{x}) = [0.15980, -28.53390, -1145.29]$

Particle 3 error: $error_3(2) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{3,2}(x_n)]^2 = 512277$

Particle 3 fitness: $F_3(2) = 1/(\varepsilon + error_3(2)) = 1.9521 \times 10^{-6}$

Particle 4 output: $\mathbf{f}_{4,2}(\mathbf{x}) = [-0.05089, 3.58761, 12.46266]$

Particle 4 error: $error_4(2) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{4,2}(x_n)]^2 = 2302.48$

Particle 4 fitness: $F_4(2) = 1/(\varepsilon + error_4(2)) = 4.3431 \times 10^{-4}$

$$\mathbf{Lbest} = \begin{bmatrix} 0.78099 & 0.09789 & -0.2717 \\ 0.05552 & 0.52886 & 0.29573 \\ -0.0953 & 0.75153 & 0.28855 \\ 0.86 & 0.22 & -0.99 \end{bmatrix}$$

LBest fitness

$$= [3.4359 \times 10^{-4} \quad 9.8110 \times 10^{-4} \quad 1.9521 \times 10^{-6} \quad 4.3431 \times 10^{-4}]^T$$

Among the fitness values, the particle 2 has the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 2's LBest fitness.

$$\mathbf{GBest\ particle} = [0.05552 \quad 0.52886 \quad 0.29573]$$

$$\mathbf{GBest\ fitness} = 9.811 \times 10^{-4}$$

With GBest, $\min error_{1\text{ to }P}(2) = error_3(2) = 1019.25 > \delta = 1 \times 10^{-4}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \mathbf{v}_3 \\ \mathbf{v}_4 \end{bmatrix} = \begin{bmatrix} 0.03099 & 0.04789 & -0.28171 \\ 0.31552 & -0.12114 & -0.50427 \\ 0.37468 & -0.20847 & -0.50145 \\ 0 & 0 & 0 \end{bmatrix}$$

$$+ 0.64 \times \left(\begin{bmatrix} 0.78099 & 0.09789 & -0.2717 \\ 0.05552 & 0.52886 & 0.29573 \\ -0.0953 & 0.75153 & 0.28855 \\ 0.86 & 0.22 & -0.99 \end{bmatrix} - \begin{bmatrix} 0.78099 & 0.09789 & -0.2717 \\ 0.05552 & 0.52886 & 0.29573 \\ -0.0953 & 0.75153 & 0.28855 \\ 0.86 & 0.22 & -0.99 \end{bmatrix} \right)$$

$$\begin{aligned}
& +0.53 \times \left(\begin{bmatrix} 0.05552 & 0.52886 & 0.29573 \\ 0.05552 & 0.52886 & 0.29573 \\ 0.05552 & 0.52886 & 0.29573 \\ 0.05552 & 0.52886 & 0.29573 \end{bmatrix} \right. \\
& \quad \left. - \begin{bmatrix} 0.78099 & 0.09789 & -0.2717 \\ 0.05552 & 0.52886 & 0.29573 \\ -0.0953 & 0.75153 & 0.28855 \\ 0.86 & 0.22 & -0.99 \end{bmatrix} \right) \\
& = \begin{bmatrix} -0.35540 & 0.27743 & 0.02051 \\ 0.31552 & -0.12114 & -0.50427 \\ 0.45502 & -0.32706 & -0.49762 \\ -0.42847 & 0.16450 & 0.68478 \end{bmatrix}
\end{aligned}$$

Particle position update:

$$\begin{aligned}
\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \mathbf{x}_4 \end{bmatrix} &= \begin{bmatrix} 0.78099 & 0.09789 & -0.2717 \\ 0.05552 & 0.52886 & 0.29573 \\ -0.0953 & 0.75153 & 0.28855 \\ 0.86 & 0.22 & -0.99 \end{bmatrix} \\
& \quad + s_c \left\{ \begin{bmatrix} -0.35540 & 0.27743 & 0.02051 \\ 0.31552 & -0.12114 & -0.50427 \\ 0.45502 & -0.32706 & -0.49762 \\ -0.42847 & 0.16450 & 0.68478 \end{bmatrix} \right\} \\
& = \begin{bmatrix} 0.42559 & 0.37532 & -0.25120 \\ 0.37104 & 0.40773 & -0.20853 \\ 0.35969 & 0.42447 & -0.20907 \\ 0.43153 & 0.38450 & -0.30522 \end{bmatrix}
\end{aligned}$$

Generation 3

The new particle positions at the start of generation 3 are

$$\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \mathbf{x}_4 \end{bmatrix} = \begin{bmatrix} 0.42559 & 0.37532 & -0.25120 \\ 0.37104 & 0.40773 & -0.20853 \\ 0.35969 & 0.42447 & -0.20907 \\ 0.43153 & 0.38450 & -0.30522 \end{bmatrix}$$

Particle 1 output: $\mathbf{f}_{1,3}(\mathbf{x}) = [0.24333, 7.12411, 46.1439]$

Particle 1 error: $error_1(3) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{1,3}(x_n)]^2 = 824.21$

Particle 1 fitness: $F_1(3) = 1/(\varepsilon + error_1(3)) = 1.2133 \times 10^{-3}$

Particle 2 output: $\mathbf{f}_{2,3}(\mathbf{x}) = [0.22823, 8.01723, 60.4423]$

Particle 2 error: $error_2(3) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{2,3}(x_n)]^2 = 433.45$

Particle 2 fitness: $F_2(3) = 1/(\varepsilon + error_2(3)) = 2.3070 \times 10^{-3}$

Particle 3 output: $\mathbf{f}_{3,3}(\mathbf{x}) = [0.21718, 8.8474, 72.2773]$

Particle 3 error: $error_3(3) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{3,3}(x_n)]^2 = 212.69$

Particle 3 fitness: $F_3(3) = 1/(\varepsilon + error_3(3)) = 4.7016 \times 10^{-3}$

Particle 4 output: $\mathbf{f}_{4,3}(\mathbf{x}) = [0.19806, 7.71361, 52.4595]$

Particle 4 error: $error_4(3) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{4,3}(x_n)]^2 = 632.73$

Particle 4 fitness: $F_4(3) = 1/(\varepsilon + error_4(3)) = 1.5804 \times 10^{-3}$

$$\text{Lbest} = \begin{bmatrix} 0.42559 & 0.37532 & -0.25120 \\ 0.37104 & 0.40773 & -0.20853 \\ 0.35969 & 0.42447 & -0.20907 \\ 0.43153 & 0.38450 & -0.30522 \end{bmatrix}$$

LBest fitness = $[0.001213 \quad 0.002307 \quad 0.004702 \quad 0.001580]^T$

At generation 3, the particle 3 has the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 3's fitness.

GBest particle = $[0.35969 \quad 0.42447 \quad -0.20907]$

GBest fitness = 4.7016×10^{-3}

With GBest, $\min error_{1 \text{ to } p}(3) = error_4(3) = 212.6 > \delta = 1 \times 10^{-4}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{aligned}
\begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \mathbf{v}_3 \\ \mathbf{v}_4 \end{bmatrix} &= \begin{bmatrix} -0.35540 & 0.27743 & 0.02051 \\ 0.31552 & -0.12114 & -0.50427 \\ 0.45502 & -0.32706 & -0.49762 \\ -0.42847 & 0.16450 & 0.68478 \end{bmatrix} \\
&+ 0.32 \times \left(\begin{bmatrix} 0.42559 & 0.37532 & -0.25120 \\ 0.37104 & 0.40773 & -0.20853 \\ 0.35969 & 0.42447 & -0.20907 \\ 0.43153 & 0.38450 & -0.30522 \end{bmatrix} \right. \\
&\quad \left. - \begin{bmatrix} 0.42559 & 0.37532 & -0.25120 \\ 0.37104 & 0.40773 & -0.20853 \\ 0.35969 & 0.42447 & -0.20907 \\ 0.43153 & 0.38450 & -0.30522 \end{bmatrix} \right) \\
&+ 0.13 \times \left(\begin{bmatrix} 0.35969 & 0.42447 & -0.20907 \\ 0.35969 & 0.42447 & -0.20907 \\ 0.35969 & 0.42447 & -0.20907 \\ 0.35969 & 0.42447 & -0.20907 \end{bmatrix} \right. \\
&\quad \left. - \begin{bmatrix} 0.42559 & 0.37532 & -0.25120 \\ 0.37104 & 0.40773 & -0.20853 \\ 0.35969 & 0.42447 & -0.20907 \\ 0.43153 & 0.38450 & -0.30522 \end{bmatrix} \right) \\
&= \begin{bmatrix} -0.36388 & 0.28376 & 0.02594 \\ 0.31406 & -0.11898 & -0.50434 \\ 0.45502 & -0.32706 & -0.49762 \\ -0.43772 & 0.16965 & 0.69717 \end{bmatrix}
\end{aligned}$$

Particle position update:

$$\begin{aligned}
\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \mathbf{x}_4 \end{bmatrix} &= \begin{bmatrix} 0.42559 & 0.37532 & -0.25120 \\ 0.37104 & 0.40773 & -0.20853 \\ 0.35969 & 0.42447 & -0.20907 \\ 0.43153 & 0.38450 & -0.30522 \end{bmatrix} \\
&+ s_c \left\{ \begin{bmatrix} -0.36388 & 0.28376 & 0.02594 \\ 0.31406 & -0.11898 & -0.50434 \\ 0.45502 & -0.32706 & -0.49762 \\ -0.43772 & 0.16965 & 0.69717 \end{bmatrix} \right\}
\end{aligned}$$

$$= \begin{bmatrix} 0.06171 & 0.65908 & -0.22527 \\ 0.68509 & 0.28875 & -0.71287 \\ 0.81471 & 0.09741 & -0.70669 \\ -0.00619 & 0.55415 & 0.39194 \end{bmatrix}$$

This is the end of generation 3. As the optimization goes on, the particle positions evolve and approach the desired solution. The particle swarm optimization continues until generation 2650.

...

Generation 2650 (PSO converged)

The new particle positions at the start of generation 2650 are

$$\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \\ \mathbf{x}_4 \end{bmatrix} = \begin{bmatrix} 4.3988 & 0.25003 & -6.7999 \\ 4.3913 & 0.25015 & -6.7846 \\ 4.3833 & 0.25028 & -6.7632 \\ 4.4014 & 0.24995 & -6.7851 \end{bmatrix}$$

Particle 1 output: $\mathbf{f}_{1,2650}(\mathbf{x}) = [-1.93838 \quad 22.6165 \quad 93.3522]$

Particle 1 error:

$$error_1(2650) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{1,2650}(x_n)]^2 = 2.59630 \times 10^{-5}$$

Particle 1 fitness: $F_1(2650) = 1/(\varepsilon + error_1(2650)) = 99.7410$

Particle 2 output, $\mathbf{f}_{2,2650}(\mathbf{x}) = [-1.93117 \quad 22.6095 \quad 93.3531]$

Particle 2 error:

$$error_2(2650) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{2,2650}(x_n)]^2 = 6.6533 \times 10^{-5}$$

Particle 2 fitness: $F_2(2650) = 1/(\varepsilon + error_2(2650)) = 99.3391$

Particle 3 output: $\mathbf{f}_{3,2650}(\mathbf{x}) = [-1.91836 \quad 22.6056 \quad 93.3505]$

Particle 3 error:

$$error_3(2650) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{3,2650}(x_n)]^2 = 1.8623 \times 10^{-4}$$

Particle 3 fitness: $F_3(2650) = 1/(\varepsilon + error_3(2650)) = 98.1718$

Particle 4 output: $\mathbf{f}_{4,2650}(\mathbf{x}) = [-1.92088 \quad 22.6317 \quad 93.3313]$

Particle 4 error:

$$error_4(2650) = \frac{1}{3} \sum_{n=1}^3 [d(x_n) - f_{4,2650}(x_n)]^2 = 2.0265 \times 10^{-4}$$

Particle 4 fitness: $F_4(2650) = 1/(\varepsilon + error_4(2650)) = 98.0137$

$$\mathbf{Lbest} = \begin{bmatrix} 4.3988 & 0.25003 & -6.7999 \\ 4.3913 & 0.25015 & -6.7846 \\ 4.3833 & 0.25028 & -6.7632 \\ 4.4014 & 0.24995 & -6.7851 \end{bmatrix}$$

LBEST fitness = $[99.7410 \quad 99.3391 \quad 98.1718 \quad 98.0137]^T$

At generation 2650, the particle 1 has the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 1's fitness.

GBest particle = $[4.3988 \quad 0.25003 \quad -6.7999]$

GBest fitness = 99.7410

With GBest, $\min error_{1 \text{ to } P}(979) = error_2(979) = 2.5963 \times 10^{-5} < \delta = 1 \times 10^{-4}$, convergence criterion has been met, no more generation is required.

Thus the optimized coefficients of the exponential function are $[4.3988, 0.25003, -6.7999]$ obtained from the GBest particle.

The convergence property of the particle swarm optimization can be observed in Figure E15.1.1 as the optimized exponential function gradually approaches the optimal polynomial function.

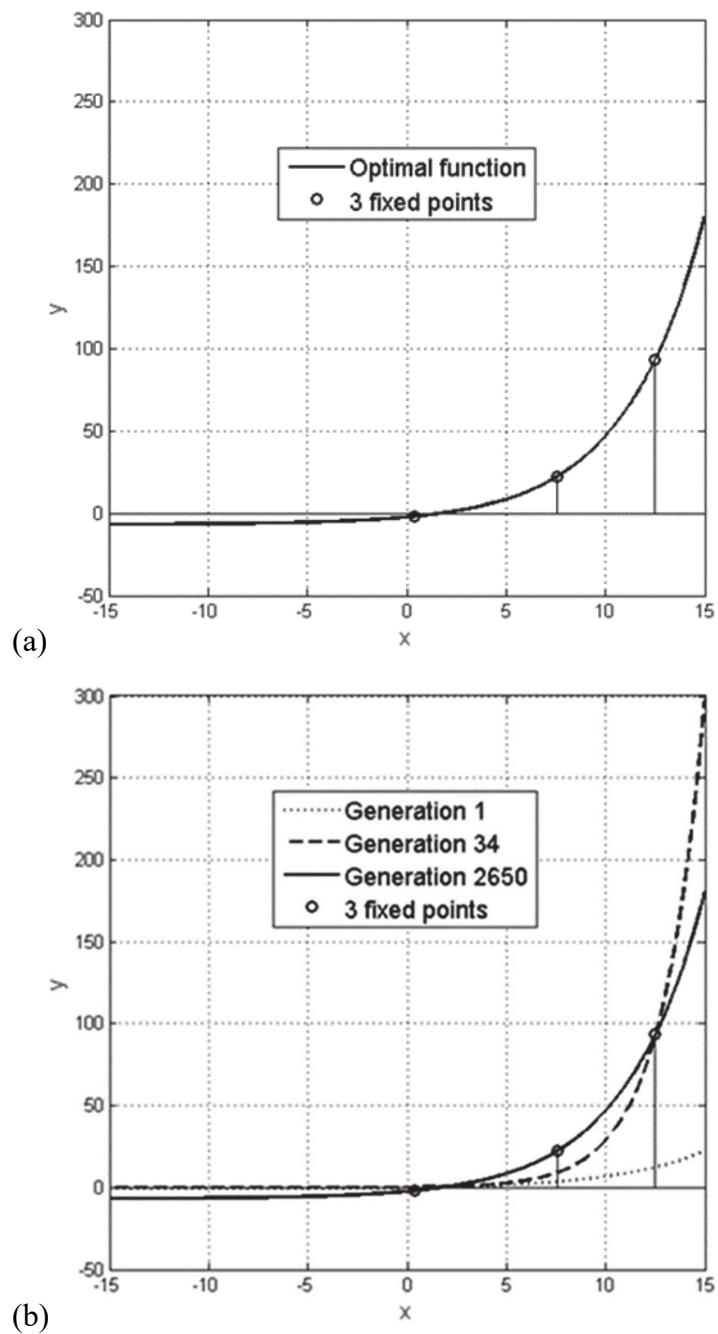


Figure E15.1.1 (a) Optimal exponential function; (b) Optimized exponential function at generations 1, 34, and 2650.

Design Example 15.2 (1-dimensional sine function interpolation by PSO)

Consider the application of the particle swarm optimization to search the coefficients of a sine function

$$y = g(x) = a_0 - a_1 x \sin(w\pi x + a_2\pi) \text{ with } w = 6$$

to fit N number of input and output data pairs.

For simplicity, a population size of six (6) is taken and the calculation steps of generations 1, 2, 8 plus the last generation are shown.

c_1 and c_2 are chosen to be 1 and the term '*rand*' represents a uniformly distributed pseudorandom number of value between 0 and 1.

The error function is taken as the mean squared error difference between the output of the target function and the output from each particle.

The fitness function is defined by the inverse of the sum of the error function plus a small positive constant.

The local best (LBest) ρ_p of a particle is taken as the best of the particle among the present and all the past generations.

Due to the small number of particles in the population, the global best (GBest) ρ_g is taken as the global best which is the best of all the particles among the present and all the past generations.

The particle velocity and position updates within each generation are carried out by implementing the following two equations in which $x_{pd}(t)$ denotes the position and $v_{pd}(t)$ denotes velocity of a particle p at time (or generation) t .

$$v_{pd}(t+1) = v_{pd}(t) + \varphi_1[\rho_{pd}(t) - x_{pd}(t)] + \varphi_2[\rho_{gd}(t) - x_{pd}(t)]$$

$$v_{pd}(t+1) = s_b[v_{pd}(t+1)] \in [-V_{max}, V_{max}]$$

$$x_{pd}(t+1) = x_{pd}(t) + v_{pd}(t+1)$$

where $\varphi_1 = c_1 \times rand_p$; and $\varphi_2 = c_2 \times rand_p$

For generation t , the error function of a particle p is defined as the mean squared error (MSE) $error_p(t)$ between the desired output $\mathbf{d}_t(\mathbf{x})$ and the present output $\mathbf{f}_{p,t}(\mathbf{x})$ due to an input vector, $\mathbf{x} (= [x_1, x_2, \dots, x_N])$ such that

$$error_p(t) = \frac{1}{N} \sum_{n=1}^N [d_t(x_n) - f_{p,t}(x_n)]^2$$

If the input $\mathbf{x}$ is kept the same in all generations, the desired output $\mathbf{d}_t(\mathbf{x})$ remains unchanged and $\mathbf{d}_t(\mathbf{x}) = \mathbf{d}(\mathbf{x})$.

The particle's fitness function is defined as the inverse of the mean squared error plus a small positive constant to avoid a zero denominator. Hence the fitness function $F_p(t)$ becomes

$$F_p(t) = \frac{1}{\varepsilon + error_p(t)}$$

During the generation t , the best among all the particles $GBest$ is defined as the particle in record that has the largest fitness or the smallest mean squared error as

$$GBest(t) = \max_{F_p} \{particle\} = \min_{error_p} \{particle\}$$

Define a small positive error limit (or convergence criterion) δ such that if $error_{GBest}(t) \leq \delta$, then the algorithm stops, and a satisfactory solution is said to have been obtained in the generation t .

Solution

Initialization

$$c_1 = 1$$

$$c_2 = 1$$

$$\varepsilon = 0.01$$

Convergence criterion: $\delta = 1 \times 10^{-5}$

Target function: $y = g(x) = a_0 - a_1 x \sin(6\pi x + a_2 \pi)$

The optimal parameters are: $[a_0, a_1, a_2] = [1, 0.5, 0.25]$

Number of particles (population size), $P = 6$

Particle dimension (number of unknown coefficients), $D = 3$

Initializing GBest particle = **0**

Initializing GBest fitness = 0

Number of fixed input, $N = 5$

$V_{max} = 2$

At the start, a set of five ($N = 5$) uniformly spaced fixed inputs $[x_1, x_2, x_3, x_4, x_5] = [-0.5, -0.25, 0, 0.25, 0.5]$ are to be used, and the corresponding 5 outputs $[d(x_1), d(x_2), d(x_3), d(x_4), d(x_5)]$ are

$$d(-0.5) = 1 - 0.5(-0.5)\sin(6\pi(-0.5) + 0.25\pi) = 0.82322$$

$$d(-0.25) = 1 - 0.5(-0.25)\sin(6\pi(-0.25) + 0.25\pi) = 1.0884$$

$$d(0) = 1 - 0.5(0)\sin(6\pi(0) + 0.25\pi) = 1$$

$$d(0.25) = 1 - 0.5(0.25)\sin(6\pi(0.25) + 0.25\pi) = 1.0884$$

$$d(0.5) = 1 - 0.5(0.5)\sin(6\pi(0.5) + 0.25\pi) = 1.1768$$

In other words, the desired outputs for all the particles are $[0.82322, 1.0884, 1, 1.0884, 1.1768]$ under fixed inputs $[-0.5, -0.25, 0, 0.25, 0.5]$.

$$\text{Initializing LBest (} 6 \times 3 \text{ matrix)} = \begin{bmatrix} 0 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 0 \end{bmatrix}$$

$$\text{Initializing LBest fitness} = [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]^T$$

$$\text{Initializing random population} = \begin{bmatrix} -0.47 & -0.64 & 0.25 \\ 0.84 & -0.91 & -0.96 \\ -0.56 & 0.44 & 0.82 \\ -0.26 & -0.31 & 0.6 \\ -0.83 & 0.32 & 0.49 \\ 0.28 & -0.24 & 0.62 \end{bmatrix}$$

$$\text{Initializing velocity (for each particle in the population)} = [0 \ 0 \ 0 \ 0 \ 0 \ 0]^T$$

Generation 1

Particle 1 output:

$$\begin{aligned} f_{1,1}(-0.50) &= -0.47 - (-0.64)(-0.5)\sin(6\pi(-0.5) + 0.25\pi) \\ &= -0.24373 \end{aligned}$$

$$\begin{aligned} f_{1,1}(-0.25) &= -0.47 - (-0.64)(-0.25)\sin(6\pi(-0.25) + 0.25\pi) \\ &= -0.58314 \end{aligned}$$

$$\begin{aligned} f_{1,1}(0.00) &= -0.47 - (-0.64)(0)\sin(6\pi(0) + 0.25\pi) \\ &= -0.47 \end{aligned}$$

$$\begin{aligned} f_{1,1}(0.25) &= -0.47 - (-0.64)(0.25)\sin(6\pi(0.25) + 0.25\pi) \\ &= -0.58314 \end{aligned}$$

$$\begin{aligned} f_{1,1}(0.50) &= -0.47 - (-0.64)(0.5)\sin(6\pi(0.5) + 0.25\pi) \\ &= -0.69627 \end{aligned}$$

In vector format:

$$f_{1,1}(\mathbf{x}) = [-0.24373, -0.58314, -0.47, -0.58314, -0.69627]$$

Particle 1 error:

$$= \frac{1}{5} \times [(0.8232 - (-0.2437))^2 + (1.0884 - (-0.5831))^2 + \dots + (1.1768 - (-0.6963))^2] = 2.4791$$

Particle 1 fitness:

$$F_1(1) = \frac{1}{\varepsilon + error_j(k)} = \frac{1}{0.01 + error_1(1)} = 0.40175$$

Similarly,

Particle 2 output:

$$f_{2,1}(\mathbf{x}) = [0.78297, 1.0657, 0.84, 1.0657, 0.89703]$$

$$\text{Particle 2 error: } error_2(1) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{2,1}(x_n)]^2 = 0.021302$$

$$\text{Particle 2 fitness: } F_2(1) = 1/(\varepsilon + error_2(1)) = 31.947$$

Particle 3 output:

$$f_{3,1}(\mathbf{x}) = [-0.67788, -0.65288, -0.56, -0.65288, -0.44212]$$

$$\text{Particle 3 error: } error_3(1) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{3,1}(x_n)]^2 = 2.6743$$

$$\text{Particle 3 fitness: } F_3(1) = 1/(\varepsilon + error_3(1)) = 0.37253$$

Particle 4 output:

$$f_{4,1}(\mathbf{x}) = [-0.11259, -0.23605, -0.26, -0.23605, -0.40741]$$

$$\text{Particle 4 error: } error_4(1) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{4,1}(x_n)]^2 = 1.6963$$

$$\text{Particle 4 fitness: } F_4(1) = 1/(\varepsilon + error_4(1)) = 0.586079$$

Particle 5 output:

$$f_{5,1}(\mathbf{x}) = [-0.98992, -0.82749, -0.83, -0.82749, -0.67008]$$

$$\text{Particle 5 error: } error_5(1) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{5,1}(x_n)]^2 = 3.4777$$

$$\text{Particle 5 fitness: } F_5(1) = 1/(\varepsilon + error_5(1)) = 0.286723$$

Particle 6 output:

$$f_{6,1}(\mathbf{x}) = [0.39157, 0.30209, 0.28, 0.30209, 0.16843]$$

$$\text{Particle 6 error: } error_6(1) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{6,1}(x_n)]^2 = 0.59161$$

$$\text{Particle 6 fitness: } F_6(1) = 1/(\varepsilon + error_6(1)) = 1.66222$$

As this is the first generation, the LBest is the current population and the fitness is the current fitness.

$$\text{LBest} = \begin{bmatrix} -0.47 & -0.64 & 0.25 \\ 0.84 & -0.91 & -0.96 \\ -0.56 & 0.44 & 0.82 \\ -0.26 & -0.31 & 0.6 \\ -0.83 & 0.32 & 0.49 \\ 0.28 & -0.24 & 0.62 \end{bmatrix}$$

LBest fitness

$$= [0.401749, 31.947, 0.37253, 0.586079, 0.286723, 1.66222]^T$$

Among the 6 fitness values, the particle 2 has the highest fitness value of 31.947. Consequently, GBest fitness is equal to the particle 2's LBest fitness.

$$\text{GBest particle} = [0.84 \quad -0.91 \quad -0.96]$$

$$\text{GBest fitness} = 31.947$$

With GBest, $\min error_{1 \text{ to } P}(1) = error_2(1) = 0.021302 > \delta = 1 \times 10^{-5}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{aligned}
& +0.383 \times \left(\begin{bmatrix} -0.47 & -0.64 & 0.25 \\ 0.84 & -0.91 & -0.96 \\ -0.56 & 0.44 & 0.82 \\ -0.26 & -0.31 & 0.6 \\ -0.83 & 0.32 & 0.49 \\ 0.28 & -0.24 & 0.62 \end{bmatrix} \right. \\
& \quad \left. - \begin{bmatrix} -0.47 & -0.64 & 0.25 \\ 0.84 & -0.91 & -0.96 \\ -0.56 & 0.44 & 0.82 \\ -0.26 & -0.31 & 0.6 \\ -0.83 & 0.32 & 0.49 \\ 0.28 & -0.24 & 0.62 \end{bmatrix} \right) \\
& +0.617 \times \left(\begin{bmatrix} 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \end{bmatrix} \right. \\
& \quad \left. - \begin{bmatrix} -0.47 & -0.64 & 0.25 \\ 0.84 & -0.91 & -0.96 \\ -0.56 & 0.44 & 0.82 \\ -0.26 & -0.31 & 0.6 \\ -0.83 & 0.32 & 0.49 \\ 0.28 & -0.24 & 0.62 \end{bmatrix} \right)
\end{aligned}$$

Hence,

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} = \begin{bmatrix} 0.80864 & -0.16667 & -0.74691 \\ 0 & 0 & 0 \\ 0.86419 & -0.83333 & -1.0988 \\ 0.67901 & -0.37037 & -0.96296 \\ 1.0309 & -0.75925 & -0.89505 \\ 0.34568 & -0.41358 & -0.9753 \end{bmatrix}$$

Particle position update:

$$\begin{aligned}
\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} &= \begin{bmatrix} -0.47 & -0.64 & 0.25 \\ 0.84 & -0.91 & -0.96 \\ -0.56 & 0.44 & 0.82 \\ -0.26 & -0.31 & 0.6 \\ -0.83 & 0.32 & 0.49 \\ 0.28 & -0.24 & 0.62 \end{bmatrix} \\
&+ s_c \left\{ \begin{bmatrix} 0.80864 & -0.16667 & -0.74691 \\ 0 & 0 & 0 \\ 0.86419 & -0.83333 & -1.0988 \\ 0.67901 & -0.37037 & -0.96296 \\ 1.0309 & -0.75925 & -0.89505 \\ 0.34568 & -0.41358 & -0.9753 \end{bmatrix} \right\} \\
&= \begin{bmatrix} 0.33864 & -0.80667 & -0.4969 \\ 0.84 & -0.91 & -0.96 \\ 0.30419 & -0.39333 & -0.27876 \\ 0.41901 & -0.68037 & -0.36296 \\ 0.20086 & -0.43925 & -0.40505 \\ 0.62568 & -0.65358 & -0.35530 \end{bmatrix}
\end{aligned}$$

Generation 2

From generation 1, a new population is produced. So the new particle positions are

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = \begin{bmatrix} 0.33864 & -0.80667 & -0.49691 \\ 0.84 & -0.91 & -0.96 \\ 0.30419 & -0.39333 & -0.27876 \\ 0.41901 & -0.68037 & -0.36296 \\ 0.20086 & -0.43925 & -0.40505 \\ 0.62568 & -0.65358 & -0.35530 \end{bmatrix}$$

Particle 1 output:

$$f_{1,2}(\mathbf{x}) = [-0.064678, 0.33668, 0.33864, 0.33668, 0.74195]$$

$$\text{Particle 1 error: } error_1(2) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{1,2}(x_n)]^2 = 0.509$$

$$\text{Particle 1 fitness: } F_1(2) = 1/(\varepsilon + error_1(2)) = 1.92679$$

Particle 2 output:

$$f_{2,2}(\mathbf{x}) = [0.78297, 1.0657, 0.84, 1.0657, 0.89703]$$

$$\text{Particle 2 error: } error_2(2) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{2,2}(x_n)]^2 = 0.021302$$

$$\text{Particle 2 fitness: } F_2(2) = 1/(\varepsilon + error_2(2)) = 31.947$$

Particle 3 output:

$$f_{3,2}(\mathbf{x}) = [0.15315, 0.24122, 0.30419, 0.24122, 0.45523]$$

$$\text{Particle 3 error: } error_3(2) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{3,2}(x_n)]^2 = 0.57784$$

$$\text{Particle 3 fitness: } F_3(2) = 1/(\varepsilon + error_3(2)) = 1.70116$$

Particle 4 output:

$$f_{4,2}(\mathbf{x}) = [0.10987, 0.34802, 0.41901, 0.34802, 0.72815]$$

$$\text{Particle 4 error: } error_4(2) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{4,2}(x_n)]^2 = 0.4288$$

$$\text{Particle 4 fitness: } F_4(2) = 1/(\varepsilon + error_4(2)) = 2.27895$$

Particle 5 output:

$$f_{5,2}(\mathbf{x}) = [-9.0725 \times 10^{-3}, 0.16858, 0.20086, 0.16858, 0.41079]$$

$$\text{Particle 5 error: } error_5(2) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{5,2}(x_n)]^2 = 0.72203$$

$$\text{Particle 5 fitness: } F_5(2) = 1/(\varepsilon + error_5(2)) = 1.36606$$

Particle 6 output:

$$f_{6,2}(\mathbf{x}) = [0.33208, 0.55393, 0.62568, 0.55393, 0.91928]$$

$$\text{Particle 6 error: } error_6(2) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{6,2}(x_n)]^2 = 0.20379$$

$$\text{Particle 6 fitness: } F_6(2) = 1/(\varepsilon + error_6(2)) = 4.67754$$

$$L_{best} = \begin{bmatrix} 0.33864 & -0.80667 & -0.49691 \\ 0.84 & -0.91 & -0.96 \\ 0.30419 & -0.39333 & -0.27876 \\ 0.41901 & -0.68037 & -0.36296 \\ 0.20086 & -0.43925 & -0.40505 \\ 0.62568 & -0.65358 & -0.35530 \end{bmatrix}$$

LBest fitness

$$= [1.92679, 31.9470, 1.70116, 2.27895, 1.36606, 4.67754]^T$$

Among the fitness values, the particle 2 has again the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 2's LBest fitness.

GBest particle $[0.84, -0.91, -0.96]$

GBest fitness = 31.9470

With GBest, $\min error_{1 \text{ to } P}(2) = error_2(2) = 0.021302 > \delta = 1 \times 10^{-5}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} = \begin{bmatrix} 0.80864 & -0.16667 & -0.74691 \\ 0 & 0 & 0 \\ 0.86419 & -0.83333 & -1.0988 \\ 0.67901 & -0.37037 & -0.96296 \\ 1.0309 & -0.75925 & -0.89505 \\ 0.34568 & -0.41358 & -0.9753 \end{bmatrix}$$

$$\begin{aligned}
& +0.5755 \times \left(\begin{bmatrix} 0.33864 & -0.80667 & -0.49691 \\ 0.84 & -0.91 & -0.96 \\ 0.30419 & -0.39333 & -0.27876 \\ 0.41901 & -0.68037 & -0.36296 \\ 0.20086 & -0.43925 & -0.40505 \\ 0.62568 & -0.65358 & -0.35530 \end{bmatrix} \right. \\
& \quad \left. - \begin{bmatrix} 0.33864 & -0.80667 & -0.49691 \\ 0.84 & -0.91 & -0.96 \\ 0.30419 & -0.39333 & -0.27876 \\ 0.41901 & -0.68037 & -0.36296 \\ 0.20086 & -0.43925 & -0.40505 \\ 0.62568 & -0.65358 & -0.35530 \end{bmatrix} \right) \\
& +0.5301 \times \left(\begin{bmatrix} 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \end{bmatrix} \right. \\
& \quad \left. - \begin{bmatrix} 0.33864 & -0.80667 & -0.49691 \\ 0.84 & -0.91 & -0.96 \\ 0.30419 & -0.39333 & -0.27876 \\ 0.41901 & -0.68037 & -0.36296 \\ 0.20086 & -0.43925 & -0.40505 \\ 0.62568 & -0.65358 & -0.35530 \end{bmatrix} \right)
\end{aligned}$$

Hence,

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} = \begin{bmatrix} 1.0744 & -0.22144 & -0.99237 \\ 0 & 0 & 0 \\ 1.1482 & -1.1072 & -1.4599 \\ 0.90216 & -0.49208 & -1.2794 \\ 1.3696 & -1.0088 & -1.1892 \\ 0.45928 & -0.54949 & -1.2958 \end{bmatrix}$$

Particle position update:

$$\begin{aligned}
 \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} &= \begin{bmatrix} 0.33864 & -0.80667 & -0.49691 \\ 0.84 & -0.91 & -0.96 \\ 0.30419 & -0.39333 & -0.27876 \\ 0.41901 & -0.68037 & -0.36296 \\ 0.20086 & -0.43925 & -0.40505 \\ 0.62568 & -0.65358 & -0.35530 \end{bmatrix} \\
 &\quad + s_c \left\{ \begin{bmatrix} 1.0744 & -0.22144 & -0.99237 \\ 0 & 0 & 0 \\ 1.1482 & -1.1072 & -1.4599 \\ 0.90216 & -0.49208 & -1.2794 \\ 1.3696 & -1.0088 & -1.1892 \\ 0.45928 & -0.54949 & -1.2958 \end{bmatrix} \right\} \\
 &= \begin{bmatrix} 1.413 & -1.0281 & -1.4893 \\ 0.84 & -0.91 & -0.96 \\ 1.4524 & -1.5005 & -1.7386 \\ 1.3212 & -1.1725 & -1.6424 \\ 1.5705 & -1.448 & -1.5943 \\ 1.085 & -1.2031 & -1.6511 \end{bmatrix}
 \end{aligned}$$

...

Generation 8

The new particle positions at the start of generation 8 are

$$\begin{bmatrix} -0.97863 & -0.53517 & 0.7198 \\ 0.84 & -0.91 & -0.96 \\ -1.1036 & 0.96416 & 1.5111 \\ -0.68709 & -0.077039 & 1.2057 \\ -1.4784 & 0.79757 & 1.053 \\ 0.2406 & -0.19286 & 0.73118 \end{bmatrix}$$

Particle 1 output:

$$f_{1,8}(\mathbf{x}) = [-0.77235, -0.89341, -0.97863, -0.89341, -1.1849]$$

$$\text{Particle 1 error: } error_1(8) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{1,8}(x_n)]^2 = 3.9787$$

$$\text{Particle 1 fitness: } F_1(8) = 1/(\varepsilon + error_1(8)) = 0.250708$$

Particle 2 output:

$$f_{2,8}(\mathbf{x}) = [0.78297, 1.0657, 0.84, \quad 1.0657, 0.89703]$$

$$\text{Particle 2 error: } error_2(8) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{2,8}(x_n)]^2 = 0.021302$$

$$\text{Particle 2 fitness: } F_2(8) = 1/(\varepsilon + error_2(8)) = 31.947$$

Particle 3 output:

$$f_{3,8}(\mathbf{x}) = [-0.62179, -1.0952, -1.1036, -1.0952, -1.5854]$$

$$\text{Particle 3 error: } error_3(8) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{3,8}(x_n)]^2 = 4.7357$$

$$\text{Particle 3 fitness: } F_3(8) = 1/(\varepsilon + error_3(8)) = 0.210719$$

Particle 4 output:

$$f_{4,8}(\mathbf{x}) = [-0.71029, -0.67172, -0.68709, -0.67172, -0.6639]$$

$$\text{Particle 4 error: } error_4(8) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{4,8}(x_n)]^2 = 2.9564$$

$$\text{Particle 4 fitness: } F_4(8) = 1/(\varepsilon + error_4(8)) = 0.337109$$

Particle 5 output:

$$f_{5,8}(\mathbf{x}) = [-1.4123, -1.675, -1.4784, -1.675, -1.5445]$$

$$\text{Particle 5 error: } error_5(8) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{5,8}(x_n)]^2 = 6.7637$$

$$\text{Particle 5 fitness: } F_5(8) = 1/(\varepsilon + error_5(8)) = 0.147629$$

Particle 6 output:

$$f_{6,8}(\mathbf{x}) = [0.31269, 0.27261, 0.2406, 0.27261, 0.1685]$$

$$\text{Particle 6 error: } error_6(8) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{6,8}(x_n)]^2 = 0.63699$$

Particle 6 fitness: $F_6(8) = 1/(\varepsilon + error_6(8)) = 1.54562$

$$Lbest = \begin{bmatrix} -0.75102 & -0.58208 & 0.50957 \\ 0.84 & -0.91 & -0.96 \\ -0.86033 & 0.7296 & 1.2018 \\ -0.49597 & -0.18129 & 0.93465 \\ -1.1882 & 0.58386 & 0.80105 \\ 0.31172 & -0.27795 & 0.53051 \end{bmatrix}$$

LBest fitness

$$= [0.304500, 31.94700, 0.256086, 0.431661, 0.193144, 1.750910]^T$$

At generation 8, the particle 2 has the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 2's fitness.

$$GBest \text{ particle} = [0.84, -0.91, -0.969]$$

$$GBest \text{ fitness} = 31.947$$

With GBest, $\min error_{1 \text{ to } P}(8) = error_2(8) = 0.021302 > \delta = 1 \times 10^{-5}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} = \begin{bmatrix} -0.22761 & 0.046913 & 0.21024 \\ 0 & 0 & 0 \\ -0.24325 & 0.23456 & 0.30928 \\ -0.19113 & 0.10425 & 0.27105 \\ -0.29016 & 0.21371 & 0.25194 \\ -0.071122 & 0.085093 & 0.20067 \end{bmatrix}$$

$$\begin{aligned}
& +0.98914 \times \left(\begin{bmatrix} -0.75102 & -0.58208 & 0.50957 \\ 0.84 & -0.91 & -0.96 \\ -0.86033 & 0.7296 & 1.2018 \\ -0.49597 & -0.18129 & 0.93465 \\ -1.1882 & 0.58386 & 0.80105 \\ 0.31172 & -0.27795 & 0.53051 \end{bmatrix} \right. \\
& \quad \left. - \begin{bmatrix} -0.97863 & -0.53517 & 0.7198 \\ 0.84 & -0.91 & -0.96 \\ -1.1036 & 0.96416 & 1.5111 \\ -0.68709 & -0.077039 & 1.2057 \\ -1.4784 & 0.79757 & 1.053 \\ 0.2406 & -0.19286 & 0.73118 \end{bmatrix} \right) \\
& +0.06695 \times \left(\begin{bmatrix} 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \\ 0.84 & -0.91 & -0.96 \end{bmatrix} \right. \\
& \quad \left. - \begin{bmatrix} -0.97863 & -0.53517 & 0.7198 \\ 0.84 & -0.91 & -0.96 \\ -1.1036 & 0.96416 & 1.5111 \\ -0.68709 & -0.077039 & 1.2057 \\ -1.4784 & 0.79757 & 1.053 \\ 0.2406 & -0.19286 & 0.73118 \end{bmatrix} \right)
\end{aligned}$$

Hence,

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} = \begin{bmatrix} 0.11928 & -0.024584 & -0.11017 \\ 0 & 0 & 0 \\ 0.12747 & -0.12292 & -0.16207 \\ 0.10016 & -0.054632 & -0.14204 \\ 0.15206 & -0.112 & -0.13203 \\ 0.039356 & -0.047086 & -0.11104 \end{bmatrix}$$

Particle position update:

$$\begin{aligned}
 \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} &= \begin{bmatrix} -0.97863 & -0.53517 & 0.7198 \\ 0.84 & -0.91 & -0.96 \\ -1.1036 & 0.96416 & 1.5111 \\ -0.68709 & -0.077039 & 1.2057 \\ -1.4784 & 0.79757 & 1.053 \\ 0.2406 & -0.19286 & 0.73118 \end{bmatrix} \\
 &+ s_c \left\{ \begin{bmatrix} 0.11928 & -0.024584 & -0.11017 \\ 0 & 0 & 0 \\ 0.12747 & -0.12292 & -0.16207 \\ 0.10016 & -0.054632 & -0.14204 \\ 0.15206 & -0.112 & -0.13203 \\ 0.039356 & -0.047086 & -0.11104 \end{bmatrix} \right\} \\
 &= \begin{bmatrix} -0.85935 & -0.55975 & 0.60963 \\ 0.84 & -0.91 & -0.96 \\ -0.9761 & 0.84124 & 1.349 \\ -0.58694 & -0.13167 & 1.0637 \\ -1.3263 & 0.68557 & 0.92096 \\ 0.27995 & -0.23994 & 0.62014 \end{bmatrix}
 \end{aligned}$$

This is the end of generation 8. As the evolution goes on, the particles evolve and eventually approaching to the optimal solution. The particle swarm optimization continues until generation 152.

...

Generation 152 (PSO converged)

The new particle positions at the start of generation 152 are

$$\begin{bmatrix} 1.0009 & -0.49442 & -0.74365 \\ 1.0004 & -0.49392 & -0.74642 \\ 1.0007 & -0.49479 & -0.74368 \\ 0.99937 & -0.49714 & -0.74305 \\ 0.99938 & -0.49333 & -0.74767 \\ 0.99942 & -0.50293 & -0.75806 \end{bmatrix}$$

Particle 1 output:

$$f_{1,152}(\mathbf{x}) = [0.82263, 1.0865, 1.0009, 1.0865, 1.1791]$$

Particle 1 error:

$$error_1(152) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{1,152}(x_n)]^2 = 2.736 \times 10^{-6}$$

$$\text{Particle 1 fitness: } F_1(152) = 1/(\varepsilon + error_1(152)) = 99.9726$$

Particle 2 output:

$$f_{2,152}(\mathbf{x}) = [0.82385, 1.0868, 1.0004, 1.0868, 1.177]$$

Particle 2 error:

$$error_2(152) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{2,152}(x_n)]^2 = 1.1931 \times 10^{-6}$$

$$\text{Particle 2 fitness: } F_2(152) = 1/(\varepsilon + error_2(152)) = 99.9881$$

Particle 3 output:

$$f_{3,152}(\mathbf{x}) = [0.82235, 1.0864, 1.0007, 1.0864, 1.1791]$$

Particle 3 error:

$$error_3(152) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{3,152}(x_n)]^2 = 2.854 \times 10^{-6}$$

$$\text{Particle 3 fitness: } F_3(152) = 1/(\varepsilon + error_3(152)) = 99.9715$$

Particle 4 output:

$$f_{4,152}(\mathbf{x}) = [0.81981, 1.0853, 0.99937, 1.0853, 1.1789]$$

Particle 4 error:

$$error_4(152) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{4,152}(x_n)]^2 = 7.1325 \times 10^{-6}$$

$$\text{Particle 4 fitness: } F_4(152) = 1/(\varepsilon + error_4(152)) = 99.9287$$

Particle 5 output:

$$f_{5,152}(\mathbf{x}) = [0.82368, 1.0859, 0.99938, 1.0859, 1.1751]$$

Particle 5 error:

$$error_5(152) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{5,152}(x_n)]^2 = 3.097 \times 10^{-6}$$

$$\text{Particle 5 fitness: } F_5(152) = 1/(\varepsilon + error_5(152)) = 99.9690$$

Particle 6 output:

$$f_{6,152}(\mathbf{x}) = [0.82616, 1.0905, 0.99942, 1.0905, 1.1727]$$

Particle 6 error:

$$error_6(152) = \frac{1}{5} \sum_{n=1}^5 [d(x_n) - f_{6,152}(x_n)]^2 = 7.0231 \times 10^{-6}$$

$$\text{Particle 6 fitness: } F_6(152) = 1/(\varepsilon + error_6(152)) = 99.9298$$

$$Lbest = \begin{bmatrix} 1.0009 & -0.49442 & -0.74365 \\ 1.0004 & -0.49392 & -0.74642 \\ 1.0007 & -0.49479 & -0.74368 \\ 0.99937 & -0.49714 & -0.74305 \\ 0.99938 & -0.49333 & -0.74767 \\ 0.99942 & -0.50293 & -0.75806 \end{bmatrix}$$

LBest fitness

$$= [99.9726 \quad 99.9881 \quad 99.9715 \quad 99.9287 \quad 99.9690 \quad 99.9298]^T$$

At generation 152, the particle 2 has the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 2's fitness.

$$GBest \text{ particle} = [1.0004, -0.49392, -0.74642]$$

$$GBest \text{ fitness} = 99.9881$$

$$\text{With GBest, } \min error_{1 \text{ to } P}(152) = error_2(152) = 1.1931 \times$$

$10^{-6} < \delta = 1 \times 10^{-5}$, convergence criterion has been met, no more generation is required.

The optimal coefficients of the function are $\mathbf{a}_o = [a_0 \ a_1 \ a_2] = [1 \ 0.5 \ 0.25]$, which is equivalent to $[1 \ 0.5 \ 0.25 + 2k]$ or $[1 \ -0.5 \ 0.25 + 2k + 1]$, where k is an integer. This is because

$$\begin{aligned} a_0 - a_1 x \sin(w\pi x + a_2\pi) &= a_0 - a_1 x \sin(w\pi x + a_2\pi + 2k\pi) \\ &= a_0 - a_1 x \sin(w\pi x + (a_2 + 2k)\pi) \end{aligned}$$

Also

$$\begin{aligned} a_0 - a_1 x \sin(w\pi x + a_2\pi) &= a_0 - (-a_1)x \sin(w\pi x + a_2\pi + \pi) \\ &= a_0 - (-a_1)x \sin(w\pi x + (a_2 + 1)\pi) \\ &= a_0 - (-a_1)x \sin(w\pi x + (a_2 + 1)\pi + 2k\pi) \\ &= a_0 - (-a_1)x \sin(w\pi x + (a_2 + 2k + 1)\pi) \end{aligned}$$

Thus from the above

$$[a_0 \ a_1 \ a_2] \equiv [a_0 \ a_1 \ a_2 + 2k] \equiv [a_0 \ -a_1 \ a_2 + 2k + 1]$$

It can be seen that at last $\mathbf{a}_1$ to $\mathbf{a}_6$ are approaching the optimal set of polynomial coefficients (for $k = -1$) given by

$$[a_0 \ -a_1 \ a_2 + 2k + 1] = [a_0 \ -a_1 \ a_2 - 1] = [1 \ -0.5 \ -0.75]$$

Thus the optimized coefficients of the sine function are $[1.0004, -0.49392, -0.74642]$.

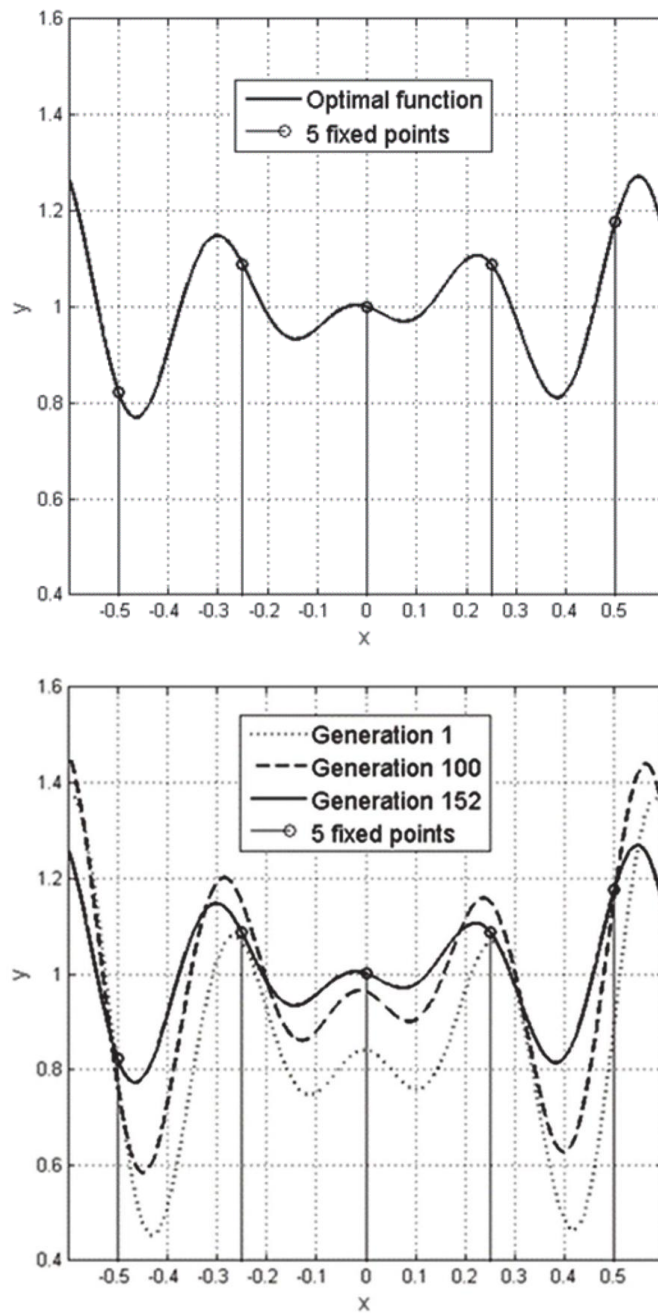


Figure E15.2.1 (a) Optimal sine function; (b) Optimized sine function at generations 1, 100, and 152.

The convergence property of the particle swarm optimization can be observed from the plots shown in Figure E15.2.1 as the optimized sine function gradually approaches the optimal sine function.

To evaluate the performance of the optimized function, forty uniformly spread values of x in between any two nearest neighbours of the five fixed values of input x are used to calculate the mean squared error MSE_q between the optimal function $g_{optimal}(x_i)$ and the optimized function $g_{optimized}(x_i)$ as

$$MSE_q = \frac{1}{40} \sum_{q=1}^{40} [f_{optimal}(x_q) - f_{optimized}(x_q)]^2 = 3.5345 \times 10^{-6}$$

The MSE_q value obtained indicates a satisfactory optimized solution for the interpolation problem has been obtained. Depending on the requirements of an application, a smaller or larger value of the convergence criterion δ could be used.

Design Example 15.3 (2-layer feedforward neural network learning of XOR problem using PSO - I)

Using the Global Optimization Matlab Toolbox particleswarm function to write a Matlab program to design a 2-layer neural network with real-valued parameters using the particle swarm optimization to learn the 4 input to output pairs of the XOR problem as described according to Chapter 4, Example 4.1.

The search continues until the output mean absolute error of a particle falls below δ .

Assuming that the LBest neighborhood of a particle p includes itself and its nearest particles with a total of 25% of the population in all generations; and the GBest neighborhood includes all the particles of a population in all generations.

The coefficient search space is

$$-s < w1, w2, w3, w4, w5, w6, b1, b2, b3 < s \quad (E15.3.1)$$

The size of the population P is taken as 90. Let the neural network parameters' range be $\pm s = \pm 16$. Each particle has $N_c = 9$ parameters with a length of $D = N_c$ real-values to be encoded as a parameter vector $\mathbf{c}$.

The slope parameter of the binary sigmoidal function at each neuron's output is taken as $r_n = 2$ with a unity slope at zero input and an input range of $\pm L = \pm 2$ before saturating to 0 and 1 outputs [Kwa9207].

The value of the slope parameter of the binary sigmoidal function to be used as the saturation function is taken as $r_f = 2$ with an input range of $\pm V_{max} = \pm 1.472$ (See Table 15.4.2).

Initial coefficients are generated randomly within $\pm s$.

The two particle velocity update constants are set to $c_1 = c_2 = 1.49$; and the sum of two acceleration coefficients φ_1 and φ_2 is less than 3. The decreasing range of the inertia weight is specified by $[w_1, w_2] = [0.1, 1.1]$.

The mean squared output error limit is taken as $\delta = 0.000001$. A summary of the suggested parameter values is given in Table E15.3.1.

Table E15.3.1 PSO parameters for 2-layer neural network.

P	N_c	D	s	r_n	L	r_f	V_{max}	c_1	c_2	$\varphi_1 + \varphi_2$	δ
90	9	N_c	16	2	2	2	1.472	1.49	1.49	≤ 3	0.000001

Solution

The Matlab program for designing the XOR problem using particle swarm optimization is listed in Table E15.3.2. The designed parameters and the mean absolute error of the 2-layer neural networks are listed in Table E15.3.3.

Table E15.3.2 Matlab program for XOR design problem using particle swarm optimization.

```

% xor_pso_mFun.m
% 2-layer neural network (with a bias at each neuron) design
% Global Optimization Matlab Toolbox particleswarm function

clc; clear; close all;
format long;

%% XOR design problem

input = [0 0; 0 1; 1 0; 1 1];
dout = [0; 1; 1; 0];
nvar=9; % Number of variables
% Lower & upper bounds of variables
Lb=[-16,-16,-16,-16,-16,-16,-16,-16,-16];
Ub=[16,16,16,16,16,16,16,16,16];

%% PSO (Set parameters for optimization)

% Default parameters
% InertiaRange: [0.1000000000000000 1.1000000000000000]
% MinNeighborsFraction: 0.2500000000000000 (local region)
% SelfAdjustmentWeight: 1.4900000000000000 (c1)
% SocialAdjustmentWeight: 1.4900000000000000 (c2)
% InitialSwarmSpan: 2000
% SwarmSize: 'min(100,10*numberofvariables)'
% MaxIterations: '200*numberofvariables'
% FunctionTolerance: 1.0000000000000000e-06

options = optimoptions(@simulannealbnd,'MaxIter', 1800); %
max iterations
options = optimoptions('particleswarm', 'TolFun', 1e-12);
options = optimoptions('particleswarm', 'Display', 'iter');

objfun = @nnet_xor_objFun; % Specify objective function
[W,fval,exitFlag,output] = particleswarm(objfun, nvar, Lb, Ub,
options);

```

```
%% Design results
```

```
mae = nnet_xor_objFun(W); % mean absolute error
yr = nnet_xor_verify(W,input); % recall output
fprintf('MAE: %18.15f\n', mae);
fprintf('Yr: %18.15f %18.15f %18.15f %18.15f\n', yr);
fprintf('W: %18.15f %18.15f %18.15f %18.15f %18.15f ...
%18.15f %18.15f %18.15f %18.15f\n', W);
figure; stem(W); grid; title('Parameters'); % designed parameters
```

```
function [mae] = nnet_xor_objFun(W)
```

```
w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);
input = [0,0; 0,1; 1,0; 1,1];
dout = [0; 1; 1; 0];
```

```
sae = 0; % set initial value for sum of absolute errors
```

```
alp=-2;
```

```
for j = 1:4
```

```
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
```

```
    y1 = 1/(1+exp((alp)*z1));
```

```
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
```

```
    y2 = 1/(1+exp((alp)*z2));
```

```
    z = y1*w5+(-1)*b3+y2*w6;
```

```
    y(j) = 1/(1+exp((-2)*z));
```

```
    ae(j) = (abs(y(j)-dout(j)));
```

```
    sae = sae + ae(j);
```

```
end
```

```
mae = sae/4; % mean absolute error
```

```
end
```

```
function yr=nnet_xor_verify(W,input)
```

```
w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);
```

```

alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z  = y1*w5+(-1)*b3+y2*w6;
    yr(j) = 1/(1+exp((-2)*z));
end
end

```

Table E15.3.3 Design parameters and result of XOR problem using particle swarm optimization.

Weights (w1 to w6)	Biases (b1 to b3)
-15.734394915099106	8.877842595930224
15.973499468096929	7.916467507328640
15.955975966819167	7.999702739680599
-16.000000000000000	
16.000000000000000	Mean absolute error
16.000000000000000	1.125361904760153e-07

Design Example 15.4 (2-layer feedforward neural network learning of XOR problem using PSO - II)

Using the IC Matlab Toolbox pso_ic function to write a Matlab program to repeat the design of a 2-layer neural network with real-valued parameters to learn the 4 input to output pairs of the XOR problem as described in Design Example 15.3.

Solution

The Matlab program is listed in Table E15.4.1, the design results are listed in Table E15.4.2, and the convergence curve is plotted in Figure E15.4.1.

Table E15.4.1 Matlab program for XOR design problem using particle swarm optimization (pso_ic.m).

```
% xor_pso_ic.m
% 2-layer neural network (with a bias at each neuron) design
% IC Matlab Toolbox pso_ic function

clc; clear; close all;
format long;

%% XOR design problem

input = [0 0; 0 1; 1 0; 1 1];
dout = [0; 1; 1; 0];
nvar=9; % Number of variables
varSize=[1 nvar];
% Lower & upper bounds of variables
Lb=-16*ones(1,nvar);
Ub=16*ones(1,nvar);

%% PSO Parameters

maxIter=200*nvar; % Maximum number of iterations
P=min(100,10*nvar); % Population size (Swarm Size)

w=1; % Inertia Weight
wdamp=0.99; % Inertia Weight Damping Ratio
c1=1.5; % Personal Learning Coefficient
c2=1.5; % Global Learning Coefficient

[BestSol,BestCost] =
pso_ic(nvar,varSize,Lb,Ub,P,w,wdamp,c1,c2,maxIter);

%% Design results

figure; %plot(BestCost, 'LineWidth', 1);
semilogy(BestCost, 'LineWidth', 1);
xlabel('Iteration'); ylabel('Best cost'); grid;
```

```

mae = nnet_xor_objFun(BestSol.Position); % mean absolute error
yr = nnet_xor_verify(BestSol.Position,input); % recall output
fprintf('MAE: %18.15f\n', mae);
fprintf('Yr: %18.15f %18.15f %18.15f %18.15f\n', yr);
fprintf('W: %18.15f  %18.15f %18.15f %18.15f %18.15f ...
%18.15f %18.15f %18.15f %18.15f  \n', BestSol.Position);
figure; stem(BestSol.Position); grid; title('Parameters');

```

```

function [mae] = nnet_xor_objFun(W)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7);  b2=W(8); b3=W(9);
input = [0,0; 0,1; 1,0; 1,1];
dout = [0; 1; 1; 0];

sae = 0;    % set initial value for sum of absolute errors
alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z  = y1*w5+(-1)*b3+y2*w6;
    y(j) = 1/(1+exp((-2)*z));
    ae(j) = (abs(y(j)-dout(j)));
    sae = sae + ae(j);
end
mae = sae/4;    % mean absolute error
end

```

```

function yr=nnet_xor_verify(W,input)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7);  b2=W(8); b3=W(9);

alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);

```

```

y1 = 1/(1+exp((alp)*z1));
z2 =w2*input(j,1)+(-1)*b2+w4*input(j,2);
y2 = 1/(1+exp((alp)*z2));
z  = y1*w5+(-1)*b3+y2*w6;
yr(j)  = 1/(1+exp((-2)*z));

end
end

```

Table E15.4.2 Design parameters and result of XOR problem using particle swarm optimization (pso_ic.m).

Weights (w1 to w6)	Biases (b1 to b3)
15.999993575822840	8.173175103422576
-15.999989281016305	8.172179836569237
-15.999999944379299	7.999999404146735
15.999990278615885	
15.999999999998234	Mean absolute error
15.999999999887224	1.125357351306150e-07

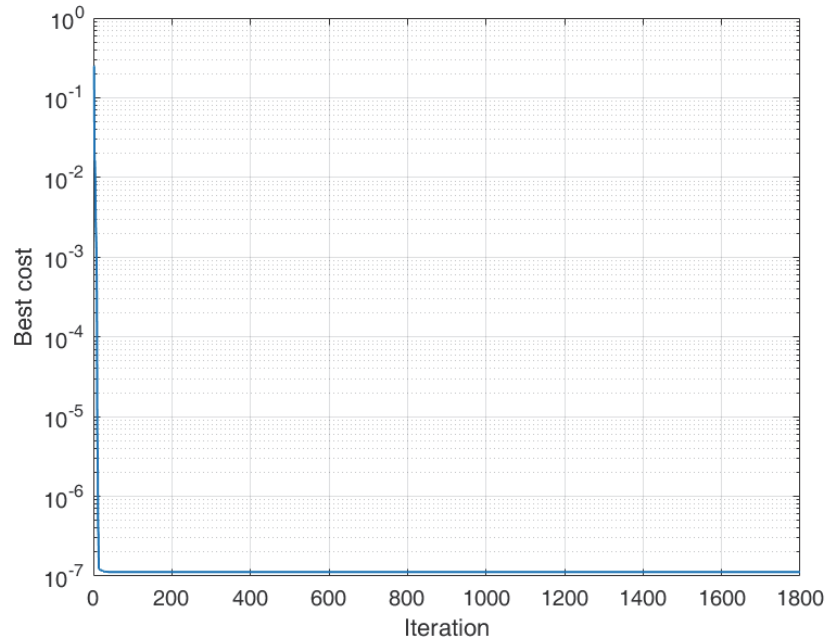


Figure E15.4.1 Convergence curve for XOR design problem using particle swarm optimization (pso_ic.m).

Design Example 15.5 (2-variable Sphere function minimization using BPSO)

This example serves to illustrate the steps on how to minimize the following objective function $f(\mathbf{X})$ with discrete values $\mathbf{X} = [x_1, x_2]$.

$$f(\mathbf{X}) = x_1^2 + x_2^2$$

using the binary particle swarm optimization.

For this problem, it can be observed that the optimal solution for $\mathbf{X} = [x_1, x_2] = [0, 0]$, and the ranges of the search space can be set as $-V_{max} < x_1, x_2 < V_{max}$.

The search continues until the objective function of a particle falls below a specified error limit δ .

For explanation, the calculation steps of the first three (3) generations plus the last generation are to be shown.

Assuming that the LBest neighborhood of a particle p includes itself only in all generations; and the GBest neighborhood includes all the particles of a population in all generations.

The size of the population P is taken as 4. Each particle has $N_c = 2$ parameters with a length of $D = 2B$ binary bits to be encoded as a parameter vector $\mathbf{c}$.

For $B_i = 1$ and B binary-bit representation, assign 1 sign bit and $B_f = B - 1$ fractional bits to represent a parameter.

The value of the slope parameter of the binary sigmoidal function to be used as the saturation function is taken as $r_f = 2$ with an input range of $\pm V_{max} = \pm 1.472$.

Initial coefficients are generated within $\pm V_{max}$ using the 2-sided exponential distribution function as explained in Chapter 16, Section 16.3.2 and then quantized to B binary bits.

The two particle velocity update constants are set to $c_1 = c_2 = 1$; consequently, the sum of two acceleration coefficients φ_1 and φ_2 is less than 2. The output error limit is taken as $\delta = 10^{-5}$.

A summary of the suggested parameter values is given in Table E15.5.1.

Table E15.5.1 BPSO parameters for sphere function minimization.

P	N_c	B_i	B_f	B	D	r_f	V_{max}	c_1	c_2	$\varphi_1 + \varphi_2$	δ
4	2	1	7	8	$N_c B$	2	1.472	1	1	≤ 2	10^{-5}

Design Example 15.6 (Third-order polynomial function interpolation using BPSO)

This example shows the steps on how the binary particle swarm optimization can be used to optimize the coefficients of a third-order polynomial function given by

$$y(x) = a_3x^3 - a_2x^2 - a_1x + a_0$$

to fit N number of input and output data pairs.

A set of five ($N = 5$) uniformly spaced fixed inputs $[x_1, x_2, x_3, x_4, x_5] = [0.5, 0.25, 0, -0.25, -0.5]$ are used to compute $y(x)$ as desired outputs.

The mean squared error between the desired output and the obtained output by a particle is used as the error measurement and for comparison (not fitness function) among particles.

The search continues until the mean squared error falls of a particle below a specified error limit δ .

For explanation, the calculation steps of the first three (3) generations plus the last generation are to be shown. Hint: The optimal interpolation function is $y(x) = 10x^3 - 3x^2 - 2.4x + 1$.

Assuming that the LBest neighborhood of a particle p includes itself only in all generations; and the GBest neighborhood includes all the particles of a population in all generations.

In this problem, it is noted that the four optimal parameters $[a_0, a_1, a_2, a_3]$ fall under different ranges of values and therefore different slope parameters are preferably be used in the probability function of (15.5.4) and different number of integer bits are preferably be used for coefficient representation as summarized in Table E15.6.1.

Table E15.6.1 Probability function slope parameters and coefficient wordlength.

	a_0	a_1	a_2	a_3
r_f	2	1	0.5	0.25
V_{max}	1.472	2.944	5.888	11.78
B_i	2	3	4	5
B_f	6	5	4	3
B	8	8	8	8

The size of the population P is taken as 4. Each particle has $N_c = 4$ parameters with a length of $D = 4B$ binary bits to be encoded as a parameter vector $\mathbf{c}$.

For binary-bit representation, assign 1 sign bit, B_i integer bits, and $B_f = B - 1$ fractional bits to represent each parameter.

Initial coefficients are generated within $\pm V_{max}$ using the 2-sided exponential distribution function as explained in Chapter 16, Section 16.3.2 and then quantized to B binary bits.

The two particle velocity update constants are set to $c_1 = c_2 = 1$; consequently, the sum of two acceleration coefficients φ_1 and φ_2 is less than 2.

The mean squared output error limit is taken as $\delta = 10^{-5}$. A summary of the suggested parameter values is given in Table E15.6.2.

Table E15.6.2 BPSO parameters for third-order polynomial function interpolation.

P	N_c	B	D	N	c_1	c_2	$\varphi_1 + \varphi_2$	δ
6	4	8	$N_c B$	5	1	1	≤ 2	10^{-5}

Design Example 15.7 (2-layer feedforward neural network learning of XOR problem using BPSO)

This example explains the steps on how to design a 2-layer neural network with discrete-valued parameters to learn using the binary particle swarm optimization the 4 input to output pairs of the XOR problem as described in Chapter 4, Example 4.1.

The search continues until the mean squared output error of a particle falls below δ .

Assuming that the LBest neighborhood of a particle p includes itself and its nearest particles with a total of 25% of a population in all generations; and the GBest neighborhood includes all the particles of a population in all generations.

The coefficient search space is

$$-s < w_1, w_2, w_3, w_4, w_5, w_6, b_1, b_2, b_3 < s \quad (\text{E15.7.1})$$

The size of the population P is taken as 90. Let the neural network parameters' range be $\pm s = \pm 16$.

For $s = 2$, we have $B_i = 2$ and for B binary-bit representation, assign 1 sign bit, 1 integer bit and $B_f = B - 2$ fractional bits to represent a parameter.

Each particle has $N_c = 9$ parameters with a length of $D = N_c B$ binary bits to be encoded as a parameter vector $\mathbf{c}$.

The slope parameter of the binary sigmoidal function at each neuron's output is taken as $r_n = 2$ with a unity slope at zero input and an input range of $\pm L = \pm 2$ before saturating to 0 and 1 outputs [Kwa9207].

The value of the slope parameter of the binary sigmoidal function to be used as the saturation function is taken as $r_f = 2$ with an input range of $\pm V_{max} = \pm 1.472$ (See Table 15.5.3).

Initial coefficients are generated within $\pm s$ using the 2-sided exponential distribution function as explained in Chapter 16, Section 16.3.2 and then quantized to B binary bits.

The two particle velocity update constants are set to $c_1 = c_2 = 1.49$; consequently, the sum of two acceleration coefficients φ_1 and φ_2 is less than 3. The range of the decreasing inertia weight is $[w_1, w_2] = [1.1, 0.1]$.

The mean squared output error limit is taken as $\delta = 0.000001$. A summary of the suggested parameter values is given in Table E15.7.1.

Table E15.7.1 BPSO parameters for 2-layer neural network.

P	N_c	s	B_i	B_f	B	D	r_n	L	r_f	V_{max}	c_1	c_2	$\varphi_1 + \varphi_2$	δ
90	9	16	2	6	8	$N_c B$	2	2	2	1.472	1.49	1.49	≤ 3	e-6

15.6 Remarks

In some problems, both continuous and binary variables are used which calls for manipulation of both continuous and binary form of variables.

A hybrid model of the particle swarm optimization integrates both continuous parameters and binary parameters in the same problem.

In the process, the continuous particle swarm optimization is used to perform a search in the continuous space and the binary particle swarm optimization is used to search in the binary space.

The particle swarm algorithm imitates human behaviours and its strength lies in its simplicity. The particle swarm optimization has been used to design neural networks, fuzzy systems, fuzzy neural networks, digital filters, adaptive digital filters, and to solve other engineering problems.

The particle swarm algorithm has also shown strength in some mathematical problems such as multiple optima, corrugation, serration, cavities and bumps, high dimensionality, infeasible regions, epistasis, and dynamic distortions and deception.

No free lunch theorem suggests that no algorithm can optimize better than any other on any general problem. This means that different problems could have different solutions best fitted to them. The only way to find which one is the best is to try them on different problems and to evaluate their strengths and weaknesses.

Chapter 16

Simulated Annealing

16.1 Introduction

Simulated annealing (SA) [Sco8305] is a metaheuristic algorithm. It is a trajectory random search based technique for global optimization.

It has been proved that SA will converge to its global optimality for sufficient randomness and very slow cooling.

Simulated annealing mimics the annealing process in material processing when a metal cools and freezes into a crystalline state with the minimum energy and larger crystal size so as to reduce the defects in metallic structures.

The annealing process involves the careful control of temperature and its cooling rate called annealing schedule.

Simulated annealing improves this strategy through the introduction of two schemes.

The first scheme is called the **Metropolis algorithm** [Met5306] in which some solutions that do not lower the energy are accepted when they serve to allow the solver to explore more of the possible space of solutions.

Such poor solutions are allowed using the criterion that

$$e^{-\frac{\Delta e}{T}} > rand \text{ for } rand \in [0, 1) \quad (16.1.1)$$

In (16.1.1), T denotes a synthetic temperature; Δe denotes the change of energy with negative for a good solution and positive for a poor solution; $rand$ is a uniformly distributed random number within $0 \leq rand < 1$; and e is the cost function corresponds to the free energy in the case of annealing a metal.

For metal annealing, the temperature parameter would actually be the kT , where k is the Boltzmann's Constant and T is the physical temperature in Kelvin.

If T is large, many poor solutions are accepted but a large part of solution space is accessed.

Solutions to be exchanged are generally chosen randomly, though more sophisticated techniques can be used.

The second scheme is called the **annealing schedule** in which the temperature is lowered, again by analogy with the annealing of a metal.

After making many exchanges and observing that the cost function declines only slowly, the temperature T is lowered to limit the size of allowed poor solutions.

After lowering the temperature T several times to a low value, one may then quench the process by accepting only good solutions in order to find the local minimum of the cost function.

There are various annealing schedules for lowering the temperature T , but the results are generally not very sensitive to the details.

There is a faster alternative for annealing schedule called **threshold acceptance** [Due9009]. In this scheme, all good solutions are accepted, as are any poor solutions that raise the cost function by less than a fixed threshold.

The threshold is then periodically lowered, similar to lowering the temperature T in annealing.

16.2 Simulated Annealing

The simulated annealing algorithm is inspired by the annealing process in metal work.

Annealing is the process of heating a metal and then cooling it to give it a desirable characteristic by altering its atomic structure.

In simulated annealing, a temperature variable is used to simulate the heating and cooling process in metal work. Initially it is set high and then allowed to slowly cool (decrease in temperature) as the algorithm progresses.

The profound connection between statistical mechanics for the behavior of a system with many degrees of freedom at a thermal equilibrium and the problem of multivariate or combinatorial optimization led to the development of the simulated annealing algorithm.

16.2.1 Algorithm

For a minimization problem, given an initial solution (or state) $\mathbf{x}$, its initial value of objective function (or error or energy) e is denoted by e_0 .

For the algorithm to progress, a new solution with error e_1 is generated by a random walk and the differential error (Δe) is calculated as

$$\Delta e = e_1 - e_0 \quad (16.2.1)$$

If the error of the new solution is lower than the current error, the algorithm accepts the new solution.

If the new solution has a larger error (with $\Delta e > 0$), then a transition occurs at iteration i according to a decreasing exponential probability function of Δe and T_i as

$$p(\Delta e, i) = e^{-\frac{\Delta e}{T_i}} \quad (16.2.2)$$

In (16.2.2), for a large T_i , $p(\Delta e, i) \rightarrow 1$; for a small T_i , $p(\Delta e, i) \rightarrow 0.0$. An alternative form for $p(\Delta e, i)$ is

$$p(\Delta e, i) = \frac{1}{1 + e^{\frac{\Delta e}{T_i}}} \quad (16.2.3)$$

where T_i is the temperature at the i th iteration. In (16.2.3), for a large T_i , $p(\Delta e, i) \rightarrow 0.5$; for a small T_i , $p(\Delta e, i) \rightarrow 0.0$.

Given $rand \in [0,1)$ is a uniformly distributed random number, the acceptance of a solution with $\Delta e > 0$ is given by (16.2.4).

$$P_r = \begin{cases} rand < p(\Delta e, i), & \text{accept} \\ rand \geq p(\Delta e, i), & \text{reject} \end{cases} \quad (16.2.4)$$

When temperature T_i is higher, the algorithm will accept solutions that are poorer than the current solution with a higher probability.

This enables the algorithm an ability to jump out of a local optimum.

As the temperature T_i is lowered, the probability of accepting poorer solutions is also reduced.

This gradually allows the simulated annealing algorithm to concentrate on an area of the search space which is most likely to contain the global optimum solution.

The temperature T_i is varied according to an annealing schedule. Many different annealing schedules can be developed and the success of a particular annealing schedule depends on a specific problem.

One such annealing schedule is the exponential cooling as given by

$$T_i = T_0 e^{-\frac{i}{K}} \quad (16.2.5)$$

where T_0 is the initial temperature and K is the annealing parameter.

The annealing schedule defined by (16.2.5) can be replaced by a simple cooling schedule as

$$T_i = \alpha T_{i-1} \text{ for } 0 < \alpha < 1 \quad (16.2.6)$$

Typical value of α ranges from 0.7 to 0.99.

16.2.2 Choices of parameter values

For solving a problem using simulated annealing, three elements are required: (i) an objective (or error) function of the problem; (ii) a random walking mechanism to generate a new solution; and (iii) an annealing schedule for the temperature and maximum number of iterations to evolve a solution.

In practice, there is a fine balance between the solution quality and the number of random walks.

There is a choice between “higher number of iterations and lower number of random walks” and “lower number of iterations and higher number of random walks”.

In general, a fixed number of random walks are applied at each higher temperature, while the number of random walks can be increased at lower temperatures to fully explore a local minima.

Given a probability p_0 and if the maximum change $\max(\Delta e)$ of the objective function is known, an initial temperature T_0 can be estimated from (16.2.2) as

$$T_0 \approx -\frac{\max(\Delta e)}{\log_e p_0} \quad (16.2.7)$$

However if a possible maximum change of the objective function is not known, a heuristic approach can be used to estimate T_0 by starting evaluations from a very high temperature (so that almost all changes are accepted) and reduce the temperature quickly until about 50% or 60% of the worse moves are accepted, and then use this temperature as the initial temperature T_0 for slow cooling.

The final temperature should be ideally zero so that no worse move can be accepted. However, if $T_{\max_iter} = 0$, more iterations are required.

In practice, a very small value $T_{\max_iter}$ such as 10^{-10} to 10^{-5} can be used depending on the desired quality of the solution and time constraint.

16.3 Pseudocode for Simulated Annealing

Table 16.3.1 Pseudocode for populated-based simulated annealing (Set $P=1$ for population=1).

Step	Description
1	Set population size, P ; Set $T = T_0$ and a fixed value for α ; Set maximum number of iterations (<code>max_iter</code>); Set maximum number of random walk (<code>max_walk</code>);

	Set initial population $[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_P]$ with each p th population member consisting of $nvar$ parameters within bounds, calculate its initial error e_p for $p = 1$ to P ;
2	for $i = 1$ to $\max_iter$
3	for $j = 1$ to $\max_walk$
4	for $p = 1$ to P
5	for $k = 1$ to M
6	Generate a new member $\mathbf{x}'_p$ by random walk (or another optimization algorithm) within bounds;
7	Calculate energy of the new member, e'_p ;
8	end
9	end
10	Sort and select top $\mathbf{x}'_p$;
11	for $p = 1$ to P
12	if $\Delta e_p = [e'_p - e_p]/e_p < 0$
13	$\mathbf{x}_p = \mathbf{x}'_p$; $e_p = e'_p$;
14	else
15	$prob(\Delta e_p, i) = e^{-\frac{\Delta e_p}{T_i}}$;
16	if $rand \leq prob(\Delta e_p, i)$
17	$\mathbf{x}_p = \mathbf{x}'_p$; $e_p = e'_p$;
18	end
19	end
20	Update $\mathbf{x}_{best}$ and display e_{min} ;
21	end
22	Check stopping criteria;
23	end
24	$T_i = \alpha T_{i-1}$;
25	end

Design Example 16.1 (2-layer feedforward neural network learning of XOR problem using SA - I)

Using the Global Optimization Toolbox of MathWorks, write a Matlab program to design a 2-layer neural network with real-valued parameters using the simulated annealing to learn the 4 input to output pairs of the XOR problem as described according to Chapter 4, Example 4.1.

The coefficient search space is

$$-s < w_1, w_2, w_3, w_4, w_5, w_6, b_1, b_2, b_3 < s \quad (\text{E16.1.1})$$

The size of the population P is taken as 90. Let the neural network parameters' range be $\pm s = \pm 16$. Each particle has $N_c = 9$ parameters with a length of $D = N_c$ real-values to be encoded as a parameter vector $\mathbf{c}$.

Initial coefficients are generated randomly within $\pm s$.

The search continues until the output mean absolute error of a particle falls below δ .

A summary of the suggested parameter values is given in Table E16.1.1.

Table E16.1.1 SA parameters for 2-layer neural network.

P	N_c	D	s	r_n	L	T_0	δ
90	9	N_c	16	2	2	100	0.000001

Solution

The Matlab program for designing the XOR problem using simulated annealing is listed in Table E16.1.2. The designed parameters and the mean absolute error of the 2-layer neural networks are listed in Table E16.1.1.

Table E16.1.2 Matlab program for XOR design problem using simulated annealing.

```
% nnet_xor_sa_mFun.m
% Use of Global Optimization Matlab Toolbox SA function for
% 2-layer neural network (with a bias at each neuron) design

clc; clear; close all;
format long;

%% XOR design problem

input = [0 0; 0 1; 1 0; 1 1];
dout = [0; 1; 1; 0];
nvar=9; % Number of variables
% Lower & upper bounds of variables
Lb=[-16,-16,-16,-16,-16,-16,-16,-16,-16];
Ub=[16,16,16,16,16,16,16,16,16];

%% SA (Set parameters for optimization)

% Default parameters
% FunctionTolerance: 1.000000000000000e-06
% InitialTemperature: 100
% MaxFunctionEvaluations: '3000*numberOfVariables'
% MaxStallIterations: '500*numberOfVariables'
% ReannealInterval: 100

options = optimoptions(@simulannealbnd,'MaxIter', 30000);
options = optimoptions(@simulannealbnd,'TolFun', 1e-6);
options = optimoptions(@simulannealbnd,'Display','iter');

objfun = @nnet_xor_objFun; % Specify objective function

W0=unifrnd(Lb,Ub,[1 nvar]);
[W,fval,exitFlag,output] = simulannealbnd(objfun, W0, Lb, Ub,
options);

%% Design results
```

```

mae = nnet_xor_objFun(W); % mean absolute error
yr = nnet_xor_verify(W,input); % recall output
fprintf('MAE: %18.15f\n', mae);
fprintf('Yr: %18.15f %18.15f %18.15f %18.15f\n', yr);
fprintf('W: %18.15f %18.15f %18.15f %18.15f %18.15f ...
%18.15f %18.15f %18.15f %18.15f\n', W);
figure; stem(W); grid; title('Parameters');

```

```

function [mae] = nnet_xor_objFun(W)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);
input = [0,0; 0,1; 1,0; 1,1];
dout = [0; 1; 1; 0];

sae = 0; % set initial value for sum of absolute errors
alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z = y1*w5+(-1)*b3+y2*w6;
    y(j) = 1/(1+exp((-2)*z));
    ae(j)= (abs(y(j)-dout(j)));
    sae= sae+ ae(j);
end
mae= sae/4; % mean absolute error
end

```

```

function yr=nnet_xor_verify(W,input)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);

% recall using binary sigmoid activation function
alp=-2;

```

```

for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z = y1*w5+(-1)*b3+y2*w6;
    yr(j) = 1/(1+exp((-2)*z));
end
end

```

Table E16.1.3 Design parameters and result of XOR problem using simulated annealing.

Weights (w1 to w6)	Biases (b1 to b3)
-14.001231410279441	-11.913576804609052
-12.7716151115861580	1.465817787941993
7.597187436395370	-7.094989893185989
11.604434688730850	
-14.088141102507640	Mean absolute error
15.953065588436454	1.575318778505788e-06

Design Example 16.2 (2-layer feedforward neural network learning of XOR problem using SA - II)

Using the IC Matlab Toolbox sa_ic function to write a Matlab program to repeat the design a 2-layer neural network with real-valued parameters using the simulated annealing to learn the 4 input to output pairs of the XOR problem as described in Design Example 16.1.

Solution

The Matlab program is listed in Table E16.2.1, the designed results are listed in Table E16.2.2, and the convergence curve is plotted in Figure E16.2.1

Table E16.2.1 Matlab program for XOR design problem using simulated annealing (sa_ic.m).

```

% xor_sa_ic.m
% 2-layer neural network (with a bias at each neuron) design using
% IC Matlab Toolbox sa_ic.m function

clc; clear; close all;
format long;

%% XOR design problem

input = [0 0; 0 1; 1 0; 1 1];
dout = [0; 1; 1; 0];
nvar=9; % Number of variables
varSize=[1 nvar];
% Lower & upper bounds of variables
Lb=-16*ones(1,nvar);
Ub=16*ones(1,nvar);

%% SA (Set parameters for optimization)

T0=0.1; % Initial temperature
alpha=0.99; % Temperature reduction rate
maxIter=200*nvar; % Maximum number of iterations
maxIterSub=100*nvar; % Maximum number of sub-iterations

[BestSol,BestCost] =
sa_ic(nvar,varSize,Lb,Ub,T0,alpha,maxIter,maxIterSub);

%% Design results

figure; semilogy(BestCost, 'LineWidth', 1);
xlabel('Iteration'); ylabel('Best cost'); grid;

mae = nnet_xor_objFun(BestSol.Position); % mean absolute error
yr = nnet_xor_verify(BestSol.Position,input); % recall output
fprintf('MAE: %18.15f\n', mae);
fprintf('Yr: %18.15f %18.15f %18.15f %18.15f\n', yr);

```

```
fprintf('W: %18.15f %18.15f%18.15f%18.15f%18.15f ...
%18.15f%18.15f%18.15f%18.15f \n', BestSol.Position);
figure; stem(BestSol.Position); grid; title('Parameters');
```

```
function [mae] = nnet_xor_objFun(W)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);
input = [0,0; 0,1; 1,0; 1,1];
dout = [0; 1; 1; 0];

sae = 0; % set initial value for sum of absolute errors
alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z = y1*w5+(-1)*b3+y2*w6;
    y(j) = 1/(1+exp((-2)*z));
    ae(j)= (abs(y(j)-dout(j)));
    sae= sae+ ae(j);
end
mae= sae/4; % mean absolute error
end
```

```
function yr=nnet_xor_verify(W,input)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);

% recall using binary sigmoid activation function
alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
```



```

    z = y1*w5+(-1)*b3+y2*w6;
    yr(j) = 1/(1+exp((-2)*z));
end
end

```

Table E16.2.2 Design parameters and result of XOR problem using simulated annealing (sa_ic.m).

Weights (w1 to w6)	Biases (b1 to b3)
12.652310808424868	-10.257319327727277
15.738285341281326	12.696647200064131
-15.301931761722866	-7.533645697969700
-12.391728181369999	
-14.912129503011808	Mean absolute error
15.913794024047327	2.769860710489019e-07

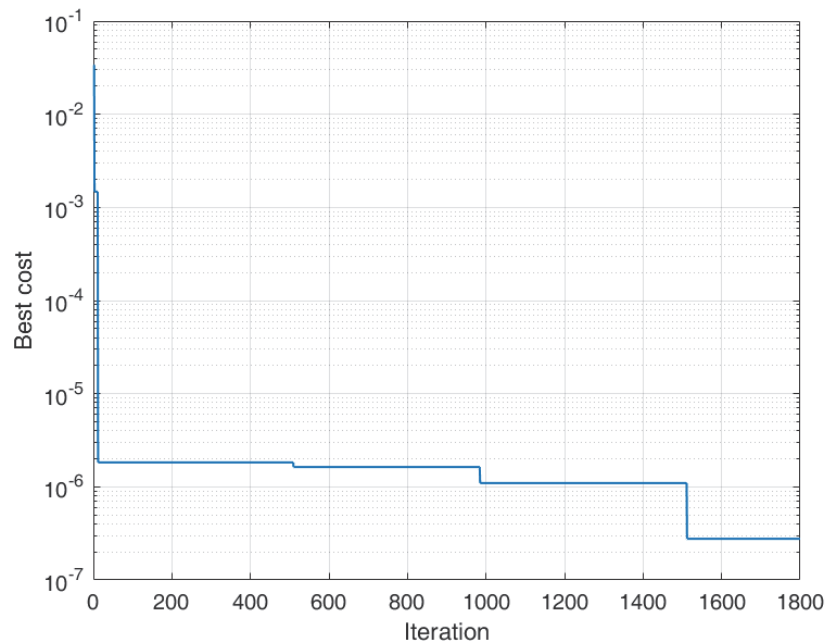


Figure E16.2.1 Convergence curve for XOR design problem using simulated annealing (sa_ic.m).

16.4 Population-Based Simulated Annealing

SA selects a starting solution, produces a new solution at the nearby region, and makes a decision by some rules to move to the new solution or not. However, the results found by SA depend on the selection of the starting point and the decisions SA makes. Very often, a local optimum is found.

To enable SA to search different regions to increase the chance of reaching a better solution or even the global optimum, the SA can be modified to become a population-based SA algorithm. In so doing, all population members are used to arrive at the best solution in each iteration.

Design Example 16.3 (2-layer feedforward neural network learning of XOR problem using population-based SA)

Using the population-based SA, write a Matlab program to design a 2-layer neural network with real-valued parameters using the simulated annealing to learn the 4 input to output pairs of the XOR problem as described according to Chapter 4, Example 4.1.

Solution

The Matlab program is listed in Table E16.3.1, the designed results are listed in Table E16.3.2, and the convergence curve is plotted in Figure E16.3.1

Table E16.3.1 Matlab program for XOR design problem using simulated annealing (sa_pop_ic.m).

```
% xor_sa_pop_ic.m
% 2-layer neural network (with a bias at each neuron) design using
% IC Matlab Toolbox sa_pop_ic.m function

clc; clear; close all;
format long;

%% XOR design problem
```

```

input = [0 0; 0 1; 1 0; 1 1];
dout = [0; 1; 1; 0];
nvar=9; % Number of variables
varSize=[1 nvar];
% Lower & upper bounds of variables
Lb=-16*ones(1,nvar);
Ub=16*ones(1,nvar);

%% SA (Set parameters for optimization)

nP=nvar; % Population size
T0=0.1; % Initial temperature
alpha=0.99; % Temperature reduction rate
maxIter=100*nvar; % Max number of iterations
maxIterSub=50*nvar; % Max number of sub-iterations

[BestSol,BestCost] =
sa_pop_ic(nvar,varSize,Lb,Ub,nP,T0,alpha,maxIter,maxIterSub);

%% Design results

figure; semilogy(BestCost, 'LineWidth', 1);
xlabel('Iteration'); ylabel('Best cost'); grid;

mae = nnet_xor_objFun(BestSol.Position); % mean absolute error
yr = nnet_xor_verify(BestSol.Position,input); % recall output
fprintf('MAE: %18.15f\n', mae);
fprintf('Yr: %18.15f %18.15f %18.15f %18.15f\n', yr);
fprintf('W: %18.15f %18.15f %18.15f %18.15f %18.15f ...
%18.15f %18.15f %18.15f %18.15f\n', BestSol.Position);
figure; stem(BestSol.Position); grid; title('Parameters');

function [mae] = nnet_xor_objFun(W)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);
input = [0,0; 0,1; 1,0; 1,1];
dout = [0; 1; 1; 0];

```

```
sae = 0; % set initial value for sum of absolute errors
alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z = y1*w5+(-1)*b3+y2*w6;
    y(j) = 1/(1+exp((-2)*z));
    ae(j)= (abs(y(j)-dout(j)));
    sae= sae+ ae(j);
end
mae= sae/4; % mean absolute error
end

function yr=nnet_xor_verify(W,input)

w1=W(1); w2=W(2); w3=W(3); w4=W(4); w5=W(5); w6=W(6);
b1=W(7); b2=W(8); b3=W(9);

% recall using binary sigmoid activation function
alp=-2;
for j = 1:4
    z1 = w1*input(j,1)+(-1)*b1+w3*input(j,2);
    y1 = 1/(1+exp((alp)*z1));
    z2 = w2*input(j,1)+(-1)*b2+w4*input(j,2);
    y2 = 1/(1+exp((alp)*z2));
    z = y1*w5+(-1)*b3+y2*w6;
    yr(j) = 1/(1+exp((-2)*z));
end
end
```

Table E16.3.2 Design parameters and result of XOR problem using simulated annealing (sa_pop_ic.m).

Weights (w1 to w6)	Biases (b1 to b3)
-7.315399046762696	8.122761758863273
-7.859347871488538	-5.586471992837126
11.931760705854384	-8.657389218940025
10.376565376908303	
15.599706477244414	Mean absolute error
-15.981175010657907	2.477413880712740e-07

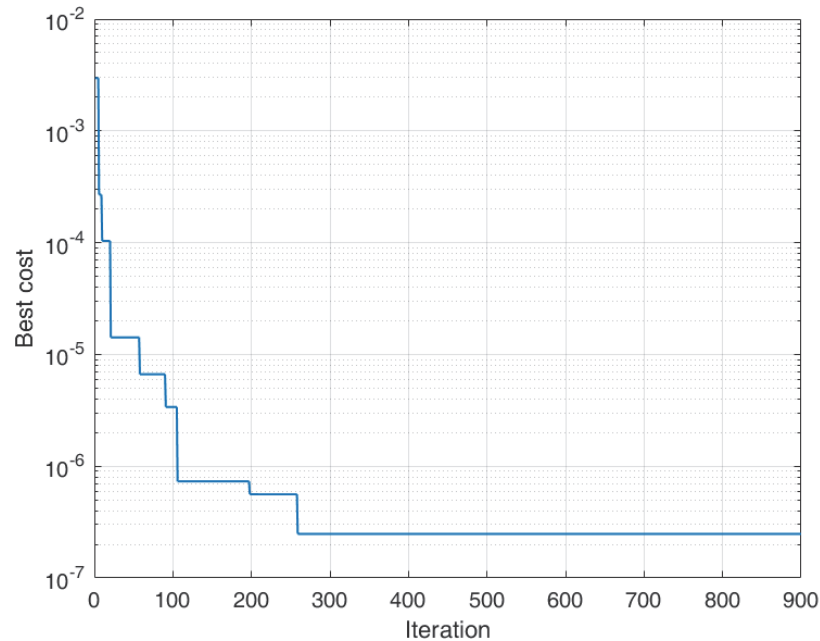


Figure E16.3.1 Convergence curve for XOR design problem using simulated annealing (sa_pop_ic.m).

16.5 Remarks

In practice, some optimization problems (like the traveling salesman problem) become unmanageable as the number of parameters (or cities)

becomes large. For such problems, simulated annealing offers a possible solution.

While simulated annealing is unlikely to find an optimum solution, it can often find a very good solution, even in the presence of noisy data.

The chance for getting a better solution or the optimal solution increases by using a population-based SA.

Chapter 17

Coding, Initialization, and Constraints

17.1 Introduction

In this eBook, four commonly known evolutionary algorithms have been explained.

This chapter describes coding, initialization, constraints, and multi-objective optimization which form a set of related background topics to facilitate the use of various evolutionary algorithms for optimization applications.

17.2 Parameter Coding Methods

There are a number of coding methods to address the number representation to be used in an evolutionary algorithm.

Coding methods can be represented in real number, integer number, binary number, octal number, and hexadecimal number.

Examples of parameter vector represented in different number systems are listed in Table 17.2.1.

Table 17.2.1 Parameter vectors represented by different coding methods.

Representation	Parameter Vector
Real number	$[-2.3574, 0.8703, -2.3582, 1.7693, 2.1298, -0.5289]$
Integer number	$[23, -11, 50, 30, -4, 65]$
Binary number	$[01101, 10010, 01011, 10110, 00101, 11010]$
Octal number (0-7)	$[5, 3, 7, 2, 4, 1]$
Hexadecimal number (0-9, A-F)	$[D2F1BD]$

Among all the number representations, the real and binary number representations are the most popular choices respectively for continuous and discrete encoding methods.

In binary encoding, each parameter of an optimization problem is mapped to an unsign-magnitude binary integer number or to a sign-magnitude binary fixed-point number.

For number representation expressed in sign, integer, and fractional values, binary coding methods include sign-magnitude binary fixed-point coding, and canonic sign-digit coding.

As compared to floating-point representation, sign-magnitude binary fixed-point coding and canonic sign-digit coding can be implemented by special-purpose but finite precision hardware for improved speed performance.

Concatenated multi-parameter unsign-magnitude integer binary coding first described below can be extended to other types of number representation, such as concatenated multi-parameter sign-magnitude binary fixed-point coding, and concatenated multi-parameter canonic sign-digit coding, as well as concatenated multi-parameter real-valued coding.

17.2.1 Unsign-magnitude binary integer coding

Consider a set of parameters with real-values $\in [V_{min}, V_{max}]$ in which each of them is to be coded by an unsigned integer $\in [0, 2^L]$ linearly within the specified interval $[V_{min}, V_{max}]$.

The precision value p_v for each coded parameter is given by

$$p_v = (V_{max} - V_{min}) / (2^L - 1) \quad (17.2.1)$$

With the above set of parameter values, real-valued parameters can be binary coded.

Multi-parameter coding can be constructed by concatenating a number of single-parameter codes.

Each code can have its sub-length and its own V_{max} and V_{min} .

The sub-length of each code can be different to accommodate the precision requirement of each individual code.

Example 17.1 (Concatenated multi-parameter unsign-magnitude binary integer coding)

Assume that a function of four parameters $f(x_1, x_2, x_3, x_4)$ is to be minimized.

The ranges of the four parameters are: $x_1 \in [-1, 1]$, $x_2 \in [-2, 2]$, $x_3 \in [-3, 3]$, and $x_4 \in [-4, 4]$.

The desired precision values for x_1, x_2, x_3, x_4 are 0.1, 0.2, 0.3, and 0.4, respectively.

- Design a concatenated multi-parameter integer binary coding for this problem.
- Determine the minimum number of bits required to obtain the desired precision.
- With the designed integer binary coding method, write down the strings that represent the following point $(x_1, x_2, x_3, x_4) = (-0.1, -1.1, -2.1, -3.1)$.

Solution

- The concatenated multi-parameter integer binary coding for this problem can be designed as shown in Table E17.1.1.

Table E17.1.1 Concatenated multi-parameter unsign-magnitude binary integer coding.

x_1	x_2	x_3	x_4
$aaa \dots a$	$bbb \dots b$	$ccc \dots c$	$ddd \dots d$

where ‘ $aaa \dots a$ ’ is a bit-parameter representing x_1 , ‘ $bbb \dots b$ ’ is a bit-parameter representing x_2 , ‘ $ccc \dots c$ ’ is a bit-parameter representing x_3 , and ‘ $ddd \dots d$ ’ is a bit-parameter representing x_4 respectively.

The actual number of bits for each individual coding is determined by its range and precision.

(b) The desired precision values p_v of x_1, x_2, x_3, x_4 are respectively given as 0.1, 0.2, 0.3, and 0.4, the minimum numbers of bits required to obtain the desired precisions can be determined as

$$p_v = \frac{V_{max} - V_{min}}{2^L - 1} \quad \text{or} \quad 2^L - 1 = \frac{V_{max} - V_{min}}{p_v}$$

For x_1 :

$$\frac{1 - (-1)}{2^L - 1} = 0.1 \rightarrow \frac{2}{0.1} = 2^L - 1 = 20 \rightarrow 2^L = 21 \rightarrow L \approx 4.3923$$

$$\rightarrow L_{x_1} = 5$$

For x_2 :

$$\frac{2 - (-2)}{2^L - 1} = 0.2 \rightarrow \frac{4}{0.2} = 2^L - 1 = 20 \rightarrow 2^L = 21 \rightarrow L \approx 4.3923$$

$$\rightarrow L_{x_2} = 5$$

For x_3 :

$$\frac{3 - (-3)}{2^L - 1} = 0.3 \rightarrow \frac{6}{0.3} = 2^L - 1 = 20 \rightarrow 2^L = 21 \rightarrow L \approx 4.3923$$

$$\rightarrow L_{x_3} = 5$$

For x_4 :

$$\frac{4 - (-4)}{2^L - 1} = 0.4 \rightarrow \frac{8}{0.4} = 2^L - 1 = 20 \rightarrow 2^L = 21 \rightarrow L \approx 4.3923$$

$$\rightarrow L_{x_4} = 5$$

(c) For $(x_1, x_2, x_3, x_4) = (-0.1, -1.1, -2.1, -3.1)$, the strings can be determined as

$$\frac{x_1 - V_{\min}(x_1)}{p_{x_1}} = \frac{-0.1 - (-1)}{0.1} = \frac{0.9}{0.1} = 9 \rightarrow 01001$$

$$\frac{x_2 - V_{\min}(x_2)}{p_{x_2}} = \frac{-1.1 - (-2)}{0.2} = \frac{0.9}{0.2} = 4.5 \approx 5 \rightarrow 00101$$

$$\frac{x_3 - V_{\min}(x_3)}{p_{x_3}} = \frac{-2.1 - (-3)}{0.3} = \frac{0.9}{0.3} = 3 \rightarrow 00011$$

$$\frac{x_4 - V_{\min}(x_4)}{p_{x_4}} = \frac{-3.1 - (-4)}{0.4} = \frac{0.9}{0.4} = 2.25 \approx 2 \rightarrow 00010$$

$$\Rightarrow (01001 \ 00101 \ 00011 \ 00010)$$

17.2.2 Sign-magnitude binary integer coding

Example 17.2 gives an illustration on how sign-magnitude binary integer number can be used to represent real-valued parameters for discrete-value optimization.

Example 17.2 (Concatenated multi-parameter sign-magnitude binary integer coding)

Assuming that a function consisting of four parameters $f(x_1, x_2, x_3, x_4)$ is to be minimized and the parameters are to be expressed in sign-magnitude binary integer format with B_i integer bits including one sign bit.

The ranges of the four parameters are $x_p \in [-s_p, s_p]$ for $p = 1$ to 4 such that $x_1 \in [-3, 3]$, $x_2 \in [-7, 7]$, $x_3 \in [-15, 15]$, and $x_4 \in [-31, 31]$.

(a) Design a concatenated multi-parameter sign-magnitude binary integer coding for this problem.

(b) With the designed sign-magnitude binary integer coding, calculate the quantized parameter vector of the parameter vector

$[x_1, x_2, x_3, x_4]^T = [-2.7, 6.2, -10.3, 16.8]^T$ and expressed the quantized parameter values in discrete integer value format.

(c) Express the quantized parameter values of the quantized parameter vector in sign-magnitude binary integer format.

Solution

(a) The binary point of each sign-magnitude binary integer parameter is determined by its range.

For $x_1 \in [-3, 3]$, it can be represented by three integer bits to be used as one sign bit and two integer bits.

For $x_2 \in [-6, 6]$, it can be represented by four integer bits to be used as one sign bit and three integer bits.

For $x_3 \in [-10, 10]$, it can be represented by five integer bits to be used as one sign bit and four integer bits.

For $x_4 \in [-17, 17]$, it can be represented by six integer bits to be used as one sign bit and five integer bits.

Therefore, the concatenated multi-parameter sign-magnitude binary integer coding for this problem can be designed as shown in Table E17.2.1.

Table E17.2.1 Concatenated multi-parameter sign-magnitude binary integer coding.

x_1	x_2	x_3	x_4
<i>aaa</i>	<i>bbbb</i>	<i>ccccc</i>	<i>dddddd</i>

where ‘*aaa*’ is a bit-format representing x_1 , ‘*bbbb*’ is a bit-format representing x_2 , ‘*ccccc*’ is a bit-format representing x_3 , and ‘*dddddd*’ is a bit-format representing x_4 respectively.

(b) Given a parameter vector $[x_1, x_2, x_3, x_4]^T = [-2.7, 6.2, -10.3, 16.8]^T$, the quantized parameters of the quantized parameter vector are obtained by rounding each parameter to the nearest integer with a

resolution of $r_p = 1$ such that $x_{pq} = r_p \times [x_p/r_p]_R = [x_p]_R$ for $p = 1$ to 4 as

$$x_{1q} = [x_1]_R = [-2.7]_R = -3$$

$$x_{2q} = [x_2]_R = [6.2]_R = 6$$

$$x_{3q} = [x_3]_R = [-10.3]_R = -10$$

$$x_{4q} = [x_4]_R = [16.8]_R = 17$$

In the above four equations, the symbol $[.]_R$ denotes rounding to the nearest integer operation.

The quantized parameter vector of $[-2.7, 6.2, -10.3, 16.8]^T$ expressed in discrete integer value format is given by

$$[-3 \ 6 \ -10 \ 17]^T$$

(c) The quantized parameter vector expressed in sign-magnitude binary integer format is given by

$$[111 \ 0110 \ 11010 \ 010001]^T$$

Note that a first bit ‘1’ on the left represents a negative signed number and a first bit ‘0’ on the left represents a positive signed number.

17.2.3 Sign-magnitude binary fixed-point coding

By assigning B_f fractional bits on the right side of the binary point (representing $2^0 = 1$) of a sign-magnitude binary integer number of B_i integer bits, we obtain a sign-magnitude binary fixed-point number which consists of B_i integer bits and B_f fractional bits.

Note that the sign-magnitude binary integer number or the sign-magnitude binary fixed-point number contains a sign bit (with 1 representing negative “ $-ve$ ”, and 0 representing positive “ $+ve$ ”) located at each of their left-most bits.

In other words, the number of integer bits including the sign bit is B_i , and the number of fractional bits is B_f where $B = B_i + B_f$ is the total number of bits for sign-magnitude binary fixed-point number representation.

Example 17.3 (Concatenated multi-parameter sign-magnitude binary fixed-point coding)

Assuming that a function consisting of four parameters $f(x_1, x_2, x_3, x_4)$ is to be minimized and the parameters are to be expressed in sign-magnitude binary fixed-point format (with B_i integer bits including one sign bit, and B_f fractional bits) with $B = B_i + B_f = 8$.

The ranges of the four parameters are $x_p \in [-s_p, s_p]$ for $p = 1$ to 4 such that $x_1 \in [-1, 1]$, $x_2 \in [-2, 2]$, $x_3 \in [-4, 4]$, and $x_4 \in [-8, 8]$.

- Design a concatenated multi-parameter binary fixed-point coding for this problem.
- Determine the resolutions of the quantized parameter values of x_1, x_2, x_3 , and x_4 .
- With the designed binary fixed-point coding, calculate the quantized parameter vector of the parameter vector $[x_1, x_2, x_3, x_4]^T = [0.1, -1.8, 3.6, -7.3]^T$ and expressed the quantized parameter values in discrete real value format.
- Express the quantized parameter values of the quantized parameter vector in sign-magnitude binary fixed-point format.

Solution

- The binary point of each sign-magnitude binary fixed-point parameter is determined by its range and the given number of binary bits B for representation.

For $x_1 \in [-1, 1]$, it can be represented by one integer bit to be used as a sign bit and the remaining ($B_f = B - 1$) fractional bits can be used to represent fractional values.

For $x_2 \in [-2, 2]$, it can be represented by two integer bits to be used as a sign bit and one integer bit, and the remaining ($B_f = B - 2$) fractional bits can be used to represent fractional values.

For $x_3 \in [-4, 4]$, it can be represented by three integer bit to be used as a sign bit and two integer bits, and the remaining ($B_f = B - 3$) fractional bits can be used to represent fractional values.

For $x_4 \in [-8, 8]$, it can be represented by four integer bits to be used as a sign bit and three integer bits, and the remaining ($B_f = B - 4$) fractional bits can be used to represent fractional values.

Therefore, the concatenated multi-parameter binary fixed-point coding for this problem can be designed as shown in Table E17.2.3.

Table E17.3.1 Concatenated multi-parameter sign-magnitude binary fixed-point coding.

x_1	x_2	x_3	x_4
<i>a.aaaaaaa</i>	<i>bb.bbbbbb</i>	<i>ccc.ccccc</i>	<i>dddd.dddd</i>

where ‘*a.aaaaaaa*’ is a bit-format representing x_1 , ‘*bb.bbbbbb*’ is a bit-format representing x_2 , ‘*ccc.ccccc*’ is a bit-format representing x_3 , and ‘*dddd.dddd*’ is a bit-format representing x_4 respectively.

(b) The individual resolution value r_p for $p = 1$ to 4 of x_1, x_2, x_3, x_4 is related to its individual range $[-s_p, s_p]$ and the total number of binary bits B assigned for representation as

$$r_p = \frac{s_p - (-s_p)}{2^B}$$

For x_1 :

$$r_p = \frac{1 - (-1)}{2^8} = 2^{-7}$$

For x_2 :

$$r_2 = \frac{2 - (-2)}{2^8} = 2^{-6}$$

For x_3 :

$$r_3 = \frac{4 - (-4)}{2^8} = 2^{-5}$$

For x_4 :

$$r_4 = \frac{8 - (-8)}{2^8} = 2^{-4}$$

(c) Given a parameter vector $[x_1, x_2, x_3, x_4]^T = [0.1, -1.8, 3.6, -7.3]^T$, the quantized parameters of the quantized parameter vector are determined as

$$x_{1q} = r_1 \times \left[\frac{x_1}{r_1} \right]_R = 2^{-7} \times \left[\frac{0.1}{2^{-7}} \right]_R = 0.1015625$$

$$x_{2q} = r_2 \times \left[\frac{x_2}{r_2} \right]_R = 2^{-6} \times \left[\frac{-1.8}{2^{-6}} \right]_R = -1.796875$$

$$x_{3q} = r_3 \times \left[\frac{x_3}{r_3} \right]_R = 2^{-5} \times \left[\frac{3.6}{2^{-5}} \right]_R = 3.59375$$

$$x_{4q} = r_4 \times \left[\frac{x_4}{r_4} \right]_R = 2^{-4} \times \left[\frac{-7.3}{2^{-4}} \right]_R = -7.3125$$

In the above four equations, the symbol $[.]_R$ denotes rounding to the nearest integer operation.

The quantized parameter vector of $[0.1, -1.8, 3.6, -7.3]^T$ expressed in discrete real value format is given by

$$[0.1015625 \quad -1.796875 \quad 3.593750 \quad -7.312500]^T$$

(d) The quantized parameter vector expressed in sign-magnitude binary fixed-point format is given by

$$[0.0001101 \quad 11.110011 \quad 011.10011 \quad 1111.0101]^T$$

Note that a first bit ‘1’ on the left represents a negative signed number and a first bit ‘0’ on the left represents a positive signed number.

17.3 Parameter Initialization Methods

For optimization methods such as gradient-based algorithms, or population-based evolutionary algorithms or non-population-based backpropagation algorithms, a method to generate an initial population of solution vectors (or an initial solution vector) is required.

For discrete optimization, initial parameter values obtained in the continuous space can first be quantized to map onto the required discrete space as initial discrete parameter values.

Below we shall described five methods for parameter initialization.

17.3.1 Initialization based on 1-side exponential function

For a solution vector consisting of D parameters with a range of parameter values $0 \leq x_d \leq x_{max}$, a 1-side exponential distribution function as defined by (17.3.1) and plotted in Figure 17.3.1 can be used to initialize parameter values x_d for $d = 1$ to D as

$$x_d = e^{-6*rand} \quad (17.3.1)$$

Where *rand* denotes a uniformly distributed pseudorandom number of value within $[0, 1]$. As shown in Figure 17.3.1, this exponential function contains many small and some large values within $[0, 1]$ which favour some optimization problems.

Note that the scaler value of 6 can be varied to adjust the slope and hence the ratio of small and large values respectively below 0.5 and above 0.5.

This exponential function could be used to replace the uniformly distributed pseudorandom number function *rand* of value within $[0, 1]$ for initializing parameter values.

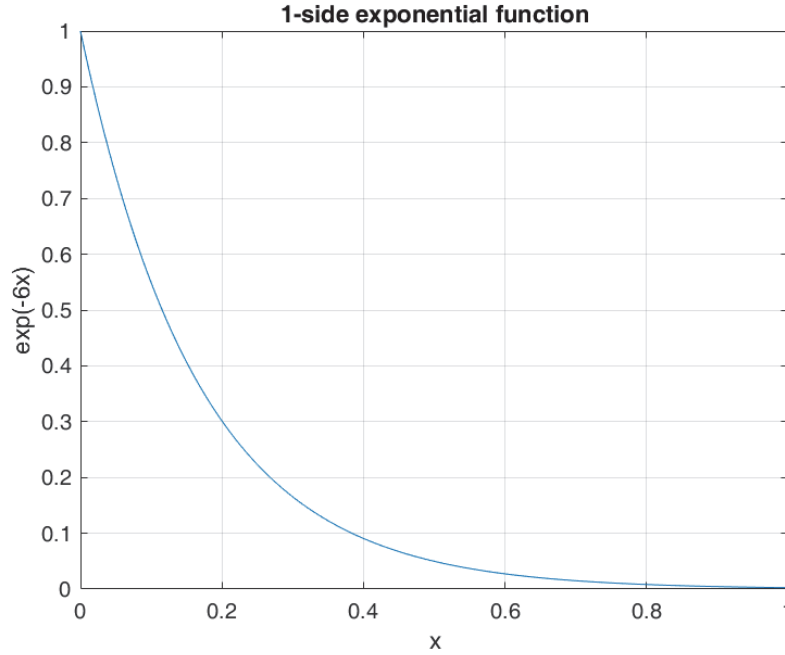


Figure 17.3.1 Plot of e^{-6x} for $x \in [0, 1]$ for parameter initialization.

17.3.2 Initialization based on 2-side exponential function

For a solution vector consisting of D parameters with a range of parameter values $-x_{max} \leq x_d \leq x_{max}$, a 2-side exponential distribution function $f(r)$ with zero mean value as defined by (17.3.2) and plotted in Figure 17.3.2 can be used to initialize parameter values x_d for $d = 1$ to D .

$$x_d = x_{max} \times \text{sign}(r) \times f(r) \quad \text{for } -z_{max} \leq r \leq z_{max}, d = 1 \text{ to } D \quad (17.3.2)$$

The value of r is a uniformly distributed random number generated within $\pm z_{max}$.

Both initialization functions contain many small absolute values and some arbitrarily large absolute values for r within $\pm z_{max}$ thus allowing an algorithm to search all possible values.

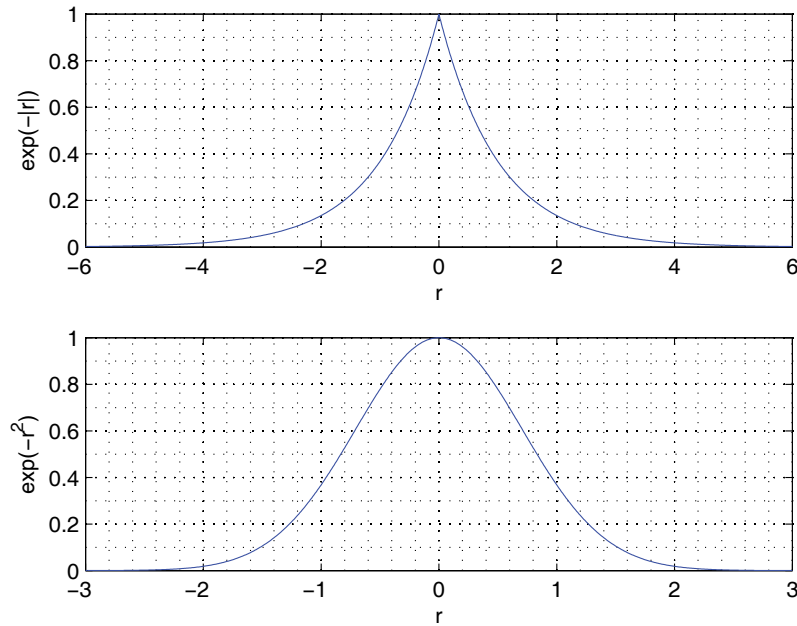


Figure 17.3.2 Two 2-side exponential functions for parameter initialization.

The absolute value exponential function $f(r) = e^{-|r|}$ generates less larger values and more smaller values as compared to those of the squared value exponential function $f(r) = e^{-r^2}$.

In Table 17.3.1, z_{max} is chosen to be 4 and 2 respectively for the two initialization functions to avoid zero initial parameter values.

Table 17.3.1 Specifications of initialization functions.

	$f(r)$	z_{max}
1	$e^{- ar }$	4
2	e^{-ar^2}	2

To allow some zero initial parameter values, z_{max} can be set to 6 and 3 respectively. The choices between them depend on the optimization problem under consideration.

17.3.3 Initialization based on random deviation from nominal value

Let c_k denote the value of the k th parameter obtained by some initial design method, for a population of P , other initial values of the k th parameter c_{pk} for $p = 1$ to P can be generated by adding a uniformly distributed random value within $[-0.5c_k, 0.5c_k]$ to c_k as

$$c_{pk} = c_k * (1 + 0.5 * (2 * rand - 1)) \quad \text{for } 1 \leq k \leq K \quad (17.3.3)$$

where $rand$ denotes a uniformly distributed random number within $[0, 1]$.

17.3.4 Initialization based on random deviation between upper bound and lower bound values of individual parameter

Let c_{uk} and c_{lk} denote the upper bound and the lower bound of a parameter k such that

$$c_{lk} \leq c_k \leq c_{uk} \quad \text{for } 1 \leq k \leq K \quad (17.3.4)$$

The initial parameter value of c_{pk} for a population member p is computed by

$$c_{pk} = c_{lk} + rand * (c_{uk} - c_{lk}) \quad \text{for } p = 1 \text{ to } P, k = 1 \text{ to } K \quad (17.3.5)$$

where $rand$ is a uniformly distributed random value between $[0, 1]$.

17.3.5 Initialization based on random deviation between upper bound and lower bound values of all parameters

Let c_u and c_l denote the upper bound and the lower bound of all parameters such that

$$c_l \leq c_k \leq c_u \quad \text{for } 1 \leq k \leq K \quad (17.3.6)$$

The initial parameter value of c_{pk} for a population member p is computed by

$$c_{pk} = c_{lk} + rand * (c_u - c_l) \quad \text{for } p = 1 \text{ to } P, k = 1 \text{ to } K \quad (17.3.7)$$

where $rand$ is a uniformly distributed random value between $[0, 1]$.

17.4 Constraint Handling Methods

A minimization problem requiring parameter constraints can be mapped into an unconstrained minimization problem by constraint handling methods which are described in this section.

Reference [Kwa0318] provides a more detailed explanation of different constraint handling methods. References [Mez1112], [Dat1500] give an overview of constraint handling methods for evolutionary optimization methods.

In the section below, a simple constraint handling method called penalty method is explained.

17.4.1 Penalty function method

To incorporate the required parameter constraints into an optimization problem, a penalty function method can be used to reduce the fitness ranking in relation to the degree of constraint violation.

In a penalty function method, a constrained optimization is reformulated as an unconstrained one by adding a penalty term in its objective function evaluation for each constraint violation.

Let $\mathbf{x}$ represent a parameter vector to be optimized. An optimization to minimize the objective function (or error function) $e(\mathbf{x})$ subject to inequality constraints

$$c_i(\mathbf{x}) \geq 0 \quad \text{for } i = 1, 2, \dots, C \quad (17.4.1)$$

can be mapped into an unconstrained optimization problem as

$$\min_{\mathbf{x}} e'(\mathbf{x}) \quad (17.4.2)$$

where

$$e'(\mathbf{x}) = e(\mathbf{x}) + \sum_{i=1}^C r_i P_f[c_i(\mathbf{x})] \quad (17.4.3)$$

The second term on the right side of (17.4.3) is a penalty term contributed by each violation of the constraints $c_i(\mathbf{x})$ for $i = 1$ to C , and r_i is the i th penalty factor [Kwa0318].

For the case of genetic algorithms, with the unconstrained objective function $e'(\mathbf{x})$ as reformulated in (17.4.3), a fitness function $f(\mathbf{x})$ can be mapped in the same manner as shown in (13.5.2) as

$$f(x) = \frac{1}{e'(\mathbf{x}) + \varepsilon} \quad (17.4.4)$$

where ε denotes a small positive constant.

17.5 Crowding Distance

The crowding distance value of a chromosome provides an estimate of the density of chromosomes surrounding that chromosome.

The crowding distance value of a particular chromosome is the average distance of its two neighboring chromosomes.

Crowding distance can be used to replace similar chromosomes by dissimilar chromosomes to diversify chromosomes within a population for single-objective optimization problems and to obtain improved Pareto optimality solution sets for multi-objective optimization problems [Zha1411].

17.6 Multi-objective Evolutionary Optimization Algorithms

Multi-objective optimization (also known as multi-criteria optimization or Pareto optimization) is a method for solving problems that requiring optimization of parameters to simultaneous satisfy more than one objective function.

Evolutionary-based algorithms have been used to solve multi-objective optimization problems.

Part V

Deep Neural Computing

V.1 Overview

With the number of hidden layers higher than one, a neural network is capable to extract deep interrelationship between input and output patterns. The learning process is called deep learning [Hin0606], [Ben0612].

Deep learning (also known as deep structured learning or hierarchical learning) is based on learning data representations. Learning can be supervised or partially supervised or unsupervised.

Deep learning architectures such as deep neural networks, deep belief networks, and deep recurrent neural networks have been applied to fields including computer vision, speech recognition, and bioinformatics with results comparable to and in some cases superior [Cir1206] to the human experts [Kri1212].

Part V on deep neural computing consists of Chapters 18-20 which cover respectively multiplierless neural networks, convolutional neural networks, and deep residual learning and networks.

Chapter 18 describes and explains the available methodologies on how multiplayer neural networks can be designed and realized. Chapter 19 describes and explains the structures, properties, operations, and applications of deep convolutional neural networks.

Chapter 20 points out and explains the phenomenon on why deep convolutional neural networks can turn from a stagnant stage to a rapidly deteriorating stage as the number of layers increases. It also describes how and explains why deep residual networks can be designed to allow additional learning as the number of layers increases.

All these chapters cover three core topics to introduce and describe the challenges encountered and the available solutions in designing and implementing deep neural networks for practical applications.

The field of deep learning and neural networks has undergone rapidly developments in most recent years and a number of new and improved neural networks including their structures, learning algorithms, and applications have been advanced.

As listed in Table 1.1, emerging and improved deep neural networks under the class of feedforward neural networks include convolutional neural networks (CNNs), and deep residual networks (ResNets); under the class of recurrent neural networks include long short-term memory networks (LSTM), restricted Boltzmann machine (RBM), deep belief networks (DBNs), and gated recurrent units (GRUs); and under the class of associative memories include autoencoders, and generative adversarial networks (GANs). These are exciting neural networks with applications being developed rapidly on daily basis.

Chapter 18

Multiplierless Neural Networks

18.1 Introduction

Digital multilayer neural networks are complex networks consisting of multipliers, adders, and activation functions among which the two most expensive components to implement are multipliers and activation functions.

The complexity of a multiplier can be drastically reduced by shortening its wordlength. The best approach is to represent a B-bit multiplier by a sum-of-powers-of-two (SOPT) multiplier such that the complexity can be controlled by the number of powers-of-two terms adopted.

The activation function of a biological neuron is a tanh-like (or sigmoid-like) function in the form of an infinite power series which requires many multiplications.

Based on [Kwa0206b] and related papers, a multiplierless approach to design and implement neural networks using SPOT weights and the simple sigmoid-like activation function (SSAF) is described in this chapter. Two design examples are given.

18.2 Simple Sigmoid Activation Function

18.2.1 Second-order activation function

In this section it is shown that a simple second-order nonlinear function $H_{s1}(z)$ exists which can be used as a sigmoid function [Kwa9207].

Define

$$H_{s1}(z) = \begin{cases} z(\beta - \theta z) & \text{for } 0 \leq z \leq L \\ z(\beta + \theta z) & \text{for } -L \leq z < 0 \end{cases} \quad (18.2.1a)$$

$$(18.2.1b)$$

A direct digital hardware implementation of $H_{s1}(z)$ can be carried out according to the signal-flow graph as shown in Figure 18.2.1.

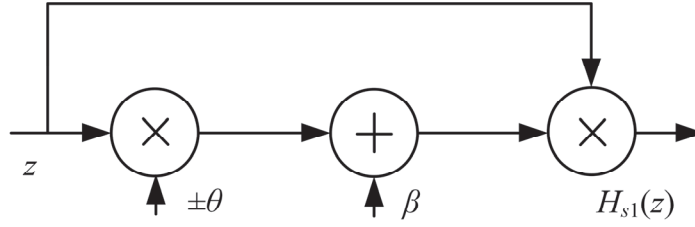


Figure 18.2.1 Signal-flow graph of $H_{s1}(z)$ for digital implementation.

A simple second-order piecewise nonlinear function which has a tanh-like transition between the lower and upper saturation regions can be constructed as

$$G_{s1}(z) = \begin{cases} 1 & \text{for } L \leq z \\ H_{s1}(z) & \text{for } -L < z < L \\ -1 & \text{for } z \leq -L \end{cases} \quad (18.2.2)$$

The piecewise activation function $G_{s1}(z)$ built from $H_{s1}(z)$ as defined in (18.2.2) can be implemented directly using digital techniques.

From (18.2.2), the binary version of the sigmoid function can be realized by

$$G_{s1}(z) = \frac{1}{2}(G_{s1}(z) + 1) \quad (18.2.3)$$

Differentiating $H_{s1}(z)$ of (18.2.1a) with respect to z and setting it equal to 0 at $z = L$, we obtain

$$\beta = 2\theta L \quad (18.2.4)$$

Setting $H_{s1}(z)$ of (18.2.1a) equal to 1 at $z = L$, together with (18.2.4), we obtain

$$\theta = 1/L^2 \quad (18.2.5)$$

$$\beta = 2/L \quad (18.2.6)$$

For unity slope at $z = 0$, we differentiate $H_{s1}(z)$ of (18.2.1a) with respect to z and set them equal to 1 at $z = 0$; and from (18.2.6), we obtain

$$L = 2 \quad (18.2.7)$$

Alternatively, identical expressions obtained above for θ , β , and L can also be derived using $H_{s1}(z)$ of (18.2.1b). The step-by-step derivations for $H_{s1}(z)$ using (18.2.1a) and (18.2.1b) are given in Problems 18.1 and 18.2.

By expressing L in terms of a single powers-of-two format (i.e. 2^{-m}), $H_{s1}(z)$ can be implemented by one binary multiplication together with one binary shift (due to $\pm\theta = 2^{-2m}$) and one binary addition (due to $\beta = 2^{-(m-1)}$).

As the total time taken by one binary shift and one binary addition is small compared to one binary multiplication, the total time required to compute $H_{s1}(z)$ is approximately of the order of one binary multiplication.

18.2.2 Comparison

For comparison, we define the bipolar sigmoid function

$$G_s(z) = \frac{1 - e^{-\tau z}}{1 + e^{-\tau z}} \quad (18.2.8)$$

where the parameter τ determines the slope in the transition region ($-L \leq z \leq L$), and the extent of z (i.e. $z = \pm L$) at which the function starts to saturate to a value of +1 at $z = L$ and starts to saturate to a value of -1 at $z = -L$.

For unity slope at $z = 0$, $\tau = 2$. A plot of $G_s(z)$ of the bipolar sigmoid function (for $\tau = 2$) and the second-order function $G_{s1}(z)$ (for $L = 2$) is shown in Figure 18.2.2.

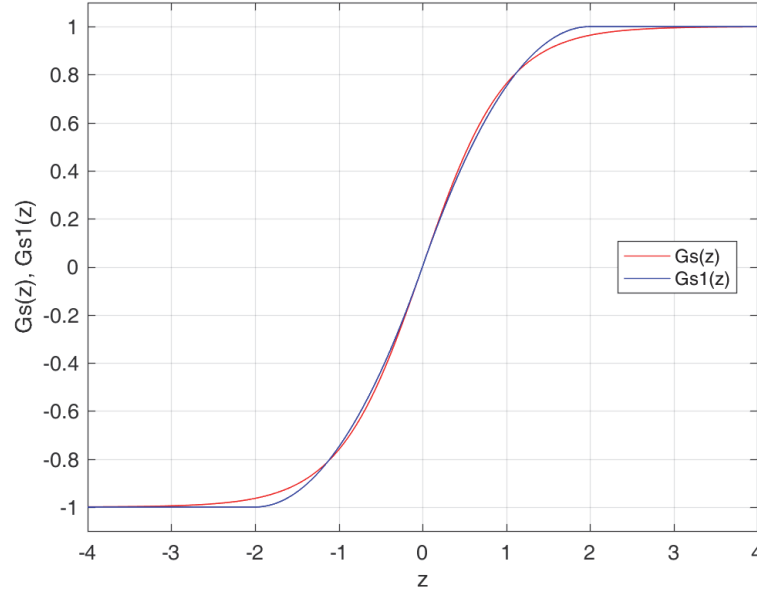


Figure 18.2.2 Bipolar sigmoid function $G_s(z)$ ($\tau = 2$) and second-order piecewise function $G_{s1}(z)$ ($\tau = 2$).

Design Example 18.1 (Pattern recognition using feedforward neural networks with simple activation function) Matlab

The two activation functions (including the simple activation function) shown in Figure 18.2.2 were used to train a two-layer and a three-layer feedforward neural network (NNet) [Kwa9207] using the backpropagation algorithm [Tan9301].

10 training pattern-pairs consisting of 10 Arabic numerals from 0 to 9 as inputs (each input is expressed in 10×10 bipolar elements of ± 1) and 10 bipolar vectors as outputs (each output vector consisted of four elements of ± 1) were used as shown in Figure E18.1.1.

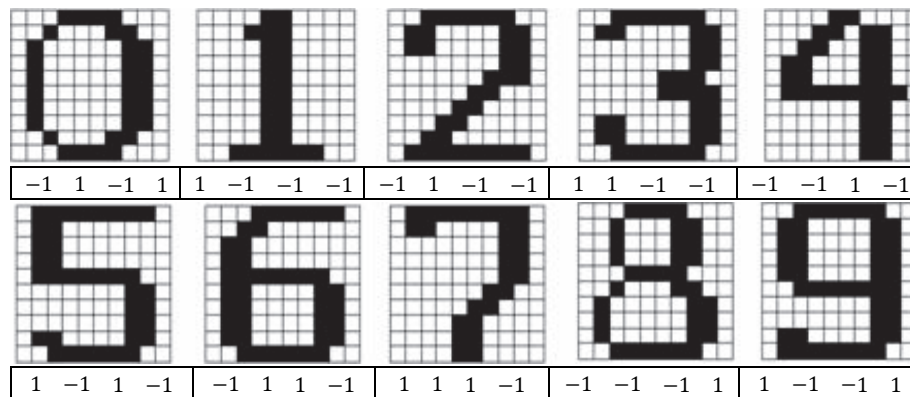


Figure E18.1.1 Ten-digit input and output pattern pairs.

Solution

The number of hidden neurons in each hidden layer of the two networks was 40. The learning rate parameter used was 0.01 in all simulations.

For each combination of function and network, three sets of initial random weights distributed within the range -0.01 to 0.1 were used.

For recall testing purposes, 100 corrupted versions were constructed for each input pattern by inverting 5% of its pixels randomly.

The results on the number of training epochs to reach a sum of squared output errors of 0.1 over 10 pattern-pairs and the generalization (expressed as the percentage of the number of correct recalls over the number of recalls of corrupted input patterns) are summarized in Table E18.1.1.

The results obtained for the two functions are quite similar in the generalization aspect but the second-order function is slightly faster in convergence speed for both networks.

A sigmoid-like second-order piecewise activation function has been described. The function has been successfully applied to training multilayer feedforward neural networks using the backpropagation algorithm [Tan9301].

Table E18.1.1 Performance of two-layer and three-layer neural networks using SSAF.

	$G_s(z)$		$G_{s1}(z)$	
	Epochs #	Generalization %	Epochs #	Generalization %
2-layer NNet	33	99.6	29	99.6
	32	99.1	27	99.1
	32	99.0	28	99.0
3-layer NNet	39	99.5	35	99.4
	41	99.9	38	99.9
	38	99.9	35	99.9

The digital hardware complexity of $H_{s1}(z)$ is of the order of one binary multiplication. This is an attractive alternative for the direct digital implementation of neuron-like processing elements especially in applications where z can be represented adequately in a short binary word length format.

18.3 Discrete Multilayer Feedforward Neural Network Design

The two most expensive items in the digital hardware implementation of a multilayer feedforward neural network (MFNN) are the sigmoid activation functions and weights (or multipliers).

The actual computational time of these two items could limit the scope of high speed applications.

In [2], the simple sigmoid activation function, which has nearly identical features to the conventional sigmoid activation function but can be implemented using one multiplier, is described.

In this section, a method for designing MFNNs using the simple sigmoid activation function and one-powers-of-two weights is described.

18.3.1 Weight and bias adaptations

In a multilayer feedforward neural network as shown in Figures 4.2.1 and 4.2.2, the output of a neuron j at a layer h due to a k th input pattern $\mathbf{X}_k$ can be expressed as

$$y_{jk}^{[h]} = F \left[\sum_{i=1}^{N_{h-1}} w_{ij}^{[h]} y_{ik}^{[h-1]} + b_j^{[h]} \right] \quad (18.3.1)$$

for $j = 1$ to N_h , $h = 1$ to L , and $k = 1$ to K .

In (18.3.1), $y_{ik}^{[h-1]}$ is the output of a neuron i at the layer $h - 1$; $w_{ij}^{[h]}$ is the weight between a neuron i at layer $h - 1$ and a neuron j at layer h ; $b_j^{[h]}$ is the bias of a neuron j at layer h ; N_h is the number of neurons at layer h ; $F[z]$ represents the operation of the SSAF. $y_{ik}^{[0]} = x_{ik}$, where x_{ik} is the i th element of $\mathbf{X}_k$.

The sum of the squared output errors (SSE) due to the input pattern k is defined as

$$e_k = \sum_{j=1}^{N_L} (t_{jk} - y_{jk}^{[L]})^2 \quad (18.3.2)$$

where t_{jk} represents the j th element of the target pattern k and $h = L$ refers to the output layer.

The change in weight is defined as

$$\Delta_k w_{ij}^{[h]} = -\varepsilon \partial e_k / \partial w_{ij}^{[h]} \quad (18.3.3)$$

where ε is a learning rate parameter for weights. After some derivations, we obtain

$$\Delta_k w_{ij}^{[h]} = \varepsilon \delta_{jk}^{[h]} y_{ik}^{[h-1]} \quad (18.3.4)$$

Where

$$\delta_{jk}^{[h]} = F' \left[z_{jk}^{[h]} \right] \sum_{i=1}^{N_{h+1}} \delta_{ik}^{[h+1]} w_{ji}^{[h+1]} \quad \text{for } h < L \quad (18.3.5)$$

and

$$\delta_{jk}^{[L]} = 2(t_{jk} - y_{jk}^{[L]})F'(z_{jk}^{[L]}) \quad (18.3.6)$$

$F'(z_{jk}^{[h]})$ represents the partial derivative of $F(z_{jk}^{[h]})$ with respect to $z_{jk}^{[h]}$. Weights are updated at the end of each epoch.

We define the change in bias as

$$\Delta_k b_j^{[h]} = -\varepsilon_b \partial e_k / \partial b_j^{[h]} \quad (18.3.7)$$

where ε_b is a learning rate parameter for biases. After some derivations, we obtain

$$\Delta_k b_j^{[h]} = \varepsilon_b \delta_{jk}^{[h]} \quad (18.3.8)$$

Biases are also updated at the end of each epoch.

18.3.2 SSAF and OPOT weights

The SSAF [Kwa9207] saturates to ± 1 at and beyond the points $z = H$ and $z = -H$ (note that L is used in [Kwa9207] to denote H).

To avoid the learning from staying at either saturation region for a long time, a small positive value σ is assigned to $G'_{s1}(z)$ at and beyond $\pm H$. Hence

$$G'_{s1}(z) = \begin{cases} \sigma & \text{for } H \leq z \\ H'_{s1}(z) & \text{for } -H < z < H \\ \sigma & \text{for } z \leq -H \end{cases} \quad (18.3.9)$$

where $G'_{s1}(z)$ and $H'_{s1}(z)$ represent, respectively, the partial derivatives of $G_{s1}(z)$ and $H_{s1}(z)$ with respect to z .

One-powers-of-two weights of the forms $\{\pm 1, \pm 2^{-1}, \pm 2^{-2}, \dots, \pm 2^{-M}, 0\}$ are used to represent weights (or multipliers) such that binary number shifting can be used to replace multiplication.

18.3.3 Design algorithm

Step 1: Starting with small random weights and zero biases, train an MFNN using the SSAF with $\beta^{[h]} = 2/H$ and $\Theta^{[h]} = 1/H^2$ for an OPOT H using (18.3.4)-(18.3.6) without adjusting biases until

$$\sum_{k=1}^K e_k < E_0 \quad (18.3.10)$$

Step 2: Find the maximum absolute value in each layer of weights $w_{ij}^{[h]}$ obtained in step 1, denoted by $w_{max}^{[h]}$, and then set $w_{max}^{[h]} = 2^{p[h]}$, where $2^{p[h]}$ is the smallest OPOT value greater than or equal to $w_{max}^{[h]}$ is the smallest OPOT value greater than or equal to $w_{max}^{[h]}$.

Step 3: Normalize weights as

$$w'_{ij}{}^{[h]} = 2^{-p[h]} w_{ij}^{[h]} \quad (18.3.11)$$

Step 4: Set $\beta^{[h]} = 2^{p[h]} \beta^{[h]}$ and $\Theta^{[h]} = 2^{2p[h]} \Theta^{[h]}$ such that they remain powers-of-two.

Step 5: Quantize $w'_{ij}{}^{[h]}$ to its nearest OPOT value $\tilde{w}_{ij}^{[h]}$ as

$$\tilde{w}_{ij}^{[h]} = \begin{cases} 0 & \text{if } |w'_{ij}{}^{[h]}| < C_M \\ \text{sgn}(w'_{ij}{}^{[h]}) 2^{-m} & \text{if } C_m < |w'_{ij}{}^{[h]}| < C_{m-1} \end{cases} \quad (18.3.12a)$$

$$C_m = \left(\frac{3}{4}\right) 2^{-m} \text{ for } m = 0, 1, \dots, M \quad (18.3.12b)$$

In (18.3.12a), the sign of $w'_{ij}{}^{[h]}$ is denoted as $\text{sgn}(w'_{ij}{}^{[h]})$.

Step 6: Calculate the SSE using (18.3.2). If the SSE is less than a specified level E , stop; otherwise, proceed to step 7.

Step 7: Compute changes in $\tilde{w}_{ij}^{[h]}$ and $b_j^{[h]}$, respectively, according to (18.3.4)-(18.3.6), (18.3.5)-(18.3.6), and (18.3.8).

Step 8: Update $\tilde{w}_{ij}^{[h]}$ and quantize them as in step 5. If this update results in a reduced SSE, accept new weights; otherwise, discard them and keep previous values.

Step 9: Update $b_j^{[h]}$.

Step 10: Go to step 6.

Design Example 18.2 (Ten-digit problem learning using discrete multilayer feedforward neural network) Matlab

In this example, a ten-digit problem learning is selected to illustrate the design procedure of a discrete multilayer feedforward neural network using simple activation functions and one-powers-of-two weights [Kwa9212], [Kwa9209a], [Kwa9308b], [Kwa9405].

As shown in Figure E4.2.1, the input patterns used in training were 10 numerals, each with 10×10 bipolar (± 0.9) pixels; corresponding output patterns were 4 bit bipolar elements (represented by ± 0.9 in simulations) shown below each input pattern.

Each MFNN was trained to obtain both continuous and OPOT weights. Sets of noisy patterns were fed to each continuous-weight MFNN and its corresponding OPOT-weight MFNN to test their generalization capabilities in terms of recall accuracy (in percent of correct recalls).

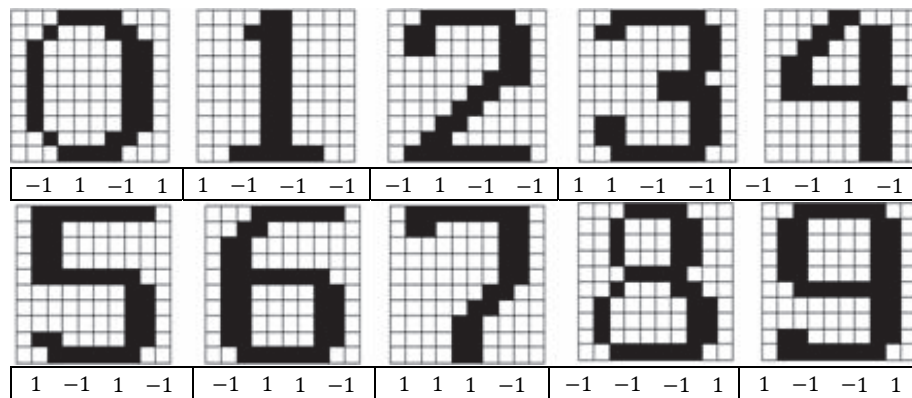


Figure E18.2.1 Ten-digit input and output pattern pairs for Design Example 18.2.

A noisy version of each of the 10 input patterns was constructed by inverting randomly 5% of its elements.

The recall accuracy was obtained by taking the average of the results among 100 noisy versions of each of the 10 input patterns.

Simulation results are summarized in Tables E4.2.1 and E4.2.2 (where CMFNN and OPOTMFNN represent, respectively, the MFNN with continuous and OPOT weights; Generalization denote generalization capability).

For MFNNs with two hidden layers, we used an identical number of neurons in both hidden layers.

All data given in these tables were averages of five designs, starting with different initial random weights uniformly distributed in $[-0.1, +0.1]$.

Table E18.2.1 Convergence speed and generalization capability of networks with one hidden layer.

Number of hidden neurons	Convergence speed		Generalization	
	CMFNN	OPOTMFNN	CMFNN	OPOTMFNN
	epoch	epoch	%	%
10	96.6	72.2	99.44	99.04
20	350.2	1.0	99.72	99.50
40	300.0	2.2	99.50	99.40
60	294.6	1.0	99.58	99.48
80	61.6	1.0	99.56	99.50
100	45.0	1.0	99.54	99.32

Table E18.2.2 Convergence speed and generalization capability of networks with two hidden layers.

Number of hidden neurons	Convergence speed		Generalization	
	CMFNN	OPOTMFNN	CMFNN	OPOTMFNN
	epoch	epoch	%	%
10	385.2	71.0	99.42	99.00
20	846.0	381.8	99.46	99.34
40	712.2	3.4	99.40	99.38
60	1021.2	1.0	99.52	99.48
80	483.4	1.0	99.56	99.50
100	307.6	1.0	99.60	99.46

The parameter values used were $\varepsilon = 0.01$; $\varepsilon_b = 0.10$; $M = 4$; $E_0 = 0.01$; $E = 0.20$; $H = 2$; and $\sigma = 10^{-2}$.

A design method for MFNNs with the SSAF and OPOT weights has been presented. As demonstrated in Example 4.2, convergence was reached in all designs.

The designed MFNNs with OPOT weights can retain the generalization capability of the corresponding MFNN with continuous weights as the degradations in performance were at most 0.42% in all designs.

The advantages of increased computational speed in applications and reduced hardware cost (due to reduced integrated chip area and power) make the proposed design method attractive for digital implementation.

In addition, multilayer feedforward neural network design with single POT weights is also described in [Tan9308], [Tan9211b].

In practice, sum-of-powers-of-two weights with uniform quantization steps can also be used but at an increased hardware cost depending on the number of OPOT terms to be used.

18.4 SOPT Quantized Activation Functions

An alternative approach to eliminate multiplications in a MFNN is to represent each activation function by a quantized activation function with one-powers-of-two output.

We shall call this OPOT quantized activation function and its associated neuron as a quantized neuron (QN) [Kwa0206a], [Tan9211b], [Tan9706]. In so doing, a multiplier, once again, is now reduced to a simple shift register.

In [Kwa9307], [Kwa9305], a method for designing MMFNNs using quantized neurons (QNs) at the hidden layer, OPOT weights at the input layer, and the SSAF and continuous weights at the output layer for continuous input to output mapping is described.

In addition, two-layer recurrent neural network design with (1, -1, 0) weights is also described in [Kwa0105] and in Chapter 4.

In practice, sum-of-powers-of-two QN with uniform quantization steps can also be used but at an increased hardware cost depending on the number of OPOT terms to be used.

18.5 Systolic Realization of Neural Networks

In addition to multiplierless neural networks, another way to speed up the processing of neural networks can be achieved through systolic realization.

In [Kwa8905], systolic architectures for Hopfield network, BAM and multilayer feedforward network are proposed for recurrent and feedforward neural networks.

In [Kwa9001], systolic realization of multilayer feedforward neural network with backpropagation learning scheme is proposed by speeding up both the architecture and the learning algorithm.

In [Kwa9004], systolic realization of the counterpropagation networks is described.

Chapter 19

Convolutional Neural Networks

19.1 Introduction

Convolutional neural networks (CNNs) are common used for deep learning. CNNs are typically used to accept inputs like images, but can also be used to accept text, patterns, signals, and other continuous inputs.

A study of the visual pathway from retina to striate cortex of cats [Web-CNN] indicated that the simple and complex cells of the visual cortex are activated by the sub-regions of a visual field.

Based on this founding, the neurons in the convolutional layer of a CNN are connected only to the sub-regions of its previous layers.

Since these sub-regions may overlap, such neurons produce spatially-correlated intermediate outputs.

In comparison, the neurons of other types of neural networks are fully connected to a previous layer.

In this Chapter, the basics of convolutional neural networks [Web-CNN], [Goo1600] are discussed.

The chapter consists of a brief introduction about the Convolutional neural networks (CNN), definitions of convolutional operations, a description on the structure and properties of convolutional neural networks, a description about various layers of CNN [Mat-Lay], [Lof1503]. Also, a design example on the training of CNNs is given using MATLAB.

In addition, two design examples on the use of CNN are given in [Zho1911] on multimodal voice conversion under adverse environment using a deep convolutional neural network and in [Wan2000] on multistage model for robust face alignment using deep neural networks.

Finally, a list of further reading on convolutional neural networks [Nai1006], [Kri1706], and [Nag1111] is given.

19.2 Definitions of Convolutional Operations

Convolutional networks are neural networks that use convolution instead of matrix multiplication in at least one of their layers. Mathematically, the operation of 1-dimensional convolution can be expressed by

$$\mathbf{y}(i) = \sum_{m=0}^{M-1} \mathbf{x}(m) \mathbf{p}(i - m + 1) \quad (19.2.1)$$

If $\mathbf{x}$ of length M , $\mathbf{p}$ of length Q , then $\mathbf{y}$ is of length $M + Q - 1$.

In convolutional network terminology, the first argument of the 1-dimensional convolution is called the input $\mathbf{x}(\cdot)$, the second argument is called the kernel $\mathbf{p}(\cdot)$, and the output $\mathbf{y}(\cdot)$ is referred to as the feature map.

In convolution network applications, an input $\mathbf{X}$ is usually a 2-dimensional data array of size $M \times N$, and the kernel $\mathbf{P}$ is usually a 2-dimensional parameter array of size $Q \times R$. The operation of 2-dimensional convolution of $\mathbf{X}$ and $\mathbf{P}$ results in an output $\mathbf{Y}$ which can be expressed as

$$\mathbf{Y}(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \mathbf{X}(m, n) \mathbf{P}(i - m, j - n) \quad (19.2.2)$$

for i of size $\max(M + Q - 1, M, Q)$, j of size $\max(N + R - 1, N, R)$.

The 2-dimensional cross-correlation of an input matrix $\mathbf{X}$ of size $M \times N$ and a kernel matrix $\mathbf{P}$ of size $Q \times R$ results in an output matrix $\mathbf{Y}$ of size $(M + Q - 1) \times (N + R - 1)$ as

$$\mathbf{Y}(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \mathbf{X}(m, n) \mathbf{P}(m - i, n - j) \quad (19.2.3)$$

for $-(P - 1) \leq i \leq (M - 1)$ and $-(Q - 1) \leq j \leq N - 1$.

19.3 Structure of Convolutional Neural Networks

For a 2-dimensional input like an image, the neurons of a convolutional neural network accepts the 2-dimensional image in its input layer 1 and transforms it into a 2-dimensional output in its output layer L .

A typical CNN consists of a number of hidden layers. Each hidden layer consist of a number of neurons arranged in a 2-dimensional structure. The neurons in each hidden layer h learn the nonlinear combinations of the original inputs at the input layer 1 to perform feature extraction.

These learned features known as activation outputs of a hidden layer h are passed onto as the inputs of the hidden layer $h + 1$ to perform feature extractions. Finally, the learned features of the last hidden layer become the inputs to a classifier or a regression function to determine the outputs at the layer L of the CNN.

The structure of a CNN varies subject to the types and numbers of layers included. The types and number of layers included depends on a particular application or data.

For categorical responses, a classification function and a classification layer is required to determine the output of a network. For continuous responses, a regression layer is required to form the output of a network.

A smaller network with only one or two convolutional layers may be sufficient to learn a small number of gray scale image data. For more complex images with pixels represented by millions of colors, a more complicated network with multiple convolutional and fully connected layers is required.

A convolutional neural network reduces the number of parameters with a reduced number of connections, shared weights, and down sampling. As compared to a neural network with fully-connected neurons, the number of connection weights increases rapidly as the number of neurons increases in its input, hidden, and output layers.

19.4 Properties of Convolutional Neural Networks

Convolutional neural networks reduce computations and memory requirements through sparse weights, weight sharing, and equivariant representations as explained below.

19.4.1 Sparse weights

Sparse weights is accomplished by making the size of kernels smaller than the size of an input.

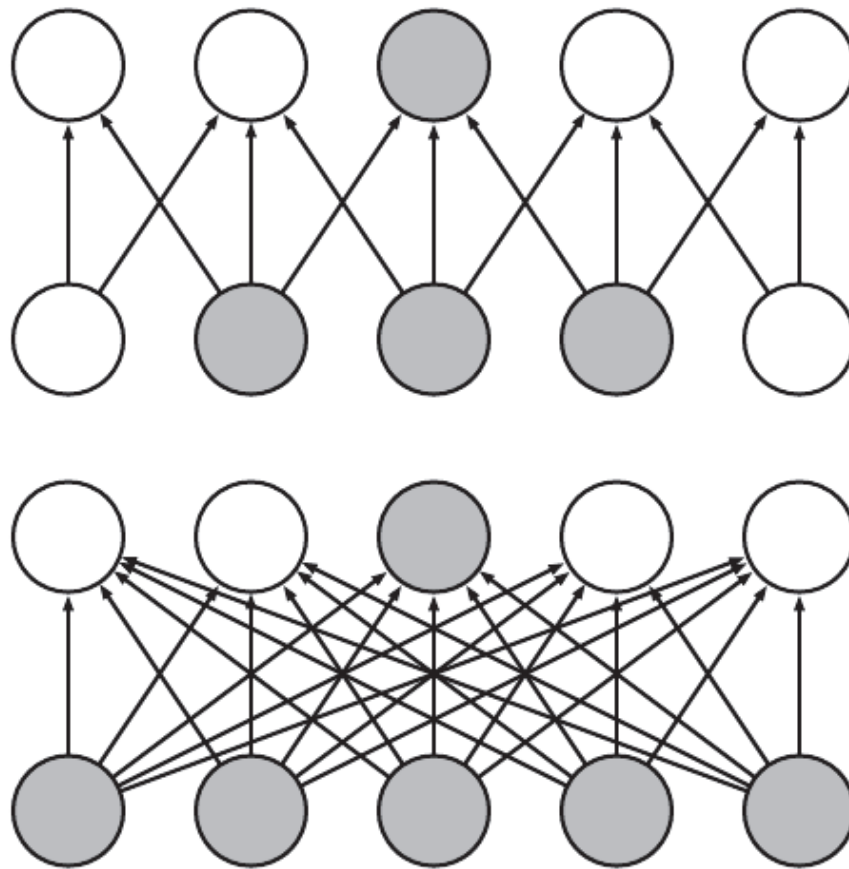


Figure 19.4.1 Sparse weights – Top view [Goo1600]. For a 3-element kernel (top), the receptive field consists of only three inputs which are

connected to each output. For fully connected (bottom), the receptive field consists of all inputs which are connected to each output.

19.4.2 Weight sharing

Weight sharing refers to using the same weight for more than one function in a model.

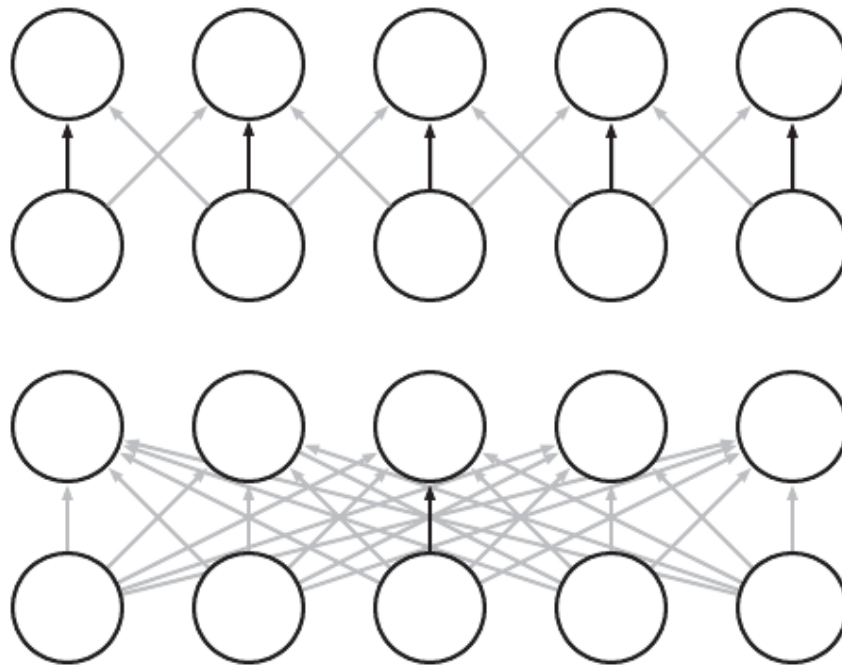


Figure 19.4.2 Weight sharing [Goo1600]. Black arrows indicate the connections that use a particular weight in two different models. The black arrows (top) indicate uses of the central weight of a 3-element kernel in a convolutional model. For weight sharing, each central weight is used for connecting the lower central neuron to its corresponding 3 upper neurons and also for connecting the upper central neuron from its 3 lower neurons. The single black arrow (bottom) indicates the use of each central weight of the weight matrix in a fully connected model. This model does not use weight sharing, so each central weight is used only once.

19.4.3 Equivariant representation

Equivariant representation refers to the property of the lower layers to change the output when the input changes.

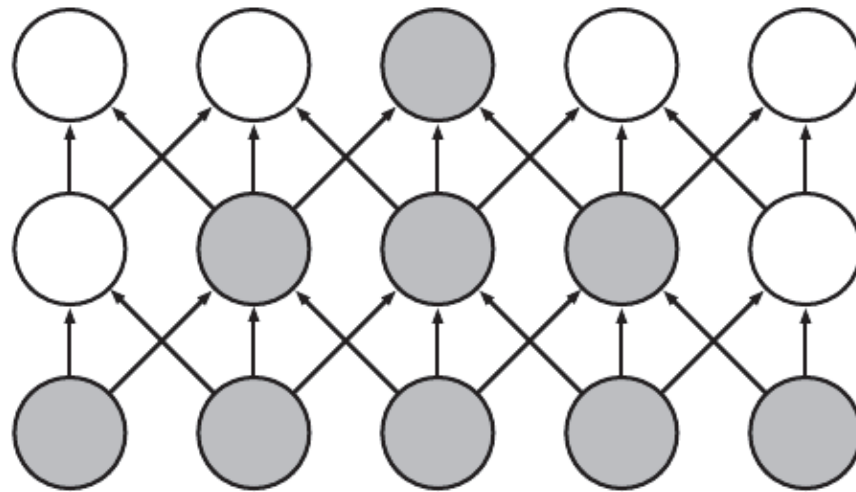


Figure 19.4.3 Equivariant representation [Goo1600]. The receptive field of the neurons in the upper layers of a convolutional network is larger than the receptive field of the neurons in the lower layers.

The effect of equivariant representation increases if a convolutional neural network includes architectural features like stride convolution (Section 19.5.1.1) or pooling (Section 19.5.5). This means that even though weight connections in a convolutional neural network are sparse, neurons in the upper layers can be indirectly connected to all or most of the lower layers as well as its input image.

19.5 Layers of Convolutional Neural Networks

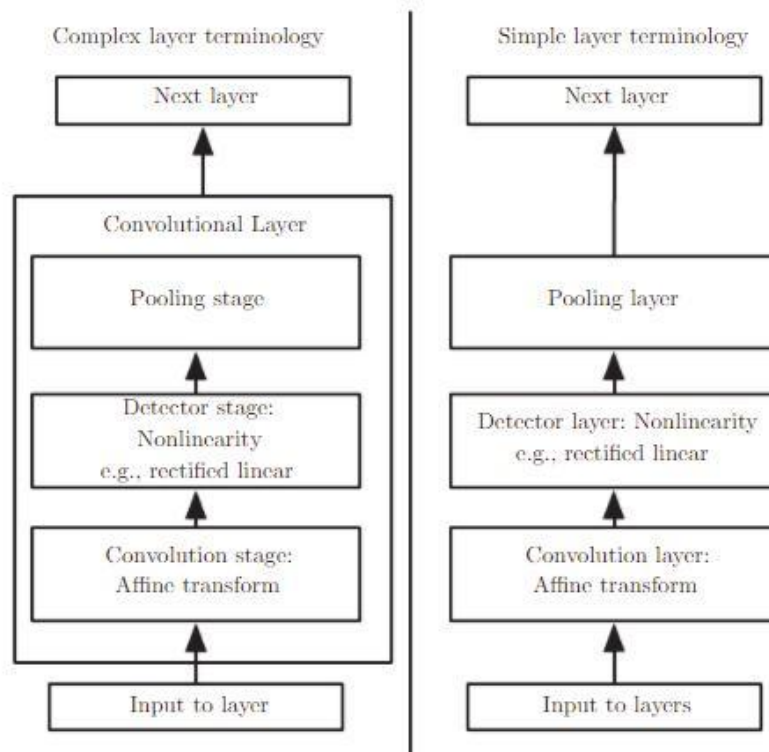


Figure 19.5.1 The components of a typical convolutional neural network layer. Left: In this terminology, the CNN is viewed as a small number of relatively complex layers, with each layer having many stages. Right: In this terminology, the CNN is viewed as a larger number of simple layers; every step of processing is regarded as a layer.

19.5.1 Convolutional layer

The first layer in a CNN is always a convolutional layer which is designed to detect features from an input.

A typical layer of a convolutional network consists of three stages. In the first stage, the layer performs several convolutions in parallel to produce a set of linear activations. The second stage is for detection, each linear activation is run through a nonlinear activation function (like

the rectified linear activation function). In the third stage, a pooling function is used to modify the output.

Different layers and their operations of a typical convolution neural network [Mat-Lay] are described below.

19.5.1.1 Filter and stride

A convolutional layer consists of neurons that connect to the subregions of an input. The layer learns the features localized by these subregions while scanning through an input.

For each subregion, a dot product of the weights and the input is computed and a bias is then added. The set of weights that is applied to a subregion in the input is called a filter.

Assuming a filter of size $F_1 \times F_2$, and nC is the number of channels (3 for color image, 2 for stereo audio) in the input, the number of weights W in a filter is

$$W = F_1 \times F_2 \times nC \quad (19.5.1)$$

The step size with which the filter moves is called a stride. The neighboring subregions that the neurons are connected can overlap.

19.5.1.2 Dilated convolution

A dilated convolution is a convolution in which a filter of size $F_1 \times F_2$ is expanded by inserting spaces between the elements of the filter. The expanded filter of size $F_{1e} \times F_{2e}$ is equal to

$$F_{1e} = (F_1 - 1) \times \text{Dilation Factor} + 1 \quad (19.5.2)$$

$$F_{2e} = (F_2 - 1) \times \text{Dilation Factor} + 1 \quad (19.5.3)$$

Dilated convolutions can be used to increase the area of the receptive field of the layer without increasing the number of weights.

Example 19.1 (Calculate size of dilated filter)

For a $F_1 \times F_2$ filter with the dilation factor $[2, 2]$, calculate the size of the expanded filter.

Solution

With zeros between the elements, the size of the expanded filter is equal to $(2F_1 - 1) \times (2F_2 - 1)$.

19.5.1.3 Zero padding

Additional rows or columns of zeros can be added to the horizontal and vertical borders of an input. By adjusting the padding sizes, you can control the output size of the layer.

19.5.1.4 Feature maps

A feature map is the output formed by convolving a 2-dimensional input of size $M \times N$ with a 2-dimensional filter of size $F_1 \times F_2$. In general, nF feature maps are formed by using nF filters, each with a different set of weights and bias.

In a convolutional layer, if an input of size $M \times N$ consists of nC channels (3 for color image, 2 for stereo audio), the total number of weights nW required by nF filters is

$$nW = (F_1 \times F_2 \times nC + 1) \times nF \quad (19.5.4)$$

where the addition of 1 is due to the bias.

Example 19.2 (Numbers of weights in a convolutional layer)

Assuming an input image of size $M \times N \times nC = 32 \times 32 \times 3$ to be processed by a convolutional layer with $nF = 8$ filters and a filter size of $F_1 \times F_2 = 5 \times 5$, calculate the total number of weights in the layer.

Solution

The number of filter weights (including one bias) per filter is

$$W = F_1 \times F_2 \times nC + 1 = 5 \times 5 \times 3 + 1 = 76$$

The total number of filter weights for all 8 filters is

$$nW = (F_1 \times F_2 \times nC + 1) \times nF = 76 \times 8 = 608$$

19.5.1.5 Output size

For an input of size $M \times N$ and a filter of size $F_1 \times F_2$, the output height M_c or width N_c of a convolutional layer can be calculated as

$$M_c = \frac{M - [(F_1 - 1) \times \text{Dilation Factor} + 1] + 2 \times \text{Padding}}{\text{Stride}} + 1 \quad (19.5.5)$$

$$N_c = \frac{N - [(F_2 - 1) \times \text{Dilation Factor} + 1] + 2 \times \text{Padding}}{\text{Stride}} + 1 \quad (19.5.6)$$

Each of the value of M_c and N_c must be an integer for the whole input to be fully covered. If not, each of M_c and N_c is to be rounded down to an integer.

In a convolutional layer, the product of the output height M_c and width N_c defines the size of the feature map and gives the total number of neurons in a feature map.

In a convolutional layer, the total number of neurons (or output size) is equal to (feature map size $\times$ number of filters).

Example 19.3 (Numbers of neurons in a convolutional layer)

If the dilation factor is $D = 1$ and the stride is $S = 2$ in each direction of the filter, and the padding size is $P = 2$, calculate the size of each feature map and the total number of neurons in the convolutional layer.

Solution

From (19.8) and (19.9), the output height M_c and width N_c of the convolutional layer due to one filter is

$$\begin{aligned} M_c &= \frac{M - [(F_1 - 1) \times \text{Dilation Factor} + 1] + 2 \times \text{Padding}}{\text{Stride}} + 1 \\ &= N_c = \frac{N - [(F_2 - 1) \times \text{Dilation Factor} + 1] + 2 \times \text{Padding}}{\text{Stride}} + 1 \end{aligned}$$

$$= \frac{32 - [(5 - 1) \times 1 + 1] + 2 \times 2}{2} + 1 = 16.5$$

Since $M_c = N_c = 16.5$, after rounding down to integers, some of the outermost zero paddings to the right and bottom of the image are discarded. After such rounding, the size (or the number of neurons) of each feature map is

$$\text{floor}(M_c) \times \text{floor}(N_c) = 16 \times 16 = 256$$

The total number of neurons in the convolutional layer due to 8 filters is

$$\text{floor}(M_c) \times \text{floor}(N_c) \times nF = 16 \times 16 \times 8 = 2048$$

19.5.1.6 Learning parameters and number of convolutional layers

The learning rates and regularization options are required to be specified for a convolutional layer.

A convolutional neural network can consist of one or multiple convolutional layers. The number of convolutional layers depends on the amount and complexity of the data.

19.5.2 Batch normalization layer

To speed up training of convolutional neural networks and reduce the sensitivity to network initialization, use batch normalization layers between convolutional layers and nonlinearities (such as ReLU layers).

A batch normalization [Lof1503] normalizes its inputs x_i by first calculating the mean μ_B and variance σ_B^2 over a mini-batch and over each input channel. Then, it calculates the normalized activations as

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (19.5.7)$$

Here, ϵ (the property Epsilon) improves numerical stability when the mini-batch variance is very small.

To allow for the possibility that inputs with zero mean and unit variance are not optimal for the layer that follows the batch normalization layer

(such as ReLU layer), the batch normalization layer further shifts and scales the activations as

$$y_i = \gamma \hat{x}_i + \beta \quad (19.5.8)$$

Here, the offset β and scale factor γ are learnable parameters that are updated during network training.

When network training is completed, the batch normalization layer calculates the mean and variance over the full training set. When using a trained network to make predictions on new images, the layer uses the mean and variance of the full training set instead of the mean and variance of a mini-batch to normalize the activations.

19.5.3 ReLU layer

A ReLU (rectified linear units) layer [Nai1006] performs a threshold operation to each element of the input, where any value less than zero is set to zero.

Convolutional and batch normalization layers are usually followed by a nonlinear activation function such as a ReLU, specified by a ReLU layer. A ReLU layer performs a threshold operation to each element, where any input value less than zero is set to zero, that is,

$$f(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases} \quad (19.5.9)$$

The ReLU layer does not change the size of its input.

There are other nonlinear activation layers that perform different operations and can improve the network accuracy for some applications.

19.5.4 Cross channel normalization layer

A channel-wise local response (or cross-channel) normalization layer carries out channel-wise normalization.

This layer performs a channel-wise local response (or cross-channel) normalization. It usually follows the ReLU activation layer. This layer replaces each element with a normalized value it obtains using the elements from a certain number of neighboring channels (elements in

the normalization window). That is, for each element x in the input, compute a normalized value x' using

$$x' = \frac{x}{\left(K + \frac{\alpha * ss}{windowChannelSize}\right)^\beta} \quad (19.5.10)$$

where K , α , and β are the hyperparameters in the normalization, and ss is the sum of squares of the elements in the normalization window [Kri1706].

The normalization formula in (19.5.10) is slightly different from that of [Kri1706] but is equivalent if the alpha value is multiplied by the window channel size.

19.5.5 Maximum or average pooling layer

A max pooling layer [Nag1111] performs down-sampling by dividing the input into rectangular pooling regions, and computing the maximum of each region.

An average pooling layer [Nag1111] performs down-sampling by dividing the input into rectangular pooling regions and computing the average values of each region.

Pooling layers follow the convolutional layers for down-sampling, hence, reducing the number of connections to the following layers. They do not perform any learning themselves, but reduce the number of parameters to be learned in the following layers. They also help reduce overfitting.

A max pooling layer returns the maximum values of rectangular regions of its input. The size of the rectangular regions is determined by the `poolSize` argument of `maxPoolingLayer`. For example, if `poolSize` equals `[2, 3]`, then the layer returns the maximum value in regions of height 2 and width 3. An average pooling layer outputs the average values of rectangular regions of its input.

Pooling layers scan through the input horizontally and vertically in specified step sizes. If the pool size is smaller than or equal to the stride, then the pooling regions do not overlap.

For nonoverlapping regions (pool size and stride are equal), if the input to the pooling layer is $n \times n$, and the pooling region size is $h \times h$, then the pooling layer down-samples the regions by h [Nag1111]. That is, the output of a max or average pooling layer for one channel of a convolutional layer is $n/h \times n/h$. For overlapping regions, the output of a pooling layer is $(\text{input size} - \text{pool size} + 2 \times \text{padding}) / \text{stride} + 1$.

19.5.6 Fully connected layer

The convolutional (and down-sampling) layers are followed by one or more fully connected layers.

A fully connected layer multiplies the input $\mathbf{X}$ by a weight matrix $\mathbf{W}$ and then adds a bias matrix $\mathbf{B}$.

As the name suggests, all neurons in a fully connected layer connect to all the neurons in the previous layer. This layer combines all of the features (local information) learned by the previous layers across an input to identify the larger patterns.

For classification problems, the last fully connected layer combines the features to classify the inputs. The output size of the last fully connected layer of the network is equal to the number of classes of the data set. For regression problems, the output size must equal to the number of response variables.

The learning rate and the regularization parameters for this layer are required to be specified when creating the fully connected layer.

If the input to the layer is a sequence (for example, in an LSTM network), then the fully connected layer acts independently on each time step.

For example, if the layer before the fully connected layer outputs an array $\mathbf{X}$ of size $\mathbf{D} \times \mathbf{N} \times \mathbf{S}$, then the fully connected layer outputs an array $\mathbf{Z}$ of size $(\text{output size}) \times \mathbf{N} \times \mathbf{S}$. At time step t , the

corresponding entry of $\mathbf{Z}$ is $\mathbf{WX}_t + \mathbf{B}$. For 1-dimensional input and output vectors, $\mathbf{X}$, $\mathbf{Y}$, $\mathbf{B}$ become respectively $\mathbf{x}$, $\mathbf{y}$, $\mathbf{b}$.

19.5.7 Regularization techniques

Regularization is a technique which makes slight modifications to the learning algorithm such that the model generalizes better. This in turn improves the model's performance on the unseen data as well.

In deep learning, it actually penalizes the weight matrices of the nodes. Assume that our regularization coefficient is so high that some of the weight matrices are nearly equal to zero.

This will result in a much simpler linear network and slight under-fitting of the training data.

Such a large value of the regularization coefficient is not that useful. We need to optimize the value of regularization coefficient in order to obtain a well-fitted model.

Now that we have an understanding of how regularization helps in reducing overfitting, we'll learn a few different techniques in order to apply regularization in deep learning.

19.5.7.1 L1 or L2 regularization

L1 and L2 are the most common types of regularization. These update the general cost function by adding another term known as the regularization term.

$$\text{Cost function} = \text{Loss function} + \text{Regularization term} \quad (19.5.11)$$

Due to the addition of this regularization term, the values of weight matrices decrease because it assumes that a neural network with smaller weight matrices leads to simpler models. Therefore, it will also reduce overfitting to quite an extent.

However, this regularization term differs in L1 and L2.

In L2, we have

$$\text{Cost function} = \text{Loss function} + \frac{\lambda}{2m} \sum_{h=1}^L \sum_i^{I_h} \sum_j^{J_h} \|w_{ij}^{[h]}\|^2 \quad (19.5.12)$$

Here, λ is the regularization parameter. It is the hyperparameter whose value is optimized for better results. L2 regularization is also known as weight decay as it forces the weights $w_{ij}^{[h]}$ to decay towards zero (but not exactly zero).

In L1, we have

$$\text{Cost function} = \text{Loss function} + \frac{\lambda}{2m} \sum_{h=1}^L \sum_i^{I_h} \sum_j^{J_h} \|w_{ij}^{[h]}\| \quad (19.5.13)$$

In this, we penalize the absolute value of the weights $w_{ij}^{[h]}$. Unlike L2, the weights may be reduced to zero here. Hence, it is useful when we are trying to compress our model. Otherwise, we usually prefer L2 over it.

19.5.7.2 Dropout

Dropout is the most frequently used regularization technique in deep learning. Dropout can be applied to both the hidden layers as well as the input layers.

At each iteration, dropout randomly selects with a given probability some nodes and set all their incoming and outgoing connections to zero.

Due to these reasons, dropout is usually preferred for a neural network with a large structure in order to introduce more randomness.

This probability of choosing how many nodes in a layer h should be dropped is the hyperparameter of the dropout function.

At training time, a layer h randomly sets its input nodes from the previous layer $h - 1$ to zero determined by the dropout mask

$$\text{rand}(\text{size}(X^{[h-1]})) < \text{Probability} \quad (19.5.14)$$

where $X^{[h-1]}$ denotes the input from layer $h - 1$ and then scales the remaining nodes by $1/(1-\text{Probability})$.

This operation effectively changes the neural network structure between iterations and helps prevent the network from overfitting [Sri1401]. At prediction time, the output of the layer is equal to its input and no nodes are dropped.

19.5.7.3 Data augmentation

The simplest way to reduce overfitting is to increase the size of the training data.

For images, there are a few ways of increasing the size of the training data by rotating each image, flipping, scaling, shifting, etc.

This technique is known as data augmentation. This usually provides a big improvement in the accuracy of predictions.

19.5.7.4 Early stopping

Early stopping is a kind of cross-validation strategy where we keep one part of the training set as a validation set.

When the performance on the validation set has reached the minimum of the loss function, the training is stopped immediately on the neural network.

19.5.8 Output layer

19.5.8.1 Softmax and classification output layer

A softmax layer applies a softmax function to the input.

A classification layer computes the cross entropy loss for multi-class classification problems with mutually exclusive classes.

For classification problems, a softmax layer and then a classification layer must follow the final fully connected layer.

The output unit activation function is the softmax activation function which normalizes the output of the fully connected layer.

$$y_r(x) = \frac{\exp[a_r(x)]}{\sum_{j=1}^k \exp[a_j(x)]} \quad (19.5.15)$$

where $0 \leq y_r(x) \leq 1$ and $\sum_{j=1}^k y_j(x) = 1$.

The output of the softmax layer consists of positive numbers that sum to one, which can then be used as classification probabilities $y_r(x) = P(c_r|x, \theta)$ by the classification layer.

The softmax function is the output unit activation function after the last fully connected layer for multi-class classification problems:

$$P(c_r|x, \theta) = \frac{P(x, \theta|c_r)P(c_r)}{\sum_{j=1}^k P(x, \theta|c_j)P(c_j)} = \frac{\exp[a_r(x, \theta)]}{\sum_{j=1}^k \exp[a_j(x, \theta)]} \quad (19.5.16)$$

where

$$0 \leq P(c_r|x, \theta) \leq 1 \text{ and } \sum_{j=1}^k P(c_j|x, \theta) = 1 \quad (19.5.17)$$

Moreover,

$$a_r(x, \theta) = \ln[P(x, \theta|c_r)P(c_r)] \quad (19.5.18)$$

and $P(x, \theta|c_r)$ is the conditional probability of the sample given class r , and $P(c_r)$ is the class prior probability.

The softmax function is also known as the normalized exponential and can be considered the multi-class generalization of the logistic sigmoid function [Bis0600].

A classification output layer must follow the softmax layer. This layer uses the probabilities returned by the softmax activation function for each input to assign the input to one of the mutually exclusive classes and compute the loss.

In the classification output layer, the network takes the values from the softmax activation function and assigns each input to one of the K

mutually exclusive classes using the cross-entropy function for a 1-of- K coding scheme and compute the loss

$$Loss(\theta) = - \sum_{i=1}^N \sum_{j=1}^K t_{ij} \ln y_j(x_i, \theta) \quad (19.5.19)$$

where N is the number of samples, K is the number of classes, t_{ij} is the indicator that the i th sample belongs to the j th class, θ is a parameter vector, and $y_j(x_i, \theta)$ is the output for sample i for class j , which in this case, is the value from the softmax function. That is, it is the probability that the network associates the i th input with class j .

19.5.8.2 Regression output layer

A regression layer computes the half-mean-squared-error loss for regression problems. For typical regression problems, a regression layer must follow the final fully connected layer.

For a single observation, the mean-squared-error is defined by

$$MSE = \sum_{i=1}^R \frac{(t_i - y_i)^2}{R} \quad (19.5.20)$$

where R is the number of responses, t_i is the target output, and y_i is the network's prediction for response i .

For image and sequence-to-one regression networks, the loss function of the regression layer is the half-mean-squared-error of the predicted responses, without normalization by R as defined by

$$Loss(\theta) = \frac{1}{2} \sum_{i=1}^R (t_i - y_i)^2 \quad (19.5.21)$$

For image-to-image regression networks, the loss function of the regression layer is the half-mean-squared-error of the predicted responses for each pixel, without normalization by R as defined by

$$Loss(\theta) = \frac{1}{2} \sum_{p=1}^{H,W,nC} (t_p - y_p)^2 \quad (19.5.22)$$

where H , W , and nC denote the height, width, and number of channels of the output respectively, and p indexes into each element (pixel) of t and y linearly.

For sequence-to-sequence regression networks, the loss function of the regression layer is the half-mean-squared-error of the predicted responses for each time step, without normalization by R as defined by

$$Loss(\theta) = \frac{1}{2S} \sum_{i=1}^S \sum_{j=1}^R (t_{ij} - y_{ij})^2 \quad (19.5.23)$$

where S is the sequence length.

When training, the software calculates the mean loss over the observations in the mini-batch.

19.5.9 Batch size

The number of training samples in a dataset can be divided into a number of batches of the same batch size. Each batch is allowed to propagate through a neural network for one pass training to update connection weights. One pass includes one forward pass and one backward pass in training.

If there are a total of 1000 training samples and a batch size of 100 is chosen. The algorithm takes the first 100 samples (from samples 1 to 100) from the training dataset to train the network. Next, it takes the second 100 samples (from samples 101 to 200) to train the network again. This procedure is repeated until all the 1000 samples are allowed to propagate through the network to complete an epoch.

If there are a total of 1060 samples instead of 1000 samples, the remaining 60 samples form the last batch of training samples. The pros and cons for adopting a batch size less than the total number of samples are described as follows.

Pros: Lesser memory and faster training

Adopting a fewer samples to train a neural network, the overall training procedure requires less memory.

Typically neural networks can be trained faster with a number of mini-batches. It is because weights are updated once after one pass of each mini-batch. If all the samples are used for training in an epoch, only one update for the network's parameters is performed in each epoch.

Cons: Reduced accuracy for gradient estimation

A smaller batch size results in a less accurate estimate of the gradient. In general, the gradient direction of a mini-batch fluctuates much more when compared to the gradient direction of the full batch.

For a batch size of 1, the gradient direction changes even more often than that of a mini-batch.

19.6 Training of Convolutional Neural Networks

Before the start of training, the weights values of a CNN are randomly initialized. The backpropagation algorithm consists of 4 distinct steps namely the forward pass, the loss function, the backward pass, and the weight update.

During the forward pass, the training input is passed through the whole network, the output of the forward pass is applied to the error function. The error function can be expressed in terms of the sum of squared output errors as

$$E = \sum_{i=1}^I (target(i) - output(i))^2 \quad (19.6.1)$$

The training algorithm tries to minimize the error function (19.6.1) so that the recall output becomes the same as the desired output. The training process can be formulated as an optimization problem to minimize the error function value.

A back pass is then performed to find out which weights contributed most to the error and finding ways to adjust them so that the error is reduced.

After all the four steps, the weights of the CNN are updated so that they change in the opposite directions of their gradients.

$$w(t) = w(t - 1) - \varepsilon \frac{dE}{dw(t - 1)} \quad (19.6.2)$$

where $w(t)$ denotes the current weight, $w(t - 1)$ denotes the previous weight, and ε is the learning rate parameter.

A large learning rate means that bigger steps are taken in weight updates and consequently, it may take less time for the CNN to converge. However, a learning rate that is too large could result in changes that are too large and not precise enough to reach the optimal point.

The process of forward pass, loss function, backward pass, and parameter update forms one training iteration. The above steps will repeat for a fixed number of iterations for a complete set of training inputs until convergence.

Design Example 19.4 (Create and train simple convolutional neural network for classification)

This example [Mat-DLN] shows how to create and train a simple convolutional neural network for deep learning classification in Matlab. The example serves to demonstrate how to load and explore image data, define the network architecture, specify training options, train the network, predict the labels of new data and calculate the classification accuracy. The Matlab program is listed in Table E19.4.1 and samples of input patterns and results are shown in Figures E19.4.1 and E19.4.2.

Table E19.4.1 Matlab program to create and train CNN.

```
% example19_4.m

clc;
close all
clear all

% Load the digit sample data as imageDatastore.
digitDatasetPath = ...
fullfile(matlabroot,'toolbox','nnet','nndemos', ...
'ndatasets','DigitDataset');
digitData = imageDatastore(digitDatasetPath, ...
'IncludeSubfolders',true,'LabelSource','foldernames');

% Display some of the images in the datastore.
figure;
perm = randperm(10000,20);
for i = 1:20
    subplot(4,5,i);
    imshow(digitData.Files{perm(i)});
end

% Calculate the number of images in each category.
labelCount = countEachLabel(digitData);

% Specify the size of the images in the input layer
% of the network.
img = readimage(digitData,1);
size(img);

% Specify Training and Validation Sets
trainNumFiles = 750;
[trainDigitData,valDigitData] = ...
splitEachLabel(digitData,trainNumFiles,'randomize');

% Define convolutional neural network architecture.
layers = [
    imageInputLayer([28 28 1])
```

```
convolution2dLayer(3,8,'Padding',1)
batchNormalizationLayer
reluLayer

maxPooling2dLayer(2,'Stride',2)

convolution2dLayer(3,16,'Padding',1)
batchNormalizationLayer
reluLayer

maxPooling2dLayer(2,'Stride',2)

convolution2dLayer(3,32,'Padding',1)
batchNormalizationLayer
reluLayer

fullyConnectedLayer(10)
softmaxLayer
classificationLayer];

% Specify training options
options = trainingOptions('sgdm', ...
    'InitialLearnRate',0.01, ...
    'MaxEpochs',4, ...
    'Shuffle','every-epoch', ...
    'ValidationData',valDigitData, ...
    'ValidationFrequency',30, ...
    'Verbose',false, ...
    'Plots','training-progress');

% Train network using training data
net = trainNetwork(trainDigitData,layers,options);

% Classify validation images and compute accuracy
predictedLabels = classify(net,valDigitData);
valLabels = valDigitData.Labels;

accuracy = sum(predictedLabels == valLabels)/numel(valLabels)
```

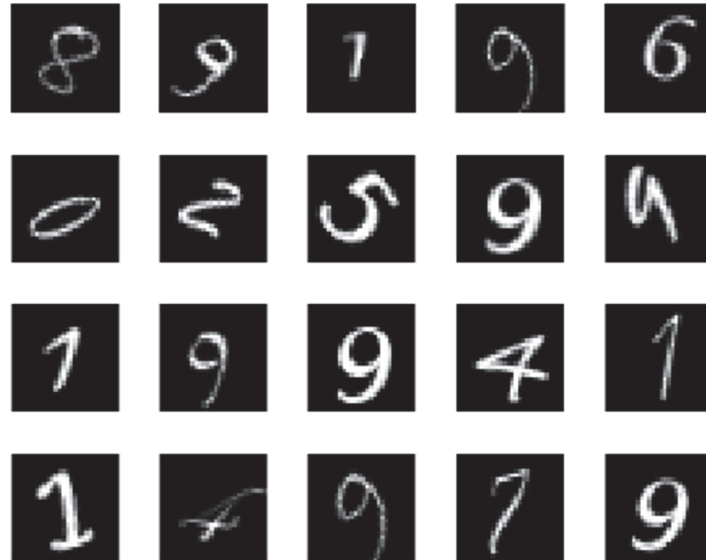


Figure E19.4.1 Samples of input patterns.

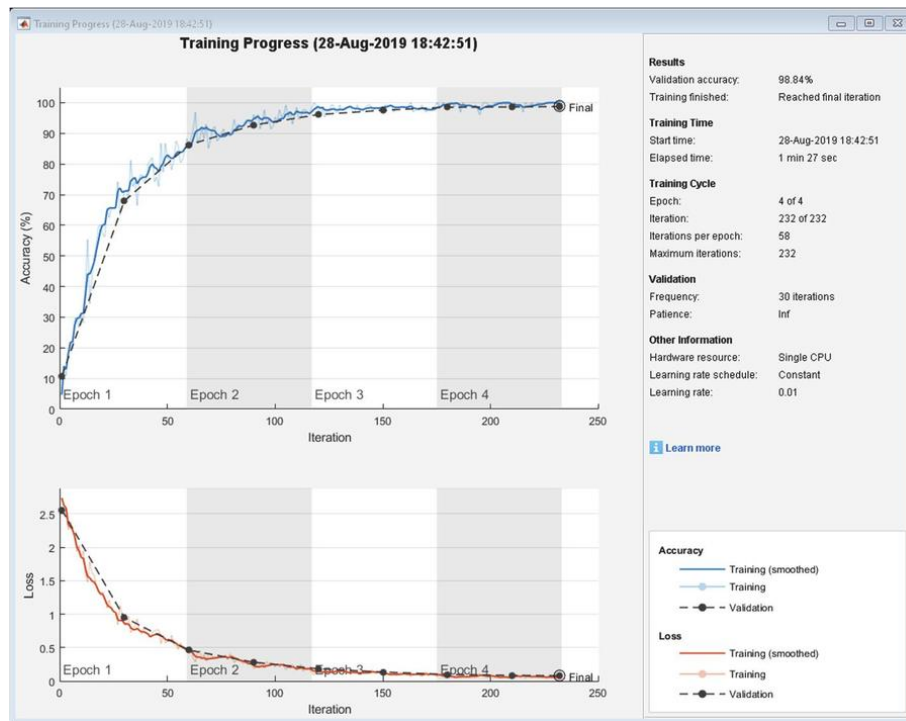


Figure E19.4.2 Training progress plot showing the mini-batch loss and accuracy and the validation loss and accuracy (=0.9884).

Chapter 20

Deep Residual Learning and Networks

20.1 Introduction

Deeper feedforward neural networks are difficult to train. When deeper neural networks are able to start converging, deterioration can emerge such that with an increasing number of layers, accuracy gets saturated and then degrades rapidly.

As reported in [He1506], [Sri1511a] and verified by [He1512a], such deterioration is not caused by overfitting, adding more layers to a suitably deep neural network can result in a higher training error. Figure 20.1.1 shows the training and testing results obtained [He1512a] using CIFAR-10 images dataset.

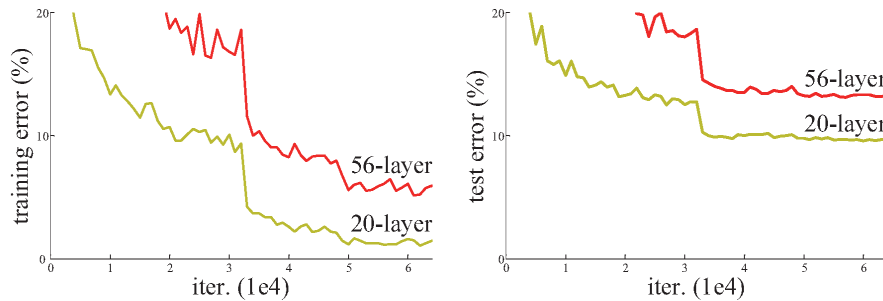


Figure 20.1.1 CIFAR-10 training errors and testing errors for CNN consisting of 20 and 56 layers.

The deterioration of training accuracy indicates that not all deep neural networks are similarly easy to be trained. Consider a deeper neural network with added layers that are identity mapped and its earlier layers are copied from its learned shallower model.

Such a deeper neural network should produce no higher training error than its shallower counterpart. Based on the similar idea, deep residual

networks (ResNets) has been developed in [He1512a] which is to be described in this chapter.

In this chapter, three deep residual networks (namely original ResNets, ResNets, and ResNeXts) are described and explained. The focus of this chapter is on the description and explanation on their structures and principles which also include key comparisons results among themselves and other state-of-the-art networks.

The original sources of the contents in this chapter and additional details of these three deep residual networks are given by the following three references [He1512a], [He1607], and [Xie1704].

20.2 Residual Units

Deep residual networks [He1512a] is formulated based on the hypothesis that a feedforward neural network with an input $\mathbf{x}$ is easier to be trained for nonlinear mapping to $F(\mathbf{x}) = 0$ than for mapping to $F(\mathbf{x}) = \mathbf{x}$.

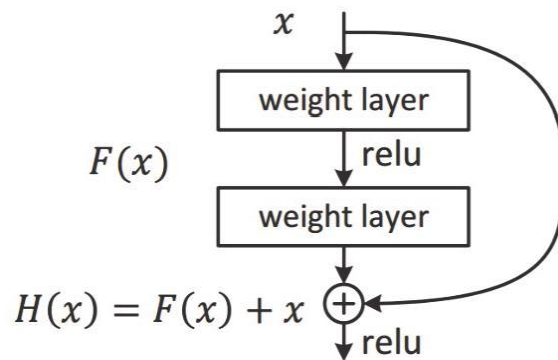


Figure 20.2.1 2-layer residual unit.

Figure 20.2.1 shows a 2-layer residual unit for residual learning which obtains $F(\mathbf{x})$ by mapping an input $\mathbf{x}$ through two weight layers. By adding $\mathbf{x}$ to $F(\mathbf{x})$, an output $H(\mathbf{x})$ is obtained.

Assuming that $H(\mathbf{x})$ is the desired mapping output, since $H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$, obtaining the desired $H(\mathbf{x})$ depends on getting the perfect

$F(\mathbf{x})$. This requires that the two weight layers in the residual unit be able to learn to recall the desired $F(\mathbf{x})$ in order to recall the desired $H(\mathbf{x})$.

In Figure 20.2.1, $F(\mathbf{x})$ is obtained from $\mathbf{x}$ as

$$\mathbf{x} \rightarrow \mathbf{W}^{[1]} \rightarrow ReLU \rightarrow \mathbf{W}^{[2]} \rightarrow F(\mathbf{x}) \quad (20.2.1)$$

Also, $H(\mathbf{x})$ is obtained from $F(\mathbf{x})$ as

$$F(\mathbf{x}) + \mathbf{x} \rightarrow ReLU \rightarrow H(\mathbf{x}) \quad (20.2.2)$$

Consider a simple example, assuming that the desired mapping is $H(\mathbf{x}) = \mathbf{x}$. As there is a stack of nonlinear layers, it would be difficult to learn the output mapping $H(\mathbf{x}) = \mathbf{x}$ such that

$$\mathbf{x} \rightarrow \mathbf{W}^{[1]} \rightarrow ReLU \rightarrow \mathbf{W}^{[2]} \rightarrow ReLU \rightarrow \mathbf{x} \quad (20.2.3)$$

However, it would be easier to learn the mapping $F(\mathbf{x}) = 0$ which can be achieved by having $\mathbf{W}^{[1]}$ or $\mathbf{W}^{[2]}$ or both equal zero such that

$$\mathbf{x} \rightarrow \mathbf{W}^{[1]} \rightarrow ReLU \rightarrow \mathbf{W}^{[2]} \rightarrow ReLU \rightarrow 0 \quad (20.2.4)$$

Then by adding $\mathbf{x}$ to (20.2.4), we can obtain

$$H(\mathbf{x}) = \mathbf{x} \quad (20.2.5)$$

20.3 Identity Mapping by Shortcuts

There are two types of residual connections:

Type 1. When the input $\mathbf{x}$ and output $F(\mathbf{x})$ are of equal dimensions, the identity shortcut $\mathbf{x}$ can be directly connected such that

$$\mathbf{y} = F(\mathbf{x}, \mathbf{W}^{[i]}) + \mathbf{x} \quad (20.3.1)$$

In (20.3.1), the function $F(\mathbf{x}, \mathbf{W}^{[i]})$ denotes operations across multiple convolutional layers.

Type 2. When the input $\mathbf{x}$ and output $F(\mathbf{x})$ are of different dimensions:

(a) The identity shortcut $\mathbf{x}$ can still perform identity mapping by padding extra zero entries to match the larger dimensions; or (b) A projection shortcut $\mathbf{w}_s$ can be used to match the dimensions by 1×1 convolutions such that

$$\mathbf{y} = F(\mathbf{x}, \mathbf{W}^{[i]}) + \mathbf{W}_s \mathbf{x} \quad (20.3.2)$$

For both Type 2 options, when shortcuts go across the feature maps of two dimensions, a stride of 2 operation can be performed.

20.4 Deep Residual Networks

In this section, the principle and structure of the original residual networks (original ResNets) and its deep residual networks based on [He1512a] is described and explained.

20.4.1 Bottleneck 3-layer residual units

2-layer and 3-layer residual units as shown in Figure 20.4.1 that can be cascaded to form a deep residual network for learning the ImageNet.

To reduce training time, a stack of 3 layers instead of 2 can be used in practice. The three layers are 1×1 , 3×3 , and 1×1 convolutions, where the 1×1 layers are responsible for reducing in layer-1 dimensions and then increasing (or restoring) in layer-3 dimensions, leaving the 3×3 layer a bottleneck with smaller input/output dimensions.

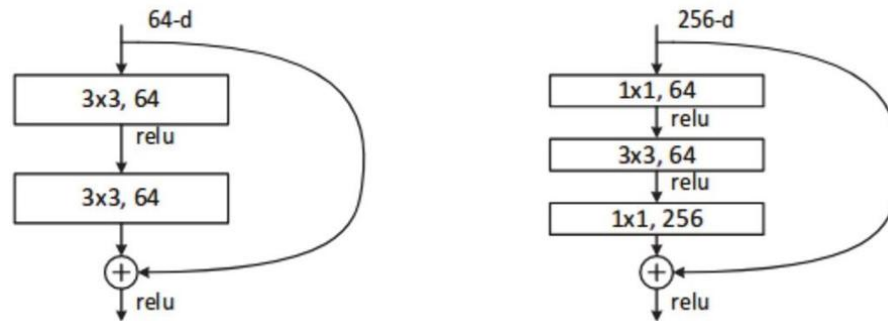


Figure 20.4.1 A deeper residual function $F(\mathbf{x})$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Figure 20.2.1 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

Table 20.4.1 Architectures for ImageNet. Building blocks are shown in brackets (details and other variants are shown for Figure 20.4.1), with the numbers of blocks stacked. Down-sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

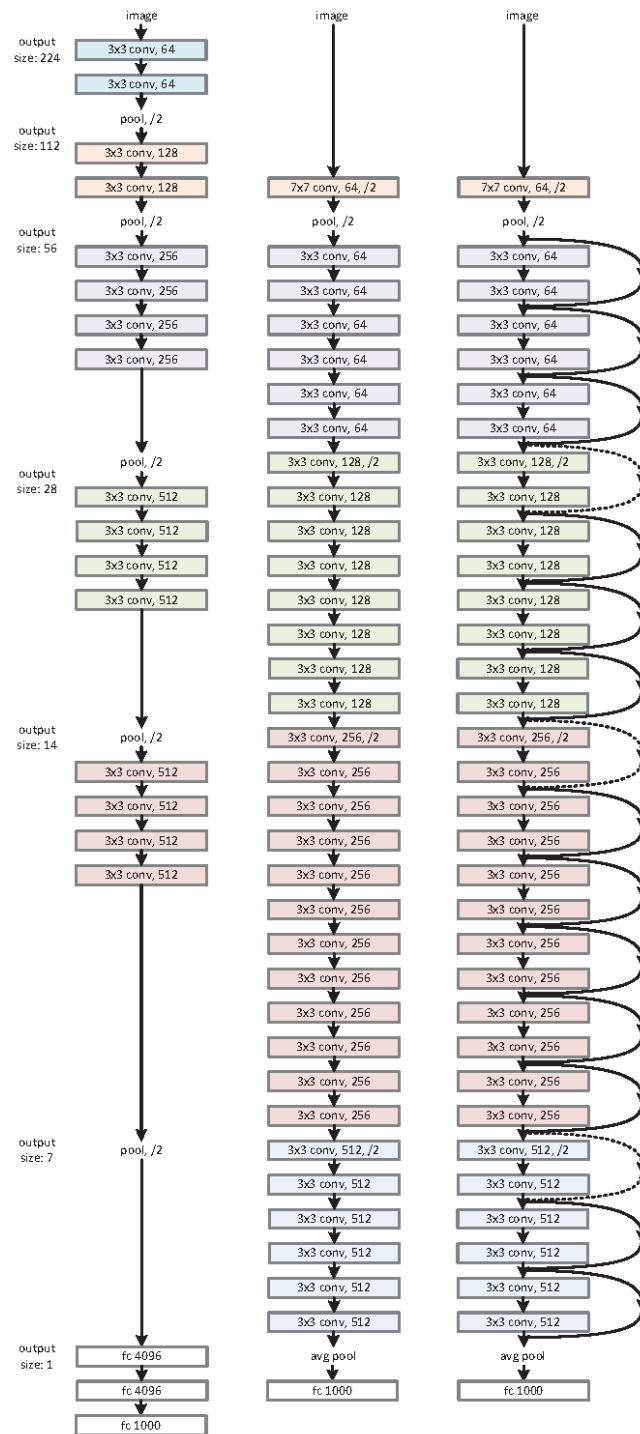


Figure 20.4.2 Example network architectures for ImageNet. Left: the VGG-19 model [Sim1504] (19.6 billion FLOPs) as a reference. Middle: a CNN with 34 parameter layers (3.6 billion FLOPs). Right: a ResNet with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions.

20.4.2 Performance and comparison of original ResNets with other networks

The method [He1512a] was applied to the ImageNet 2012 classification dataset [Rus1400] that consists of 1000 classes. The models are trained on the 1.28 million training images, and evaluated on the 50k validation images. A final result on the 100k test images, reported by the test server was obtained.

In each of the three networks as shown in Figure 20.4.2: (a) 3x3 convolutional filters are used except for one 7x7 convolutional filter is used in the first layer immediate after the input for networks 2 and 3; (b) Down-sampling with stride 2 are used; (c) 1000-way fully connected output layer by Softmax and global average pooling. More details on the configuration of the two 34 parameter layers are listed in Table 20.4.1.

Using projection shortcuts of (20.7) in all the layers resulted a very small incremental gain. Consequently, all ResNet units used only identity shortcuts with projections shortcuts used only when dimensions change.

With the same number of parameters, Figure 20.4.3 shows that (a) CNN-18 performs better than CNN-34 in terms of training and validation errors; and (b) ResNet-34 performs better than ResNet-18 in terms of training and validation errors.

The results indicate that the ResNets can achieve lower training and validation errors as the number of layers increases from 18 to 34. On the other hand, the increase of the number of layers from 18 to 34 cannot achieve lower training and validation errors.

From Figure 20.4.3, it can be observed that ResNets can converge faster than CNNs using the same number of layers.

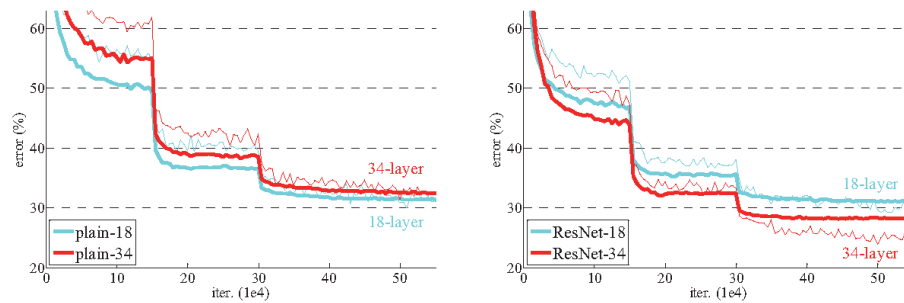


Figure 20.4.3 ImageNet training errors (thin curves) and validation errors (bold curves) for CNN (left) and ResNets (right) with same number of parameters.

In Table 20.4.2, three options are compared. A: Zero-padding shortcuts are used for increasing dimensions, and all shortcuts are parameter free (Same in Figure 20.4.2 right); B: Projection shortcuts are used for increasing dimensions, and other shortcuts are identity; and C: All shortcuts are projections.

Table 20.4.2 shows that all three options are considerably better than the corresponding CNN-34. B is slightly better than A. According to [He1512a], this is because the zero-padded dimensions in A indeed have no residual learning. C is marginally better than B, and according to [He1512a], this is due to the extra parameters introduced by thirteen projection shortcuts.

However, the small differences among A, B, and C indicate that projection shortcuts are not essential for addressing the learning deterioration problem. Consequently, option C is no longer used to reduce memory and time complexity, and model dimensions.

As shown in Table 20.4.2, the ResNets are compared to the previous best single-network results. The baseline 34-layer ResNets have achieved very competitive accuracy.

Table 20.4.2 Error rates (10-crop testing) on ImageNet validation. VGG-16 is based on [He1512a].

Network	Top-1 error %	Top-5 error %
VGG-16 [Sim1504]	28.07	9.33
GoogLeNet [Sze1506]	-	9.15
PReLU-net [He1512b]	24.27	7.38
CNN-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50 B	22.85	6.71
ResNet-101 B	21.75	6.05
ResNet-152 B	21.43	5.71

As shown in Table 20.4.3, the 152-layer ResNet has a single-network top-5 validation error of 4.49%. This single-network result outperforms all previous ensemble results.

Table 20.4.3 Error rates of single-network results on the ImageNet validation set. VGG is based on [He1512a].

Network	Top-1 error %	Top-5 error %
VGG [Sim1504] (ILSVRC'14)	-	8.43
GoogLeNet[Sze1506](ILSVRC'14)	-	7.89
VGG [Sim1504]	24.40	7.10
PReLU-net [He1512b]	21.59	5.71
BN-inception [Iof1503]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

As shown in Table 20.4.4, six networks of different depths are combined to form an ensemble (only with two 152-layer ones). This leads to 3.57% top-5 error on the test set.

Table 20.4.4 Error rates of ensembles. The top-5 error is on the test set of ImageNet and reported by the test server.

Network	Top-5 error % (test)
VGG [Sim1504] (ILSVRC'14)	7.32
GoogLeNet [Sze1506] (ILSVRC'14)	6.66
VGG [Sim1504]	6.80
PReLU-net [He1512b]	4.94
BN-inception [Iof1503]	4.82
ResNet (ILSVRC'15)	3.57

20.5 Identity Mappings in Deep Residual Networks

In this section, the principle of identity mappings in deep residual networks based on [He1607] is described and explained.

Deep residual networks (ResNets) [He1512a] consist of many stacked residual units. A residual unit represented by (20.3.1) can be generalized to a form expressed by

$$\mathbf{y}_l = g(\mathbf{x}_l) + F(\mathbf{x}_l, \mathbf{W}^{[l]}) \quad (20.5.1)$$

$$\mathbf{x}_{l+1} = f(\mathbf{y}_l) \quad (20.5.2)$$

where $\mathbf{x}_l$ and $\mathbf{x}_{l+1}$ are the input and output of the l th unit, and $F(\cdot)$ is a residual function. In [He1512a], $g(\mathbf{x}_l) = \mathbf{x}_l$ is an identity mapping. In (20.5.3), $f(\cdot)$ is a ReLU function [Nai1006].

Here $\mathbf{x}_l$ is the input feature to the l th residual unit. $\mathbf{W}^{[l]}$ denotes the set of weights (and bias) associated with the l th residual unit for $l = 1$ to L ($L = 2$ or 3 in [He1512a]). $F(\cdot)$ denotes the residual function, e.g., a stack of two 3×3 convolutional layers in [He1512a]. The function $f(\cdot)$ is the operation after element-wise addition, and in [He1512a] $f(\cdot)$ is a ReLU function. The function $g(\cdot)$ is set as an identity mapping $g(\mathbf{x}_l) = \mathbf{x}_l$.

If $f(\cdot)$ is also an identity mapping such that $\mathbf{x}_{l+1} = \mathbf{y}_l$, we can put (20.5.2) into (20.5.1) and obtain

$$\mathbf{x}_{l+1} = \mathbf{x}_l + F(\mathbf{x}_l, \mathbf{W}^{[l]}) \quad (20.5.3)$$

The central idea of ResNets is to learn the additive residual function $F(\mathbf{x}_l, \mathbf{W}^{[l]})$ with respect to $g(\mathbf{x}_l)$, with a key choice of using an identity mapping $g(\mathbf{x}_l) = \mathbf{x}_l$ which can be realized by attaching an identity skip connection (shortcut) as shown on the left side of each of Figure 20.5.1 (a) and (b). The residual unit shown in Figure 20.5.1 (b) proposed in [He1607] enables ResNet-1001 to be trained easier.

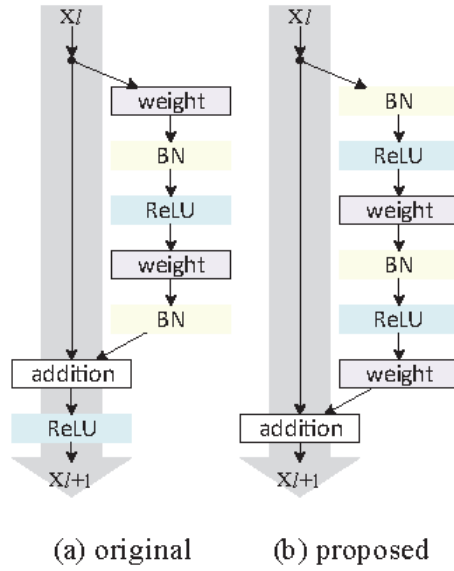


Figure 20.5.1 (a) Original residual unit [He1512a]; (b) Residual unit [He1607]. The wide grey arrows (on left of (a) and (b)) indicate the easiest paths for the information to propagate, corresponding to the additive term $\mathbf{x}_l$ in (20.5.1) (forward propagation).

Hence, from (20.5.3), the L th layer output can be expressed as

$$\mathbf{x}_L = \mathbf{x}_l + \sum_{h=l}^{L-1} F(\mathbf{x}_h, \mathbf{W}^{[h]}) \quad (20.5.4)$$

Starting from the input layer with $l = 0$, (20.5.4) becomes

$$\mathbf{x}_L = \mathbf{x}_0 + \sum_{h=0}^{L-1} F(\mathbf{x}_h, \mathbf{W}^{[h]}) \quad (20.5.5)$$

The following properties can be seen from (20.5.4) and (20.5.5):

(i) From (20.5.4), the feature $\mathbf{x}_L$ of any deeper unit L can be represented as the feature $\mathbf{x}_l$ of any shallower unit l plus a residual function in a form of $\sum_{l=1}^{L-1} F(\mathbf{x}_l, \mathbf{W}^{[l]})$, indicating that the model is in a residual fashion between any units L and l .

(ii) From (20.5.5), the feature $\mathbf{x}_L$ of any deeper unit L can be obtained from the summation of the outputs of all preceding residual functions plus $\mathbf{x}_0$.

The above is different from a feedforward neural network and according to (4.2.1)-(4.2.3), the feature $\mathbf{x}_L$ is equal to a series of matrix-vector products as

$$\mathbf{x}_k^{[l]} = G\left(\boldsymbol{\theta}^{[l]} + \mathbf{W}^{[l]}\mathbf{x}_k^{[l-1]}\right) \text{ for } l = 1 \text{ to } L \quad (20.5.6)$$

In (20.5.6), the activation function $G(\cdot)$ has embedded both batch normalization and ReLU operations. Equation (20.5.4) also leads to nice backward propagation properties. Denoting the loss function as E , according to the chain rule of backpropagation in Chapter 4, we have

$$\frac{\partial E}{\partial \mathbf{x}_l} = \frac{\partial E}{\partial \mathbf{x}_L} \frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_l} = \frac{\partial E}{\partial \mathbf{x}_L} \left(1 + \frac{\partial}{\partial \mathbf{x}_l} \sum_{l=1}^{L-1} F(\mathbf{x}_l, \mathbf{W}^{[l]})\right) \quad (20.5.7)$$

Equation (20.5.7) indicates that the gradient $\partial E / \partial \mathbf{x}_l$ can be decomposed into two additive terms as

$$\frac{\partial E}{\partial \mathbf{x}_l} = \frac{\partial E}{\partial \mathbf{x}_L} + \left(\frac{\partial E}{\partial \mathbf{x}_L} \frac{\partial}{\partial \mathbf{x}_l} \sum_{l=1}^{L-1} F(\mathbf{x}_l, \mathbf{W}^{[l]})\right) \quad (20.5.8)$$

Equation consists of a term of $\partial E / \partial \mathbf{x}_L$ that propagates information directly without concerning any weight layers, and another term that propagates through the weight layers.

The additive term of $\partial E / \partial \mathbf{x}_L$ ensures that information is directly propagated back to any shallower residual unit l . Equation (20.5.8) also suggests that it is unlikely for the gradient $\partial E / \partial \mathbf{x}_l$ to be canceled out for a mini-batch, because the term $(\partial E / \partial \mathbf{x}_L) \sum_{l=1}^{L-1} F(\mathbf{x}_l, \mathbf{W}^{[l]})$ in general cannot be always -1 for all samples in a mini-batch. This implies

that the gradient of a layer does not vanish even when the weights are arbitrarily small.

Equations (20.5.4) and (20.5.7) suggest that the signal can be directly propagated from any unit to another, both forward and backward. The basic of (20.5.4) is two identity mappings: (i) the identity skip connection $g(\mathbf{x}_l) = \mathbf{x}_l$, and (ii) the condition that $\mathbf{x}_{l+1} = f(\mathbf{y}_l)$ is an identity mapping. These directly propagated information flows are represented by the grey arrows in Figure 20.5.1. And the above two conditions are true when these grey arrows cover no operations (except addition) and thus are clean. In the following two sections we separately investigate the impacts of the two conditions.

20.5.1 Importance of identity skip connections

Consider a simple modification, $g(\mathbf{x}_l) = \lambda_l \mathbf{x}_l$, is used to break the identity shortcut such that

$$\mathbf{x}_{l+1} = \lambda_l \mathbf{x}_l + F(\mathbf{x}_l, \mathbf{W}^{[l]}) \quad (20.5.9)$$

where λ_l is a modulating scalar (for simplicity, assume $f(\cdot)$ is identity). Recursively applying this formulation we obtain an equation similar to (20.5.4) such that

$$\mathbf{x}_L = \left(\prod_{i=l}^{L-1} \lambda_i\right) \mathbf{x}_l + \sum_{i=l}^{L-1} \left(\prod_{j=i+1}^{L-1} \lambda_j\right) F(\mathbf{x}_i, \mathbf{W}^{[i]}) \quad (20.5.10)$$

Applying the chain rule of backpropagation to (20.17), we have

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{x}_l} = \frac{\partial E}{\partial \mathbf{x}_L} & \left(\prod_{i=l}^{L-1} \lambda_i \right. \\ & \left. + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \left(\prod_{j=i+1}^{L-1} \lambda_j \right) F(\mathbf{x}_i, \mathbf{W}^{[i]}) \right) \end{aligned} \quad (20.5.11)$$

Unlike (20.5.7), the first additive term of (20.5.11) is modulated by a factor $\prod_{i=l}^{L-1} \lambda_i$. For an extremely deep network (L is large), if $\lambda_i > 1$ for all i , this factor can be exponentially large; if $\lambda_i < 1$ for all i , this factor can be exponentially small and vanish, which blocks the backpropagated signal from the shortcut and forces it to flow through

the weight layers. This results in optimization difficulties as observed by simulations.

In the above analysis, the original identity skip connection in (20.5.3) is replaced with a simple scaling $g(\mathbf{x}_l) = \lambda_l \mathbf{x}_l$. If the skip connection $g(\mathbf{x}_l)$ represents more complicated transforms (such as gating and 1×1 convolutions), in (20.5.11) the first term becomes $\prod_{i=l}^{L-1} g'(i)$ where $g'(i)$ is the derivative of $g(i)$. This product may also impede information propagation and hamper the training procedure as observed by simulations.

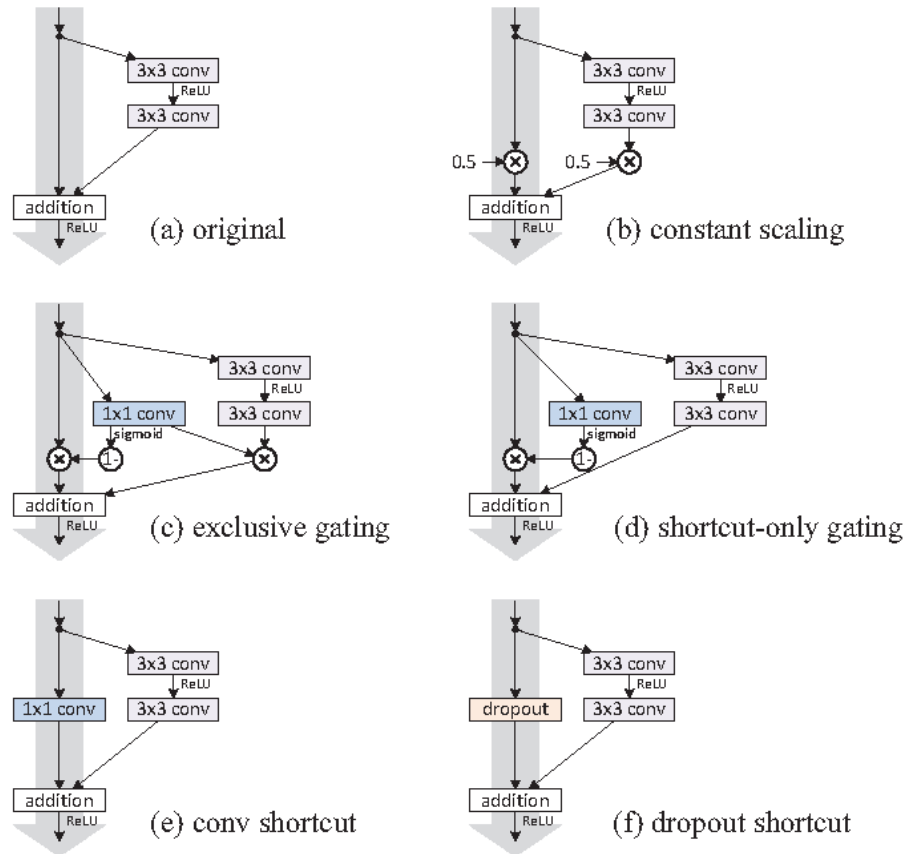


Figure 20.5.2 Various types of shortcut connections. The grey arrows indicate the easiest paths for the information to propagate. The shortcut connections in (b)-(f) are impeded by different components. For

simplicity, the batch normalization layers right after the weight layers for all units are not displayed.

As indicated by the grey arrows in Figure 20.5.2, the shortcut connections are the most direct paths for the information to propagate. Multiplicative manipulations (scaling, gating, 1×1 convolutions, and dropout) on the shortcuts can hamper information propagation and lead to optimization problems.

It is noteworthy that the gating and 1×1 convolutional shortcuts introduce more parameters, and should have stronger representational abilities than identity shortcuts. In fact, the shortcut-only gating and 1×1 convolution cover the solution space of identity shortcuts (i.e., they could be optimized as identity shortcuts). However, their training error is higher than that of identity shortcuts, indicating that the degradation of these models is caused by optimization issues, instead of representational abilities.

20.5.2 Performance and comparison of original ResNets and ResNets

ResNets with over 100 layers have demonstrated an ability to reach the state-of-the-art accuracy for several challenging recognition tasks on ImageNet [Rus1400] and MS COCO [Lin1409] competitions.

Improved results on ImageNet using a 200-layer ResNet can be obtained as compared to the counterpart of [He1512a] where overfitting can be observed. Improved results using a ResNet-1001 can achieve 4.62% error on CIFAR-10 and 22.71% error on CIFAR-100. These results suggest that there is much room to exploit the dimension of network depth, a key to the success of modern deep learning. Details are given in [He1607].

In [He1607], deep residual networks are analyzed by focusing on creating a direct path for propagating information not only within a residual unit, but through the entire network. The derivations reveal that if both $g(\mathbf{x}_l)$ and $f(\mathbf{y}_l)$ are identity mappings, the signal could be directly propagated from one unit to any other units, in both forward and backward passes. The experiments empirically show that training in

general becomes easier when the architecture is closer to the above two conditions.

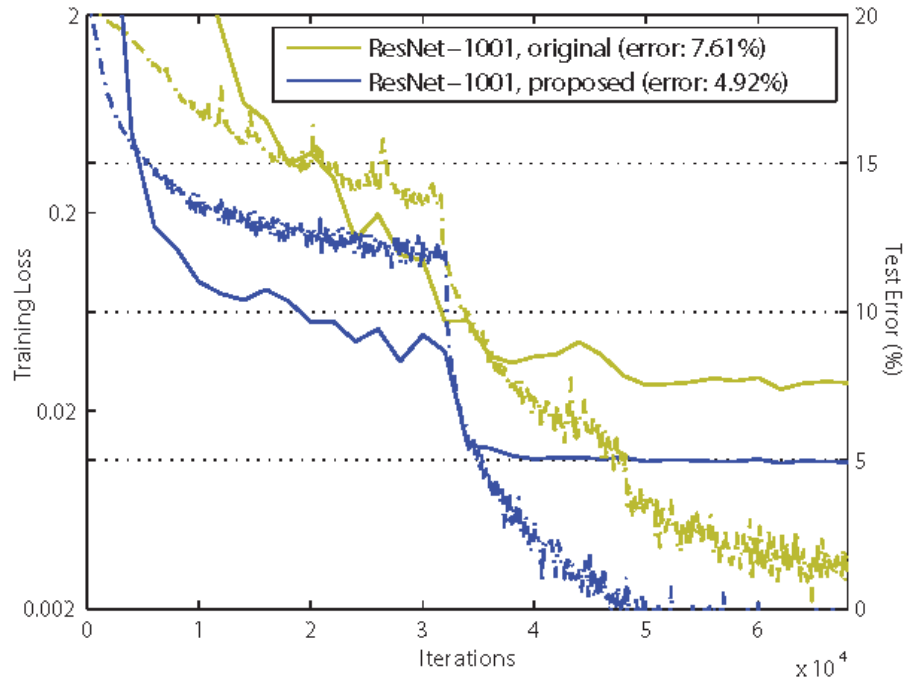


Figure 20.5.3 Training curves on CIFAR-10 of 1001-layer ResNets. Solid lines denote test error (y-axis on right), and dashed lines denote training loss (y-axis on left).

To construct an identity mapping $f(\mathbf{y}_l) = \mathbf{y}_l$, the activation functions (ReLU and BN [Iof1503]) are viewed as the pre-activation of the weight layers, in contrast to conventional wisdom of post-activation. This point of view leads to a new residual unit design, shown in (Figure 20.5.1 (b)). Based on this unit, competitive results on CIFAR-10/100 with a 1001-layer ResNet are presented, which shows that it is much easier to be trained and can generalize better than the original ResNet in [He1512a].

In [He1607], the propagation formulations behind the connection mechanisms of deep residual networks has been investigated. The derivations obtained imply that identity shortcut connections and

identity after-addition activation are essential for making information propagation smooth.

Table 20.5.1 Comparisons with state-of-the-art methods on CIFAR-10 and CIFAR-100 using moderate data augmentation (flip/translation), except for ELU [Cle1602] with no augmentation. Better results of [Gra1505], [Spr1504] have been reported using stronger data augmentation and ensembling. For the ResNets, the number of parameters is listed. Results are obtained by taking the median of 5 runs with mean $\pm$ std in the brackets. All ResNets results are obtained with a mini-batch size of 128 except * with a mini-batch size of 64.

CIFAR-10	Error %
NIN [Lin1403]	8.81
DSN [Lee1505]	8.22
FitNet [Rom1505]	8.39
Highway [Sri1511b]	7.72
All-CNN [Spr1504]	7.25
ELU [Cle1602]	6.55
FitResNet, LSUUV [Mis1602]	5.84
ResNet-100 (1.7M) [He1512a]	6.61
ResNet-1202 (19.4M) [He1512a]	7.93
ResNet-164 (1.7M) [He1607]	5.46
ResNet-1001 (10.2M) [He1607]	4.92 (4.89 $\pm$ 0.14)
ResNet-1001 (10.2M)* [He1607]	4.62 (4.69 $\pm$ 0.20)
CIFAR-100	Error %
NIN [Lin1403]	35.68
DSN [Lee1505]	34.57
FitNet [Rom1505]	35.04
Highway [Sri1511b]	32.39
All-CNN [Spr1504]	33.71
ELU [Cle1602]	24.28
FitResNet, LSUUV [Mis1602]	27.66
ResNet-164 (1.7M) [He1607]	25.16
ResNet-1001 (10.2M) [He1512a]	27.82
ResNet-164 (1.7M) [He1607]	24.33
ResNet-1001 (10.2M) [He1607]	22.71 (22.68 $\pm$ 0.22)

To understand the role of skip connections, various types of $g(\mathbf{x}_l)$ are analyzed and compared and the results shows that the identity mapping $g(\mathbf{x}_l) = \mathbf{x}_l$ chosen in [He1512a] achieves the fastest error reduction and the lowest training loss among all the variants, whereas skip connections of scaling, gating [Hoc9712], [Sri1511a], [Sri1511b], and 1×1 convolutions all lead to a higher training loss and error. These experiments suggest that keeping a direct information path (indicated by the wide grey arrows in Figure 20.5.1) is helpful for easing optimization.

20.6 Aggregated Transformations for Deep Residual Networks

In this section, the principle of a simple highly modularized network architecture for image classification based on [Xie1704] is described and explained.

The network is constructed by repeating a building block that aggregates a set of transformations with the same topology. The simple design results in a homogeneous, multi-branch architecture that has only a few hyper-parameters to set. This strategy exposes a new dimension which is call cardinality (the size of the set of transformations) as an essential factor in addition to the dimensions of depth and width.

On the ImageNet-1K dataset, even under the restricted condition of maintaining complexity, it can be shown empirically that increasing cardinality is able to improve classification accuracy. Moreover, increasing cardinality is more effective than going deeper or wider for increasing the capacity.

20.6.1 Template

A highly modularized design following VGG/ResNets is adopted. The network consists of a stack of residual blocks. These blocks have the same topology, and are subject to two simple rules inspired by VGG/ResNets: (i) if producing spatial maps of the same size, the blocks share the same hyper-parameters (width and filter sizes), and (ii) each time when the spatial map is downsampled by a factor of 2, the width of the blocks is multiplied by a factor of 2.

The second rule ensures that the computational complexity, in terms of FLOPs (floating-point operations, in # of multiply-adds), is roughly the same for all blocks.

With these two rules, only a template module needs to be designed, and all modules in a network can be determined accordingly. So these two rules greatly narrow down the design space and allow us to focus on a few key factors. The networks constructed by these rules are in Table 20.6.1.

Table 20.6.1 ResNet-50 (3rd column) and ResNeXt-50 (4th column) with a $32 \times 4d$ template (using the reformulation in Figure 20.6.2 (c)). Inside the brackets are the shape of a residual block, and outside the brackets is the number of stacked blocks on a stage. “ $C=32$ ” suggests grouped convolutions [Kri1212] with 32 groups. The numbers of parameters and FLOPs are similar between these two networks.

stage	output	ResNet-50	ResNeXt-50 ($32 \times 4d$)
conv1	112×112	$7 \times 7, 64$, stride 2	$7 \times 7, 64$, stride 2
conv2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128, C=32 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256, C=32 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512, C=32 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 1024 \\ 3 \times 3, 1024, C=32 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax
# params.		25.5×10^6	25.0×10^6
FLOPs		4.1×10^9	4.2×10^9

20.6.2 Neuron operation

The simplest neurons in artificial neural networks perform inner product (weighted sum), which is the elementary transformation done by fully-connected and convolutional layers. Inner product can be interpreted as a form of aggregating transformation:

$$\sum_{i=1}^D w_i x_i \quad (20.6.1)$$

where $\mathbf{x} = [x_1; x_2; \dots; x_D]$ is a D -channel input vector to the neuron and w_i is a filter's weight for the i th channel. This operation (usually including some output nonlinearity) is referred to as a neuron.

The above operation can be recast as a combination of splitting, transforming, and aggregating. (i) Splitting: the vector $\mathbf{x}$ is sliced as a low-dimensional embedding, and in the above, it is a single-dimension subspace x_i . (ii) Transforming: the low-dimensional representation is transformed, and in the above, it is simply scaled by $w_i x_i$. (iii) Aggregating: the transformations in all embeddings are aggregated by the summation in (20.6.1).

20.6.3 Aggregated transformations

Based on the above analysis of a simple neuron, consider replacing the elementary transformation ($w_i x_i$) with a more generic function, which in itself can also be a network. In contrast to Network-in-Network [Lin1403] that turns out to increase the dimension of depth, it can be shown that the Network-in-Neuron expands along a new dimension.

Define the aggregated transformations as

$$F(\mathbf{x}) = \sum_{i=1}^C T_i(\mathbf{x}) \quad (20.6.2)$$

where $T_i(\mathbf{x})$ can be an arbitrary function. Analogous to a simple neuron, $T_i(\mathbf{x})$ should project $\mathbf{x}$ into an (optionally low dimensional) embedding and then transform it. In (20.6.2), C is the size of the set of transformations to be aggregated. C is referred as cardinality. In (20.6.2), C is in a position similar to D in (20.6.1), but C needs not equal D and can be an arbitrary number. While the dimension of width is related to

the number of simple transformations (inner product), the dimension of cardinality controls the number of more complex transformations. As shown by experiments, cardinality is an essential dimension and can be more effective than the dimensions of width and depth.

In [Xie1704], a simple way of designing the transformation functions is considered in which all $T_i(\mathbf{x})$ have the same topology. This extends the VGG-style strategy of repeating layers of the same shape, which is helpful for isolating a few factors and extending to any large number of transformations. The individual transformation $T_i(\mathbf{x})$ is chosen to be the bottleneck shaped architecture [He1512a], as shown in Figure 20.6.1 (right). In this case, the first 1×1 layer in each $T_i(\mathbf{x})$ produces the low dimensional embedding.

The aggregated transformation in (20.6.2) serves as the residual function [Spr1504] (Figure 20.6.1 right) as

$$\mathbf{y} = \mathbf{x} + \sum_{i=1}^C T_i(\mathbf{x}) \quad (20.6.3)$$

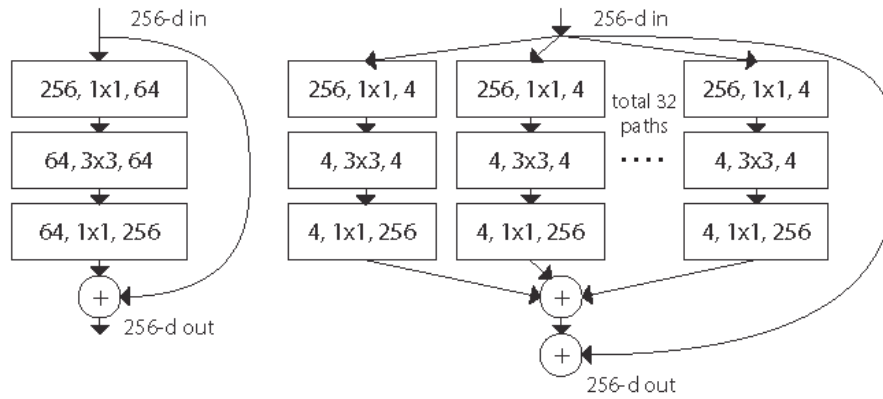


Figure 20.6.1 A block of ResNet (left) [He1512a] and a block of ResNeXt (right) with cardinality = 32, with roughly the same complexity. A layer is labelled by (# in channels, # filter size, # out channels).

Although the present reformulations exhibit concatenation (Figure 20.6.2 (b)) or grouped convolutions (Figure 20.6.2 (c)), such reformulations are not always applicable for the general form of (20.6.3),

e.g., if the transformation $T_i(\mathbf{x})$ takes arbitrary forms and are heterogenous. In [Xie1704], homogenous forms are used because they are simpler and extensible. Under this simplification, grouped convolutions in the form of Figure 20.6.2 (c) are helpful to simplify implementation.

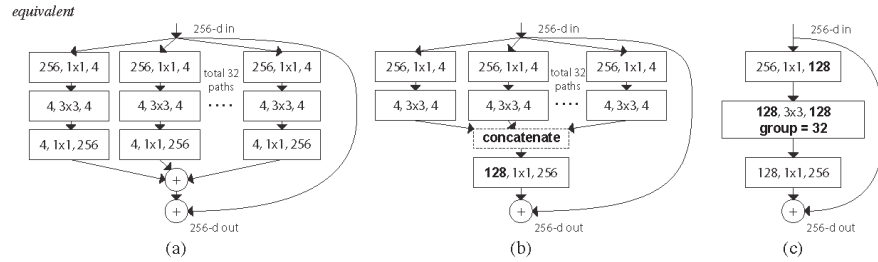


Figure 20.6.2 Equivalent building blocks of ResNeXt. (a): Aggregated residual transformations, the same as Figure 20.6.1 (right). (b): A block equivalent to (a), implemented as early concatenation. (c): A block equivalent to (a, b), implemented as grouped convolutions [Kri1212]. Notations in bold text highlight the reformulation changes. A layer is denoted as (# input channels, # filter size, # output channels).

20.6.4 Model capacity

Experiments in the next section will show that the models improve accuracy when maintaining the model complexity and number of parameters. This is not only interesting in practice, but more importantly, the complexity and number of parameters represent inherent capacity of models and thus are often investigated as fundamental properties of deep networks [Eig1402].

When evaluating different cardinalities C while preserving complexity, the modification of other hyper-parameters needs to be minimized. To isolate from the input and output of the block, the width of the bottleneck (e.g., 4-d in Figure 20.6.1 (right)) is adjusted. This strategy introduces no change to other hyper-parameters (depth or input/output width of blocks), which allows focus to be placed on the impact of cardinality.

In Figure 20.6.1 (left), the original ResNet bottleneck block [He1512a] has $256 \times 64 + 3 \times 3 \times 64 \times 64 + 64 \times 256 \approx 70\text{k}$ parameters and proportional FLOPs (on the same feature map size). With bottleneck width d , the template in Figure 20.6.1 (right) has the following number of parameters and proportional FLOPs.

$$C \times (256 \times d + 3 \times 3 \times d \times d + d \times 256) \quad (20.6.4)$$

When $C = 32$ and $d = 4$, $(20.6.4) \approx 70\text{k}$. Table 20.6.2 shows the relationship between cardinality C and bottleneck width d .

Table 20.6.2 Relations between cardinality and width (for the template of conv2), with roughly preserved complexity on a residual block. The number of parameters is $\sim 70\text{k}$ for the template of conv2. The number of FLOPs is ~ 0.22 billion ($\# \text{ parameters} \times 56 \times 56$ for conv2).

Cardinality C	1	2	4	8	32
Width of bottleneck d	64	40	24	14	4
Width of group convolution	64	80	96	112	128

Due to the adaptation of the two rules in Section 20.6.1, the above approximate equality is valid between a ResNet bottleneck block and our ResNeXt on all stages (except for the subsampling layers where the feature maps size changes). Table 20.6.1 compares the original ResNet-50 and the ResNeXt-50 of similar capacity. It is noted that the complexity can only be preserved approximately, but the difference of the complexity is small and does not affect the results.

20.6.5 Performance and comparison of ResNets and ResNeXts

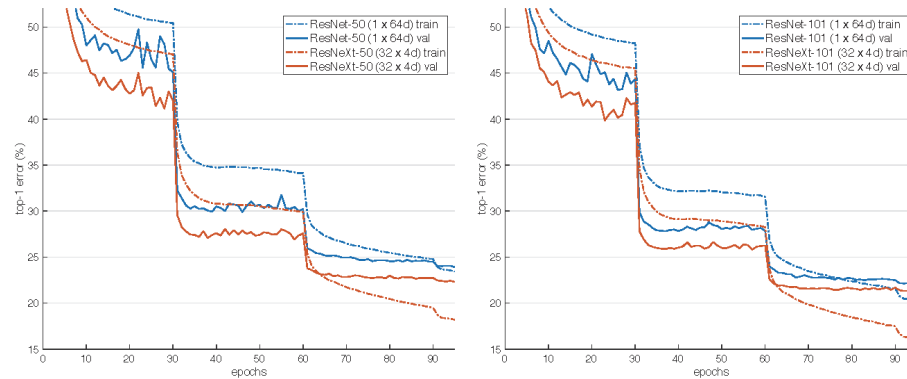


Figure 20.6.3 Training curves on ImageNet-1K. ResNet/ResNeXt-50 (left) with preserved complexity (~ 4.1 billion FLOPs, ~ 25 million parameters); ResNet/ResNeXt-101 (right) with preserved complexity (~ 7.8 billion FLOPs, ~ 44 million parameters).

Table 20.6.3 State-of-the-art networks on the ImageNet-1K validation set (single-crop testing). The test size of ResNet/ResNeXt is 224×224 and 320×320 as in [He1704] and of the Inception networks is 299×299 . Top-1: Top-1 error; Top-5: Top-5 error.

Network	224×224		320×320/299×299	
	Top-1	Top-5	Top-1	Top-5
ResNet-101 [He1512a]	22.0	6.0	-	-
ResNet-101 [He1704]	21.7	5.8	20.1	4.8
Inception-v3 [Sze1606]	-	-	21.2	5.6
Inception-v3 [Sze1608]	-	-	20.0	5.0
Inception-ResNet-v2 [Sze1608]	-	-	19.9	4.9
ResNet-101 (64×4d)	20.4	5.3	19.1	4.4

References

- [And8800] J. A. Anderson and E. Rosenfeld (Editors), "Neurocomputing - Foundations of Research," Cambridge, MA, USA: MIT Press, 1988, page xvi.
- [Arb0211] M. A. Arbib (Editor), The Handbook of Brain Theory and Neural Networks, Second Edition, Cambridge, MA, USA: MIT Press, November 2002.
- [Arb8900] M. A. Arbib and Shun-ichi Amari, Dynamic Interactions in Neural Networks: Models and Data, New York, NY, USA: Springer-Verlag, 1989.
- [Bas0208] S. Basu, P. F. Henshaw, N. Biswas, and H. K. Kwan, "Prediction of gas-phase adsorption isotherms using neural nets," The Canadian Journal of Chemical Engineering, vol. 80, no. 4, pp. 506-512, August 2002.
- [Bay0108] I. C. Baykal, A. Yilmaz, and H. K. Kwan, "Detection of late potentials in electrocardiogram signals in both time and frequency domains using artificial neural networks," in Proceedings of IEEE International Midwest Symposium on Circuits and Systems (ISCAS 2001), Dayton, Ohio, USA, vol. 2, 13-17 August 2001, pp. 576-579.
- [Bay0008] C. Baykal, A. Yilmaz, H. K. Kwan, and G. A. Jullien, "Detection of late potentials in electrocardiogram signals using artificial neural networks," in Proceedings 43rd IEEE International Midwest Symposium on Circuits and Systems (MWSCAS 2000), East Lansing, Michigan, USA, vol. 3, 8-11 August 2000, pp. 1352-1355.
- [Ben0612] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, "Greedy layer-wise training of deep networks," in Advances in Neural Information Processing Systems 19 (NIPS 2006), Y. Weiss, B. Schölkopf, and J. Platt, (Editors), New Jersey, USA: MIT Press, 2007, pp. 153-160.
- [Bis0600] C. M. Bishop, "Pattern Recognition and Machine Learning," New York, NY, USA: Springer, 2006.

[Cai9806] L. Y. Cai and H. K. Kwan, "Fuzzy classification using fuzzy inference networks," *IEEE Transactions on Systems, Man, Cybernetics, Part B: Cybernetics*, vol. 28, no. 3, pp. 334-347, June 1998.

[Cai9505] Y. Cai and H. K. Kwan, "A fuzzy inference network for classification," in *Proceedings of IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP 1995)*, Detroit, Michigan, USA, vol. 5, 8-12 May 1995, pp. 3499-3502.

[Cai9502] Y. Cai, H. K. Kwan, and B. Zhang, "Fuzzy neural network: a soft classifier," *AMSE Periodicals: Advances in Modeling and Analysis, B*, vol. 35, no. 2, pp. 51-56, 1995.

[Cai9410] Y. Cai and H. K. Kwan, "Fuzzy neural network: a soft classifier," in *Proceedings of International AMSE Conference on Systems, Control, Information: Methodologies and Applications*, Wuhan, China, vol. II, 20-22 October 1994, pp. 894-899.

[Cai9305] Y. Cai and H. K. Kwan, "A fuzzy neural classifier for pattern classification," in *Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS1993)*, Chicago, Illinois, USA, 3-6 May 1993, vol. 4, pp. 2367-2370.

[Cha0608] C.-Y. Chang and S.-Y. Fu, "Image classification using a module RBF neural network," in *Proceedings of First International Conference on Innovative Computing, Information and Control*, vol. I (ICICIC 2006), Beijing, China, 30 August - 1 September 2006, pp. 270-273. <https://doi.org/10.1109/ICICIC.2006.295>

[Che0410] L. Chen, S. Nilufar, and H. K. Kwan, "Districted matching approach for 1D object classification," in *Proceedings of International Symposium on Intelligent Multimedia, Video, and Speech Processing (ISIMP 2004)*, Hong Kong, China, 20-22 October 2004, pp. 206-209.

[Cir1206] D. Ciresan, U. Meier, and J. Schmidhuber, "Multi-column deep neural networks for image classification," in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2012)*, Providence, RI, USA, 16-21 June 2012, pp. 3642-3649. <https://doi.org/10.1109/cvpr.2012.6248110>

[Cle1602] D. A. Clevert, T. Unterthiner, and S. Hochreiter, “Fast and accurate deep network learning by exponential linear units (ELUs),” arXiv: 1511.07289v5, 22 February 2016.

[Cor9509] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273-297, September 1995. <https://doi.org/10.1007/BF00994018>

[Cri8600] F. H. C. Crick and C. Asanuma, “Certain aspects of the anatomy and physiology of the cerebral cortex,” in “Parallel distributed processing, Explorations in the microstructure of cognition, vol. 2: Psychological and biological models,” James L. McClelland, David E. Rumelhart and the PDP Research Group (Editors), Cambridge, MA, USA: MIT Press, 1986, Chapter 20, pp. 333-371.

[Dat1500] R. Datta and K. Deb (Editors), *Evolutionary Constrained Optimization*, Infosys Science Foundation Series in Applied Sciences and Engineering, New York, NY, USA: Springer, 2015.

[Don0306] X. Dong and H. K. Kwan, “Phoneme response sequence recognition using hidden Markov model,” in *Proceedings of IEEE Electro/Information Technology Conference (EIT 2003)*, Indianapolis, Indiana, USA, Session 5B-Digital Signal Processing, 5-6 June 2003, pp. 1-7.

[Due9009] G. Dueck and T. Scheuer, “Threshold accepting: A general purpose optimization algorithm appearing superior to simulated annealing” *Journal of Computational Physics*, vol. 90, issue 1, pp. 161-175, September 1990.

[Esm0905] A. Esmaili and N. Mozayani, “Adjusting the parameters of radial basis function networks using particle swarm optimization,” in *Proceedings of International Conference on Computational Intelligence for Measurement Systems and Applications (CIMSA 2009)*, Hong Kong, China, 11-13 May 2009, pp. 1-3. <https://doi.org/10.1109/CIMSA.2009.5069942>

[Eig1402] D. Eigen, J. Rolfe, R. Fergus, and Y. LeCun, “Understanding deep architectures using a recursive convolutional network,” arXiv: 1312.1847v2, 19 February 2014.

[Fan1800] S. Fan. “Understanding Deep Residual Networks.” <https://shuzhanfan.github.io/2018/11/ResNet/> (accessed 31 January 2020)

[Faq1704] A. Faqih, B. Krisnandhika, and B. Kusumoputro, “Mixed odors classification by neural network using radial basis function,” in Proceedings of 3rd International Conference on Control, Automation and Robotics (ICCAR 2017), Nagoya, Japan, 24-26 April 2017, pp. 567-570. <https://doi.org/10.1109/ICCAR.2017.7942761>

[Fau9400] L. Fausett, Fundamentals of Neural Networks, Prentice-Hall, New Jersey, 1994, Chapter 4, pp. 192-195.

[Fra9800] S. Franklin, Artificial Minds, Third Printing, Cambridge, MA, USA: MIT Press, 1998.

[Fus9907] J. M. Fuster, Memory in the Cerebral Cortex - An Empirical Approach to Neural Networks in the Human and Nonhuman Primate, Cambridge, MA, USA: MIT Press, July 1999.

[Glo8900] D. E. Goldberg, Genetic Algorithm in Search Optimization, and Machine Learning, Reading, Massachusetts, USA: Addison-Wesley, 1989.

[Goo1600] I. Goodfellow, Y. Bengio, and A. Courville. “Chapter 9 Convolutional Networks” in “Deep Learning,” MIT Press, 2016, Page 326-366. <http://www.deeplearningbook.org>

[Gra1505] B. Graham, “Fractional max-pooling,” arXiv: 1412.6071v4, 12 May 2015.

[Gra9800] D. B. Graham and N. M. Allinson, "Characterising virtual eigensignatures for general purpose face recognition," in H. Wechsler, P. J. Phillips, V. Bruce, F. Fogelman-Soulie, and T. S. Huang, (Editors), Face Recognition: From Theory to Application, NATO Advanced Study Institute (ASI), Series F, Computer and Systems Sciences, vol. 163, 1998, pp. 446-456.

[Gre9600] S. A. Greenfield, Human Mind Explained: An Owner's Guide to the Mysteries of the Mind, Montreal, Quebec, Canada: Reader's Digest, 1996.

[Gu0405] J.-J. Gu, L. Tao, and H. K. Kwan, "Fast learning algorithms for new L2 SVM based on active set iteration method," in Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS 2004), Vancouver, BC, Canada, 23-26 May 2004, vol. V, pp. 812-815.

[Hay9400] S. Haykin, Neural Networks, A Comprehensive Foundation, Macmillan College Publishing Co. Inc., 1994, Section 10.4, pp. 405-407.

[He1607] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," arXiv: 1603.05027v3, 25 July 2016.

[He1606] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016), Las Vegas, NV, USA, 27-30 June 2016, pp. 770-778.
<https://doi.org/10.1109/CVPR.2016.90>

[He1512a] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," arXiv: 1512.03385v1 10 December 2015.

[He1512b] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imageNet classification," in Proceedings of IEEE International Conference on Computer Vision (ICCV 2015), Santiago, Chile, 7-13 December 2015, pp. 1026-1034.

- [He1506] K. He and J. Sun, "Convolutional neural networks at constrained time cost," in Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2015), Boston, MA, USA, 7-12 June 2015, pp. 5353-5360.
<https://doi.org/10.1109/CVPR.2015.7299173>
- [Hec9000] R. Hecht-Nielsen, Neurocomputing, Reading, MA, USA: Addison-Wesley, 1990.
- [Hin0606] G. E. Hinton, S. Osindero, and Y.-W. Teh, "A fast learning algorithm for deep belief nets," Neural Computation, vol. 18, no. 7, pp. 1527-1554, June 2006. <https://doi.org/10.1162/neco.2006.18.7.1527>
- [Hoc9712] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, December 1997.
- [Koh9009] T. Kohonen, "The self-organizing map," IEEE Proceedings, vol. 78, no. 9, pp. 1464-1480, September 1990.
- [Koh9006] T. Kohonen, "Improved versions of learning vector quantization," Proceedings of International Joint Conference on Neural Networks, San Diego, CA, USA, 17-21 June 1990, vol. 1, pp. 545-550.
- [Hol7500] J. H. Holland (Editor), Adaptation in Neural and Artificial Systems, Ann Arbor, MI, USA: University of Michigan Press, 1975.
- [Hol6207] J. H. Holland, "Outline for a logical theory of adaptive systems," Journal of ACM, vol. 9, issue 3, pp. 297-314, July 1962.
- [Hou9005] P. Houselander and J. T. Taylor, "Improving the Hamming binary associative memory," Electronics Letters, vol. 26, issue 11, pp. 705- 707, 24 May 1990.
- [Iof1503] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," arXiv: 1502.03167v3, 2 March 2015.
- [Jan9700] J.-S. R. Jang, C.-T. Sun, and E. Mizutani, Neuro-Fuzzy and Soft Computing, Prentice-Hall, New Jersey, 1997.

[Joh9105] L. G. Johnson and S. M. S. Jalaliddine, "MOS implementation of winner-take-all network with application to content-addressable memory," *Electronics Letters*, vol. 27, issue 11, pp. 957-958, 23 May 1991.

[Jon7500] K. A. De Jong, *An Analysis of the Behavior of a Class of Genetic Adaptive Systems*, Doctoral Dissertation, Ann Arbor, MI, USA: University of Michigan, 1975.

[Kap1901] A. Kaplan and M. Haenlein, "Siri, Siri in my hand: who's the fairest in the land? On the interpretations, illustrations and implications of artificial intelligence," *Business Horizons*, vol. 62, issue 1, pp. 15-25, January-February 2019.

<https://doi.org/10.1016/j.bushor.2018.08.004>

[Ken0100] J. Kennedy and R. C. Eberhart, *Swarm Intelligence*, San Francisco, CA, USA: Morgan Kaufmann Publishers, 2001. ISBN: 1558605959, 9781558605954.

[Ken9710] J. Kennedy and R. C. Eberhart, "A discrete binary version of the particle swarm algorithm," in *Proceedings of International Conference on Systems, Man, and Cybernetics*, Orlando, FL, USA, 12-15 October 1997, pp. 4104-4109.

[Ken9704] J. Kennedy, "The particle swarm: social adaptation of knowledge," in *Proceedings of IEEE International Conference on Evolutionary Computation*, Indianapolis, IN, USA, 13-16 April 1997, pp. 303-308.

[Kli9500] G. J. Klir and B. Yuan, *Fuzzy Sets and Fuzzy Logic, Theory and Applications*, Prentice-Hall, New Jersey, 1995.

[Kos8801] B. Kosko, "Bidirectional associative memories," *IEEE Transactions on Systems, Man, Cybernetics*, January/February 1988, vol. 18, no. 1, pp. 49-60.

[Kos8712] B. Kosko, "Adaptive bidirectional associative memories," *Applied Optics*, December 1987, vol. 26, no. 23, pp. 4979-4960.

[Kwa1811a] H. K. Kwan, “Allpass group delay equalizer design using harmony search algorithm,” in Proceedings of IEEE 23rd International Conference on Digital Signal Processing (DSP 2018), Shanghai, China, 19-21 November 2018, pp. 1-4.

[Kwa1811b] H. K. Kwan and M. Zhang, “2-D linear phase FIR digital filter design using teaching-learning-based optimization,” in Proceedings of IEEE 23rd International Conference on Digital Signal Processing (DSP 2018), Shanghai, China, 19-21 November 2018, pp. 1-4.

[Kwa1808a] H. K. Kwan, “Group delay equalizer design using harmony search algorithm,” in Proceedings of IEEE 61st International Midwest Symposium on Circuits and Systems (MWSCAS 2018), Windsor, Ontario, Canada, 5-8 August 2018, pp. 688-691.

[Kwa1808b] H. K. Kwan, J. Liang, and A. Jiang, “Sparse FIR filter design using iterative MOCSA,” in Proceedings of IEEE 61st International Midwest Symposium on Circuits and Systems (MWSCAS 2018), Windsor, Ontario, Canada, 5-8 August 2018, pp. 952-955.

[Kwa1808c] H. K. Kwan and M. Zhang, “2-D FIR digital filter design using teaching-learning-based optimization,” in Proceedings of IEEE 61st International Midwest Symposium on Circuits and Systems (MWSCAS 2018), Windsor, Ontario, Canada, 5-8 August 2018, pp. 684-687.

[Kwa1805a] H. K. Kwan, “Unconstrained asymmetric FIR filter design using harmonic search algorithm,” in Proceedings of IEEE 31th Canadian Conference on Electrical and Computer Engineering (CCECE 2018), Quebec City, Quebec, Ontario, Canada, 13-16 May 2018, pp. 1-4.

[Kwa1805b] H. K. Kwan and M. Zhang, “IIR filter design using multiobjective teaching learning-based optimization,” in Proceedings of IEEE 31th Canadian Conference on Electrical and Computer Engineering (CCECE 2018), Quebec City, Quebec, Ontario, Canada, 13-16 May 2018, pp. 1-4.

[Kwa1803] H. K. Kwan, Global Optimization Algorithms and Design Applications, Edition 1.2, dfisp.org, 13 March 2018.

[Kwa1704] H. K. Kwan, “Asymmetric FIR filter design using evolutionary optimization,” in Proceedings of IEEE 30th Canadian Conference on Electrical and Computer Engineering (CCECE 2017), Windsor, Ontario, Canada, 30 April - 3 May 2017, pp. 1-4.

[Kwa1610a] H. K. Kwan and R. Raju, “Minimax design of linear phase FIR differentiators using artificial bee colony algorithm,” in Proceedings of 8th International Conference on Wireless Communications and Signal Processing (WCSP 2016), Yangzhou, Jiangsu, China, 13-15 October 2016, pp. 1-4.

[Kwa1610b] H. K. Kwan and J. Liang, “Design of linear phase FIR filters using cuckoo search algorithm,” in Proceedings of 8th International Conference on Wireless Communications and Signal Processing (WCSP 2016), Yangzhou, Jiangsu, China, 13-15 October 2016, pp. 1-4.

[Kwa1610c] H. K. Kwan and M. Zhang, “Minimax design of linear phase FIR Hilbert transformer using teaching-learning-based optimization,” in Proceedings of 8th International Conference on Wireless Communications and Signal Processing (WCSP 2016), Yangzhou, Jiangsu, China, 13-15 October 2016, pp. 1-4.

[Kwa1301] H. K. Kwan, “Intelligent digital filters with application to salt and pepper noise reduction in magnetic resonance brain images,” in Proceedings of Constantinides International Workshop on Signal Processing (CIWSP 2013), London, England, UK, 25 January 2013, pp. 1-4.

[Kwa1112] B. Y. M. Kwan and H. K. Kwan, “Impulse noise reduction in brain magnetic resonance imaging using fuzzy filters,” World Academy of Science, Engineering and Technology, issue 60, December 2011, Article number 264, pp. 1344-1347.

[Kwa0505] H. K. Kwan, "Three-layer symmetrical and asymmetrical associative memories for image applications," in Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS 2005), Kobe, Japan, 23-26 May 2005, pp. 2759-2762.

[Kwa0312] H. K. Kwan and X. Dong, "Phoneme sequence pattern recognition using fuzzy neural networks," in Proceedings of IEEE International Conference on Neural Networks and Signal Processing, Nanjing, China, 14-17 December 2003, vol. 1, pp. 535-538.

[Kwa0305a] H. K. Kwan, "Three-layer bidirectional asymmetrical associative memory," in Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS 2003), Bangkok, Thailand, vol. III, 25-28 May 2003, pp. III-482 to III-485.

[Kwa0305b] H. K. Kwan, "Fuzzy filters for noisy image filtering," in Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS 2003), Bangkok, Thailand, 25-28 May 2003, pp. IV-161 to IV-164.

[Kwa0303] H. K. Kwan, "Fuzzy filters for noise reduction in images," in "Fuzzy Filters for Image Processing," M. Nachtegaal, D. Van der Weken, D. Van De Ville, E. E. Kerre, (Editors), Studies in Fuzziness and Soft Computing, vol. 122, Berlin, Heidelberg, German: Springer Verlag, March 2003, Chapter 2, pp. 25-53.
https://doi.org/10.1007/978-3-540-36420-7_2

[Kwa0208a] H. K. Kwan, "Multilayer recurrent neural networks," in Proceedings of 45th IEEE International Midwest Symposium on Circuits and Systems (MWSCAS 2002), Tulsa, Oklahoma, USA, vol. III, 4-7 August 2002, pp. III-97 to III-100.

[Kwa0208b] H. K. Kwan and Y. Cai, "Fuzzy filters for image filtering," in Proceedings of 45th IEEE International Midwest Symposium on Circuits and Systems (MWSCAS 2002), Tulsa, Oklahoma, USA, vol. III, 4-7 August 2002, pp. III-672 to III-675.

[Kwa0206a] H. K. Kwan and C. Z. Tang, "Multiplierless multilayer feedforward neural network design using quantised neurons," *Electronics Letters*, vol. 38, no. 13, pp. 645-646, 20 June 2002.

[Kwa0206b] H. K. Kwan, "Multiplierless designs for artificial neural network," in *Neural Networks and Systolic Array Design (Machine Perception and Artificial Intelligence)*, vol. 49, D. Zhang and S. K. Pal, (Editors). Singapore: World Scientific Publishing Company Pte Ltd., June 2002, Chapter 13, pp. 301-325.
https://doi.org/10.1142/9789812778086_0013

[Kwa0205] H. K. Kwan, "Fuzzy neural network for phoneme sequence recognition," in *Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS 2002)*, Scottsdale, Arizona, USA, vol. II, 26-29 May 2002, pp. 847-850.

[Kwa0105] H. K. Kwan and J. Yan, "Second-order recurrent neural network for word sequence learning," in *Proceedings of IEEE International Symposium on Intelligent Multimedia, Video and Speech Processing (ISIMP2001)*, Hong Kong, China, 2-4 May 2001, pp. 405-408.

[Kwa0008a] H. K. Kwan and J. Yan, "Recurrent neural network design for temporal sequence learning," in *Special Section S6 - Neural Networks and Applications of Proceedings 43rd IEEE International Midwest Symposium on Circuits and Systems (MWSCAS 2000)*, East Lansing, Michigan, USA, vol. 2, 8-11 August 2000, pp. 832-835.

[Kwa0008b] H. K. Kwan and L. Y. Cai, "Supervised fuzzy inference network for invariant pattern recognition," in *Special Section S6 - Neural Networks and Applications of Proceedings 43rd IEEE International Midwest Symposium on Circuits and Systems (MWSCAS 2000)*, East Lansing, Michigan, USA, vol. 2, 8-11 August 2000, pp. 850-854.

[Kwa9905] H. K. Kwan and L. Tao, "Adaptive IIR filtering using an analog neural network," in *Proceedings of Canadian Conference on Electrical and Computer Engineering (CCECE 1999)*, Edmonton, Alberta, Canada, 9-12 May 1999, vol. 2, pp. 827-830.

[Kwa9408] H. K. Kwan and Y. Cai, "A fuzzy neural network and its application to pattern recognition," *IEEE Transactions on Fuzzy Systems*, vol. 2, no. 3, pp. 185-193, August 1994.

[Kwa9405] H. K. Kwan and C. Z. Tang, "A multilayer feedforward neural network model for digital hardware implementation," in *Proceedings of IEEE International Symposium on Circuits and Systems*, London, UK, vol. 6, 30 May 30 - 2 June 1994, pp. 343-346.

[Kwa9308a] H. K. Kwan and Y. Cai, "Median filtering using fuzzy concept," in *Proceedings of 36th IEEE International Midwest Symposium on Circuits and Systems (MWSCAS 1993)*, Detroit, Michigan, USA, vol. 2, 15-18 August 1993, pp. 824-827.

[Kwa9308b] H. K. Kwan and C. Z. Tang, "Multiplierless multilayer feedforward neural networks," in *Proceedings of 36th IEEE International Midwest Symposium on Circuits and Systems (MWSCAS 1993)*, Detroit, Michigan, USA, vol. 2, 15-18 August 1993, pp. 1085-1088.

[Kwa9307] H. K. Kwan and C. Z. Tang, "Multiplierless multilayer feedforward neural network design suitable for continuous input-output mapping," *Electronics Letters*, vol. 29, no. 14, pp. 1259-1260, 8 July 1993.

[Kwa9306] H. K. Kwan, Y. Cai, and B. Zhang, "Membership function learning in fuzzy classification," *International Journal of Electronics*, vol. 74, no. 6, pp. 845-850, June 1993.

[Kwa9305] H. K. Kwan and C. Z. Tang, "A design method for multilayer feedforward neural networks for simple hardware implementation," in *Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS1993)*, Chicago, Illinois, USA, 3-6 May 1993, vol. 4, pp. 2363-2366.

[Kwa9304] H. K. Kwan, Z. Wang, and J. J. Soltis, "Method for generating partially correlated vector sets for neural network simulations," *International Journal of Electronics*, vol. 74, no. 4, pp. 523-528, April 1993.

[Kwa9301] H. K. Kwan and C. K. Lee, "A neural network approach to pulse radar detection," IEEE Transactions on Aerospace and Electronic Systems, vol. 29, no. 1, pp. 9-21, January 1993.

[Kwa9212] H. K. Kwan and C. Z. Tang, "Designing multilayer feedforward neural networks using simplified sigmoid activation functions and one-powers-of-two weights," Electronics Letters, vol. 28, no. 25, pp. 2343-2344, 3 December 1992.

[Kwa9211] H. K. Kwan and Y. Cai, "Fuzzy neural network for pattern recognition," in Proceedings of International Joint Conference on Neural Networks (IJCNN 1992), Beijing, China, 3-6 November 1992, vol. III, pp. 691-696.

[Kwa9209a] H. K. Kwan and C. Z. Tang, "Multilayer feedforward neural networks with powers-of-two weights and piecewise activation functions," in Proceedings of Canadian Conference on Electrical and Computer Engineering (CCECE 1992), Toronto, Ontario, Canada, 13-16 September 1992, vol. II, pp. TM6.14.1-4.

[Kwa9209b] H. K. Kwan, "Multilayer feedforward and feedbackward neural networks," in Proceedings of Canadian Conference on Electrical and Computer Engineering (CCECE 1992), Toronto, Ontario, Canada, 13-16 September 1992, vol. II, pp. WA7.26.1-4.

[Kwa9208] H. K. Kwan, "One-layer feedforward neural network for fast maximum/minimum determination," Electronics Letters, vol. 28, no. 17, pp. 1583-1585, 13 August 1992.

[Kwa9207] H. K. Kwan, "Simple sigmoid-like activation function suitable for digital hardware implementation," Electronics Letters, vol. 28, no. 15, pp. 1379-1380, 16 July 1992.

[Kwa9205] H. K. Kwan and Y. Yang, "Ring unidirectional associative memories," in Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS 1992), San Diego, California, USA, vol. 3, 10-13 May 1992, pp. 1557-1560.

[Kwa9109a] H. K. Kwan, "Noise filtering using multilayer feedforward neural network," in Proceedings of Canadian Conference on Electrical and Computer Engineering (CCECE 1991), Quebec, Canada, 25-27 September 1991, vol. 1, pp. 1.4.1-4.

[Kwa9109b] H. K. Kwan, "Cascaded recursive artificial neural networks," in Proceedings of Canadian Conference on Electrical and Computer Engineering (CCECE 1991), Quebec, Canada, 25-27 September 1991, vol. 1, pp. 25.6.1-4.

[Kwa9106] H. K. Kwan, "High-order feedbackward neural networks," in Proceedings of International Conference on Circuits and Systems, Shenzhen, China, 16-17 June 1991, vol. 1, pp. 49-51.

[Kwa9105] H. K. Kwan, "Multilayer feedbackward neural networks," in Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 1991), Toronto, Ontario, Canada, 14-17 May 1991, vol. 2, pp. 1145-1148.

[Kwa9004] H. K. Kwan and P. C. Tsang, "Systolic implementation of counterpropagation networks," in Proceedings of IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP 1990), Albuquerque, New Mexico, USA, 3-6 April 1990, vol. 2, pp. 953-956.

[Kwa9001] H. K. Kwan and P. C. Tsang, "Systolic implementation of multilayer feed-forward neural network with back-propagation learning scheme," in Proceedings of International Joint Conference on Neural Networks (IJCNN 1990), Washington, DC, USA, 15-19 January 1990, vol. II, pp. 155-158.

[Kwa8909a] H. K. Kwan and P. C. Tsang, "Multi-layer recursive neural network," in Proceedings of Canadian Conference on Electrical and Computer Engineering (CCECE 1989), Montreal, Quebec, Canada, 17-20 September 1989, vol. I, pp. 428-431.

[Kwa8909b] H. K. Kwan and C. K. Lee, "Temporal associative memories using cascade and ring architectures," in Proceedings of Canadian Conference on Electrical and Computer Engineering (CCECE 1989), Montreal, Quebec, Canada, vol. II, 17-20 September 1989, pp. 779-782.

[Kwa8906a] H. K. Kwan and C. K. Lee, "Pulse radar detection using a multi-layer neural network," in Proceedings of International Joint Conference on Neural Networks (IJCNN 1989), Washington, DC, USA, 18-22 June 1989, vol. II, pp. 75-80.

[Kwa8906b] H. K. Kwan and C. K. Lee, "Temporal associative memories using cascade and ring architectures," Abstract in Proceedings of the International Joint Conference on Neural Networks, Washington, DC, USA, vol. II, 18-22 June 1989, page 570.

[Kwa8905] H. K. Kwan, "Systolic architectures for Hopfield network, BAM and multi-layer feed-forward network," in Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS 1989), Portland, Oregon, USA, 9-11 May 1989, vol. 2, pp. 790-793.

[Koc9800] C. Koch and I. Segev (Editors), Methods in Neuronal Modeling - From Ions to Networks, Second Edition Cambridge, MA, USA: MIT Press, 1998.

[Kri1706] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," Communications of the ACM, vol. 60 no. 6, pp. 84-90, June 2017. <https://dl.acm.org/doi/10.1145/3065386>

[Kri1212] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," in Proceedings of International Conference on Neural Information Processing Systems (NIPS 2012), Lake Tahoe, Nevada, USA, 3-6 December 2012, pp. 1097-1105.

[Lar8001] P. M. Larsen, "Industrial applications of fuzzy logic control," International Journal of Man-Machine Studies, vol. 12, issue 1, pp. 3-10, January 1980.

[Lee1505] C.-Y. Lee, S. Xie, P. W. Gallagher, Z. Zhang, and Z. Tu, “Deeply-supervised nets,” in Proceedings of the 18th International Conference on Artificial Intelligence and Statistics (AISTATS), San Diego, CA, USA, 9-12 May 2015, pp. 562-570 (JMLR: W&CP volume 38).

[Lee9006] C. K. Lee and H. K. Kwan, “Learning algorithms for autoassociators with full storage capacity,” in Proceedings of International Joint Conference on Neural Networks (IJCNN 1990), San Diego, California, USA, 17-21 June 1990, vol. III, pp. 173-178.
<https://doi.org/10.1109/IJCNN.1990.137841>

[Li0712] H.-Q. Li and L. Li, “A novel hybrid real-valued genetic algorithm for optimization problems,” in Proceedings of International Conference on Computational Intelligence and Security, Harbin, Heilongjiang, China, 15-19 December 2007, pp. 91-95.

[Lia1811a] J. Liang and H. K. Kwan, “Sparse 2-D FIR filter design using iterative CSA with step-descendant coefficient thresholding,” in Proceedings of IEEE 23rd International Conference on Digital Signal Processing (DSP 2018), Shanghai, China, 19-21 November 2018, pp. 1-4.

[Lia1811b] J. Liang, H. K. Kwan, and A. Jiang, “Sparse linear phase FIR filter design using iterative CSA,” in Proceedings of IEEE 23rd International Conference on Digital Signal Processing (DSP 2018), Shanghai, China, 19-21 November 2018, pp. 1-4.

[Lia1805] J. Liang and H. K. Kwan, “IIR filter design using constrained multiobjective cuckoo search algorithm,” in Proceedings of IEEE 31th Canadian Conference on Electrical and Computer Engineering (CCECE 2018), Quebec City, Quebec, Ontario, Canada, 13-16 May 2018, pp. 1-4.

[Lia1704] J. Liang and H. K. Kwan, “FIR filter design using multiobjective cuckoo search algorithm,” in Proceedings of IEEE 30th Canadian Conference on Electrical and Computer Engineering (CCECE 2017), Windsor, Ontario, Canada, 30 April - 3 May 2017, pp. 1-4.

[Lin1409] T. Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollar, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Computer Vision - ECCV 2004 (Lecture Notes in Computer Science, vol. 8693)*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars (Editors), Cham: Springer, 2004, pp. 740-7555.
https://doi.org/10.1007/978-3-319-10602-1_48

[Lin1403] M. Lin, Q. Chen, and S. Yan. "Network in network," arXiv: 1312.4400v3, 4 March 2014

[Lin9600] C.-T. Lin and C. S. George Lee, *Neural Fuzzy Systems, a Neuro-Fuzzy Synergism to Intelligent Systems*, Prentice-Hall, New Jersey, 1996.

[Lip8704] R. P. Lippmann, "An introduction to computing with neural nets," *IEEE ASSP Magazine*, pp. 4-22, April 1987.

[Lof1503] S. Loffe and C. Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." preprint, arXiv: 1502.03167v3 (2 March 2015)

[Mac8700] Ronald J. MacGregor, *Neural and Brian Modeling*, Toronto, Ontario, Canada: Academic Press, Inc., 1987.

[Mam7501] E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man-Machine Studies*, vol. 7, issue 1, pp. 1-13, January 1975.

[Man0712] C.-T. Man, X.-X. Li, and L.-Y. Zhang, "Radial basis function neural network based on ant colony optimization," in *Proceedings of Computational Intelligence and Security Workshops (CISW 2007)*, Heilongjiang, Harbin, China, 15-19 December 2007, pp. 59-62. <https://doi.org/10.1109/CISW.2007.4425446>

[Mat-CNN] "Learn About Convolutional Neural Networks." Mathworks.com
<https://www.mathworks.com/help/nnet/ug/introduction-to-convolutional-neural-networks.html> (accessed 28 January 2020)

[Mat-DLN] “Create Simple Deep Learning Network for Classification.” Mathworks

<https://www.mathworks.com/help/deeplearning/examples/create-simple-deep-learning-network-for-classification.html> (accessed 28 January 2020)

[Mat-Lay] “Specify Layers of Convolutional Neural Network.” Mathworks.com

<https://www.mathworks.com/help/nnet/ug/layers-of-a-convolutional-neural-network.html> (accessed 28 January 2020)

[Met5306] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, “Equation of state calculations by fast computing machines,” *Journal of Chemical Physics*, vol. 21, no. 6, page 1087-1092, June 1953.

[Mez1112] E. Mezura-Montesa and C. A. Coello, “Constraint-handling in nature-inspired numerical optimization: Past, present and future,” *Swarm and Evolutionary Computation*, vol. 1, issue 4, pp. 173-194, December 2011.

[McC9800] J. L. McClelland and D. E. Humelhart, *Explorations in Parallel Distributed Processing*, Cambridge, MA, USA: MIT Press, 1988, Chapter 4, pp. 83-119.

[Mis1602] D. Mishkin and J. Matas, “All you need is a good init,” *arXiv:1511.06422v7*, 19 February 2016.

[Nag1111] J. Nagi, F. Ducatelle, G. A. Di Caro, D. Ciresan, U. Meier, A. Giusti, F. Nagi, J. Schmidhuber, and L. M. Gambardella, “Max-pooling convolutional neural networks for vision-based hand gesture recognition,” in *Proceedings of IEEE International Conference on Signal and Image Processing Applications (ICSIPA 2011)*, Kuala Lumpur, Malaysia), 16-18 November 2011, pp. 342-347.

[Nai1006] V. Nair and G. E. Hinton, “Rectified linear units improve restricted Boltzmann machines,” in *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*, Haifa, Israel, 21-24 June 2010, pp. 807-814.

[Nil0505] S. Nilufar, L. Chen, and H. K. Kwan, "A fuzzy query based image retrieval system," in Proceedings of International Conference on Communications, Circuits and Systems (ICCCAS 2005), Hong Kong, China, 27-30 May 2005, pp. 1017-1021.

[Pan9003] J. Pankove, C. Radehaus, and K. Wagner, "Winner-take-all neural net with memory," *Electronics Letters*, vol. 26, issue 6, pp. 349-350, 15 March 1990.

[Pri0500] K. V. Price, R. M. Storn, and J. A. Lampinen, *Differential Evolution, A practical approach to Global Optimization*, Berlin Heidelberg: Springer-Verlag, 2005.

[Qas0905] S. N. Qasem and S. M. H. Shamsuddin, "Improving performance of radial basis function network based with particle swarm optimization," in Proceedings of IEEE Congress on Evolutionary Computation (CEC 2009), Trondheim, Norway, 18-21 May 2009, pp. 3149-3156. <https://doi.org/10.1109/CEC.2009.4983342>

[Raj1808] R. Raju, H. K. Kwan, and A. Jiang, "Sparse FIR filter design using artificial bee colony algorithm," in Proceedings of IEEE 61st International Midwest Symposium on Circuits and Systems (MWSCAS 2018), Windsor, Ontario, Canada, 5-8 August 2018, pp. 956-959.

[Raj1805] R. Raju and H. K. Kwan, "IIR filter design using multiobjective artificial bee colony algorithm," in Proceedings of IEEE 31st Canadian Conference on Electrical and Computer Engineering (CCECE 2018), Quebec City, Quebec, Ontario, Canada, 13-16 May 2018, pp. 1-4.

[Raj1704] R. Raju and H. K. Kwan, "FIR filter design using multiobjective artificial bee colony algorithm," in Proceedings of IEEE 30th Canadian Conference on Electrical and Computer Engineering (CCECE 2017), Windsor, Ontario, Canada, 30 April - 3 May 2017, pp. 1-4.

- [Rom1505] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. “Hints for thin deep nets,” in Proceedings of International Conference on Learning Representations (ICLR 2015), San Diego, CA, USA, 7-9 May 2015, pp. 1-13.
- [Rus1400] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, “ImageNet large scale visual recognition challenge,” arXiv: 1409.0575v3, 30 January 2015.
- [Sco8305] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, issue 4598, pp. 671-680, 13 May 1983.
- [Shi9805] Y. Shi and R. C. Eberhart, “A modified particle swarm optimizer,” in Proceedings of IEEE World Congress on Computational Intelligence, Anchorage, AK, USA, 4-9 May 1998, pp. 69-73.
- [Sim1504] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” arXiv: 1409.1556v6, 10 April 2015.
- [Siv0800] S. N. Sivanandam and S. N. Deepa, *Introduction to Genetic Algorithms*, Berlin, Heidelberg: Springer-Verlag, 2008.
- [Spr1504] J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, “Striving for simplicity: The all convolutional net,” arXiv: 1412.6806v3, 13 April 2015.
- [Sri1511a] R. K. Srivastava, K. Greff, and J. Schmidhuber, “Highway networks,” arXiv: 1505.00387v2, 3 November 2015.
- [Sri1511b] R. K. Srivastava, K. Greff, and J. Schmidhuber, “Training very deep networks,” arXiv: 1507.06228v2, 23 November 2015.
- [Sri1401] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929-1958, January 2014.

[Sug8810] M. Sugeno and G. T. Kang, "Structure identification of fuzzy model," *Fuzzy Sets and Systems*, vol. 28, issue 1, pp. 15-33, October 1988.

[Sze1608] C. Szegedy, S. Ioffe, and V. Vanhoucke. "Inceptionv4, inception-resnet and the impact of residual connections on learning," *arXiv: 1602.07261v2*, 23 August 2016.

[Sze1606] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. "Rethinking the inception architecture for computer vision," in *Proceedings of Computer Vision and Pattern Recognition (CVPR 2016)*, Las Vegas, NV, USA, 27-30 June 2016, pp. 2818-2826. <https://doi.org/10.1109/CVPR.2016.308>

[Sze1506] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2015)*, Boston, MA, USA, 7-12 June 2015, pp. 1-9.

[Tak8501] T. Takagi and M. Sugeno, "Fuzzy identification of systems and its applications to modeling and control," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 15, no. 1, pp. 116-132, January-February 1985.

[Tan9706] C. Z. Tang and H. K. Kwan, "Digital implementation of neural networks with quantized neurons," in *Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS 1997)*, Hong Kong, vol. 1, 9-12 June 1997, pp. 649-652.

[Tan9308] C. Z. Tang and H. K. Kwan, "Multilayer feedforward neural networks with single powers-of-two weights," *IEEE Transactions on Signal Processing*, vol. 41, no. 8, pp. 2724-2727, August 1993.

[Tan9301] C. Z. Tang and H. K. Kwan, "Parameter effects on convergence speed and generalization capability of backpropagation algorithm," *International Journal of Electronics*, vol. 74, no. 1, pp. 35-46, January 1993.

- [Tan9211a] C. Z. Tang and H. K. Kwan, "Feedforward neural networks without multipliers," in Proceedings of International Conference on Signal Processing Applications and Technology, Boston, Massachusetts, USA, 2-5 November 1992, vol. 2, pp. 1194-1201.
- [Tan9211b] C. Z. Tang and H. K. Kwan, "Feedforward neural networks with powers-of-two weights," in Proceedings of International Joint Conference on Neural Networks (IJCNN 1992), Beijing, China, 3-6 November 1992, vol. II, pp. 93-98.
- [Tan9205] C. Z. Tang and H. K. Kwan, "Convergence and generalization properties of multilayer feedforward neural networks," in Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS 1992), San Diego, California, USA, 10-13 May 1992, vol. 1, pp. 65-68.
- [Tao9905] L. Tao and H. K. Kwan, "A neural network method for adaptive noise cancellation," in Proceedings of IEEE International Symposium of Circuits and Systems (ISCAS 1999), Orlando, Florida, USA, 30 May - 2 June 1999, vol. V, pp. 567-570.
- [The0512] J. Thevaril and H. K. Kwan, "Speech enhancement using adaptive neuro-fuzzy filtering," in Proceedings of International Symposium on Intelligent Signal Processing and Communication Systems, Hong Kong, China, 13-16 December 2005, pp. 753-756.
- [Tsu7900] Y. Tsukamoto, "An approach to fuzzy reasoning method," in M. M. Gupta, R. K. Ragade, and R. R. Yager (Editors), Advances in Fuzzy Set Theory and Applications, North-Holland, Amsterdam, pp. 137-149, 1979.
- [Wan2000] H. Wang, R. Cheng, J. Zhou, L. Tao, and H. K. Kwan, "Multistage model for robust face alignment using deep neural networks," Neurocomputing, 2020.
- [Wan1810] P. Wang, A. Jiang, X. Liu, J. Shang, and L. Zhang, "LSTM-based EEG classification in motor imagery tasks," IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 86, no. 11, Page 2086-2095, 18 October 2018.

[Wik6895] “68–95–99.7 rule.” wikipedia.org
https://en.wikipedia.org/wiki/68%E2%80%9395%E2%80%9399.7_rule
(access March 21, 2017)

[WikCoIn] “Computational Intelligence.” wikipedia.org
https://en.wikipedia.org/wiki/Computational_intelligence (access February 2019)

[WikArIn] “Artificial Intelligence.” wikipedia.org
https://en.wikipedia.org/wiki/Artificial_intelligence (accessed February 2019)

[WikNeur] “Neuron.” wikipedia.org
<http://en.wikipedia.org/wiki/Neuron> (accessed January 2016)

[Wu0510] F. Wu, Y. Zhao, and Y. Zhang, “A novel radial basis function network on ant colony clustering,” in Proceedings of 11th joint International Computer Conference (JICC 2005), Hackensack: World Scientific Publishing Company, Incorporated, October 2005, pp. 464-469. https://doi.org/10.1142/9789812701534_0105

[Xie1704] S. Xie, R. Girshick, P. Dollar, Z. Tu, and K. He, “Aggregated residual transformations for deep neural networks,” arXiv: 1611.05431v2, 11 April 2017.

[Yen9203] J. -C. Yen and S. Chang, “Improved winner-take-all neural network,” Electronics Letters, vol. 28, issue 7, pp. 662-664, 26 March 1992.

[Zad7801] L. A. Zadeh, “Fuzzy sets as a basis for a theory of possibility,” Fuzzy Sets and Systems, vol. 1, issue 1, pp. 3-28, January 1978.

[Zad7500a] L. A. Zadeh, “The concept of a linguistic variable and its application to approximate reasoning, Part I,” Information Sciences, vol. 8, issue 3, pp. 199-249, 1975.

[Zad7500b] L. A. Zadeh, “The concept of a linguistic variable and its application to approximate reasoning, Part II,” Information Sciences, vol. 8, issue 4, pp. 301-357, 1975.

[Zad7500c] L. A. Zadeh, "The concept of a linguistic variable and its application to approximate reasoning, Part III," *Information Sciences*, vol. 9, pp. 43-80, 1975.

[Zad7301] L. A. Zadeh, "Outline of a new approach to the analysis of complex systems and decision processes," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 3, no. 1, pp. 28-44, January 1973.

[Zad7104a] L. A. Zadeh, "Quantitive fuzzy semantics," *Information Sciences*, vol. 3, issue 2, pp. 159-176, April 1971.

[Zad7104b] L. A. Zadeh, "Similarity relations and fuzzy ordering," *Information Sciences*, vol. 3, issue 2, pp. 177-206, April 1971.

[Zad6506] L. A. Zadeh, "Fuzzy Sets," *Information and Control*, vol. 8, issue 3, pp. 338-353, June 1965.

[Zha1704] M. Zhang and H. K. Kwan, "FIR filter design using multiobjective teaching-learning-based optimization," in *Proceedings of IEEE 30th Canadian Conference on Electrical and Computer Engineering (CCECE 2017)*, Windsor, Ontario, Canada, 30 April - 3 May 2017, pp. 1-4.

[Zha1411] J. Zhang and H.-Q. Li, "A global-crowding-distance based multi-objective particle swarm optimization algorithm," in *Proceedings of Tenth International Conference on Computational Intelligence and Security*, Kunming, China, 15-16 November 2014, pp. 1-6.

[Zho1911] J. Zhou, Y. Hu, H. Lian, H. Wang, L. Tao, and H. K. Kwan, "Multimodal voice conversion under adverse environment using a deep convolutional neural network," vol. 7, pp. 170878 - 170887, 26 November 2019. <https://doi.org/10.1109/ACCESS.2019.2955982>

[Zho1012] J. Zhou, Z. Wen, and J. Qu, "RBF neural network using improved differential evolution for groundwater table prediction," in *Proceedings of International Conference on Computational Intelligence and Software Engineering (CiSE 2010)*, Wuhan, China, 10-12 December 2010, pp. 1-4. <https://doi.org/10.1109/CISE.2010.5676973>

Description: The Edition 1.5 of this eBook on Intelligent Computing gives a clear description of the theories, models, algorithms, and design methodologies of intelligent computing to mimic and explain human-like reasoning, learning, and memory. The eBook consists of 20 chapters organized into 5 parts. The Part I on Intelligent Computing consists of Chapter 1 which gives overviews on computational intelligence, artificial intelligence, intelligent computing, and intelligent system design; and the basics of biological and artificial neural networks. The Part II on Neural Computing starts with an overview and consists of Chapters 2 to 8 covering various neural networks and learning algorithms. The Part III on Fuzzy Neural Computing first provides an overview and consists of Chapters 9 to 12 explaining fuzzy set, fuzzy logic and reasoning, fuzzy systems; and fuzzy neural networks. The Part IV on Evolutionary Computing begins with an overview and consists of Chapter 13 to 17 describing 4 popular evolutionary algorithms, and aspects on coding, initialization, and constraints. Finally, the Part V is on Deep Neural Computing first presents an overview and consists of Chapters 18-20 on multiplierless neural networks; convolutional neural networks; and deep residual learning and networks. This eBook is well suitable for senior undergraduate and graduate students as a textbook, and practicing engineers and computing professionals for self-learning. No specific prerequisites are required but a background of elementary courses in linear algebra and calculus would be useful.

Key Features: (1) A comprehensive coverage of intelligent computing. (2) Topics are clearly described and explained with the aid of step-by-step hand-calculation examples, Matlab and design examples, figures, tables, and easy-to-understand equations. (3) The eBook contains 98 examples in which 37 of them are design examples; and over 150 tables, 176 figures, 195 references, and 721 pages to support and clarify explanations. (4) Accompanied by a supplementary eBook on “Matlab and Worked Problems in Intelligent Computing” to reinforce learning. (5) A clickable table of contents also comes with the eBook to allow a quick link of a specific topic. (6) The eBook has been designed and formatted to be readable and usable directly by an instructor in classes and also by teaching assistants in tutorial classes. (7) The contents of this eBook have been developed under research and classroom environments for courses taught by the author.

dfisp.org

